

Архив статей wasm.ru. Том 1: Основы и уроки lczelion

Собрание русскоязычных статей сайта wasm.ru о программировании, ассемблере, реверс-инжиниринге и компьютерной безопасности. Том 1 из 11. Разделы: 1. Низкоуровневое программирование для дзенствующих; 2. Уроки lczelion'a. Все приложенные файлы находятся в wasm_files.zip.

Больше крутых материалов в моём телеграм канале [@grogrigs](#)

1. Низкоуровневое программирование для дзенствующих

Введение в машинный код

Автор: Serrgio / HI-TECH

Дата: Не указана

Этот документ предназначен для начинающих кодеров и только для них. Профессиональным программистам надлежит относиться к нему снисходительно ;) Выбор "среды обучения" был совершен в трезвом уме и в здравой памяти, желающие поиронизировать по этому поводу - милости прошу serrgio@gorki.unibel.by.

_Вы читали "Хроники Амбера" Роджера Желязны? Там есть такой эпизод:

Главный герой находится в заточении. В абсолютной тьме. У него были выколоты глаза, но за год они регенерировали, и зрение постепенно к нему возвращается...

И однажды каким-то чудом в одной камере с ним оказывается загадочный Дворкин — создатель Лабиринта. Именно "чудом" — он просто появился неизвестно откуда. Он тоже находится "в заключении", но, в отличие от Корвина (главного героя), может спокойно ходить через каменные стены.

Удивленный Корвин спрашивает его:

— Как ты оказался в моей камере? Ведь здесь нет дверей.

Дворкин отвечает:

— Двери есть везде. Просто нужно знать, как в них войти.

Будем считать это эпиграфом..._

1.1. Система счисления

#1. Наверняка среди ваших знакомых есть "крутые" программисты, или люди, таковыми себя считающие ;) . Попробуйте как-нибудь проверить их "на вшивость". Предложите им в уме перевести число 12 из шестнадцатеричной в двоичную систему счисления. Если над подобным вопросом "крутой программист" будет думать дольше 10 секунд - значит он вовсе не так крут, как говорит...

#2. Система счисления (сие не подвластное человеческой логике определение взято из математической энциклопедии) - это совокупность приемов представления обозначения натуральных чисел. Этих "совокупностей приемов представления" существует очень много, но самая совершенная из всех - та, которая подчиняется позиционному принципу. А согласно этому принципу один и тот же цифровой знак имеет различные значения в зависимости от того места, где он расположен. Такая система счисления основывается на том, что некоторое число n единиц ($radix$) объединяются в единицу второго разряда, n единиц второго разряда объединяются в единицу третьего разряда и т. д.

#3. "Разрядам" нас учили еще в начальных классах школы. Например, у числа 35672 цифра "2" имеет первый разряд, "7" - второй, "6" - третий, "5" - четвертый и "3" - пятый. А "различные значения" цифрового знака "в зависимости от того места, где он расположен" и "объединение в единицу старшего разряда" на тех же уроках арифметики "объяснялось" следующим образом:

$$\begin{aligned} 35672 &= 30000 + 5000 + 600 + 70 + 2 \\ 35672 &= 3 \cdot 10000 + 5 \cdot 1000 + 6 \cdot 100 + 7 \cdot 10 + 2 \cdot 1 \\ 35672 &= 3 \cdot 10^4 + 5 \cdot 10^3 + 6 \cdot 10^2 + 7 \cdot 10^1 + 2 \cdot 10^0 \quad (1) \end{aligned}$$

#4. Очень наглядно это отображают обыкновенные счета. Набранное на них число 35672 будет выглядеть... см. рисунок слева в общем...

Чтобы набрать число 35672 мы должны передвинуть влево две "костяшки" на первом "прутике", 7 на втором, 6 на третьем, 5 на четвертом и 3 на пятом. (У нас ведь 1 "костяшка" на втором - это то же самое, что и 10 "костяшек" на первом, а одна на третьем равна десяти на втором, и так далее...) Пронумеруем наши "прутики" снизу вверх - да так, чтобы номером первого был "0"... И снова посмотрим на наши выражения:

$$35672 = 3 \cdot 10^4 + 5 \cdot 10^3 + 6 \cdot 10^2 + 7 \cdot 10^1 + 2 \cdot 10^0$$

Это (если сверху вниз считать) сколько на каждом "прутике" "костяшек" влево отодвинуто.

$$35672 = 3 \cdot 10^4 + 5 \cdot 10^3 + 6 \cdot 10^2 + 7 \cdot 10^1 + 2 \cdot 10^0$$

Это номер прутика (самый нижний - 0), на котором отодвинуто определенное число костяшек.

$$35672 = 3 \cdot 10^4 + 5 \cdot 10^3 + 6 \cdot 10^2 + 7 \cdot 10^1 + 2 \cdot 10^0$$

Это на каждом прутике - по 10 костяшек нанизано, не все влево отодвинуты, но всего-то их - 10! Кстати, красненькое 10 в последнем выражении соответствует основанию (radix) системы счисления (number system).

#5. Пальцев на руках у человека 10, поэтому и считать мы привыкли в системе счисления с основанием 10, то есть в десятичной. Если вы хорошо представляете себе счеты и немного поупражнялись в разложении чисел аналогично выражению 1, то перейти на систему счисления с основанием, отличным от привычной, особого труда для вас не составит. Нужно всего лишь представить себе счеты, на каждый прут которых нанизано не привычные 10 костяшек, а... скажем, 9 или 8, или 16, или 32, или 2 и... попробовать мысленно считать на них.

#6. Для обозначения десятичных чисел мы используем цифры от 0 до 9, для обозначения чисел в системах счисления с основанием менее 10 мы используем те же цифры:

radix 9 - 0, 1, 2, 3, 4, 5, 6, 7, 8;
radix 8 - 0, 1, 2, 3, 4, 5, 6, 7;
radix 2 - 0, 1 и т. д.

Если же основание системы счисления больше десяти, то есть больше, чем десять привычных нам чисел, то начинают использоваться буквы английского алфавита. Например, для обозначения чисел в системе счисления с основанием 11 "как цифра" будет использоваться буква A:

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A

В системе счисления с основанием 16 - буквы от A до F:

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

И так далее...

Правда, при определенном основании (при каком?) буквы английского алфавита закончатся...

Но нам это, пока что, глубоко фиолетово, так как работать мы будем только с тремя radix-ами: 10 (ну естественно), 16 и 2. Правда, если кто на ДВК поизучать это дело собирается, тому еще и radix 8 понадобится.

#7. Числа в любой системе счисления строятся аналогично десятичной. Только на "счетах" не с 10, а с другим количеством костяшек.

Например, когда мы пишем десятичное число 123, то имеем в виду следующее:

1 раз 100 (10 раз по 10)
+ 2 раза 10
+ 3 раза 1

Если же мы используем символы 123 для представления, например, шестнадцатеричного числа, то подразумеваем следующее:

1 раз 256 (16 раз по 16)
+ 2 раза 16
+ 3 раза 1

Короче - полный беспредел. Говорим одно, а подразумеваем другое. И последнее не для красного словца сказано. А потому, что так оно и есть...

Истина где-то рядом...

#8. Трудность у вас может возникнуть при использовании символов А, В, С и т. д. Чтобы решить эту проблему раз и навсегда, необходимо на зубок вызубрить ма-а-аленькую табличку "соответствия" между употребляемыми в "компьютерном деле" систем счисления:

radix 10	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
radix 16	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
radix 2	0	1	10	11	100	101	110	111	1000	1001	1010	1011	1100	1101	1110	1111

Следуя этой таблице, число 5BC в шестнадцатеричном формате "строится" так:

5 раз 256 (16 раз по 16)
+ 11 раз 16 (10 - потому что по таблице B как бы равно 11)
+ 12 раз 1

А теперь, если пораскинуть мозгами, с легкостью переведем 5BC из шестнадцатеричной в десятичную систему счисления:

$$5*256 + 11*16 + 12 = 1468$$

Вот и объединили цифры с буквами. Пространство со временем поучимся объединять немного позже - если не испугаетесь сложностей низкоуровневого программирования.

В общем-то решать вам. В Delphi тоже много чего объединять можно.

#9. Двоичная система по-компьютерному обзывается "bin", "родная" десятичная - "dec", а шестнадцатеричная - "hex". Это так компьютерщики обозвали те системы счисления, с которыми имеют дело... А обозвали потому, что у них ведь полный бардак в голове, оказывается!

Например, 10 - что это за число? Да это вообще не число! Палка и барабан - и только... А вот 10d или же 10_10 - уже понятно, что это - число, соответствующее количеству пальцев на обеих руках. И именно на обеих, а не на двух. Почему не на двух? - А потому что на двух в какой системе? Ежели в двоичной, так это на десяти! То бишь 100, если в десятичной...

Вот и придумали программисты после числа букву писать - b, d или h. А самые ленивые еще и директиву специальную придумали: напишут в самом начале программы какой-нибудь .radix 16 и будут автоматически все числа, которые без этих букв, за шестнадцатеричные приниматься.

#10. Еще немного про перевод между "радиками". (Вообще-то это плевое дело, конечно, если представляешь себе, что такое "совокупность приемов представления обозначения натуральных чисел").

Например, преобразование числа 42936 из десятичного в шестнадцатеричный формат проводится следующим образом (в скобках - остаток):

```
42936/16 = 2683(8) 8 - младшая цифра
2683/16 = 167(11)  В (11d=Bh по таблице)
167/16 = 10(7)    7
10/16 = 0(10)     А - старшая цифра
-----
42936d=A7B8h
```

А вот и обратный процесс - перевод из HEX в DEC числа A7B8h:

```
10*16=160 160+7=167 (10 - потому что Ah=10d)
167*16=2672 2672+11=2683
2683*16=42928 42928+8=42936
-----
```

Преобразования чисел в системы счисления с другим основанием проводятся аналогично... Счеты! Обыкновенные счеты, только с "плавающим" числом "костяшек" на каждом "прутике"...

#11. Если честно, то конкретный "рисунок" цифр - единица там палкой обозначается, двойка - лебедем - это все лишь историческая случайность. Мы запросто можем считать в "троичной" системе счисления с цифрами %, *, _ (где запятая - это знак препинания, а вовсе не число):

%, *, _, **%, **, *_%, _%, _*, __, **%, **%, *%_, **%...

Или использовать родные цифры в десятичной системе счисления, но по другому "вектору упорядоченных цифр" - 1324890576:

1, 3, 2, 4, 8, 9, 0, 5, 7, 6, 31, 33, 34, 34, 38, 39, 30, 35, 37...

Правда, этим немножко затрудняется понимание происходящего? А ведь тоже десятичная система! И рисунок цифр как бы знакомый :-)))

Или вообще считать в 256-ричной системе счисления, используя в качестве "рисунка цифр" таблицу ASCII-символов! (По сравнению с вами, извращенцами, любой Биллгейтс будет девственником казаться!!).

#12. Теперь самая интересная часть Марлезонского балета.

Компьютер, как известно, считает только в двоичной системе счисления. Человеку привычна десятичная. Так нахрена еще и шестнадцатеричную какую-то знать нужно?

Все очень просто. В умных книжках пишут, что "шестнадцатеричная нотация является удобной формой представления двоичных чисел". Что это значит?

Переведите число A23F из шестнадцатеричной "нотации" в двоичную. (Один из возможных алгоритм приведен в п.10.). В результате длительных манипуляций у вас должно получиться 1010001000111111.

А теперь еще раз посмотрите на таблицу в п. 8. (которую вы как бы уже и выучили) и попробуйте то же самое сделать в уме :

```
Ah=1010b
2h=0010b
3h=0011b
Fh=1111b
A23Fh = 1010 0010 0011 1111b
```

Каждой шестнадцатеричной цифре соответствует тетрада (4 штуки) ноликов и единичек. Все, что потом нужно сделать - "состыковать" эти тетрады. Круто? Вас еще не то ждет!

#13. Кстати (наверняка вы это уже знаете):

```
00000123 = 123, но!!
123 <> 12300000
```

... но это так... кстати...

#14. И, напоследок, еще несколько слов про HEX и BIN :). Зайдите в Norton Commander, наведите указатель на какой-нибудь файл и нажмите там F3. А когда он откроется - на F4. Там какие-то нездоровые циферки попарно сгруппированы. Это и есть "нолики и единички" (которыми в компьютере все-все-все описывается), но в шестнадцатеричном формате...

Следует основательно разобраться с системой счисления. Минимум, что должен вынести из этой главы юзвер, вступивший на скользкий путь низкоуровневого программирования - это научиться переводить числа между DEC, HEX и BIN... хе-хе... В УМЕ!

1.2. Регистры

#1. Наверняка вы имеете представление о том, что такое переменная. Наиболее продвинутые даже знают, что переменная имеет тип. Кажется вполне естественным, что любой высокоуровневый язык

программирования позволяет создавать любое количество переменных того или иного типа...

Так вот, господа - при программировании на ассемблере вас ждет большая неожиданность. Потому что для всех ваших навороченных вычислений разрешается использовать только несколько переменных с фиксированными "именами собственными" и имеющих фиксированную "длину". Эти "предопределенные переменные" называются регистрами, и каждая из них имеет свою специализацию.

О специализации нам пока что говорить рано, описание наподобие "регистр-указатель базы кадра стека" вам вряд ли о чем-то скажет. Поэтому для начала познакомимся только с так называемыми **registрами общего назначения** (РОН), и то не со всеми, а только с четырьмя основными, которые являются своего рода "рабочими лошадками" микропроцессора.

Вот их "имена собственные" - AX, CX, DX, BX (именно в такой последовательности они "упорядочены" в Intel'овских микропроцессорах).

А сейчас мы поближе посмотрим на эти "рабочие лошадки" микропроцессора.

1. Запустите программу DEBUG.EXE1.

2. Когда появится приглашение в виде "минусика", введите букву "R" (можно и "r" - регистр символов значения не имеет) и нажмите на "Enter".

Не правда ли, весьма похоже на то, что показывают в художественных фильмах про хакеров?

```
AX=0000 BX=0000 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=18B2 ES=18B2 SS=18B2 CS=18B2 IP=0100 NV UP EI PL NZ NA PO NC
18B2:0100 6A          DB          6A
```

- Ну и что это такое? - скептически спросите вы.

- А черт его знает! Будем разбираться!

#2. То, что у вас должно появиться - это список доступных регистров и текущее значение каждого из них. Как видите, значения регистров AX, BX, CX, DX равны 0. Не правда ли, создается впечатление, что они просто-напросто ждут того, чтобы в них внесли какое-либо значение?

Природа не терпит пустоты. Писателей приводит в ужас чистый лист бумаги...

Весьма скоро и вы при виде "пустых" регистров будете испытывать непреодолимое наркотическое желание чем-нибудь их заполнить...

Однако прежде чем мы сделаем это в первый раз, давайте уточним тип этих "переменных".

А он очень простой, этот тип - шестнадцатеричное число в диапазоне 0...FFFF. Или, если в BIN, то - от 0 до 1111 1111 1111 1111.

Маловато будет? А вот создатели первых "IBM-совместимых" :) компьютеров посчитали, что и этого много! 16-битная переменная еще и на две части дробится - для совместимости с языками ассемблера для предыдущих моделей процессора Intel, работавших только с 8-битными регистрами; да и просто ради удобства...

В общем, в умных книжках рисуют вот такую вот "нездоровую" схемку3:

AX	
AH	AL

А означает она следующее.

Физически существует один регистр - AX, а вот логически он делится на два - на старшую (AH) и младшую (AL) части (от английского - high и low).

Очевидно, что присвоить AX значение, например, 72F9h, мы можем следующими способами:

1. AX = 72F9h (одной командой);
2. AH = 72h; AL = F9h (двумя командами).

Точно так же присвоить значение 78h регистру AH можно двумя способами:

1. AH = 78h;
2. AX = 7800h.

То же самое, но для регистра AL:

1. AL = 78h;
2. AX = 0078h .

Тех, кого смущают числа с буквами, мы со зловредной ухмылкой отсылаем к *1.1. Система счисления :-]*

#3. Если рассматривать регистр "целиком", то каждый из них имеет "длину" 16 бит, которые принято нумеровать справа налево4 . Так, для числа 2F4Dh, внесенного, например, в регистр AX, мы можем нарисовать такую вот "навороченную" табличку:

AX	2F4D															
AH	AL	2F	4D													
Значение бита	0	0	1	0	1	1	1	1	0	1	0	0	1	1	0	1
Номер бита	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Тетрады	Старшая AH	Младшая AH	Старшая AL	Младшая AL												

Внимательно смотрим на таблицу: одной шестнадцатеричной цифре соответствует тетрада двоичных цифр (4 шт., они же - 4 бита). "Емкость" регистров AH и AL - две тетрады, т. е. 8 бит. Точно такую "длину" имеют: коды символов, скан-коды клавиш, номера функций прерываний и куча всего прочего, чего вы пока еще не знаете.

Емкость AX (состоящего из двух половинок) - 4 тетрады, т. е. 16 бит; они же (эти 16 бит) иначе еще называются "словом"...

#4. "Принудительно" присвоить регистру значение можно при помощи той же команды "R", только с параметром "имя собственное регистра".

Например, команда

- R AX [Enter]

выбросит вам на монитор

:

Введите после двоеточия, например, число 123 и снова нажмите на Enter:

:123 [Enter]

На дисплее опять появится приглашение "-", на которое мы отвечаем командой "R" без параметров и таким образом вновь просматриваем значения наших регистров:

```
AX=0123 BX=0000 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=18B2 ES=18B2 SS=18B2 CS=18B2 IP=0100 NV UP EI PL NZ NA PO NC
18B2:0100 6A          DB          6A
```

Смотрим внимательно - AX=0123, что и требовалось доказать...

Примечания

1). В W9X она находится в папке WINDOWS\COMMAND\ В Y2K и XP - WINDOWS\SYSTEM32\ В обоих случаях достаточно набрать в командной строке "debug", чтобы она запустилась.

2). Не забудьте, что при тогдашней технологической базе и это было большим прорывом. А экстенсивное расширение, например, разрядности, во-первых, нужно правильно предвидеть (вспомните, сколько в том же MS-DOS закладок на будущее, которые никуда не пошли за ненадобностью), а во-вторых, правильно оценить (в буквальном смысле). Неужели вы думаете, что, например, производители памяти не могут легким мановением руки увеличить ширину шины, соединяющую память с процессором? Могут, но во-первых - это резко повысит стоимость памяти, а во-вторых - не гарантирует повышения производительности.

3). Впоследствии мы немного усложним эту схемку - регистры современных процессоров 32-разрядные и называются немного иначе ;)

4). "Первый справа" бит мы будем называть "нулевым". Однако нам попадались руководства, в которых это же бит обозван как "первый". Можно долго обсуждать тонкости русского языка (которые, к сожалению, не всегда понимает переводчик), однако это выходит за рамки данной книги. Просто имейте это ввиду, что можете с этим столкнуться, и будьте бдительнее, читая документацию.

1.3. Память

#1. Первым видом памяти, с которым мы войдем (придется!) в тесный физический контакт, будет оперативная, она же - RAM (от английского - *Random Access Memory*). Оперативная память - это своего рода "рабочая площадка", по которой суетится этаким шустрый многорукий дядька-процессор - чего-то там собирает, от кучи к куче бегают, всех ругает... :)

Оперативная память - это ряд пронумерованных ячеек размером в байт. Мы можем получить доступ к первому байту памяти, ко второму, к третьему и т.д.

Короче - пришло время испробовать еще одну команду из скромного арсенала DEBUG'a! Запустите debug и введите команду D (от английского - DUMP).

"Картинка", которую вы увидели, называется "дамп памяти" (что в переводе с английского означает "свалка") и она насыщена не только важной информацией, но и специальной низкоуровневой энергетикой. Да чего уж там греха таить - каждый ассемблерщик знает, что рассматривание дампа памяти поднимает настроение, жизненный тонус и другие, не менее важные вещи ;)

```
18B2:0100  6A 00 68 4B 01 66 83 7E-E0 00 74 05 B8 4C 01 EB   j.hk.f.~.t..L..
18B2:0110  03 B8 4A 01 2B D2 52 50-57 FF 36 C4 34 00 A1 18   ..J.+.RPW.6.4...
18B2:0120  F7 7F 83 C4 12 56 9A 16-44 F7 7F FF 76 FE 9A 59   ....V..D...v..Y
18B2:0130  04 8F 17 B8 FE FF 1F 5E-5F C9 CA 06 00 90 C8 54   .....^.....T
18B2:0140  04 00 57 56 8B 76 04 33-C0 89 46 D6 B9 0B 00 8D   ..WV.v.3..F....
18B2:0150  7E D8 16 07 F3 AB 89 46-BC B9 0C 00 8D 7E BE F3   ~.....F.....~..
18B2:0160  AB 9A 21 9C 8F 17 89 46-FA A1 08 01 8E 46 06 26   ..!....F.....F.&
18B2:0170  39 44 02 0F 84 55 01 C7-46 BC 1A 00 C4 5E 0C 26   9D...U..F....^.&
```

Слева - это адрес памяти. В центре - 16 столбцов из спаренных цифр...

А здесь и повториться лишний раз не грех. Каждая пара шестнадцатеричных цифр - это байт. Смотрите внимательно на дамп! Байт по адресу 100 имеет значение 6A, байт по адресу 101 - 00, байт по адресу 102 - 68... Эти "сладкие парочки" - и есть неделимая "единица адресации" оперативной памяти.

Тех, кого смущает наличие буковок в адресе, в очередной раз отсылаем ознакомиться с шестнадцатеричной системой счисления, так как все числа, отображаемые программой debug - именно шестнадцатеричные.

И, наконец, столбец справа - это символы, соответствующие шестнадцатеричным кодам центрального столбца (например, коду 6A соответствует символ J). Большинству кодов не

соответствует никакой из "печатных символов" - таким в колонке справа соответствуют точки.

#2. А теперь потренируем наши пальчики дампитровать память - пройдемся по некоторым "историческим местам" нашей оперативной памяти... Для этого мы будем вводить команду D с параметром.

Например, команда (параметр L8 означает "вывести 8 байтов"):

```
- D FFFF:5 L8 [Enter]
```

покажет вам системную дату в правом столбце дампа.

Короче, искателям приключений выдаем "простыню" самых интересных адресов (большинство слов в описании вам пока должны быть непонятны, но вы не пугайтесь - понимание придет!).

- **0:417** - два байта разрядов состояния клавиатуры. Они активно используются ROM-BIOS для управления интерпретаций действий клавиатуры. Изменение этих байтов изменяет значение нажатых клавиш (например, верхний или нижний регистр).
- **0:41A** - слово по этому адресу указывает на начало буфера BIOS для ввода с клавиатуры, расположенного начиная с адреса 41E. В этом буфере хранятся и ждут обработки результаты нажатия на клавиши. Конец буфера - слово по адресу 41C.
- **0:43E** - байт указывает, необходима ли проверка дискеты перед подводом головки на дорожку. Разряды 0...3 соответствуют дисководам 0...3. Если разряд установлен в 0, то необходима проверка дискеты. Как правило, вы можете обнаружить, что разряд установлен в 0, если при предыдущем обращении к дисководу имели место какие-либо проблемы. Например, разряд проверки будет равен 0, если вы попытаетесь запросить каталог на дисковом, на котором нет дискеты, и затем на запрос, появившийся на экране дисплея: "Not ready reading drive B: Abort, Retry, Ignore?" вы ответите: "A".
- **0:44C** (2 байта) - длина регенерации экрана. Это число байтов, используемых для страницы экрана. Зависит от режима.
- **0:44E** (2 байта) - смещение для адреса начала текущей страницы в памяти дисплея. Этот адрес указывает, какая страница в данный момент используется (маленькая, но неприятная подробность - это смещение внутри текущего сегмента видеопамати, без учета самого сегмента. Например, для нулевой страницы смещение всегда будет равно нулю.)
- **0:460** (2 байта) - размер курсора, представленный в виде диапазона строк развертки. Первый байт задает конечную, а второй - начальную строку развертки.
- **0:449** - значение этого байта определяет текущий видеорежим. Для расшифровки требуется целая таблица. Например, 3 - 80-колоный текст, 16 цветов; 13h (19) - 256-цветный графический режим 320x200 и т. д.

Ну и хватит для первого раза. Кому мало - ищите дополнительную документацию :-р "

1.4. Программа

#1. Любая программа выполняется последовательно (мы ведь пока обсуждаем "простой" IBM PC, а не какой-нибудь крутой векторный параллельный суперкомпьютер). То есть пока не выполнилась текущая "строка" (инструкция) программы, следующая не выполнится. Совсем другой вопрос, какая "строка" будет выполнена после "текущей" (здесь мы имеем дело со всевозможными логическими "ветвлениями", "циклами" и т. д.), или же строчку из какой программы процессор выполнит следующей, а какая - будет ждать своей очереди (так называемая "многозадачность", которую пока трогать не будем - в большинстве случаев мы можем прекрасно прожить и без нее, поскольку все заботы об этом все равно берут на себя операционные системы).

Итак, у нас есть оперативная память, в которую загружается программа перед ее выполнением (сразу же по нажатию на Enter из Norton Commander). Операционная система, которая, собственно, и загружает программу, сообщает процессору, что надо начать обрабатывать команды, которые в памяти начинаются с такого-то адреса. И здесь первый подводный камень, вернее скала, которую

трудно не заметить.

Начало программы в памяти процессор различает легко - ему указывает на это командный интерпретатор, а вот конец программы программист должен указывать сам!

Каким образом? А очень легко! Компьютер "распознает" как выход из программы специальную последовательность байтов. Например, для исполнимых файлов типа com (именно с этим типом файлов мы будем работать на начальном этапе) достаточно последовательности CD и 20.

Пробуем-проверяем? Ну конечно же! Только для этого вам понадобится какой-нибудь шестнадцатеричный редактор, например, HexWorkshop.

Все очень просто - создаем новый файл, единственным содержимым которого является последовательность CD 20, и сохраняем его как, например, myprg_1.com. Если вы позаботились о том, чтобы после CD 20 не было никаких прочих символов, то исполнимая программа будет "весить" только 2 байта.

Запускать это ваше первое творение лучше из Norton или Volcov Commander (все же это пока что DOS'овская программка).

Что же она делает, эта 2-байтовая малышка? А ничего, просто этот файл обладает двумя важными свойствами:

- это - программа;
- эта программа - программа с корректным выходом.

Последнее и является единственным, что она пока что может делать (корректно выгружаться из памяти)...

Еще к вопросу о выгружаемости - если после CD 20 вы напишите еще что-нибудь (чепуху), она все равно будет проигнорирована. Дело до ее выполнения просто-напросто не дойдет. Другое дело - если вы напишите чепуху до...

#2. Честно говоря, опасно при низкоуровневом программировании чепуху писать. Можно невзначай и винт отформатировать :)). Поэтому лабуду писать не будем, вернее - будем, но не лабуду...

Итак, продолжим наше извращение. Познакомимся еще с некоторыми "машинными командами" (в нашем случае - последовательностями шестнадцатеричных цифр).

- B82301 - внести значение 0123h в AX;
- 052500 - прибавить значение 0025h к AX;
- 8BD8 - переслать содержимое AX в BX;
- 03D8 - прибавить содержимое AX к BX;
- 8BCB - переслать содержимое BX в CX;
- 31C0 - очистка AX;
- CD20 - конец программы. Передача управления операционной системе.

Вот и давайте создадим еще одну программу типа com со следующим "шестнадцатеричным содержимым":

```
B8-23-01-05-25-00-8B-D8-03-D8-8B-CB-31-C0-CD-20
```

Если вы все ввели правильно, то прога у вас без проблем запустится, а операционная система не будет ругаться... Правда, визуально (в смысле на мониторе) вы ее работу так и не заметите, но поверьте на слово - она работает! В этом вы еще убедитесь, когда посмотрите на ее работу изнутри - не различающими цветов глазами компьютера... Только сначала еще немного теории...

#3. Теперь поговорим о втором подводном камне :). Один из принципов фон Неймана звучит приблизительно так: машине безразлично целевое назначение данных... Одна и та же цепочка битов может быть и машинными командами, и данными (например, символами, выраженными в

виде кодов - есть такая "таблица символов ASCII", наверняка вы ее знаете).

Что из этого следует? А то, что компьютеру нужно указывать, что подразумевается под той или иной "простыней" из битов - данные или код.

На высоком уровне это делает операционная система. Например, она не пытается загрузить в память для выполнения файлы с расширениями, отличными от COM, EXE и BAT (последний вообще не из этой оперы, но принцип сохраняется).

Хотя..., вы всегда можете поэкспериментировать! Смените, например, у какого-нибудь текстового файла тип с TXT на COM и попробуйте его запустить на выполнение (хотя мы это делать настоятельно не рекомендуем!). В большинстве случаев ваш компьютер зависнет! Потому что:

1. Он пытается интерпретировать данные как код. Соответственно, в процессор "попадает" всякая ерунда.
2. Вряд ли он натолкнется на последовательность CD 20 в вашем тексте :). Даже в том случае, если этот код выполнится "успешно" - ваша программа не возвратит управление операционной системе, а пойдет выполняться хлам, содержащийся в оперативной памяти. Как-то - остатки ранее выполненных программ, куски чьих-то данных, интерпретированные как код... и прочая многочисленная ерунда...

Почти такой же эффект, но с потенциально большей разрушительной силой может получиться, если управление получит ИСПОРЧЕННЫЙ код, который вроде бы "в основном" правильный, но часть его вместо инициализации переменных и прочих подготовительных действий в лучшем случае ничего не делает, а в худшем портит другой код и данные...

Как вам тяжело "въехать" в смысл повествования, состоящего из кусков различных книг, так и компьютеру тяжело понять подобную "мешанину". С той лишь разницей, что любую "неинтересную книгу" вы можете использовать в качестве туалетной бумаги, а вот "компьютер" подобного права выбора лишен - он должен в это "въезжать", его процессор начинает перегреваться, а мозги кипят и вытекают через низкоуровневые порты ввода-вывода (командами IN и OUT соответственно).

#4. Еще немного идеологии. О программе, которая выполняется в памяти...

Сколько бы ни было "мозгов" в вашей навороченной тачке, любая программа выполняется в 640 килобайтах "нижней" (или основной) памяти. Если отнять от этой цифры "резидентную часть" операционной системы, многочисленные драйвера и т.д., то оставшееся и есть объем памяти, в котором выполняется ваша программа. А остальные мегабайты - это место для кэширования диска, хранения промежуточных данных и т.п.

Страшно? Медитируйте!

#5. Как уже говорилось в #3, одна и та же последовательность битов в памяти может быть:

- кодом (т.е. что компьютеру нужно делать) - последовательностью инструкций;
- данными (т.е. с чем компьютеру нужно выполнять ту или иную работу). Именно данные являются исходной "задачей" и конечным результатом работы процессора.
- стеком - это область памяти, позволяющая писать реентерабельный/рекурсивный код и служащая для хранения адресов возврата и локальных данных и передачи параметров.

Соответственно, и программа состоит из трех частей (сегментов): сегмента данных (data), сегмента кода (code) и сегмента стека (stack)...

Оставим пока что "гнилой базар" про смысл словосочетаний "реентерабельный/рекурсивный код" и "адрес возврата". Чтобы не затруднять себе понимание происходящего, мы попытаемся абстрагироваться от всех этих ужасающих вещей и для начала заняться только кодом.

#6. Помните, как в конце фильма "Matrix" Нео в конце концов увидел ее - черно-зеленую "матрицу"? Сейчас с вами произойдет нечто подобное! ;)

Посмотрите на машинные коды, и "что они делают" в #2. Немножко дополним эту "простыню". Например, командой "внести значение" 1234 последовательно в каждый из "регистров общего пользования":

```
B83412 - AX=1234
BB3412 - BX=1234
B93412 - CX=1234
BA3412 - DX=1234
```

Наиболее наблюдательные должны для себя отметить, что первый байт - это команда "переместить в регистр", а второй и третий - само число, только байты почему-то "наоборот".

Однако никто не пишет программы в шестнадцатеричных редакторах! Никто! Это большая глупость! Единственное, зачем мы вам про это рассказываем - это чтобы вы поняли, что могут означать загадочные пары шестнадцатеричных цифр в дампе...

Нет необходимости заучивать, что B8 - это "переместить в регистр AX", BB - "переместить в регистр BX" и так далее... Когда-нибудь это может пригодиться тому, кто будет писать компилятор, умеющий генерировать исполняемый код, упаковщики исполняемых файлов, самомодифицирующийся код или, на худой конец, конструкторы полиморфных самошифрующихся вирусов. Но это мы оставим на будущее...

Все намного проще!

В этом вы можете убедиться, загрузив вашу программу myprg_1.com в debug (например, командной строкой

```
debug myprg_1.com
```

и введя команду "u".

А вот дальше начинается самое интересное :)))

#7. Вот что вы должны увидеть:

```
11B7:0100 B82301 MOV AX,0123 ; Внести значение 0123h в AX
11B7:0103 052500 ADD AX,0025 ; Прибавить значение 0025h к AX
11B7:0106 8BD8 MOV BX,AX ; Переслать содержимое AX в BX
11B7:0108 03D8 ADD BX,AX ; Прибавить содержимое AX к BX
11B7:010A 8BCB MOV CX,BX ; Переслать содержимое BX в CX
11B7:010C 31C0 XOR AX,AX ; Очистка AX
11B7:010E CD20 INT 20 ; Конец программы
```

Возвратившись к #2, перенесем сюда "описание" машинных команд.

Эти mov, add, xor, int - так называемые "мнемонические команды" (более или менее понятные человеку), на основе которых формируется (это debug делает) "машинный код". Не правда ли, так намного легче?

Соответственно, вместо шестнадцатеричных кодов мы легко могли вводить эти команды при помощи команды "A" (однако этим мы займемся позже).

#8. А теперь мы выполним нашу программу пошагово - произведем так называемую "трассировку" при помощи команды "T".

Итак, вводим "T" и жмем на Enter!

Вот что мы видим:

```
AX=0123 BX=0000 CX=0010 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=11B7 ES=11B7 SS=11B7 CS=11B7 IP=0103 NV UP EI PL NZ NA PO NC
11B7:0103 052500 ADD AX,0025
```

Смотрим на значение AX и вспоминаем предыдущую инструкцию - "внести значение 0123h в AX". Внесли? И правда! А в самом низу - код и мнемоника команды, которая будет выполняться

следующей...

Вводим команду "Т" снова:

```
AX=0148 BX=0000 CX=0010 DX=0000 SP=FFFF BP=0000 SI=0000 DI=0000
DS=11B7 ES=11B7 SS=11B7 CS=11B7 IP=0106 NV UP EI PL NZ NA PE NC
11B7:0106 8BD8 MOV BX,AX
```

AX=0148 - "прибавить значение 0025h к AX". Сделали? Сделали!!

Вводим команду "Т" снова:

```
AX=0148 BX=0148 CX=0010 DX=0000 SP=FFFF BP=0000 SI=0000 DI=0000
DS=11B7 ES=11B7 SS=11B7 CS=11B7 IP=0108 NV UP EI PL NZ NA PE NC
11B7:0108 03D8 ADD BX,AX
```

AX=0148=BX - "переслать содержимое AX в BX". Сделали? Сделали!!

Вводим команду "Т" снова:

```
AX=0148 BX=0290 CX=0010 DX=0000 SP=FFFF BP=0000 SI=0000 DI=0000
DS=11B7 ES=11B7 SS=11B7 CS=11B7 IP=010A NV UP EI PL NZ AC PE NC
11B7:010A 8BCB MOV CX,BX
```

"Прибавить содержимое AX к BX". Оно? А то!

Вводим команду "Т" снова:

```
AX=0148 BX=0290 CX=0290 DX=0000 SP=FFFF BP=0000 SI=0000 DI=0000
DS=11B7 ES=11B7 SS=11B7 CS=11B7 IP=010C NV UP EI PL NZ AC PE NC
11B7:010C 31C0 XOR AX,AX
```

"Переслать содержимое BX в CX". Сделано!

Вводим команду "Т" снова:

```
AX=0000 BX=0290 CX=0290 DX=0000 SP=FFFF BP=0000 SI=0000 DI=0000
DS=11B7 ES=11B7 SS=11B7 CS=11B7 IP=010E NV UP EI PL ZR NA PE NC
11B7:010E CD20 INT 20
```

"Очистка AX"? И точно: AX=0000!

Вводим команду "Т" снова... И ГРОМКО РУГАЕМСЯ!!

Потому что, по идее, сейчас наша программа должна была завершиться - у нас же там код выхода прописан, а она куда лезет? НОРы какие-то (если продолжать команду "Т" вводить), CALL 1085 (да вы продолжайте "трассировку", продолжайте!)

Для тех, кому лень продолжать жать на букву "Т", введите для разнообразия команду "G" (от английского GO). На монитор должна вывалиться надпись "Нормальное завершение работы программы".

- Уф , - должны сказать вы - *Работаем!*

А то!

#9. Только непонятно вот, почему вдруг между int 20 (CD 20) и надписью "Нормальное завершение работы программы" куча всяких "левых" непонятных команд (в том случае, если вы и дальше производили тарассировку, а не воспользовались "халявной" командой "G")?

А потому, дорогие наши, что вы имели счастье нарваться на прерывание (interrupt)!

Понимаете ли, завершить программу - дело непростое :). Нужно восстановить первоначальное значение регистров, восстановить переменные среды и кучу всего другого! Знаете, как это сложно?

Однако эта процедура насколько сложная, настолько и типичная для исполняемых программ. А по сему разработчики операционной системы решили избавить программистов от необходимости делать это вручную, и включили эту стандартную процедуру в ядро операционной системы. И

сказали: "да будешь ты (процедура обработки прерывания) вызываться как int 20, и будешь ты обеспечивать корректную передачу управления из выполняемой программы - назад в ядро". И стало так...

Ну, посудите сами, должна же операционная система ну хоть что-нибудь делать!!

1.5. Прерывания

#1. Прерывание - это СИГНАЛ процессору, что одно из устройств в компьютере НУЖДАЕТСЯ в обслуживании со стороны программного обеспечения. В развитие этой же идеи, программам позволили самим посылать запросы на обслуживание через механизм прерываний. Получив этот сигнал, процессор временно переключается на выполнение другой программы ("обработчика прерывания") с последующим ВОЗОБНОВЛЕНИЕМ выполнения ПРЕРВАННОЙ программы.

Когда же и "кем" генерируются эти "сигналы" (в смысле "прерывания")?

1. Многочисленными "схемами" компьютера, его устройствами. Например, соответствующее прерывание генерируется при нажатии клавиши на клавиатуре.
2. Также прерывания генерируются как "побочный продукт" при некоторых "необычных" ситуациях (например, при делении на "букву О"), из которых компьютеру хочешь, не хочешь, но приходится как-то выкручиваться...
3. Наконец, прерывания могут преднамеренно генерироваться программой, для того чтобы произвести то или иное "низкоуровневое" действие.

Когда процессор получает сигнал прерывания, он останавливает работу приложения и активизирует "программу обработки прерывания", соответствующую "номеру прерывания" (т.е. разных сигналов прерываний больше одного - точнее, их 256). После того как обработчик свое отработает, снова продолжает выполняться основная программа.

Для тех, кто не понял. Представьте себе, что вы сидите за компом и выполняете какую-либо работу. И вдруг ловите себя на мысли, что вам СРОЧНО НУЖНО сходить в туалет (терпеть вы больше уже не можете). Вот это СРОЧНО НУЖНО и есть сигнал-прерывание, по которому вы начинаете выполнять определенную СТАНДАРТНУЮ последовательность инструкций (программу обработки прерывания), как-то: встать, пойти туда-то, включить свет ... вернуться, сесть за комп и ПРОДОЛЖИТЬ РАБОТУ с того же самого места, на котором вы остановились перед выполнением программы "поход в туалет". В данном случае наш мозг выполняет роль процессора, наши внутренние органы сигнализируют мозгу о потребности в обслуживании, а само обслуживание проводится "программой-навыком", заложенным в процессе нашего развития и (хм!) воспитания.

#2. Программы обработки прерывания располагаются в оперативной памяти (ну а где же еще им располагаться?!) и, следовательно, имеют свой АДРЕС. Однако генератору прерывания этот адрес знать не обязательно :). Есть такая замечательная штука (спросите у тех, кто пишет вирусы) - таблица векторов прерываний. Это таблица соответствия номеров и адресов памяти, по которым находятся программы их обработки.

Почему "спросите у вирмейкеров?". А потому, что поменять адрес "программы обработки прерывания" на другой - проще пареной репы (мы этим еще займемся), в результате чего при запуске классической программы "HELLO, WORLD" может получиться еще более классический format c:...

Программы обработки прерывания автоматически сохраняют значения регистра флагов, регистра кодового сегмента CS и указателя инструкции IP, чтобы по завершении "обработки прерывания", к нашей безумной радости, снова возвратиться к выполняемой программе (просто программе)... Остальные регистры, содержимое которых меняется в обработчике, должен сохранять сам обработчик - и если он этого делать не будет, то нарушится выполнение основной программы. Ведь она даже не знает, что ее "ненадолго" прервали!

Однако на самом деле все намного сложнее :)). Но ведь это только "первое погружение" в прерывания, верно? А посему - пока что без особых "наворотов"...

#3. Одно прерывание мы с вами уже знаем. Это 20-е прерывание, обеспечившее "выход" из нашей СОМ-программы. Сегодня мы пойдем немножко дальше - помимо "выхода" попробуем поработать еще с одним прерыванием.

Итак, я достаю свой толстый талмуд с описанием прерываний и выбираю, каким бы это прерыванием вас занять на ближайшие 1/2 часа ;)...

Ну, например, вот одно симпатичное, под названием "прокрутить вверх активную страницу".

Внимательно читаем описание (и наши комментарии):

INT 10h, AH=06h (_1) - прокручивает вверх произвольное окно на дисплее на указанное количество строк.

ВХОДНЫЕ ПАРАМЕТРЫ: (_2)

- AH=06h; (_3)
- AL - число строк прокрутки (0...25) (AL=0 означает гашение всего окна); (_4)
- BH - атрибут, использованный в пустых строках (00h...FFh); (_5)
- CH - строка прокрутки - верхний левый угол; (_6)
- CL - столбец прокрутки - верхний левый угол;
- DH - строка прокрутки - нижний правый угол;
- DL - столбец прокрутки - нижний правый угол;

Далее представим входные параметры в виде таблички: (_7)

АН	06h	AL	Число строк
ВН	Атрибут	BL	Не имеет значения
СН	Строка (верх)	CL	Столбец (верх)
ДН	Строка (низ)	DL	Столбец (низ)

Плюс подробнейшее толкование, что подразумевается под словом "атрибут" (регистр ВН):

Цвет символа (разряды 3210)	0000	черный		Цвет фона (разряды 654)
	0001	синий		
	0010	зеленый		
	0011	голубой		
	0100	красный		
	0101	пурпурный		
	0110	коричневый		
	0111	белый		
Мерцание (разряд 7)	1000	серый		Повышенная яркость (разряд 3)
	1001	светло-синий		
	1010	светло-зеленый		
	1011	светло-голубой		
	1100	светло-красный		
	1101	светло-пурпурный		
	1110	желтый		
	1111	яркий белый		

ВЫХОДНЫЕ ПАРАМЕТРЫ: отсутствуют (т.е. ни один регистр не меняется). (_8)

Входные строки гасятся в нижней части окна. (_9)

Нормальное значение байта атрибута - 07h. (_10)

Совсем недавно, если бы вам показали подобное "описание", вы бы ничего в нем не поняли и ужаснулись. Теперь же, после прочтения предыдущих глав курса, в эти "таблицы" вы, более или менее, но "въехать" должны! Тем более, что сейчас я сделаю комментарии для... хм... "отстающих" учеников (внимательно смотрим на циферки в скобках):

_1. Черным по белому, в толстом талмуде, описывающем функции прерываний, написано: "Драйвер видео вызывается по команде INT 10h и выполняет все функции, относящиеся к управлению дисплеем".

И далее. "...ДЕЙСТВИЕ: после входа управление передается одной из 18 программ в соответствии с кодом функции в регистре АН. При использовании запрещенного кода функции, управление возвращается вызывающей программе.

НАЗНАЧЕНИЕ: прикладная программа может использовать INT 10h для прямого выполнения функций видео..."

Вот что из этого следует:

- выполнив команду INT 10h, мы ВЫПОЛНЯЕМ одну из "функций видео";
- так как функций видео - много, необходимо УКАЗАТЬ, КАКУЮ именно ФУНКЦИЮ из МНОЖЕСТВА мы хотим ВЫПОЛНИТЬ.

Дело в том, что "прерывание номер десять" - это не только "прокрутка окна", но и, например, "установка режима видео", "установка типа курсора", "установка палитры" и многое другое. Нас же интересует именно первое, поэтому из списка возможных значений (он приведен ниже) мы выбираем именно АН=06h.

Нижеследующая табличка называется "Функции, реализуемые драйвером видео":

АН=00h	Установить режим видео
АН=01h	Установить тип курсора
АН=02h	Установить позицию курсора
АН=03h	Прочитать позицию курсора
АН=04h	Прочитать позицию светового пера
АН=05h	Выбрать активную страницу видеопамати
АН=06h	Прокрутить вверх активную страницу
АН=07h	Прокрутить вниз активную страницу
АН=08h	Прочитать атрибут/символ
АН=09h	Записать символ/атрибут
АН=0Ah	Записать только символ
АН=0Bh	Установить палитру
АН=0Ch	Записать точку
АН=0Dh	Прочитать точку
АН=0Eh	Записать TTY
АН=0Fh	Прочитать текущее состояние видео
АН=13h	Записать строку

Соответственно, если перед выполнением INT 10 в регистре АН будет значение 06h, то выполнится именно "прокрутить вверх активную страницу", а не что-то другое из "простыни" функций десятого прерывания...

Теперь читаем описание дальше (смотрим на циферки в скобках):

_2. Входные параметры? Что тут может быть непонятного? Даже запуск ракеты с атомной боеголовкой требует прежде всего указать координаты цели... Чего уж тут говорить об обыкновенной функции?

_3. То, о чем мы уже говорили - номер функции из "простыни".

_4. Т.е. на СКОЛЬКО строчек прокручивать. Вспомните так называемый "скроллинг" в любой прикладной программе. На кнопки Up, Down подвешен скроллинг на одну строчку (не путать с координатами курсора), а вот на PgUp и PgDown - штук на 18 строк (AL=01h и AL=12h соответственно). А вот AL=0, вместо того чтобы вообще не скроллировать (по идее), поступает

наоборот - "скроллирует" все, что может.

_5. Скажем так - какого цвета будет окно и символы в нем после скроллинга.

_6. Как известно из школьного курса геометрии, прямоугольник можно построить по двум точкам. Это утверждение справедливо и для окна, в котором мы желаем проскроллить наш текст.

_7. Резюме того, что было написано выше.

_8. К примеру, попала ли наша ракета в цель или нет ;).

_9. Если бы мы использовали функцию 07h, то было бы глубокомысленно написано, что "строки гасятся в верхней части окна".

_10. Это то самое, которое в DOS по умолчанию. Т.е. белыми буквами на черном фоне. Правда, это 07h лучше все же рассматривать как 00000111b :) но это уже совсем другая проблема...

#4. А теперь мы напишем программу. Ручками, без использования компилятора. Запускаем наш любимый debug.exe, вводим команду "a" и судорожно стучим по клавиатуре:

```
-a
119A:0100 xor al,al ;гашение всего окна
119A:0102 mov bh,70 ;белое окно
119A:0104 mov ch,10 ;четыре координаты прямоугольника
119A:0106 mov cl,10
119A:0108 mov dh,20
119A:010A mov dl,20
119A:010C mov ah,06 ;такая-то функция прерывания
119A:010E int 10 ;Go!!
119A:0110 int 20 ;выход...
119A:0112
-r cx ;в CX - сколько байтов программы
;писать 112h-100h=12h

CX 0000
:12
-n @int10.com
-w
Запись: 00012 байтов
```

Сначала запускаем из-под Norton Commander. Затем запускаем из-под debug. Трассируем. Открываем в HEX-редакторе. Смотрим на "бессмыслицу" шестнадцатеричных циферек. Медитируем, медитируем и еще раз медитируем...

1.6. Немножко программируем и немножко отлаживаем

#1. Тем, кто не в курсе - НАСТОЯТЕЛЬНО рекомендую проштудировать предыдущие части курса, иначе "въехать" будет сложно. Тем же, кто внимательно читал предыдущие главы, нижеследующие упражнения для ума и пальцев покажутся детским лепетом...

Давайте немножко видоизменим программу, которую мы писали в прошлый раз. Сделаем так, чтобы наше "окошко скроллинга" располагалось более или менее посередине экрана.

```
:0100 XOR AL,AL ;ПРИМЕЧАНИЕ: с целью экономии пространства и
:0102 MOV BH,10 ;времени мы немножко сократили наш DEBUG-й
:0104 MOV CH,05 ;листинг, т.е. убрали адрес сегмента и
:0106 MOV CL,10 ;машинные коды, соответствующие мнемоническим
:0108 MOV DH,10 ;командам :))
:010A MOV DL,3E ;А так, конечно, в оригинале первая строка вот
:010C MOV AH,06 ;как должна выглядеть:
:010E INT 10 ;11B7:0100 30C0 XOR AL,AL
:0110 INT 20
```

Теперь наша задача - написать программу, которая последовательно выводит пять таких окошек, причем каждое последующее окно "вложено" в предыдущее, а значение атрибута в шестнадцатеричной нотации на 10 больше предыдущего.

Если мы будем программировать ЭТО линейно (именно так для начала), то очевидно, что все, что мы должны сделать - это заданное количество раз (5) заставить машину выполнить вышеуказанные операции, изменяя перед "запуском прерывания" значения регистров AL, BH, CX, DX (полное описание 6-й функции 10-го прерывания ищите в прошлых главах).

#2. Вот к каким умозаключениям вы должны были придти, пораскинув мозгами. За атрибут (то бишь цвет) у нас отвечает регистр BH. Он был равен 10h, а нужно на 10h больше... это, значит, 20h будет...

Ладно... CX (он же CH и CL, как известно) - это ТОЖЕ ШЕСТНАДЦАТЕРИЧНЫЕ координаты левого верхнего угла нашего окошка. Чтобы "окно в окне" получилось, все это нужно на строчку больше сделать и на колонку больше тоже, и все считать в HEX'е. Получается, что в регистр CH нужно вместо значения 05h внести 06h, а в регистр CL вместо 10h - 11h.

А еще можно одним махом в CX записать число 0510h той же командой mov.

Ладно... DX (DH и DL соответственно) - это координаты правого нижнего угла прямоугольника. DH=10h-1h=Fh и DL=3Eh-1h=3Dh.

Ну, а AL=0 и AH=6 - это уже и ежу понятно из описания данной функции (mov AH,6) данного прерывания (INT 10h).

Все, что осталось - это набить в debug'е после команды "а" эти мнемоники энное количество раз. (кажется 5). Набиваем!!

```
:0100 XOR AL,AL ;первый раз
:0102 MOV BH,10
:0104 MOV CH,05
:0106 MOV CL,10
:0108 MOV DH,10
:010A MOV DL,3E
:010C MOV AH,06
:010E INT 10
:0110 INT 20
:0112 XOR AL,AL ;второй раз
:0114 MOV BH,10
:0116 MOV CH,06
:0118 MOV CL,11
:011A MOV DH,0F
:011C MOV DL,3D
:011E MOV AH,06
:0120 INT 10
:0122 INT 20 ;третий раз
:0124 XOR AL,AL
: и т.д
```

Правда, красивые циферки-буквы? Набиваем-набиваем! Если сейчас к вам подойдут недЗенствующие приятели/коллеги и посмотрят, что вы тут колупаете, то ни черта не поймут и покрутят пальцем у виска. Привыкайте к этому. Только не говорите им, что пытаетесь сейчас получить Матрицу, ПОТОМУ ЧТО это неправда. А неправда это потому, что сейчас Матрица в очередной раз обманула вас!

#3. ВСЕ ПОТОМУ, ЧТО МОЗГАМИ ДУМАТЬ НАДО, А НЕ ТОЛЬКО СЛЕПО СЛЕДОВАТЬ РУКОВОДСТВУ!

У вас только первое окно прорисовывается, сразу же после чего программа натолкнется на INT 20h и благополучно завершится! А следовательно, и все, что после первого CD 20 написано будет - останется проигнорированным! Исправляйте! (Т.е. уберите все INT 20 КРОМЕ ПОСЛЕДНЕГО).

Второй момент. ВОЗВРАЩАЕТ ЛИ это прерывание ЧТО-НИБУДЬ В РЕГИСТР AX? Смотрите описание. Ничего? Ну так какого черта тогда по новой вводить XOR AL,AL и MOV AH,06 и переприсваивать AH значение 6h, если и без того AH = 6h? Один раз ввести - более чем достаточно!

Скажите, какая мелочь- байтом больше, байтом меньше! А я скажу вот что - на то он и assembler, чтобы "байтом меньше".

Исправляйте!

#4. - Исправляйте? - возмутитесь вы - *Да это же по-новому все вводить нужно!*

- *По-новому?* - возмутимся мы в свою очередь! - *Зачем по-новому? Вы что, с ума сошли?*

1. Что вам мешает после команды "а" указать адрес, который вы желаете переассемблировать? И благополучно заменить старую команду на новую!

- *А что делать, если не переассемблировать нужно, а вообще удалить?*

2. Существует куча способов, что вы в самом-то деле! Например, в HEX Workshop с блоками шестнадцатеричных цифр запросто можно работать. Да и в других программах это можно делать - например, в HIEW или даже в Volcov Commander.

Кстати, если процессор встретит команду NOP, то он просто побездельничает некоторое очень короткое время.

ПРОБУЙТЕ!! В конце концов, ваша прога должна принять такой вот вид:

```
:0100 XOR AL,AL ;окошко первое
:0102 MOV BH,10
:0104 MOV CH,05
:0106 MOV CL,10
:0108 MOV DH,10
:010A MOV DL,3E
:010C MOV AH,06
:010E INT 10
:0110 MOV BH,20 ;окошко второе
:0112 MOV CH,06
:0114 MOV CL,11
:0116 MOV DH,0F
:0118 MOV DL,3D
:011A INT 10
:011C MOV BH,30 ;окошко третье
:011E MOV CH,07
:0120 MOV CL,12
:0122 MOV DH,0E
:0124 MOV DL,3C
:0126 INT 10
:0128 MOV BH,40 ;окошко четвертое
:012A MOV CH,08
:012C MOV CL,13
:012E MOV DH,0D
:0130 MOV DL,3B
:0132 INT 10
:0134 MOV BH,50 ;окошко пятое
:0136 MOV CH,09
:0138 MOV CL,14
:013A MOV DH,0C
:013C MOV DL,3A
:013E INT 10
:0140 INT 20 ;конец проги...
```

О, да! Получившаяся у вас программа написана долго и бездарно! Имейте это в виду :)). Мы же торжественно обещаем, что в последующих главах обязательно ее усовершенствуем. Да, вот еще что - особенно извращенные могут попытаться заменить INT 20 на JMP 100. Получится, конечно, не ахти, но все же - "анимация" ;)

#5. А теперь мы попробуем ОТЛАДОЧНЫЙ прием! Все кракеры его знают и пользуются им для взлома софта. Имейте в виду, пока вы будете использовать его для своих исполняемых программ - вы программер, исправляющий ошибки, а как только попытаетесь использовать это для отвязки чужой программы от какого-нибудь серийного номера - ваша деятельность станет считаться

неэтичной или незаконной. Так что думайте сами, что лучше - флаг в руки или барабан вместе с петлей на шею.

Итак, вводим приблизительно такую командную строку - *debug имя_проги.com* или же подгружаем прогу в отладчик командой "I" (от слова load) и трассируем, как вы уже неоднократно это делали.

Цель - "на лету" (без изменения кода) заставить первое окошко "рисоваться" не синим (BH=10h), а красным (BH=40h) цветом.

Мы просто приведем вам последовательность действий, а вывод "зачем это нужно" и прочие возможные выводы вы уже сами делать будете. Ок?

```
-t
AX=0000 BX=0000 CX=0043 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=11B7 ES=11B7 SS=11B7 CS=11B7 IP=0102 NV UP EI PL ZR NA PE NC
11B7:0102 B710 MOV BH,10
-
```

Состояние: обнулится регистр AX (первую команду MOV AL,AL мы не видим). Процессор готовится выполнить команду MOV BH,10. Дадим ему это сделать!

```
-t
AX=0000 BX=1000 CX=0043 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=11B7 ES=11B7 SS=11B7 CS=11B7 IP=0104 NV UP EI PL ZR NA PE NC
11B7:0104 B505 MOV CH,05
-
```

Состояние - в BX уже внесен код синего цвета, который нам по условию необходимо заменить на красный (т. е. заменить значение регистра BX с 1000h на 4000h).

Вот теперь-то мы и делаем это "на лету":

```
-r bx
BX 1000
:4000
-
```

А действительно ли сделали? Проверим!

```
-r
AX=0000 BX=4000 CX=0043 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=11B7 ES=11B7 SS=11B7 CS=11B7 IP=0104 NV UP EI PL ZR NA PE NC
11B7:0104 B505 MOV CH,05
-
```

Состояние? BH теперь равно 40h! Мы "вклинились" между строчками:

```
:0102 MOV BH,10
:0104 MOV CH,05
```

И изменили текущую цепь событий, заставив программу делать ТО, ЧТО НАМ НУЖНО! Поздравляю!

А дальше - вводим команду "g" и даем нашей тупорылой программе исполниться. 1:0 не в пользу Матрицы!

Медитируйте!

1.7. Стек

#1. В средней школе, где когда-то учился автор, учителем физкультуры был настоящий зверь. Помимо того, что он заставлял нас школьников бегать-прыгать-подтягиваться, у него еще любимое наказание было - проходило оно в так называемом "зале тяжелой атлетики".

Что такое тяжелая атлетика вы наверняка знаете. Видели по телевизору, когда выходит на помост этакий здоровяк, и рвет собственное здоровье, поднимая штангу.

Штанга - это такая "палка", по бокам которой навешиваются так называемые "блины" - круглые плоские с дыркой посередине диски невероятной тяжести.

Хранятся же эти диски на штырях, которые представляют собой те же "палки", но воткнутые вертикально в пол. На них и хранились диски - один на другой положенные.

"Наказание" заключалось вот в чем - тот "педагог" заставлял нерадивого ученика комплектовать штангу! Это элементарно сделать, если диски просто валяются на полу. Но когда они аккуратно сложены на штырь - это намного сложнее :(.

Садист! Чтобы достать со штыря диск заданной тяжести (который обычно находился внизу) необходимо было снять со штыря все "вышележащие" диски, достать самый нижний, а остальные снова надеть на штырь. А потом точно так же достать диск другой "тяжести" со второго штыря. А он как бы случайно тоже в самом низу. И так далее - до полной победы идиотизма над здравым рассудком. Не правда ли, изощренная пытка?

К чему это лирическое бредисловие? А к тому, что стек - это тоже своего рода штырь с блинами. И уж поверьте, упражняться вы с ним будете намного чаще, чем мы делали это на уроках физкультуры. С той лишь несущественной разницей, что у нас на следующий день болела спина, а у вас на следующий день будут болеть мозги.

Так вот, о стеке: "штырь" для блинов находится в оперативной памяти (где же еще?). А роль блинов выполняют хорошо знакомые нам всем регистры, вернее - их "значения".

Правила работы с ним те же - вы можете снять только верхний "блин". Чтобы получить самый нижний "блин" - вам нужно прежде снять все те, которые НАД ним.

Очевидно, что из десяти "блинов", которые вы надели на "штырь", первым будет сниматься последний из надетых (верхний), а последним - первый, то есть самый нижний.

Все очень просто: "первый пришел - последним уйдешь" и наоборот "пришел последним - уйдешь первым".

Это вам не очередь времен социализма... Это очередь "загрузки-разгрузки" стека!

#2. Для работы со стеком вам пока что необходимо знать только две команды: push и pop. Так как в качестве "блинов" у нас регистры, то, соответственно, необходимо после этих команд указывать и "имена собственные" помещаемых в стек значений регистров.

Соответственно:

```
push AX ;ПОМЕЩАЕТ В СТЕК значение регистра AX
pop AX  ;ИЗВЛЕКАЕТ ИЗ СТЕКА значение регистра AX
```

Ну а как делать то же самое с остальными регистрами вы, наверняка, уже и сами догадались.

Очень важно помнить, каким "нездоровым" образом в стеке реализована ОЧЕРЕДЬ -поместить/извлечь. Помните, мы вас предупреждали, что нам нельзя верить на слово? Не верьте! А посему - обязательно убедитесь в истинности/ложности нашего голословного утверждения при помощи следующей программку:

```
:0100 MOV AX,0001 ;AX = 1
:0103 PUSH AX      ;В стек записана 1
:0104 MOV AX,0002
:0107 PUSH AX      ;В стек записана 2
:0108 MOV AX,0003
:010B PUSH AX      ;В стек записана 3
:010C MOV AX,0004
:010F PUSH AX      ;В стек записана 4
:0110 MOV AX,0005
:0113 PUSH AX      ;В стек записана 5
:0114 POP AX
:0115 POP AX
:0116 POP AX
```

```
:0117 POP AX
:0118 POP AX
:0119 INT 20
```

С очередностью заполнения стека, наверное, все понятно :). Я много про абстрактные "блины" загружал. А вот с адреса 114 начинается извлечение из стека. В какой последовательности это делается, вы можете увидеть сами, произведя трассировку этой небольшой проги.

```
-r
AX=0000 BX=0000 CX=001B DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=14DC ES=14DC SS=14DC CS=14DC IP=0100 NV UP EI PL NZ NA PO NC
14DC:0100 B80100      MOV AX,0001
-
```

Анализируем. Прога еще не начала работать, готовится выполниться команда по адресу 100. Делаем ШАГ!

```
-t
AX=0001 BX=0000 CX=001B DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=14DC ES=14DC SS=14DC CS=14DC IP=0103 NV UP EI PL NZ NA PO NC
14DC:0103 50          PUSH AX
-
```

Анализируем. AX=0001 - значит, команда выполнилась правильно :). Следующая команда, по идее, должна поместить 1 в стек.

```
-t
AX=0001 BX=0000 CX=001B DX=0000 SP=FFFC BP=0000 SI=0000 DI=0000
DS=14DC ES=14DC SS=14DC CS=14DC IP=0104 NV UP EI PL NZ NA PO NC
14DC:0104 B80200      MOV AX,0002
-
```

И что? Команда выполнилась, но где мы можем увидеть, что в стек действительно "ушла" единица? Увы, но здесь это не отображается :). Проверим потом. Ведь логично предположить, что если эти значения действительно сохранились в стеке, то мы их потом без проблем оттуда извлечем, т.е. если найдем "там" наши 1, 2, 3, 4, 5 - значит все Ок.

А поэтому - дадим программе работать дальше до адреса 114 (не включительно), не вдаваясь в подробный анализ. Что тут анализировать? Если значение регистра AX последовательно меняется от 1 до 5 - значит, команда mov работает. А стек (команда push) проверим потом, как и договорились.

Проехали до адреса 114.

```
-g 114
AX=0005 BX=0000 CX=001B DX=0000 SP=FFF4 BP=0000 SI=0000 DI=0000
DS=14DC ES=14DC SS=14DC CS=14DC IP=0114 NV UP EI PL NZ NA PO NC
14DC:0114 58          POP AX
-
```

А вот теперь снова анализируем :). При следующем шаге выполнится команда, извлекающая некогда "запомненное" значение AX из стека.

Обратите внимание, регистр IP указывает на адрес (114) выполняемой команды. Мы с вами это уже проходили, не так ли?

Поехали дальше!!

```
-t
AX=0005 BX=0000 CX=001B DX=0000 SP=FFF6 BP=0000 SI=0000 DI=0000
DS=14DC ES=14DC SS=14DC CS=14DC IP=0115 NV UP EI PL NZ NA PO NC
14DC:0115 58          POP AX
-
```

Выполнился первый POP. Готовится выполниться второй. AX=5. Т.е., по сравнению с предыдущим шагом, вроде ничего не изменилось... Но на самом деле это не так. AX=5 - эта пятерка

"загрузилась" из стека :)). В этом вы легко убедитесь, сделав следующий шаг трассировки.

```
-t
AX=0004 BX=0000 CX=001B DX=0000 SP=FFF8 BP=0000 SI=0000 DI=0000
DS=14DC ES=14DC SS=14DC CS=14DC IP=0116 NV UP EI PL NZ NA PO NC
14DC:0116 58          POP AX
-
```

Ууупс... AX=4 :). А команда, вроде, та же :) - POP AX :)

```
-t
AX=0003 BX=0000 CX=001B DX=0000 SP=FFFA BP=0000 SI=0000 DI=0000
DS=14DC ES=14DC SS=14DC CS=14DC IP=0117 NV UP EI PL NZ NA PO NC
14DC:0117 58          POP AX
-
```

AX=3 :)

```
-t
AX=0002 BX=0000 CX=001B DX=0000 SP=FFFC BP=0000 SI=0000 DI=0000
DS=14DC ES=14DC SS=14DC CS=14DC IP=0118 NV UP EI PL NZ NA PO NC
14DC:0118 58          POP AX
-
```

AX=2 :)

```
-t
AX=0001 BX=0000 CX=001B DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=14DC ES=14DC SS=14DC CS=14DC IP=0119 NV UP EI PL NZ NA PO NC
14DC:0119 CD20        INT 20
-
```

AX=1 :) То есть нашлись-таки наши 1, 2, 3, 4, 5 :). Восстановились из стека. Теперь поверили? А то!

Еще раз обращаю ваше внимание на то, что последовательность записи (четыре PUSH'a) была - 1, 2, 3, 4, 5, а вот последовательность извлечения (четыре POP'a) - 5, 4, 3, 2, 1. Т.е. "последний пришел - первый ушел". Зарубите это себе на носу! (Как сделал это на своем перебитом носе наш школьный учитель физкультуры).

Медитируйте над этой темой до полного просветления! Иначе потом придется туго!

1.8. Цикл

#1. Наша программа для работы со стеком линейна. А линейное программирование - это плохо. Хотя и не всегда :)

Итак, давайте еще раз посмотрим на нашу программу для работы со стеком. С 100-го до 113-го адреса у нас имеется пять почти идентичных блоков. Изменяется только значение AX, но на одно и то же число - на единицу в большую сторону. То есть AX = предыдущее значение + 1. Это очевидно.

Еще более очевидно, что простая команда POP AX (с 114 по 119) повторяется у нас тоже 5 раз.

Мне почему-то сразу вспомнился анекдот о том, как два мента едут в машине, и один спрашивает у другого: "Глянь, работает ли у нас мигалка на крыше". Тот высунул в голову в форточку и говорит: "Работает-не работает-работает-не работает-работает-не работает..."

Так вот, не будем уподобляться этим нехорошим людям и сделаем нашу прогу более нормальной.

Добьемся мы этого с помощью так называемого "цикла". Цикл - это... Не буду давать общепринятые определения; кто хочет - поищите в книжках, благо, их навалом.

Скажу только: "сесть-встать, сесть-встать, сесть-встать" - это не цикл, а вот "сесть-встать и так три раза" - уже можно считать циклом.

Реализуется же он (цикл), например, при помощи регистра CX и команды LOOP следующим образом.

Число циклов заносится в регистр CX. После этого следует "простыня" из команд, которые вы хотите "зациклить", т. е. выполнить энное количество раз. Заканчиваться все это должно LOOP'ом с указанием адреса "строки", с которой необходимо начать цикл (обычно это "строка", следующая сразу же после mov CX).

Давайте мы сначала "набьем" нелинейный вариант нашей проги, а потом разберемся, что там к чему. Набиваем:

```
:0100 XOR AX,AX ; AX=0
:0102 MOV CX,0005 ; нижеследующий до команды LOOP кусок повторить CX раз
:0105 ADD AX,0001 ; AX=AX+1 (у нас же значение AX на 1 увеличивается...)
:0108 PUSH AX ; помещаем в стек
:0109 LOOP 0105 ; конец цикла; иницируем повторение; CX уменьшается на 1
:010B MOV CX,0005 ; второй цикл повторить тоже 5 раз
:010E POP AX ; достаем из стека
:010F LOOP 010E ; конец цикла; повторить! ; CX=CX-1
:0111 INT 20 ; выход из нашей "правильной" проги...
```

Наверное, вы уже поняли, что цикл повторяется до тех пор, пока CX не станет равен 0. Несмотря на то что CX - он как бы регистр общего назначения, для "зацикливания" используется именно он :). С остальными такой фокус не проходит. Это и есть так называемая "специализация регистров", о которой мы уже вскользь упоминали.

Протрассируйте эту программу! Искренне надеюсь, что вы поняли, чем это я вас тут загружал.

Медитируйте!

#2. А теперь вопрос на засыпку ;). Сколько раз выполнится следующий цикл:

```
:0102 MOV CX,0000
:0105 ADD AX,0001
:0108 LOOP 0105
```

Очевидный ответ - 0 раз. В CX же у нас занесен 0. Так вот - ответ неправильный.

Менее очевидный ответ - 1 раз! Ведь перед LOOP'ом сложение один раз все-таки выполнится. Так вот, этот ответ тоже неправильный.

Самые подозрительные могут сразу же посмотреть на этот цикл под отладчиком, и с удивлением обнаружат, что LOOP сначала уменьшает значение CX (0-1=FFFF), а потом уже проверяет, не равен ли он нулю. И с гордостью за задний ум своей головы воскликнут: FFFFh раз!

Так вот: этот ответ близок к истине, но тоже неправильный ;)

Правильный ответ - цикл выполнится 10000h (65536d) раз.

Но только вы и мне не верьте! Истинно только то утверждение, которое вы сами проверили на практике. Медитируйте!

1.9. Немножко оптимизации

Как мы уже говорили, линейное программирование - это плохо, но не всегда. Сравните размеры ваших линейной и нелинейной программ. Не знаю, как у вас, но у нас линейная "весит" 27, а нелинейная - 19 байтов. Как по-вашему, какая быстрее работать будет?

- Ну, естественно, нелинейная, потому что она меньше! - скажете вы и будете неправы.

Попытайтесь оттрассировать "зацикленную". Не правда ли, она трассируется намного дольше своего линейного аналога?

Угу, всё поняли? Сам знаю, что ни черта. :(

Объясняю: в "зацикленной" программе "компьютеру" приходится выполнять БОЛЬШЕ команд, нежели в "незацикленной".

Аргументирую это голословное утверждение следующей таблицей (построенной на основе трассировки):

Что делает линейная	Что делает нелинейная
$AX=1$	$AX=0$
Помещаем в стек 1	$CX=5$
$AX=2$	$AX=AX+1=1$
Помещаем в стек 2	Помещаем в стек 1
$AX=3$	Конец цикла - переход
Помещаем в стек 3	$AX=AX+1=2$
$AX=4$	Помещаем в стек 2
Помещаем в стек 4	Конец цикла - переход
$AX=5$	$AX=AX+1=3$
Помещаем в стек 5	Помещаем в стек 3
Достаем из стека 5	Конец цикла - переход
Достаем из стека 4	$AX=AX+1=4$
Достаем из стека 3	Помещаем в стек 4
Достаем из стека 2	$AX=AX+1=5$
Достаем из стека 1	Помещаем в стек 5
Выход	$CX=5$
	Достаем из стека 5
	Конец цикла - переход
	Достаем из стека 4
	Конец цикла - переход
	Достаем из стека 3
	Конец цикла - переход
	Достаем из стека 1
	Выход

Ну и как по-вашему, какую из двух простыней процессор быстрее обработает? Сказать вам по секрету? А вот ничего я вам не скажу! Сами думайте! :]

Как сейчас помню, был в моем Турбо-Си в преференсах к компилятору такой радиобуттон: "оптимизировать" по размеру или по скорости выполнения. Угадайте, на чем основан принцип этой оптимизации?

Только не вздумайте писать линейные проги! Пишите "нелинейные"! Нелинейную в линейную "переоптимизировать" - как два пальца намочить! А вот наоборот - :(

Резюме - бесплатный сыр бывает только в мышеловке, и за все надо платить. Компактность и скорость - обычно параметры конфликтующие, поэтому в каждом конкретном случае нужно выбирать, что предпочтительнее. Исследования, проведенные в свое время еще Кнудом, показали, что 80% времени затрачивается на выполнение 20% программы. Соответственно, рекомендуется тратить время на оптимизацию скорости именно тех 20% программы, а остальные можно оптимизировать по размеру (в частности, за счет циклов). Так и получается баланс между компактностью и скоростью программы ;).

1.10. Разборка с процедурами

#1. В разделе 1.7. #4 мы сделали глупую линейную программку, выводящую окошки. Обещали, что в следующей главе сделаем ее менее "тупой", да отвлеклись почему-то на циклы и стек. То есть я-то знаю, ПОЧЕМУ, но вот вам об этом - не скажу! Догадайтесь сами. Итак, поехали...

Шаг первый. Внимательно посмотрев на "линейную" прогу из 1.7. #4 и прочитав "условие задачи" из главы 1.7. #1, вы обязаны возмутиться - зачем мы использовали команду MOV, если и ежу понятно, что отличия следующего окошка от предыдущего можно выразить более лаконично: BH=BH+10, CH=CH+1, CL=CL+1, DH=DH-1, DL=DL-1? И не нужно напрягать мозги, подсчитывая новое значение регистра вручную.

Если вы так подумали, то оказались совершенно правы! Программу из #4 запросто можно было представить в таком вот виде:

```
:0100 XOR AL,AL ;окошко первое
:0102 MOV BH,10
:0104 MOV CH,05
:0106 MOV CL,10
:0108 MOV DH,10
:010A MOV DL,3E
:010C MOV AH,06
:010E INT 10
:0110 ADD BH,10 ;окошко второе
:0113 ADD CH,01
:0116 ADD CL,01
:0119 SUB DH,01
:011C SUB DL,01
:011F INT 10
:0121 ADD BH,10 ;окошко третье
:0124 ADD CH,01
:0127 ADD CL,01
:012A SUB DH,01
:012D SUB DL,01
:0130 INT 10
:0132 ADD BH,10 ;окошко четвертое
:0135 ADD CH,01
:0138 ADD CL,01
:013B SUB DH,01
:013E SUB DL,01
:0141 INT 10
:0143 ADD BH,10 ;окошко пятое
:0146 ADD CH,01
:0149 ADD CL,01
:014C SUB DH,01
:014F SUB DL,01
:0152 INT 10
:0154 INT 20 ;конец программы
```

И несмотря на то, что размер ее оказался несколько большим, она тоже будет работать правильно :).

Но тут любой более или менее наблюдательный программмер возмутится повторно - да что это за программа такая? В ней целых четыре раза повторяется один и тот же кусок:

```
:0143 ADD BH,10
:0146 ADD CH,01
:0149 ADD CL,01
:014C SUB DH,01
:014F SUB DL,01
:0152 INT 10
```

И знаете что? Этот наблюдательный программмер будет прав! А если он еще и нехорошо выразится по поводу такого "неправильного" стиля программирования - будет прав... или почти прав...

Есть такой процесс - оптимизация, одной из особенностей которой является уменьшение параметризации (параметризация - вставка процедур в место вызова или фиксация значения

отдельных параметров) и развертка циклов. То, что обычно такими вещами должен заниматься компилятор, суть меняет не сильно - тем более, что ассемблер оптимизацией сам не занимается :). Но до ассемблера мы с вами еще не добрались, поэтому говорить об этом пока еще рано.

Внимательно всмотритесь в полный текст программы и в этот выделенный кусок. И помедитируйте над ним до полного просветления текущей "обстановки"...

#2. Итак, у нас есть ПОВТОРЯЮЩАЯСЯ ЧАСТЬ программы. А еще у нас есть пальцы, которым, как правило, лень набивать длинные "простыни" программного кода. Это одна из многочисленных причин, по которым и придумали такого "зверя" как ПОДПРОГРАММУ (она же - ПРОЦЕДУРА, она же - ФУНКЦИЯ). Остальные причины мы рассмотрим попозже, а вот на счет "лени" поговорим прямо сейчас:

Если мы возьмем наш "часто повторяющийся" кусок программы и допишем в конец команду RET, то получится у нас именно ПРОЦЕДУРА - во всей своей красе:

```
:011E ADD BH,10 ; "точка входа"; она же - начало "тела".
:0121 ADD CH,01
:0124 ADD CL,01
:0127 SUB DH,01
:012A SUB DL,01
:012D INT 10 ; конец "тела"
:012F RET
```

Красота ее вот в чем заключается - процедуру можно "вызвать" командой CALL :)))

Все более чем просто. Когда в программе встречается CALL с указанием АДРЕСА-НАЧАЛА-ПРОЦЕДУРЫ (в нашем случае это 011E), то компьютер "идет" по этому адресу и выполняет все команды, расположенные между "точкой входа" (включительно) и командой RET, то есть так называемое "тело" процедуры.

RET - это тоже команда, но к "телу" (адреса 11E... 12D) она не относится. Она является "ОРГАНИЗАТОРОМ" этого "тела". Процессор, встретив команду RET, возвращает управление обратно после последнего CALL (т.е. "перепрыгивает" на строчку ниже "вызвавшего" данную процедуру CALL'a)...

Короче, CALL XXXX означает - "выполнить процедуру, начинающуюся по адресу XXXX". А RET означает - "конец процедуры" и, соответственно, переход на строчку ниже вызвавшего его CALL'a.

Если же говорить более формально и строго, то процедура - это средство обобщения, когда некоторая общая последовательность действий получает "имя", и потом при необходимости ПОВТОРНОГО ИСПОЛЬЗОВАНИЯ данного кода к нему просто идет обращение по "имени" (напомним, что в отличие от языков высокого уровня и даже ассемблера, в машинном коде от имен остаются одни только адреса, а язык, принимаемый DEBUG, является промежуточным между машинным кодом и ассемблером). Более того, следующим логическим шагом после обобщения является параметризация, когда некоторые части общего кода зависят от передаваемой извне информации (параметров), с чем очень хорошо знакомы программисты на языках высокого уровня. Но о параметризации и ее применении в ассемблере мы поговорим в другой раз.

Не ругайтесь. Мы знаем, что вы ни черта не поняли. А по сему набьем в debug'e эту прогу и посмотрим, что она делает.

Те, кто читал внимательно, могут отметить, что инструкция CALL по своему действию очень похожа на инструкцию генерации прерывания INT, с той лишь разницей, что аргументом CALL является адрес процедуры, а не индекс в таблице "векторов прерываний", где и хранится адрес обработчика прерывания (той же процедуры). А для особо продвинутых отметим, что в ранних моделях процессоров от Intel при подаче запроса на обработку от внешнего устройства контроллер прерываний, помимо собственно сигнала прерывания, посылал в процессор инструкцию CALL.

#3. Кстати, вы уже поняли, почему мы называем debug "до боли любимой программой"? Нет? Неужели вы еще не полюбили это произведение программмерского гения всеми фибрами своей души? Еще нет? М-да... мы в вас разочаровались.

А по сему: - НАБИВАЕМ! - злобно кричим, брызгая слюной на эргономичный коврик:

```
:0100 XOR AL,AL ;первое окошко рисуем, как и раньше...
:0102 MOV BH,10
:0104 MOV CH,05
:0106 MOV CL,10
:0108 MOV DH,10
:010A MOV DL,3E
:010C MOV AH,06
:010E INT 10
:0110 CALL 011E ;четыре раза вызываем подпрограмму,
:0113 CALL 011E ;начинающуюся по адресу 011E
:0116 CALL 011E
:0119 CALL 011E
:011C INT 20 ;выход из программы...
:011E ADD BH,10 ;начало процедуры
:0121 ADD CH,01
:0124 ADD CL,01
:0127 SUB DH,01
:012A SUB DL,01
:012D INT 10
:012F RET ;конец процедуры
```

Не правда ли, красиво получилось?

Первое, что вас может смутить - это то, что команда выхода (INT 20) расположена не там, где вы привыкли, то есть не в конце программы.

Ну что я вам могу на это ответить? Концы - они-то разные бывают! Последняя строчка в листинге вовсе не означает, что последней будет выполняться именно она. И это не должно вас смущать! А если все же смущает - смотрим, как работает эта прога из-под отладчика.

Итак, до адреса 0110 вам все должно быть понятно, мы это рассматривали. Трассируем дальше...

- *Что значит "трассируем"? - попросите вы напомнить.*

Мысленно мы ругаем вас нехорошими словами (ну сколько раз повторять-то можно!), а вслух скажем: Команда "T" и Enter. Команда "T" и Enter. Команда "T" и Enter...

Команда CALL 011E по адресу 0110 говорит процессору: "Дальше мы не пойдем, пока не выполним простыню, начинающуюся по адресу 011E". И далее, естественно, следует переход на этот адрес.

Входим в тело процедуры, начиная с 011E, и выполняем команды до 012D включительно...

А теперь внимательно смотрим, на какой адрес нас "перекинет" команда RET.

На 113-й? И это правильно! По 113-му адресу у нас какая команда? Да вот опять CALL 011E!

Опять процедура с адреса 011E, опять RET[URN] на строку ниже, то есть на 116...

И так далее, до того момента, пока следующей строчкой не окажется INT 20 - собственно, на этом и программе конец.

Ну оно и ежу понятно, что, несмотря на то что INT 20 - не в конце программы, последним выполнится именно он.

Короче, куда бы вас не посылали всяческие "столбы с указателями", конец вашего пути только один... А плутать вокруг да около этого конца вы можете сколько вам заблагорассудится...

Кстати, именно это и является одной из многочисленных тайн программирования.

Кто после этого скажет, что программисты - недЗенствующие люди?

#4. Те, кто внимательно ознакомились с циклами, они и на этом не остановятся! Посмотрев на адреса 110...119, они вообще возьмут и возомнят себя воистину крутыми парнями! Знаете, что они напишут? А вот что (предвидим!):

```
:0100 XOR AL,AL
:0102 MOV BH,10
:0104 MOV CH,05
:0106 MOV CL,10
:0108 MOV DH,10
:010A MOV DL,3E
:010C MOV AH,06
:010E INT 10
:0110 MOV CX,0004
:0113 CALL 011A
:0116 LOOP 0113
:0118 INT 20
:011A ADD BH,10
:011D ADD CH,01
:0120 ADD CL,01
:0123 SUB DH,01
:0126 SUB DL,01
:0129 INT 10
:012B RET
```

То бишь еще и CALL в цикл при помощи MOV CX,4 и LOOP'a "закрутят". И что? А попробуйте!

Что, не "пашет"? А что надо делать, если "не пашет, а должно бы"? Правильно! Смотреть из-под отладчика!

Смотрим? Если посмотрите, то сразу же и "загвоздку" увидите - CX, использованный в качестве "счетчика" циклов, "перебивает" тот же CX, но используемый как "координаты верхнего левого угла окна". И что с этим делать прикажете?

Вот вы и столкнулись с одной из самых больших проблем. В процессорах фирмы Intel есть только 4 регистра общего назначения (и то в большинстве случаев - специализированных). Помните, мы вам говорили об этом?

А теперь попробуйте выкрутиться из этой нехорошей ситуации с использованием стека :). Кстати, весьма "мозгопрочищающая" задачка :).

1.11. Переходы

#1. "Переходы" бывают разные. Если вы пришли в гости, а вас просто послали к черту - такой переход называется "безусловный". А нежели вам сказали: "Если без пива - то иди к черту, а если с пивом - тогда проходи", - то это уже "условный" переход.

Соответственно, для успешного перехода необходимо указать: ПРИ КАКОМ УСЛОВИИ выполнить переход, КУДА ПЕРЕЙТИ, ну и, наконец, сам пинок под зад нужно СДЕЛАТЬ, чтобы переход "гостя" в заданном направлении все-таки "состоялся".

Безусловный переход у нас "делает" мнемоническая команда JMP, после которой следует указать адрес, на который "компьютер" должен пойти "на" ;). В данном случае УСЛОВИЕМ у нас будет "при любых обстоятельствах": хоть пустой, хоть с пивом, хоть с ... все равно. Когда рисовали окошки, вы уже использовали эту команду для создания "спецдефекта". Если кто еще не понял, что делает эта команда - к нему (см. 1.7 #4) и отсылаю. Сделайте "спецдефект" и посмотрите на него под отладчиком. Когда до вас дойдет, почему мы там не предусмотрели выхода (INT 20h) - можете переходить к п.2 текущей главы.

#2. Условный переход у нас организуется в два шага. На первом шаге мы вычисляем условие ("принес ли пиво?"), на втором шаге "посылаем" или не "посылаем" - в зависимости от результатов вычислений. Можно привести такую аналогию - на первом шаге два груза кладутся на весы, сравнивающие их массу. Соответственно, возможны только три их положения: наклон влево (груз в

левой чашке тяжелее), наклон вправо (груз в правой чашке тяжелее) и равновесие. На втором шаге мы предпринимаем действия в зависимости от положения весов.

Например, на первом шаге можно использовать как "аптекарские весы" инструкцию CMP, которой обязательно нужно указать, ЧТО и С ЧЕМ она будет сравнивать.

Пишем, например,

```
CMP AX, BX
```

В зависимости от значений регистров у нас возможны следующие состояния: "наклон влево" ($AX < BX$), "наклон вправо" ($AX > BX$) и "равновесие" ($AX = BX$). Таким образом ВЫЧИСЛЕНИЕ УСЛОВИЯ у нас уже организовано :). Только условие не бинарное, а есть еще и "серединный вариант" (и даже несколько других!). Это нормально. Это для того сделано, чтобы мы могли выражения типа "больше-или-равно", "меньше-или-равно" да и просто "равно" в своих программах использовать...

Итак, УСЛОВИЕ есть. Теперь решаем, что нам делать при том или ином условии. Вот далеко не полный список возможных "прыг-скоков":

- JE - переход если равно;
- JNE - переход если не равно;
- JA - переход, если больше;
- JAE - переход, если больше или равно;
- JB - переход, если меньше;
- JBE - переход, если меньше или равно...
- и т.д.

Естественно, что после мнемоники ("прыгнуть, если") должен стоять АДРЕС, куда нужно "прыгнуть", если условие соблюдено. Если же условие не соблюдено, то прыжок не происходит, и выполняется нижеследующая строка программы.

Задание на медитирование - зрительно представьте себе "весы правосудия". И побросайте на их чашки разную шестнадцатеричную дрянь в различных "пропорциях" и "комбинациях". Просветлиться вы должны следующим образом - в какую бы сторону эти ваши "весы" ни склонялись, вы все равно заставите систему работать так, как ЭТО вам угодно! "Весы" - они только констатируют факт. А вот "приговор" выносят судьи. Хе... и пусть после этого только кто-нибудь скажет, что программерам чужда политика - дело, как известно, весьма грязное.

А о чем это мы? Ах да, переходы...

#3. Продолжим программировать, что ли? Напишем что-нибудь красивое и неизменно тупое? С использованием условных и безусловных переходов?

Поехали! Слабаем мы сейчас что-то наподобие графического редактора :)). Не верите?

У-у-у... Сложная задачка! Если въедете, что да как - значит, молодцы! Значит, разобрались-таки с джебагом! Значит, подключились-таки к программерскому эгрегору и более или менее привели в порядок свои мозги :)... А это сложная штука, мы вам скажем - мозги в порядок приводить! Особенно когда есть куча инструментов, которые "порядок в коде" сами как бы наводят :).

Думаете, вы по нашим текстам программировать учитесь? По-настоящему программировать мы еще не начали! Все, чем мы пока занимаемся - это приводим в порядок свои мозги и тренируемся на кнопки клавиатуры нажимать :)). А вот ско-о-оро НАЧНЕМ... тогда "прощай, здоровье" будет настоящим!

Итак, сначала рассмотрим прерывания, которые в нашем "графическом редакторе" будут использоваться. Их три штуки, и все - BIOS'овские:

```
mov AH, 00 ; функция 0 прерывания 10h устанавливает "режим видео"
```

```

mov AL,04 ;"на входе" (в регистре AL) - номер режима .
int 10h ;если AL=4, то устанавливается цветной 320x200 графический режим
mov CX,64h ; CX и DX - координаты точки
mov DX,64h
mov AH,0Ch; функция 0Ch (в регистре AH!) прерывания 10h рисует точку.
mov AL,1Bh ;в AL - "код цвета" этой точки.
int 10h

```

Опять-таки - подробности о координатах и "кодах" цвета ищите сами!! Благо, знаете, где искать.

```

mov AH,00 ;функция 0 прерывания 16h
int 16h ;"читать код нажатой клавиши"

```

Эта функция считывает код сканирования и код символа (клавиши на клавиатуре и соответствующий ей ASCII-код) из буфера клавиатуры (есть такой). Если в буфере ничего нет - она ждет, пока там что-нибудь появится. То есть ЖДЕТ, чтобы вы нажали на какую-нибудь клавишу, код которой будет занесен в регистр AX. Причем в AL - "символ", а вот в AH - так называемый "код сканирования"...

Кодами вы пока голову не забивайте. Достаточно знать, что после нажатия клавиши Up в AH "попадет" значение 48h, Down - 50h, Left - 4Bh, Right - 4Dh.

Как работает последний кусок кода, обязательно проверьте под отладчиком, это полезно :).

#4. И лезем, лезем в наш горячо любимый DZEBUG, дабы набить там драгоценные строчки машинного mnemonicного никому-кроме-вас-непонятного кода!

```

-a
:0100 MOV AH,00 ;устанавливаем графический режим
:0102 MOV AL,04
:0104 INT 10
:0106 MOV CX,0064 ;координаты Первой Точки
:0109 MOV DX,0064
:010C MOV AH,0C ;рисуем точку!
:010E MOV AL,1B
:0110 INT 10
:0112 MOV AH,00 ;ждем нажатия на клавишу
:0114 INT 16
:0116 CMP AH,4B ;а не нажат ли у нас Left?
:0119 JE 012A ;если да - то "прыг"!
;если нет - то следующая строчка
:011B CMP AH,4D ;а не нажат ли у нас Right?
:011E JE 012D
:0120 CMP AH,48 ;а не нажат ли у нас Up?
:0123 JE 0130
:0125 CMP AH,50 ;а не нажат ли у нас Down?
:0128 JE 0133
:012A DEC CX ;задаем новые координаты, в зависимости
:012B JMP 010C ;от нажатой клавиши - и скок в начало!
:012D INC CX
:012E JMP 010C
:0130 DEC DX
:0131 JMP 010C
:0133 INC DX
:0134 JMP 010C

```

Тут один из автору вставили шпильку:

"Не хватает проверки и выхода (со сбросом видеорежима!) по Esc - такие действия должны быть обязательным атрибутом, а не домашним заданием."

Совершенно верная шпилька, товарищи! Но все равно - пусть это будет домашним заданием.

Если вы все ввели правильно - должно заработать! Полубуйтесь плодами своей медитации... Красиво?

#5. А сейчас мы это все дело прокомментируем:

- адреса 100...200 - устанавливаем графический режим, указываем функцию (100), указываем номер режима (102) и вызываем прерывание (104);
- адреса 106...109 - инициализируем координаты первой точки. Координаты последующих точек будут определяться "динамически" - в зависимости от нажатой клавиши;
- адреса 10C...110 - рисуем точку. Первый раз - в координатах, инициализированных командами по адресам 106 и 109. Все последующие разы - по координатам, "инкрементированным" или "декрементированным" (во словеса!) по адресам: 12A, 12D, 130 и 133;
- адреса 112...114 - ждем нажатия на клавишу;
- адреса 116...128 - "щемим" нужные нам клавиши. "Взвешиваем". На каждую из курсорных клавиш по адресам 12A...134 приготовлены "обработчики". Если найдена "нужная клавиша", то делаем прыг на "обработчик" этой клавиши;
- адреса 12A...134 - в этом блоке определяется, что делать с координатами следующей точки. После чего - прыжок на "рисуем точку" :).

Правда, здорово получилось?

1.12. Данные

#1. Работать с кодом мы с вами научились. Сейчас поучимся заставить наш код обрабатывать данные...

Итак, запускаем DZEBUG и вводим следующую команду:

```
-e cs:115
```

Которая означает: "набиваем память всяким дерьмом начиная со смещения 115".

В ответ он вам выплюнет:

```
17B3:0115 00.
```

Что означает: байт по смещению 115 равно 00. И точка. Но это не простая точка - это приглашение ввести НОВОЕ ЗНАЧЕНИЕ этого байта. Когда вы его ввели, нужно нажать на пробел.

Если вы вознамеритесь последовательно ввести 1,2,3,4,5, то это будет выглядеть приблизительно так:

```
17B3:0115 00.1 75.2 AD.3
17B3:0118 66.4 FF.5 [Enter]
```

А теперь делаем дампы памяти и смотрим, что за дрянь у нас получилась...

А ведь получилось же!!

#2. Мы запросто умеем "присваивать" регистру любое значение (mov AL,1C какой-нить), запросто можем "копировать" содержимое одного регистра в другой (mov AL,BL например)... А сейчас мы с вами научимся при помощи той же команды MOV еще и с данными из памяти работать.

Все проще пареной репы... Если мы напишем

```
MOV AL,[115]
```

то в результате выполнения этой команды в регистр AL "внесутся" две шестнадцатеричные циферки (байт), которые по адресу 115 находятся. То есть в нашем случае AL станет равным 1.

А теперь посмотрите, что делает "обратная" команда:

```
mov AL,55
mov [115],AL
```

В первой строчке мы присвоили AL значение 55, а второй строчкой "скопировали" значения регистра в байт по адресу 115. Правда, проще некуда?

Обязательно посмотрите на этот процесс под отладчиком!

#3. А еще вот какой изврат с этим можно делать:

```
mov BX,115
mov AL,[BX]
```

Сие присваивает регистру AL значение байта по адресу 115 :). Ну... через посредника "BX" присваивает! Который у нас "переменная", как известно :).

```
mov AL,1C
mov BX,115
mov [BX],AL
```

А этот кусок кода у нас "записал" 1C в сегмент данных по адресу 115 :). Ну, и извращения наподобие:

```
mov AL,[BX+1]
```

и

```
mov [BX+1],AL
```

Тоже весьма и весьма полезны в программном деле :).

Короче: все, что в квадратных скобках, - это адрес в памяти, с которым вы собираетесь "работать". Другой вопрос, что этот адрес может быть "составным"...

#4. Низкоуровневый Paint мы с вами уже писали. Сегодня напишем низкоуровневый дЗенский EXCEL.

Задание простое... Есть у нас табличка типа:

```
1 8 ?
2 9 ?
3 1 ?
4 2 ?
5 2 ?
```

в которой данные в формате HEX. И все, что нам нужно с ними сделать - это просуммировать каждую "строчку", а сумму занести в третий "столбец"... В EXCEL'е это делается элементарно... А на машинном уровне, в общем-то, не намного сложнее!!

Для начала мы наберем "исходные данные" и зарезервируем место (например, забьем нулями) под третий столбец, в который собираемся помещать результат...

Набиваем блок данных, начиная с адреса, например, 115:

```
-e ds:115
17EA:0115 01.1 08.8 02.0
17EA:0118 09.2 02.9 00.0 03.3 03.1 00.0 04.4 04.2
17EA:0120 00.0 05.5 05.2 00.0
```

Вот так это у меня в DZEBUG'е выглядело :). Только я еще дампы посмотрел, правильно ли я ввел:

```
17EA:0110 03 E2 F3 CD 20 01 08 00-02 09 00 03 01 00 04 02 ....
17EA:0120 00 05 02 00 6A 87 04 FF-76 FE 57 57 9A 5C 6C 87 ....j...v.WW.\1.
```

Вроде правильно :)). Ну а программу я вот какую придумал:

```
17EA:0100 BB1501      MOV     BX,0115
17EA:0103 B90500      MOV     CX,0005
17EA:0106 8A07        MOV     AL,[BX]
17EA:0108 024701      ADD     AL,[BX+01]
17EA:010B 884702      MOV     [BX+02],AL
17EA:010E 83C303      ADD     BX,3
17EA:0111 E2F3        LOOP    0106
17EA:0113 CD20        INT     20
```

В BX я занес адрес начала блока данных (он же - верхний левый угол нашей таблицы). В CX внес 5, чтобы столько раз цикл выполнялся (LOOP по адресу 111). А тело цикла вообще простое:

Команда по адресу 106 забирает в AL цифирь из первого столбца.

108 - суммирует "цифирь из первого столбца с цифирью из второго столбца" (сумма, само собой, в AL'e остается).

10B - записывает сумму в третий столбец :).

Ну и ADD BX,3 для перехода на следующую строчку :).

И все на этом...

Сделайте трассировку (внутри INT 20 залезать не надо) и посмотрите на дампы нашего блока данных :)

Я и говорю: ПРОЩЕ ПАРЕНОЙ РЕПЫ!! ;)

#5. Видите? В качестве переменных "в компьютере" можно использовать не только регистры, но и "куски" памяти! А уж там вы можете клепать свои переменные в почти неограниченном количестве! Единственное, что нужно иметь ввиду: с переменными-регистрами компьютер работает намного быстрее, чем с переменными-в-памяти :).

Кстати, если вы хотите сохранить плод своих сегодняшних трудов на веник, то имейте ввиду, что вы и сегмент данных тоже должны сохранить! То есть: вам нужно сохранить весь "диапазон" от адреса 100 до 123 включительно :).

Ну и, само собой, при попытке дизассемблирования с адреса 115 у вас абракадабра пойдет... мы об этом уже говорили и упоминали один из принципов фон Неймана.

Диагноз

Полагаю, вы уже поняли, что значит "выучить язык ассемблера" :) и теперь с удовольствием кинете грязью в того, кто скажет вам, что это сложно ;)

Что значит "выучить язык", и что значит "программировать"? А проводите сами границы между этими понятиями! Только имейте в виду, кто скажет "выучить - значит все команды запомнить - тот дурак :((.

Слова и понятия извращать можно по-всякому. Переопределите собственный тип и носите свежееобмоченные пеленки в полной уверенности, что это носки... В моем понимании "знать ассемблер" и "изучить на ассемблере" - синонимы (хотя лингвисты могут придраться, но мне это пофиг). Согласно границам, которые провел для себя (гы... ну и для вас немножко) автор курса, ассемблер вы уже знаете, а вот программировать на нем пока еще не умеете...

Да о чем это я, в общем-то? (Утомлен кофием, поэтому речь несвязна)? Просто хотел сообщить вам, что первая часть курса закончилась. Вооружившись справочником команд и прерываний, вы уже можете программировать под дос. Если вы внимательно штудировали предыдущие главы, то идеология этого дела (под дос) вам уже должна быть понятна как $2 \times 2 = 100b$.

DZebug: руководство юZверя

Автор: Nyron / HI-TECH

Дата: Не указана

Прога эта неимоверно крута и сильна! Она позволяет писать программы и вмешиваться в ход выполнения программ на самом низком, можно сказать "на самом дЗенском" уровне! С помощью нее можно отображать и изменять значения регистров :), запускать и останавливать выполнение программы в любой момент :), вносить изменения в программу :), работать с винтом на физическом уровне :); работать с машинным кодом, ассемблировать и дизассемблировать его с той легкостью, которая присуща только продуктам корпорации Micro\$oft...

- Бредисловие
- Запуск dZebug'a
- Отображение и изменение значений регистров
- Дамп памяти
- Поиск байтов
- Сравнение участков памяти
- ДиЗассемблирование
- Размещение данных в памяти
- Размещение данных одинакового значения
- Перемещение данных
- Ассемблирование
- Имя, сестра, имя
- Грузить
- Писать (ударение на "а")
- Чтение данных из порта
- Запись данных в порт
- GO! GO! GO! Але-але-але...
- "Трррассиррровка" - сказал пулеметчик
- Плюс-минус
- Нирвана!

Бредисловие

... В 945-ом году отправился князь Игорь к программистам за данью. Когда программисты узнали размеры дани, их лица сразу стали озабоченными, и они побили Игоря и его дружину. Тогда жена Игоря Ольга с огнем и мечом пошла на программистов. Отдавайте, говорит, законную дань, а не желаете, так поставьте на каждую тачку нашу новую навороченную ОСь. Обрадовались программисты, что могут отделаться малым, и их лица опять стали веселыми. А Ольга приказала в каждую ОСь зашить BUG. Программисты инсталлировали ОСь, и BUG уничтожил все их данные... И дело даже не в том, что жалко программистов, а в том, что история учит, какими б не казались крутыми ОСы, нужно уметь работать независимо от программного обеспечения. Ведь сила не в мегагерцах и не гигабайтах, и даже не в DЗеньгах, сила - она в ньютонах..._

Запуск dzebug'a

Я знаю только два способа запуска DEBUG с файлом.

1. С командной строкой:

```
debug имяфайла.тип [ENTER]
```

2. Без командной строки:

```
debug [ENTER]
```

Тут вы получите приглашение DZEBUG в виде черточки "-".
Далее следуют команды:

```
-n имяфайла.тип [ENTER]
-l [ENTER]
```

Ууупс! DZEBUG загрузил вашу программу и готов к работе :)))

Отображение и изменение значений регистров

Первым делом мы посмотрим содержимое регистров, используя команду R. В качестве объекта извращения (Serrgio сегодня будет извращаться несколько иначе) - ваша же прога из #7 п. 4...

Если вы ввели R без параметров, значения регистров будут выведены примерно так:

```
AX=0000 BX=0000 CX=0043 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=16BB ES=16BB SS=16BB CS=16BB IP=0100 NV UP DI PL NZ NA PO NC
15A3:0100 30C0 XOR AL,AL
```

CX содержит длину файла (0043h или 67d). Если размер файла превышает 64K, то BX будет содержать старшую часть размера файла. Это очень важно знать при использовании команды Write - размер файла содержится именно в этих регистрах.

Запомните: когда файл находится в памяти, DZebug не знает его размер. При записи данные о размере берутся из регистров CX и BX. Страшно даже подумать, что может произойти, если мы по ошибке введем команду Write, а в это время BX и CX будут содержать FFFF! Страшно, но приятно... Если мы хотим изменить значение одного из регистров, мы вводим R и имя регистра. Давайте поместим в AX слово *UCK (те, кто не знаком с английским, пусть спросят у своих умных друзей, что означает это слово).

```
-R AX
```

Вывалится:

```
AX 0000
:
```

":" - это приглашение ввести новое значение. Мы отвечаем *UCK

```
:*UCK
```

Вывалится:

```
^Error
```

DZebug выдал сообщение об ошибке.

Тут дело даже не в том, что DЗебугу не понравилось слово, которое мы ввели. Ему, в сущности, глубоко наплевать, какое значение мы хотим занести в регистр, главное - чтобы цифры были HEX'овые. И если F является таковой, то U - нет. DЗебуг заботливо указал нам на нее значком " ^ ". Ну и фиг с ним. Попробуем ввести что-нибудь более пристойное, например D3E0:

```
-R AX
AX 0000
:D3E0
```

Теперь, если мы посмотрим регистры, мы увидим следующее:

```
AX=D3E0 BX=0000 CX=0043 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=16BB ES=16BB SS=16BB CS=16BB IP=0100 NV UP DI PL NZ NA PO NC
15A3:0100 30C0 XOR AL,AL
```

Вы увидите, что ничего не изменилось кроме регистра AX. Ему присвоили новое значение, как мы помним.

Еще один важный момент: команда Register может использоваться только для 16-битных регистров (AX, BX и т. д.). Она не может изменять значения 8-битных регистров (AH, AL, BH и т. д.). Например, чтобы изменить AH, вы должны ввести новое значение в регистр AX с новым AH и старым

значением AL.

Помедитируйте немного и поехали дальше...

Дамп памяти

Если вы не очень хорошо умеете читать машинные команды процессора, команду Dump можно использовать для вывода на экран данных (тексты, флаги и т. д.). Для вывода кода лучше использовать команду Unassemble.

Если мы теперь введем команду Dump, DZEBUG определит начало программы. Для этого он использует регистр DS, и, так как это COM-файл, начинает вывод с адреса DS:0100. Он выведет 80h (128d) байт данных (или то количество, которое вы сами определите)... Следующее употребление команды Dump отобразит следующие 80h байт и так далее.

Например, первая команда Dump выведет 80h байт начиная с адреса DS:0100, вторая команда выведет 80h байт начиная с адреса DS:0180...

Конечно, можно самому определять нужный вам сегмент и смещение при использовании Dump, только нужно использовать для определения только шестнадцатеричные цифры.

Например, запись D DS:BX не верна. Загрузив нашу программу и введя команду Dump, мы увидим очень непонятные буквы и цифры.

Вы наверное уже знаете, что эти буквы и цифры - не что иное как наша программа. Невероятно, но это так.

Вы видите, что выводимые командой Dump данные разделены на три части.

Самая левая содержит адрес первого байта в строке. Ее формат - сегмент:смещение.

Следующая колонка содержит шестнадцатеричные данные по указанному адресу. Каждая строка содержит 16 байт данных.

Третья колонка - это ASCII представление данных. Отображаются только стандартные символы ASCII. Специальные символы IBMPC не отображаются, вместо них выводятся точки ".". Это делает поиск простого текста более удобным.

Dump не может выводить данные, выходящие за границы сегмента. Например, команда

```
-D 0100 L F000
```

правильная (выводятся байты начиная с DS:0100 до DS:F0FF), а команда

```
-D 9000 L 8000
```

некорректна (8000h + 9000h = 11000h - выходит за границу сегмента).

Так как 64К - это 10000h то невозможно задать этот адрес четырьмя шестнадцатеричными цифрами, поэтому DEBUG использует 0000 для задания 10000h.

Чтобы вывести на экран весь сегмент - введите

```
-D 0 L 0.
```

Помедитируйте немного и поехали дальше...

Поиск байтов

Search служит для поиска заданного байта или последовательности байт в пределах сегмента. Параметры задания адреса точно такие, как для команды Dump, поэтому мы не будем здесь опять о них рассказывать.

Еще мы должны задать данные, которые нужно искать. Они могут быть введены как в шестнадцатеричном, так и в символьном формате. Шестнадцатеричные данные вводятся как байты, через пробел или запятую. Символьные данные должны быть заключены в одинарные или двойные кавычки.

Шестнадцатеричные и символьные данные могут чередоваться в любом порядке, например команда

```
-S 0 L 100 12 34 'abc' 56 [ENTER]
```

- правильная, в результате произойдет поиск от DS:0000 до DS:00FF последовательности 12h 34h а b с 56h.

Символы верхнего регистра отличаются от символов нижнего регистра. Например, 'ABC'- это не то же самое, что 'Abc' или 'abc' или другие комбинации верхних и нижних регистров. Однако, 'ABC' и "ABC" считаются одинаковыми, потому что кавычки - это всего лишь разделитель.

Попробуем найти слово 'пИво'. У нас получится следующее:

```
-S 0 L 0 'пИво' [ENTER]
-
```

Ну нет в этой программе пИва!

Попробуем пИво поискать в другом месте, а в нашей программе лучше поищем слово B7 20 B5 06:

```
-S 0 L 0 B7 20 B5 06 [ENTER]
15A3:0110
-
```

Итак, по адресу 15A3:0170 DZebug нашел слово B7 20 B5 06.

Опять же: значение сегмента у вас может быть иным, но смещение должно быть такое же...

Если мы выведем данные на экран, то по этому адресу мы найдем строку '.=...0..'. Мы можем попробовать найти строчку '_T- _:-.', или еще какую-нибудь нездоровую последовательность символов.

Если мы захотим найти все места, где употребляется команда Int 10h (в машинном коде эта команда имеет вид CD 10), мы сделаем следующее:

```
-S 0 L 0 cd 10 [ENTER]
```

и получим дли-и-инную простыню:

```
15A3:010E
15A3:011A
15A3:0126
15A3:0132
15A3:013E
15A3:0AE6
15A3:0F23
-
```

DZEBUG нашел последовательность CD 10 по этим адресам. Это не значит, что все CD 10 - это команды Int 10h. Просто по указанным адресам расположены искомые данные. Это может быть (чаще всего) инструкция, но это также может быть и адрес, вторая часть инструкции JMP и т.д. Вы должны внимательно изучить код программы на данном участке памяти, чтобы быть уверенным, что это действительно INT 10.

Вы ведь не думаете, что комп все сделает за вас? Конечно, компьютеры заменили человека во многих сферах деятельности, преимущественно там, где нужно работать нижними полушариями мозга, а там, где нужны еще и верхние полушария, без человека ну никак не обойтись! И это звучит гордо!

Помедитируйте немного и поехали дальше...

Сравнение участков памяти

Архиполезнейшая штука! Лично я долго над ней дЗенствовал :).

Команда compare берет два заданных участка памяти и сравнивает их, байт за байтом. Если два адреса содержат разную информацию, они выводятся на экран вместе с их содержимым. Для примера мы сравним 2 байта DS:0100 с DS:0200 .

```
-C 0100 L 8 0200
15A3:0100 30 0E 15A3:0200
15A3:0101 C0 00 15A3:0201
```

Все 2 байта различны, поэтому они все выведены на экран. Если какие-нибудь байты окажутся одинаковыми, то они не будут выводиться на экран. Если две области окажутся полностью одинаковыми, DEBUG просто ответит новым приглашением. Это очень удобный способ сравнения данных из памяти с данными из файла или ROM-BIOS :)

Дизассемблирование

Unassemble - основная команда, которую вы будете использовать при отладке. Эта команда берет машинный код и преобразует его в инструкции ассемблера. Способ задания адреса такой же, как и в предыдущих командах, с одной лишь разницей: поскольку мы теперь будем работать с кодом (предыдущие команды в основном предназначены для работы с данными), регистр по умолчанию - CX. В .COM программах это делает небольшое отличие, если только вы сами не очистите DS. Однако в .EXE файлах это чревато некоторыми осложнениями, потому что изначально регистрам CS и DS присвоены разные значения.

```
-u
15A3:0100 XOR AL,AL
15A3:0102 MOV BH,10
15A3:0104 MOV CH,05
15A3:0106 MOV CL,10
15A3:0108 MOV DH,10
15A3:010A MOV DL,3E
15A3:010C MOV AH,06
15A3:010E INT 10
15A3:0110 MOV BH,20
15A3:0112 MOV CH,06
15A3:0114 MOV CL,11
15A3:0116 MOV DH,0F
15A3:0118 MOV DL,3D
15A3:011A INT 10
15A3:011C MOV BH,30
15A3:011E MOV CH,07
```

Мы видим уже знакомую нам программу. Если мы опять введем "u", то DZebug выдаст нам очередную порцию кода. В нашем случае все обошлось благополучно, но бывает и так, когда вы не знаете, что вы в данный момент дизассемблируете - действительно код программы, или же ее данные. DZebug сделает все, что вы ему прикажете. Если вы скажете дизассемблировать данные, он сделает это, ничего не заметив.

Размещение данных в памяти

Команда Enter используется для размещения данных в памяти. Она имеет два режима: Display/Modify и Replace. Отличия между ними в расположении помещаемых данных - в самой команде Enter или после приглашения.

Если вы ввели E <адрес>, вы будете находиться в режиме Display/Modify. DZEBUG предложит вам изменить значение байта, отображая его текущее значение. Вы можете ввести один или два шестнадцатеричных символа. Если вы нажмете пробел, DZEBUG не будет изменять текущий байт, а перейдет к следующему. Если вы зашли далеко, нажатие минуса "-" возвратит на один байт назад.

```
-E 100 [Enter]
15A3:0100 30.41 C0.42 B7.43 10. B5.45
15A3:0105 05.46 B1.40 10.-
15A3:0106 40.47 10.
```

В нашем примере мы ввели E 100. Dzebug ответил адресом и значением байта по этому адресу (30). Мы ввели 41, и DZebug автоматически перешел к следующему байту данных (C0). Опять, мы вводим 42, и DEBUG переходит к следующему байту (B7). Мы изменили его на 43. По адресу 104 байт 10 нас вполне удовлетворяет, поэтому мы нажали пробел. DEBUG не изменил его значение и перешел к следующему байту.

После ввода 40 в позиции 105 мы обнаружили, что ввели неправильное значение. Нажимаем минус, и DEBUG возвращается на одну позицию назад, отображая адрес и содержимое. Обратите внимание, что оно отличается от первоначального (B5) и имеет значение, которое мы ввели (40). Мы вводим правильное значение и завершаем работу нажатием ENTER.

Как вы видите, это утомительная работа, особенно когда вы работаете с большими объемами данных или с ASCII-текстом - вы должны знать шестнадцатеричное значение каждого символа. В этом случае можно воспользоваться режимом Replace.

Режим Display/Modify предназначен для изменения значения небольшого количества байтов по различным смещениям. Replace предназначен для изменения любого количества идущих подряд байтов.

Данные можно вводить как в символьном, так и в шестнадцатеричном формате, и все байты можно ввести за один раз, не ожидая приглашения отладчика. Если вы хотите разместить строку 'Wind0yZ must Die', заканчивающуюся на 0, по адресу 100, то вы должны ввести : E 100 'Wind0yZ must Die' 0 Люди!!! То, что вы сейчас сделали, подобно самоубийству!!! Вместо МАТРИЦЫ вы только что поимели свою собственную программу. Вы отредактировали ее КОД!!! Причем самым извращенным образом!!!

Если теперь вы введет команду U, то увидите ЭТО:

```
15A3:0100 57      PUSH DI
15A3:0101 69      DB      69
15A3:0102 6E      DB      6E
15A3:0103 64      DB      64
15A3:0104 30795A XOR     [BX+DI+5A],BH
15A3:0107 206D75 AND     [DI+75],CH
15A3:010A 7374      JNB     0180
15A3:010C 204469 AND     [SI+69],AL
```

А если вам, не дай бог, взбредет в голову ввести команду G, то... Ну, в общем, сами увидите. Как и в команде Search, данные в символьном и HEX формате могут чередоваться в любом порядке. Это наиболее удобный способ размещения больших объемов данных в память.

Размещение данных одинакового значения

Команда Fill удобна для размещения данных одинакового значения. Она отличается от команды Enter тем, что завершает свою работу только тогда, когда будет полностью заполнен заданный участок памяти. Как и Enter, она работает и с символьными данными, и с шестнадцатеричными значениями. В отличие от Enter, с помощью этой команды можно заполнять большой объем памяти за один раз, без определения значения каждого символа.

Например, чтобы очистить 32K (8000h) памяти, вы всего лишь должны ввести команду:

```
-F 0 L 8000 0 [Enter]
```

В итоге все байты памяти начиная с DS:0000 и до DS:8000 будут обнулены. Если бы вместо нуля мы ввели '1234' , то память была бы заполнена повторяющейся последовательностью '123412341234', и так далее. Обычно, для задания небольших объемов данных лучше использовать команду Enter, потому что ошибка в длине при вызове команды Fill может наделать много бед. Команда Enter изменяет значения только тех байт, которые вы непосредственно укажете, что позволяет минимизировать вероятность ошибки.

Помедитируйте немного и поехали дальше...

Перемещение данных

Команда MOVE перемещает данные "внутри компьютера". Она берет данные, расположенные по одному адресу, и копирует их "в другой" адрес.

Если вы хотите выполнить эту команду во время трассировки, это может нарушить ход ее выполнения и получится очень здорово ;) - данные и инструкции, расположенные после "вставки", будут теперь расположены в са-а-авсем другом месте...

Команда MOVE может быть использована для сохранения части программы в свободной памяти, пока вы будете вносить в нее изменения. (Завернул??) Тогда программу можно будет восстановить в любой момент...

Вы можете вносить изменения "в BIOS" не утруждая себя программированием ROM:

```
- M 100 L 200 ES:100 [Enter]
```

Мы копируем данные с адреса DS:0100 до DS:02FF (длина - 200) в область памяти, которая начинается с ES:100. Позднее мы можем их восстановить. Нужно только ввести:

```
- M ES:100 L 200 100 [Enter],
```

, что скопирует данные туда, где они должны быть. Если мы не изменяли данные в памяти по адресу ES:0100, то эта команда восстановит первоначальное состояние памяти DS:0100 - DS:02FF. Вроде правильно написал, а? А если неправильно, то все равно ведь ничего страшного. ДЗЕНСТВУЮЩИЙ Sashok подправит, если что не так... верно?

Помедитируйте с часок и поехали дальше...

Ассемблирование

А вот теперь начинается самое интересное!

Команда ASSEMBLE запрашивает мнемоники (то бишь команды микроассемблера) и преобразует их в машинный код.

Есть некоторые операции, которая она не может проделать (в отличие от MASM/TASM/NASM): ссылка на метки, использование макроса или чего-нибудь еще, что не может СРАЗУ транслироваться в машинный код.

Обращение к данным должно происходить по их физическому адресу в памяти, сегментные регистры, если они отличаются от установленного значения, должны быть определены, и при использовании команды RET должен быть указан тип возврата (NEAR или FAR).

А еще, если инструкция обращается к данным, а не к регистрам (например, MOV [278], 5), то нужно указывать их длину - Byte ptr или Word ptr. Чтобы указать DZebug различие между пересылкой 1234h в AX и пересылкой слова по адресу 1234 в регистр AX, используют квадратные скобки - последнее будет иметь вид MOV AX, [1234].

Всяческие разновидности инструкции JMP автоматически ассемблируются в Short, Near или Far переходы.

А теперь мы... гы... напишем еще одно "окошко" :). Только оно будет не очень красивое :). Размеры его будут максимально возможными, а атрибут - стандартный досовский. Работать эта программа будет по образу и подобию команды CLS (очистка экрана).

```
-A 100
15A3:0100 mov ax,600
15A3:0103 mov cx,0
15A3:0106 mov dx,184f
15A3:0109 mov bh,07
15A3:010B int 10
15A3:010D int 20
15A3:010F
-
```

Мы использовали прерывание BIOS 10h, которое предназначено для работы с экраном. Мы обращаемся к BIOS с AX=600, BH=7, CX=0, and DX=184Fh. Сначала необходимо установить регистры, что мы и сделали, введя первые четыре инструкции. Команда по адресу 15A3:010B - команда обращения к BIOS. INT 20 (по адресу 010D) служит для безопасности. Нам эта команда практически не нужна, но когда она есть, программа остановится автоматически. Без INT 20, и если мы сами не остановим программу, DEBUG продолжит выполнение программы (от 010F и дальше). А так как после 010D начинается неопределенная область, то, скорее всего, система зависнет. Теперь поможет только ctrl-alt-del (может быть) или же выключение и включение питания. Будьте осторожны и дважды проверяйте, прежде чем что-нибудь делать. А еще лучше - трижды...

Теперь мы должны запустить программу. Чтобы это сделать, введите команду G и нажмите Enter. Если вы правильно ввели свою программу, экран должен очиститься и должно появиться сообщение "Program terminated normally". Более подробно команда Go будет рассмотрена ниже.

Опять же, я не могу выразить всей важности правильного введения инструкций при использовании Assemble. Особенно нужно быть осторожным с инструкциями типа JMP и CALL. Они изменяют ход выполнения программы, поэтому может случиться так, что выполнение начнется с середины какой-нибудь инструкции, что приведет к крайне нежелательным результатам.

Как говорили монашки, натягивая презервативы на свечи: "Береженого бог бережет"...

Предо... (тьфу) ... помедитируйте с часок и поехали дальше...

Имя, сестра, имя

Команда NAME служит только для одной цели - определить имя файла, который DZEBUG должен загрузить или сохранить. Она не изменяет память и не выполняет программу, она только формирует "блок контроля" для файла, с которым будет работать DZEBUG. Если вы хотите загрузить программу, то можете указать это в этой же строчке параметры, как при работе с ДОС. Единственное отличие - должно быть задано расширение.

Расширений по умолчанию не существует. DEBUG загрузит или запишет на диск любой файл, если указано его полное имя.

```
-n format.com c:/s
```

Мы приготовили DZEBUG к загрузке программы FORMAT.COM с заданным ключом. Когда мы введем команду Load (см. ниже), DZEBUG загрузит программу format.com с параметрами c:/s.

Ну, тут и ежу все понятно, можно не медитировать....

Грузить

Команда LOAD имеет два формата. Первый загружает программу, которая была определена командой NAME, устанавливает все регистры, готовит все необходимое для исполнения. Все заданные параметры программы будут помещены в PSP, и программа "приготовится" к выполнению.

Если файл в формате HEX, он должен содержать правильные шестнадцатеричные символы, которые определяют размер памяти. Загруженные файлы выполняются с адреса CS:0100 или с адреса, указанного в команде. Для файлов .COM, .HEX and .EXE регистры содержат адрес первой инструкции программы. Для других типов файлов регистры не определены. Сегментные регистры имеют значение, указанное в PSP (100h байт перед кодом программы), в BX и CX содержится размер файла. Остальные регистры не определены.

```
-n format.com  
-l
```

Эта последовательность команд загрузит format.com в память, поместит в IP точку входа - 0100, а CX будет содержать HEX-размер файла. Программа теперь готова к работе :)

Другой формат команды LOAD не использует команду NAME. Он предназначен для чтения секторов с диска (гибкого или жесткого) в память.

За один раз можно прочитать 80h (128d) секторов. При использовании этой команды вы должны указать начальный адрес, диск (0=A, 1=B и т. д.), начальный сектор и количество читаемых секторов.

Например

```
-l 100 0 10 20
```

указывает DZEBUG загрузить в память с DS:0100 20h секторов с диска A начиная с сектора 10h.

Таким образом, DZEBUG можно иногда использовать для восстановления части информации в поврежденном секторе. Но это уже изврат!

Вот. Кратко и лаконично. Медитируйте...

Писать (ударение на "а")

Команда WRITE очень похожа на команду LOAD. Обе имеют два режима работы, и обе могут работать как с файлами, так и с физическими секторами. Как вы наверное уже поняли, WRITE производит запись на диск. Поскольку все параметры такие же, как и в LOADe, мы не будем их опять описывать.

Отметим только одну вещь - при использовании этой команды размер записываемых данных определен в BX и CX, где BX содержит старшую часть размера файла. Начальный адрес должен быть определен по умолчанию - CS:0100. Файлы с расширением .EXE или .HEX не могут быть записаны (появится сообщение об ошибке). Если вы хотите изменить .EXE или .HEX файл, просто переименуйте его, загрузите, сделайте необходимые изменения, сохраните его и дайте ему прежнее имя.

Медитируйте...

Чтение данных из порта

Команда INPUT предназначена для чтения данных из любого I/O порта PC. Адрес порта может быть как однобайтовым, так и двухбайтовым. DZEBUG произведет чтение из порта и отобразит на экране его содержимое.

```
-i 3fd 7D
```

Этой командой мы прочитали данные из "входного" порта "первого асинхронного адаптера". Результат, который вы получите, может отличаться от приведенного :) - все зависит от текущего состояния порта. У меня в момент чтения регистр порта имел значение 7Dh.

Естественно, чтение из разных портов может привести к различным результатам.

Медитируйте...

Запись данных в порт

Вы можете использовать команду OUTPUT для того, чтобы послать один байт или последовательность данных в порт. Помните, что запись в некоторые порты может привести к непредсказуемым последствиям. Будьте осторожны при работе с этой командой!

```
-o 3fc 1
```

Порт 3FCh - это "регистр контроля модема" для "первого асинхронного порта". Запись в него 01h устанавливает бит DTR. 00h сбрасывает все биты. Если у вас есть модем, который отображает состояния этих бит, вы сможете увидеть вспышку лампочки, когда будете устанавливать и сбрасывать этот бит.

Медитируйте...

Go! Go! Go! Але-але-але...

Команда GO начинает выполнение программы. Она позволяет запускать программу с любой точки и останавливать ее в любой из десяти брекпоинтов программы. Если брекпоинты не установлены (или не выполнены), выполнение программы будет продолжаться до конца, после чего будет выведено сообщение "Program terminated normally". (Именно последнее, кстати и является "але-але-але").

Если выполнение программы дошло до брекпоинта, программа будет остановлена, отобразится содержимое регистров и появится обычное приглашение DZEBUG. Теперь можно вводить любые команды DZEBUG, включая и команду Go для продолжения выполнения программы.

Команда Go не может быть прервана нажатием Ctrl-break. Это одна из тех немногих команд, которые не могут быть прерваны во время исполнения.

```
-g =100
```

Команда GO без брекпоинтов начинает выполнение программы с адреса, указанного в параметре.

Кстати, перед адресом должен стоять знак "равно" - без этого знака адрес воспринимается как брекпоинт...

Если не указан стартовый адрес, выполнение программы начинается с CS:IP.

Что еще тут сказать? Пример если только привести...

```
-g 176 47d 537 647
```

Данной командой мы запускаем программу и устанавливаем брекпоинты по адресам CS:176, CS:47D, CS:537 и CS:647.

Несколько слов о бряках теперь...

Работа проги останавливается непосредственно ПЕРЕД брекпоинтами. Установка их на текущую инструкцию приведет к тому, что программа не будет выполняться. DZEBUG сначала устанавливает брекпоинт, и только потом пытается выполнить программу. Бряки (разновидность бряк) используют INT 3 для остановки выполнения. DZEBUG вызывает прерывание 3 для остановки программы и отображает содержание регистров.

Брекпоинты не сохраняются между двумя вызовами команды GO. Все брекпоинты вы должны указывать при КАЖДОМ обращении к этой замечательной команде :)

С зеленым чаем в "точках останова"... медитируем...

"Трррассиррровка" - сказал пулеметчик

Команда TRACE имеет сходство с командой GO. Различие между ними заключается в том, что GO выполняет целый блок кода за один раз, а TRACE выполняет инструкции по одной, каждый раз отображая содержимое регистров.

Как и в GO, выполнение можно начинать с любой точки. Перед стартовым адресом должен стоять знак "равно". При вызове команды можно указать количество инструкций, которые нужно выполнить.

```
-t =100 5
```

Эта команда начнет работу с адреса CS:100 и выполнит пять инструкций. Без указания адреса, выполнение начнется с CS:IP. Команда T без параметров выполнит только одну инструкцию. При использовании TRACE желательно обходить обращения к DOS и другие прерывания. DOS нельзя трассировать - это может привести к плохим последствиям. Можно трассировать программу до инструкции прерывания, затем командой GO выполнить прерывание и продолжать трассировку дальше.

Медитируем-с... скоро нирвана...

Плюс-минус

С помощью команды с завернутым названием HEXARITHMETIC можно складывать (+) и вычитать (-) шестнадцатеричные числа. Она имеет два параметра: два числа, которые нужно сложить или вычесть. Числа должны иметь длину не более четырех шестнадцатеричных цифр. Сложение и вычитание беззнаковое, не учитывается переполнение старшего разряда.

```
-h 5 6  
000B FFFF
```

```
-h 5678 1234  
68AC 4444
```

В первом примере мы хотели сложили 0005 и 0006. Их сумма - 000B, а разность -1. Однако, т.к. числа беззнаковые, мы получили FFFF.

Во втором примере сумма 5678 и 1234 - 68AC, а разность - 4444.

Медитируем...

Нирвана!

Имею счастье заявить: я кончил.

Программирование на Ассемблере под DOS

Автор: Serrgio / HI-TECH

Дата: 2002-08-16

2.1. Проба молотка

#1. Теоретически гвозди можно забивать и голыми руками. Но намного быстрее и безболезненнее делать это с помощью молотка. Пользоваться им, как известно, каждый дурак умеет. Чего там сложного? Взял оный в руки - и молоти: раз по гвоздю, два раза по пальцам (понимание приходит с опытом). Молотки бывают разные: большие и маленькие, с длинной ручкой и с короткой ручкой, железные и деревянные, приспособленные для забивания гвоздей и приспособленные для пробивания черепов... Да и гвоздей разнообразие еще то! И разной длины, и разной толщины, и со шляпками разной формы, из разного материала сделанные... есть даже специализированные для прибивания рук к кресту! :(.

Так вот: виды молотков и особенности их использования мы пока что оставим в покое. Для начала просто возьмем гвоздь, который уже не раз забивали голыми руками (больно было?) и забьем его при помощи молотка! Помните, как мы программу, выводящую окошки, без компилятора одними мнемониками писали?

А теперь ее же, дуру, "под компилятор" перелопатим... В общем, настало время, братья, тайну нервную и страшную узнать. Сегодня мы познакомимся с **компилятором**. **#2.** Нижеследующий текст набираем в **текстовом редакторе** :

```
;-[блок 2]-----
CODESG segment
assume CS:CODESG
org 100

;-[блок 3]-----
MAIN proc
    xor AL,AL
    mov BH,10h
    mov CH,5
    mov CL,10h
    mov DH,10h
    mov DL,3Eh
    mov AH,6
    int 10h
    call WINDOW
    call WINDOW
    call WINDOW
    call WINDOW
    int 20h
MAIN endp

WINDOW proc
    ADD BH,10h
    ADD CH,1
    ADD CL,1
    SUB DH,1
    SUB DL,1
    INT 10h
    RET
WINDOW endp

;-[блок 4]-----
CODESG ends

end MAIN
```

Сохраняем получившуюся дуру под именем **proga_1.asm**.

Далее запускаем файл `tasm.exe` и в качестве параметра передаем ему имя файла с исходным текстом программы...То есть командная строка у вас (в том же Windows Commander'e) вот какая должна быть:

```
tasm proga_1.asm
```

Если вы правильно набрали текст программы, то TASM должен выплюнуть вам вот что:

```
Turbo Assembler Version 4.1 Copyright (c) 1988, 1996 Borland International
Assembling file: proga_1.asm
Error messages: None
Warning messages: None
Passes: 1
Remaining memory: 406k
```

Самое тут главное - это чтоб **"Error messages"** был **"None"**. Это значит, что ошибок в программе нет.

Поехали дальше... Если ошибок у вас действительно никаких не было, то в том же каталоге, что и **proga_1.asm** ищите файл **proga_1.OBJ**. Можете даже по F3 попробовать просмотреть его содержимое... Что-нибудь поняли? Если нет - отсылаю вас к *"Введению в машинный код"*.

А теперь запускаем хорошую программу **TLINK** следующим образом:

```
tlink /t proga_1.obj
```

Обратите внимание: линкуем мы именно файл с типом **OBJ**, а не **ASM**.

Что получилось? А получился файл **proga_1.COM**!! И этот **.COM** работает!

Посмотрите на его содержимое в DZEBUG'e :). Отличается ли оно чем-то от той проги, что мы делали ранее?

Нет, не отличается!

Медитируем...

#3. А теперь несколько слов о том, почему не стоит писать программы так, как мы это делали раньше:

1. Эти проклятые адреса! Пойди рассчитай на какой адрес прыгнуть нужно, с какого смещения подпрограмма начинается, с какого блок данных... В принципе, как мы уже убедились, в этом нет ничего сложного... просто занятие это очень уж муторное и неинтересное...

Как абсолютно верно заметил некто Евгений Олейников в **RTFM_Helpers** : *"Когда я пишу программу, я не знаю точного адреса начала будущей подпрограммы"*...

2. Очень сложно было в том случае, когда возникала необходимость ВСТАВИТЬ ту или иную команду в середину кода. Приходилось перебивать код по новой... по новой пересчитывать адреса... ужас, в общем!

Так вот: основная и самая главная функция "ассемблерного компилятора" - это как раз и есть "просчитывание адресов"!

Смотрите, как хорошо и приятно: мы готовим исходный текст в обыкновенном текстовом редакторе :). Просто набиваем строчка за строчкой - нам не важны адреса (мы их вообще не видим!), не важны "точки входа" в процедуру... Спокойненько вставляем какую надо команду, спокойненько удаляем... Без лишнего напряжения!

Да вы посмотрите на блок 3 программы :). Там все те же хорошо знакомые команды :).

Особое внимание обратите на команду **CALL**, которая у нас, как известно, вызывает процедуру. После нее идет не привычный адрес начала процедуры, а всего лишь ее "имя собственное"! А сама процедура находится между строчками **WINDOW proc**(начало процедуры) и **WINDOW endp** (конец процедуры).

"WINDOW" - понятно, это "имя собственное". **"Proc"** - потому что процедура. **"Endp"** - потому что конец процедуры...

Тут еще один момент... подобные "слова" КОМАНДАМИ АССЕМБЛЕРА НЕ ЯВЛЯЮТСЯ. Они называются иначе - **"директивами"**. Это не ПРИКАЗ делать то-то и то-то, а ЦЕННОЕ УКАЗАНИЕ компилятору (не процессору!), что и как ему делать с данным куском кода...

Процедуры - вещь хорошая! Все процедуры хороши, и в большой программе их чертовски много! Это вам не языки высокого уровня, где можно длиннющие простыни кода лабать и все будет работать! (*"Дельфи-компилятор не даст вам выстрелить себе в ногу, но будет так удивлен попыткой сделать это, что через некоторое время сделает это сам."* (C) DZ WhiteUnicorn).

Это ASM! Тут запутаться легче простого! Исходники неимоверно длинны и сложны! Все делается ручками (хоть и с помощью компилятора), а ошибки довольно трудно отслеживаются (для не-дЗенствующих это вообще занятие безнадежное)! Вот и выкручиваются низкоуровневые программисты таким вот образом: всю программу (даже в тех случаях, когда это не обоснованно!) делят на маленькие кусочки-процедуры... Напишут процедуру, протестируют ее так и сяк... если работает - за следующую берутся...

Сии "методы проектирования" мы с вами еще не раз рассмотрим :). Пока что знайте вот что: любую прогу можно/нужно рассматривать как КУЧУ ПРОЦЕДУР, которые все между собой повязаны...

Но есть среди этих процедур САМАЯ ГЛАВНАЯ! Это та, С КОТОРОЙ НАЧИНАЕТСЯ ВЫПОЛНЕНИЕ ПРОГРАММЫ! Ее никто не вызывает. Она - босс! Она всеми командует, все гребет под себя... Описывают ее те же самые директивы, что и прочие "подчиненные процедуры"... Но есть у нас еще одна директива, которая указывает, КАКАЯ ИМЕННО процедура ИЗ ВСЕХ вроде бы "равноправных" является ГЛАВНОЙ.

Видите строчку **end MAIN** в конце исходного текста программы? Именно она и указывает ГЛАВНУЮ ПРОЦЕДУРУ ("MAIN" - ее имя собственное). Если бы мы написали **end WINDOW**, то выполнение программы у нас началось бы с первой строчки процедуры WINDOW, и ни одна строчка из MAIN выполнена не была бы...

Уф... в общем, долго и упорно медитируйте...

#4. Наличие многочисленных директив - это своего рода плата за то, что компилятор избавляет нас от необходимости просчитывать адреса. Как говорит дЗенская программистская мудрость "любишь кататься, люби и саночки возить"...

Как и в DZEBUG'e, "в TASM'e" мы также должны четко инструктировать компилятор (дабы он в свою очередь также четко проинструктировал процессор), что у нас является кодом, а что - данными...

Посмотрите на исходник. Весь текст программы у нас хранится между директивами **CODESG segment** и **CODESG ends**. Где **CODESG** - это "имя собственное", **"segment"** - потому что "CODESG" он, собственно, и есть сегмент :), и **"ends"** - потому что конец сегмента... (сравните с процедурными директивами).

Но тут такой вопрос:

Директива **ASSUME** производит связывание сегментного регистра CS с "именем собственным".

Далее у нас следует директива **org 100h**. Нужна она нам для того, чтобы компилятор понял, что мы хотим получить именно COM-файл, который, как известно, помещается в сегмент памяти начиная со смещения 100 (в общем-то, это необходимое, но вовсе не достаточное условие для получения COM-файла). Директива очень интересная, о ней мы, надеюсь, еще поговорим подробнее, когда коснемся вирмейкерства. **#5.** Ладно... с директивами более-менее разобрались, исходник приготовили, пора разбираться что там дальше происходит...

Дальше происходит так называемое "ассемблирование", т.е. перевод команд в соответствующие машинные коды. При этом просчитываются адреса меток, адреса начала подпрограмм, адреса начала/конца сегментов... и многое другое...

Причем ассемблирование происходит как минимум в два приема. Посудите сами: откуда компилятору знать, с какого адреса начнется процедура WINDOW, если не известно, какая еще простыня команд ПОСЛЕ этого CALL'а будет? В DZEBUG'e мы это "в уме на листике" считали...

TASM это тоже аналогичным образом делает :).

При первом проходе он подсчитывает, сколько какая команда занимает места, с каких адресов начинаются процедуры и т. д., и только при втором проходе подставляет в call'ы КОНКРЕТНЫЕ АДРЕСА начала этих процедур... всего лишь... ну еще и ваши "d" и "b" в машинные "h" (которые на самом деле "b") переводит... (во завернул!)...

А вообще, TASM много еще чего делает... программеры - народ ленивый...

В результате ассемблирования мы получаем так называемый "объектный файл".

"И что это за дрянь?" - спросите вы, и правильно спросите...

А вы сравните шестнадцатеричное содержимое **OBJ** и **COM** файлов. В OBJ присутствует та же последовательность байтов, что и в OBJ. Но помимо этого и еще какая-то шестнадцатеричная ерунда присутствует: имя сассемблированного файла, версия ассемблера, "имя собственное" сегмента и т. д.

Это своего рода "служебная" информация, предназначенная для тех случаев, когда ваш исполнимый файл вы хотите собрать из нескольких. При разработке больших приложений исходный текст, как правило, хранится в нескольких файлах; в каждом из них прописаны свои сегменты кода/данных/стека. А исполнимый получить нам нужно только один - с единым сегментом кода/данных/стека. Именно это **TLINK** и делает: завершает определение адресных ссылок и объединяет, если это требуется, несколько программных модулей в один...

И этот один у нас и является исполнимым... УРА!

#6. Итак, первую прогу с использованием компилятора мы написали. Теперь напишем вторую. Набиваем ее исходный текст и компилим:

```
assume CS:SUXXX, ES:SUXXX

SUXXX segment
org 100h

MAIN proc
    lea bp,ABC
    mov AH,13h
    mov AL,3
    xor bh,bh
    mov bl,07h
    mov cx,16d
    xor DX,DX
    int 10h
    int 20h
MAIN endp

ABC db 'H',0Ah,'e',0Bh,'l',0Dh,'l',0Ch
    db 'o',0Bh,',',' ',0Ah,'w',09h
    db 'o',08h,'r',07h,'l',06h,'d',05h
    db '!',02h,'!',02h,'!',02h

SUXXX ends

end MAIN
```

Итак, в этой программе мы использовали функцию 13h прерывания 10h (INT 10h, AH=13h).

Вот ее описание:

[INT 10h, ФУНКЦИЯ 13h] - записывает на экран символьную строку, начиная от указанной позиции.

ВХОДНЫЕ ПАРАМЕТРЫ:

AH = 13h;

AL - код формата(0-3):

AL=0, формат строки{симв., симв.,..., симв.} и курсор не перемещается,

AL=1, формат строки{симв., симв.,..., симв.} и курсор перемещается,

AL=2, формат строки{симв., атр.,...,симв., атр.} и курсор не перемещается,

AL=3, формат строки{симв., атр.,...,симв., атр.} и курсор перемещается;

BH - страница дисплея;

BL - атрибут (для режимов AL=0, AL=1);

CX - длина строки;

DX - позиция курсора для записи строки;

ES:BP - указатель строки.

ВЫХОДНЫЕ ПАРАМЕТРЫ: отсутствуют.

А еще мы использовали команду LEA, которая загружает в регистр адрес (смещение), с которого у нас начинается блок данных.

2.2. Несколько "тупых" процедур

#1. Дабы сходу "обломать" нездоровую критику, сразу предупреждаем, что нами сознательно допущена некоторая излишняя "процедуризация" исходного кода. А по сему, прежде чем взяться за изучение нижеследующего материала, твердо уясните: в данном случае (как есть сейчас) дробление кода на процедуры - дурь полная. А обосновываем мы эту дурь только тем, что впоследствии будем совершенствовать эти процедуры до уровня, когда, собственно, сие "дробление" и будет обосновано.

И еще. Подобный модульный подход (это понятие немножко шире, чем просто "использование процедур") в некоторых случаях снижает быстроедействие и увеличивает размер исполнимого файла, однако он же и значительно увеличивает "читабельность" исходника, что вполне обоснованно с точки зрения обучения. И с точки зрения "командной" разработки программ - тоже. (В общем, критика по этому поводу не принимается!).

Итак, для начала набиваем вот какой текст (исходник это!):

```
assume CS:PROGA

PROGA segment
org 100h

;-[TESTING]-----
;Здесь мы будем тестировать процедуры
;-----
TESTING proc
    call EXIT_COM
TESTING endp

;-[EXIT_COM, V1]-----
;Завершение работы программы
;На входе: пофиг
;На выходе: нихрена
;Прерывания: INT 20h
;Процедуры: ан нэту
;-----
EXIT_COM proc
    int 20h
EXIT_COM endp

PROGA ends

end TESTING
```

Здесь вам все должно быть понятно. INT 20h вынесен в отдельную процедуру, и только. Плюс еще какие-то нездоровые заголовки добавлены, которые и весят-то больше, чем сам код. Это нормально. После точки с запятой в исходнике вы можете писать все, что угодно. Все равно при компиляции это будет проигнорированно и, следовательно, на размер исполнимого файла не повлияет. (Точно так, как и длина "имен собственных" процедур и меток).

Мы предлагаем использовать именно такую "шапку" комментария к каждой из ваших процедур. Все очень просто. Первая строчка - это "что делает процедура". Вторая - какие ей необходимо передать параметры. Третья - какие она возвращает параметры. Четвертая и пятая, соответственно, - какие в процедуре использовались прерывания и хм... ранее написанные процедуры :). Это намного облегчит понимание вашего исходника как вами самими, так и теми, кому вы его предоставите на поругание...

Для тех, кто в танке: последующие процедуры вставляйте между процедурами TESTING и EXIT_COM - не ошибетесь :-p.

#2. Следующая процедура также основана на одном-единственном прерывании. Все, что она делает - это возвращает текущие координаты курсора.

```
;-[CURSOR_READ, V1]-----
;Возвращает координаты курсора
;На входе: пофиг
;На выходе: DH - строка, DL - столбец
;Прерывания: INT 10h, AH=03h
;Процедуры: ан нэту
;-----
CURSOR_READ proc
    push AX
    push BX
    push CX
    mov AH,3
    xor BH,BH
    int 10h
    pop CX
    pop BX
    pop AX
    ret
CURSOR_READ endp
```

Прежде всего обратите внимание на цепочку push'ей и pop'ов (далее - просто "поп"), на очередность записи в стек (AX, BX, CX) и извлечения (CX, BX, AX - обратное то есть). Все регистры, которые мы собираемся изменять внутри процедуры, должны обязательно сохраняться в ее начале и восстанавливаться в ее конце. В важности соблюдения этого правила вы еще не раз убедитесь на своем горьком опыте. Те, кто медитировал над заданием из главы 1.10 #4, уже знают, о чем тут идет речь (а кто не пытался - самое время!).

Давайте посмотрим на нашу процедуру с точки зрения пушей и поп. AH мы изменяли для указания функции прерывания, которую мы хотим использовать. BH обнуляли для указания видеостраницы (пока будем только одну-единственную, нулевую, юзать). "А CX зачем" - спросите. - "Вроде мы его не трогали...". И точно, мы - не трогали. А посмотрите в описании, что в этот регистр нам засунуло прерывание в "результате" своего выполнения... Посмотрели? Оно вам надо?? То, что нам надо - координаты - засунуты в DX (DH, DL), поэтому их значения мы не сохраняем. Если бы нам нужно было получить информацию о типе курсора - мы бы запустили DX, а CX бы оставили в покое...

Что? Без пива не разобраться?

Так в чем, черт подери, дело? Разбирайтесь под пиво!!

Вот вам еще одна аналогичная процедура, которая не определяет, а УСТАНОВЛИВАЕТ курсор в заданные координаты...

```
;-[CURSOR_SET, V1]-----
;Устанавливает курсор в заданные координаты
;На входе: DH - строка, DL - столбец
;На выходе: ни хрена
;Прерывания: INT 10h, AH=02h
;Процедуры: ан нэту
;-----
CURSOR_SET proc
    push AX
    push BX
    push CX
    mov AH,2
    xor BH,BH
    int 10h
    pop CX
    pop BX
    pop AX
    ret
CURSOR_SET endp
```

Если кто чего не понимает - смотрите комментарии к предыдущей процедуре (+ описание прерываний и команд! это обязательно!). Если кто не понял и предыдущую - снова отсылаю к главе

1.10

#3. На основе двух предыдущих процедур мы напишем третью "курсорную" :), которая будет сдвигать курсор на одну позицию вправо.

```
;-[CURSOR_RIGHT, V1]-----
;Перемещает курсор на одну позицию вправо
;На входе: пофиг
;На выходе: нихрена
;Прерывания: ан нэту
;Процедуры: CURSOR_READ, CURSOR_SET
;-----
CURSOR_RIGHT proc
    push DX
    call CURSOR_READ
    inc DL
    call CURSOR_SET
    pop DX
    ret
CURSOR_RIGHT endp
```

А здесь очень простая идеология :)). Из двух процедур мы собрали третью :)). Вызвали CURSOR_READ, получили в DX текущие координаты курсора. Ту координату, что столбец (DL, младшая часть DX), увеличили на единицу. А потом вызвали процедуру CURSOR_SET, которая у нас устанавливает координаты курсора. Новые координаты в нее передаются опять таки через тот же DX. Улавливаете?

Естественно, мы запросто можем отказаться от процедур CURSOR_SET и CURSOR_READ и решить данную задачу внутри одной процедуры... В общем, свой выбор вы сделаете сами. Страшна Сцилла оптимизации по быстродействию, еще страшнее - Харибда оптимизации по размеру, но тварь самая страшная - это Программер, который оптимизирует свой код по собственной "удобноваримости"... (Хм... интересно, что бы сказали по этому поводу программеры Мелкософта...)

Обратите также внимание на push/pop DX внутри процедуры. Мы просто сдвигаем курсор вправо. ПРОСТО СДВИГАЕМ на одну позицию. То что в DX нам надо? Сто лет оно нам не надо... херим... А остальные регистры - еще процедурами CURSOR_SET и CURSOR_READ неоднократно "похерены". В каком смысле "похерены"? А в таком, что состояние регистров ПОСЛЕ вызова CURSOR_RIGHT в точности такое же, как и было ДО. Хотя, сами помните, что всю четверку регистров мы еще как юзали...

Вот теперь можно сделать паузу и (это американцы пускай свой пластмассовый твикс кушают) ПОМЕДИТИРОВАТЬ...

#4. Следующая процедура ну вааще элементарна:

```
;-[WRITE_CHAR, V1]-----
;Печатает символ и переводит курсор на позицию вправо
;На входе: DL - код символа.
;На выходе: нихрена
;Прерывания: INT 10h, AH=09h
;Процедуры: CURSOR_RIGHT
;-----
WRITE_CHAR proc
    push AX
    push BX
    push CX
    mov AH,9
    xor BH,BH
    mov BL,00000111b
    mov CX,1
    mov AL,DL
    int 10h
    call CURSOR_RIGHT
    pop CX
    pop BX
    pop AX
```

```
ret
WRITE_CHAR endp
```

Она символ на монитор выводит. Через 9-ю функцию 10-го прерывания. А потом (после вывода) курсор на позицию вправо перемещает. Догадаться сами, "путем вызова" какой процедуры... Ага, правильно :)), CURSOR_RIGHT.

Посмотрите на описание этого прерывания. Код символа должен быть в AL. А у нас в комментариях он в DL прописан. А перед INT 10h mov AL,DL нездоровый стоит. Нахрена он тут?? И правильно!! Этот mov можно удалить, и передавать значение через AL. Но я тварь вредная. Привык я, понимаете-ли, через DX передавать... Привычка - сила страшная!! Лень с ней бороться... Лень, а поэтому и не буду... Кто-то в подобном мове может и более глубокий смысл найдет - наверняка найдет!! В общем - ищите сами. На блюдецке с голубой каемочкой вам это не преподнесу :)). Вредный.

mov BL,00000111b (не 07h) написано специально. Это чтоб вы посмотрели, как кодируется атрибут (фон, цвет) этого символа. В одном из предыдущих номеров даже табличка есть, из справочника содранная...

#5. В языке "командного интерпретатора DOS" есть хорошая команда - CLS (то бишь очистка дисплея). Хорошая команда! Кто не поленится заглянуть внутрь command.com'a, увидят приблизительно следующее (на самом деле все чуть-чуть навороченнее, но прерывание то же):

```
;-[CLS, V1]-----
;Очистка дисплея
;На входе: пофиг
;На выходе: ни хрена
;Прерывания: INT 10h, AH=06h
;Процедуры: ан нэту
;-----
CLS proc
push AX
push BX
push CX
push DX
mov AH,6
xor AL,AL
mov BH,00000111b
xor CX,CX
mov DH,24d
mov DL,79d
int 10h
pop DX
pop CX
pop BX
pop AX
ret
CLS endp
```

Короче, элементарный скроллинг, только заданы максимально возможные координаты скроллируемого окошка и CX=0... в общем, окошки рисовали, помните...

#6. Тестируем, штоль?? Дописываем процедуру TESTING:

```
TESTING proc
mov DL,'*'
call WRITE_CHAR
call WRITE_CHAR
call WRITE_CHAR
call EXIT_COM
TESTING endp
```

Компилим... CLS тож тестируем... Все работает??

А теперь самое интересное :)) и благоприятно влияющее на нервную систему :). Прелесть модульного подхода вот в чем: написали процедуру, протестировали успешно - и МОЖЕТЕ ЗАБЫТЬ нафиг, как она у вас работает, какие функции там используются, какие хитроПОПые

алгоритмы там применены...

Просто смотрите на заголовок, че она делает, чего ей надобно на входе, чего возвращает... а ее "внутренности" вам глубоко фиолетовы. Работает - и ладно. Аааааа?? Круто?

Уф... медитируйте!!

2.3. Печать "шестнадцатеричных циферек"

#1. Мы уже неоднократно юзали хорошую мнемоническую (ака ассемблерную) команду ADD :). Напомню, что в результате выполнения команд

```
mov AX,2
mov BX,3
add AX,BX
```

в регистр AX у нас помещалась сумма (AX=AX+BX).

Мы смотрели на это дело под отладчиком, и, к своей неопишуемой радости, убеждались в том, что эта дрянь действительно работает. Но толку нам знать, что она работает?? Программа ведь не только работать должна, но еще и диалог какой-нить между юзверем и компьютером обеспечивать! Например, спрашивать у него эти два числа и выплевывать на монитор результат их сложения.

Вот именно - выводить на монитор, а не заносить в какой-то абстрактный регистр.

С клавиатурным вводом пока обождем, а вот с выводом (на монитор) разберемся прямо сейчас.

Как мы это сделаем? Вы уже неоднократно слышали, что "в ассемблере" все делается "ручками" :). Сейчас вы лишний раз убедитесь в том (некоторые замрут в ужасе), что это утверждение истинно. Для вывода значения регистра мы вовсе не "познакомимся с новым прерыванием". Даже такая простейшая операция, как "вывод на дисплей значения регистра (переменной)" - это целая процедура. И не одна, как вы скоро в этом убедитесь. Страшно?

Поехали!!

#2. Задача: вывести на монитор значение регистра DL.

Народ! Давайте сразу расставим границы между КОДОМ СИМВОЛА и его НАЧЕРТАНИЕМ.

Например, у нас F3h в DL. Как мы хотим это вывести? Как символы 'F3' или же как ASCII символ, соответствующий коду F3h? Определяемся. Если в DL у нас F3h - то надо чтоб именно 'F3' у нас на монитор и выводилась. Не 'э перевернутое', не '46 33', а именно 'F3'. Помедитируйте. Уловите разницу между 'э', '46 33' и 'F3'.

Эту задачу мы немножко упростим :). Для начала напишем процедуру, которая выводит только младшую тетраду регистра DL (цифру "3" в нашем примере). Для этого мы обратимся к процедуре WRITE_CHAR из прошлого номера. Именно она печатает нам на монитор символ, ASCII-код которого находится в DL.

Но тут загвоздка: в DL-код, а печатается-то символ :). А нам, собственно, именно две циферки шестнадцатеричного кода, как два символа, и нужно напечатать. Ну, или хотя бы младшую циферку этого кода...

Решается эта задача элементарно :). Главное - это правильно ее сформулировать!

Вот что я тут "нарисовал":

ЕСТЬ		НУЖНО	
код	символ	символ	код
00h	'0'	'0'	30h
01h	'1'	'1'	31h
02h	'2'	'2'	32h
03h	'3'	'3'	33h
04h	'4'	'4'	34h
05h	'5'	'5'	35h
06h	'6'	'6'	36h
07h	'7'	'7'	37h
08h	'8'	'8'	38h
09h	'9'	'9'	39h

0Ah	'?'	'A'	41h
0Bh	'?'	'B'	42h
0Ch	'?'	'C'	43h
0Dh	'?'	'D'	44h
0Eh	'?'	'E'	45h
0Fh	'?'	'F'	46h

Только во второй колонке вместо вопросительных знаков должны быть соответствующие всякие символы (посмотрите в ASCII-таблице, какие они на вид страшные!).

Процедура наша вот что должна делать - Всего-навсего перевести (конвертировать) тетраду в код соответствующего ей символа... Завернуто? Если разобраться, то не очень-то и завернуто.

Смотрите: в DL у нас 03h. Хотим мы эту '3' на монитор вывести. Если вызовем WRITE_CHAR, то у нас символ "сердечко" выплунется. А надо, чтоб символ '3' вывелся, код которого 33h.

Соответственно и для остальных смотри по табличке.

А теперь обратите внимание, насколько "шестнадцатеричная циферка" (тетрада) отличается от ASCII-кода, этой "циферке" соответствующего. Сам скажу: на 30h для цифр от '1' до '9', и на 37h для цифр от 'A' до 'F'. То есть "переконвертацию" мы запросто можем сделать командами add DL,30h (если тетрада в диапазоне 0...9) и add DL,37h (если тетрада в диапазоне A...F).

Короче, вот код (пропиваю!):

```
;-[WRITE_HEX_DIGIT, V1]-----
;Печатает одну шестнадцатеричную цифру (младшую тетраду DL)
;(старшая тетрада должна быть равна 0)
;На входе: DL - цифра
;На выходе: нхрена
;Прерывания: ан нэту
;Процедуры: WRITE_CHAR
;-----
WRITE_HEX_DIGIT proc
    push DX
    cmp DL,0Ah
    jae HEX_LETTER
    add DL,30h
    JMP WRITE_DIGIT
HEX_LETTER:
    add DL,37h
WRITE_DIGIT:
    call WRITE_CHAR
    pop DX
    ret
WRITE_HEX_DIGIT endp
```

Сначала, ессно, изменяемые регистры сохраняем (мы ж их изменяем!). (Ну, и восстанавливаем в конце процедуры (PUSH и POP соответственно)).

Потом у нас логическое ветвление организовано. Сравниваем значение DL с "общей границей" наших двух диапазонов (команда - CMP, "граница" - 0Ah). Если это значение больше или равно 0Ah, то прыжок на метку HEX_LETTER, прибавление к DL 37h и печать цифры (WRITE_CHAR). Иначе добавляем 30h и безо всяких условий перепрыгиваем на вызов WRITE_CHAR (минуя add DL,37h то бишь).

Все. Тестируем.

#3. Про тестирование - базар отдельный. В данном случае мы можем запросто проверить нашу процедуру на абсолютно всех возможных значениях этой тетрады (всего-то ничего- 16 вариантов). Но намного правильнее, проанализировав алгоритм, установить своего рода "критические" значения, на которых целесообразно проводить проверку. Плюс, естественно, минимальное и максимальное значения.

```
TESTING proc
    mov DL,00h
    call WRITE_HEX_DIGIT
    mov DL,01h
```

```

call WRITE_HEX_DIGIT
mov DL,09h
call WRITE_HEX_DIGIT
mov DL,0Ah
call WRITE_HEX_DIGIT
mov DL,0fh
call WRITE_HEX_DIGIT
call EXIT_COM
TESTING endp

```

Если сия "тестовая" (она же - главная) процедура выведет на монитор

019AF

, значит мы с высокой долей вероятности можем быть уверенными, что процедура WRITE_HEX_DIGIT работает правильно на всех значениях младшей тетрады DL.

Кто не просто скопировал процедуру из буфера обмена, а действительно разобрался с тем, как она работает - сами знают, что значение старшей тетрады нашей процедуре НЕбезразлично. Оно должно быть равным 0.

#4. Что мы имеем? Процедуру для вывода на дисплей одной шестнадцатеричной циферки - младшей тетрады (в DL). Но нам-то нужно две вывести! Сначала старшую циферку-тетраду, и только потом - младшую!

Таким образом очередная задача разбивается на две части: печать старшей тетрады DL и печать младшей тетрады DL.

Первая "подзадача" решается легко: нужно просто старшую тетраду переместить на место младшей и вызывать процедуру (основательно протестированную и 99,9%-но работающую) WRITE_HEX_DIGIT.

А вторая подзадача - хм... заключается в восстановлении предыдущего (мы ж тетраду перенесли) значения DL и снова - вызове WRITE_HEX_DIGIT.

(Хе! Вот теперь-то вы уж точно почувствуете всю прелесть "дробления кода на процедуры"!)

Перенос тетрады мы осуществим при помощи команды SHR, которую в умных книжках обзывают как "логический сдвиг разрядов операнда вправо". Объясню.

Представьте себе деревянную доску, длиной в 8 бутылок пива и шириной в одну. (В принципе, доску эту можно и в два раза длиннее представить, но тогда на ней надо "DX" написать, мы же пока только "DL" напишем). А еще дурня, у которого на лбу SHR написано. Так вот, если этому придурку стукнуть по хребту, то он слева от доски поставит ПУСТУЮ бутылку, а остальные сдвинет на одну позицию вправо, в результате чего самая правая бутылка, ессно, с доски упадет.

Бутылки, которые сразу стояли, могут быть пустыми или полными, а вот дурень SHR - только пустые ставит. И только слева.

```

"исходное" 11110011
SHR        01111001
SHR        00111100
SHR        00011110
SHR        00001111

```

Ну тут и ежу все понятно. Четыре раза дурню по хребту надо дать, чтоб старшая тетрада на место младшей переместилась.

(На самом деле самая правая бутылка перед тем как об землю разбиться, на некоторое время в воздухе зависает, но вы пока этим голову не забивайте).

Реализовывается этот сдвиг вот как:

```

mov DL,11110011b
mov CL,4
shr DL,CL

```

В DL - наша цепочка битов. (11110011b = F3h, естественно).

В CL заносим "на сколько позиций" нам нашу цепочку сдвинуть.

Ну и SHR - это дурень, который сдвигает вправо, а слева нули дописывает.

#5. Думаете, это все?? Разогнались!! Не все так просто :).

WRITE_HEX_DIGIT у нас требует, чтобы первой тетрадой были только одни нули. Я заострял на этом ваше внимание.

При печати первой тетрады это условие соблюдается. SHR слева нули дописывает.

А вот при печати второй цифры нужно вот что: ничего никуда не сдвигая, обнулить старшую тетраду, а младшую (которая, собственно, и есть "цифра") оставить в покое.

Решим мы эту команду при помощи логической операции "и" (and по-аглицкому).

```
0 0 1 1
0 1 0 1
-----
0 0 0 1
```

А теперь и для особо одаренных:

```
0 and 0 = 0
0 and 1 = 0
1 and 0 = 0
1 and 1 = 1
```

Смотрите, интересно как получается:

Если мы AND чего-либо (нуля или единички) с 0 делаем, то у нас в результате 0, и только 0 получается.

А если AND с единичкой - то ЧТО БЫЛО, ТО И ОСТАЕТСЯ.

(Это и есть потаенный дЗенский смысл команды AND)

Решение нашей проблемы (обнулить старшую тетраду, а младшую оставить без изменений) таким образом сводится к тому, что старшую тетраду нужно "AND 0", а младшую - "AND 1".

То есть значению DL с 00001111b (оно же - 0Fh) "AND" сделать.

На ассемблере это вот как выглядеть будет:

```
and DL,00001111b
```

Естественно, 00001111b = 0Fh

Аминь!!

#6. Уфф... Вот что получится в итоге должно:

```
;-[WRITE_HEX, V1]-----
;Печатает две шестнадцатеричные цифры
;На входе: DL - типа цифры две :))
;На выходе: нихрена
;Прерывания: ан нэту
;Процедуры: WRITE_HEX_DIGIT
;-----
WRITE_HEX proc
    push CX
    push DX
    mov DH,DL
    mov CL,4
    shr DL,CL
    call WRITE_HEX_DIGIT
    mov DL,DH
    and DL,0Fh
    call WRITE_HEX_DIGIT
    pop DX
    pop CX
    ret
WRITE_HEX endp
```

Ну че тут объяснять?? Я уже объяснил все!! Единственное, что могу добавить - это про mov DH,DL. Этой командой мы значение регистра копируем перед тем как биты "ему" сдвинуть. А потом

статус-кво mov DL,DH восстанавливаем, чтоб и младшую цифру напечатать.
Все. Тестируем. Вроде должно работать.

#7. Так сказать "к вопросу о шаблонах мышления"...

Мы тут доооолго трахались с тетрадами. Вроде успешно.

Когда при тестировании понимаемости материала мы предложили пяти "подопытным" самостоятельно написать процедуру для вывода на монитор "большого" регистра (DX), они все как один начали сдвигать байты... :(

Народ!! Это не есть правильно!!

```
;-[WRITE_HEX_WORD, V1]-----
;Печатает шестнадцатеричное слово
;На входе: DX - слово
;На выходе: нихрена
;Прерывания: ан нэту
;Процедуры: WRITE_HEX
;-----
WRITE_HEX_WORD proc
    push    DX
    xchg    DL,DH
    call    WRITE_HEX
    xchg    DL,DH
    call    WRITE_HEX
    pop     DX
    ret
WRITE_HEX_WORD endp
```

Команды xchg DL,DH и xchg DH,DL, кстати, работают абсолютно одинаково. Операнды просто меняются между собой значениями. В качестве одного из операндов может выступить память.

#8. Ну, и напоследок, - информация к размышлению:

Команду shr можно использовать для деления целочисленных операндов без знака на степени 2 :)).

```
mov     cl,4
shr     ax,cl
```

Думаете эти "фокусники" который в уме офигенны уравнения считать умеют, шибко умные?? Нифига!! Они просто люди ЗНАЮЩИЕ. А делить на десять и вы умеете...

Над этим я настоятельно рекомендую дооооолго помедитировать. А еще над тем, что девушки весьма и весьма любят, когда им фокусы показывают. Впрочем, это (вычисления в уме) - тема отдельная. Мы ее тоже когда-нить коснемся :). MATRIX MUST DIE!!

2.4. Печать десятичных циферек

#1. Для тех, кто медитировал над главой 1.1, алгоритм должен быть понятен как $2+2=$ (вписать желаемое). Кто не въедет в алгоритм WRITE_DECIMAL - научитесь сначала переводить числа между радиками на листике в клеточку.

Итак, ставим новую задачу. В DX у нас шестнадцатеричное число. Нам нужно напечатать его на монитор в "десятичном формате". Еще раз обращаю ваше внимание на то, что существует множество способов ее решения. Мы же выбрали способ, который:

а). Вам легче всего будет понять.

б). Требуем минимального количества "новых" команд.

Кто скажет, что мы не правы - пусть первый кинет камень в эхо-конференцию RTFM_Helpers.

#2. Прежде всего посмотрите на процедуру WRITE_HEX_DIGIT и вспомните, какой алгоритм положен в основу ее работы. Вспомнили? Рад за вас!!

А теперь мы познакомимся к командой деления. Что будем делить?? Ну естественно, "регистры" :).

Новая команда называется "деление беззнаковое" (DIVide unsigned).

Что такое делимое/делитель/частное, я вас грузить не буду, это в учебнике арифметики для младших классов более чем понятно расжевано. Ну во всяком случае детишки это понимают.

(Кто скажет, что $10/3=3$ а остаток 333 в периоде - тот дурак. Кто скажет, что остаток будет равен 1, скажет правильно.)

Следующий кусок кода демонстрирует деление значения регистра AX на значение регистра BL.

```
mov ax,10d
mov bl,3d
div bl
```

Обратите внимание: ДЕЛИМОЕ у нас может располагаться только в регистре AX (другими словами, делимое задается неявно), а ДЕЛИТЕЛЬ - в любом регистре.

Кто не верит - посмотрите под отладчиком...

1. Если делитель размером в байт, то после операции частное помещается в AL, а остаток - в AH.
2. Если делитель размером в слово, то делимое должно быть расположено в паре регистров DX:AX (младшая часть делимого в AX). После операции частное помещается в AX, а остаток - в DX.
2. Если делитель размером в двойное слово, то делимое должно быть расположено в паре регистров EDX:EAX (младшая часть делимого находится в EAX.) После операции частное помещается в EAX, а остаток - в EDX.

Внимание, подводный камень! В следующем примере:

```
mov ax,10d
mov bx,3d
div bx
```

у вас вовсе не AX на BX делиться будет, а парочка DX:AX на BX.

Помедитируйте над этим :))

#3. А теперь, собственно, пропиваю саму процедуру вместе с традиционным расжевыванием оной:

```
;-[write_decimal, v1]-----
;печатает десятичное беззнаковое число
;на входе: dx - типа число
;на выходе: ни хрена
;прерывания: ан нэту
;процедуры: write_hex_digit
;-----
write_decimal proc
    push ax
    push cx
    push dx
    push bx
    mov ax,dx ;(1)
    mov bx,10d ;(2)
    xor cx,cx ;(3)
non_zero:
    xor dx,dx ;(4)
    div bx ;(5)
    push dx ;(6)
    inc cx ;(7)
    cmp ax,0 ;(8)
    jne non_zero
write_digit_loop:
    pop dx ;(9)
    call write_hex_digit ;(10)
    loop write_digit_loop
    pop bx
    pop dx
    pop cx
    pop ax
    ret
write_decimal endp
```

Алгоритм простой: пока частное не равно 0, делим его, делим и еще раз делим на 10d, запихивая остатки в стек. Потом - извлекаем из стека. Вот и вся "конвертация" из HEX в BIN :). Это если в двух словах.

А если подробно, то вот что получается:

Бряк 1 - подготавливаем делимое. Как уже говорилось, оно у нас задается неявно - обязательно через AX. А параметр у нас - через DX процедуре передается. Вот и перемещаем.

Бряк 2 - это, собственно, делитель.

Бряк 3 - очищаем CX. Он у нас будет в качестве счетчика. О нем мы еще поговорим.

Бряк 4 - очищаем DX. Если не очистим, то мы не 1234h какое-нить на 10 делить будем, а 12341234h. Первое 1234 нам надо? Вот и я говорю - очищаем!

Бряк 5 - делим! Частное - в AX, остаток - в DX.

Бряк 6 - заносим остаток (DX) в стек ;).

Бряк 7 - CX=CX+1. Это мы считаем сколько раз "щемили остаток", который "щемятся", по кругу (прыжок на метку pop_zero), пока AX не равно 0 (бряк 8). То есть делим, делим AX, пока он не окажется таким, что делить, собственно, нечего.

Так-с... Деление закончено, число раз, которое мы поделили AX до его полного обнуления, хранится в CX.

Дальше все просто. Нам нужно такое же количество раз (CX) извлечь значение (DX) из стека. И это будет "HEX", переведенный в DEC. (Оно же: число в двоичном коде, разобранный на последовательность десятичных цифр).

Помните, как у нас организуется цикл? Через loop и CX?

Если бы в качестве счетчика мы использовали какой-нибудь другой регистр, то пришлось бы извращаться со всякими метками и прыжками... а так все просто, все продуманно :). Цикл, в теле которого ИЗВЛЕЧЬ ЦИФЕРКУ (бряк 9) и НАПЕЧАТАТЬ ЦИФЕРКУ (бряк 10). Столько же раз, сколько мы и делили наше исходное шестнадцатеричное число.

Для тех, кто не понял: бряк - это брекпоинт. Для тех, кто еще не знает, что такое брекпоинт - ищите объяснение в 'DZebug. Руководство юЗверя'

#4. Тестируем!!

```
testing proc
    mov     dx,12345d
    call    write_decimal
    call    exit_com
testing endp
```

Двое из десяти "подопытных" чайников (есть такие) возмутились:

- В DX же только четыре цифры влезают!

- Ага! Аж два раза четыре!

```
mov     dx,'DZ'
```

Как видите, туда еще и две буквы "влезают" ;)

А то и все четыре, если кавычки считать...

Медитируйте!!

2.5. Hello, world! или Изврат-2

#1. Мы уже писали программу "Hello, World!" с использованием 13-й функции 10 прерывания. Посмотрите на ее исходник...

Сегодня мы слабаем еще одно "Hello, World!", но уже несолько другим способом. Какой из этих способов более дЗенский - решайте сами ;).

Для начала мы создадим блок данных после всех-всех-всех процедур но между директивами начала и конца сегмента.

Блок данных будет выглядеть следующим образом:

```
abc db 'Hello, World-2$'
```

Вы должны спросить "А почему мы хотим напечатать Hello, World-2, а в конце строки у нас 2\$? \$ - это что? World за баксы продавать, штоль?? Да ну вас..."

Эту строчку мы будем выводить на монитор особым дЗенским способом - посимвольно. То есть: возьмем первый символ из блока данных, выведем, потом второй и т. д. аналогично пока не встретим символ '\$'.

Опять-таки, это если в двух словах. Но ведь наверняка вам этого покажется мало ;).

"Щемить" символы мы будем двумя способами. (Тут я хотел было написать "неправильным" и "правильным", но потом передумал и решил обозвать их "первым" и "вторым").

В алгоритм первого способа вы и сами без труда въедете (я только "рабочую часть" приведу, пуши с попами сами проставляйте):

```
...
next:
  mov dl,[BX]
  cmp dl,'$'
  je finish
  call write_char
  inc BX
  jmp next
finish:
...
```

#2. А вот второй способ немножко навороченнее :). Чтобы в нем разобраться, мы сначала познакомимся со следующей группой команд: LODSB, LODSW, LODSD

Их назначение - это загрузка элемента из последовательности (строки, цепочки) в регистр-аккумулятор al/ax/eax.

Для тех, кто не понял - наша строчка "Hello, World-2\$" как раз и является "последовательностью/строкой/цепочкой" из элементов размером в байт.

Адрес цепочки передается через ds:esi/si, сами "элементы" (фиксированной ширины) возвращаются в al (байт, команда LODSB), ax (слово, команда LODSW) или eax (двойное слово, команда LODSD). В общем, последние буквы команд как раз и указывают на размерность элемента: [B]yte, [W]ord, [D]ouble word. Т. е. размер мы определяем неявно.

После выполнения одной из этих команд значение регистра si изменяется на величину, равную длине элемента, но... хм...

Пришло время еще одну большую тайну познать, братья. В справочнике Юрова написано, "знак этой величины зависит от состояния флага df:

df=0 — значение положительное, то есть просмотр от начала цепочки к ее концу;

df=1 — значение отрицательное, то есть просмотр от конца цепочки к ее началу."

Обидно, да? Флаги-то мы с вами еще не расколупали... Чего-ж дальше-то делать, а?

Как что? "Пропивать" стандартную процедуру и колупать ее, колупать, колупать, ногами ее, ногами, и по морде, по морде, по морде...

```
;-[write_string, v1]-----
;печать строки символов на мониторе.
;(Строчка оканчивается символом '$'
;на входе: ds:dx - адрес строки
;на выходе: нихрена
;прерывания: ан нату
;процедуры: write_char
;-----
write_string proc
  push ax
  push dx
  push si
  pushf                ;(1)
  cld                  ;(2)
  mov si,dx            ;(3)
string_loop:
  lodsb                ;(4)
  cmp al,'$'           ;(5)
  jz end_of_string    ;(6)
```

```

mov dl,al      ;(7)
call write_char ;(8)
jmp string_loop ;(9)
end_of_string:
popf           ;(10)
pop si
pop dx
pop ax
ret
write_string endp

```

Итак, что делает команда `lodsb`, вы уже поняли. А вот с `df=0/df=1` пока что непонятки. Будем разбираться.

Нам нужно, чтобы "просмотр цепочки" осуществлялся командой `lodsb` слева направо, для этого нужно установить значение `df=0`. Делаем мы это при помощи команды `cld` (бряк 2).

Если нам нужно, чтобы "просмотр цепочки" осуществлялся справа налево, мы используем команду `std`. (Можете попробовать. Ерунда получится.)

А теперь вспомним "золотое правило". Все регистры, которые мы изменяли ВНУТРИ процедуры, "на выходе" должны восстанавливать свои ПРЕДЫДУЩИЕ значения (кроме тех регистров, через которые мы возвращаем РЕЗУЛЬТАТ).

Так вот: то же самое касается и флагов, которые мы изменяем.

Для регистров мы использовали команды `PUSH` и `POP`. Для регистра флагов (изменять у которого мы можем только БИТЫ) используются команды `pushf` (записать в стек значения флагов, бряк 1) и `popf` (извлечь из стека, бряк 10).

ПРИМЕЧАНИЕ: Это несколько вольное положение (хотя, вообще-то, верное). Видимо, здесь следует хотя бы добавить, что сохранять/восстанавливать флаги нужно при изменении специальных флагов (`if`, `df`). (C) Хемуль

Теперь колупаем дальше... (C) Serrgio

Бряк 3. Мне удобнее передавать данные "в процедуру" через регистр `DX`, о чем и написано в заголовке. А `lodsb` (бряк 4) хочет, чтоб адрес ему в `SI` подавали. Удовлетворим его желание :)

Бряк 4. После выполнения этой команды ASCII-код первого символа "цепочки" "Hello, World-2\$" помещается в `AL`, а значение регистра `SI` увеличивается на 1 и указывает теперь на второй элемент цепочки (символ 'е').

Бряк 7. Удовлетворяем "пожелания" процедуры `write_char`. Из `AL` переносим в `DL` и печатаем (бряк 9) `write_char`ом.

Тэкс... Мы пропустили бряки 5 и 6-й...

Смотрите: на бряке 9 мы "безусловно" зацикливаем "извлечение" и печать элементов цепочки. Безусловно - это значит до потери пульса. Чтоб этого не произошло, каждый из элементов цепочки мы сравниваем с символом-конца-цепочки (в нашем случае это '\$', но можно использовать и любой другой). Если текущий символ равен символу-конца-цепочки, то выпрыгиваем (бряк 6) из этого безусловного цикла, восстанавливаем статус-кво (бряк 10) и все на этом...

#3. Тестируем!!

```

testing proc
    lea dx,abc
    call write_string
    call exit_com
testing endp

```

Напечаталось то, что надо?

Если true - читайте дальше.

Если false - расколупывайте и медитируйте до полного просветления...

Диагноз

Когда-то автор этого текста хотел осветить все вопросы низкоуровневого программирования, начиная от процессора 8086 до p3, от DOS'а до W2000 и линуха. Однако, практика показала, что при таких черепаших темпах написания... к тому времени когда дойдет очередь хотя бы до w3.11 - windows-2000 уже станет вчерашним днем :(. Поэтому этот tutorial, мягко говоря, не совсем завершен ;).

Win32ASM: Минимальное приложение

Автор: Serrgio / HI-TECH

Дата: 2002-08-17

Как я и обещал, мы с вами займемся программированием под win32. Выполняю свое обещание. Хотя, честно говоря, нижеследующий кусок текста мне уже совсем не нравится. То есть сначала он мне понравился, но потом некоторые товарищи, чье мнение я весьма уважаю и ценю, его раскритиковали в пух и перья...

А вообще-то в этом тексте дофига плагиата.

Portion by svet(R)off

#1. Однажды студент по имени Дениска Ричи сел изучать абсолютно новый для него язык программирования. Первое, что он сделал - написал программу "Hello, World", и, что самое удивительное, она у него заработала. Когда же он перелистнул следующую страницу своей умной книжки, то ни черта не понял. То есть задним умом своей головы он, конечно же, понимал, что большинство известных ему программ умеют намного больше, чем тупо приветствовать мир, однако понять, каким это извращенным образом такие здоровские штуки можно запрограммировать, он не мог.

Однако ум Дениска имел пытливый (в школе его часто драли в раздевалке - всех гениев в детстве драли в раздевалке!) и сразу же просек, что если в природе существуют программы, умеющие больше этого, то наверняка должны быть и программы, умеющие меньше.

И тогда Дениска, опасаясь быть сначала проигнорированным, потом замодерированным, и в конце концов посланным, забросил свой вопрос в одну из эх, приготовившись, если чего вдруг не так, притвориться кроликом и начать жевать морковку.

Последующие события подняли дыбом все волосы на его тщедушном тельце. Со всех концов бескрайней саванны мгновенно слетелись, сбежались (а некоторые даже и сползли) бесчисленные гуру, сенсеи и йоды. Возникла крутейшая разборка, поднявшая большое облако пыли, из которого долго доносились рыки, ржание, визги и предсмертные хрипы, а во все стороны разлетались зубы, перья и клоки шерсти... Сначала великие считали и пересчитывали строчки в исходниках, потом буквочки в строчках, а потом даже и байтики в экзешниках, при этом постоянно гоняя получившиеся цифры между двоичным, десятичным и шестнадцатеричным радиками...

Страшна и беспощадна была эта битва, пока, наконец, из-под обломков не выполз только что пришедший в себя Бьярни Страус (весь помятый и бледный, как труп; те, кто видели, так и начали его с тех пор обзывать - Страус-труп) и не предложил подсчитать биты, установленные в 1...

И тогда вопли стихли, ибо на всех вдруг нашло прозрение, по какой это такой причине двери в психушках открываются исключительно вовнутрь.

Вывалив языки и тяжело дыша, великие расползлись по своим кельям зализывать раны и готовиться к грядущим битвам, а посередине вытопанной поляны, одинокий, жалкий и забытый, остался остывать трупик несчастного кролика...

#2. Между тем, вопрос о минимальном приложении - это вам не "кто идет за "Клинским" (конечно же, тот, кого в прошлый раз от "Балтики-медового" мутило), и даже не тот, "кто потом пойдет сдавать бутылки" (конечно же тот, кого систематически дерут в раздевалке!). Минимальное приложение - это есмь альфа и омега программирования, сцилла и харибда отладки, инь и янь сопровождения и даже (давайте не будем бояться этой правды жизни!) эпос и анус раскрутки программного продукта! И именно поэтому за много-много лет никто так и не усомнился в правоте Кнута, который утверждал, что если:

$$f(\sigma, j) = (\sigma, a_j)$$

(где θ_j - это программа, а σ - операционная система)

то бессильна даже банка со сметаной, и все, что в этом случае остается - брать в руки биту и идти в лес, выгонять из дубовых дупел пчел.

Как свидетельствует вышеозначенная формула, вопрос о минимальном приложении (речь идет именно о приложениях, то бишь applications, а вовсе не о каких-нибудь там VxD) - совсем не простейший, и представляет отнюдь не академический интерес. Даже прибалдевшему от морковки кролику должно быть очевидно, что любое приложение обязано выполнить как минимум две не такие уж и простые задачи:

- а) **стартовать** , получив при этом нормальный доступ к ресурсам рабочей среды,
- и
- б) **завершиться** , оставив эту рабочую среду работоспособной.

В языках высокого уровня подобные процессы обставлены всяческим сервисом - тут тебе и табун девок, накачанных силиконом, и водитель, готовый хоть на руках тащить автомобиль на стоянку, и услужливый швейцар, который при необходимости занесет на повороте хвост, если таковой, конечно же, имеется, и крашеный негр Ванька-каин, который в любое время суток до блеска готов отполировать ботинки...

Программа же на ассемблере - это дитя совершенно иных, воистину спартанских условий. Лишенный всего вышеперечисленного, ассемблер, с одной стороны, дает программисту абсолютную власть над компьютером, но с другой... Если вы вдруг достанете свою ракетницу и возжелаете сделать ракетджемп в надежде допрыгнуть до мегахелса, то служба охраны отнюдь не кинется прикрывать своими бронированными телами остальных жильцов пятизвездочной с двумя плюсами гостиницы!

Ракетница - одно из самых крутых оружий в Quake. Ракетджемп - это такой прыжок, когда из ракетницы (в прыжке!) стреляешь себе под ноги и из-за этого очень высоко подпрыгиваешь, при этом сжигая определенное количество пунктов здоровья. Мегахелс, однако, тебе их потом с лихвой восстанавливает. Quake - рулез! Каждый низкоуровневый программист должен любить Кваку!!)

Практически это означает то, что вы берете в руки крутейший инструмент, который не имеет абсолютно никакой защиты от дурака. И если какой-нибудь дельфийский компилятор будет некоторое время удивляться вашим попыткам выстрелить себе в ногу, (а потом возьмет и выстрелит в ногу сам!) то ассемблерный не скажет ни слова, полагая, что имеет дело с профессиональным террористом, который знает что делает...

А раз любвеобильной мамочки рядом с нами не будет, то прежде чем взять в руки это мощнейшее "оружие", надо сначала сходить в аптеку и закупиться не только зеленкой и йодом, но и всевозможными средствами контрацепции. И только тогда, в полном вооружении, брать в руки черный с черепушкой флаг, вешать на шею барабан и отправляться в густой таежный лес, учиться делать ракетджемп и брать медведя голыми руками...

#3. Итак, с криком "банзай" (восточная школа) либо "мастдай" (западная школа), открываем текстовый редактор и набираем там исходный текст (в обиходе называется сырец) минимального приложения на языке ассемблера.

```
;Сырец 1. Минимальное приложение на Assembler'e (minimal.asm)
```

```
.386
.model flat,stdcall
```



```

includelib kernel32.lib

ExitProcess PROTO :DWORD

.code

WinMain PROC
;...
push 0
call ExitProcess
WinMain ENDP

end WinMain

```

Вот она, эта песнь души измученного кролика, из которой, как говорится, ни строчки вырезать, ни матерного слова добавить. Однако, это всего лишь своего рода нотный стан с нарисованным на нем скрипичным ключом, размером и нотами. А разве многие из нас могут наслаждаться музыкой, нарисованной на бумаге? Вот именно! Поэтому давайте сначала загрузим эту партитурку в мясорубку, покрутим ручку и посмотрим, какая бяка появится у нас на выходе.

Мясорубкой мы будем пользоваться специальной, винدوزовской. Называется она MASM32, версию имеет 7.0, весит 5 Мб, а скачать ее можно [здесь...](#)

Как говорится, клик хере фор доунлоад, а пока вы будете эту мясорубку скачивать и привинчивать к столу своей камеры пыток, я прочту вам некоторые строки из прилагаемого к ней буклетика.

#4. Фирма Microsoft никогда не документировала, каким это извращенным способом можно писать приложения под Win32 на Ассемблере. Конечно же, в ее Driver Development Kit (DDK) есть парочка соответствующих топигов, но они относятся исключительно к разработке виртуальных драйверов VxD и прочих специфических штук. Ни для кого не секрет, что для программирования в среде Windows необходимо иметь многочисленные ссылки на данные, такие как прототипы функций, структуры, типы и определения констант, макросы... и так далее. Так вот, все вышеперечисленное богатство как раз и поставляется с DDK. Однако, вот беда, "заточено"-то оно под Си :).

Естественно, маргинальная часть программирующего сообщества неоднократно задавала мелкософту вопрос на тему "assembly language programming for Win32". Но все, что могла ответить служба поддержки - это "assembly language programming for Win32 is not supported by Microsoft" - на первый бесплатный звонок, и "no, it cannot be done" - на второй. Третий же звонок уже стоил немалых денег, и находились лишь считанные единицы, решавшиеся на это.

Однако, как говорится, если гора не идет к Магомету, то Магомет идет к горе. Осознав простую народную мудрость, что "спасение утопающих - дело рук самих утопающих", некто Hutch (потом ему начала помогать целая команда во главе с Iczelion'ом), взял все вышеозначенное "богатство" и начал "перезатачивать" его под Ассемблер...

Подозреваю, что это был весьма тяжелый и тернистый путь, потребовавший от Hutch'a как огромных моральных усилий, так и определенных материальных затрат. Однако он был успешно выполнен, в результате чего мы с вами имеем огромнейшую халяву и собираемся самым бессовестным образом сберечь на этом большое число своих драгоценнейших нервных клеток ;).

Не правда ли, последнее радует? Но, тем не менее (а как же без ложки дегтя?), необходимо учесть и тот простой факт, что пакет MASM32 делался живыми людьми, и в нем могут содержаться (и они там действительно есть!) ошибки. Поэтому отнюдь не стоит брезговать время от времени проверять соответствующие заголовочные файлы ;).

Вот такая вот предыстория. Надеюсь вы уже скачали MASM32 и установили его на свой жесткий диск? Тогда давайте вернемся к нашим баранам :).

#5. Делать EXE'шник, как всегда, мы будем в два этапа. Для начала возьмем исходник minimal.asm и "заведем" нашу мясорубку следующей командной строкой (каталог с исходником должен быть текущим):

```

c:\masm32\bin\ml /c /coff minimal.asm

```

Ключ /с говорит о том, что мы хотим только оттранслировать исходный файл, но не компоновать. Ключ /coff означает, что мы хотим создать объектный файл в формате COFF (Common Object File Format), стандартном для "Окон" формате объектных файлов (TASM, в отличие от MASM, создает объектные файлы в интеловском формате OMF (Object Module Format), который мелкосовфтовский линкер запросто конвертирует в COFF).

Теперь, если вы правильно набрали исходник, в той же папке, что и minimal.asm, должен появиться объектный файл minimal.obj.

Далее приступим к компоновке экзешника. Затаиваем дыхание и дергаем за стартер:

```
c:\masm32\bin\link /SUBSYSTEM:WINDOWS /LIBPATH:c:\masm32\lib minimal.obj
```

Ключ /SUBSYSTEM с параметром WINDOWS говорит линкеру о том, что мы собираемся собрать экзешник для подсистемы Windows. Другие возможные значения: CONSOLE - если мы собираемся делать программу с текстовым интерфейсом а-ля ДОС, NATIVE - если драйвер устройства, и POSIX - если мы собираемся писать программы, ориентированные на стандарты POSIX и более или менее переносимые под разные Юниксы (для которых эти стандарты и писались).

Ключ /LIBPATH указывает путь к библиотекам импорта, которые нам обязательно необходимо прилинковать к программе, если мы хотим использовать возможности, предоставляемые не только процессором, но и операционной системой. Подробнее о том, что из себя представляют библиотеки импорта, мы поговорим чуть позже.

Если вы все сделали правильно, то в нашей папке, наряду с исходным и объектным, должен появиться еще и третий файл - исполнимый. Возрадуемся же этому и перейдем к следующему пункту повестки дня - разборке исходника. Неужели вам не интересно знать, что означают эти загадочные строчки в minimal.asm?

#6..386 - это директива ассемблера, определяющая набор инструкций процессора, которые могут быть использованы в программе (позже мы проведем четкую границу между инструкциями, директивами и командами). По умолчанию транслятор полагает, что программа пишется для процессора 8086 и сопроцессора 8087. Но, посудите сами, какая под него может быть Винда? Для приложений win32 необходимо указывать либо .386, либо выше (486, 586, 686) - в зависимости от того, собираетесь ли вы использовать возможности, предоставляемые процессорами/сопроцессорами последующих поколений или нет. Впоследствии мы будем также использовать r-версию этой директивы (486r, 586r), что даст нам доступ к страшным и ужасным привилегированным командам, которые нехороший "дядька" Микрософт подмял под себя, и выпросить их у "мелкомягких" будет не так уж и просто. **#7. .model flat, stdcall**. Первый параметр - это модель памяти. Например, COM'овские программки, которые мы с вами раньше писали, соответствовали модели tiny, то есть "крошечной" - они запросто помещались в 64 Кб памяти, и никаких проблем с адресацией данных у нас не было. Однако если бы мы захотели написать под ДОС программу во много раз большую, чем 64 Кб, т.е. превышающую размер сегмента, нам пришлось бы познакомиться с таким динозавром, как оффсетно-сегментная адресация, и использовать, в зависимости от навороченности программы, модели small, medium, compact или large. Под Windows же у нас есть одна-единственная "правильная" модель памяти - flat, то бишь "плоская", позволяющая нашей программе благодаря страничной адресации легко и просто работать с 4 Гб виртуальной несегментированной памяти. И это есть хорошо! Ибо теперь, как сказал некто Вал.Ик., нам не нужно смотреть на мир сквозь замочную скважину 64 Кб-сегмента. (С) Второй же параметр указывает на так называемое соглашение о вызове процедур. Каждый язык имеет свои "соглашения". Так, второй параметр может принимать значения: c, basic, fortran, pascal... Мы не будем вдаваться в особенности каждого, просто скажем, что при программировании под win32 на макроассемблере нужно использовать соглашение stdcall, ведущее свою родословную в части наименования] функций - от языка C (плодовитый, однако, мужчина!), а в части передачи аргументов - от языка Pascal (курица, как говорится, не птица, но яйца мы предпочитаем куриные). Подробнее об этом соглашении будет рассказано ниже.

#8. `.includelib kernel32.lib` - эта директива передается компоновщику и сообщает, что наша программа должна быть слинкована с указанной библиотекой, в данном случае с `kernel32.lib`. Библиотека представляет собой энное количество готовых к употреблению процедур, оттранслированных в объектные файлы и собранных в большую кучу под названием "библиотека". Если помните, есть такое понятие как программное прерывание, то бишь механизм, при помощи которого можно обратиться за обслуживанием своих запросов к операционной системе. Так вот, в среде Windows пользовательских прерываний как таковых нет, а задействование ресурсов операционной системы производится совершенно иначе - при помощи так называемых функций Windows API. А для того, чтобы получить доступ к этим ресурсам, вы должны обязательно слинковать свою программу с соответствующей библиотекой импорта, в которой как раз и находится куча маленьких подпрограммок, собственно, эти функции и вызывающих. Слово "слинковать" выделено не случайно. Это связано со следующей строчкой нашего исходника.

#9. Посмотрите внимательно на тело `WinMain`. Там есть команда `call`, то есть вызов подпрограммы. А где, спрашивается, сама подпрограмма `ExitProcess`? Ан нету ее! А почему тогда, спрашивается, транслятор на это не ругается? А вы удалите строчку `ExitProcess PROTO :DWORD` и попробуйте оттранслировать свой исходник... Ага, хорошо знакомое: "undefined symbol : ExitProcess"? Еще бы, такой подпрограммы в нашем исходнике действительно нет!

Посмотрите на `call ExitProcess` - это вызов API'шной функции, завершающей работу нашей программы (виндозный аналог хорошо нам известного `INT 20h`). Именно подпрограмма с таким именем будет прилинкована к нашему экзешнику, и уж она-то знает способ, как ей обратиться к DLL-ке `kernel32.dll` и заставить ее прервать работу нашей программы! Еще раз повторюсь, прилинкуется не сама API'шная функция, а только подпрограммка, которая умеет эту внешнюю функцию вызывать.

Но это на этапе линковки, когда соединяются все "концы" между объектными файлами. А на этапе транслирования ассемблер не имеет никакого понятия о том, что в природе существует какой-то `ExitProcess`... И чтобы он (транслятор) не ругался, мол, "кто такой, почему не знаю", нужно их "познакомить" - при помощи так называемого прототипа. Другими словами, прототип - это своего рода уведомление транслятора о том, что "немного попозже я собираюсь обращаться к товарищу `ExitProcess`, и форма вызова для него такая-то". Ну а далее следуют "паспортные данные" на этого товарища - сколько он "переваривает" параметров, и какой они должны быть размерности, дабы тот, не дай бог, не подавился.

Вообще-то говоря, существует несколько способов подобного "уведомления", однако вышеприведенный, несмотря на свою кажущуюся навороченность, является самым эффективным, так как позволяет проверять соответствие количества и типа аргументов параметрам, тем самым помогая отслеживать целый ряд трудноуловимых ошибок.

#10. Как известно, каждая программа состоит из кода и данных. Раньше мы их называли сегментами - это было связано с тем, что из-за сегментной адресации приходилось мерить мир "спичками" по 64 Кб. Теперь же мы, слава Богу, имеем единое адресное пространство, и термин сегмент, во избежание терминологической путаницы наподобие "говорим сегмент, подразумеваем 64 Кб", заменим на термин "секция", хотя, по большому счету, это одно и то же.

Определяются эти секции, а-ля сегменты, следующими упрощенными директивами:

- `.data` - определяет секцию инициализированных данных;
- `.data?` - определяет секцию неинициализированных данных, для тех случаев, когда необходимо предварительно выделить определенное количество памяти, но инициализировать ее заранее нет необходимости. Фишка заключается вот в чем - сколько бы мы ни определяли неинициализированных элементов данных, размер файла программы на диске остается неизменным. Мы просто таким образом ставим систему в известность: "когда программа загрузится, я хочу иметь в своем распоряжении такой-то объем памяти";
- `.const` - определяет секцию констант, то есть элементов данных, которые наша программа не сможет (во всяком случае, не должна) изменять ни в коем разе;

- .code - собственно код, то есть последовательность инструкций, которые должен выполнить твой компьютер.

Из всех вышеперечисленных секций в нашем исходнике есть только CODE. Это единственная секция, которая обязательно должна присутствовать в любой программе. А разве может быть иначе?

#11. В секции кода у нас есть одна-единственная процедура. На всякий случай напомним, что о ее начале свидетельствует строка WinMain PROC, где WinMain - это "имя собственное", которое вы можете заменить любым словом, в том числе и своим любимым. А ее конец - это строка WinMain ENDP. Кстати, настоящим ассемблерщикам должно быть приятно, что, опустив ненужную в связи с этим команду ret, мы сэкономили аж 4 байта кода!

О конце модуля транслятору сообщает директива end, и в качестве параметра у нее метка, на которую будет передано управление при старте программы. В данном случае - это имя "главной", а в нашем случае еще и единственной, процедуры WinMain.

;... - этот комментарий поставлен на месте всякой прочей мелочи, которую вы хотели бы заставить делать ваше приложение в тот примечательный момент, когда решите сделать его немного большим, чем минимальное.

#12. Как я уже говорил в п.7, вызов функций API из программы на ассемблере подчиняется соглашению "stdcall". Выражаясь официально, "с точки зрения прикладного программиста, с учетом специфики Windows и MASM", эти соглашения заключаются в следующем:

1. Регистр символов в имени функции не имеет значения. Например, функции с именами WindowsMustDie и windowsmustdie - это одна и та же функция. Здесь мы наблюдаем отличие от требования языков C и C++, в которых идентификаторы регистро-зависимы. Зато в Паскале регистр имен не имеет значения.

2. Аргументы передаются вызываемой функции через стек. Если аргумент укладывается в 32-битное значение и не подлежит модификации, вызываемой функцией, он обычно записывается в стек непосредственно. В остальных случаях программист должен разместить значение аргумента в памяти, а в стек записать 32-битный указатель на него. Таким образом, все параметры, передаваемые функции API, представляются 32-битными величинами.

3. Вызывающая программа загружает аргументы в стек последовательно, начиная с последнего, указанного в описании функции, и кончая первым. После загрузки всех аргументов программа вызывает функцию командой call.

4. За возвращение стека в исходное состояние после возврата из функции API отвечает сама вызываемая функция. Программисту заботиться о восстановлении указателя стека esp нет необходимости. Эта идея тоже пришла из реализаций Паскаля и прилично экономит на размере кода.

5. Вызываемая функция API гарантированно сохраняет регистры общего назначения ebp, esi, edi. Регистр eax, как правило, содержит возвращаемое значение. Состояние остальных регистров после возврата из функции API следует считать неопределенным (полный набор соглашений stdcall регламентирует также сохранение системных регистров ds и ss. Однако, для flat-модели памяти, используемой в win32, эти регистры значения не имеют.)

В применении к нашему примеру это означает следующее:

- аргументы мы должны передать через стек;
- сама функция вызывается командой call;
- не "парим" себе мозги, задаваясь вопросом, "если мы сделали PUSH, то когда же нам сделать POP?", ибо правильный ответ - никогда. Об этом позаботится сама функция.

#13. И, напоследок, топик #13 ;)

Читая эту главу, вы написали свою первую программу для Windows на языке ассемблера. Если вы уже имеете хотя бы небольшой опыт программирования, то даже беглого прочтения этой главы будет достаточно, чтобы сказать: "да, теперь я знаю, как ЭТО делается на асме". Если же вы еще

совсем молоды и зелены, то просто "тупо" выполните мои инструкции, а комментарии расцените как своего рода предварительные наброски тех тем, которые мы подробно будем рассматривать впоследствии.

Более того, это единственная глава, которая действительно необходима для обучения программированию под Windows на ассемблере. Как только программист узнает, как вызывать API, все остальное он станет способен делать самостоятельно, без учебников и подсказок. Потребуется только справочник Platform SDK, да знание двух языков: С и английского (оба - в объеме церковно-приходской школы).

И да пребудет с вами сила!

Win32ASM: Форматы данных от лукавого

Автор: Serrgio / HI-TECH

Дата: 2002-08-19

#1. Как известно, объем жидкости измеряется в см³ ;). Но когда мы покупаем, например, пиво, то измеряем его вовсе не в кубических сантиметрах, а в совершенно других единицах измерения: бутылках, банках, ящиках, канистрах, бокалах или, на худой конец, в литрах. Точно так и информация: с одной стороны, она, конечно же, измеряется битами, байтами, килобайтами и т.д., но с другой стороны - существуют и её элементарные "потребительские куски". Как пиво мы пьем глотками, так и процессор потребляет ее своими небольшими "глоточками" ;).

Ранее мы уже разбирались что такое регистры и какими кусками они могут держать в себе информацию. Однако со времен процессора 8086 прошло очень много времени, и регистры немного подросли. Поэтому мы для начала познакомимся с *современным* вариантом "нездоровой" схемки из главы 1.2 **#2** (Ведение в машинный код), и только потом поговорим о данных.

Типичный **регистр общего назначения** выглядит следующим образом:

EAX

AX

AH — AL

Что в нем изменилось? Да просто его (регистра) стало в два раза больше и максимальное число, которое мы можем в этот регистр записать - это FFFFFFFFh (4294967295d, 11111111111111111111111111111111b). Однако обратите внимание (см. рисунок), что EAX - это все лишь **расширенная версия** регистра AX. То есть мы не можем обратиться к "старшей половине" EAX, как мы делали это, например, при обращении к частям AX через "половинки" AH|AL. Другими словами, если **mov AX,1234h** и парочка **mov AH,12h ; mov AL,34h** - это один чёрт, то про **mov EAX,12345678h** ничего похожего мы сказать не можем.

"Неделимые" регистры SI, DI, SP и BP также имеют свои расширенные версии (ESI, EDI, ESP и EBP), а вот сегментные регистры CS, SS, DS, ES, GS и FS сия участь минула, так как парочек наподобие ES:EBX (сегмент:смещение) вполне достаточно и сейчас. Заметим: сегментные регистры по-прежнему задействованы во всех инструкциях доступа к памяти (например **mov ax,[var]** всегда идёт через DS, а **call [func-var]** использует оба DS и CS), однако при программировании под Win32 об этом обычно можно не беспокоиться.

_Повторюсь опять и опять - если простеньким прожкам под Win32 сегментные регистры явно использовать нет надобности, то это вовсе не значит, что они перестали использоваться процессором при адресации данных в "сегменте данных", "на стеке" и в строковых инструкциях. Не говоря уже про явное использование сегментных регистров в сУрьёзных программах ;)

На первых порах мы будем "замалчивать" существование сегментных регистров, но знайте: утверждение об их "устарелости" есмь брехня._

Однако, вернемся к нашим баранам ;)

#2. Для описания простейших типов данных на языке ассемблера используются так называемые **директивы резервирования и инициализации** данных, которые по сути являются указаниями **транслятору** на выделение определённого объёма памяти.

Например, мы неоднократно использовали директиву **DB** (*define byte*), которая инициализировала столько байт, сколько нашей душеньке было угодно. Например, **DB 'Hello, World'** "резервировала и инициализировала" столько байт, сколько символов нам вздумалось написать между кавычками

или апострофами.

Однако помимо DB существует еще куча подобных директив. Я приведу вам их все. Некоторые из них вас ужаснут, однако расслабьтесь - вовсе не обязательно что вы будете все их использовать ;). Но ознакомиться с ними - крайне желательно желательного ;) Подозреваю, что впоследствии мы вернемся к этой главе еще не раз.

Для начала мы "построим" эти директивы согласно размера памяти, который они резервируют. А заодно с трех сторон посмотрим на диапазон значений, которые эти "элементы данных" могут принимать.

Расшифровывается эта табличка следующим образом: первая колонка - это сама инструкция. Первая буква - D - от буржуйского *define*, то есть "определить", вторая - B, W, D и т. д. - от размера определяемого элемента данных, то есть *byte* (байт), *word* (слово), *doubleword* (двойное слово), и т. д. Третья колонка - размер в байтах. Четвертая - диапазон принимаемых значений в шестнадцатеричной системе счисления (хе... я понимаю, что последние две строчки несколько длинноваты ;), однако вроде никто еще не придумал писать шестнадцатеричные числа в экспоненциальном виде). Пятая колонка - это соответствующий диапазон десятичных беззнаковых чисел, шестой - знаковых десятичных.

DB	байт, byte	1	0...FFh	0...255	-128...+127
DW	слово, word	2	0...FFFFh	0...65 535	-32768...+32767
DD	двойное слово, double word	4	0...FFFFFFFFh	0...4294967295	-2147483 648...+2147483647
DF	указатель дальнего слова, far word	6	0...FFFFFFFFFFFFh	0...248-1	-247...+247-1
DQ	учетверенное слово, quadword	8	0...FFFFFFFFFFFFFFFFh	0...264-1	-263...263-1
DT	10 байт, ten bytes	10	0...FFFFFFFFFFFFFFFFFFFFh	0...280-1	-279...279-1

Во-первых, обратите внимание на DF - это как раз случай *длинного сегментированного указателя* (32 бит смещения + 16 бит сегмента). И для него, кстати, есть синоним: DP. Во-вторых, помимо целых чисел как аргументы можно указывать *нецелые*. DD может использоваться для описания и хранения *коротких* (single-precision) нецелых чисел $\pm 10^{-38} \dots 10^{38}$, DQ для длинных (long, double) нецелых $\pm 10^{-308} \dots 10^{308}$, а DT для "*временных*" (temporary) нецелых (диапазон не помню). Например:

```
dd 3.141
dq 1.2345678987654321
dt 0.0000000001
```

Вообще, если быть совсем точным, то помимо интерпретации значений в соответствующих ячейках как целых, они могут интерпретироваться следующим образом: DB - байт или знак; DW - int/unsigned или 16-битное смещение; DD - long, float, 16-битный far (сегмент:смещение) или 32-битный близкий (смещение) указатель; DF - 32-битный far (сегмент:смещение) указатель; DQ - double; DT - упакованное десятичное (packed BCD) число длиной 20 цифр или long double (оба - формат FPU).

Вот еще одна интересная табличка. Все таки у нас программирование для дЗенствующих ;), а это подразумевает в первую очередь рассмотрение любого "объекта" с нескольких точек зрения. Первая колонка - это директива, а вторая - так называемое *адресное выражение* :

DW	16-битный адрес сегмента; смещение в 16-битном сегменте
DD	16-битный адрес + 16-битное смещение; 32-битный адрес
DF	16-битный адрес сегмента + 32-битное смещение

В принципе, все выше перечисленные директивы можно использовать для "задания" строкового значения. Однако имейте в виду, что в памяти они могут выглядеть совсем не так, как вы задали их в директиве, поэтому для инициализации строк мы всегда будем использовать DB. Следующая же простынка приводится исключительно в "образовательных" целях и ни в коем случае не следует воспринимать ее как руководство к действию ;).

```
String_1 DB 'A'
String_2 DW 'As'
String_3 DD 'Asse'
String_4 DF 'Assemb'
String_5 DQ 'Assemble'
String_6 DT 'Assembler ' ;после r - пробел
String_7 DB 'Assembler is ruleZ forever!'
```

Четко вы должны усвоить одно: существует только 6 типов переменных - *байт* , *слово* , *двойное слово* , *"дальнее слово"* (давайте будем именно так называть "дальний указатель" в формате сегмент : 32-битное смещение), *четверенное слово* и *"тенбайт"* ("10 байт"). Все остальное - от лукавого ;).

И, наконец, третья табличка. В тех случаях когда мы захотим инициализировать локальную переменную, существующую только в пределах какой-либо функции и умирающую, как только функция свое "отработает", мы будем использовать несколько другие директивы. Вот табличка их "соответствия":

DB	BYTE/SBYTE
DW	WORD/SWORD
DD	DWORD/SDWORD
DF	FWORD
DQ	QWORD
DT	TBYTE

Два варианта "соответствия" через слэш - это, *беззнаковое* (unsigned) и *знаковое* (signed) соответственно.

#3. Как я уже говорил в прошлом выпуске, вся документация, которую добренький микрософт задарма предоставляет своим девелоперам, заточена под си. А си - это, да будет вам известно, ;) все же язык программирования, претендующий на высокоуровневость. И те же 32 бита двойного слова могут быть обозваны и как указатель на строку символов (LPCSTR), и как результат, возвращаемый функцией (LRESULT), и даже, страшно подумать, цветом (COLORREF).

Вам это нравится? Мне тоже - нет. Однако, если мы хотим программировать приложения под win32, нам нужно научиться использовать функции API. Информация же об этих функциях находится в си-заточенном MSDN'е. Что ж делать? Будем учиться перезатачивать!

Загрузите свой MSDN и поищите там, например, функцию **WriteConsole** (в закладке Index). Смотрим на страничку в описании:

Функция **WriteConsole** записывает в строку символов буфер экрана консоли (console screen buffer), начиная с текущей позиции курсора.

```
BOOL WriteConsole(
    HANDLE hConsoleOutput    // хэндл буфера экрана
    CONST *lpBuffer VOID     // буфер для записи
```



```

DWORD nNumberOfCharsToWrite    // число символов для записи
LPDWORD lpNumberOfCharsWritten // число записанных символов (указатель)
LPVOID lpReserved              // резерв
);

```

Слово BOOL перед нашей функцией означает, что результат, который она нам вернет через регистр EAX (все апишные функции возвращают результат через этот регистр) - булевский.

Читаем описание. Возвращаемое значение не-ноль, если функция выполнялась успешно и 0, если сглючила. А дальше, в скобках, какие-то непонятные парочки слов наподобие **HANDLE hConsoleOutput**... Как их прикажете понимать?

Первое слово из "сладкой парочки" - это тип переменной, а второе - это её "имя собственное". Извлечь необходимую нам информацию мы можем как из первого, так и из второго слова ;). Первое слово - это тип переменной, которых в сях целый вагон с маленькой тележкой, расгрузить его попробуйте самостоятельно, обратившись к разделу *Platform SDK MSDN'a*, к топику *Data Types*. Второе же слово - это "имя собственное" переменной, однако имя хитрое - с **префиксом**, о котором мы с той или иной степенью вероятности может судить о **типе** этой переменной.

Это не потому, что правила языка таковы, это потому, что в мелкософте решили все кишки переменной описывать в её имени, причём как префикс, а не как суффикс - это чтобы жизнь мёдом не казалась при работе с сорсами от мелкософта; называется сия гадость "венгерская нотация".

Расшифровываю:

b	байт	BYTE
w	слово	WORD
dw	двойное слово	DWORD
h	хэндл	DWORD
lp	указатель	DWORD
n	"количество"	DWORD

Это - небольшой кусочек из так называемой венгерской нотации, которой мелкософт старается следовать. Суть её состоит в том, что имя переменной должно быть не просто идентификатором, но еще и нести в себе некоторые целевые характеристики. Притом не обязательно "размерные", но и "функциональные", с которыми вы можете ознакомиться в MSDN'овском топики *"Hungarian Notation"*

#4. Теперь, когда мы краешком глаза взглянули на многообразие типов данных великого и претендующего на сильную могучесть языка Си, давайте попробуем перевести топик про **WriteConsole** на язык Элпочки-людоедки, то бишь ассемблерный.

hConsoleOutput. Префикс **h**, значит (смотрим на табличку) - **DWORD**.

CONST lpBuffer VOID *. **CONST** и **VOID**, конечно же, смущают, но в имени переменной есть префикс **lp**, значит - **DWORD**. **DWORD nNumberOfCharsToWrite**. Префикс **n**, значит **DWORD** (хотя, собсно, при чём тут префикс? Про **DWORD** же прямым текстом написано!)

И так далее...

Остается открытым вопрос, чтож это за "указатель" за такой. По большому секрету скажу, что это самый обыкновенный адрес. А вот хэндл... брррр... в общем, это индекс в массиве данных, который позволяет таскать вместо всей структуры данных только её относительно короткий "идентификатор". Сама же структура хранится, разумеется, где-то в глубинах системы, что к тому же позволяет обеспечить её целостность несмотря на неосторожные или агрессивные действия со стороны программы.

Короче, **хэндл** - тот же адрес, только адрес является индексом памяти (массива байт), а хендл - индексом в некотором массиве, который лежит в этой памяти. :) Попробуйте это представить. Получилось?

На этой оптимистической ноте ;) переходим к следующей части - написанию "Hello, World" под Win32.

Win32ASM: "Hello, World" и три халявы MASM32

Автор: Serrgio / HI-TECH

Дата: 2002-08-21

#1. С легкой левой руки Дениса Ричи повелось начинать освоение нового языка программирования с создания простейшей программы "Hello, World". Ничто человеческое нам не чуждо - давайте и мы совершим сей грех.

В позапрошлом выпуске я уже рассказал о том, как работать в ассемблере с апишными функциями, однако вы наверняка не поняли ;). Это нормально, и не нужно из-за этого беспокоиться. Все станет более чем ясным после того как мы с вами напишем одну-две простенькие программки и разберем их по строчкам.

Заново перечитайте "Минимальное приложение" и набейте следующий исходник:

```
;-----  
;          ПРОЦ, МОДЕЛЬ, ОПЦИИ, ИНКЛУДЫ, БИБЛИОТЕКИ ИМПОРТА  
;-----  
  
.386  
.model flat,stdcall  
option casemap:none  
  
includelib kernel32.lib  
  
SetConsoleTitleA PROTO :DWORD  
GetStdHandle PROTO :DWORD  
WriteConsoleA PROTO :DWORD, :DWORD, :DWORD, :DWORD, :DWORD  
ExitProcess PROTO :DWORD  
Sleep PROTO :DWORD  
  
;-----  
;          СЕКЦИЯ КОНСТАНТ  
;-----  
  
.const  
  
sConsoleTitle db 'My First Console Application',0  
sWriteText db 'hEILo, Wo(R)LD!!'  
  
;-----  
;          СЕКЦИЯ КОДА  
;-----  
  
.code  
  
;-----  
;          Самая Главная Процедура  
;-----  
  
Main PROC  
    LOCAL hStdout :DWORD ;(1)  
  
    ;титл консоли  
    push offset sConsoleTitle ;(2)  
    call SetConsoleTitleA  
  
    ;получаем хэндл вывода ;(3)  
    push -11  
    call GetStdHandle  
    mov hStdout,EAX  
  
    ;выводим HELLO, WORLD! ;(4)  
    push 0  
    push 0
```

```

push 16d
push offset sWriteText
push hStdout
call WriteConsoleA

;задержка, чтобы полюбоваться ;(5)
push 2000d
call Sleep

;выход ;(6)
push 0
call ExitProcess

Main ENDP

;-----=

end Main

```

Вот две строчки из моего батника (*.bat), который позволяет не "париться" с командной строкой:

```

c:\tools\masm32\bin\ml /c /coff hello.asm
c:\tools\masm32\bin\link /SUBSYSTEM:CONSOLE /LIBPATH:c:\masm32\lib hello.obj

```

Обращаю внимание, что для сборки консольного приложения необходимо использовать ключ /SUBSYSTEM:CONSOLE. Несмотря на то что окошко, в котором оно запустится, до боли напоминает "сеанс MS-DOS", получившаяся программа - полноценное винدوزное 32-битное приложение в формате PE. Ассемблируем, линкуем, запускаем, наслаждаемся...

#2. А теперь давайте устроим этому исходнику разборку. **Бряк 1.** Таким образом мы определяем локальную переменную с именем *hStdout* и размером двойное слово (DWORD). Почему локальная? А потому, что она существует только внутри процедуры *Main*, и если бы мы попытались обращаться к переменной *hStdout* за пределами этой процедуры, ассемблер бы ругал нас всякими нехорошими словами - в отличие от, скажем, константы *sWriteText*, имя которой "известно" в любом месте нашей программы.

Обратите внимания на префикс *h* в названии переменной. Это я просто оставил для себя памятку, что переменная заведена под хэндл.

Бряк 2. Апишная функция *SetConsoleTitleA* - устанавливаем титл (заголовок) для нашего консольного окошка. Вот выдержка из MSDN'a:

```

BOOL SetConsoleTitle(
    LPCTSTR lpConsoleTitle // new console title
);

```

Как видим, функция требует один-единственный параметр - указатель на строку символов, которую мы хотим вывести в заголовке окна. Строка должна заканчиваться нулем.

Команда *push offset sConsoleTitle* помещает в стек (push) адрес (offset) строки символов (помеченной как *sConsoleTitle*). Ну а далее следует, собственно, сам вызов (call) функции *SetConsoleTitle*.

Заметьте, для указания адреса используется префикс под названием *offset*. Это потому, что берется смещение (offset) относительно начала сегмента, которое и является "ближним адресом". Есть еще "дальние" адреса, в которых задействуется также сам сегмент, но это тема будущих разговоров - сейчас это нас не должно волновать.

Здесь у вас должен возникнуть вполне закономерный вопрос - почему мы дописали букву А в конец функции? В MSDN'e ведь нет никакой буквы А... Я отвечу на этот вопрос немного позже.

Бряк 3. Консоль мы можем использовать как *устройство ввода* (input device), *устройство вывода* (output device), *устройство для отчета об ошибках* (error device). Для того чтобы работать с этим "девайсом", мы должны получить его хэндл при помощи следующей функции:

```

HANDLE GetStdHandle(
    DWORD nStdHandle // input, output, or error device

```

);

Единственный параметр, который она от нас требует - указание, на какое устройство мы желаем получить "квиток"-хэндл. Вот табличка:

Хэндл стандартного ввода	-10
Хэндл стандартного вывода	-11
Хэндл "ошибок"	-12

Что нам нужно? Вывести строчку! Значит - запрашиваем хэндл для стандартного вывода, то есть перед вызовом функции "суюм" в стек -11. После выполнения функции регистр EAX содержит столь желанный "хэндл стандартного вывода". Кладем этот хэндл в переменную *hStdout* (которую мы столь предусмотрительно определили на бряке 1) для последующего использования.

- Это ж что за безобразие? - воскликните вы. - *Что это за таблица такая нездоровая? Какие-то отрицательные числа, которые ни в жисть не запомнить! Хотим таблицу как в MSDN'e! Чтобы не -10, -11, -12, а длинные мнемонические STD_INPUT_HANDLE, STD_OUTPUT_HANDLE, STD_ERROR_HANDLE!*

Спокойно! Исходник, который мы сейчас рассматриваем, весьма точно отображает реальные процессы, происходящие в программе. Чуть позже мы приведем его к варианту в стиле Си и посмотрим, как можно использовать некоторые высокоуровневые конструкции, значительно облегчающие жизнь низкоуровневому программисту.

Бряк 4. Ну наконец-то, самое главное - функция, которая, собственно, и выводит на консоль строку символов. Вот ее описание:

```
BOOL WriteConsole(  
    HANDLE hConsoleOutput,      // handle to screen buffer  
    CONST VOID *lpBuffer,      // write buffer  
    DWORD nNumberOfCharsToWrite, // number of characters to write  
    LPDWORD lpNumberOfCharsWritten, // number of characters written  
    LPVOID lpReserved          // reserved  
);
```

Расшифровываем. Перед вызовом функции *WriteConsole* мы должны поместить в стек целых пять параметров:

1. Хэндл. Какие проблемы? Мы его уже получили и предусмотрительно сохранили в переменной *hStdout*. Командой *push hStdout* заносим его в стек, и все дела.
2. Указатель на строку символов, которую мы хотим напечатать. Сама строка у нас определена в секции констант под именем *sWriteText*. Получить ее адрес мы можем при помощи *offset*. Укладываем все в одну строчку - *push offset sWriteText*. Два в одном - и адрес получаем и в стек его заталкиваем :).
3. Число символов, которые мы хотим напечатать. В смысле - число "буковок" из строки *sWriteText*. Сколько символов в строке "hElLo, Wo(R)LD!!"? Включая пробелы - 16d. Пишем - *push 16d*. Заметьте, функция *WriteConsole* не требует нуля в конце буфера!
4. Указатель на переменную, в которой будет возвращено число напечатанных символов. Функция нам любезно сообщает, сколько символов из шестнадцати ей удалось напечатать. И требует переменную, в которую эту информацию ей занести. Давайте сделаем вид, что она нам не нужна, то есть напишем 0. Ничего страшного не случится, а в ошибочности подобного рода игнорирований убедимся в следующей главе. Пишем - *push 0*, но для себя оставляем пометку, что что-то функция от нас все же хотела.
5. Резерв. Так сказать, зарезервировано для следующих версий. Смело пишем - *push 0*.

Теперь, когда мы разобрали все параметры, обратите внимание на то, что MSDN'овская очередность параметров не соответствует той очередности, в которой мы записываем их в стек в нашем исходнике. Вернитесь еще раз к Минимальному приложению, п.12 и внимательно

прочитайте пункты соглашения *stdcall*. Теперь понятно?

Бряк 5. Дабы мы успели полюбоваться результатом трудов своих праведных, при помощи функции *Sleep* вызываем программную задержку в 2 секунды. Думаю, с параметрами вы без труда разберетесь.

И, наконец, **бряк 6** - выход из программы.

Вообще-то, правильный стиль предполагает явное освобождение всех занятых ресурсов по минованию надобности в них, в том числе и хэндлов, несмотря на то что они автоматически закрываются *ExitProcess* 'ом. Но будем надеяться, что если мы не сделаем это в такой маленькой программuline как наша, ничего страшного не случится. Естественно, "формат цэ" не в счет.

#3. Теперь делаем первый шаг по приведению нашего сырца в более читабельный вид.

Итак, первое, с чем мы ознакомимся - это эквиваленты, прописанные в файле */MASM32/windows.inc*.

Мы уже сталкивались с MSDN'овской табличкой:

Value	Meaning
STD_INPUT_HANDLE	Standard input handle
STD_OUTPUT_HANDLE	Standard output handle
STD_ERROR_HANDLE	Standard error handle

Однако вместо мнемонического интуитивно-понятного аргумента *STD_OUTPUT_HANDLE* вносили в стек значение -11, неизвестно откуда взятое. Давайте напишем сразу же после директивы *includelib* следующую строчку:

```
STD_OUTPUT_HANDLE equ -11d
```

А строчку *push -11* заменим на *push STD_OUTPUT_HANDLE*.

Что получилось? Программа откомпилировалась без проблем, ибо в самом начале листинга мы прописали *equ*[валент]. Проще говоря, мы сказали ассемблеру: "если ты встретишь в тексте программы *STD_OUTPUT_HANDLE*, то имей в виду, что это то же самое, что и -11". Другими словами, завели нечто типа константы (не переменную!) с именем *STD_OUTPUT_HANDLE* и значением -11.

Теперь откройте файл *windows.inc* и полюбуйте его содержимым. Там целая куча "эквивалентов", наподобие вышерассмотренного! И чтобы воспользоваться этой халявой - вовсе не обязательно копировать ту или иную константу через буфер обмена. Можно поступить намного проще - добавить в исходник директиву

```
include [путь к файлу] windows.inc
```

В ответ на это ассемблер сам извлечет из *windows.inc* всю имеющуюся в этом файле информацию и преподнесет ее транслятору на блюдечке с голубой каемочкой.

#4. Вторая халява, которой мы с вами воспользуемся - это "инклюды" (давайте именно так будем называть файлы *.inc) с прототипами функций. Мы уже рассматривали, что такое прототипы, и какую роль они играют при линковке нашей программы с библиотеками импорта. Конечно же, мы можем сами, на основе MSDN'овского описания функции, вывести ее прототип, но зачем нам приумножать сущности сверх необходимого? Ведь в MASM32 для каждой из библиотек импорта есть и одноименный файл с прототипами. В нашем примере мы использовали функции *kernel32* и для этого линковали его с библиотекой *kernel32.lib*? Ну а соответствующий файл с прототипами называется *kernel32.inc*!

Что может быть проще? Из нашего исходника вырезаем к черту блок с прототипами, а на его место лепим директиву *include [путь] kernel32.inc*. Компилим, и, как говорят по телику, "теперь вы можете забыть об этих неудобных промокающих :)" (ууупс... опять пошли брутальные фантазии; время

начинать новый абзац...).

Теперь, пожалуй, пришло время сдержать свое обещание и объяснить - какого черта мы к концу функции *WriteConsole* прилепили букву "A". Объясню - а потому что нет в винде функции *WriteConsole!*

#5. ...зато есть функции *WriteConsoleA* и *WriteConsoleW*. "A" - это если вы хотите напечатать строку в формате ASCII (т.е. каждый знак занимает один байт), а "W" - если в Unicode (W - от wide, широкий. В Unicode знаки не 8-битные, а 16-битные, и занимают два байта). Подобные окончания имеют только те функции, которые тем или иным образом работают со строковыми значениями. Функция *ExitProcess*, например, подобного буквенного окончания не имеет - посудите сами, не все ли равно, на каком национальном языке завершать работу приложения?

Откроем файл *kernel32.inc* и пристально посмотрим на его содержимое, в частности, на следующее:

```
WriteConsoleA PROTO :DWORD,:DWORD,:DWORD,:DWORD,:DWORD
WriteConsole equ <WriteConsoleA>
```

Как видим, команда разработчиков MASM32 позаботилась не только о простыне прототипов, но и о "независимости" нашего исходника от выбранной кодировки. То есть для того, чтобы "перезаточить" программу под UNICODE, нам вовсе не нужно заменять окончание A на W в имени функции. Достаточно просто приинклудить другой файл с прототипами и эквивалентами наподобие

```
WriteConsoleW PROTO :DWORD,:DWORD,:DWORD,:DWORD,:DWORD
WriteConsole equ <WriteConsoleW>
```

и не "париться" с переписыванием исходника.

Надо отметить, в MASM32 подобного "юникодного" инклуда нет, однако вы легко можете сделать его сами.

#6. И, наконец, третья, самая большая "халява" - это маленькая фенечка, использование которой сразу же превращает макроассемблер из языка кодирования в язык программирования!

С помощью этой "фенечки" целый блок инструкций:

```
push 0
push 0
push 16d
push offset sWriteText
push hStdout
call WriteConsoleA
```

мы с легкостью можем заменить одной-единственной строчкой:

```
invoke WriteConsoleA, hStdout, offset sWriteText, 16d, 0, 0
```

Обратите внимание, что при использовании этой команды параметры мы передаем слева направо, в той же очередности, что и вещает нам MSDN. В отличие от простыни "пушей" с "каллом" в конце.

#7. Теперь самый главный момент... Затаите дыхание!

В свете вышесказанного, вышерасписанного и вышерасжеванного наш исходник принимает весьма красивый "высокоуровневый" вид:

```
.386
.model flat,stdcall
option casemap:none

includelib kernel32.lib
include windows.inc
include kernel32.inc

.const

sConsoleTitle db 'My First Console Application',0
sWriteText db 'hEILo, Wo(R)LD!!!'
.code
```

```
Main PROC
    LOCAL hStdout :DWORD

    invoke SetConsoleTitle, offset sConsoleTitle
    invoke GetStdHandle, STD_OUTPUT_HANDLE
    mov hStdout,EAX
    invoke WriteConsole, hStdout, offset sWriteText, 16d, NULL, NULL
    invoke Sleep, 2000d
    invoke ExitProcess, NULL

Main ENDP

end Main
```

А что? Самое время выпить бутылочку пива ;).

Win32ASM: Консольный ввод, томограф IDA и скальпель SoftICE

Автор: Serrgio / HI-TECH

Дата: 2002-08-22

В этом tutorialе мы напишем простенькую консольную программу, познакомимся с Идой и Сайсом и с их помощью проведем небольшое исследование на тему что такое локальные переменных и с чем их едят.

Консольный ввод и томограф IDA

#1. В прошлый раз мы разобрались с консольным выводом. Сегодня - разберемся с консольным вводом. Для этого напишем простую программу, запрашивающую строку символов, а затем её же (строку) и выводющую. Это будет очередной маленький шаг, который для вас вполне может стать решающим, так как именно эта программа впоследствии послужит нам жертвой для негуманных экспериментов, в которых мы впервые используем два хирургических инструмента - отладчик SoftIce (на правах скальпеля) и дизассемблер IDA Pro (на правах томографа). И если при виде окровавленных внутренностей программы вам не поплохее, более того - если вам ЭТО понравится, значит вы имеете неплохие шансы стать либо серийным маньяком (крэкером), либо - исследователем программ (реверсером). Все же остальные профессии отныне перестанут для вас существовать :)

Итак, открываем ASM Editor и набиваем там следующий текст:

```
.386
.model flat, stdcall
option casemap :none ; case sensitive

; #####

include \tools\masm32\include\windows.inc
include \tools\masm32\include\kernel32.inc
includelib \tools\masm32\lib\kernel32.lib

; #####

.data

Msg1      db "Type something > "
Msg2      db "You typed > "
ConsoleTitle db 'Input & Output',0

; #####

.code

; #####

Main proc
LOCAL InputBuffer[128] :BYTE ;буффер для ввода
LOCAL hOutPut          :DWORD ;хэндл для вывода
LOCAL hInput           :DWORD ;хэндл для ввода
LOCAL lpszBuffer       :DWORD ;адрес буфера
LOCAL nRead            :DWORD ;прочитано байт
LOCAL nWritten         :DWORD ;напечатано байт

;устанавливаем титл окна
invoke SetConsoleTitle, addr ConsoleTitle

;получаем хэндл для вывода
invoke GetStdHandle, STD_OUTPUT_HANDLE
```

```

mov hOutPut, eax

;печатаем "Type something > "
invoke WriteConsole, hOutPut, addr Msg1, 17, addr nWritten, NULL

;получаем хэндл для ввода
invoke GetStdHandle, STD_INPUT_HANDLE
mov hInput, eax

;ВВОДИМ
invoke ReadConsole, hInput, addr InputBuffer, 10, ADDR nRead, NULL

;печатаем "You typed > "
invoke WriteConsole, hOutPut, addr Msg2, 12, addr nWritten, NULL

;печатаем то, что ввели
invoke WriteConsole, hOutPut, addr InputBuffer, nRead, addr nWritten, NULL

;задержка, чтобы полюбоваться
invoke Sleep, 2000d

;выход
invoke ExitProcess, 0
Main endp

; #####

end Main

```

ПРИМЕЧАНИЕ: Ручной подсчет числа выводимых символов хорошим стилем программирования, конечно же, не назовешь :(Немного попозже я расскажу, как делать это дело правильно ;). И да простят меня продвинутые программисты...

Строка `LOCAL InputBuffer[128] :BYTE` резервирует 128 байт памяти под строку символов, которую мы будем запрашивать при помощи апишной функции *ReadConsole*. Вот ее описание:

```

BOOL ReadConsole(
    HANDLE hConsoleInput,      // handle to console input buffer
    LPVOID lpBuffer,           // data buffer
    DWORD nNumberOfCharsToRead, // number of characters to read
    LPDWORD lpNumberOfCharsRead, // number of characters read
    LPVOID lpReserved           // reserved
);

```

Попутно даю урок английского языка, который сам знаю хреново (академиков не кончал, но высшее образование вам даду): "number of characters to read" переводится как "число символов, подлежащих чтению", а "number of characters read" - как "число прочитанных символов". Наверное. Согласитесь, это существенная разница ;).

#2. Далее обратите внимание, что адрес переменной мы получаем не при помощи `offset`, а при помощи `addr`. Все их различие заключается в том, что `addr` может работать с локальными переменными, а вот `offset` - нет.

Открою вам страшную тайну! На самом деле локальная переменная - это всего лишь зарезервированное место в стеке. Когда компилятор встречает `addr`, он сначала проверяет локальная это переменная или глобальная. Если глобальная, он помещает адрес этой переменной в объектный файл, то есть работает аналогично `offset`. Если же это локальная переменная, то перед вызовом функции генерируется следующая последовательность инструкций:

```

lea eax, LocalVar
push eax

```

Как обычно, я надеюсь на то, что вы не поверили мне на слово. Мы обязательно разберемся с тем, как происходит резервирование места в стеке и обращение к локальным переменным, но сделаем

немного позже. Для начала нам нужно хотя бы чуть-чуть ознакомиться с инструментарием. Начнем мы, пожалуй, с *IDA - The Interactive Disassembler*.

#3. IDA относится и интенсивно развивающимся продуктам, то есть вследствие постоянного совершенствования даже близкие версии могут вести себя по-разному. А по сему оговорюсь: описываемая мной последовательность действий рассчитана на версию 4.1.5.520, самым честным образом купленную. Если у вас другая версия, и мои советы "не проходят", то: во-первых, я не виноват, а во-вторых - для разнообразия попробуйте не только тупо следовать руководству, но еще и головой думать (сорри за грубость).

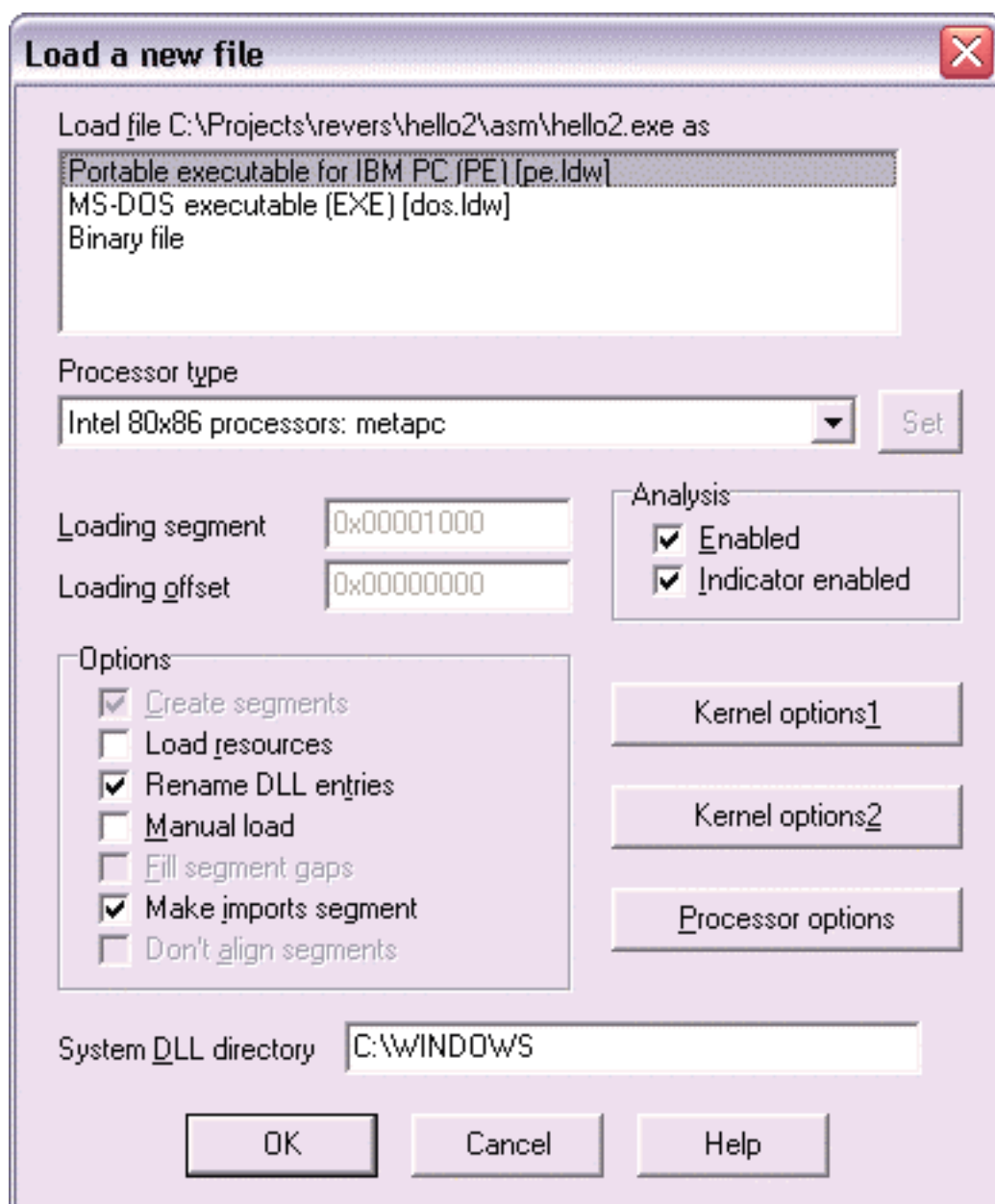
Что касается дизассемблирования вообще, то тут четко необходимо уяснить для себя одну вещь: вследствие того, что ассемблирование - это однонаправленный процесс с потерями, автоматическое восстановление исходного текста невозможно. Хотя, казалось бы, чего тут сложного - перевод двоичного кода процессора в удобочитаемые мнемоники... а фиг вам, задача еще та!

Существуют две категории дизассемблеров: автономные и интерактивные. Автономные требуют у юзера все необходимые им указания до начала процесса дизассемблирования и не позволяют вмешиваться в сам процесс. Соответственно, если результат нас не устраивает или мы желаем попробовать какую-либо дополнительную фишку из предоставляемых дизассемблером, то весь процесс (а для больших программ он может длиться часами!) придется повторять, и, скорее всего, не один раз.

А вот интерактивные позволяют "вручную" управлять процессом "препарирования" программы. В любой момент мы можем сказать дизассемблеру "парень, ты гонишь" и помочь этому парню, например, отличить адреса от констант либо определить границы инструкций и т. д. Соответственно, интерактивные дизассемблеры имеют хорошо развитый пользовательский интерфейс, а некоторые (не буду показывать пальцем, все наверняка уже догадались, какой дизассемблер я имею в виду) имеют даже собственный си-подобный язык скриптов! И более того - являются виртуальной программируемой машиной!

#4. Итак, давайте проведем первое знакомство с этим плодом человеческого гения... Запускаем ИДУ и озадачиваем ее нашим исполнимым файлом (*File > Open*).

Вот что мы увидим:



Я не менял настройки, оставил все дефолтом. Но посмотрите, например, на список *Processor type*, разве он не впечатляет? Жмем на **OK** и получаем дизассемблированный листинг нашей программы.

Наверняка вы некоторое время потягаете вверх-вниз вертикальный скроллбар и помедитируете над полученным результатом ;). И только потом начнете читать дальше... Что ж, совершенно правильное поведение ;)) Именно это я называю "медитацией" ;)

Теперь пойдем дальше...

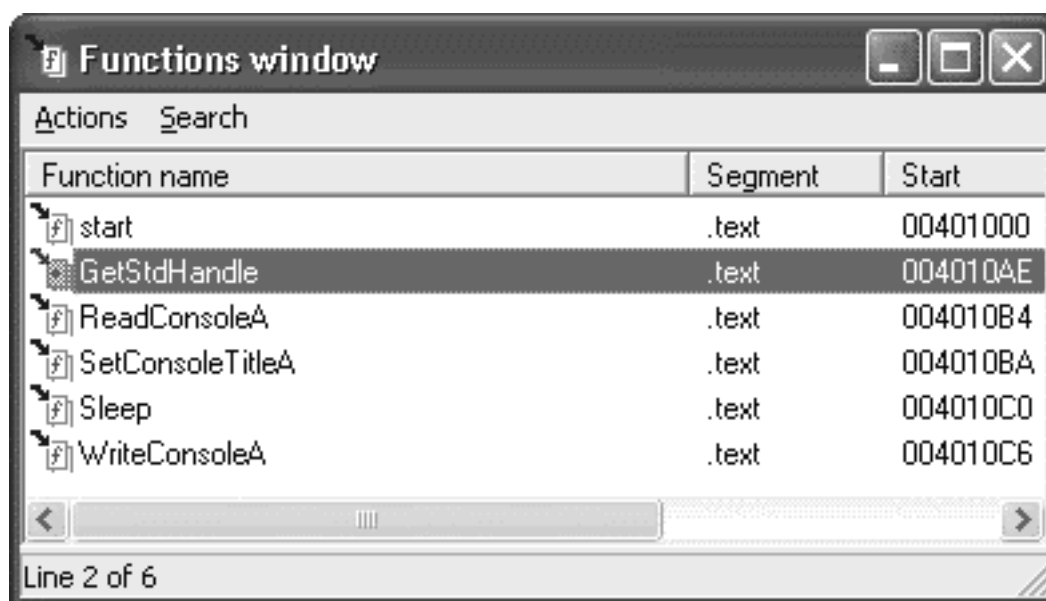
Лезем в пункт меню *Views* , и знакомимся с некоторыми его подпунктами. Сразу же совет: запоминайте горячие клавиши того или иного пункта меню. Это сэкономит вам кучу времени ;)

Итак, *View > Toggle dump view* переключит режим отображения из дизассемблированного листинга в режим дампа, то есть мы увидим простыню шестнадцатеричных цифр:

```
.text:00401000 55 8B EC 81 C4 6C FF FF-FF 68 1D 30 40 00 E8 A7 "UEuA-1 h_0@.oc"
.text:00401010 00 00 00 6A F5 E8 94 00-00 00 89 85 7C FF FF FF "...j?o0...EA| "
.text:00401020 6A 00 8D 85 6C FF FF FF-50 6A 11 68 00 30 40 00 "j.IA1 Pj_h.0@"
.text:00401030 FF B5 7C FF FF FF E8 8B-00 00 00 6A F6 E8 6C 00 " || oE...j?o1."
...
```

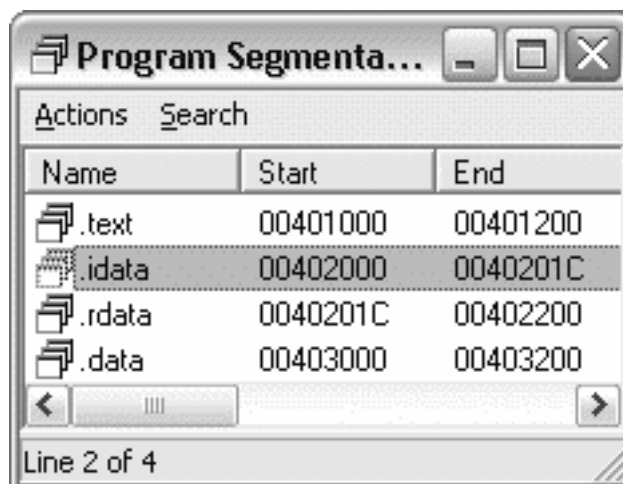
Не правда ли, до боли знакомо? Переключаемся назад в режим листинга нажатием клавиши **F4**.

View > Open subview > Functions выдаст нам окошко со всеми имеющимися в программе функциями:



Как видим, здесь в одну простыню собраны как наши собственные (в нашем примере это одна-единственная с именем start), так и обращения к внешним, апишным функциям. К каждой из функций в прилагается еще и дополнительная информация, подробнее об этом мы еще поговорим.

Кликаем **View > Open subviews > Segments** и видим следующую картинку:



Как вы уже, должно быть, догадались, это окошко показывает нам, из каких секций состоит дизассемблированная программа.

.text - эта секция содержит исполняемый код. Благодаря 32-битной плоской адресации содержимое аналогичных секций всех объектных файлов, подаваемых на вход линкера, собирается в одной секции .text исполняемого PE-файла. **.idata** - содержит данные об импортируемых приложением функциях, то бишь таблица импорта. Эта таблица состоит из 1) массива с описанием используемых DLL'ок, 2) двух массивов с адресами импортируемых функций и 3) массива имен импортируемых функций. Страшно?! Не переживайте, эту секцию мы еще рассмотрим детально... пока что просто знайте, что такая есть ;). **.rdata** - содержит данные, доступные только для чтения, как-то: литеральные строки, константы, отладочную информацию... Это тоже не берите в голову, попозже мы копнем глубже и эту секцию ;). **.data** - содержит инициализированные и глобальные переменные. Как и для секции .text одержимое .data-секций всех объектных файлов, подаваемых на вход линкера, собирается в одной секции .data исполняемого PE. На всякий случай напомним, что

локальные переменные в этой секции вы не найдете. **#5.** Двойной щелчок по той или иной секции переместит указатель на то место дизассемблированного листинга, где эта секция начинается.

Делаем кликаем .data и перемещаемся вот в это место нашего листинга:

```
.data:00403000 ; Segment type: Pure data
.data:00403000 _data      segment para public 'DATA' use32
.data:00403000          assume cs:_data
.data:00403000          ;org 403000h
.data:00403000 unk_403000 db 54h ; T      ; DATA XREF: start+2B^o
.data:00403001          db 79h ; y
.data:00403002          db 70h ; p
.data:00403003          db 65h ; e
.data:00403004          db 20h ;
.data:00403005          db 73h ; s
.data:00403006          db 6Fh ; o
.data:00403007          db 6Dh ; m
.data:00403008          db 65h ; e
.data:00403009          db 74h ; t
.data:0040300A          db 68h ; h
.data:0040300B          db 69h ; i
.data:0040300C          db 6Eh ; n
.data:0040300D          db 67h ; g
.data:0040300E          db 20h ;
.data:0040300F          db 3Eh ; >
.data:00403010          db 20h ;
.data:00403011 unk_403011 db 59h ; Y      ; DATA XREF: start+6D^o
.data:00403012          db 6Fh ; o
.data:00403013          db 75h ; u
.data:00403014          db 20h ;
.data:00403015          db 74h ; t
.data:00403016          db 79h ; y
.data:00403017          db 70h ; p
.data:00403018          db 65h ; e
.data:00403019          db 64h ; d
.data:0040301A          db 20h ;
.data:0040301B          db 3Eh ; >
.data:0040301C          db 20h ;
.data:0040301D unk_40301D db 49h ; I      ; DATA XREF: start+9^o
.data:0040301E          db 6Eh ; n
.data:0040301F          db 70h ; p
.data:00403020          db 75h ; u
.data:00403021          db 74h ; t
.data:00403022          db 20h ;
.data:00403023          db 26h ; &
.data:00403024          db 20h ;
.data:00403025          db 4Fh ; O
.data:00403026          db 75h ; u
.data:00403027          db 74h ; t
.data:00403028          db 70h ; p
.data:00403029          db 75h ; u
.data:0040302A          db 74h ; t
.data:0040302B          db 0 ;
.data:0040302C          align 200h
.data:0040302C _data      ends
.data:0040302C
.data:0040302C
.data:0040302C          end start...
```

Что мы видим? Длинную простыню из байтов! Ида нам подсказывает, какой символ соответствует тому или иному шестнадцатеричному значению. Не нужно быть семи пядей во лбу, чтобы догадаться, что эта простыня соответствует следующему куску нашего исходника:

```
.data
    Msg1      db "Type something > "
    Msg2      db "You typed > "
    ConsoleTitle db 'Input & Output',0
```

То есть unk_403000 (смотрим на дизассемблированный листинг) - это глобальная переменная Msg1 (смотрим на исходник нашей программы), unk_403011 - это Msg2, а unk_40301D - это ConsoleTitle. Теперь посмотрим на нашу дизассемблированную секцию данных. Напротив каждой метки имеется комментарий наподобие DATA XREF: start+2B^o. Но это не просто комментарий - это перекрестная ссылка, которая свидетельствует о том, что к текущему адресу произошло обращение из процедуры start. Более того, указывается и адрес, по которому происходит обращение - смещение в 2Bh от начала процедуры start. Стрелка указывает на относительное расположение источника перекрестной ссылки, а буква "o" свидетельствует о том, что обращение произошло по смещению (offset).

Теперь о главном. Если есть ссылка, то по ней можно (и нужно!) куда-нибудь проследовать, как по обычной html-ной гиперссылке, ;) Итак, перемещаем указатель на слово start+2B^o в комментарии и жмем на Enter!

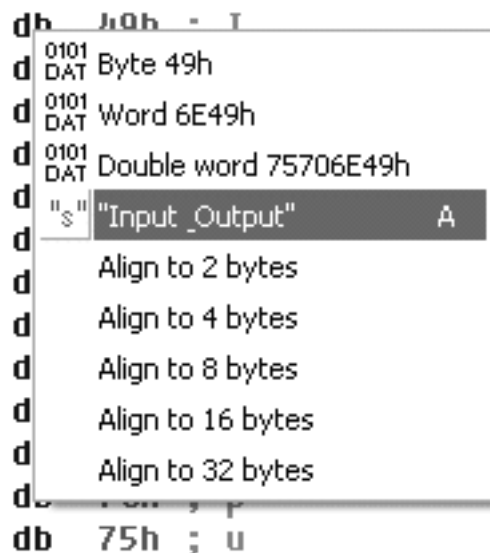
Перепрыгиваем на следующую строчку:

```
.text:0040102B push offset unk_403000 ; lpBuffer
```

И в самом деле, мы видим, что обращение к переменной (пихание оной в стек) происходит по смещению 2B при помощи префикса offset. И тут же видим ну вообще потрясающую вещь - IDA смекнула, что череда пушей перед call WriteConsoleA - это передача параметров соответствующей апишной функции, проанализировала там чего-то... и решила, что эта переменная - lpBuffer, совсем как в MSDN'овском описании функции WriteConsole! А ниже и хорошо знакомые нам переменные lpReserved, lpNumberOfCharsWritten, nNumberOfCharsToWrite, hConsoleOutput. Не правда ли, впечатляет? Сравните с листингами, генерируемыми другими дизассемблерами и вы поймете, что Иду не зря называют седьмым чудом света ;)

Перемещаем указатель на unk_403000, жмем на *Enter* и перепрыгиваем назад в секцию данных. **#6.** Честно говоря, мне не нравится то, какой простыней Ида дизассемблировала блок данных. Хотелось бы лицезреть его в таком виде, каков он был в исходном тексте ;). Для этого ставим указатель на последний db в секции данных и жмем на правую кнопку мыши, а вывалившейся контекстной менюшке

```
.data:0040301D unk_40301D
.data:0040301E
.data:0040301F
.data:00403020
.data:00403021
.data:00403022
.data:00403023
.data:00403024
.data:00403025
.data:00403026
.data:00403027
.data:00403028
.data:00403029
```



выбираем "s", то бишь "переопределить в строку". В итоге получим красивую, и, что самое главное, сходу понятную, строчку:

```
.data:0040301D aInputOutput db 'Input & Output',0 ; DATA XREF: start+9_o
```

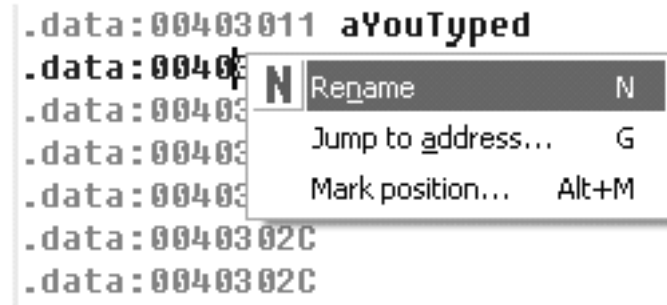
Прделаем такую же процедуру и с двумя "вышележащими" метками, получив в итоге следующее отображение секции данных:

```

.data:00403000 _data      segment para public 'DATA' use32
.data:00403000          assume cs:_data
.data:00403000          ;org 403000h
.data:00403000 aTypeSomething db 'Type something > ' ; DATA XREF: start+2B_o
.data:00403011 aYouTyped   db 'You typed > ' ; DATA XREF: start+6D_o
.data:0040301D aInputOutput db 'Input & Output',0 ; DATA XREF: start+9_o
.data:0040302C          align 200h
.data:0040302C _data      ends

```

Желающие могут обозвать метки по-своему. Для этого кликните правой кнопкой по адресу и выберите пункт Rename.



И можно вводить любые ругательные слова, которые только придут на ум ;)

#7. Обратите внимание вот на что:

Когда переменные были помечены как unk_403000, unk_403011 и unk_40301D, то обращение к ним осуществлялось следующим образом:

```

...
.text:00401009 push offset unk_40301D ; lpConsoleTitle
...
.text:0040102B push offset unk_403000 ; lpBuffer
...
.text:0040106D push offset unk_403011 ; lpBuffer
...

```

Когда мы переопределили данные из байтовых в строковые, то Ида переобозвала переменные в aTypeSomething, aYouTyped и aInputOutput, а обращение стало производиться уже к совершенно другим "именам собственным".

А вот если бы мы переопределили данные начиная не с последней метки, а с первой, то получилась бы строка:

```

.data:00403000 aTypeSomethingY db 'Type something > You typed > Input & Output',0

```

А обращение к ней осуществлялось бы вот как:

```

...
.text:00401009 push (offset aTypeSomethingY+1Dh) ; lpConsoleTitle
...
.text:0040102B push offset aTypeSomethingY ; lpBuffer
...
.text:0040106D push (offset aTypeSomethingY+11h) ; lpBuffer
...

```

Как я уже говорил, на сегодняшний день не существует ни одного полностью автоматического диасемблера, способного генерировать безупречно работоспособный листинг (вырожденные примеры наподобие нашего - не в счет), поэтому в той или иной мере, но доводить его готовности приходится человеку. Что мы помаленьку и будем учиться делать.

За сим будем считать что первое знакомство с Идой состоялось, а в качестве домашнего задания - попробуйте диасемблировать нашу программу Sourcer'ом и WinDASM'ом, чтобы, как говорится, почувствовать разницу ;)

Медитируйте!

Консольный ввод и скальпель SoftICE

#1. Сначала был 8086-й процессор, операционная система DOS и отладчик *debug* фирмы Microsoft. Отладчик неудобный, со скромными возможностями - он был (однако, есть и будет!) пригоден разве что для разного рода низкоуровневых забав и изучения ассемблера ;). В то время отладчики росли подобно грибам после мягкого кислотного дождика. И с такой же скоростью уходили в забвение - ибо от своего мелкомягкого прототипа отличались лишь интерфейсом. Это было золотое время для разработчиков всевозможных защит, так как достаточно было "запереть" клавиатуру, запретить прерывания, сбросить флаг трассировки, и у незадачливого хакера надолго отпадала охота копаться в чужом исполняемом коде...

Потом был 80286-й, такие шедевры программирования как отладчики *AFD Pro* и *Turbo Debugger* ... Золотое время разработчиков защит плавно перетекло в серебряное. А затем, с выходом на рынок фирмы NuMega и ее отладчика *SoftIce*, их жизнь вовсе превратилась в адский, неблагодарный, и, что самое важное, малоэффективный труд... Было вот как - разработчики защит были отдельными личностями либо небольшими фирмами, а за разработчиками *SoftIce* стояла намного более многочисленная группа людей, да не абы каких, а хакеров ;). Притом хакеров не в (увы!) современном понимании этого слова (т.е. прыщавых компьютерных вандалов, постоянных покупателей клирасила), а в единственно верном его толковании, т.е. людей в первую очередь высокоинтеллектуальных и стремящихся во всем разобраться до мелочей... Поэтому несколько не удивительно, что на появление 80386-го, который имел специальные механизмы, обеспечивающие контроль за исполнением кода на аппаратном уровне, NuMega отреагировала намного быстрее, чем противоположный лагерь, который "разобрался" с этими новыми возможностями только годы спустя; и как очевидно, при этом безнадежно проиграл "хакерским средствам".

А потом был Прозорливый Билли и его "принципиально новая архитектура", перед которой пасовали все существующие отладчики. Взамен им мелкософт предлагал свои, но ориентированы эти неповоротливые монстры были на программиста, отлаживающего собственный код, но никак не исполняемый файл, избавленный от отладочной информации. При этом документацию, необходимую для написания отладчиков под Windows, мелкомягкие не распространяли... некоторое время. И несмотря на то что конкурирующие фирмы все-таки вытянули ее посредством каленых клещей US'овской судебной системы, легче им от этого не стало - их отладчики все равно не превосходили мелкософтовский. Ибо это были отладчики под Windows. Неповоротливые отладчики под такой же неповоротливый виндовс.

NuMega же пошла своим путем, и повторить ее шедевр до сих пор никто не решается. Если операционная система не позволяет отлаживать программы - "забиваем" на операционную систему! Если стену не перепрыгнуть и не обойти, то почему бы ее не подкопать? Результат такого подхода оказался выше всяких похвал. "Виндозный" отладчик нумеги ни в чем не полагался на операционную систему, а опирался исключительно на аппаратное обеспечение и, вследствие этого, позволял отлаживать практически **любую** программу, в том числе и ядро операционной системы...

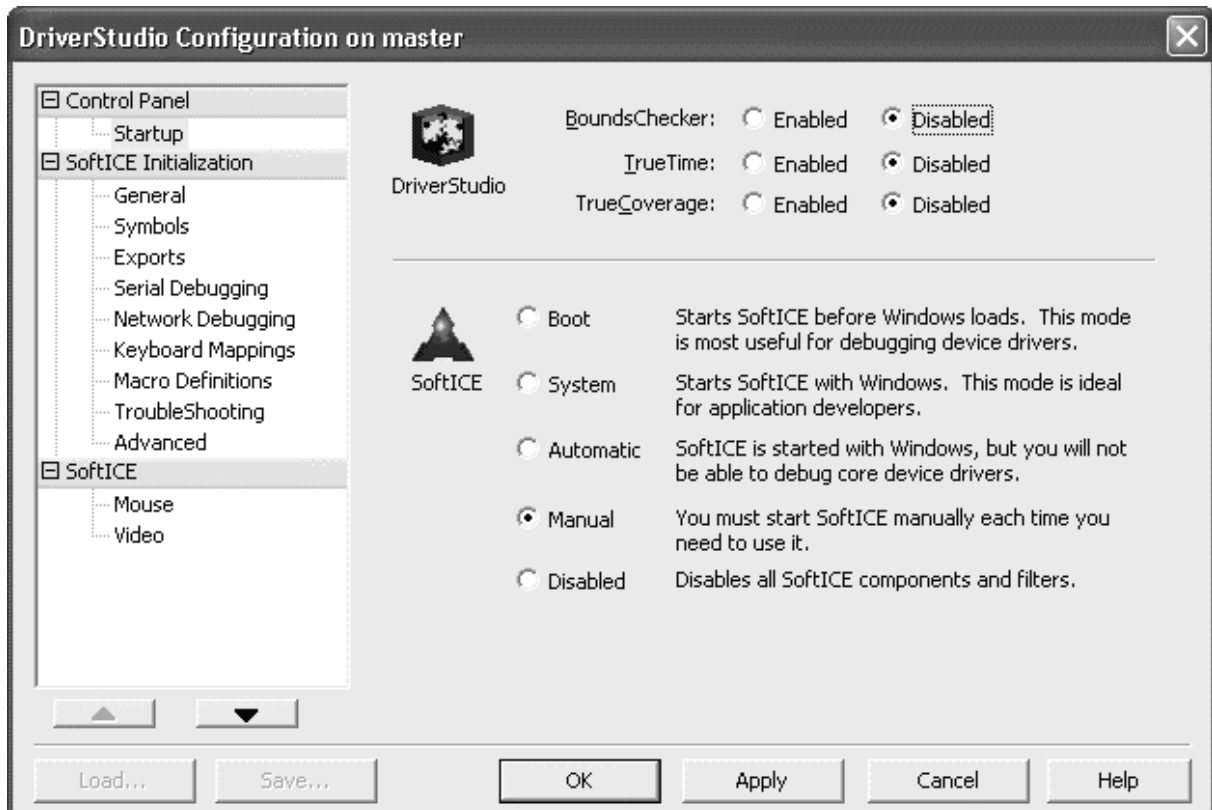
Что мы имеем в результате всего этого? Фирму NuMega - единственного "поставщика" высококачественных хакер-ориентированных инструментов под Windows, оперативно реагирующего на все изменения операционной системы от мелкософта...

Подробнее обо всем этом вы можете прочитать в книге К.Касперски "Техника и философия хакерских атак", настоятельно рекомендую (при всем своем уважении к автору, не могу не отметить, что второе издание этой книги, выпущенное издательством Солон, имеет отвратительный переплет - по первом же прочтении книга буквально развалилась на отдельные страницы).

#2. Как вы, должно быть, уже поняли, способ запуска сайса (именно так мы будем называть *SoftIce*) несколько сложнее, чем всенародно любимого пасьянса "косынка". В Windows 9X для этого нужно было редактировать *autoexec.bat* и делать мультikonфигурацию, но в NT с этим дела обстоят

немного проще. Всю последовательность действий я привожу из предположения, что у читателя установлена операционная система *Windows 2000* (которая, как известно, "build on NT technology"), английская, а сайс берется из пакета *DriverStudio* версии 2.5. При этом желающие "сходу" произвести полную установку этого пакета должны иметь в виду, что программа инсталляции попросит, помимо всего прочего, указать пути к *Driver Development Kit* ;).

Итак, после установки (и появления на панели задач соответствующей ветки меню), первое, что мы должны сделать, это выбрать тип его загрузки. Для этого запускаем программу конфигурирования (*Start > NuMega DriverStudio > SoftIce > Settings*) и видим следующее окошко:



Честно говоря, меня вполне устраивает ручной режим загрузки, то есть *Manual* - наверное, это потому что я еще не сталкивался с отладкой "core device driver" ;). Поэтому насчет остальных режимов ничего сказать не могу. Итак, ставим *Manual* ;). Чтобы запустить сайс в этом режиме, необходимо ввести команду:

```
NET START NTICE
```

либо запустить "*Start SoftICE*" из менюшки (pif на обыкновенный батник, где эта команда написана). Все! Отладчик запущен. Только вот не увидите вы его ни в *Applications* , ни в *Processes* (CTRL+ALT+DEL). Ни даже на экране - пока не нажмете на волшебную комбинацию клавиш **Ctrl+D** _ ;).

Жмем на Ctrl+D! Если отладчик установился успешно, то всплывет приблизительно следующая "картинка":

```
-----
EAX=00005305  EBX=C4920074  ECX=C14698E4  EDX=00000000  ESI=C1476EC0
EDI=C49202B0  EBP=67890000  ESP=C4687E2C  EIP=000080D2  o d i s z a P c
CS=0128  DS=0030  SS=0030  ES=0030  FS=0078  GS=0030
-----byte-----PROT16-
0030:00000000  9E 0F C9 D8 65 04 70 00-16 00 C9 09 65 04 70 00  ....e.p.....e.p.
0030:00000010  65 04 70 00 54 00 FF F0-58 7F 00 F0 FF E7 00 F0  e.p.T.....
0030:00000020  00 00 00 C9 D2 08 A3 0A-6F EF 00 F0 6F 00 F0 00  ....o...o...
0030:00000030  6F EF 00 F0 6F EF 00 F0-9A 00 C9 09 65 04 70 00  o...o.....e.p.
-----Cancel_Call_When_Idle+002C-----PROT16-
0128:80D1  POPF
```

```

0128:80D2 CLS
0128:80D3 RETF
0128:80D4 POPF
0128:80D5 STC
0128:80D6 RETF
0128:80D7 CMP      AL,13
0128:80D9 NOP
0128:80DA NOP
0128:80DB JBE      80E1
-----
:rs
:g
WINICE: Free32 Obj=01 Mod=NOTEPAD
WINICE: Free32 Obj=02 Mod=NOTEPAD
WINICE: Free32 Obj=03 Mod=NOTEPAD
WINICE: Free32 Obj=04 Mod=NOTEPAD
WINICE: Free32 Obj=05 Mod=NOTEPAD
WINICE: Free16 Sel=351F
:X
-----
X, XFRAME, XG, XP, XRSET, XT                                KERNEL32
-----

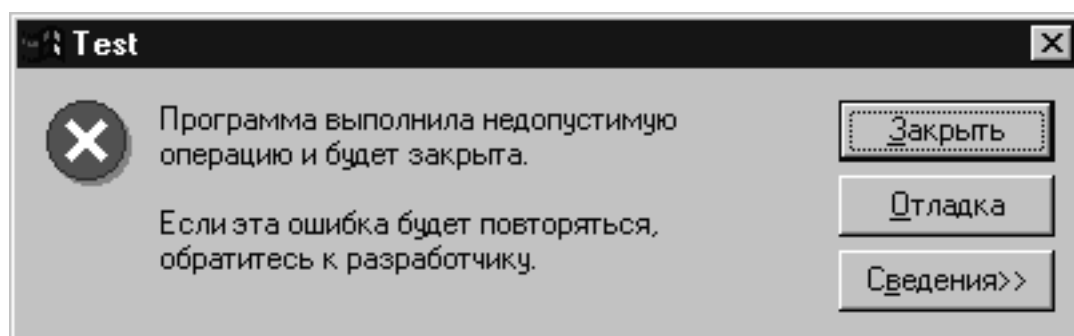
```

Сверху, как вы уже, должно быть, догадались, регистры. Чуть ниже - дампы памяти. Еще ниже - дизассемблированные команды процессора. Далее следуют окно диспетчера, в котором мы можем вводить команды и читать различные матюгальники, и контекстная подсказка.

#3. Не знаю, как на вашем дисплее, но на моем 17-дюймовом с разрешением 1024x768 окошко отладчика получилось больно уж маленьким. Чтобы не напрягать глаза, его можно немножко "под себя" настроить. Ок! Давайте попробуем ввести несколько команд, которые позволяют выполнить подобную настройку. Для этого пишем следующие команды:

- **SET FONT 2** - и шрифт в окошке немного увеличивается, как и сам размер окна;
- **LINES 60** - увеличиваем число строк в окне отладчика;
- **WD 22** - задаем число строк под дампы;
- **WC 25** - задаем число строк под код;
- **CODE ON** - разрешить отображать байты инструкций.;
- **COLOR A A 20 20 2** - устанавливаем "извращенную" цветовую схему (подробнее о параметрах смотрите в *SoftICE Command Reference*, битовое кодирование цвета мы рассматривали).

Еще одна команда (я настоятельно рекомендую ее использовать, особенно пользователям W9X) - это **FAULTS OFF**, которая предотвращает "всплытие" отладчика при возникновении GPF - General Protection Fault, в просторечии также известную как "ваши ручки выполнили недопустимую операцию и будут ампутированы".



Теперь, когда мы настроили "под себя" внешний вид отладчика, давайте позаботимся о том, чтобы нам не нужно было при его запуске каждый раз вводить эти семь команд. Т.е. пропишем все эти команды в строку инициализации - простую команду, которые автоматически будут выполняться при загрузке отладчика. Для этого ищем конфигурационный файл *winice.dat* (в 2000 я его нашел в *Windows\system32\drivers*; в 9X, насколько я помню, он находился в том же каталоге, что и проинсталлированный SoftICE) и дополняем строчку `INIT="X;"` нашими командами ("X;" в самом

конце - это выход из окна сайса):

```
INIT="SET FONT 2; LINES 60; WD 22; WC 25; CODE ON; COLOR A A 20 20 2; FAULTS OFF; X;"
```

Далее раскомментируйте в *winice.dat* все строки наподобие

```
EXP=\SystemRoot\System32\kernel32.dll
```

Это необходимо для того чтобы сайс загрузил имена экспортируемых функций, находящихся в этих библиотечках. Иначе вместо понятных команд, наподобие

```
call USER32!MessageBoxA
```

мы увидим безобразия типа

```
call 0044F2A1
```

Теперь нам нужно перезапустить отладчик, чтобы проверить, подхватывает ли сайс наши настройки. Для этого нужно... В общем, как я уже говорил, сайс - прога, весьма специфическая, и выгрузить ее "из компьютера" можно только одним способом - перезагрузкой ;). *Start > Shutdown > Restart...*

#4. Существуют две области применения сайса - отладка собственных программ и исследование чужих, так называемый *reverse engineering*. Для начала давайте научимся использовать сайс в качестве инструмента для отладки и изучения собственных приложений.

Что мы имеем в этом случае? Исходные тексты программы, и как следствие - возможность откомпилировать ее отладочную версию, т.е. тот же экзешник, но содержащий, помимо всего прочего, еще и кучу дополнительной информации. Как в самом исполняемом файле, так и в специально для этой цели сгенерированных дополнительных файлах.

Итак, мы должны:

1. Сассемблировать исходник с ключем Zi:

```
ml /c /coff /Zi src.asm
```

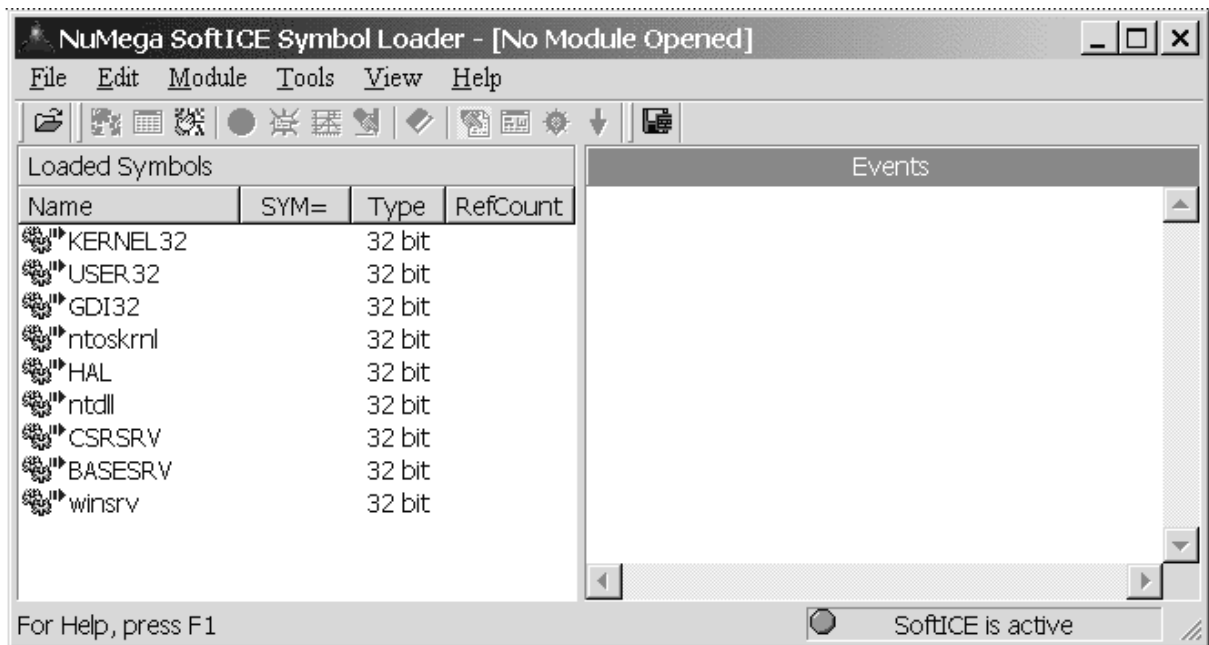
В результате этого мы получим объектный файл, содержащий отладочную информацию для отладчика *CodeView*. Легко заметить, что размер этого файла намного больше, чем у его "нормального" аналога.

2. Слинковать объектный файл с ключами /DEBUG и /DEBUGTYPE:CV

```
link.exe /SUBSYSTEM:CONSOLE /DEBUG /DEBUGTYPE:CV src.obj
```

После этого, помимо экзешника, мы получим отладочные файлы *src.ilc* и *simple.pdb*, *Microsoft Linker Database* и *Microsoft C/C++ program database* соответственно.

Теперь загружаем нашу отладочную версию программы в отладчик. Для этого запускаем *Symbol Loader*:



В левом окне мы видим, из каких файлов подгружена символическая информация. Зеленая лампочка в строке статуса свидетельствует о том, что отладчик подгружен (конечно, если вы это сделали после перезагрузки). Правое окно - это окно отчета, там появляется информация о выполненных действиях, ошибках и т.д. Ах да..., еще есть заголовок окна, там помимо названия программы есть еще и надпись в скобках *[No Module Opened]*, то есть "не открыт модуль". Не правда ли, не очень тонкий намек?

- Жмем *File > Open* и открываем наш *src.exe* (отладочные файлы, как, желательно, и исходник, должны находиться в этом же каталоге). На первый взгляд ничего не изменилось, но посмотрите внимательно на заголовок программы - надпись "не открыт модуль" заменилась на "*src.exe*". Значит, что-то все-таки произошло ;).

- Далее жмем *Module > Setting*, и всплывает диалоговое окно, в котором можно настроить все, что мы желаем сделать с модулем, перед тем как он будет загружен в отладчик. Настраиваем:

- Из закладки *General* нам ничего не нужно - все поля оставляем пустыми, никаких галочек не ставим.

- В *Debugging* выбираем *Load Executable*("загрузить экзешник") и ставим галочку на *Stop at WinMain, Main, DllMain, etc...* т.е. "остановиться на точке входа".

- В *Translation* выбираем *Symbols only (included locals and structures)*, т.е. "только символическую информацию, включая локальные переменные и структуры".

Теперь мы готовы свершить самое главное действие - загрузить программу под отладчик. Жмем *Module > Load*, и вот оно, долгожданное! У нас всплывает окно сайса с указателем, установленным на точку входа в нашу программу:

```
Main
001B:00401010 55          PUSH EBP
```

#5. Для начала рассмотрим команду **display memory** (отобразить память). Ее синтаксис:

```
D[size] [address [l length]]
```

Если ее использовать без параметров, то просто будет отображаться следующая страница дампа. Необязательный параметр *size* - это размерность, в которой выводится дамп. Вот расшифровка его значений:

```
b      Byte
```

W	Word
D	Double Word
S	Short Real
L	Long Real
T	10-Byte Real

Размерности byte, word и double word вам, конечно же, хорошо знакомы. А вот загадочные short long и 10-byte real мы рассмотрим немного позже.

Обратите внимание на то, что первый параметр (размерность) мы должны писать "в одно слово" с, собственно, самой командой "d", т.е. - "db", "dw", "dd" и т.д.

Необязательный параметр *address* - это адрес памяти, дамп которого вы хотите получить. Притом вовсе необязательно писать адрес цифрами - "составные" адреса сайс также принимает "за милую душу".

Таким образом, если мы введем команду:

```
DD EIP
```

то увидим дамп той области памяти, в которой в настоящее время располагается секция кода нашей программы (конечно, если после всплытия отладчика на точке входа вы ничего не успели напортачить).

Если же мы начнем использовать параметр length, то данные, запрашиваемые с использованием этого параметра, начнут отображаться в командной строке. Например, команда

```
DW EIP L 1000
```

выведет в командную строку 1000 байтов памяти, начиная с текущего значения регистра EIP, группированных пословно, и вы будете вынуждены несколько раз нажать на *any key*, прежде чем все это просмотрите. По большому секрету скажу, что нажатие на *ESC* сразу же прервет просмотр.

#6. Пожалуй, одна из главных возможностей любого отладчика - это трассировка, которая позволяет выполнять программу пошагово. Итак, загрузим нашу программу в *SymbolLoader*, нажмем на *Load*, а в появившемся окне отладчика введем команду "p" (она же - клавиша F10). В результате этого выполнится один логический шаг нашей программы (program step). Под "логическим шагом" подразумевается, своего рода "поверхностная" трассировка, без входа в процедуры, циклы и т.д.

Команда "t" (она же - клавиша F8) выполняет трассировку одной инструкции (trace one instruction), с "заходом" во все функции, в том числе и апишные. Подобная трассировка - это очень длинный путь, поэтому у данной команды существует еще и два параметра:

```
T [=start-address] [count]
```

Первый - это адрес, с которого вы желаете начать трассировку, а count - это число инструкций, которое сайс протрассирует, прежде чем остановиться (и не забывайте о том, что он "заходит" в апишные функции!).

Конечно же, трассировочные возможности сайса намного больше, чем те немногие, которые мы рассмотрели. Впоследствии мы обязательно ознакомимся со всем их многообразием - по мере необходимости ;).

#7. Еще одна полезная команда - это *rs* (она же F4), restore the program screen (восстановление экрана программы). А как же без этого? Ведь внешне работа программы заключается вовсе не в выполнении команд процессора и пересылках данных между регистрами ;).

Например в нашем случае программа печатает приглашение ввести строку символов. Мы можем оттрассировать программу до (включительно) строки:

```
001B:00401046 E8BD000000 CALL _WriteConsoleA
```

и затем нажать на F4, для того чтобы подсмотреть, действительно ли в консольном окне появилась строчка "Type something >". А затем нажать на *any key* и снова очутиться в отладчике.

Или же, например, выполнив апишную функцию

001B: 00401070 E881000000 CALL _ReadConsoleA

которая "просит" ввести строку символов, нажать на F4 и ввести необходимую строку символов, а по нажатию на Enter (типа "ввод закончен") снова оказаться в отладчике.

#8. Теперь, когда мы поверхностно ознакомились с "хакерскими" инструментами, пришло время вспомнить о цели, которую мы преследовали, начав знакомство с идой и сайсом. Напоминаю - мы хотели исследовать, каким образом происходит деклариование локальных переменных в стеке, и как происходит к ним обращение. **Стек и локальные переменные**

#1. Представьте себе картину: накачанный колесами и протеиновыми коктейлями шварценеггер бьет со всей дури по боксерской груше. Та отлетает в сторону, а потом по каким-то абсолютно нефизическим законам кинематографа возвращается назад и бьет этому боксеру по морде лица, в результате чего шварценеггер отлетает на несколько метров, и, обязательно задев что-нибудь из мебели, размазывается соплями по стене - к неописуемому удовольствию зрителя. Посмотрев на такую картину, Станиславский бы сказал: "не верю"! А вот дзенствующей программист, увидя это безобразие, подумал бы: "ба! Да совсем как стек. Помнища, в одной из своих кулхацкерных прог я на похожие грабли как раз и напоролся".

Ранее мы уже разобрались с очередностью записи в стек и чтения из него. Напомню, что доступ к стеку осуществляется в соответствии с принципом LIFO (Last In First Out – Последним Пришел, Первым Ушел). Однако это отнюдь не единственное, что нам нужно знать о стеке - конечно же, если мы не собираемся время от времени получать "отдачу" от "боксерской груши" ;).

Нам уже хорошо известно, что стек можно использовать для временного хранения данных – с его помощью мы выкручивались из такой проблемы, как недостаточное для полета нашей фантазии количество регистров. То есть мы временно сохраняли значения регистров в стеке, активно юзали их для наших нужд, а потом снова восстанавливали "статус кво", за исключением, в большинстве случаев, одного-единственного регистра, в котором хранился результат проделанной работы.

Также известно, что в инструкциях процессоров от Интел переменные (которые в памяти) не могут выступать в качестве приемника и источника одновременно, то есть инструкция:

```
mov [dwVar1],[dwVar2]
```

не проходит. А выкрутиться из такой ситуации можно двумя способами - либо использовать в качестве посредника регистр (которых, как всегда, мало):

```
mov eax,[dwVar2]
mov [dwVar1],eax
```

либо задействовав стек:

```
push [dwVar2]
pop [dwVar1]
```

Кстати, недавно в почтовой рассылке RTFM_helpers прошло обсуждение того, как можно копировать из памяти в память - там было упомянуто, например, использование movs. А если подумать, можно найти и другие нетривиальные способы.

Для тех, кто ещё не понял. Вот этот кусок кода:

```
push 1
push 2
push 3
pop  eax
pop  ebx
pop  ecx
```

делает то же самое, что и следующий код (если только не считать разницу в скорости, размере кода и побочных эффектах):

```

mov  eax,3
mov  ebx,2
mov  ecx,1

```

Сомневающиеся могут проверить под отладчиком:). Также попробуйте сравнить:

```

push 1
pop  eax

```

и психоделическое:

```

sub  esp,4
mov  dword ptr [esp],1
mov  eax,[esp]
add  esp,4

```

Вы можете мне не поверить, но эти два куска кода тоже "делают" одно и то же, хотя последний и кажется плодом больного воображения.

- *Что за esp такой?* – спросите вы.

Пошире откройте глаза и слушайте – сейчас я поведаю вам страшные тайны. ;)

#2. Помните мою аналогию с блинами от штанги и вделанным в пол штырём, на который они надевались для хранения. Там еще был учитель физкультуры, который приставал к нашим неполовозрелым одноклассникам, в результате чего ему перебили нос. Так вот, адрес самого верхнего блина – это *вершина стека* и хранится он (адрес) в регистре *esp/sp*. Другими словами, вершина стека – это адрес последнего занесенного в стек элемента.

Давайте посмотрим на поведение esp при трассировке следующего кода:

```

Main proc
push 1
push 2
push 3
pop  eax
pop  ebx
pop  ecx
invoke ExitProcess,0
Main endp

```

При загрузке мы видим, что регистр *esp* инициализирован значением 12FFC4 (в других условиях стартовое значение может быть другим, но суть от этого не меняется). Давайте выполним один шаг (команда "t" - trace). В результате этого в стек ляжет 1, а значение регистра esp поменяется на 12FFC0. Трассируем дальше и наблюдаем, как изменяется *esp* :

```

push 1    ;esp=12FFC0
push 2    ;esp=12FFBC
push 3    ;esp=12FFB8
pop  eax  ;esp=12FFBC
pop  ebx  ;esp=12FFC0
pop  ecx  ;esp=12FFC4

```

Протрассировав первые три строчки, мы можем сделать вывод, что *стек "растет" в сторону младших адресов* (12FFC0 > 12FFBC > 12FFB8, логично?), а шагом изменения регистра esp является 4. И это правильно, так как при 32-битном режиме адресации в стеке сохраняются двойные слова, они же - 4 байта (хотя допускается класть в стек также и 2-байтные слова - при этом шаг равен 2). К слову сказать: если бы мы писали под DOS (точнее, в реальном или виртуальном режиме), то стек у нас адресовался бы регистром **_sp_** и изменялся бы он на плюс-минус 2.

Обобщаем. Алгоритм работы команды *push <источник>* следующий:

1. Уменьшение значения указателя стека **_esp/sp_** на 4/2.
2. Запись значения источника по адресу **_ss:esp/sp_** (вершина стека).

Об алгоритме работы команды **_pop_** догадайтесь сами...

Для последователей дЗена предлагаем тему для исследования: какое значение будет лежать в стеке после инструкции *push esp*. :) **ПРЕДУПРЕЖДЕНИЕ** : *ни в коем случае не принимайте результаты отдельных экспериментов в конкретных условиях за абсолютную истину, верную всюду и всегда!*

Теперь, после вышесказанного, вы легко сообразите, какое значение примет регистр *eax* в следующем извращенном случае:

```
push 1    ;esp=12FFC0
push 2    ;esp=12FFBC
push 3    ;esp=12FFB8
add esp,4
pop  eax  ;esp=12FFC0
pop  ebx  ;esp=12FFC4
```

[Правильный ответ – 2. :)]

#3. Есть ещё один регистр, ассоциируемый со стеком - *ebp/bp*, и описывается его функция так, что выговорить страшно – *указатель базы кадра стека*. Такое название этого регистра я нашел в книжке Юрова & Хорошенко "ASSEMBLER, учебный курс". Нет, конечно же, можно назвать калоши "мокrostупами", а *bitmap* "двоично-точечной картинкой", но... *"У меня нет слов, у меня есть только выражения в адрес того, кто заворачивает такие коленца"* (С) Аркадий Белоусов. А по сему давайте заменим словосочетание "кадр стека" простым народным :) словом "фрейм", а "указатель базы" заменим на просто "база" (или "указатель" - по вкусу).

В результате подобных терминологических "подстановок" получается следующая картина: есть у нас в компьютере некие "фреймы", располагаются они в стеке и адреса этих пока что непонятных нам штук завязаны с регистром *ebp/bp*.

"Дело в следующем". Любая процедура/функция в терминах любого процедурного языка имеет (может иметь) нуль и больше параметров и локальных переменных. *Область памяти, создаваемая (выделяемая) при вызове процедур для аргументов и локальных переменных*, и называется фреймом. А чтобы процедура могла быть рекурсивной (т.е. могла вызывать саму себя или вызываться из других процедур, которые она вызывает) или реентерабельной (т.е. чтобы код процедуры мог использоваться в параллельных процессах), фреймы для неё должны размещаться в стекоподобной структуре данных - "стеке". А поскольку процедурные языки ближе к "естественному" мышлению, то в наборы инструкций процессоров и ассемблеры вводят поддержку подобных высокоуровневых конструкций.

Существуют нюансы и различия в реализациях, например:

- Писюковые сишные компиляторы генерируют код, в котором часть фрейма с аргументами после вызова процедуры удаляется вызывающим кодом, а в паскалевских соглашениях о вызове фрейм всегда удаляется самой процедурой (что экономит на размере кода, поэтому это соглашение было принято как стандартное в Windows).
- В Паскале существует понятие локальных процедур, которые могут обращаться к параметрам и локальным переменным всех родительских (в смысле статического размещения, а не динамического порядка вызовов) процедур и при этом могут быть рекурсивными (родительские процедуры тоже могут быть рекурсивными, и доступ к переменным должен идти к последнему, активному экземпляру). Для поддержки этой идеи во фреймы добавляются указатели на родительские фреймы, обычно организованные в виде списка.
- Фортран вообще язык не рекурсивный, поэтому IBM в реализации Фортрана на IBM/360 (где, кстати, нет поддержки стека) для каждой процедуры заводила фрейм статически, во время компиляции.

Подобные "нюансы" сущности фрейма, конечно же, не меняют. Приведены они по одной единственной причине – чтобы вы поняли некоторую условность такого термина как "фрейм".

Теперь, собсна, про *ebp/ep*. В случае стекового фрейма его адрес не фиксирован, поэтому адресация параметров и переменных в нём должна быть "базисно-индексной", относительно начала фрейма. На пискюке под это идеально подошёл (или изначально проектировался) *BP/EBP* - в отличие от *SP*, он может служить базой, и также адресуется относительно *SS*.

Я знаю, что вы мало что поняли из всего этого бреда, а по сему будем познавать истину путем долгой и продолжительной медитации.

#4. Мы привыкли к тому, что извлекать данные из стека можно только повинаясь очередности LIFO. А что, если нам понадобилось обратиться к произвольному элементу стека? Один из возможных способов мы уже рассмотрели: это изменение значения регистра *esp/sp* плюс команда *pop*. Это далеко не совсем хорошая идея, и вам не стоит издеваться над стеком таким изощренным способом. Если, конечно же, вы не хотите уподобляться Штирлицу и Мюллеру, которые стреляли по очереди...

Напомню: регистр *ebp/bp* ведет себя приблизительно таким же образом, как и хорошо нам знакомый *ebx/bx*. То есть он может выполнять роль *базы*.

Напомню, что, например, следующий код:

```
mov ebx,12FFC0h
mov al,[ebx]
```

присвоит регистру AL значение байта по адресу 12FFC0 из сегмента, задаваемого регистром *DS*.

Точно таким же образом можно использовать и регистр *ebp/bp*. Говоря другими словами, это один из немногих регистров, которые можно "брать в квадратные скобки" не увеличивая при этом размер инструкции :). То есть (и это будут уже третьи слова) он позволяет работать с ячейкой памяти, адрес которой находится в регистре.

Проиллюстрирую это на простом примере. Допустим, занесли вы в стек разных параметров кучу:

адрес	значение
0012FFC4	77E7EB69
0012FFC8	0047E4AC
0012FFCC	0012DAB4
0012FFD0	7FFDF080

И возжелалось вам в силу какой-нибудь нездоровой производственной необходимости прочитать "здесь и сейчас", например, предпоследний элемент. В этом случае делаем вот что:

```
mov ebp,esp
mov eax,[ebp+4]
```

Расшифровываю. Мы принимаем адрес самого последнего из записанных в стек элемента за точку отсчета, и путем прибавления к этой "точке отсчета" четвёрок можно легко получить доступ к тому элементу стека, который нашей программмерской душеньке возжелался. В принципе, в данном примере можно было бы использовать *ESP* вместо *EBP*, но, во-первых, должны же мы были показать использование *EBP*, :) а, во-вторых, в больших фрагментах кода использование *ESP* непосредственно имеет свои недостатки (большой размер инструкций и необходимость отслеживать изменение вершины стека).

Вот именно такое безобразие и называется "организация произвольного доступа к данным внутри стека".

#5. Перед тем, как мы пойдем дальше и разберемся-таки с локальными переменными – пара слов для особо продвинутых:

В защищённом режиме (или в реальном режиме с префиксами смены разрядности) базой может служить любой регистр, поэтому, если мы точно знаем состояние регистра *ESP* (т.е. мы точно знаем, сколько раз мы делали *push*, а сколько *pop*), то для доступа к фрейму можно использовать *ESP* (при этом индексы одной и той же перменной в разных местах процедуры могут отличаться

из-за промежуточных push/pop). Собсно, подобного рода оптимизацией занимаются, насколько я знаю, последние версии BC и VC. А в их "асмах" появилась директива "фреме-поин-оммисинс" как раз для таких извратов и предназначенная.

Однако, здесь есть недостатки по сравнению с использованием более-менее статичного *EBP*, как об этом было упомянуто выше: во-первых, с *[ESP]* инструкции длиннее, во-вторых, нужно быть очень аккуратным в подсчёте промежуточных push/pop, чтобы верно подставлять смещение до аргумента или переменной в *[ESP+xx]*(а ведь есть относительно непредсказуемые инструкции вида *push [esp+xxx]*). Наконец, поскольку индекс *xx* может постоянно меняться, поэтому использовать встроенные директивы типа *ARGS* или даже вручную расставленные *EQU* становится малореальным. Поэтому возможность использования *ESP* в качестве базы (при ручной кодогенерации – глюкалово полное) отнюдь не умаляет полезности *EBP*. **#6.** Ну вот, мы и подошли к самой интересной части марлезонского балета. Сейчас мы готовы проанализировать нашу программу на предмет того, чего она там вытворяет с локальными переменными.

```
:00401000 NumberOfCharsWritten= dword ptr -90h
:00401000 nNumberOfCharsToWrite= dword ptr -8Ch
:00401000 hConsoleInput      = dword ptr -88h
:00401000 hConsoleOutput    = dword ptr -84h
:00401000 Buffer            = byte ptr -80h
:00401000
:00401000 push     ebp
:00401001 mov     ebp, esp
:00401003 add     esp, 0FFFFFF70h
:00401009 push     offset aInputOutput ; lpConsoleTitle
:0040100E call    SetConsoleTitleA
:00401013 push     0FFFFFFF5h ; nStdHandle
:00401015 call    GetStdHandle
:0040101A mov     [ebp+hConsoleOutput], eax
:00401020 push     0 ; lpReserved
:00401022 lea     eax, [ebp+NumberOfCharsWritten]
:00401028 push     eax ; lpNumberOfCharsWritten
:00401029 push     11h ; nNumberOfCharsToWrite
:0040102B push     offset aTypeSomething ; lpBuffer
:00401030 push     [ebp+hConsoleOutput] ; hConsoleOutput
:00401036 call    WriteConsoleA
:0040103B push     0FFFFFFF6h ; nStdHandle
:0040103D call    GetStdHandle
:00401042 mov     [ebp+hConsoleInput], eax
:00401048 push     0 ; lpReserved
:0040104A lea     eax, [ebp+nNumberOfCharsToWrite]
:00401050 push     eax ; lpNumberOfCharsRead
:00401051 push     80h ; nNumberOfCharsToRead
:00401056 lea     eax, [ebp+Buffer]
:00401059 push     eax ; lpBuffer
:0040105A push     [ebp+hConsoleInput] ; hConsoleInput
:00401060 call    ReadConsoleA
:00401065 push     0 ; lpReserved
:00401067 lea     eax, [ebp+NumberOfCharsWritten]
:0040106D push     eax ; lpNumberOfCharsWritten
:0040106E push     0Ch ; nNumberOfCharsToWrite
:00401070 push     offset aYouTyped ; lpBuffer
:00401075 push     [ebp+hConsoleOutput] ; hConsoleOutput
:0040107B call    WriteConsoleA
:00401080 push     0 ; lpReserved
:00401082 lea     eax, [ebp+NumberOfCharsWritten]
:00401088 push     eax ; lpNumberOfCharsWritten
:00401089 push     [ebp+nNumberOfCharsToWrite] ; nNumberOfCharsToWrite
:0040108F lea     eax, [ebp+Buffer]
:00401092 push     eax ; lpBuffer
:00401093 push     [ebp+hConsoleOutput] ; hConsoleOutput
:00401099 call    WriteConsoleA
:0040109E push     7D0h ; dwMilliseconds
:004010A3 call    Sleep
:004010A8 push     0 ; uExitCode
:004010AA call    ExitProcess
```

Начнем с того, что полегче ;). Нетрудно заметить, что команда *addr* в применении к локальным переменным (в контексте *invoke*) , во всех случаях генерит следующий код:

```
lea eax,[ebp-X]
push eax
```

Предвидя грабли, на которые может натолкнуться начинающий, сразу оговорюсь, эти строки принципиально отличаются от

```
push [ebp-X]
```

Вы не поверите, но почему-то многие новички здесь путаются... Как, вы тоже? ;))

Первое – заносит в стек указатель на. А второе – значение. Всем медитировать!

Обратите внимание, что Ида услужливо вынесла вверх листинга блок констант, каждая из которых имеет отрицательное значение. Но мы-то с вами ещё со школы умеем решать простенькие задачки на сложение отрицательных чисел и без труда высчитаем, что система уравнений:

```
a = -84
b = x+a
```

имеет более чем тривиальное решение:

```
b = x-84
```

Хе-хе... слышала бы меня сейчас моя школьная учительница математики ;)).

#7. Очевидно, что *ebp*– это некая "точка отсчета", относительно которой адресуются локальные переменные. А что ж у нас в ***_ebp_***? Смотрим на начало процедуры, и ищем там строчки

```
:00401001 mov     ebp, esp
:00401003 add     esp, 0FFFFFFF70h
```

И медленно ("брюки превращаются... брюки превращаются...") приоткрываем завесу над тайной. "Дело в следующем": ассемблер смотрит, сколько мы задекларировали локальных переменных и какова размерность каждой. На основании этой информации определяется размер фрейма. В нашем случае задекларировано (смотрим исходник):

```
LOCAL InputBuffer[128] :BYTE ;буффер для ввода
LOCAL hOutPut          :DWORD ;хэндл для вывода
LOCAL hInput           :DWORD ;хэндл для ввода
LOCAL nRead            :DWORD ;прочитано байт
LOCAL nWriten          :DWORD ;напечатано байт
```

То есть 128 штук байт плюс 4 двойных слова по 4 байта в каждом. Итого для этого всего богатства нужно выделить 144 байта памяти.

Ну и замечательно! Сохраняем адрес вершины стека (память под переменные еще не отведена!) в регистре *ebp*. Теперь это у нас вовсе не вершина, а "точка отсчёта" для локальных переменных. А саму вершину стека передвигаем на 144 байта "вверх", в сторону младших адресов (там у нас будет область для хранения локальных переменных). Как видите, все очень просто. :)

Задание для медитации: чем будут отличаться генерируемые инструкции для директивы *addr* в случае, если *addr* будет применяться к аргументам процедуры, а не к локальным переменным.

Если вас смущает то, что для "поднятия планки" используется инструкция *add* и столь большое число, то вернитесь к выпуску о кодировании отрицательных чисел и особенностях дополнительного кода. Либо поставьте в Иде указатель на смущающее вас число 0FFFFFFF70h и нажмите на Ctrl+., после чего долго и упорно медитируйте. :)

Если подключить фантазию и пару раз протрассировать это безобразие под отладчиком, то получится такая картинка:

		0012FF20	
		0012FF24	
		0012FF28	
	esp	0012FF2C	БЕРИЩНА
[ebp - 90h]		0012FF30	nWriten
[ebp - 8Ch]		0012FF34	nRead
[ebp - 88h]		0012FF38	hInput
[ebp - 84h]		0012FF3C	hOutPut
[ebp - 80h]		0012FF40	InputBuffer[128]
		0012FF44	
		0012FF48	
		0012FF4C	
		0012FF50	
		0012FF54	
		0012FF58	
		0012FF5C	
		0012FF60	
		0012FF64	
		0012FF68	
		0012FF6C	
		0012FF70	
		0012FF74	
		0012FF78	
		0012FF7C	
		0012FF80	
		0012FF84	
		0012FF88	
		0012FF8C	
		0012FF90	
		0012FF94	
		0012FF98	
		0012FF9C	
		0012FFA0	
		0012FFA4	
		0012FFA8	
		0012FFAC	
		0012FFB0	
		0012FFB4	
		0012FFB8	
		0012FFBC	
	ebp	0012FFC0	БАЗА ФРЕЙМА
		0012FFC4	
		0012FFC8	
		0012FFCC	
		0012FFD0	

Распечатайте её и повесьте над своей кроватью. И пусть она время от времени напоминает вам о смерти...

OLE Variant 2 ANSI

Автор: murder

Дата: 2008-04-10

Исходники к статье (находится в архиве wasm_files.zip: files/0485/Variant2Str.7z)

Для кого эта статья?

Этот материал полезен для начинающих воинов дзена и может поведать о том как:

- конвертировать целые числа и float'ы единым алгоритмом
- узнать количество десятичных цифр в целом числе через логарифм
- избавиться от условных переходов при помощи команд stovXX, bt и модификации кода

Здесь также будет рассмотрен исходник автономной процедуры для преобразования типа variant в ANSI строку.

OLE Variant

Этот тип данных используется для передачи информации через COM. Переменная такого типа может содержать до 64 бит полезной информации (это могут быть числа, указатели или флаги). Изнутри эта структура выглядит так:

```
struct VARIANT
  vt  rw 1 ;тип данных
  wReserved1 rw 1 ;зарезервировано
  wReserved2 rw 1 ;зарезервировано
  wReserved3 rw 1 ;зарезервировано
  value  rq 1 ;данные
ends
```

Единственное поле которое заслуживает подробного описания это vt - поле типа данных. Оно может содержать следующие значения:

нотация Microsoft	нотация Delphi	hex значение	описание
vt_empty	varEmpty	00h	пустая структура
vt_null	varNull	01h	пустая структура
vt_i2	varSmallint	02h	целое со знаком 2 байта
vt_i4	varInteger	03h	целое со знаком 4 байта
vt_r4	varSingle	04h	число с плавающей точкой 4 байта
vt_r8	varDouble	05h	число с плавающей точкой 8 байт
vt_cy	varCurrency	06h	х.з.
vt_date	varDate	07h	число с плавающей точкой 8 байт (01.01.1900=2.0)
vt_bstr	varOleStr	08h	указатель на unicode строку
vt_dispatch	varDispatch	09h	указатель на интерфейс IDispatch
vt_error	varError	0Ah	код ошибки 4 байта
vt_bool	varBoolean	0Bh	1 бит
vt_variant	varVariant	0Ch	указатель на переменную типа variant

нотация Microsoft	нотация Delphi	hex значение	описание
vt_unknown	varUnknown	0Dh	х.з.
vt_i1	varShortInt	10h	целое со знаком 1 байт
vt_ui1	varByte	11h	целое без знака 1 байт
vt_ui2	varWord	12h	целое без знака 2 байта
vt_ui4	varLongWord	13h	целое без знака 4 байта
vt_i8	varInt64	14h	целое со знаком 8 байт
vt_clsid	varStrArg	48h	указатель на CLSID

Единый алгоритм преобразования чисел

Математический сопроцессор аппаратно поддерживает упакованные двоично-десятичные числа, следовательно можно переложить всю работу на инструкцию `fbstp`. для особо непродвинутых приведу формат упакованного BCD числа.

9	8	7	6	5	4	3	2	1	0
sxxxxxxx	17,16	15,14	13,12	11,10	9,8	7,6	5,4	3,2	1,0

Как видно из таблицы BCD - число содержит 18 тетрад при этом значение тетрады лежит в диапазоне от 0 до 9 (в общем это перевёрнутое текстовое отображение числа).

S - это знаковый бит (1 - число отрицательное)

Теперь перейдём к числам в формате с плавающей точкой. Для начала нужно преобразовать число с плавающей точкой в число с фиксированной точкой. Допустим у нас есть число:

87954.6465

В идеале его нужно привести к 18-значному виду вот так:

879546465000000000

Напрашивается формула $Y = X \cdot 10^{(18 - \text{длина целой части})}$. Вопрос: как найти длину целой части? Самый быстрый и маленький способ решить эту задачу - использовать логарифмы.

Итак определение:

Логарифм данного числа X при основании Y - это показатель степени, в которую нужно возвести число Y, чтобы получить X.

Можно проще:

$$X = Y^{(\log_Y X)}$$

То есть если взять логарифм по основанию 2 мы получим количество значащих битов в целом числе. Если же взять логарифм по основанию 10 - это будет количество десятичных символов в числе. Фактически сопроцессор способен рассчитывать только логарифмы по основанию 2 и для того чтобы найти логарифм с произвольным основанием применяется формула:

$$\log_Y X = (\log_2 X) / (\log_2 Y)$$

Заботливые инженеры intel предусмотрели ряд часто используемых констант, их можно получить при помощи инструкций `fildXXX` (Нас интересует `fild2t` - загрузить $\log_2 10$).

В результате всего вышесказанного вырисовывается следующий алгоритм:

1. Загрузить число в FPU
2. Подсчитать длину целой части

3. Умножить число на $10^{(18-\text{длина целой части})}$ (при этом лучше всего воспользоваться таблицей значений)
4. сохранить BCD
5. Если знаковый бит установлен вывести минус
6. Вывести целую часть
7. Вывести точку
8. Вывести оставшиеся цифры

Однако для умножения на 10^x необходимо использовать только точные целые значения (иначе будет жуткая погрешность). Сопроцессор оперирует числами с мантиссой в 64 бита и порядком в 15 бит, а этого не достаточно чтобы точно представить число 10^{18} . Умножение на 64 битовое целое число невозможно по той же причине. Поэтому придётся использовать 32 разрядные целые числа с диапазоном от 0 до 10^9 .

Следовательно число:

```
0.00000123456789
```

Будет урезано до:

```
1234
```

В связи с этим обстоятельством не получится выровнять число по старшему байту, то есть в большинстве случаев строка будет начинаться нулями. В конце строки также может появиться много нулей.

Оптимизация

Это несомненно самая дзенская часть статьи. Итак рассмотрим техники оптимизации, которые я применил от простого к сложному.

- Инструкции `movXX` или инструкции условной пересылки данных выглядят так:

```
cmovXX reg16,reg16
cmovXX reg32,reg32
```

Это позволяет копировать данные из одного регистра в другой при исполнении некоторого условия. Например чтобы поместить в `eax` наибольшее число из `eax` и `edx` нужно написать

```
cmp    eax,edx
cmovl  eax,edx
```

- Маски позволяют обнулить регистр или память при некотором условии. Маска принимает 2 значения: -1 и 0.

Вот стандартный способ применения маски из флага `cf`:

```
cmp ax,dx ;если ax>=dx то обнулить ax
sbb dx,dx
and ax,dx
```

Хотя можно получить маску из любого флага используя `setXX`:

```
xor    ax,ax ;если cx=dx, то ax=-1, иначе ax=0
cmp    cx,dx
setne  al
dec    ax
```

- Битовые множества могут существенно сократить количество сравнений и условных переходов.

```
ensemble db 01000110b ;множество чисел 1,2,6
-//-
movzx  eax,al
bt     [ensemble],eax
jc     quit            ;если (al=1) или (al=2) или (al=6) то выход
```


- Модификация кода. Самомодифицирующаяся программа похожа на поезд, который сам прокладывает себе рельсы

```
mov byte[dirrection],0FDh ;загружаем Маш. код инструкции std
rcr al,1 ;если al нечётный то
sbb byte[dirrection],0 ;std превращается в cld
dirrection:
rb 1
```

Можно даже например создать процедуру в стеке, которая сама выбросит себя от туда при завершении:

```
push 0004C2ACh ;маш-коды инструкций 0ACh-lodsb 0004C2h-ret 4
call esp ;вызываем процедуру из стека
```

Процедура Variant2Str

Процедура понимает следующие типы:

- varSmallint
- varInteger
- varSingle
- varDouble
- varOleStr
- varBoolean
- varByte
- varWord
- varLongWord
- varInt64

иначе выводит NaN

ограничения:

- длина числа не более 18 десятичных цифр
- требуется наличие WideCharToMultiByte в секции импорта
- логический тип представляется как да/нет

Параметры:

value - указатель на переменную типа variant

lpasz - указатель на выходную строку

```
procedure Variant2Str(value,lpasz: pointer);assembler;
const
power10:array[0..9] of longword=(1000000000, //таблица степеней десяти
100000000,
10000000,
1000000,
100000,
10000,
1000,
100,
10,
1);
opcodes:array[0..3] of byte=($df {fild word[ecx]}, //опкоды для загрузки данных
$db {fild dword[ecx]},
$d9 {fld dword[ecx]},
$dd {fld qword[ecx]});
NaNSet: integer=229436; //1111000000000111100b множество
//поддерживаемых типов

asm
finit
pusha
mov edi,lpasz
mov ecx,value
```

```

movzx eax,word[ecx]                //в eax тип переменной

cmp    al,$0B                      //если это vt_bool
jne    @notbool                   //то выводим да или нет
mov     edx,'ад'                   //и выходим
mov     eax,'теН'
test    [ecx+8],al
cmovne  eax,edx
mov     [edi],eax
popa
ret
@notbool:

cmp     al,8                      //если это vt_bstr
jne    @notstring                 //то воспользуемся
xor     eax,eax                   //функцией WideCharToMultiByte
push    eax                       //и выходим
push    eax
push    65536
push    edi
push    -1
push    [ecx+8]
push    eax
push    eax
call    WideCharToMultiByte
popa
ret
@notstring:

push    ax
fstcw   [esp]                     //устанавливаем максимальную точность FPU
or      word[esp],000001110000000b //и режим округления в меньшую сторону
fldcw   [esp]

add     ecx,8                     //теперь ecx указывает на поле данных
cmp     ax,20                     //если тип не поддерживается
ja      @NaN                      //выводим NaN и выходим
bt      [NaNSet],eax
jnc     @NaN

mov     edx,$00C30100             //подготавливаем опкоды
mov     dl,byte[opcodes+eax-2]    //для загрузки данных в зависимости от типа.
mov     bx,$29bf                 //в bx опкод для fild qword[ecx]
cmp     al,16
cmova   dx,bx

push    edx
call    esp                       //загружаем данные
pop     edx

fld1
fld     st(1)                     //вычисляем длину целой части числа
fabs
fyl2x
fldl2t
fdivp
fistp   dword[esp-4]

mov     eax,dword[esp-4]
add     eax,1                     //если log(X)=-1,
adc     eax,0                     //то X<1=>
dec     eax                       //=>длина числа=1 (eax=0)
cmp     al,17                     //если число слишком длинное то
ja      @NaN                      //выводим NaN и выходим
mov     edx,eax
sub     al,8
cmc
sbb     cl,cl                     //умножаем на 10^(18-длина числа)
and     al,cl                     //при этом если (18-длина числа)>9
fimul   dword[power10+eax*4]      //то умножаем на 10^9
fbstp   [edi]

```

```

fldcw [esp]                //восстанавливаем настройки FPU
pop    cx

neg     eax
add     eax,edx
lea     esi,[esp-26+eax+9]  //Адрес первого символа в буфере

push esi
push edi
mov     esi,edi
lea     edi,[esp-18]
mov     ecx,9
@unpack:xor  ax,ax          //распаковываем BCD число в буфер
        lodsb
        shl  ax,4
        shr  al,4
        add  ax,$3030      //и преобразуем его в ASCII
        stosw
loop @unpack
pop edi
pop esi

shl     byte[edi+9],1      //если число отрицательное
mov     byte[edi], '-'    //ставим минус
adc     edi,0

lea     eax,[esp-25]      //dx -  длина целой части
mov     ecx,esi           //ecx -  общая длина
sub     ecx,eax
std

@copy: lodsb              //переворачиваем строку из буфера
mov     [edi],al          //и отделяем целую часть
inc     edi               //от дробной точкой
mov     byte[edi], '.'
sub     dl,1
adc     edi,0

loop @copy
mov     [edi],cl          //завершающий ноль
scasb
mov     al,'0'            //удаляем лишние нули
mov     cl,18             //в конце строки
repe    scasb
mov     byte[edi+2],ch
cmp     byte[edi+1],47     //если последний символ - это точка
cmc                                           //то удаляем её
sbb     al,al
and     byte[edi+1],al
cld
popa

ret
@NaN:
mov     dword[edi], 'NaN'
pop     ax
popa
end;

```

2. Уроки Iczelion'a

Win32 API. Урок 1. Основы

Автор: Iczelion, пер. Aquila

Дата: 2002-05-01

Этот tutorial предполагает, что читатель знает, как использовать MASM. Если нет, то для начала скачайте win32asm и прочитайте текст, входящий в состав этого пакета, и только затем продолжите чтение моего бреда.

Хорошо. Будем считать, что вы это сделали ;) Давайте приступим.

ТЕОРИЯ, МАТЬ СКЛЕРОЗА

Win32 программы выполняются в защищенном режиме, который доступен начиная с 80286. Но 80286 теперь история. Поэтому мы предполагаем, что имеем дело только с 80386 и его шелудивыми потомками.

Как известно, каждую Win32 программу Windows запускает в отдельном виртуальном пространстве. Это означает, что каждая Win32 программа будет иметь 4-х гигабайтовое адресное пространство, но вовсе не означает, что каждая программа имеет 4 гигабайта физической памяти, а только то, что программа может обращаться по любому адресу в этих пределах. А Windows сделает все необходимое, чтобы сделать память, к которой программа обращается, "существующей". Конечно, программа должна придерживаться правил, установленных Windows, или это вызовет General Protection Fault.

Каждая программа одна в своем адресном пространстве, в то время как в Win16 дело обстоит не так. Все Win16 программы могут *видеть* друг друга, что невозможно в Win32. Эта особенность помогает снизить шанс того, что одна программа запишет что-нибудь поверх данных или кода другой программы.

Модель памяти также коренным образом отличается от 16-битных программ. Под Win32, мы больше не должны беспокоиться о моделях памяти или сегментах! Теперь только одна модель памяти: плоская, без 64-ти килобайтных сегментов. Теперь память - это большое последовательное 4-х гигабайтовое пространство. Это также означает, что можно не париться с сегментными регистрами, зато можно использовать любой сегментный регистр для адресации к любой точке памяти. Это ОГРОМНОЕ подспорье для программистов. Это то, что делает программирование на ассемблере под Win32 таким же простым, как C ;)

При программировании под Win32 вы должны помнить несколько важных правил. Самое важное следующее: Windows использует esi, edi, ebp и ebx для своих целей и не ожидает, что вы измените значение этих регистров. Если же вы используете какой-либо из этих четырех регистров в вызываемой функции, то не забудьте восстановить их перед возвращением управления Windows.

СУТЬ, МАТЬ ШИЗОФРЕНИИ

Вот шаблон программы. Если что-то из кода вы не понимаете, не паникуйте. Ибо кода здесь в общем-то, пока что и нету.

```
.386
.MODEL Flat, STDCALL
.DATA
    <Ваша инициализированные данные>
    .....
.DATA?
    <Ваши неинициализированные данные>
    .....
.CONST
    <Ваши константы>
    .....
.CODE
    <метка>
    <Ваш код>
    .....
end <метка>
```

Вот и все! Давайте проанализируем этот "каркас".

.386

Это ассемблерная директива, указующая ассемблеру использовать набор операций для процессора 80386. Можно использовать и .486, .586, но самый безопасный выбор - это указывать .386. Также есть два практически идентичных выбора для каждого варианта CPU. .386/.386p, .486/.486p. Эти "p"-версии необходимы только в тех случаях, когда ваша программа использует привилегированные инструкции, то есть инструкции, зарезервированные процессором/операционной системой для защищенного режима. Они могут быть использованы только в защищенном коде, например, vdx-драйверами. Как правило, ваши программы будут работать в непривилегированном режиме, так что лучше использовать не-"p" версии.

.MODEL FLAT, STDCALL

.MODEL - это ассемблерная директива, определяющая модель памяти вашей программы. Под Win32 есть только одна модель - плоская.

STDCALL говорит MASM'у о порядке передачи параметров, слева направо или справа налево, а также о том, кто уравнивает стек после того как функция вызвана.

Под Win16 существует два типа передачи параметров, C и PASCAL. По C-договоренности, параметры передаются справа налево, то есть самый правый параметр кладется в стек первым. Вызывающий должен уравнивать стек после вызова. Например, при вызове функции с именем foo(int first_param, int second_param, int third_param), используя C-передачу параметров, ассемблерный код будет выглядеть так:

```
push [third_param] ; Положить в стек третий параметр
push [second_param] ; Следом - второй
push [first_param] ; И, наконец, первый
call foo
add sp, 12 ; Вызывающий уравнивает стек
```

PASCAL-передача параметров - это C-передача наоборот. Согласно ей, параметры передаются слева направо и вызываемый должен уравнивать стек.

Win16 использует этот порядок передачи данных, потому что тогда код программы становится меньше. C-порядок полезен, когда вы не знаете, как много параметров будут переданы функции, как например, в случае sprintf(), когда функция не может знать заранее, сколько параметров будут положены в стек, так что она не может его уравнивать. STDCALL - это гибрид C и PASCAL. Согласно ему, данные передаются справа налево, но вызываемый ответственен за выравнивание стека.

Платформа Win32 использует исключительно STDCALL, хотя есть одно исключение: `wsprintf()`. Вы в последнем случае должны следовать сишному порядку вызова.

`.DATA`

`.DATA?`

`.CONST`

`.CODE`

Все четыре директивы - это то, что называется секциями. Вы помните, что в Win32 нет сегментов? Но вы можете поделить пресловутое адресное пространство на логические секции. Начало одной секции отмечает конец предыдущей. Есть две группы секций: данных и кода.

`.DATA` - Эта секция содержит инициализированные данные вашей программы.

`.DATA?` - Эта секция содержит неинициализированные данные вашей программы. Иногда вам нужно только *предварительно* выделить некоторое количество памяти, но вы не хотите инициализировать ее. Эта секция для этого и предназначена. Преимущество неинициализированных данных следующее: они не занимают места в исполняемом файле. Например, если вы хотите выделить 10.000 байт в вашей `.DATA?` секции, ваш exe-файл не увеличится на 10kb. Его размер останется таким же. Вы всего лишь говорите компилятору, сколько места вам нужно, когда программа загрузится в память.

`.CONST` - Эта секция содержит объявления констант, используемых программой. Константы не могут быть изменены ей. Это всего лишь *константы*.

Вы не обязаны задействовать все три секции. Объявляйте только те, которые хотите использовать.

Есть только одна секция для кода: `.CODE`, там где содержится весь код.

```
<метка>
.....
end <метка>
```

где `<метка>` - любая произвольная метка, устанавливающая границы кода. Обе метки должны быть идентичны. Весь код должен располагаться между

```
<метка>
```

и

```
end <метка>
```

Win32 API. Урок 2. MessageBox

Автор: lczelion, пер. Aquila

Дата: 2002-05-02

В этом уроке мы создадим полнофункциональную Windows программу, которое выводит сообщение "Win32 assembly is great!".

Скачайте пример [здесь](#) (находится в архиве `wasm_files.zip: files/recovered/1001002/tut02.zip`).

ТЕОРИЯ, МАТЬ СКЛЕРОЗА

Windows предоставляет огромное количество ресурсов Windows-программам через *Windows API* (Application Programming Interface). Windows API - это большая коллекция очень полезных функций, располагающихся непосредственно в операционной системе и готовых для использования программами. Эти функции находятся в нескольких динамически подгружаемых библиотек (DLLs), таких как *kernel32.dll*, *user32.dll* и *gdi32.dll*. *Kernel32.dll* содержит API функции, взаимодействующие с памятью и управляющие процессами. *User32.dll* контролирует пользовательский интерфейс. *Gdi32.dll* ответственен за графические операции. Кроме этих трех "основных", существуют также другие dll, которые вы можете использовать, при условии, что обладаете достаточным количеством информации о нужных API функциях. Windows программы динамически подсоединяется к этим библиотекам, то есть код API функций не включается в исполняемый файл. Информация находится в библиотеках импорта. Вы должны слинковать ваши программы с правильными библиотеками импорта, иначе они не смогут найти эти функции. Когда Windows программа загружается в память, Windows читает информацию, сохраненную в программе. Эта информация включает имена функций, которые программа использует и DLL-ок, в которых эти функции располагаются. Когда Windows находит подобную информацию в программе, она вызывает библиотеки и исправляет в программе вызовы этих функций, так что контроль всегда будет передаваться по правильному адресу.

Существует две категории API функций: одна для ANSI и другая для Unicode. На конце имен API функций для ANSI стоит "A", например, `MessageBoxA`. В конце имен функций для Unicode находится "W". Windows 95 от природы поддерживает ANSI и Windows NT Unicode. Мы обычно имеем дело с ANSI строками (массивы символов, оканчивающиеся NULL-ом. Размер ANSI-символа - 1 байт. В то время как ANSI достаточна для европейских языков, она не поддерживает некоторые восточные языки, в которых есть несколько тысяч уникальных символов. Вот в этих случаях в дело вступает Unicode. Размер символа UNICODE - 2 байта, и поэтому может поддерживать 65536 уникальных символов.

Но по большей части, вы будете использовать include-файл, который может определить и выбрать подходящую для вашей платформы функцию. Просто обращайтесь к именам API функций без постфикса.

ПРАКТИКА, МАТЬ ШИЗОФРЕНИИ

Я приведу голый скелет программы ниже. Позже мы разберем его.

```
.386
.model flat, stdcall

.data
.code
start:
end start
```

Выполнение начинается с первой инструкции, следующей за меткой, установленной после конца директив. В вышеприведенном каркасе выполнение начинается непосредственно после метки

'start'. Будут последовательно выполняться инструкция за инструкцией, пока не встретится операция плавающего контроля, такая как `jmp`, `jne`, `je`, `ret` и так далее. Эти инструкции перенаправляют поток выполнения другим инструкциям. Когда программа выходит в Windows, ей следует вызвать API функцию `ExitProcess`.

```
ExitProcess proto uExitCode:DWORD
```

Строка выше называется прототипом функции. Прототип функции указывает ассемблеру/линкеру атрибуты функции, так что он может делать для вас проверку типов данных. Формат прототипа функции следующий:

```
ИмяФункции PROTO [ИмяПараметра]:ТипДанных, [ИмяПараметра]:ТипДанных, ...
```

Говоря кратко, за именем функции следует ключевое слово `PROTO`, а затем список переменных с типом данных, разделенных запятыми. В приведенном выше примере с `ExitProcess`, эта функция была определена как принимающая только один параметр типа `DWORD`. Прототипы функций очень полезны, когда вы используете высокоуровневый синтаксический вызов - `invoke`. Вы можете считать об `invoke` как обычный вызов с проверкой типов данных. Например, если вы напишете:

```
call ExitProcess
```

Линкер уведомит вас, что вы забыли положить в стек двойное слово. Я рекомендую вам использовать `invoke` вместо простого вызова. Синтакс `invoke` следующий:

```
invoke выражение [, аргументы]
```

Выражение может быть именем функции или указателем на функцию. Параметры функции разделены запятыми.

Большинство прототипов для API-функций содержатся в `include`-файлах. Если вы используете `hutch`'евский `MASM32`, они будут находиться в директории `MASM32/INCLUDE`. Файлы подключения имеют расширение `.inc` и прототипы функций DLL находятся в `.inc` файле с таким же именем, как и у этой DLL. Например, `ExitProcess` экспортируется `kernel32.lib`, так что прототип `ExitProcess` находится в `kernel32.inc`. Вы также можете создать прототипы для ваших собственных функций. Во всех моих экземплярах я использую `hutch`'евский `windows.inc`, который вы можете скачать с <http://win32asm.cjb.net> Возвращаясь к `ExitProcess`: параметр `uExitCode` - это значение, которое программа вернет Windows после окончания программы. Вы можете вызвать `ExitProcess` так:

```
invoke ExitProcess, 0
```

Поместив эту строку непосредственно после стартовой метки, вы получите Win32 программу, немедленно выходящую в Windows, но тем не менее полнофункциональную.

```
.386
.model flat, stdcall
option casemap:none

include \masm32\include\windows.inc
include \masm32\include\kernel32.inc
includelib \masm32\lib\kernel32.lib
.data

.code
start:
    invoke ExitProcess, 0
end start
```

`option casemap:none` говорит MASM сделать метки чувствительными к регистрам, то есть `ExitProcess` и `exitprocess` - это различные имена. Отметьте новую директиву - `include`. После нее следует имя файла, который вы хотите вставить в то место, где эта директива располагается. В примере выше, когда MASM обрабатывает линию `include \masm32\include\windows.inc`, он открывает `windows.inc`, находящийся в директории `\MASM32\INCLUDE`, и далее анализирует содержимое `windows.inc` так, как будто вы "вклеили" подключаемый файл. Хатчевский `windows.inc`

содержит в себе определения констант и структур, которые вам могут понадобиться для программирования под Win32. Этот файл не содержит в себе прототипов функций. Windows.inc ни в коем случае не является исчерпывающим и всеобъемлющим. Hutch и я пытаемся заполнить его как можно большим количеством констант и структур, но есть еще дохрена, чего мы еще не сделали. Он постоянно обновляется. Заходите на хатчевскую и мою странички за свежими апдейтами ;) Из windows.inc, ваша программа будет брать определения констант и структур. Что касается прототипов функций, вы должны подключить другие include-файлы. Они находятся в директории \masm32\include.

В вышеприведенном примере, мы вызываем функцию, экспортированную из kernel32.dll, для чего мы должны подключить прототипы функций из kernel32.dll. Этот файл - kernel32.inc. Если вы открываете его текстовым редактором, вы увидите, что он состоит из прототипов функций из соответствующей dll. Если вы не подключите kernel32.inc, вы все еще можете вызвать ExitProcess, но уже с помощью ассемблерной команды call. Вы не сможете вызвать эту функцию с помощью invoke. Дело вот в чем: для того, чтобы вызвать функцию через invoke, вы должны поместить в исходном коде ее прототип. В примере выше, если вы не подключите kernel32.inc, вы можете определить прототип для ExitProcess где-нибудь до вызова этой функции и это будет работать. Файлы подключения нужны для того, что избавить вас от лишней работы и вам не пришлось набирать все прототипы самим.

Теперь мы встречаем новую директиву - includelib. Она работает не так, как include. Это всего лишь способ сказать ассемблеру какие библиотеки использует ваша программа должна прилинковать. Хотя вы вовсе не обязаны использовать именно этот метод. Вы можете указать имена библиотек импорта к командной строке при запуске линкера, но поверьте мне, это весьма скучно и утомительно, да и командная строка может вместить максимум 128 символов.

Теперь возьмите весь исходный текст примера этого урока, сохраните его как msgbox.asm и сассемблируйте его так:

```
ml /c /coff /Cp msgbox.asm
```

/c говорит MASM'у создать .obj файл в формате COFF. MASM использует вариант COFF (Common Object File Format), использующийся под Unix, как его собственный объектный и исполняемый формат файлов.

/Cp говорит MASM'у сохранять регистр имен, заданных пользователем. Если вы используете hutch'евский MASM32 пакет, вы можете вставить "option casemap:none" в начале вашего исходника, сразу после директивы .model, чтобы добиться того же эффекта.

После успешной компиляции msgbox.asm, вы получите msgbox.obj. Это объектный файл, от которого один шаг до исполняемого. Obj содержит инструкции/данные в двоичной форме. Отсутствуют только необходимая корректировка адресов, которая проводится линкером.

Теперь сделайте следующее:

```
link /SUBSYSTEM:WINDOWS /LIBPATH:c:\masm32\lib msgbox.obj
```

/SUBSYSTEM:WINDOWS информирует линкер о том, какого вида является будущий исполняемый модуль.

/LIBPATH:<путь к библиотекам импорта> говорит линкеру, где находятся библиотеки импорта. Если вы используете MASM32, они будут в MASM32\lib.

Линкер читает объектный файл и корректирует его, используя адреса, взятые из библиотек импорта. После окончания линковки вы получите файл msgbox.exe. Запустите его. Вы увидите, что она ничего не делает.

Да, мы не поместили в код ничего не интересного. Но тем не менее полноценная Windows программа. И посмотрите на размер! На моем PC - 1.536 байт.

Теперь мы готовы создать окно с сообщением. Прототип функции, которая нам для этого необходима следующая:

```
MessageBox PROTO hwnd:DWORD, lpText:DWORD, lpCaption:DWORD, uType:DWORD
```

Thwnd - это хэндл родительского окна. Вы можете считать хэндл числом, представляющим окно, к которому вы обращаетесь. Его значение для вас не важно. Вы только должны знать, что оно представляет окно. Когда вы захотите сделать что-нибудь с окном, вы должны обратиться к нему, используя его хэндл.

lpText - это указатель на текст, который вы хотите отобразить в клиентской части окна сообщения. Указатель - это адрес чего-либо. Указатель на текстовую строку == адрес этой строки.

lpCaption - это указатель на заголовок окна сообщения.

uType устанавливает иконку, число и вид кнопок окна.

Давайте изменим msgbox.asm для отображения сообщения.

```
.386

.model flat,stdcall
option casemap:none
include \masm32\include\windows.inc
include \masm32\include\kernel32.inc

includelib \masm32\lib\kernel32.lib
include \masm32\include\user32.inc
includelib \masm32\lib\user32.lib

.data
MsgBoxCaption db "Iczelion Tutorial No.2",0
MsgBoxText    db "Win32 Assembly is Great!",0

.code
start:

invoke MessageBox, NULL, addr MsgBoxText, addr MsgBoxCaption, MB_OK
invoke ExitProcess, NULL
end start
```

Скомпилируйте и запустите. Вы увидите окошко с сообщением "Win32 Assembly is great!".

Давайте снова взглянем на исходник.

Мы определили две оканчивающиеся NULL'ом строки в секции .data. Помните, что каждая ANSI строка в Windows должна оканчиваться NULL'ом (0 в шестнадцатичной системе). Мы используем две константы, NULL и MB_OK. Эти константы прописаны в windows.inc, так что вы можете обратиться к ним, указав их имя, а не значение. Это улучшает читаемость кода. Оператор addr используется для передачи адреса метки (и не только) функции. Он действителен только в контексте директивы invoke. Вы не можете использовать его, чтобы присвоить адрес метки регистру или переменной, например. В данном примере вы можете использовать offset вместо addr. Тем не менее, есть некоторые различия между ними.

1. addr не может быть использован с метками, которые определены впереди, а offset может. Например, если метка определена где-то дальше в коде, чем строка с invoke, addr не будет работать.

```
invoke MessageBox,NULL, addr MsgBoxText,addr MsgBoxCaption,MB_OK
.....
MsgBoxCaption db "Iczelion Tutorial No.2",0
MsgBoxText    db "Win32 Assembly is Great!",0
```

MASM доложит об ошибке. Если вы используете offset вместо addr, MASM без проблем скомпилирует указанный отрывок кода. 2. Addr поддерживает локальные переменные, в то время как offset нет. Локальная переменная - это всего лишь зарезервированное место в стеке. Вы только знаете ее адрес во время выполнения программы. Offset интерпретируется во время компиляции ассемблером, поэтому неудивительно, что он не поддерживает локальные переменные. Addr же работает с ними, потому что ассемблер сначала проверяет - глобальная переменная или локальная. Если она глобальная, он помещает адрес этой переменной в объектный файл. В этом

случае оператор работает как `offset`. Если это локальная переменная, компилятор генерирует следующую последовательность инструкций перед тем как будет вызвана функция:

```
lea eax, LocalVar  
push eax
```

Учитывая, что `lea` может определить адрес метки в "рантайме", все работает прекрасно.

Win32 API. Урок 3. Простое окно

Автор: lczelion, пер. Aquila

Дата: 2002-05-03

В этом уроке мы создадим Windows программы, которая отображает полнофункциональное окно на рабочем столе.

Скачайте файл примера [здесь](#) (находится в архиве `wasm_files.zip: files/recovered/1001003/tut03.zip`).

ТЕОРИЯ

Windows программы для создания графического интерфейса пользуются функциями API. Этот подход выгоден как пользователям, так и программистам. Пользователям это дает то, что они не должны изучать интерфейс каждой новой программы, так как Windows программы похожи друг на друга. Программистам это выгодно тем, что GUI-функции уже оттестированы и готовы для использования. Обратная сторона - это возросшая сложность программирования. Чтобы создать какой-нибудь графический объект, такой как окно, меню или иконка, программист должен следовать строгим правилам. Но процесс программирования можно облегчить, используя модульное программирование или ООП-философию. Я вкратце изложу шаги, требуемые для создания окна:

1. Взять хэндл вашей программы (обязательно)
2. Взять командную строку (не нужно до тех пор, пока программе не потребуется ее проанализировать)
3. Зарегистрировать класс окна (необходимо, если вы не используете один из предопределенных классов окна, таких как `MessageBox` или диалоговое окно)
4. Создайте окно (необходимо)
5. Отобразите его на экране
6. Обновить содержимое экрана на окне
7. Запустите бесконечный цикл, в котором будут проверяться сообщения от операционной системы.
8. Прибывающие сообщения передаются специальной функции, отвечающая за обработку окна
9. Выйти из программы, если пользователь закрывает окно.

Как вы можете видеть, структура Windows программы довольно сложна по сравнению с досовской программой. Но мир Windows разительно отличается от мира DOS'a. Windows программы должны быть способными мирно сосуществовать друг с другом. Они должны следовать более строгим правилам. Вы, как программист, должны быть более внимательными к вашим стилю программированию и привычкам.

СУТЬ

Ниже приведен исходник нашей программы простого окна. Перед тем как углубиться в описание деталей программирования на ассемблере под Win32, я покажу вам несколько трюков, могущие облегчить программирование.

Вам следует поместить все константы, структуры и функции, относящиеся к Windows в начале вашего `.asm` файла. Это сэкономит вам много сил и времени. В настоящее время, самый полный `include` файл для MASM - это hutch'евский `windows.inc`, который вы можете скачать с его или моей страницы. Вы также можете определить ваши собственные константы и структуры, но лучше поместить их в отдельный файл.

Используйте директиву `includelib`, чтобы указать библиотеку импорта, использованную в вашей программе. Например, если ваша программа вызывает `MessageBox`, вам следует поместить строку

"`includelib user32.lib`" в начале кода. Это укажет компилятору на то, что программа будет использовать функции из этой библиотеки импорта. Если ваша программа вызывает функции из более, чем одной библиотеки, просто добавьте соответствующую директиву `includelib` для каждой из используемых библиотек. Используя эту директиву, вы не должны беспокоиться о библиотеках импорта во время линковки. Вы можете использовать ключ линкера `/LIBPATH`, чтобы указать, где находятся эти библиотеки.

Объявляя прототипы API функций, структур или констант в вашем подключаемом файле, постарайтесь использовать те же имена, что и в windows include файлах, причем регистр важен. Это избавит вас от головной боли в будущем.

Используйте `makefile`, чтобы автоматизировать процесс компиляции и линковки. Это избавит вас лишних усилий. (Лично я использую `wmake` из пакета Watcom C/C++ - переводчик.)

```
.386
.model flat,stdcall

option casemap:none
include \masm32\include\windows.inc
include \masm32\include\user32.inc
includelib \masm32\lib\user32.lib ; calls to functions in user32.lib and kernel32.lib
include \masm32\include\kernel32.inc
includelib \masm32\lib\kernel32.lib

WinMain proto :DWORD,:DWORD,:DWORD,:DWORD

.DATA
; initialized data

ClassName db "SimpleWinClass",0 ; Имя нашего класса окна
AppName db "Our First Window",0 ; Имя нашего окна

.DATA?
; Неинициализируемые данные
hInstance HINSTANCE ? ; Хэндл нашей программы
CommandLine LPSTR ?
.CODE
; Здесь начинается наш код
start:
invoke GetModuleHandle, NULL ; Взять хэндл программы
; Под Win32, hmodule==hinstance mov hInstance,eax
mov hInstance,eax

invoke GetCommandLine ; Взять командную строку. Вы не обязаны
; вызывать эту функцию ЕСЛИ ваша программа не обрабатывает командную строку.
mov CommandLine,eax
invoke WinMain, hInstance,NULL,CommandLine, SW_SHOWDEFAULT ; вызвать основную функцию
invoke ExitProcess, eax ; Выйти из программы.
; Возвращаемое значение, помещаемое в eax, берется из WinMain'a.

WinMain proc
hInst:HINSTANCE,hPrevInst:HINSTANCE,CmdLine:LPSTR,CmdShow:DWORD
LOCAL wc:WNDCLASSEX ; создание локальных переменных в стеке
LOCAL msg:MSG
LOCAL hwnd:HWND

mov wc.cbSize,SIZEOF WNDCLASSEX ; заполнение структуры wc
mov wc.style, CS_HREDRAW or CS_VREDRAW
mov wc.lpfnWndProc, OFFSET WndProc
mov wc.cbClsExtra,NULL

mov wc.cbWndExtra,NULL
push hInstance
pop wc.hInstance
mov wc.hbrBackground,COLOR_WINDOW+1

mov wc.lpszMenuName,NULL
```

```

mov     wc.lpszClassName,OFFSET ClassName
invoke  LoadIcon,NULL,IDI_APPLICATION
mov     wc.hIcon,eax

mov     wc.hIconSm,eax
invoke  LoadCursor,NULL,IDC_ARROW
mov     wc.hCursor,eax
invoke  RegisterClassEx, addr wc ; регистрация нашего класса окна
invoke  CreateWindowEx,NULL,\
        ADDR ClassName,\
        ADDR AppName,\
        WS_OVERLAPPEDWINDOW,\
        CW_USEDEFAULT,\
        CW_USEDEFAULT,\
        CW_USEDEFAULT,\
        CW_USEDEFAULT,\
        NULL,\
        NULL,\
        hInst,\
        NULL
mov     hwnd,eax

invoke  ShowWindow, hwnd,CmdShow ; отобразить наше окно на десктопе
invoke  UpdateWindow, hwnd ; обновить клиентскую область

.WHILE TRUE ; Enter message loop
    invoke  GetMessage, ADDR msg,NULL,0,0
.BREAK .IF (!eax)
    invoke  TranslateMessage, ADDR msg
    invoke  DispatchMessage, ADDR msg
.ENDW
mov     eax,msg.wParam ; сохранение возвращаемого значения в eax
ret

WinMain endp

WndProc proc hwnd:HWND, uMsg:UINT, wParam:WPARAM, lParam:LPARAM

    .IF uMsg==WM_DESTROY ; если пользователь закрывает окно
        invoke  PostQuitMessage,NULL ; выходим из программы
    .ELSE
        invoke  DefWindowProc,hwnd,uMsg,wParam,lParam ; Дефолтная функция обработки окна
        ret
    .ENDIF
    xor     eax,eax

    ret
WndProc endp

end start

```

АНАЛИЗ

Вы можете быть ошарашены тем, что простая Windows программа требует так много кода. Но большая его часть - это *шаблонный* код, который вы можете копировать из одного исходника в другой. Или, если вы хотите, вы можете скомпилировать часть этого кода в библиотеку, которая будет использоваться как прологовый и эпилоговый код. Вы можете писать код уже только в функции WinMain. Фактически, это то, что делают С-компилятор. Они позволяют вам писать WinMain без беспокойства о коде, который должен быть в каждой программе. Единственная хитрость это то, что вы должны написать функцию по имени WinMain, иначе С-компиляторы не смогут скомбинировать ваш код с проловым и эпиловым. Такого ограничения нет в ассемблерном программировании. Вы можете назвать эту функцию так ка вы хотите. Готовьтесь! Это будет долгий, долгий tutorial. Давайте же проанализируем эту программу до самого конца.

```

.386
.model flat,stdcall
option casemap:none

```

```

WinMain proto :DWORD,:DWORD,:DWORD,:DWORD

include \masm32\include\windows.inc
include \masm32\include\user32.inc

include \masm32\include\kernel32.inc
includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib

```

Первые три линии обязательны в высшей степени. .386 говорит MASM'у, что намереваемся использовать набор инструкций процессора 80386 в этой программе. .Model flat, stdcall говорит MASM'у, что наша программа будет использовать плоскую модель памяти. Также мы использовать передачу параметров типа STDCALL по умолчанию. Следом идет прототип функции WinMain. Перед тем, как мы вызовем в дальнейшем эту функцию, мы должны сначала определить ее прототип.

Мы должны подключить windows.inc в начале кода. Он содержит важные структуры и константы, которые потребуются нашей программе. Этот файл всего лишь текстовый файл. Вы можете открыть его с помощью любого текстового редактора. Пожалуста заметьте, что windows.inc не содержит все структуры и константы (пока). Hutch и я работаем над этим. Вы можете добавить в него что-то новое, если этого там нет. Наша программа вызывает API функции, находящиеся в user32.dll (CreateWindowEx, RegisterWindowClassEx, например) и kernel32.dll (ExitProcess), поэтому мы должны прописать пути к этим двум библиотекам. Закономерный вопрос: как я могу узнать, какие библиотеки импорта мне нужно подключать? Ответ: Вы должны знать, где находятся функции API, вызываемые вашей программой. Например, если вы вызываете API функцию в gdi32.dll, вы должны подключить gdi32.lib. Это - подход MASM'a. Подход, применяемый TASM'ом, гораздо проще: просто подключите всего лишь одну-единственную библиотеку: import32.lib.

```

.DATA
    ClassName db "SimpleWinClass",0
    AppName   db "Our First Window",0

.DATA?
    hInstance HINSTANCE ?
    CommandLine LPSTR ?

```

Далее идет секции "DATA".

В .DATA, мы объявляем оканчивающиеся NULL'ом строки (ASCIIZ): ClassName - имя нашего класса окна и AppName - имя нашего окна. Отметьте, что обе переменные проинициализированны. В .DATA? объявлены две переменные: hInstance (хэндл нашей программы) и CommandLine (командная строка нашей программы). Незнакомые типы данных - HINSTANCE и LPSTR - на самом деле новые имена для DWORD. Вы можете увидеть их в windows.inc. Обратите внимание, что все переменные в этой секции не инициализированны, так как они не должны содержать какое-то определенное значение при загрузке программа, но мы хотим зарезервировать место на будущее.

```

.CODE

start:
    invoke GetModuleHandle, NULL
    mov   hInstance,eax
    invoke GetCommandLine

    mov   CommandLine,eax
    invoke WinMain, hInstance,NULL,CommandLine, SW_SHOWDEFAULT
    invoke ExitProcess,eax
    ....

end start

```

.CODE содержит все ваши инструкции. Ваш код должен располагаться между <стартовая метка>: и end <стартовая метка>. Имя метки несущественно. Вы можете назвать ее как пожелаете, до тех

пор, пока оно уникально и не нарушает правила именования в MASM'e.

Наша первая инструкция - вызов `GetModuleHandle`, чтобы получить хэндл нашей программы. Под Win32, instance хэндл и module хэндл - одно и то же. Вы можете воспринимать хэндл программы как ее ID. Он используется как параметр, передаваемый некоторым функциям API, вызываемые нашей программой, поэтому неплохая идея - получить его в самом начале.

Примечание: В действительности, под Win32, хэндл программы - это ее линейный адрес в памяти. По возвращению из Win32 функции, возвращаемое ею значение находится в `eax`. Все другие значения возвращаются через переменные, переданные в параметрах функции.

Функция Win32, вызываемая вами, практически всегда сохранит значения сегментных регистров и регистров `ebx`, `edi`, `esi` и `ebp`. Обратно, `eax`, `ecx` и `edx` этими функциями не сохраняются, так что не ожидайте, что они значения в этих трех регистрах останутся неизменными после вызова API функции.

Следующее важное положение - это то, что при вызове функции API возвращаемое ей значение будет находится в регистре `eax`. Если какая-то из ваших функций будет вызываться Windows, вы также должны играть по правилам: сохраняйте и восстанавливайте значения используемых сегментных регистров, `ebx`, `edi`, `esi` и `ebp` до выхода из функции, или же ваша программа повиснет очень быстро, включая функцию обработки сообщений к окну, да и все остальные тоже. Вызов `GetCommandLine` не нужен, если ваша программа не обрабатывает командную строку. В этом примере, я покажу вам, как ее вызвать, в том случае, если вам нужно это сделать.

Далее идет вызов `WinMain`. Она получает четыре параметра: хэндл программы, хэндл предыдущего экземпляра программы, командную строку и состояние окна при первом появлении. Под Win32 нет такого понятия, как предыдущий экземпляр программы. Каждая программа одна-единешенька в своем адресном пространстве, поэтому значение переменной `hPrevInst` всегда 0. Это пережиток времен Win16, когда все экземпляры программы запускались в одном и том же адресном пространстве, и экземпляр мог узнать, был ли запущены еще копии этой программы. Под Win16, если `hPrevInst` равен `NULL`, тогда этот экземпляр является первым.

Примечание: Вы не обязаны объявлять функцию `WinMain`. На самом деле, вы совершенно свободны в этом отношении. Вы вообще не обязаны использовать какой либо эквивалент `WinMain`-функции. Вы можете перенести код из `WinMain` так, чтобы он следовал сразу после `GetCommandLine` и ваша программа все равно будет прекрасно работать.

По возвращению из `WinMain`, `eax` заполняется значением кода выхода. Мы передаем код выхода как параметр функции `ExitProcess`, которая завершает нашу программу.

```
WinMain proc
```

```
Inst:HINSTANCE,hPrevInst:HINSTANCE,CmdLine:LPSTR,CmdShow:DWORD
```

В вышенаписанной строке объявление функции `WinMain`. Обратите внимание на параметры. Вы можете обращаться к этим параметрам, вместо того, чтобы манипулировать со стеком. В добавление, MASM будет генерировать прологовый и эпилоговый код для функции. Так что мы не должны беспокоиться о состоянии стека при входе и выходе из функции.

```
LOCAL wc:WNDCLASSEX
```

```
LOCAL msg:MSG
```

```
LOCAL hwnd:HWND
```

Директива `LOCAL` резервирует память из стека для локальных переменных, использованных в функции. Все директивы `LOCAL` должны следовать непосредственно после директивы `PROC`. После `LOCAL` сразу идет `<имя_переменной>:<тип переменной>`. То есть `LOCAL wc:WNDCLASSEX` говорит MASM'у зарезервировать память из стека в объеме, равном размеру структуры `WNDCLASSEX` для переменной размером `wc`. Мы можем обратиться к `wc` в нашем коде без всяких

трудностей, связанных с манипуляцией со стеком. Это действительно ниспослано нам свыше, я думаю. Обратной стороной этого является то, что локальные переменные не могут быть использованы вне функции, в которой они были созданы и будут автоматически уничтожены функцией по возвращении управления вызывающему. Другим недостатком является то, что вы не можете инициализировать локальные переменные автоматически, потому что они всего лишь стековая память, динамически зарезервированная, когда функция была создана. Вы должны вручную присвоить им значения.

```
mov    wc.cbSize,SIZEOF WNDCLASSEX

mov     wc.style, CS_HREDRAW or CS_VREDRAW
mov     wc.lpfnWndProc, OFFSET WndProc
mov     wc.cbClsExtra,NULL
mov     wc.cbWndExtra,NULL

push    hInstance
pop     wc.hInstance
mov     wc.hbrBackground,COLOR_WINDOW+1
mov     wc.lpszMenuName,NULL

mov     wc.lpszClassName,OFFSET ClassName
invoke  LoadIcon,NULL,IDI_APPLICATION
mov     wc.hIcon,eax
mov     wc.hIconSm,eax

invoke  LoadCursor,NULL,IDC_ARROW
mov     wc.hCursor,eax
invoke  RegisterClassEx, addr wc
```

Все написанное выше в действительности весьма просто. Это инициализация класса окна. Класс окна - это не что иное, как наметки или спецификации будущего окна. Он определяет некоторые важные характеристики окна, такие как иконка, курсор, функцию, ответственную за окно и так далее. Вы создаете окно из класса окна. Это некоторый сорт концепции ООП. Если вы создаете более, чем одно окно с одинаковыми характеристиками, есть резон для того, чтобы сохранить все характеристики только в одном месте, и обращаться к ним в случае надобности. Эта схема спасет большое количество памяти путем избегания повторения информации. Помните, Windows создавался во времена, когда чипы памяти стоили непомерно высоко и большинство компьютеров имели 1 MB памяти. Windows должен был быть очень эффективным в использовании скудных ресурсов памяти. Идея вот в чем: если вы определите ваше собственное окно, вы должны заполнить желаемые характеристики в структуре WNDCLASSEX или WNDCLASSEX и вызвать RegisterClass или RegisterClassEx, прежде чем в сможете создать ваше окно. Вы только должны один раз зарегистрировать класс окна для каждой их разновидности, из которых вы будете создавать окна.

В Windows есть несколько предопределенных классов, таких как класс кнопки или окна редактирования. Для этих окно (или контролов), вы не должны регистрировать класс окна, необходимо лишь вызвать CreateWindowEx, передав ему имя предопределенного класса. Самый важный член WNDCLASSEX - это lpfnWndProc. lpfn означает дальний указатель на функцию. Под Win32 нет "близких" или "дальних" указателей, а лишь просто указатели, так как модель памяти теперь FLAT. Но это опять же пережиток времен Win16. Каждому классу окна должен быть сопоставлена процедура окна, которая ответственна за обработку сообщения всех окон этого класса. Windows будут слать сообщения процедуре окна, чтобы уведомить его о важных событий, касающихся окон, за которые ответственна эта процедура, например о вводе с клавиатуры или перемещении мыши. Процедура окна должна выборочно реагировать на получаемые ей сообщения. Вы будете тратить большую часть вашего времени на написания обработчиков событий.

Ниже я объясню каждый из членов структуры WNDCLASSEX:

```

WNDCLASSEX STRUCT DWORD
    cbSize          DWORD    ?
    style           DWORD    ?

    lpfnWndProc     DWORD    ?
    cbClsExtra      DWORD    ?
    cbWndExtra      DWORD    ?
    hInstance       DWORD    ?

    hIcon           DWORD    ?
    hCursor         DWORD    ?
    hbrBackground   DWORD    ?
    lpszMenuName    DWORD    ?

    lpszClassName   DWORD    ?
    hIconSm         DWORD    ?
WNDCLASSEX ENDS

```

- **cbSize:** Размер структуры WNDCLASSEX в байтах. Мы можем использовать оператор `sizeof`, чтобы получить это значение.
- **style:** Стилль окон, создаваемых из этого класса. Вы можете комбинировать несколько стилей вместе, используя оператор "or".
- **lpfnWndProc:** Адрес процедуры окна, ответственной за окна, создаваемых из класса.
- **cbClsExtra:** Количество дополнительных байтов, которые нужно зарезервировать (они будут следовать за самой структурой). По умолчанию, операционная система инициализирует это количество в 0. Если приложение использует WNDCLASSEX структуру, чтобы зарегистрировать диалоговое окно, созданное директивой `CLASS` в файле ресурсов, оно должно приравнять этому члену значение `DLGWINDOWEXTRA`.
- **hInstance:** Хэндл модуля.
- **hIcon:** Хэндл иконки. Получите его функцией `LoadIcon`.
- **hCursor:** Хэндл курсора. Получите его функцией `LoadCursor`.
- **hbrBackground:** Цвет фона
- **lpszMenuName:** Хэндл меню для окон, созданных из класса по умолчанию.
- **lpszClassName:** Имя класса окна.
- **hIconSm:** Хэндл маленькой иконки, которая сопоставляется классу окна. Если этот член равен `NULL`'у, система ищет иконку, определенную для члена `hIcon`, чтобы использовать ее как маленькую иконку.

```

invoke CreateWindowEx, NULL,\
    ADDR ClassName,\

    ADDR AppName,\
    WS_OVERLAPPEDWINDOW,\
    CW_USEDEFAULT,\
    CW_USEDEFAULT,\

    CW_USEDEFAULT,\
    CW_USEDEFAULT,\
    NULL,\
    NULL,\

    hInst,\
    NULL

```

После регистрации класса окна, мы должны вызвать `CreateWindowEx`, чтобы создать наше окно, основанное на этом классе. Заметьте, что этой функции передаются этой функции.

```

CreateWindowExA proto dwExStyle:DWORD,\
    lpClassName:DWORD,\
    lpWindowName:DWORD,\
    dwStyle:DWORD,\
    X:DWORD,\
    Y:DWORD,\
    nWidth:DWORD,\

```

```

nHeight:DWORD,\
hWndParent:DWORD ,\
hMenu:DWORD,\
hInstance:DWORD,\
lpParam:DWORD

```

Давайте посмотрим детальное описание каждого параметра:

- **dwExStyle:** Дополнительные стили окна. Это новый параметр, который добавлен в старую функцию `CreateWindow`. Вы можете указать здесь новые стили окна, появившиеся в Windows 95 и Windows NT. Обычные стили окна указываются в `dwStyle`, но если вы хотите определить некоторые дополнительные стили, такие как `topmost` окно (которое всегда наверху), вы должны поместить их здесь. Вы можете использовать `NULL`, если вам не нужны дополнительные стили.
- **lpClassName:** (Обязательный параметр). Адрес ASCIIZ строки, содержащую имя класса окна, которое вы хотите использовать как шаблон для этого окна. Это может быть ваш собственный зарегистрированный класс или один из предопределенных классов. Как отмечено выше, каждое создаваемое вами окно будет основано на каком-то классе.
- **lpWindowName:** Адрес ASCIIZ строки, содержащей имя окна. Оно будет показано на title bar'e окна. Если этот параметр будет равен `NULL`'у, он будет пуст.
- **dwStyle:** Стили окна. Вы можете определить появление окна здесь. Можно передать `NULL` без проблем, тогда у окна не будет кнопок изменения размеров, закрытия и системного меню. Большого прока от этого окна нет. Самый общий стиль - это `WS_OVERLAPPEDWINDOW`. Стиль окна всегда лишь битовый флаг, поэтому вы можете комбинировать различные стили окна с помощью оператора "or", чтобы получить желаемый результат. Стиль `WS_OVERLAPPEDWINDOW` в действительности комбинация большинства общих стилей с помощью этого метода.
- **X, Y:** Координаты вернего левого угла окна. Обычно эти значения равны `CW_USEDEFAULT`, что позволяет Windows решить, куда поместить окно. **nWidth, nHeight:** Ширина и высота окна в пикселях. Вы можете также использовать `CW_USEDEFAULT`, чтобы позволить Windows выбрать соответствующую ширину и высоту для вас.
- **hWndParent:** Хэндл родительского окна (если существует). Этот параметр говорит Windows является ли это окно дочерним (подчиненным) другого окна, и, если так, кто родитель окна. Заметьте, что это не родительско-дочерние отношения в окна MDI (multiply document interface). Дочерние окна не ограничены границами клиентской области родительского окна. Эти отношения нужны для внутреннего использования Windows. Если родительское окно уничтожено, все дочерние окна уничтожаются автоматически. Это действительно просто. Так как в нашем примере всего лишь одно окно, мы устанавливаем этот параметр в `NULL`.
- **hMenu:** Хэндл меню окна. `NULL` - если будет использоваться меню, определенное в классе окна. Взгляните на код, объясненный ранее, член структуры `WNDCLASSEX` `lpszMenuName`. Он определяет меню *по умолчанию* для класса окна. Каждое окно, созданное из этого класса будет иметь тоже меню по умолчанию, до тех пор пока вы не определите специально меню для какого-то окна, используя параметр `hMenu`. Этот параметр - двойного назначения. В случае, если ваше окно основано на предопределенном классе окна, оно не может иметь меню. Тогда `hMenu` используется как ID этого контрола. Windows может определить действительно ли `hMenu` - это хэндл меню или же ID контрола, проверив параметр `lpClassName`. Если это имя предопределенного класса, `hMenu` - это идентификатор контрола. Если нет, это хэндл меню окна.
- **hInstance:** Хэндл программного модуля, создающего окно.
- **lpParam:** Опциональный указатель на структуру данных, передаваемых окну. Это используется окнами MDI, чтобы передать структуру `CLIENTCREATESTRUCT`. Обычно этот параметр установлен в `NULL`, означая, что никаких данных не передается через `CreateWindow()`. Окно может получать значение этого параметра через вызов функции `GetWindowsLong`.

```

mov     hwnd, eax
invoke  ShowWindow, hwnd, CmdShow
invoke  UpdateWindow, hwnd

```

После успешного возвращения из `CreateWindowsEx`, хэндл окна находится в `eax`. Мы должны сохранить это значение, так как будем использовать его в будущем. Окно, которое мы только что создали, не покажется на экране автоматически. Вы должны вызвать `ShowWindow`, передав ему хэндл окна и желаемый тип отображения на экране, чтобы оно появилось на рабочем столе. Затем вы должны вызвать `UpdateWindow` для того, чтобы окно перерисовало свою клиентскую область. Эта Функция полезна, когда вы хотите обновить содержимое клиентской области. Вы Тем не менее, вы можете пренебречь вызовом этой функции.

```
.WHILE TRUE
    invoke GetMessage, ADDR msg,NULL,0,0
.BREAK .IF (!eax)
    invoke TranslateMessage, ADDR msg
    invoke DispatchMessage, ADDR msg
.ENDW
```

Теперь наше окно на экране. Но оно не может получать ввод из внешнего мира. Поэтому мы должны проинформировать его о соответствующих событиях. Мы достигаем этого с помощью цикла сообщений. В каждом модуле есть только один цикл сообщений. В нем функцией `GetMessage` последовательно проверяется, есть ли сообщения от Windows. `GetMessage` передает указатель на `MSG` структуру Windows. Эта структура будет заполнена информацией о сообщении, которые Windows хотят послать окну этого модуля. Функция `GetMessage` не возвращается, пока не появится какое-нибудь сообщение. В это время Windows может передать контроль другим программам. Это то, что формирует схему многозадачности в платформе Win16. `GetMessage` возвращает `FALSE`, если было получено сообщение `WM_QUIT`, что прерывает цикл обработки сообщений и происходит выход из программы. `TranslateMessage` - это вспомогательная функция, которая обрабатывает ввод с клавиатуры и генерирует новое сообщение (`WM_CHAR`), помещающееся в очередь сообщений. Сообщение `WM_CHAR` содержит ASCII-значение нажатой клавиши, с которым проще иметь дело, чем непосредственно со скан-кодами. Вы можете не использовать эту функцию, если ваша программа не обрабатывает ввод с клавиатуры. `DispatchMessage` пересылает сообщение процедуре соответствующего окна.

```
mov  eax,msg.wParam
ret
```

```
WinMain endp
```

Если цикл обработки сообщений прерывается, код выхода сохраняется в члене `MSG` структуры `wParam`. Вы можете сохранить этот код выхода в `eax`, чтобы вернуть его Windows. В настоящее время код выхода не влияет никаким образом на Windows, но лучше подстраховаться и играть по правилам.

```
WndProc proc hWnd:HWND, uMsg:UINT, wParam:WPARAM, lParam:LPARAM
```

Это наша процедура окна. Вы не обязаны называть ее `WndProc`. Первый параметр, `hWnd`, это хэндл окна, которому предназначается сообщение. `uMsg` - сообщение. Отметьте, что `uMsg` - это не `MSG` структура. Это всего лишь число. Windows определяет сотни сообщений, большинством из которых ваша программа интересоваться не будет. Windows будет слать подходящее сообщение, в случае если произойдет что-то относящееся к этому окну. Процедура окна получает сообщение и реагирует на это соответствующе. `wParam` и `lParam` всего лишь дополнительные параметры, использующиеся некоторыми сообщениями. Некоторые сообщения шлют сопроводительные данные в добавление к самому сообщению. Эти данные передаются процедуре окна в переменных `wParam` и `lParam`.

```
.IF uMsg==WM_DESTROY
    invoke PostQuitMessage,NULL
.ELSE

    invoke DefWindowProc,hWnd,uMsg,wParam,lParam
ret
```

```
.ENDIF
xor eax, eax

ret
WndProc endp
```

Это ключевая часть - там где располагается логика действий вашей программы. Код, обрабатывающий каждое сообщение от Windows - в процедуре окна. Ваш код должен проверить сообщение, чтобы убедиться, что это именно то, которое вам нужно. Если это так, сделайте все, что вы хотите сделать в качестве реакции на это сообщение, а затем возвратитесь, оставив в `eax` ноль. Если же это не то сообщение, которое вас интересует, вы ДОЛЖНЫ вызвать `DefWindowProc`, передав ей все параметры, которые вы до этого получили. `DefWindowProc` - это API функция, обрабатывающая сообщения, которыми ваша программа не интересуется.

Единственное сообщение, которое вы ОБЯЗАНЫ обработать - это `WM_DESTROY`. Это сообщение посылается вашему окну, когда оно закрывается. В то время, когда процедура окна его получает, окно уже исчезло с экрана. Это всего лишь напоминание, что ваше окно было уничтожено, поэтому вы должны готовиться к выходу в Windows. Если вы хотите дать шанс пользователю предотвратить закрытие окна, вы должны обработать сообщение `WM_CLOSE`. Относительно `WM_DESTROY` - после выполнения необходимых вам действий, вы должны вызвать `PostQuitMessage`, который пошлет сообщение `WM_QUIT`, что вынудит `GetMessage` вернуть нулевое значение в `eax`, что в свою очередь, повлечет выход из цикла обработки сообщений, а значит из программы.

Вы можете послать сообщение `WM-DESTROY` вашей собственной процедуре окна, вызвав функцию `DestroyWindow`.

Win32 API. Урок 4. Отрисовка текста

Автор: lczelion, пер. Aquila

Дата: 2002-05-04

В этом разделе мы научимся как "рисовать" текст в клиентской части окна. Мы также узнаем о контекстах устройств.

Вы можете скачать исходный код здесь (находится в архиве `wasm_files.zip: files/recovered/1001004/tut04.zip`).

ТЕОРИЯ

Текст в Windows - это вид GUI объекта. Каждый символ создан из множества пикселей (точек), которые соединены в различные рисунки. Вот почему мы "рисует" их, а не "пишем". Обычно вы рисуете текст в вашей клиентской области (на самом деле, вы можете рисовать за пределами клиентской области, но это другая история). Помещение текста на экран в Windows разительно отличается от того, как это делается в DOS'e. В DOS'e размерность экрана 80x25. Но в Windows, экран используется одновременно несколькими программами. Необходимо следовать определенным правилам, чтобы избежать того, чтобы программы рисовали поверх чужой части экрана. Windows обеспечивает это ограничивая область рисования его клиентской частью. Размер клиентской части окна совсем не константа. Пользователь может изменить его в любой момент, поэтому вы должны определять размеры вашей клиентской области динамически. Перед тем, как вы нарисуете что-нибудь на клиентской части, вы должны спросить разрешения у операционной системы. Действительно, теперь у вас нет абсолютного контроля над экраном, как это было в DOS'e. Вы должны спрашивать Windows, чтобы он позволил вам рисовать в вашей собственной клиентской области. Windows определит размер вашей клиентской области, фонт, цвета и другие графические атрибуты и пошлет хэндл контекста устройства (device context) программе. Тогда вы сможете использовать его как пропуск к рисованию.

Что такое контекст устройства? Это всего структура данных, использующаяся Windows внутренне. Контекст устройства сопоставлен определенному устройству, такому как принтер или видео-адаптер. Для видеодисплея, контекст устройства обчно сопоставлен определенному окну на экране.

Некоторые из значений в этой структуре - это графические атрибуты, такие как цвета, фонт и т.д. Это значения по умолчанию, которые вы можете изменять по своему желанию.

Они существуют, чтобы помочь снизить загрузку из-за необходимости указывать эти атрибуты при каждом вызове функций GDI.

Когда программе нужно отрисовать что-нибудь, она должна получить хэндл контекста устройства. Как правило, есть несколько путей достигнуть этого.

- Вызовите `BeginPaint` в ответ на сообщение `WM_PAINT`.
- Вызовите `GetDC` в ответ на другие сообщения.
- Вызовите `CreateDC`, чтобы создать ваш собственный контекст устройства.

Вы должны помнить одну вещь. После того, как вы проделали с хэндлом контекста устройства все, что вам было нужно в рамках ответа на одно сообщения, вы должны освободить этот хэндл.

Нельзя делать так: получить хэндл, обрабатывая одно сообщение, и освободить его, обрабатывая другое.

Windows посылает сообщение `WM_PAINT` окну, чтобы уведомить его о том, что настало время для перерисовки клиентской области. Windows не сохраняет содержимое клиентской части окна.

Взамен, когда происходит ситуация, служащая основанием для перерисовки окна, Windows помещает в очередь сообщений окна WM_PAINT. Окно должно само перерисовать свою клиентскую область. Вы должны поместить всю информацию о том, как перерисовывать клиентскую область в секции WM_PAINT вашей процедуры окна, так чтобы она могла отрисовать всю клиентскую часть, когда будет получено сообщение WM_PAINT. Также вы должны представлять себе, что такое invalid rectangle. Windows определяет i.r. как наименьшую прямоугольную часть окна, которая должна быть перерисована. Когда Windows обнаруживает i.r. в клиентской области окна, оно посылает сообщение WM_PAINT этому окну. В ответ на сообщение, окно может получить структуру PAINTSTRUCT, которая среди прочего содержит координаты i.r.. Вы вызываете функцию BeginPaint в ответ на сообщение WM_PAINT, чтобы сделать неполноценный прямоугольник снова нормальным. Если вы не обрабатываете сообщение WM_PAINT, то по крайней мере вам следует вызвать DefWindowProc или ValidateRect, иначе Windows будет слать вам WM_PAINT постоянно.

Ниже показаны шаги, которые вы должны выполнить, обрабатывая сообщение WM_PAINT:

- Получить хэндл контекста устройства с помощью BeginPaint.
- Отрисовать клиентскую область.
- Освободить хэндл функцией EndPaint.

Заметьте, что вы не обязаны думать о том, чтобы пометить неполноценные прямоугольники как нормальные, так как это делается автоматически при вызове BeginPaint. Между связкой BeginPaint-EndPaint, вы можете вызвать любую другую графическую функцию, чтобы рисовать в вашей клиентской области. Практически все из них требуют хэндл контекста устройства.

СОДЕРЖИМОЕ

Мы напишем программу, ображающую текстовую строку "Win32 asstmble is great and easy!" в центре клиентской области.

```
.386

.model flat,stdcall
option casemap:none

WinMain proto :DWORD,:DWORD,:DWORD,:DWORD

include \masm32\include\windows.inc

include \masm32\include\user32.inc
includelib \masm32\lib\user32.lib
include \masm32\include\kernel32.inc
includelib \masm32\lib\kernel32.lib

.DATA
ClassName db "SimpleWinClass",0

AppName db "Our First Window",0
OurText db "Win32 assembly is great and easy!",0

.DATA?
hInstance HINSTANCE ?
CommandLine LPSTR ?

.CODE
start:
    invoke GetModuleHandle, NULL

    mov     hInstance,eax
    invoke GetCommandLine
    mov     CommandLine,eax
    invoke WinMain, hInstance,NULL,CommandLine, SW_SHOWDEFAULT
```

```

        invoke ExitProcess,eax

WinMain proc hInst:HINSTANCE, hPrevInst:HINSTANCE, CmdLine:LPSTR,
CmdShow:DWORD

    LOCAL wc:WNDCLASSEX
    LOCAL msg:MSG
    LOCAL hwnd:HWND

    mov     wc.cbSize,SIZEOF WNDCLASSEX
    mov     wc.style, CS_HREDRAW or CS_VREDRAW
    mov     wc.lpfnWndProc, OFFSET WndProc
    mov     wc.cbClsExtra,NULL
    mov     wc.cbWndExtra,NULL
    push    hInst
    pop     wc.hInstance
    mov     wc.hbrBackground,COLOR_WINDOW+1
    mov     wc.lpszMenuName,NULL
    mov     wc.lpszClassName,OFFSET ClassName
    invoke  LoadIcon,NULL,IDI_APPLICATION
    mov     wc.hIcon,eax
    mov     wc.hIconSm,eax
    invoke  LoadCursor,NULL,IDC_ARROW
    mov     wc.hCursor,eax
    invoke  RegisterClassEx, addr wc

    invoke  CreateWindowEx,NULL,ADDR ClassName,ADDR AppName,\
        WS_OVERLAPPEDWINDOW,CW_USEDEFAULT,\

        CW_USEDEFAULT,CW_USEDEFAULT,CW_USEDEFAULT,NULL,NULL,\
        hInst,NULL

    mov     hwnd,eax
    invoke  ShowWindow, hwnd,SW_SHOWNORMAL
    invoke  UpdateWindow, hwnd
    .WHILE TRUE

        invoke GetMessage, ADDR msg,NULL,0,0
        .BREAK .IF (!eax)
        invoke TranslateMessage, ADDR msg
        invoke DispatchMessage, ADDR msg

    .ENDW
    mov     eax,msg.wParam
    ret
WinMain endp

WndProc proc hWnd:HWND, uMsg:UINT, wParam:WPARAM, lParam:LPARAM
    LOCAL hdc:HDC

    LOCAL ps:PAINTSTRUCT
    LOCAL rect:RECT
    .IF uMsg==WM_DESTROY
        invoke PostQuitMessage,NULL

    .ELSEIF uMsg==WM_PAINT
        invoke BeginPaint,hWnd, ADDR ps
        mov     hdc,eax
        invoke GetClientRect,hWnd, ADDR rect

        invoke DrawText, hdc,ADDR OurText,-1, ADDR rect, \
            DT_SINGLELINE or DT_CENTER or DT_VCENTER
        invoke EndPaint,hWnd, ADDR ps
    .ELSE

        invoke DefWindowProc,hWnd,uMsg,wParam,lParam
        ret
    .ENDIF
    xor     eax, eax
    ret

```



```

WndProc endp
end start

```

АНАЛИЗ

Большая часть этого кода точно такая же, как и пример из урока 3. Я объясню только важные изменения.

```

LOCAL hdc:HDC

LOCAL ps:PAINTSTRUCT

LOCAL rect:RECT

```

Это несколько переменных, использующихся в нашей секции WM_PAINT. Переменная hdc используется для сохранения хэндла контекста устройства, возвращенного функцией BeginPaint. ps - это структура PAINTSTRUCT. Обычно вам не нужны значения этой структуры. Она передается функции BeginPaint и Windows заполняет ее подходящими значениями. Затем вы передаете ps функции EndPaint, когда заканчиваете отрисовку клиентской области. rect - это структура RECT, определенная следующим образом:

```

RECT Struct
    left    LONG ?
    top     LONG ?
    right   LONG ?
    bottom  LONG ?
RECT ends

```

Left и top - это координаты вернего левого угла прямоугольника. Right и bottom - это координаты нижнего правого угла. Помните одну вещь: начала координатных осей находятся в левом верхнем углу клиентской области, поэтому точка y=10 НИЖЕ, чем точка y=0.

```

invoke BeginPaint,hWnd, ADDR ps
mov     hdc, eax
invoke GetClientRect,hWnd, ADDR rect
invoke DrawText, hdc, ADDR OurText, -1, ADDR rect, \
    DT_SINGLELINE or DT_CENTER or DT_VCENTER
invoke EndPaint,hWnd, ADDR ps

```

В ответ на сообщение WM_PAINT, вы вызываете BeginPaint, передавая ей хэндл окна, в котором вы хотите рисовать и неинициализированную структуру типа PAINTSTRUCT в качестве параметров. После успешного вызова, eax содержит хэндл контекста устройства. После вы вызываете GetClientRect, чтобы получить размеры клиентской области. Размеры возвращаются в переменной rect, которую вы передаете функции DrawText как один из параметров. Синтаксис DrawText'а таков:

```

DrawText proto hdc:HDC, lpString:DWORD, nCount:DWORD, lpRect:DWORD,
    uFormat:DWORD

```

DrawText = это высокоуровневая API функция вывода текста. Она берет на себя такие вещи как перенос слов, центровка и т.п., так что вы можете сконцентрироваться на строке, которую вы хотите нарисовать. Ее низкоуровневый брат, TextOut, будет описан в следующем уроке. DrawText подгоняет строку под прямоугольник. Она использует выбранный в настоящее время шрифт, цвет и фон для отрисовки текста. Слова переносятся так, чтобы строка влезла в границы прямоугольника. DrawText возвращает высоту выводимого текста в единицах устройства, в нашем случае в пикселях. Давайте посмотрим на ее параметры:

- hdc - хэндл контекста устройства
- lpString - указатель на строку, которую вы хотите нарисовать в прямоугольнике. Строка должна заканчиваться NULL'ом, или же вам придется указывать ее длину в следующем параметре, nCount.
- nCount - количество символов для вывода. Если строка заканчивается NULL'ом, nCount должен быть равен -1. В противном случае, nCount должен содержать количество символов в

строке.

- `lpRect` - указатель на прямоугольник (структура типа `RECT`), в котором вы хотите рисовать строку. Заметьте, что прямоугольник ограничен, то есть вы не можете нарисовать строку за его пределами.
- `uFormat` - значение, определяющее как строка отображается в прямоугольнике. Мы используем три значения, скомбинированные оператором "or":
- `DT_SINGLELINE` указывает, что текст будет располагаться в одну линию
- `DT_CENTER` центрирует текст по горизонтали
- `DT_VCENTER` центрирует текст по вертикали. Должен использоваться вместе с `DT_SINGLELINE`.

После того, как вы отрисовали клиентскую область, вы должны вызвать функцию `EndPaint`, чтобы освободить хэндл устройства контекста.

Вот и все. Мы можем указать главные идеи:

- Вы вызываете связку `BeginPaint-EndPaint` в ответ на сообщение `WM_PAINT`. Делайте все, что вам нужно с клиентской областью между вызовами этих двух функций.
- Если вы хотите перерисовать вашу клиентскую область в ответе на другие сообщения, у вас есть два выбора:
- Используйте связку `GetDC-ReleaseDC` и делайте отрисовку между вызовами этих функций.
- Вызовите `InvalidateRect` или `UpdateWindow`, чтобы Windows послала сообщение `WM_PAINT` вашему окну.

Win32 API. Урок 5. Больше о тексте

Автор: lczelion, пер. Aquila

Дата: 2002-05-05

Мы еще немного поэкспериментируем, то есть шрифт и цвет.

Вы можете скачать пример [здесь](#) (находится в архиве `wasm_files.zip: files/recovered/1001005/tut05.zip`).

ТЕОРИЯ

Цветовая система Windows базируется на RGB значениях, R=красный, G=зеленый, B=синий. Если вы хотите указать Windows цвет, вы должны определить желаемый цвет в системе этих трех основных цветов. Каждое цветовое значение имеет область определения от 0 до 255. Например, если вы хотите чистый красный цвет, вам следует использовать 255, 0, 0. Или если вы хотите чистый белый цвет, вы должны использовать 255, 255, 255. Вы можете видеть из примеров, что получение нужного цвета очень сложно, используя эту систему, так что вам нужно иметь хорошее "чувство на цвета", как мешать и составлять их. Для установки цвета текста или фона, вы можете использовать `SetTextColor` и `SetBkColor`, оба из которых требуют хэндл контекста устройства и 32-битное RGB значение. Структура 32-битного RGB значения определена как:

```
RGB_value struct
    unused    db 0
    blue      db ?
    green      db ?
    red        db ?
RGB_value ends
```

Заметьте, что первый байт не используется и должен быть нулем. Порядок оставшихся байтов перевернут, то есть `blue`, `green`, `red`. Тем не менее, мы не будем использовать эту структуру, так как ее тяжело инициализировать и использовать. Вместо этого мы создадим макрос. Он будет получать три параметра: значения красного, зеленого и синего. Он будет выдавать желаемое 32-битное RGB значение и сохранять его в `eax`. Макрос определен следующим образом:

```
RGB macro red,green,blue

    xor     eax,eax
    mov     ah,blue
    shl     eax,8
    mov     ah,green
    mov     al,red

endm
```

Вы можете поместить этот макрос в `include` файл для использования в будущем. Вы можете "создать" шрифт, вызвав `CreateFont` или `CreateFontIndirect`. Разница между ними заключается в том, что `CreateFontIndirect` получает только один параметр: указатель на структуру логического шрифта, `LOGFONT`.

`CreateFontIndirect` более гибкая функция из этих двух, особенно если вашей программе необходимо часто менять шрифты. Тем не менее, в нашем примере мы "создадим" только один шрифт для демонстрации, поэтому будем делать это через `CreateFont`. После вызова этой функции, она вернет хэндл шрифта, который вы должны выбрать в определенном контексте устройства. После этого, каждая текстовая API функция будет использовать шрифт, который мы выбрали.

СОДЕРЖИМОЕ

```

.model flat,stdcall
option casemap:none

WinMain proto :DWORD,:DWORD,:DWORD,:DWORD

include \masm32\include\windows.inc
include \masm32\include\user32.inc
include \masm32\include\kernel32.inc

include \masm32\include\gdi32.inc
includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib
includelib \masm32\lib\gdi32.lib

RGB macro red,green,blue
    xor eax,eax

    mov ah,blue
    shl eax,8
    mov ah,green
    mov al,red

endm

.data

ClassName db "SimpleWinClass",0
AppName db "Our First Window",0
TestString db "Win32 assembly is great and easy!",0
FontName db "script",0

.data?
hInstance HINSTANCE ?

CommandLine LPSTR ?

.code

start:
    invoke GetModuleHandle, NULL
    mov hInstance,eax
    invoke GetCommandLine

    mov CommandLine,eax
    invoke WinMain, hInstance,NULL,CommandLine, SW_SHOWDEFAULT
    invoke ExitProcess,eax

WinMain proc
hInst:HINSTANCE,hPrevInst:HINSTANCE,CmdLine:LPSTR,CmdShow:DWORD
LOCAL wc:WNDCLASSEX
LOCAL msg:MSG
LOCAL hwnd:HWND

    mov wc.cbSize,SIZEOF WNDCLASSEX
    mov wc.style,CS_HREDRAW or CS_VREDRAW
    mov wc.lpfnWndProc,OFFSET WndProc
    mov wc.cbClsExtra,NULL
    mov wc.cbWndExtra,NULL
    push hInst
    pop wc.hInstance
    mov wc.hbrBackground,COLOR_WINDOW+1
    mov wc.lpszMenuName,NULL
    mov wc.lpszClassName,OFFSET ClassName
    invoke LoadIcon,NULL,IDI_APPLICATION
    mov wc.hIcon,eax
    mov wc.hIconSm,eax

```

```

invoke LoadCursor,NULL,IDC_ARROW
mov    wc.hCursor,eax

invoke RegisterClassEx, addr wc
invoke CreateWindowEx,NULL,ADDR ClassName,ADDR AppName,\
    WS_OVERLAPPEDWINDOW,CW_USEDEFAULT,\
    CW_USEDEFAULT,CW_USEDEFAULT,CW_USEDEFAULT,NULL,NULL,\
    hInst,NULL

mov    hwnd,eax

invoke ShowWindow, hwnd,SW_SHOWNORMAL

invoke UpdateWindow, hwnd
.WHILE TRUE
    invoke GetMessage, ADDR msg,NULL,0,0
    .BREAK .IF (!eax)

    invoke TranslateMessage, ADDR msg
    invoke DispatchMessage, ADDR msg
.ENDW
mov    eax,msg.wParam

ret
WinMain endp

WndProc proc hwnd:HWND, uMsg:UINT, wParam:WPARAM, lParam:LPARAM
LOCAL hdc:HDC
LOCAL ps:PAINTSTRUCT
LOCAL hfont:HFONT

.IF uMsg==WM_DESTROY
    invoke PostQuitMessage,NULL

.ELSEIF uMsg==WM_PAINT
    invoke BeginPaint,hwnd, ADDR ps
    mov    hdc,eax
    invoke CreateFont,24,16,0,0,400,0,0,0,OEM_CHARSET,\
        OUT_DEFAULT_PRECIS,CLIP_DEFAULT_PRECIS,\
        DEFAULT_QUALITY,DEFAULT_PITCH or
        FF_SCRIPT,\
        ADDR FontName

    invoke SelectObject, hdc, eax
    mov    hfont,eax
    RGB    200,200,50

    invoke SetTextColor,hdc,eax
    RGB    0,0,255
    invoke SetBkColor,hdc,eax
    invoke TextOut,hdc,0,0,ADDR TestString,SIZEOF TestString

    invoke SelectObject,hdc, hfont
    invoke EndPaint,hwnd, ADDR ps
.ELSE
    invoke DefWindowProc,hwnd,uMsg,wParam,lParam

    ret
.ENDIF
xor     eax,eax
ret

WndProc endp

end start

```

АНАЛИЗ

```

invoke CreateFont,24,16,0,0,400,0,0,0,OEM_CHARSET,\

```

```

OUT_DEFAULT_PRECIS,CLIP_DEFAULT_PRECIS,\
DEFAULT_QUALITY,DEFAULT_PITCH or FF_SCRIPT,\
ADDR FontName

```

CreateFont создает логический шрифт, который наиболее близок к данным параметрам и доступным данным шрифта. Эта функция имеет больше параметров, чем любая другая в Windows. Она возвращает логический шрифт, который можно выбрать функцией SelectObject. Мы в подробностях обсудим ее параметры.

```

CreateFont proto nHeight:DWORD,\
                nWidth:DWORD,\
                nEscapement:DWORD,\
                nOrientation:DWORD,\
                nWeight:DWORD,\
                cItalic:DWORD,\
                cUnderline:DWORD,\
                cStrikeOut:DWORD,\
                cCharSet:DWORD,\
                cOutputPrecision:DWORD,\
                cClipPrecision:DWORD,\
                cQuality:DWORD,\
                cPitchAndFamily:DWORD,\
                lpFacename:DWORD

```

- nHeight - желаемая высота символов. Ноль значит использовать размер по умолчанию.
- nWidth - желаемая ширина символов. Обычно этот параметр равен нулю, что позволяет Windows подобрать ширину соответственно высоте. Однако, в нашем примере, дефолтная ширина делает символы нечитаемыми, поэтому я установил ширину равную 16.
- nEscapement - указывает ориентацию вывода следующего символа, относительно предыдущего в десятых градусах. Как правило его устанавливают в 0. Установка в 900 вынуждает идти все символы снизу вверх, 1800 - справа налево, 2700 - сверху вниз.
- nOrientation - указывает насколько символ должен быть повернут в десятых градусах. 900 - все символы будут "лежать" на спине, и далее по аналогии с предыдущим параметром.
- nWeight - устанавливает толщину линии. Windows определяет следующие размеры:
- FW_DONTCARE equ 0
- FW_THIN equ 100
- FW_EXTRALIGHT equ 200
- FW_ULTRALIGHT equ 200
- FW_LIGHT equ 300
- FW_NORMAL equ 400
- FW_REGULAR equ 400
- FW_MEDIUM equ 500
- FW_SEMIBOLD equ 600
- FW_DEMIBOLD equ 600
- FW_BOLD equ 700
- FW_EXTRABOLD equ 800
- FW_ULTRABOLD equ 800
- FW_HEAVY equ 900
- FW_BLACK equ 900
- cItalic - 0 для обычных символов, любое другое значение для романских.
- cUnderline - 0 для обычных символов, любое другое значение для подчеркнутых.
- cStrikeOut - 0 для обычных символов, любое другое значение для перечеркнутых.
- cCharSet - символьный набор шрифта. Обычно должен быть установлен в OEM_CHARSET, который позволяет Windows выбрать системно-зависимый шрифт.
- cOutputPrecision - указывает насколько должен близко должен приближаться шрифт к характеристикам, которые мы указали. Обычно этот параметр устанавливается в OUT_DEFAULT_PRECIS.

- `cClipPrecision` определяет, что делать с символами, которые вылезают за пределы отрисовочного региона.
- `cQuality` - указывает качества вывода, то есть насколько внимательно GDI пытаться подогнать атрибуты логического фонта к атрибутам фонта физического. Есть выбор из трех значений: `DEFAULT_QUALITY`, `PROOF_QUALITY` и `DRAFT_QUALITY`.
- `cPitchAndFamily` - указывает питч и семейство фонта. Вы должны комбинировать значение питча и семьи с помощью оператора "or".
- `lpFacename` - указатель на заканчивающуюся NULL'ом строку, определяющую гарнитуру фонта.

Вышеприведенное описание ни в коем случае не является исчерпывающим. Вам следует обратиться к вашему Win32 API Справочнику за деталями.

```
invoke SelectObject, hdc, eax
mov     hfont, eax
```

После получения хэндла логического фонта, мы должны выбрать его в контексте устройства, вызвав `SelectObject`. Функция устанавливает новые GDI объекты, такие как перья, кистья и фонты в контекст устройства, используемые GDI функциями. `SelectObject` возвращает хэндл замещенного объекта в `eax`, который нам следует сохранить для будущего вызова `SelectObject`. После вызова `SelectObject` любая функция вывода текста будет использовать шрифт, который мы выбрали в данном контексте устройства.

```
RGB      200,200,50
invoke SetTextColor, hdc, eax
RGB      0,0,255
invoke SetBkColor, hdc, eax
```

Используйте макрос `RGB`, чтобы создать 32-битное RGB значение, которое будет использоваться функциями `SetTextColor` и `SetBkColor`.

```
invoke TextOut, hdc, 0, 0, ADDR TestString, SIZEOF TestString
```

Вызываем функцию `TextOut` для отрисовки текста на клиентской области экрана. Будет использоваться ранее выбранные нами шрифт и цвет.

```
invoke SelectObject, hdc, hfont
```

После этого мы должны восстановить старый шрифт обратно в данном контексте устройства. Вам всегда следует восстанавливать объект, который вы заменили.

Win32 API. Урок 6. Клавиатура

Автор: lczelion, пер. Aquila

Дата: 2002-05-06

Мы изучим, как Windows программа получает сообщения от клавиатуры.

Скачайте пример здесь (находится в архиве wasm_files.zip: files/recovered/1001006/tut06.zip).

ТЕОРИЯ

Как правило, у каждого компьютера есть только одна клавиатура, поэтому все запущенные Windows программы должны разделять ее между всеми. Windows ответственна за то, чтобы отсылать информацию о нажатых клавишах активному в данный момент окну.

Хотя на экране может быть сразу несколько окон, только одно из них имеет фокус ввода, и только оно может получать сообщения от клавиатуры. Вы можете отличить окно, которое имеет фокус ввода от окна, которое его не имеет, посмотрев на его title bar - он будет подсвечен, в отличии от других.

В действительности, есть два типа сообщений от клавиатуры, зависящих от того, чем вы считаете клавиатуру. Вы можете считать ее набором кнопок. В этом случае, если вы нажмете кнопку, Windows пошлет сообщение WM_KEYDOWN активному окну, уведомляя о нажатии клавиши. Когда вы отпустите клавишу, Windows пошлет сообщение WM_KEYUP. Вы думаете о клавише как о кнопке. Другое взгляд на клавиатуру предполагает, что это устройство ввода символов. Тогда, Windows шлет сообщения WM_KEYDOWN или WM_KEYUP окну, в котором есть фокус ввода, и эти сообщения будут транслированы в сообщение WM_CHAR функцией TranslateMessage. Процедура окна может обрабатывать все три сообщения или только то, в котром оно заинтересованно. Большую часть времени вы можете игнорировать WM_KEYDOWN и WM_KEYUP, так как вызов функции TranslateMessage в цикле обработки сообщений транслирует сообщения WM_KEYDOWN и WM_KEYUP в WM_CHAR. Мы будем опираться именно на это сообщение в данном уроке.

ПРИМЕР

```
.386
.model flat,stdcall
option casemap:none

WinMain proto :DWORD,:DWORD,:DWORD,:DWORD

include \masm32\include\windows.inc
include \masm32\include\user32.inc
include \masm32\include\kernel32.inc
include \masm32\include\gdi32.inc

includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib
includelib \masm32\lib\gdi32.lib

.data
ClassName db "SimpleWinClass",0
AppName db "Our First Window",0

char WPARAM 20h ; the character the program
receives from keyboard

.data?
```



```
hInstance HINSTANCE ?
CommandLine LPSTR ?
```

```
.code
```

```
start:
```

```
    invoke GetModuleHandle, NULL
```

```
    mov     hInstance,eax
    invoke GetCommandLine
    mov     CommandLine,eax
```

```
    invoke WinMain, hInstance,NULL,CommandLine, SW_SHOWDEFAULT
```

```
    invoke ExitProcess,eax
```

```
WinMain proc
```

```
hInst:HINSTANCE,hPrevInst:HINSTANCE,CmdLine:LPSTR,CmdShow:DWORD
```

```
    LOCAL wc:WNDCLASSEX
```

```
    LOCAL msg:MSG
```

```
    LOCAL hwnd:HWND
```

```
    mov     wc.cbSize,SIZEOF WNDCLASSEX
    mov     wc.style, CS_HREDRAW or CS_VREDRAW
    mov     wc.lpfnWndProc, OFFSET WndProc
    mov     wc.cbClsExtra,NULL
```

```
    mov     wc.cbWndExtra,NULL
    push    hInst
    pop     wc.hInstance
    mov     wc.hbrBackground,COLOR_WINDOW+1
```

```
    mov     wc.lpszMenuName,NULL
    mov     wc.lpszClassName,OFFSET ClassName
    invoke LoadIcon,NULL,IDI_APPLICATION
    mov     wc.hIcon,eax
```

```
    mov     wc.hIconSm,eax
    invoke LoadCursor,NULL,IDC_ARROW
    mov     wc.hCursor,eax
    invoke RegisterClassEx, addr wc
```

```
    invoke CreateWindowEx,NULL,ADDR ClassName,ADDR AppName,\
        WS_OVERLAPPEDWINDOW,CW_USEDEFAULT,\
        CW_USEDEFAULT,CW_USEDEFAULT,CW_USEDEFAULT,NULL,NULL,\
        hInst,NULL
```

```
    mov     hwnd,eax
    invoke ShowWindow, hwnd,SW_SHOWNORMAL
    invoke UpdateWindow, hwnd
    .WHILE TRUE
```

```
        invoke GetMessage, ADDR msg,NULL,0,0
        .BREAK .IF (!eax)
        invoke TranslateMessage, ADDR msg
        invoke DispatchMessage, ADDR msg
```

```
    .ENDW
```

```
    mov     eax,msg.wParam
    ret
```

```
WinMain endp
```

```
WndProc proc hwnd:HWND, uMsg:UINT, wParam:WPARAM, lParam:LPARAM
```

```
    LOCAL hdc:HDC
```

```
    LOCAL ps:PAINTSTRUCT
```

```
    .IF uMsg==WM_DESTROY
        invoke PostQuitMessage,NULL
```

```

.ELSEIF uMsg==WM_CHAR
    push wParam
    pop char

    invoke InvalidateRect, hWnd,NULL,TRUE
.ELSEIF uMsg==WM_PAINT
    invoke BeginPaint,hWnd, ADDR ps
    mov     hdc, eax

    invoke TextOut,hdc,0,0,ADDR char,1
    invoke EndPaint,hWnd, ADDR ps
.ELSE
    invoke DefWindowProc,hWnd,uMsg,wParam,lParam

    ret
.ENDIF
xor     eax,eax
ret

WndProc endp
end start

```

АНАЛИЗ

char WPARAM 20h ; символ, который программа получает от клавиатуры

Это переменная, в которой будет сохраняться символ, получаемый от клавиатуры. Так как символ шлется в WPARAM процедуры окна, мы для простоты определяем эту переменную как обладающую типом WPARAM. Начальное значение - 20h или "пробел", так как когда наше окно обновляет свою клиентскую область в первое время, символ еще не введен, поэтому мы делаем так, чтобы отображался пробел.

```

.ELSEIF uMsg==WM_CHAR

    push wParam
    pop char
    invoke InvalidateRect, hWnd,NULL,TRUE

```

Это было добавлено в процедуру окна для обработки сообщения WM_CHAR. Она всего лишь помещает символ в переменную char и затем вызывает InvalidateRect, что вынуждает Windows послать сообщение WM_PAINT процедуре окна. Синтаксис этой функции следующий:

```

InvalidateRect proto hWnd:HWND,\
                    lpRect:DWORD,\
                    bErase:DWORD

```

- lpRect - указатель на прямоугольник в клиентской области, который мы хотим объявить требующим перерисовки. Если этот параметр равен NULL'у, тогда вся клиентская область объявляется такой. bErase - флаг, говорящий Windows, нужно ли уничтожить бэкграунд. Если он равен TRUE, тогда она делает это при вызове функции BeginPaint.
- Таким образом, мы будем использовать следующую стратегию: мы сохраним всю необходимую информацию, относящуюся к отрисовке клиентской области и генерирующую сообщение WM_PAINT, чтобы перерисовать ее. Конечно, код в секции WM_PAINT должен знать заранее, что от него ожидают. Это кажется обходным путем делать дела, но это путь Windows.
- На самом деле, мы можем отрисовать клиентскую область в ходе обработки сообщения WM_CHAR, между вызовами функций GetDC и ReleaseDC. Нет никаких проблем с этим. Но вся забава начнется, когда приложению понадобится перерисовать клиентскую область. Так как код, рисующий символ находится в секции WM_CHAR, программа не сможет перерисовать символ в клиентской части. Поэтому помещайте все необходимые данные и код, отвечающий за рисование в WM_PAINT. Вы можете послать это сообщение из любого места вашего кода, где вам нужно перерисовать клиентскую область.

```

invoke TextOut,hdc,0,0,ADDR char,1

```

Когда `InvalidateRect` вызвана, она шлет сообщение `WM_PAINT` обратно процедуре окна, поэтому вызывается код в секции `WM_PAINT`. Он вызывает `BeginPaint`, чтобы получить хэндл контекста устройства, и затем вызывает `TextOut`, рисуя наш символ в клиентской области в `x=0, y=0`. Когда вы запускаете программу и нажимаете любую клавишу, вы увидите, что символьное эхо в верхнем левом углу клиентского окна. И когда окно минимизируется и максимизируется, символ все равно там, так как все код и все данные, необходимые для перерисовки располагаются в секции `WM_PAINT`.

Win32 API. Урок 7. Мышь

Автор: lczelion, пер. Aquila

Дата: 2002-05-07

Мы научимся как получать и отвечать на ввод с мыши в нашей процедуре окна. Программа-пример будет ждать нажатия на левую кнопку мыши и отображать текстовую строку в точности в том месте клиентской области, где кликнули на мышь.

Скачайте пример здесь (находится в архиве wasm_files.zip: files/recovered/1001007/tut07.zip).

ТЕОРИЯ

Так же, как и при вводе с клавиатуры, Windows определяет и шлет уведомления об активности мыши относительно какого-либо окна. Эта активность включает в себя нажатия на правую и левую клавишу, передвижения курсора через окно, двойные нажатия. В отличие от клавиатуры, сообщения от которой направляются окну, имеющему в данный момент фокус ввода, сведения о которой передаются окну, над которым находится мышь, независимо от того, активно оно или нет. Вдобавок, есть сообщения от мыши, связанные с не-клиентской части окна, но, к счастью, мы можем их как правило игнорировать. Мы можем сфокусироваться на связанных с клиентской областью.

Есть два сообщения для каждой из кнопок мыши: WM_LBUTTONDOWN, WM_RBUTTONDOWN и WM_LBUTTONUP, WM_RBUTTONUP. Если мышь трехкнопочная, то есть еще WM_MBUTTONDOWN и WM_MBUTTONUP. Когда курсор мыши движется над клиентской областью, Windows шлет WM_MOUSEMOVE окну, над которым он находится. Окно может получать сообщения о двойных нажатиях, WM_LBUTTONDOWNDBCLK или WM_RBUTTONDOWNDBCLK, тогда и только тогда, когда окно имеет стиль CS_DBLCLKS, или же оно будет получать только серию сообщений об одинарных нажатиях.

Во всех этих сообщениях значение lParam содержит позицию мыши. Нижнее слово - это x-координата, верхнее слово - y-координата верхнего левого угла клиентской области окна. wParam содержит информацию о состоянии кнопок мыши, Shift'a и Ctrl'a.

ПРИМЕР

```
.386
.model flat,stdcall

option casemap:none

WinMain proto :DWORD,:DWORD,:DWORD,:DWORD

include \masm32\include\windows.inc
include \masm32\include\user32.inc

include \masm32\include\kernel32.inc
include \masm32\include\gdi32.inc
includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib

includelib \masm32\lib\gdi32.lib

.data

ClassName db "SimpleWinClass",0
AppName db "Our First Window",0
MouseClicked db 0 ; 0=no click yet
.data?
```

```

hInstance HINSTANCE ?
CommandLine LPSTR ?

hitpoint POINT <>

.code

start:
    invoke GetModuleHandle, NULL
    mov     hInstance,eax
    invoke GetCommandLine

    mov     CommandLine,eax
    invoke WinMain, hInstance,NULL,CommandLine, SW_SHOWDEFAULT
    invoke ExitProcess,eax

WinMain proc
hInst:HINSTANCE,hPrevInst:HINSTANCE,CmdLine:LPSTR,CmdShow:DWORD
LOCAL wc:WNDCLASSEX

LOCAL msg:MSG
LOCAL hwnd:HWND
mov     wc.cbSize,SIZEOF WNDCLASSEX
mov     wc.style, CS_HREDRAW or CS_VREDRAW

mov     wc.lpfnWndProc, OFFSET WndProc
mov     wc.cbClsExtra,NULL
mov     wc.cbWndExtra,NULL
push    hInst

pop     wc.hInstance
mov     wc.hbrBackground,COLOR_WINDOW+1
mov     wc.lpszMenuName,NULL
mov     wc.lpszClassName,OFFSET ClassName

invoke LoadIcon,NULL,IDI_APPLICATION
mov     wc.hIcon,eax
mov     wc.hIconSm,eax
invoke LoadCursor,NULL,IDC_ARROW

mov     wc.hCursor,eax
invoke RegisterClassEx, addr wc
invoke CreateWindowEx,NULL,ADDR ClassName,ADDR AppName,\
    WS_OVERLAPPEDWINDOW,CW_USEDEFAULT,\
    CW_USEDEFAULT,CW_USEDEFAULT,CW_USEDEFAULT,NULL,NULL,\
    hInst,NULL
mov     hwnd,eax
invoke ShowWindow, hwnd,SW_SHOWNORMAL

invoke UpdateWindow, hwnd
.WHILE TRUE
    invoke GetMessage, ADDR msg,NULL,0,0
    .BREAK .IF (!eax)

    invoke DispatchMessage, ADDR msg
.ENDW
mov     eax,msg.wParam
ret

WinMain endp

WndProc proc hwnd:HWND, uMsg:UINT, wParam:WPARAM, lParam:LPARAM

LOCAL hdc:HDC

LOCAL ps:PAINTSTRUCT

.IF uMsg==WM_DESTROY
    invoke PostQuitMessage,NULL

```

```

.ELSEIF uMsg==WM_LBUTTONDOWN
    mov eax,lParam

    and eax,0FFFFh
    mov hitpoint.x,eax
    mov eax,lParam
    shr eax,16

    mov hitpoint.y,eax
    mov MouseClick,TRUE
    invoke InvalidateRect,hWnd,NULL,TRUE
.ELSEIF uMsg==WM_PAINT

    invoke BeginPaint,hWnd, ADDR ps
    mov hdc,eax
    .IF MouseClick
        invoke lstrlen,ADDR AppName

        invoke TextOut,hdc,hitpoint.x,hitpoint.y,ADDR AppName,eax
    .ENDIF
    invoke EndPaint,hWnd, ADDR ps
.ELSE

    invoke DefWindowProc,hWnd,uMsg,wParam,lParam
    ret
.ENDIF
xor    eax,eax

ret
WndProc endp
end start

```

АНАЛИЗ

```

.ELSEIF uMsg==WM_LBUTTONDOWN

    mov eax,lParam
    and eax,0FFFFh
    mov hitpoint.x,eax
    mov eax,lParam

    shr eax,16
    mov hitpoint.y,eax
    mov MouseClick,TRUE
    invoke InvalidateRect,hWnd,NULL,TRUE

```

Процедура окна ждет нажатия на левую клавишу мыши. Когда она получает WM_LBUTTONDOWN, lParam содержит координаты курсора мыши в клиентской области. Процедура сохраняет их в переменной типа POINT, определенной следующим образом:

```

POINT STRUCT
    x    dd ?

    y    dd ?

POINT ENDS

```

Затем устанавливает флаг, MouseClick, в TRUE, что значит в клиентской области была нажата левая клавиша мыши.

```

    mov eax,lParam
    and eax,0FFFFh
    mov hitpoint.x,eax

```

Так как x-координата - это нижнее слово lParam и члены структуры POINT размером в 32 бита, мы должны обнулить верхнее слово eax, прежде чем сохранить значение в hitpoint.x.

```

    shr eax,16

    mov hitpoint.y,eax

```

Так как у-координата - это верхнее слово IParam, мы должны ее в нижнее слово, прежде чем сохранять в hitpoint.y. Мы делаем это сдвигая eax на 16 битов вправо. После сохранения позиции мыши, мы устанавливаем флаг, MouseClick, в TRUE для того, чтобы отрисовывающий код в секции WM_PAINT, знал, что было нажатие в клиентской области, и значит поэтому он может нарисовать строку в позиции, где была мышь при нажатии. Затем мы вызываем функцию InvalidateRect, чтобы заставить окно полностью перерисовать ее клиентскую область.

```
.IF MouseClick  
  
    invoke strlen,ADDR AppName  
    invoke TextOut,hdc,hitpoint.x,hitpoint.y,ADDR AppName,eax  
  
.ENDIF
```

Отрисовывающий код в секции WM_PAINT должен проверять, установлен ли флаг MouseClick в TRUE, потому что когда окно создается, процедура окна получает сообщение WM_PAINT в то время, когда не было сделано еще ни одного нажатия, то есть строку отрисовывать нельзя. Мы инициализируем MouseClick в FALSE и меняем ее значение в TRUE, когда происходит нажатие на мышь. Если по крайней мере одно нажатие на мышь произошло, она вырисовывает строку в клиентской области в позиции, где была мышь при нажатии. Заметьте, что она вызывает strlen для того, чтобы определить длину строки и шлет полученное значение в качестве последнего параметра функции TextOut.

Win32 API. Урок 8. Меню

Автор: lczelion, пер. Aquila

Дата: 2002-05-08

В этом tutorialе мы научимся, как вставить в наше окно меню.

Скачайте пример 1 (находится в архиве `wasm_files.zip: files/recovered/1001008/tut08-1.zip`) и пример 2 (находится в архиве `wasm_files.zip: files/recovered/1001008/tut08-2.zip`).

ТЕОРИЯ

Меню - это один из важнейших компонентов вашего окна. В меню является списком всех возможностей, которые программа предлагает пользователю. Пользователь не обязан читать мануал, поставляемый с программой, чтобы использовать ее (весьма спорная точка зрения - прим. пер.), он может досконально исследовать меню, чтобы получить представление о возможностях данной программы и начать 'играть' с ней немедленно. Так как меню - это инструмент для того, чтобы дать пользователю 'быстрый старт', вы должны следовать стандарту.

Короче говоря, первый два пункта меню должны быть "File" и "Edit", а последний - "Help". Вы можете вставить ваши собственные пункты между "Edit" и "Help". Если пункт меню вызывает диалоговое окно, вам нужно заканчивать название пункта эллипсисом (...).

Меню - это разновидность ресурсов. Есть несколько видов ресурсов, таких как диалоговые окна, строковые таблицы, иконки, битмапы, меню и т.д. Ресурсы описываются в отдельном файле, называемом файлом ресурсов, который, как правило, имеет расширение .rc. Вы можете соединять ресурсы с исходным кодом во время стадии линковки. Окончательный продукт - это исполняемый файл, который содержит как инструкции, так и ресурсы.

Вы можете писать файлы ресурсов, используя любой текстовый редактор. Они состоят из набора фраз, определяющих внешний вид и другие атрибуты ресурсов, используемых в программе. Хотя вы можете писать файлы ресурсов в текстовом редакторе, это довольно тяжело. Лучшей альтернативой является использование редактора ресурсов, который позволит вам визуальное создавать дизайн ваших ресурсов. Редакторы ресурсов обычно входят в пакет с компиляторами, такими как Visual C++, Borland C++ и т.д.

Вы описываете ресурс меню примерно так:

```
MyMenu MENU
{
    [menu list here]
}
```

Си-программисты могут заметить, что это похоже на объявление структуры. MyMenu - это имя меню, за ним следует ключевое слово MENU и список пунктов меню, заключенный в фигурные скобки. Вместо них вы можете использовать BEGIN и END. Этот вариант больше понравится программистам на Паскале.

Список меню включает в себя выражения 'MENUITEM' или 'POPUP'.

'MENUITEM' определяет пункт меню, который не является подменю. Его синтаксис следующий:

```
MENUITEM "&text", ID [,options]
```

Выражение начинается ключевым словом 'MENUITEM', за которым следует текст, который будет отображаться. Обратите внимание на амперсанд. Его действие заключается в том, что следующий за ним символ будет подчеркнут. Затем идет строка в качестве ID пункта меню. ID - это номер, который будет использоваться для обозначения пункта меню в сообщении, посылаемое процедуре окно, когда этот пункт меню будет выбран. Каждое ID должно быть уникальным.

Опции опциональны. Доступны следующие опции:

- *GRAYED - пункт меню неактивен, и он не генерирует сообщение WM_COMMAND. Текст серого цвета.
- *INACTIVE - пункт меню неактивен, и он не генерирует сообщение WM_COMMAND. Текст отображается нормально.
- *MENUBREAK - этот пункт меню и последующие пункты отображаются после новой линии меню.
- *HELP - этот пункт меню и последующие пункты выравнены по правой стороне.

Вы можете использовать одну из вышеописанных опций или комбинировать их оператором "or". Учтите, что 'INACTIVE' и 'GRAYED' не могут комбинироваться вместе. Выражение 'POPUP' имеет следующий синтаксис:

```
POPUP "&text" [,options]
{
    [menu list]
}
```

Выражение 'POPUP' определяет пункт меню, при выборе которого выпадает список пунктов в маленьком роруп-окне. Список меню может быть выражением 'MENUITEM' или 'POPUP'. Есть специальный вид выражения 'MENUITEM' - 'MENUITEM SEPARATOR', который отрисовывает горизонтальную линию в роруп-окне.

Последний шаг - это ссылка на ваш скрипт ресурса меню в программе.

Вы можете сделать это в двух разных местах.

- В члене lpszMenuName структуры WNDCLASSEX. Скажем, если у вас было меню под названием "FirstMenu", вы можете присоединить меню к вашему окну следующим образом:

```
.DATA
    MenuName db "FirstMenu",0
    .....
    .....
.CODE
    .....
    mov     wc.lpszMenuName, OFFSET MenuName
    .....
```

- *С помощью параметра-хэндла меню в функции CreateWindowEx:

```
.DATA
    MenuName db "FirstMenu",0
    hMenu HMENU ?
    .....
    .....
.CODE
    .....
    invoke LoadMenu, hInst, OFFSET MenuName
    mov     hMenu, eax
    invoke CreateWindowEx,NULL,OFFSET ClsName,\
        OFFSET Caption, WS_OVERLAPPEDWINDOW,\
        CW_USEDEFAULT,CW_USEDEFAULT,\
        CW_USEDEFAULT,CW_USEDEFAULT,\
        NULL,\
        hMenu,\
        hInst,\
        NULL\
    .....
```

Вы можете спросить, в чем разница между этими двумя методами? Когда вы делаете ссылку на меню в структуре WNDCLASSEX, меню становится меню по умолчанию для данного класса окна. Каждое окно этого класса будет иметь такое меню.

Если вы хотите, чтобы каждое окно, созданное из одного класса, имело разное меню, вы можете выбрать второй подход. В этом случае, любое окно, которому передается хэндл меню в функции `CreateWindowEx` будет иметь меню, которое замещает меню по умолчанию, указанное в структуре `WNDCLASSEX`. Сейчас мы узнаем, как меню уведомляет процедуру окна о том, что пользователь выбрал пункт меню.

Когда пользователь выберет пункт меню, процедура окна получит сообщение `WM_COMMAND`. Нижнее слово `wParam`'а содержит ID выбранного пункта меню.

Теперь у нас достаточно информации для того, чтобы создать и использовать меню. Давайте сделаем это.

ПРИМЕР

Первый пример показывает нам как создать и использовать меню, указав имя меню в классе окна.

```
.386
.model flat,stdcall
option casemap:none

WinMain proto :DWORD,:DWORD,:DWORD,:DWORD

include \masm32\include\windows.inc
include \masm32\include\user32.inc
include \masm32\include\kernel32.inc
includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib

.data

ClassName db "SimpleWinClass",0
AppName db "Our First Window",0
MenuName db "FirstMenu",0 ; The name of our menu in the resource file.

Test_string db "You selected Test menu item",0
Hello_string db "Hello, my friend",0
Goodbye_string db "See you again, bye",0

.data?

hInstance HINSTANCE ?
CommandLine LPSTR ?

.const

IDM_TEST equ 1 ; Menu IDs
IDM_HELLO equ 2
IDM_GOODBYE equ 3
IDM_EXIT equ 4

.code

start:

    invoke GetModuleHandle, NULL
    mov hInstance,eax
    invoke GetCommandLine

    mov CommandLine,eax
    invoke WinMain, hInstance,NULL,CommandLine, SW_SHOWDEFAULT
    invoke ExitProcess,eax

WinMain proc hInst:HINSTANCE,hPrevInst:HINSTANCE,CmdLine:LPSTR,CmdShow:DWORD
    LOCAL wc:WNDCLASSEX
    LOCAL msg:MSG
    LOCAL hwnd:HWND

    mov wc.cbSize,SIZEOF WNDCLASSEX
    mov wc.style, CS_HREDRAW or CS_VREDRAW
```

```

mov     wc.lpfWndProc, OFFSET WndProc
mov     wc.cbClsExtra,NULL
mov     wc.cbWndExtra,NULL
push    hInst
pop      wc.hInstance
mov     wc.hbrBackground,COLOR_WINDOW+1
mov     wc.lpszMenuName,OFFSET MenuName      ; Put our menu name here
mov     wc.lpszClassName,OFFSET ClassName
invoke  LoadIcon,NULL,IDI_APPLICATION
mov     wc.hIcon,eax
mov     wc.hIconSm,eax
invoke  LoadCursor,NULL,IDC_ARROW
mov     wc.hCursor,eax

invoke  RegisterClassEx, addr wc
invoke  CreateWindowEx,NULL,ADDR ClassName,ADDR AppName,\
        WS_OVERLAPPEDWINDOW,CW_USEDEFAULT,\
        CW_USEDEFAULT,CW_USEDEFAULT,CW_USEDEFAULT,NULL,NULL,\
        hInst,NULL

mov     hwnd,eax
invoke  ShowWindow, hwnd,SW_SHOWNORMAL
invoke  UpdateWindow, hwnd

.WHILE TRUE

        invoke GetMessage, ADDR msg,NULL,0,0
        .BREAK .IF (!eax)
        invoke DispatchMessage, ADDR msg

.ENDW

mov     eax,msg.wParam
ret

WinMain endp

WndProc proc hwnd:HWND, uMsg:UINT, wParam:WPARAM, lParam:LPARAM
    .IF uMsg==WM_DESTROY
        invoke PostQuitMessage,NULL
    .ELSEIF uMsg==WM_COMMAND
        mov     eax,wParam
        .IF ax==IDM_TEST
            invoke MessageBox,NULL,ADDR Test_string,OFFSET AppName,MB_OK
        .ELSEIF ax==IDM_HELLO
            invoke MessageBox, NULL,ADDR Hello_string, OFFSET AppName,MB_OK
        .ELSEIF ax==IDM_GOODBYE
            invoke MessageBox,NULL,ADDR Goodbye_string, OFFSET AppName, MB_OK
        .ELSE
            invoke DestroyWindow,hwnd
        .ENDIF
    .ELSE
        invoke DefWindowProc,hwnd,uMsg,wParam,lParam
        ret
    .ENDIF
    xor     eax,eax
    ret
WndProc endp

end start

*****Menu.
rc*****

#define IDM_TEST 1
#define IDM_HELLO 2
#define IDM_GOODBYE 3
#define IDM_EXIT 4

FirstMenu MENU

```

```

{
    POPUP "&PopUp"
    {
        MENUITEM "&Say Hello",IDM_HELLO
        MENUITEM "Say &GoodBye", IDM_GOODBYE
        MENUITEM SEPARATOR
        MENUITEM "E&xit",IDM_EXIT
    }

    MENUITEM "&Test", IDM_TEST
}

```

Анализ:

Давайте сначала проанализируем файл ресурсов.

```

#define IDM_TEST 1                /* все равно, что IDM_TEST equ 1*/
#define IDM_HELLO 2
#define IDM_GOODBYE 3
#define IDM_EXIT 4

```

Вышенаписанные линии определяют ID пунктов меню. Вы можете присваивать ID любое значение, главное, чтобы оно было уникально.

FirstMenu MENU

Определите ваше меню ключевым словом 'MENU'.

```

POPUP "&PopUp"
{
    MENUITEM "&Say Hello",IDM_HELLO
    MENUITEM "Say &GoodBye", IDM_GOODBYE
    MENUITEM SEPARATOR
    MENUITEM "E&xit",IDM_EXIT
}

```

Определите роупер-меню с четырьмя пунктами меню, третье - это сепаратор.

MENUITEM "&Test", IDM_TEST

Определите пункт меню в основном меню.

Далее мы изучим исходный код.

```

MenuName db "FirstMenu",0          ; Имя нашего меню в файле ресурсов
Test_string db "You selected Test menu item",0
Hello_string db "Hello, my friend",0
Goodbye_string db "See you again, bye",0

```

'MenuName' - это имя меню в файле ресурсов. Заметьте, что вы можете определить более, чем одно меню в файле ресурсов, поэтому вы можете указывать, какое меню хотите использовать. Оставшиеся три линии определяют текстовые строки, которые будут отображаться в messagebox'e при выборе соответствующего пункта меню пользователем.

```

IDM_TEST equ 1                    ; ID меню
IDM_HELLO equ 2
IDM_GOODBYE equ 3
IDM_EXIT equ 4

```

Определите ID меню для использования в процедуре окна. Эти значения должны совпадать с теми, что были определены в файле ресурсов.

```

.ELSEIF uMsg==WM_COMMAND
    mov eax,wParam
    .IF ax==IDM_TEST
        invoke MessageBox,NULL,ADDR Test_string,OFFSET AppName,MB_OK
    .ELSEIF ax==IDM_HELLO
        invoke MessageBox, NULL,ADDR Hello_string, OFFSET AppName,MB_OK
    .ELSEIF ax==IDM_GOODBYE
        invoke MessageBox,NULL,ADDR Goodbye_string, OFFSET AppName, MB_OK
    .ELSE

```

```

        invoke DestroyWindow,hWnd
    .ENDIF

```

В процедуре окна мы обрабатываем сообщение WM_COMMAND. Когда пользователь выбирает пункт меню, его ID посылается процедуре окна в нижнем слове wParam'a вместе с сообщением WM_COMMAND. Поэтому, когда мы сохраняем значение wParam в eax, мы сравниваем значение в eax с ID пунктов меню, определенными ранее, и поступаем соответствующим образом. В первых трех случаях, когда пользователь выбирает 'Test', 'Say Hell' и 'Say GoodBye', мы отображаем текстовую строку в messagebox'e.

Если пользователь выбирает пункт 'Exit', мы вызываем DestroyWindow с хэндлом нашего окна в качестве его параметра, которое закрывает наше окно.

Как вы можете видеть, указание имени меню в классе окна довольно просто и прямолинейно. Тем не менее, вы также можете использовать альтернативный метод для того, чтобы загружать меню в ваше окно. Я не буду воспроизводить здесь весь исходный код. Файл ресурсов такой же. Есть небольшие изменения в исходнике, которые я покажу ниже.

```

.data?
hInstance HINSTANCE ?
CommandLine LPSTR ?
hMenu HMENU ? ; handle of our menu

```

Определите переменную типа HMENU, чтобы сохранить хэндл нашего меню.

```

invoke LoadMenu, hInst, OFFSET MenuName
mov     hMenu,eax
INVOKE CreateWindowEx,NULL,ADDR ClassName,ADDR AppName,\
    WS_OVERLAPPEDWINDOW,CW_USEDEFAULT,\
    CW_USEDEFAULT,CW_USEDEFAULT,CW_USEDEFAULT,NULL,hMenu,\
    hInst,NULL

```

Перед вызовом CreateWindowEx, мы вызываем LoadMenu, передавая ему хэндл процесса и указатель на имя меню. LoadMenu возвращает хэндл нашего меню, который мы передаем CreateWindowEx.

Win32 API. Урок 9. Дочерние окна

Автор: lczelion, пер. Aquila

Дата: 2002-05-09

В этом tutorialе мы изучим дочерние элементы управления (child window controls), которые являются важными частями ввода и вывода нашей программы.

Скачайте пример здесь (находится в архиве wasm_files.zip: files/recovered/1001009/tut09.zip).

ТЕОРИЯ

Windows предоставляет несколько предопределенных классов окон, которые мы можем сразу же использовать в своих программах. Как правило, мы будем использовать их как компоненты dialog box'ов, поэтому они носят название дочерних элементов управления. Эти элементы обрабатывают сообщения от клавиатуры и мыши и уведомляют родительское окно, если их состояние изменяется. Они снимают с программистов огромный груз, поэтому вам следует использовать их так часто, как это возможно. В этом tutorialе, я положу их на обычное окно, только для того, чтобы продемонстрировать как их можно создать и использовать, но в реальности вам лучше класть их на dialog box.

Примерами предопределенных классов окон являются кнопки, списки, checkbox'ы, радиокнопки и т.д.

Чтобы использовать дочернее окно, вы должны создать его с помощью функции `CreateWindow` или `CreateWindowEx`. Заметьте, что вы не должны регистрировать класс окна, так как он уже был зарегистрирован Windows. Имя класса окна должно быть именем предопределенного класса. Скажем, если вы хотите создать кнопку, вы должны указать "button" в качестве имени класса в `CreateWindowEx`. Другие параметры, которые вы должны указать - это хэндл родительского окна и ID контрола. ID контрола должно быть уникальным. Вы используете его для того, чтобы отличать данный контрол от других.

После того, как контрол был создан, он посылает сообщение, уведомляющее родительское окно об изменении своего состояния. Обычно вы создаете дочернее окно во время обработки сообщения `WM_CREATE` главного окна. Дочернее окно посылает сообщение `WM_COMMAND` родительскому окну со своим ID в нижнем слове `WParam`'а, код уведомления в верхнем слове `WParam`'а, а ее хэндл в `LPARAM`'е. Каждое окно имеет разные коды уведомления, сверьтесь с вашим справочником по Win32 API, чтобы получить подробную информацию.

Родительское окно также может посылать команды дочерним окнам, вызывая функцию `SendMessage`. Функция `SendMessage` посылает определенные сообщения с сопутствующими значениями в `WParam` и `LPARAM` окну, чей хэндл передается функции. Это очень полезная функция, так как она может посылать сообщения любому окну, хэндл которого у вас есть.

Поэтому, после создания дочерних окон, родительское окно должно обрабатывать сообщения `WM_COMMAND`, чтобы быть способным получать коды уведомления от дочерних окон.

ПРИМЕР

Мы создадим окно, которое содержит edit-контрол и pushbutton. Когда вы нажмете на кнопку, появится окно, отображающее текст, введенный в edit box'е. Также имеется меню с 4 пунктами:

1. Say Hello - ввести текстовую строку в edit box
2. Clear Edit Box - очистить содержимое edit box'а
3. Get Text - отобразить окно с текстом в edit box'е

4. Exit - закрыть программу

```
.386
.model flat,stdcall
option casemap:none

WinMain proto :DWORD,:DWORD,:DWORD,:DWORD

include \masm32\include\windows.inc
include \masm32\include\user32.inc
include \masm32\include\kernel32.inc
includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib

.data

ClassName db "SimpleWinClass",0
AppName db "Our First Window",0
MenuName db "FirstMenu",0
ButtonClassName db "button",0
ButtonText db "My First Button",0
EditClassName db "edit",0
TestString db "Wow! I'm in an edit box now",0

.data?

hInstance HINSTANCE ?
CommandLine LPSTR ?
hwndButton HWND ?
hwndEdit HWND ?
buffer db 512 dup(?) ; buffer to store the text retrieved from the edit box

.const

ButtonID equ 1 ; The control ID of the button control
EditID equ 2 ; The control ID of the edit control
IDM_HELLO equ 1
IDM_CLEAR equ 2
IDM_GETTEXT equ 3
IDM_EXIT equ 4

.code

start:
    invoke GetModuleHandle, NULL
    mov hInstance,eax
    invoke GetCommandLine
    mov CommandLine,eax
    invoke WinMain, hInstance,NULL,CommandLine, SW_SHOWDEFAULT
    invoke ExitProcess,eax

WinMain proc hInst:HINSTANCE,hPrevInst:HINSTANCE,CmdLine:LPSTR,CmdShow:DWORD
    LOCAL wc:WNDCLASSEX
    LOCAL msg:MSG
    LOCAL hwnd:HWND

    mov wc.cbSize,SIZEOF WNDCLASSEX
    mov wc.style, CS_HREDRAW or CS_VREDRAW
    mov wc.lpfnWndProc, OFFSET WndProc
    mov wc.cbClsExtra,NULL
    mov wc.cbWndExtra,NULL
    push hInst
    pop wc.hInstance
    mov wc.hbrBackground,COLOR_BTNFACE+1
    mov wc.lpszMenuName,OFFSET MenuName
    mov wc.lpszClassName,OFFSET ClassName

    invoke LoadIcon,NULL,IDI_APPLICATION
    mov wc.hIcon,eax
    mov wc.hIconSm,eax
    invoke LoadCursor,NULL,IDC_ARROW
```

```

mov     wc.hCursor,eax
invoke RegisterClassEx, addr wc
invoke CreateWindowEx,WS_EX_CLIENTEDGE,ADDR ClassName, \
                    ADDR AppName, WS_OVERLAPPEDWINDOW,\
                    CW_USEDEFAULT, CW_USEDEFAULT,\
                    300,200,NULL,NULL, hInst,NULL

mov     hwnd,eax
invoke ShowWindow, hwnd,SW_SHOWNORMAL
invoke UpdateWindow, hwnd
.WHILE TRUE
    invoke GetMessage, ADDR msg,NULL,0,0
    .BREAK .IF (!eax)
    invoke TranslateMessage, ADDR msg
    invoke DispatchMessage, ADDR msg
.ENDW
mov     eax,msg.wParam
ret

WinMain endp

WndProc proc hwnd:HWND, uMsg:UINT, wParam:WPARAM, lParam:LPARAM
    .IF uMsg==WM_DESTROY
        invoke PostQuitMessage,NULL
    .ELSEIF uMsg==WM_CREATE
        invoke CreateWindowEx,WS_EX_CLIENTEDGE, ADDR EditClassName,NULL,\
                    WS_CHILD or WS_VISIBLE or WS_BORDER or ES_LEFT or\
                    ES_AUTOHSCROLL,\
                    50,35,200,25,hwnd,8,hInstance,NULL

        mov     hwndEdit,eax
        invoke SetFocus, hwndEdit
        invoke CreateWindowEx,NULL, ADDR ButtonClassName,ADDR ButtonText,\
                    WS_CHILD or WS_VISIBLE or BS_DEFPUSHBUTTON,\
                    75,70,140,25,hwnd,ButtonID,hInstance,NULL

        mov     hwndButton,eax
    .ELSEIF uMsg==WM_COMMAND
        mov     eax,wParam
        .IF lParam==0
            .IF ax==IDM_HELLO
                invoke SetWindowText,hwndEdit,ADDR TestString
            .ELSEIF ax==IDM_CLEAR
                invoke SetWindowText,hwndEdit,NULL
            .ELSEIF ax==IDM_GETTEXT
                invoke GetWindowText,hwndEdit,ADDR buffer,512
                invoke MessageBox,NULL,ADDR buffer,ADDR AppName,MB_OK
            .ELSE
                invoke DestroyWindow,hwnd
            .ENDIF
        .ELSE
            .IF ax==ButtonID
                shr     eax,16
                .IF ax==BN_CLICKED
                    invoke SendMessage,hwnd,WM_COMMAND,IDM_GETTEXT,0
                .ENDIF
            .ENDIF
        .ENDIF
    .ELSE
        invoke DefWindowProc,hwnd,uMsg,wParam,lParam
        ret
    .ENDIF
xor     eax,eax

ret

WndProc endp
end start

```

АНАЛИЗ

Давайте проанализируем программу.

```
.ELSEIF uMsg==WM_CREATE
```



```

invoke CreateWindowEx,WS_EX_CLIENTEDGE, \
    ADDR EditClassName,NULL,\
    WS_CHILD or WS_VISIBLE or WS_BORDER or ES_LEFT\
    or ES_AUTOHSCROLL,\
    50,35,200,25,hWnd,EditID,hInstance,NULL
mov  hWndEdit,eax
invoke SetFocus, hWndEdit
invoke CreateWindowEx,NULL, ADDR ButtonClassName,\
    ADDR ButtonText,\
    WS_CHILD or WS_VISIBLE or BS_DEFPUSHBUTTON,\
    75,70,140,25,hWnd,ButtonID,hInstance,NULL
mov  hWndButton,eax

```

Мы создаем контролы во время обработки сообщения WM_CREATE. Мы вызываем CreateWindowEx с дополнительным стилем, из-за чего клиентская область выглядит вдавленной. Имя каждого контрола предопределенно - "edit" для edit-контрола, "button" для кнопки. Затем мы указываем стили дочерних окон. У каждого контрола есть дополнительные стили, кроме обычных стилей окна. Например, стили кнопок начинаются с "BS_", стили edit'a - с "ES_". Вы должны посмотреть информацию об этих стилях в вашем справочнике по Win32 API. Заметьте, что вместо хэндла меню вы передаете ID контрола. Это не вызывает никаких противоречий, поскольку дочерний элемент управления не может иметь меню. После создания каждого контрола, мы сохраняем его хэндл в соответствующей переменной для будущего использования.

SetFocus вызывается для того, чтобы направить фокус ввода на edit box, чтобы пользователь мог сразу начать вводить в него текст.

```

.ELSEIF uMsg==WM_COMMAND
    mov  eax,wParam
    .IF lParam==0

```

Обратите внимание, что меню тоже шлем сообщение WM_COMMAND, чтобы уведомить окно о своем состоянии. Как мы можем провести различие между сообщениями WM_COMMAND, исходящими от меню и контролов? Вот ответ:

Нижнее слово wParam	Верхнее слово wParam	lParam
ID меню	0	0
ID контрола	Код уведомления	Хэндл дочернего окна

Вы можете видеть, что вы должны проверить lParam. Если он равен нулю, текущее сообщение WM_COMMAND было послано меню. Вы не можете использовать wParam, чтобы различать меню и контрол, так как ID меню и ID контрола могут быть идентичными и код уведомления должен быть равен нулю.

```

. IF ax==IDM_HELLO
    invoke SetWindowText,hWndEdit,ADDR TestString
.ELSEIF ax==IDM_CLEAR
    invoke SetWindowText,hWndEdit,NULL
.ELSEIF ax==IDM_GETTEXT
    invoke GetWindowText,hWndEdit,ADDR buffer,512
    invoke MessageBox,NULL,ADDR buffer,ADDR AppName,MB_OK

```

Вы можете поместить текстовую строку в edit box с помощью вызова SetWindowText. Вы очищаете содержимое edit box'a с помощью вызова SetWindowText, передавая ей NULL. SetWindowText - это функция общего назначения. Вы можете использовать ее, чтобы изменить заголовок окна или текст на кнопке. Чтобы получить текст в edit box'e, вы можете использовать GetWindowText.

```

. IF ax==ButtonID
    shr  eax,16
    . IF ax==BN_CLICKED
        invoke SendMessage,hWnd,WM_COMMAND,IDM_GETTEXT,0
    .ENDIF
.ENDIF

```

Приведенный выше кусок кода является обработкой нажатия на кнопку. Сначала он проверяет нижнее слово `wParam`'а, чтобы убедиться, что ID контрола принадлежит кнопке. Если это так, он проверяет верхнее слово `wParam`'а, чтобы убедиться, что был послан код уведомления `BN_CLICKED`, то есть кнопка была нажата.

После этого идет собственно обработка нажатия на клавиш. Мы хотим получить текст из `edit box`'а и отобразить его в `message box`'е. Мы можем продублировать код в секции `IDM_GETTEXT` выше, но это не имеет смысла. Если мы сможем каким-либо образом послать сообщение `WM_COMMAND` с нижним словом `wParam`, содержащим значение `IDM_GETTEXT` нашей процедуры окна, то избежим дублирования кода и упростим программу. Функция `SendMessage` - это ответ. Эта функция посылает любое сообщение любому окну с любым `wParam`'ом и `lParam`'ом, которые нам понадобятся. Поэтому вместо дублирования кода мы вызываем `SendMessage` с хэндлом родительского окна, `WM_COMMAND`, `IDM_GETTEXT` и 0. Это дает тот же эффект, что и выбор пункта меню "Get Text". Процедура окна не почувствует никакой разницы.

Вы должны использовать эту технику так часто, насколько возможно, чтобы сделать ваш код более упорядоченным.

И напоследок. Не забудьте функцию `TranslateMessage` в очереди сообщений. Так как вам нужно печатать текст в `edit box`'е, ваша программа должна транслировать ввод в читабельный текст. Если вы пропустите эту функцию, вы не сможете напечатать что-либо в вашем `edit box`'е.

Win32 API. Урок 10. Диалоговое окно как основное

Автор: lczelion, пер. Aquila

Дата: Не указана

Теперь время для действительно интересной темы, относящейся к GUI, о диалоговом окне. В этом tutorialе (и в следующем) мы научимся как использовать диалоговое окно в качестве основного.

Скачайте первый (находится в архиве `wasm_files.zip: files/recovered/1001010/tut10-1.zip`) и второй (находится в архиве `wasm_files.zip: files/recovered/1001010/tut10-2.zip`) примеры.

ТЕОРИЯ

Если вы изучили примеры в предыдущем tutorialе достаточно подробно, вы заметили, что вы не могли перемещать фокус ввода от одного дочернего окна на другой, используя кнопку Tab. Вы могли сделать это только кликнув на нужном контроле, чтобы перевести на него фокус. Это довольно неудобно. Также вы могли заметить, что изменил цвет родительского окна на серый. Это было сделано для того, чтобы цвет дочерних окон не контрастировал с клиентской областью родительского окна. Есть путь, чтобы обойти эту проблему, но он не очень прост. Вы должны сабклассить все дочерние элементы управления в вашем родительском окне.

Причина того, почему возникают подобные неудобства состоят в том, что дочерние окна изначально проектировались для работы с диалоговым окном, а не с обычным. Цвет дочернего окна по умолчанию серый, так как это обычный цвет диалогового окна.

Прежде чем мы углубимся в детали, мы должны сначала узнать, что такое диалоговое окно. Диалоговое окно - это ничто иное, как обычное окно, которое спроектировано для работы с дочерними элементами управления. Windows также предоставляет внутренний "менеджер диалоговых окон", который воплощает большую часть диалоговой логики, такую как перемещение фокуса ввода, когда юзер нажимает Tab, нажатие кнопки по умолчанию, если нажата кнопка 'Enter', и так далее, так чтобы программисты могли заниматься более высокоуровневыми задачами. Поскольку диалоговое окно можно считать "черной коробкой" (это означает то, что вы не обязаны знать, как работает диалоговое окно, для того, чтобы использовать его), вы должны только знать, как с ним взаимодействовать. Это принцип объектно-ориентированного программирования, называемого скрытием информации. Если черная коробка спроектирована **совершенно**, пользователь может использовать ее не зная, как она работает. Правда, загвоздка в том, что черная коробка должна быть совершенной, это труднодостижимо в реальном мире. Win32 API также спроектирован как черная коробка.

Ладно, похоже, что мы немного отклонились. Давайте вернемся к нашему сюжету. Диалоговые окна спроектированы так, чтобы снизить нагрузку на программиста. Обычно, если вы помещаете дочерний контрол на обычное окно, вы должны сабклассить их и самостоятельно обрабатывать нажатия на клавиши. Но если вы помещаете их на диалоговое окно, оно обработает их за вас. Вы только должны как получать информацию, вводимую пользователем, или как посылать команды окну. Диалоговое окно определяется как ресурс (похожим образом, как и меню). Вы пишете шаблон диалогового окна, описывая характеристики диалогового окна и его контролов, а затем компилируете его с помощью редактора ресурсов.

Обратите внимание, что все ресурсы располагаются в одной скрипте ресурсов. Вы можете использовать любой текстовый редактор, чтобы написать шаблон диалогового окна, но я бы не рекомендовал это. Вы должны использовать редактор ресурсов, чтобы сделать визуально расположить дочерние окна. Существует несколько прекрасных редакторов ресурсов. К большинству из основных компиляторов прилагаются подобные редакторы. Вы можете использовать их, чтобы создать скрипт ресурса. После этого стоит вырезать лишние линии,

например, те, которые относятся к MFC.

Есть два основных вида диалоговых окон: модальные и независимые. Независимые диалоговые окна дают вам возможность перемещать фокус ввода на другие окна. Пример - диалоговое окно 'Find' в MS Word. Есть два подтипа модальных диалоговых окон: модальные к приложению и модальные к системе. Первые не дают вам переключаться на другое окно того же приложения, но вы можете переключиться на другое приложение. Вторые не дают вам возможности переключиться на любое другое окно.

Независимое диалоговое окно создается с помощью вызова функции `CreateDialogParam`. Модальное диалоговое окно создается вызовом `DialogBoxParam`. Единственное различие между диалоговым окном, модальным отношению к приложению, и диалоговым окном, модальным по отношению к системе, - это стиль `DS_SYSMODAL`. Если вы включите стиль `DS_SYSMODAL` в шаблон диалогового окна, это диалоговое окно будет модальным к системе.

Вы можете взаимодействовать с любым дочерним элементом управления на диалоговом окне с помощью функции `SendDlgItemMessage`. Ее синтаксис следующий:

```
SendDlgItemMessage proto hwndDlg:DWORD,\
                                idControl:DWORD,\
                                uMsg:DWORD,\
                                wParam:DWORD,\
                                lParam:DWORD
```

Эта API-функция неоценимо полезна при взаимодействии с дочерним окном. Например, если вы хотите получить текст с контрола `edit`, вы можете сделать следующее:

```
call SendDlgItemMessage, hDlg, ID_EDITBOX, WM_GETTEXT, 256, ADDR text_buffer
```

Чтобы знать, какое сообщение когда посылать, вы должны проконсультироваться с вашим Win32 API-справочником.

Windows также предоставляет несколько специальных API-функций, заточенных под дочерние окна, для быстрого получения и установки нужных данных, например, `GetDlgItemText`, `CheckDlgButton` и т.д. Эти специальные функции созданы, чтобы программисту не приходилось выяснять каждый раз значения `wParam` и `lParam`. Как правило, вы должны использовать данные функции, если хотите, чтобы управление кодом было легче. Используйте `SendDlgItemMessage` только, если нет соответствующей API-функции. Менеджер диалоговых окон посылает некоторые сообщения специальной callback-функции, называемой процедурой диалогового окна, которая имеет следующий формат:

```
DlgProc proto hDlg:DWORD ,\
                                iMsg:DWORD ,\
                                wParam:DWORD ,\
                                lParam:DWORD
```

Процедура диалогового окна очень похожа на процедуру окна, если не считать тип возвращаемого значения - `TRUE/FALSE`, вместо обычных `LRESULT`. Внутренний менеджер диалоговых окон внутри Windows - истинная процедура для диалоговых окон. Она вызывает нашу процедуру диалоговых окон, передавая некоторые из полученных сообщений. Поэтому главное правило следующее: если наша процедура диалогового окна обрабатывает сообщение, она должна вернуть `TRUE` в `eax` и если она не обрабатывает сообщение, тогда она должна вернуть в `eax` `FALSE`. Заметьте, что процедура диалогового окна не передает сообщения функции `DefWindowProc`, так как это не настоящая процедура окна.

Диалоговое окно можно использовать в двух целях. Вы можете использовать ее как основное окно или как вспомогательное для получения информации, вводимой пользователем. В этом tutorialе мы изучим первый вариант.

"Использование диалогового окна как основное окно" можно понимать двумя образами.

- Вы можете использовать шаблон диалогового окна как шаблон класса, который вы регистрируете с помощью функции RegisterClassEx. В этом случае, диалоговое окно ведет себя как "нормальное": оно получает сообщения через процедуру окна, на которую ссылается lpfnWndProc, а не через процедуру диалогового окна. Выгода данного подхода состоит в том, что вы не должны самостоятельно создавать дочерние элементы управления, Windows создает их во время создания диалогового окна. Также Windows берет на себя логику нажатий на клавиши (Tab и т.д.). Плюс вы можете указать курсор и иконку вашего окна в структуре класса окна.

- Ваша программа создает диалоговое окно без создания родительского окна. Этот подход делает цикл сообщений ненужным, так как сообщения шлются напрямую процедуре диалогового окна. Вам даже не нужно регистрировать класс окна!

Похоже, что этот tutorial будет довольно долгим.

ПРИМЕР

```

                                dialog.asm

.386
.model flat,stdcall
option casemap:none
WinMain proto :DWORD,:DWORD,:DWORD,:DWORD
include \masm32\include\windows.inc
include \masm32\include\user32.inc
include \masm32\include\kernel32.inc
includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib

.data

ClassName db "DLGCLASS",0
MenuName db "MyMenu",0
DlgName db "MyDialog",0
AppName db "Our First Dialog Box",0
TestString db "Wow! I'm in an edit box now",0

.data?

hInstance HINSTANCE ?
CommandLine LPSTR ?
buffer db 512 dup(?)

.const

IDC_EDIT            equ 3000
IDC_BUTTON          equ 3001
IDC_EXIT            equ 3002
IDM_GETTEXT         equ 32000
IDM_CLEAR           equ 32001
IDM_EXIT            equ 32002

.code

start:

    invoke GetModuleHandle, NULL
    mov     hInstance,eax
    invoke GetCommandLine
    mov     CommandLine,eax
    invoke WinMain, hInstance,NULL,CommandLine, SW_SHOWDEFAULT
    invoke ExitProcess,eax

WinMain proc hInst:HINSTANCE,hPrevInst:HINSTANCE,CmdLine:LPSTR,CmdShow:DWORD

    LOCAL wc:WNDCLASSEX
    LOCAL msg:MSG
    LOCAL hDlg:HWND
    mov     wc.cbSize,SIZEOF WNDCLASSEX

```

```

mov     wc.style, CS_HREDRAW or CS_VREDRAW
mov     wc.lpfWndProc, OFFSET WndProc
mov     wc.cbClsExtra, NULL
mov     wc.cbWndExtra, DLGWINDOWEXTRA
push    hInst
pop      wc.hInstance
mov     wc.hbrBackground, COLOR_BTNFACE+1
mov     wc.lpszMenuName, OFFSET MenuName
mov     wc.lpszClassName, OFFSET ClassName
invoke  LoadIcon, NULL, IDI_APPLICATION
mov     wc.hIcon, eax
mov     wc.hIconSm, eax
invoke  LoadCursor, NULL, IDC_ARROW
mov     wc.hCursor, eax
invoke  RegisterClassEx, addr wc
invoke  CreateDialogParam, hInstance, ADDRDlgName, NULL, NULL, NULL
mov     hDlg, eax
invoke  ShowWindow, hDlg, SW_SHOWNORMAL
invoke  UpdateWindow, hDlg
invoke  GetDlgItem, hDlg, IDC_EDIT
invoke  SetFocus, eax

.WHILE TRUE
    invoke GetMessage, ADDR msg, NULL, 0, 0
    .BREAK .IF (!eax)
    invoke IsDialogMessage, hDlg, ADDR msg
    .IF eax == FALSE
        invoke TranslateMessage, ADDR msg
        invoke DispatchMessage, ADDR msg
    .ENDIF
.ENDW
mov     eax, msg.wParam

ret

WinMain endp

WndProc proc hWnd:HWND, uMsg:UINT, wParam:WPARAM, lParam:LPARAM

    .IF uMsg==WM_DESTROY
        invoke PostQuitMessage, NULL
    .ELSEIF uMsg==WM_COMMAND
        mov     eax, wParam
        .IF lParam==0
            .IF ax==IDM_GETTEXT
                invoke GetDlgItemText, hWnd, IDC_EDIT, ADDR buffer, 512
                invoke MessageBox, NULL, ADDR buffer, ADDR AppName, MB_OK
            .ELSEIF ax==IDM_CLEAR
                invoke SetDlgItemText, hWnd, IDC_EDIT, NULL
            .ELSE
                invoke DestroyWindow, hWnd
            .ENDIF
        .ELSE
            mov     edx, wParam
            shr     edx, 16
            .IF dx==BN_CLICKED
                .IF ax==IDC_BUTTON
                    invoke SetDlgItemText, hWnd, IDC_EDIT, ADDR TestString
                .ELSEIF ax==IDC_EXIT
                    invoke SendMessage, hWnd, WM_COMMAND, IDM_EXIT, 0
                .ENDIF
            .ENDIF
        .ENDIF
    .ELSE
        invoke DefWindowProc, hWnd, uMsg, wParam, lParam
    .ENDIF

    ret

WndProc endp

```

```
WndProc endp
```

```
end start
```

Dialog.rc

```
#include "resource.h"

#define IDC_EDIT 3000
#define IDC_BUTTON 3001
#define IDC_EXIT 3002
#define IDM_GETTEXT 32000
#define IDM_CLEAR 32001
#define IDM_EXIT 32003
MyDialog DIALOG 10, 10, 205, 60
STYLE 0x0004 | DS_CENTER | WS_CAPTION | WS_MINIMIZEBOX |
WS_SYSMENU | WS_VISIBLE | WS_OVERLAPPED | DS_MODALFRAME | DS_3DLOOK
CAPTION "Our First Dialog Box"
CLASS "DLGCLASS"
BEGIN
    EDITTEXT IDC_EDIT, 15,17,111,13, ES_AUTOHSCROLL | ES_LEFT
    DEFPUSHBUTTON "Say Hello", IDC_BUTTON, 141,10,52,13
    PUSHBUTTON "E&xit", IDC_EXIT, 141,26,52,13, WS_GROUP
END

MyMenu MENU
BEGIN
    POPUP "Test Controls"
    BEGIN
        MENUITEM "Get Text", IDM_GETTEXT
        MENUITEM "Clear Text", IDM_CLEAR
        MENUITEM "", 0x0800 /*MFT_SEPARATOR*/
        MENUITEM "E&xit", IDM_EXIT
    END
END
```

АНАЛИЗ

Давайте проанализируем первый пример.

Этот пример показывает, как зарегистрировать диалоговый шаблон как класс окна и создает "окно" из этого класса. Это упрощает вашу программу, так как вам не нужно создавать дочерние контролы самостоятельно.

Давайте проанализируем шаблон диалогового окна.

```
MyDialog DIALOG 10, 10, 205, 60
```

Объявление имя диалога, в данном случае - "MyDialog", за которым следует ключевое слово "DIALOG". Следующие четыре номера - это x, y, ширина и высота диалогового окна в специальных единицах (не в пикселях).

```
STYLE 0x0004 | DS_CENTER | WS_CAPTION | WS_MINIMIZEBOX |
WS_SYSMENU | WS_VISIBLE | WS_OVERLAPPED | DS_MODALFRAME | DS_3DLOOK
```

Объявление стилей диалогового окна.

```
CAPTION "Our First Dialog Box"
```

Это текст, который появится в title bar'e.

```
CLASS "DLGCLASS"
```

Это ключевая строка. Ключевое слово 'CLASS' позволяет нам использовать шаблон диалогового окна в качестве класса окна. Следующее слово - это имя "класса окна".

```
BEGIN
    EDITTEXT IDC_EDIT, 15,17,111,13, ES_AUTOHSCROLL | ES_LEFT
    DEFPUSHBUTTON "Say Hello", IDC_BUTTON, 141,10,52,13
    PUSHBUTTON "E&xit", IDC_EXIT, 141,26,52,13
END
```

Данный блок определяет дочерние элементы управления в диалоговом окне. Они определены между ключевыми словами BEGIN и END. Общий синтаксис таков:

```
control-type "text" ,controlID, x, y, width, height [,styles]
```

control-type'ы - это константы компилятора ресурсов, вам нужно свериться с руководством.

Теперь мы углубляемся непосредственно в ассемблерный код. Интересующая нас часть находится в структуре класса окна.

```
mov wc.cbWndExtra,DLGWINDOWEXTRA
mov wc.lpszClassName,OFFSET ClassName
```

Обычно этот параметр оставляется равным нулю, но если мы хотим зарегистрировать шаблон диалогового окна как класс окна, мы должны установить этот параметр равным DLGWINDOWEXTRA. Заметьте, что имя класса должно совпадать с именем, что определено в шаблон диалогового окна. Остаточные параметры инициализируются как обычно. После того, как вы заполните структуру класса окна, зарегистрируйте ее с помощью RegisterClassEx. Звучит знакомо. Точно также вы регистрируете обычный класс окна.

```
invoke CreateDialogParam,hInstance,ADDRDlgName,NULL,NULL,NULL
```

После регистрации "класса окна", мы создаем наше диалоговое окно. В этом примере я создал его как независимое диалоговое окно функцией CreateDialogParam. Эта функция получает 5 параметров, но вам нужно заполнить только первые два: хэндл процесса и указатель на имя шаблона диалогового окна. Заметьте, что 2-ой параметр - это не указатель на имя класса.

В этот момент, диалоговое окно и его дочерние элементы управления создаются Windows. Ваша процедура окна получит сообщение WM_CREATE как обычно.

```
invoke GetDlgItem,hDlg, IDC_EDIT
invoke SetFocus, eax
```

После того, как диалоговое окно создано, я хочу установить фокус ввода на edit control. Если я помещу соответствующий код в секцию WM_CREATE, вызов GetDlgItem провалится, так как дочерние окна еще не созданы. Единственный путь сделать это - вызвать эту функцию после того, как диалоговое окно и все его дочерние окна будут созданы. Поэтому я помещаю данные две линии после вызова UpdateWindow. Функция GetDlgItem получает ID контрола и возвращает соответствующий хэндл окна. Так вы можете получить хэндл окна, если вы знаете его control ID.

```
invoke IsDialogMessage, hDlg, ADDR msg
.IF eax ==FALSE
    invoke TranslateMessage, ADDR msg
    invoke DispatchMessage, ADDR msg
.ENDIF
```

Программа входит в цикл сообщений и перед тем, как мы транслируем и передаем сообщения, мы вызываем функцию IsDialogMessage, чтобы позволить менеджеру диалоговых сообщений обрабатывать логику сообщений за нас. Если эта функция возвращает TRUE, это значит, что сообщение сделано для диалогового окна и обрабатывается менеджером диалоговых сообщений. Отметьте другое отличие от предыдущего tutorиала. Когда процедура окна хочет получить текст с edit контрола, она вызывает функцию GetDlgItemText, вместо функции GetWindowText. GetDlgItemText принимает ID контрола вместо хэнгла окна. Это делает вызов проще в том случае, если вы используете диалоговое окно.

Теперь давайте перейдем ко второму подходу использования диалогового окна как основного окна. В следующем примере, я создам программно-модальное диалоговое окно. Вы не увидите цикл сообщений или процедуру окна, потому что они не нужны!

dialog.asm (part 2)


```

.model flat,stdcall
option casemap:none

DlgProc proto :DWORD,:DWORD,:DWORD,:DWORD

include \masm32\include\windows.inc
include \masm32\include\user32.inc
include \masm32\include\kernel32.inc
include lib \masm32\lib\user32.lib
include lib \masm32\lib\kernel32.lib

.data

DlgName db "MyDialog",0
AppName db "Our Second Dialog Box",0
TestString db "Wow! I'm in an edit box now",0

.data?

hInstance HINSTANCE ?
CommandLine LPSTR ?
buffer db 512 dup(?)

.const

IDC_EDIT equ 3000
IDC_BUTTON equ 3001
IDC_EXIT equ 3002
IDM_GETTEXT equ 32000
IDM_CLEAR equ 32001
IDM_EXIT equ 32002

.code

start:

    invoke GetModuleHandle, NULL
    mov hInstance,eax
    invoke DialogBoxParam, hInstance, ADDR DlgName,NULL, addr DlgProc, NULL
    invoke ExitProcess,eax

DlgProc proc hWnd:HWND, uMsg:UINT, wParam:WPARAM, lParam:LPARAM
    .IF uMsg==WM_INITDIALOG
        invoke GetDlgItem, hWnd,IDC_EDIT
        invoke SetFocus,eax
    .ELSEIF uMsg==WM_CLOSE
        invoke SendMessage,hWnd,WM_COMMAND,IDM_EXIT,0
    .ELSEIF uMsg==WM_COMMAND
        mov eax,wParam
        .IF lParam==0
            .IF ax==IDM_GETTEXT
                invoke GetDlgItemText,hWnd,IDC_EDIT,ADDR buffer,512
                invoke MessageBox,NULL,ADDR buffer,ADDR AppName,MB_OK
            .ELSEIF ax==IDM_CLEAR
                invoke SetDlgItemText,hWnd,IDC_EDIT,NULL
            .ELSEIF ax==IDM_EXIT
                invoke EndDialog, hWnd,NULL
            .ENDIF
        .ELSE
            mov edx,wParam
            shr edx,16
            .if dx==BN_CLICKED
                .IF ax==IDC_BUTTON
                    invoke SetDlgItemText,hWnd,IDC_EDIT,ADDR TestString
                .ELSEIF ax==IDC_EXIT
                    invoke SendMessage,hWnd,WM_COMMAND,IDM_EXIT,0
                .ENDIF
            .ENDIF
        .ENDIF
    .ELSE
        mov eax,FALSE
    
```

```

        ret
    .ENDIF
    mov eax,TRUE

    ret

DlgProc endp

end start

```

dialog.rc (part 2)

```

#include "resource.h"

#define IDC_EDIT 3000
#define IDC_BUTTON 3001
#define IDC_EXIT 3002
#define IDR_MENU1 3003
#define IDM_GETTEXT 32000
#define IDM_CLEAR 32001
#define IDM_EXIT 32003
MyDialog DIALOG 10, 10, 205, 60
STYLE 0x0004 | DS_CENTER | WS_CAPTION | WS_MINIMIZEBOX |
WS_SYSMENU | WS_VISIBLE | WS_OVERLAPPED | DS_MODALFRAME | DS_3DLOOK
CAPTION "Our Second Dialog Box"

MENU IDR_MENU1
BEGIN
    EDITTEXT IDC_EDIT, 15,17,111,13, ES_AUTOHSCROLL | ES_LEFT
    DEFPUSHBUTTON "Say Hello", IDC_BUTTON, 141,10,52,13
    PUSHBUTTON "E&xit", IDC_EXIT, 141,26,52,13
END

IDR_MENU1 MENU
BEGIN
    POPUP "Test Controls"
    BEGIN
        MENUITEM "Get Text", IDM_GETTEXT
        MENUITEM "Clear Text", IDM_CLEAR
        MENUITEM "", , 0x0800 /*MFT_SEPARATOR*/
        MENUITEM "E&xit", IDM_EXIT
    END
END

```

АНАЛИЗ

```
DlgProc proto :DWORD,:DWORD,:DWORD,:DWORD
```

Мы объявляем прототип функции для DlgProc, так что мы можем сослаться на нее оператором addr:

```
invoke DialogBoxParam, hInstance, ADDR DlgName,NULL, addr DlgProc, NULL
```

В вышеприведенной строке вызывается функция DialogBoxParam, которая получает 5 параметров: хэндл процесса, имя шаблона диалогового окна, хэндл родительского окна, адрес процедуры диалогового окна и специальные данные для диалогового окна. DialogBoxParam создает модальное диалоговое окно. Она не возвращается, пока диалоговое окно не будет уничтожено.

```

    .IF uMsg==WM_INITDIALOG
        invoke GetDlgItem, hWnd,IDC_EDIT
        invoke SetFocus,eax
    .ELSEIF uMsg==WM_CLOSE
        invoke SendMessage,hWnd,WM_COMMAND,IDM_EXIT,0

```

Процедура диалогового окна выглядит как процедура окна, не считая того, что она не получает сообщение WM_CREATE. Первое сообщение, которое она получает - это WM_INITDIALOG. Обычно вы помещаете здесь инициализационный код. Заметьте, что вы должны вернуть в eax значение TRUE, если вы обрабатываете это сообщение.

Внутренний менеджер диалогового окна не посылает нашего процедуре сообщение WM_DESTROY, а вот WM_CLOSE шлет. Поэтому если мы хотим отреагировать на то, что юзер нажимает кнопку закрытия на нашем диалоговом окне, мы должны обработать сообщение WM_CLOSE. В нашем примере мы посылаем сообщение WM_CLOSE со значением IDM_EXIT в wParam. Это произведет тот же эффект, что и выбор пункта 'Exit' в меню. EndDialog вызывается в ответ на IDM_EXIT.

Обработка сообщений WM_COMMAND остается такой же.

Когда вы хотите уничтожить диалоговое окно, единственный путь - это вызов функции EndDialog. Не пробуйте DestroyWindow! EndDialog не уничтожает диалоговое окно немедленно. Она только устанавливает флаг для внутреннего менеджера диалогового окна и продолжает выполнять следующие инструкции.

Теперь давайте изучим файл ресурсов. Заметное изменение - это то, что вместо использования текстовой строки в качестве имени меню, мы используем значение IDR_MENU1. Это необходимо, если вы хотите прикрепить меню к диалоговому окну, созданному DialogBoxParam'ом. Заметьте, что в шаблоне диалогового окна вы должны добавить ключевое слово 'MENU', за которым будет следовать ID ресурса меню.

Различие между двумя примерами в этом руководстве, которое вы можете легко заметить - это отсутствие иконки в последнем примере. Тем не менее, вы можете установить иконку, послав сообщение WM_SETICON диалоговому окну во время обработки WM_INITDIALOG.

Win32 API. Урок 11. Больше о диалоговых окнах

Автор: lczelion, пер. Aquila

Дата: Не указана

В этом tutorialе мы узнаем больше о диалоговых окнах. В частности, мы узнаем, как использовать диалоговые окна в качестве устройств ввода-вывода. Если вы читали предыдущий tutorial, то этот будет для вас достаточно прост, так как небольшая модификация - все, что требуется для использования диалоговых окон как дополнение к основному окну. Также в этом tutorialе мы научимся тому, как использовать предопределенные диалоговые окна.

Скачайте примеры здесь (находится в архиве wasm_files.zip: files/recovered/1001011/tut11-1.zip) и здесь (находится в архиве wasm_files.zip: files/recovered/1001011/tut11-2.zip). Пример предопределенного диалогового окна скачайте здесь (находится в архиве wasm_files.zip: files/recovered/1001011/tut11-3.zip).

ТЕОРИЯ

Очень немного будет сказано о том, как использовать диалоговые окна в качестве устройств ввода-вывода. Программа создает основное окно как обычно, и когда вы хотите отобразить диалоговое окно, просто-напросто вызовите `CreateDialogParam` или `DialogBoxParam`. Вызвав `DialogBoxParam`, вам не нужно делать что-либо еще, просто обработайте сообщения в процедуре диалогового окна. При использовании `CreateDialogParam`, вам будет нужно вставить вызов `IsDialogMessage` в цикле сообщений, чтобы позволить менеджеру диалогового окна обработать навигацию клавиатуры в вашем диалоговом окне за вас. Поскольку эти два случая тривиальны, я не привожу здесь исходный код. Вы можете скачать примеры и изучить их самостоятельно.

Давайте перейдем к предопределенным диалоговым окнам, которые Windows предоставляет для использования вашими приложениями. Эти диалоговые окна существуют, чтобы обеспечить стандартизованный пользовательский интерфейс. Существуют файловое диалоговое окно, принтер, цвет, фон и поисковое диалоговое окно. Вам следует использовать их так часто, как это возможно.

Диалоговые окна находятся в `comdlg32.dll`. Чтобы использовать их, вы должны прилинковать `comdlg32.lib`. Вы создаете эти диалоговые окна вызовом соответствующих функций из библиотеки предопределенных диалоговых окон. Для открытия файлового диалогового окна существует функция `GetOpenFileName`, для сохранения - `GetSaveFileName`, для диалогового окна принтера - `PrintDlg` и так далее. Каждая из этих функций берет указатель на структуру в качестве параметра. Вам следует посмотреть их в справочнике Win32 API. В этом tutorialе я продемонстрирую как создавать и использовать файловое диалоговое окно.

Ниже приведен прототип функции `GetOpenFileName`.

```
GetOpenFileName proto lpofn:DWORD
```

Вы можете видеть, что она получает только один параметр, указатель на структуру `OPENFILENAME`. Возвращаемое значение `TRUE` значит, что пользователь выбрал файл, который нужно открыть, `FALSE` означает обратное. Сейчас мы рассмотрим структуру `OPENFILENAME`:

```
OPENFILENAME STRUCT
    IStructSize DWORD ?
    hwndOwner HWND ?
    hInstance HINSTANCE ?
    lpstrFilter LPCSTR ?
    lpstrCustomFilter LPSTR ?
    nMaxCustFilter DWORD ?
    nFilterIndex DWORD ?
    lpstrFile LPSTR ?
```

```

nMaxFile DWORD ?
lpstrFileName LPSTR ?
nMaxFileName DWORD ?
lpstrInitialDir LPCSTR ?
lpstrTitle LPCSTR ?
Flags DWORD ?
nFileOffset WORD ?
nFileExtension WORD ?
lpstrDefExt LPCSTR ?
lCustData LPARAM ?
lpfnHook DWORD ?
lpTemplateName LPCSTR ?
OPENFILENAME ENDS

```

Давайте рассмотрим значение часто используемых параметров.

IStructSize	Размер структуры OPENFILENAME в байтах.
hwndOwner	Хэндл файлового диалогового окна.
hInstance	Хэндл процесса, который создает файловое диалоговое окно.
lpstrFilter	Строка-фильтр состоит из парных строк, разделенных null'ом. Первая строка в каждой паре - это описание. Вторая строка - это шаблон фильтра. Например: FilterString db "All Files (*.*)",0, ".*",0 db "Text Files (*.txt)",0,".txt",0,0 Отметьте, что шаблон во второй строке каждой пары действительно используется для отфильтровки файлов. Также отметьте, что вам нужно добавить дополнительный 0 в конце фильтровых строк, чтобы указать конец. Определите, какая пара фильтровых строк будет использоваться при первом отображении файлового диалогового окна. Индекс основывается на единице, то есть первая пара - 1, вторая - 2 и так далее. Поэтому в вышеприведенном экземпляре, если мы укажем nFilterIndex как 2, будет использован второй шаблон - "*.txt".
lpstrFile	Указатель на буфер, который содержит имя файла, используемого для инициализации edit control'a имени файла на диалоговом окне. Буфер должен быть длиной по крайней мере 260 байтов. После того, как юзер выберет файл для открытия, имя файла с полным путем будет сохранено в этом буфере. Вы можете извлечь информацию из него позже.
nMaxFile	Размер буфера.
lpstrTitle	Указатель на заголовок открытого файлового диалогового окна.
Flags	Определите стили и характеристики диалогового окна.
nFileOffset	После того, как юзер выбрал файл для открытия, этот параметр содержит индекс первого символа собственно названия файла. Например, если полное имя с путем "c:\windows\system\lz32.dll", то этот параметр будет содержать значение 18.
nFileExtension	После того, как пользователь выберет файл для открытия, этот параметр содержит индекс первого символа расширения файла.

ПРИМЕР

Нижеприведенная программа отображает диалоговое окно открытия файла, когда пользователь выбирает пункт File->Open в меню. Когда пользователь выберет файл в диалоговом окне, программа отобразит сообщение, содержащее полное имя, собственно имя файла и расширение выбранного файла.

```

.model flat,stdcall
option casemap:none

WinMain proto :DWORD,:DWORD,:DWORD,:DWORD

include \masm32\include\windows.inc
include \masm32\include\user32.inc
include \masm32\include\kernel32.inc
include \masm32\include\comdlg32.inc

includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib
includelib \masm32\lib\comdlg32.lib

.const

IDM_OPEN equ 1
IDM_EXIT equ 2
MAXSIZE equ 260
OUTPUTSIZE equ 512

.data

ClassName db "SimpleWinClass",0
AppName db "Our Main Window",0
MenuName db "FirstMenu",0
ofn OPENFILENAME <>
FilterString db "All Files",0,"*.*",0
               db "Text Files",0,"*.txt",0,0
buffer db MAXSIZE dup(0)
OurTitle db "--Our First Open File Dialog Box--: Choose the file to open",0
FullPathName db "The Full Filename with Path is: ",0
FullName db "The Filename is: ",0
ExtensionName db "The Extension is: ",0
OutputString db OUTPUTSIZE dup(0)
CrLf db 0Dh,0Ah,0

.data?

hInstance HINSTANCE ?
CommandLine LPSTR ?

.code

start:

    invoke GetModuleHandle, NULL
    mov     hInstance,eax
    invoke GetCommandLine
    mov     CommandLine,eax
    invoke WinMain, hInstance,NULL,CommandLine, SW_SHOWDEFAULT
    invoke ExitProcess,eax

WinMain proc

hInst:HINSTANCE,hPrevInst:HINSTANCE,CmdLine:LPSTR,CmdShow:DWORD
LOCAL wc:WNDCLASSEX

LOCAL msg:MSG
LOCAL hwnd:HWND

    mov     wc.cbSize,SIZEOF WNDCLASSEX
    mov     wc.style, CS_HREDRAW or CS_VREDRAW
    mov     wc.lpfnWndProc, OFFSET WndProc
    mov     wc.cbClsExtra,NULL
    mov     wc.cbWndExtra,NULL
    push     hInst
    pop     wc.hInstance
    mov     wc.hbrBackground,COLOR_WINDOW+1
    mov     wc.lpszMenuName,OFFSET MenuName
    mov     wc.lpszClassName,OFFSET ClassName

```

```

invoke LoadIcon,NULL,IDI_APPLICATION
mov wc.hIcon,eax
mov wc.hIconSm,eax
invoke LoadCursor,NULL,IDC_ARROW
mov wc.hCursor,eax

invoke RegisterClassEx, addr wc
invoke CreateWindowEx,WS_EX_CLIENTEDGE,ADDR ClassName,ADDR AppName,\
WS_OVERLAPPEDWINDOW,CW_USEDEFAULT,\
CW_USEDEFAULT,300,200,NULL,NULL,\
hInst,NULL

mov hwnd,eax
invoke ShowWindow, hwnd,SW_SHOWNORMAL
invoke UpdateWindow, hwnd

.WHILE TRUE
    invoke GetMessage, ADDR msg,NULL,0,0
    .BREAK .IF (!eax)
    invoke TranslateMessage, ADDR msg
    invoke DispatchMessage, ADDR msg
.ENDW

mov eax,msg.wParam

ret

WinMain endp

WndProc proc hwnd:HWND, uMsg:UINT, wParam:WPARAM, lParam:LPARAM

    .IF uMsg==WM_DESTROY
        invoke PostQuitMessage,NULL
    .ELSEIF uMsg==WM_COMMAND
        mov eax,wParam
        .if ax==IDM_OPEN
            mov ofn.lStructSize,SIZEOF ofn
            push hwnd
            pop ofn.hwndOwner
            push hInstance
            pop ofn.hInstance
            mov ofn.lpstrFilter, OFFSET FilterString
            mov ofn.lpstrFile, OFFSET buffer
            mov ofn.nMaxFile,MAXSIZE
            mov ofn.Flags, OFN_FILEMUSTEXIST or \
                OFN_PATHMUSTEXIST or OFN_LONGNAMES or\
                OFN_EXPLORER or OFN_HIDEREADONLY
            mov ofn.lpstrTitle, OFFSET OurTitle
            invoke GetOpenFileName, ADDR ofn
            .if eax==TRUE
                invoke lstrcat,offset OutputString,OFFSET FullPathName
                invoke lstrcat,offset OutputString,ofn.lpstrFile
                invoke lstrcat,offset OutputString,offset CrLf
                invoke lstrcat,offset OutputString,offset FullName
                mov eax,ofn.lpstrFile
                push ebx
                xor ebx,ebx
                mov bx,ofn.nFileOffset
                add eax,ebx
                pop ebx
                invoke lstrcat,offset OutputString,eax
                invoke lstrcat,offset OutputString,offset CrLf
                invoke lstrcat,offset OutputString,offset ExtensionName
                mov eax,ofn.lpstrFile
                push ebx
                xor ebx,ebx
                mov bx,ofn.nFileExtension
                add eax,ebx
                pop ebx
                invoke lstrcat,offset OutputString,eax
                invoke MessageBox,hwnd,OFFSET OutputString,ADDR AppName,MB_OK
            .endif
        .endif
    .endif

```

```

        invoke RtlZeroMemory,offset OutputString,OUTPUTSIZE
    .endif
    .else
        invoke DestroyWindow, hWnd
    .endif
    .ELSE
        invoke DefWindowProc,hWnd,uMsg,wParam,lParam

        ret

    .ENDIF

    xor     eax,eax

    ret

WndProc endp

end start

```

АНАЛИЗ

```

mov ofn.lStructSize,SIZEOF ofn
push hWnd
pop  ofn.hwndOwner
push hInstance
pop  ofn.hInstance

```

Мы заполняем в процедуре члены структуры ofn.

```
mov ofn.lpstrFilter, OFFSET FilterString
```

FilterString - это фильтр имен файлов, который мы определяем следующим образом.

```

FilterString db "All Files",0,"*.*",0
              db "Text Files",0,"*.txt",0,0

```

Заметьте, что все четыре строки заканчиваются нулем. Первая строка - это описание следующей строки. Первая строка является описанием первой. В качестве фильтра мы можем определить все, что захочем. Мы **должны** добавить дополнительный ноль после последнего фильтра, чтобы указать конец. Не забудьте сделать это, иначе ваше диалоговое окно поведет себя весьма странно.

```

mov ofn.lpstrFile, OFFSET buffer
mov ofn.nMaxFile,MAXSIZE

```

Мы указываем, где диалоговое окно поместить имена файлов, выбранные пользователем. Учтите, что мы должны указать размер буфера в nMaxFile. Мы можем затем извлечь имя файла из этого буфера.

```

mov ofn.Flags, OFN_FILEMUSTEXIST or \
               OFN_PATHMUSTEXIST or OFN_LONGNAMES or \
               OFN_EXPLORER or OFN_HIDEREADONLY

```

Флаги определяю характеристики окна.

- OFN_FILEMUSTEXIST и OFN_PATHMUSTEXIST указывают то, что имя файла и путь, который пользователь набирает в edit control'e имени файла, **должен** существовать.
- OFN_LONGNAMES указывает диалоговому окну показывать длинные имена.
- OFN_EXPLORER указывает на то, что появление диалогового окна должно быть похоже на explorer.
- OFN_HIDEREADONLY прячет неизменяемый checkbox на диалоговом окне. Есть много других флагов, которые вы можете использовать. Проконсультируйтесь с вашим справочником по Win32 API.

```
mov ofn.lpstrTitle, OFFSET OurTitle
```

Указываем имя диалогового окна.


```
invoke GetOpenFileName, ADDR ofn
```

Вызов функции GetOpenFileName. Передача указателя на структуру ofn в качестве параметров.

В тоже время, диалоговое окно открытия файла отображается на экране. Функция не будет возвращаться, пока пользователь не выберет файл или не нажмет кнопку 'Cancel' или закроет диалоговое окно.

Функция возвратит TRUE, если пользователь выбрал файл, в противном случае FALSE.

```
.if eax==TRUE
    invoke lstrcat,offset OutputString,OFFSET FullPathName
    invoke lstrcat,offset OutputString,ofn.lpstrFile
    invoke lstrcat,offset OutputString,offset CrLf
    invoke lstrcat,offset OutputString,offset FullName
```

В случае, если пользователь выбирает файл, мы подготавливаем строку вывода, которая будет отображаться в окне сообщения. Мы резервируем блок памяти в переменной OutputString и затем используем API-функцию, lstrcat, чтобы соединить обе строки. Чтобы разместить строку в несколько рядов, мы должны использовать символы переноса каретки.

```
mov  eax,ofn.lpstrFile
push ebx
xor  ebx,ebx
mov  bx,ofn.nFileOffset
add  eax,ebx
pop  ebx
invoke lstrcat,offset OutputString,eax
```

Вышеприведенные строки требуют некоторых объяснений. nFileOffset содержит индекс в ofn.lpstrFile. Но вы не можете сложить их в месте, так размерности этих переменных разные. Поэтому я поместил значение nFileOffset в нижнее слово ebx'a и сложил его со значением lpstrFile'a.

```
invoke MessageBox,hWnd,OFFSET OutputString,ADDR AppName,MB_OK
```

Мы отображаем строку в окне сообщения.

```
invoke RtlZeroMemory,offset OutputString,OUTPUTSIZE
```

Мы **должны** очистить OutputString перед тем, как заполнить его другой строкой. Поэтому мы используем функцию RtlZeroMemory для этого.

Win32 API. Урок 12. Память и файлы

Автор: lczelion, пер. Aquila

Дата: 2002-05-12

Мы выучим основы менеджмента памяти и файловых операций ввода/вывода в этом уроке. Также мы используем обычные диалоговые окна как устройства ввода/вывода.

Скачайте [пример](#) [здесь](#) (находится в архиве [wasm_files.zip: files/recovered/1001012/tut12.zip](#)).

ТЕОРИЯ

Менеджмент памяти под Win32 с точки зрения приложения достаточно прост и прямолинеен. Используемая модель памяти называется плоской моделью памяти. В этой модели все сегментные регистры (или селекторы) указывают на один и тот же стартовый адрес и смещение 32-битное, так что приложение может обратиться к любой точке памяти своего адресного пространства без необходимости изменять значения селекторов. Это очень упрощает управление памятью. Больше нет "дальних" и "ближних" указателей.

Под Win16 существует две основные категории функций API памяти: глобальные и локальные. Функции глобального типа взаимодействуют с памятью в других сегментах, поэтому они функции "дальней" памяти. Функции локального типа взаимодействуют с локальной кучей процессом, поэтому они функции "ближней" памяти.

Под Win32 оба этих типа идентичны. Используйте ли вы GlobalAlloc или LocalAlloc, вы получите одинаковый результат.

1. Выделите блок памяти с помощью вызова GlobalAlloc. Эта функция возвращает хэндл на запрошенный блок памяти.
2. "Закройте" блок памяти, вызвав GlobalLock. Эта функция принимает хэндл на блок памяти и возвращает указатель на блок памяти.
3. Вы можете использовать указатель, чтобы читать или писать в память.
4. "Откройте" блок памяти с помощью вызова GlobalUnlock. Эта функция возвращает указатель на блок памяти.
5. Освободите блок памяти с помощью GlobalFree. Эта функция принимает хэндл на блок памяти.

Вы также можете заменить "Global" на "Local", т.е. LocalAlloc, LocalLock и т.д.

Вышеуказанный метод может быть упрощен использованием флага GMEM_FIXED при вызове GlobalAlloc. Если вы используете этот флаг, возвращаемое значение от Global/LocalAlloc будет указателем на зарезервированный блок памяти, а не хэндл этого блока. Вам не надо будет вызывать Global/LocalLock вы сможете передать указатель Global/LocalFree без предварительного вызова Global/LocalUnlock. Но в этом tutorialе я использую "традиционный" подход, так как вы можете столкнуться с ним при изучении исходников других программ.

Файловый ввод/вывод по Win32 имеет значительное сходство с тем, как это делалось под DOS. Все требуемые шаги точно такие же. Вам только нужно изменить прерывания на вызовы API функций.

1. Откройте или создайте файл функцией CreateFile. Эта функция имеет очень многонаправленна: не считая файла, она может открывать компорты, пайпы, дисковые приводы и консоли. В случае успеха она возвращает хэндл файла или устройства. Затем вы можете использовать этот хэндл, чтобы выполнить определенные действия над файлом или устройством.
2. Переместите файловый указатель в желаемое местоположение функцией SetFilePointer.

3. Проведите операцию чтения или записи с помощью вызова ReadFile или WriteFile. Перед этим вы должны зарезервировать достаточно большой блок памяти для данных.

4. Закройте файл с помощью CloseHandle. Эта функции принимает хэндл файла.

Приведенная ниже програма отображает открытый файловое диалоговое окно. Оно позволяет пользователю использовать текстовый файл, чтобы открыть и показать содержимое файла в клиентской области edit control'a. Пользователь может изменять текст в edit control'e по своему усмотрению, а затем может сохранить содержимое в файл.

```
.386
.model flat,stdcall

option casemap:none
WinMain proto :DWORD,:DWORD,:DWORD,:DWORD
include \masm32\include\windows.inc
include \masm32\include\user32.inc

include \masm32\include\kernel32.inc
include \masm32\include\comdlg32.inc
includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib

includelib \masm32\lib\comdlg32.lib

.const

IDM_OPEN equ 1
IDM_SAVE equ 2
IDM_EXIT equ 3
MAXSIZE equ 260

MEMSIZE equ 65535

EditID equ 1 ; ID of the edit control

.data
ClassName db "Win32ASMEEditClass",0

AppName db "Win32 ASM Edit",0
EditClass db "edit",0
MenuName db "FirstMenu",0
ofn OPENFILENAME <>

FilterString db "All Files",0,"*.*",0
db "Text Files",0,"*.txt",0,0
buffer db MAXSIZE dup(0)

.data?
hInstance HINSTANCE ?
CommandLine LPSTR ?

hwndEdit HWND ? ; Handle to the edit control

hFile HANDLE ? ; File handle
hMemory HANDLE ? ;handle to the allocated
memory block

pMemory DWORD ? ;pointer to the allocated
memory block
SizeReadWrite DWORD ? ; number of bytes actually read or
write

.code
start:
invoke GetModuleHandle, NULL
```

```

    mov     hInstance,eax
    invoke  GetCommandLine
    mov     CommandLine,eax

    invoke  WinMain, hInstance,NULL,CommandLine, SW_SHOWDEFAULT
    invoke  ExitProcess,eax

WinMain proc
hInst:HINSTANCE,hPrevInst:HINSTANCE,CmdLine:LPSTR,CmdShow:SDWORD
    LOCAL  wc:WNDCLASSEX
    LOCAL  msg:MSG

    LOCAL  hwnd:HWND
    mov     wc.cbSize,SIZEOF WNDCLASSEX
    mov     wc.style, CS_HREDRAW or CS_VREDRAW
    mov     wc.lpfnWndProc, OFFSET WndProc

    mov     wc.cbClsExtra,NULL
    mov     wc.cbWndExtra,NULL
    push    hInst
    pop     wc.hInstance

    mov     wc.hbrBackground,COLOR_WINDOW+1
    mov     wc.lpszMenuName,OFFSET MenuName
    mov     wc.lpszClassName,OFFSET ClassName
    invoke  LoadIcon,NULL,IDI_APPLICATION

    mov     wc.hIcon,eax
    mov     wc.hIconSm,eax
    invoke  LoadCursor,NULL,IDC_ARROW
    mov     wc.hCursor,eax

    invoke  RegisterClassEx, addr wc
    invoke  CreateWindowEx,WS_EX_CLIENTEDGE,ADDR ClassName,ADDR AppName,\
        WS_OVERLAPPEDWINDOW,CW_USEDEFAULT,\
        CW_USEDEFAULT,300,200,NULL,NULL,\
        hInst,NULL
    mov     hwnd,eax
    invoke  ShowWindow, hwnd,SW_SHOWNORMAL
    invoke  UpdateWindow, hwnd

    .WHILE TRUE
        invoke  GetMessage, ADDR msg,NULL,0,0
        .BREAK .IF (!eax)
        invoke  TranslateMessage, ADDR msg

        invoke  DispatchMessage, ADDR msg
    .ENDW
    mov     eax,msg.wParam
    ret

WinMain endp

WndProc proc uses ebx hwnd:HWND, uMsg:UINT, wParam:WPARAM, lParam:LPARAM

    .IF uMsg==WM_CREATE
        invoke  CreateWindowEx,NULL,ADDR EditClass,NULL,\
            WS_VISIBLE or WS_CHILD or ES_LEFT or ES_MULTILINE or\
            ES_AUTOHSCROLL or ES_AUTOVSCROLL,0,\
            0,0,0,hwnd,EditID,\
            hInstance,NULL
        mov     hwndEdit,eax
        invoke  SetFocus,hwndEdit

;=====
;      Initialize the members of OPENFILENAME structure
;=====
        mov     ofn.lStructSize,SIZEOF ofn

```

```

    push hWnd
    pop ofn.hwndOwner
    push hInstance
    pop ofn.hInstance

    mov ofn.lpstrFilter, OFFSET FilterString
    mov ofn.lpstrFile, OFFSET buffer
    mov ofn.nMaxFile, MAXSIZE
.ELSEIF uMsg==WM_SIZE

    mov eax, lParam
    mov edx, eax
    shr edx, 16
    and eax, 0ffffh

    invoke MoveWindow, hWndEdit, 0, 0, eax, edx, TRUE
.ELSEIF uMsg==WM_DESTROY
    invoke PostQuitMessage, NULL
.ELSEIF uMsg==WM_COMMAND

    mov eax, wParam
    .if lParam==0
        .if ax==IDM_OPEN
            mov ofn.Flags, OFN_FILEMUSTEXIST or \
                OFN_PATHMUSTEXIST or OFN_LONGNAMES or \
                OFN_EXPLORER or OFN_HIDEREADONLY
            invoke GetOpenFileName, ADDR ofn
            .if eax==TRUE

                invoke CreateFile, ADDR buffer, \
                    GENERIC_READ or GENERIC_WRITE, \
                    FILE_SHARE_READ or FILE_SHARE_WRITE, \
                    NULL, OPEN_EXISTING, FILE_ATTRIBUTE_ARCHIVE, \
                    NULL
                mov hFile, eax
                invoke GlobalAlloc, GMEM_MOVEABLE or
                    GMEM_ZEROINIT, MEMSIZE
                mov hMemory, eax
                invoke GlobalLock, hMemory
                mov pMemory, eax

                invoke ReadFile, hFile, pMemory, MEMSIZE-1, ADDR
                    SizeReadWrite, NULL
                invoke SendMessage, hWndEdit, WM_SETTEXT, NULL, pMemory
                invoke CloseHandle, hFile

                invoke GlobalUnlock, pMemory
                invoke GlobalFree, hMemory
            .endif
            invoke SetFocus, hWndEdit

        .elseif ax==IDM_SAVE
            mov ofn.Flags, OFN_LONGNAMES or \
                OFN_EXPLORER or OFN_HIDEREADONLY
            invoke GetSaveFileName, ADDR ofn

            .if eax==TRUE
                invoke CreateFile, ADDR buffer, \
                    GENERIC_READ or GENERIC_WRITE, \
                    FILE_SHARE_READ or FILE_SHARE_WRITE, \
                    NULL, CREATE_NEW, FILE_ATTRIBUTE_ARCHIVE, \
                    NULL
                mov hFile, eax
                invoke GlobalAlloc, GMEM_MOVEABLE or GMEM_ZEROINIT, MEMSIZE

```

```

        mov  hMemory,eax
        invoke GlobalLock,hMemory
        mov  pMemory,eax
        invoke SendMessage,hwndEdit,WM_GETTEXT,MEMSIZE-1,pMemory
        invoke WriteFile,hFile,pMemory,eax,ADDR SizeReadWrite,NULL
        invoke CloseHandle,hFile

        invoke GlobalUnlock,pMemory
        invoke GlobalFree,hMemory
    .endif
    invoke SetFocus,hwndEdit

    .else
        invoke DestroyWindow, hwnd
    .endif
    .endif
    .endif

    .ELSE
        invoke DefWindowProc,hWnd,uMsg,wParam,lParam
        ret
    .ENDIF

xor    eax,eax
ret
WndProc endp
end start

```

Анализ:

```

        invoke CreateWindowEx,NULL,ADDR EditClass,NULL,\
\
        WS_VISIBLE or WS_CHILD or ES_LEFT or ES_MULTILINE or\
        ES_AUTOHSCROLL or ES_AUTOVSCROLL,0,\
        0,0,0,hwnd,EditID,\
        hInstance,NULL

        mov  hwndEdit,eax

```

В секции WM_CREATE мы создаем edit control. Отметим, что параметры, которые определяют x, y, width, height контрола равны нулю, поскольку мы изменим размер контрола позже, что покрыть всю клиентскую область родительского окна.

Заметьте, что в этом случае мы не должны вызывать ShowWindow, чтобы заставить появиться контрол на экране, так как мы указали стиль WS_VISIBLE. Вы можете использовать этот трюк и для родительского окна.

```

;=====
;      Инициализируем структуру
;=====
        mov  ofn.lStructSize,SIZEOF ofn

        push hwnd
        pop  ofn.hwndOwner
        push hInstance
        pop  ofn.hInstance

        mov  ofn.lpstrFilter, OFFSET FilterString
        mov  ofn.lpstrFile, OFFSET buffer
        mov  ofn.nMaxFile,MAXSIZE

```

После создания edit control'a edit control'a, мы используем это время, чтобы проинициализировать члены ofn. Так как мы хотим использовать ofn повторно в диалоговом окне, мы заполняем только **общие** члены, которые используются и GetOpenFileName и GetSaveFileName. Секция WM_CREATE - это прекрасное место для одноразовой инициализации.

```

    .ELSEIF uMsg==WM_SIZE
        mov  eax,lParam
        mov  edx,eax

```

```
shr edx,16
and eax,0ffffh
invoke MoveWindow,hwndEdit,0,0,eax,edx,TRUE
```

Мы получаем сообщения WM_SIZE, когда размер клиентской области нашего основного окна изменяется. Мы также получаем его, когда окно создается. Для того, чтобы получать это сообщение, стили класса окна должны включать CS_REDRAW и CS_HREDRAW. Мы используем эту возможность для того, чтобы сделать размер нашего edit control'a равным клиентской области окна. Для начала мы должны узнать текущую ширину и высоту клиентской области родительского окна. Мы получаем эту информацию из IParam. Верхнее слово IParam содержит высоту, а нижнее слово - ширину клиентской области. Затем мы используем эту информацию для того, чтобы изменить размер edit control'a с помощью вызова функции MoveWindow, которая может изменять позицию и размер окна на экране.

```
.if ax==IDM_OPEN
mov ofn.Flags, OFN_FILEMUSTEXIST or \
               OFN_PATHMUSTEXIST or OFN_LONGNAMES or\
               OFN_EXPLORER or OFN_HIDEREADONLY

invoke GetOpenFileName, ADDR ofn
```

Когда пользователь выбирает пункт меню File/Open, мы заполняем в структуре параметр Flags и вызываем функцию GetOpenFileName, чтобы отобразить окно открытия файла.

```
.if eax==TRUE
invoke CreateFile,ADDR buffer,\
               GENERIC_READ or GENERIC_WRITE ,\
               FILE_SHARE_READ or FILE_SHARE_WRITE,\
               NULL,OPEN_EXISTING,FILE_ATTRIBUTE_ARCHIVE,\
               NULL
mov hFile,eax
```

После того, как пользователь выберет файл для открытия, мы вызываем CreateFile, чтобы открыть файл. Мы указываем, что функция должна попробовать открыть файл для чтения и записи. После того, как файл открыт, функция возвращает хэндл на открытый файл, который мы сохраняем в глобальной переменной для будущего использования. Эта функция имеет следующий синтаксис:

```
CreateFile proto lpFileName:DWORD,\
               dwDesiredAccess:DWORD,\
               dwShareMode:DWORD,\
               lpSecurityAttributes:DWORD,\
               dwCreationDistribution:DWORD,\
               dwFlagsAndAttributes:DWORD,\
               hTemplateFile:DWORD
```

- dwDesireAccess указывает, какую операцию вы хотите выполнить над файлом.
- Открыть файл для проверки его атрибутов. Вы можете писать и читать из файла.
- GENERIC_READ Открыть файл для чтения.
- GENERIC_WRITE Открыть файл для записи.
- dwShareMode указывает, какие операции вы хотите позволить выполнять вашим процессам над открытыми файлами.
- 0 Не разделять файл с другими процессами.
- FILE_SHARE_READ позволяет другим процессам прочесть информацию из файла, который был открыт
- FILE_SHARE_WRITE позволяет другим процессам записывать информацию в открытый файл.
- lpSecurityAttributes не имеет значения под Windows 95.
- dwCreationDistribution указывает действие, которое будет выполнено над файлом при его открытии.
- CREATE_NEW Создание нового файла, если файла не существует.
- CREATE_ALWAYS Создание нового файла. Функция перезаписывает файл, если он существует.
- OPEN_EXISTING Открытие существующего файла.

- OPEN_ALWAYS Открытие файла, если он существует, в противном случае, функция создает новый файл.
- TRUNCATE_EXISTING Открытие файла и обрезание его до нуля байтов. Вызывающий функцию процесс должен открывать файл по крайней мере с доступом GENERIC_WRITE. Если файл не существует, функция не срабатывает.
- dwFlagsAndAttributes указывает атрибуты файла
- FILE_ATTRIBUTE_ARCHIVE Файл является архивным файлом. Приложения используют этот атрибут для бэкапа или удаления.
- FILE_ATTRIBUTE_COMPRESSED Файл или директория сжаты. Для файла это означает, что вся информация в файле заархивирована. Для директории это означает, что сжатие подразумевается по умолчанию для создаваемых вновь файлов и поддиректорий.
- FILE_ATTRIBUTE_NORMAL У файла нет других атрибутов. Этот атрибут действителен, только если используется один.
- FILE_ATTRIBUTE_HIDDEN Файл скрыт. Он не включается в обычные листинги директорий.
- FILE_ATTRIBUTE_READONLY Файл только для чтения. Приложения могут читать из файла, но не могут писать в него или удалить его.
- FILE_ATTRIBUTE_SYSTEM Файл - часть операционной системы или используется только ей.

```

invoke GlobalAlloc,GMEM_MOVEABLE or GMEM_ZEROINIT,MEMSIZE
mov hMemory,eax

invoke GlobalLock,hMemory
mov pMemory,eax

```

Когда файл открыт, мы резервируем блок памяти для использования функциями ReadFile и WriteFile. Мы указываем флаг GMEM_MOVEABLE, чтобы позволить Windows перемещать блок памяти, чтобы уплотнить последнюю.

Когда GlobalAlloc возвращает положительный результат, eax содержит хэндл зарезервированного блока памяти. Мы передаем этот хэндл функции GlobalLock, который возвращает указатель на блок памяти.

```

invoke ReadFile,hFile,pMemory,MEMSIZE-1,ADDR SizeReadWrite,NULL
invoke SendMessage,hwndEdit,WM_SETTEXT,NULL,pMemory

```

Когда блок памяти готов к использованию, мы вызываем функцию ReadFile для чтения данных из файла. Когда файл только что открыт или создан, указатель на смещение равен нулю. В этом случае, мы начинаем чтение с первого байта. Первый параметр ReadFile - это хэндл файла, из которого необходимо произвести чтение, второй - это указатель на блок памяти, затем - количество байтов, которое нужно считать из файла, четвертый параметр - это адрес переменной размера DWORD, который будет заполнен количеством байтов, в реальности считанных из файла.

После заполнения блока памяти данными, мы помещаем данные в edit control, посылая сообщение WM_SETTEXT контролю, причем lParam содержит указатель на блок памяти. После этого вызова edit control отображает данные в его клиентской области.

```

invoke CloseHandle,hFile
invoke GlobalUnlock,pMemory
invoke GlobalFree,hMemory
.endif

```

В этой месте у нас нет необходимости держать файл открытым, так как нашей целью является запись модифицированных данных из edit control'a в другой файл, а не в оригинальный. Поэтому мы закрываем файл функцией CloseHandle, передав ей в качестве параметра хэндл файла. Затем мы открываем блок памяти и освобождаем его. В действительности, вам не нужно освобождать ее сейчас, вы можете использовать этот же блок во время операции сохранения. Но в демонстрационных целях я освобождаю ее сейчас.

```

invoke SetFocus,hwndEdit

```


Когда на экране отображается окно открытия файла, фокус ввода сдвигается на него. Поэтому, когда это окно закрывается, мы должны передвинуть фокус ввода обратно на edit control.

Это заканчивает операцию чтения из файла. В этом месте пользователь должен отредактировать содержимое edit control'a. И когда он хочет сохранить данные в другой файл, он должен выбрать File/Save, после чего отобразиться диалоговое окно. Создание окна сохранения файла не слишком отличается от создание окна открытия файла. Фактически, они отличаются только именем функций. Вы можете снова использовать большинство из параметров структуры ofn, кроме параметра Flags.

```
mov ofn.Flags,OFN_LONGNAMES or\  
OFN_EXPLORER or OFN_HIDEREADONLY
```

В нашем случае, мы хотим создать новый файл, так чтобы OFN_FILEMUSTEXIST и OFN_PATHMUSTEXIST должны быть убраны, иначе диалоговое окно не позволит нам создать файл, который уже не существует.

Параметр dwCreationDistribution функции CreateFile должен быть установлен в CREATE_NEW, так как мы хотим создать новый файл.

Оставшийся код практически одинаков с тем, что используется при создании окна открытия файла, за исключением следующего:

```
invoke SendMessage,hwndEdit,WM_GETTEXT,MEMSIZE-1,pMemory  
invoke WriteFile,hFile,pMemory,eax,ADDR SizeReadWrite,NULL
```

Мы посылаем сообщение WM_GETTEXT edit control'у, чтобы скопировать данные из него в блок памяти, возвращаемое значение в eax - это длина данных внутри буфера. После того, как данные оказываются в блоке памяти, мы записываем их в новый файл.

Win32 API. Урок 13. Memory Mapped файлы

Автор: lczelion, пер. Aquila

Дата: Не указана

Я покажу вам, что такое MMF и как использовать их для вашей выгоды. Использование MMF достаточно просто, как вы увидите из этого туториала.

Скачайте пример здесь (находится в архиве wasm_files.zip: files/recovered/1001013/tut13.zip).

ТЕОРИЯ

Если вы хорошо изучили пример из прошлого туториала, вы увидите, что у него есть серьезный недостаток: что, если файл, который вы хотите прочитать больше, чем зарезервированный блок памяти? или если строка, которую вы хотите найти будет обрезана посередине, потому что кончился блок памяти? Традиционный ответ на первый вопрос - это то, что вам нужно последовательно читать данные из файла, пока он не кончится. Ответом на второй вопрос является то, что вы должны обрабатывать подобную возможность. Это называется проблемой пограничного значения. Она представляет собой головную боль для программистов и вызывает неисчислимо много багов.

Было бы неплохо, если бы мы могли зарезервировать очень большой блок памяти, достаточный для того, чтобы сохранить весь файл, но наша программа стала бы очень прожорливой в плане ресурсов. File mapping - это спасение. Используя его, вы можете считать весь файл уже загруженным в память и использовать указатель на память, чтобы читать или писать данные в файл. Очень просто. Нет нужды использовать API памяти и файловые API одновременно, в FM это одно и то же. FM также используется для обмена данными между процессами. При использовании FM таким образом, реально не используется никакой файл. Это больше похоже на блок памяти, который могут видеть все процессы. Но обмен данными между процессами - весьма деликатный предмет. Вы должны будете обеспечить синхронизацию между процессами и ветвями, иначе ваше приложение очень скоро повиснет.

Мы не будем касаться того, как использовать FM для создания общего региона памяти в этом туториале. Мы сконцентрируемся на том, как использовать FM для "загрузки" файла в память. Фактически, PE-загрузчик использует FM для загрузки исполняемых файлов в память. Это очень удобно, так как только необходимые порции файла будут считываться с диска. Под Win32 вам следует использовать FM так часто, как это возможно.

Правда, существует несколько ограничений при использовании FM. Как только вы создали такой файл, его размер не может изменяться до закрытия сессии. Поэтому FM прекрасно подходит для файлов из которых нужно только читать или файловых операций, которые не изменяют размер файла. Это не значит, что вы не можете использовать FM, если хотите увеличить размер файла. Вы можете установить новый размер и создать MMF нового размера и файл увеличится до этого размера. Это просто неудобно, вот и все.

Достаточно объяснений. Давайте перейдем к реализации FM. Для того, чтобы его использовать, должны быть выполнены следующие шаги.

- Вызов `CreateFile` для открытия файла.
- Вызов `CreateFileMapping`, которой передается хэндл файла, возвращенный `CreateFile`. Эта функция создает FM-объект из файла, созданного `CreateFile`ом.
- Вызов `MapViewOfFile`, чтобы загрузить выбранный файловый регион или весь файл в память. Эта функция возвращает указатель на первый байт промэппированного файлового региона.
- Используйте указатель, чтобы писать или читать из файла.

- Вызовите UnmapViewOfFile, чтобы выгрузить файл.
- Вызов CloseHandle, передав ему хэндл промэппированного файла в качестве одного из параметра, чтобы закрыть его.
- Вызов CloseHandle снова, передав ему в этот раз хэндл файла, возвращенный CreateFile, чтобы закрыть сам файл.

ПРИМЕР

Программа, листинг которой приведен ниже, позволит вам открыть файл с помощью окна открытия файла. Она откроет файл, используя FM, если это удастся, заголовок окна изменится на имя открытого файла. Вы можете сохранить файл под другим именем, выбрав пункт меню File/Save. Программа скопирует все содержимое открытого файла в новый файл. Учтите, что вы не должны вызывать GlobalAlloc для резервирования блока памяти в этой программе.

```
.386
.model flat,stdcall

WinMain proto :DWORD,:DWORD,:DWORD,:DWORD

include \masm32\include\windows.inc
include \masm32\include\user32.inc
include \masm32\include\kernel32.inc
include \masm32\include\comdlg32.inc

includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib
includelib \masm32\lib\comdlg32.lib

.const
IDM_OPEN equ 1
IDM_SAVE equ 2

IDM_EXIT equ 3
MAXSIZE equ 260

.data
ClassName db "Win32ASMFileMappingClass",0
AppName db "Win32 ASM File Mapping Example",0
MenuName db "FirstMenu",0

ofn OPENFILENAME <>
FilterString db "All Files",0,"*.*",0
            db "Text Files",0,"*.txt",0,0
buffer db MAXSIZE dup(0)

hMapFile HANDLE 0                                ; Указатель на MMF, должен быть инициализирован
                                                ; нулем, так как мы также используем его
                                                ; в качестве флага в секции WM_DESTROY

.data?
hInstance HINSTANCE ?
CommandLine LPSTR ?
hFileRead HANDLE ?                               ; Хэндл источника
hFileWrite HANDLE ?                              ; Хэндл выходного файла
hMenu HANDLE ?
pMemory DWORD ?                                  ; Указатель на данные в исходном файле
SizeWritten DWORD ?                             ; количество байтов
actually written by WriteFile

.code
start:

        invoke GetModuleHandle, NULL
        mov     hInstance,eax
```

```

        invoke GetCommandLine
        mov  CommandLine,eax

        invoke WinMain, hInstance,NULL,CommandLine, SW_SHOWDEFAULT
        invoke ExitProcess,eax

WinMain proc
hInst:HINSTANCE,hPrevInst:HINSTANCE,CmdLine:LPSTR,CmdShow:DWORD
    LOCAL wc:WNDCLASSEX
    LOCAL msg:MSG

    LOCAL hwnd:HWND
    mov  wc.cbSize,SIZEOF WNDCLASSEX
    mov  wc.style, CS_HREDRAW or CS_VREDRAW
    mov  wc.lpfnWndProc, OFFSET WndProc

    mov  wc.cbClsExtra,NULL
    mov  wc.cbWndExtra,NULL
    push hInst
    pop  wc.hInstance

    mov  wc.hbrBackground,COLOR_WINDOW+1
    mov  wc.lpszMenuName,OFFSET MenuName
    mov  wc.lpszClassName,OFFSET ClassName
    invoke LoadIcon,NULL,IDI_APPLICATION

    mov  wc.hIcon,eax
    mov  wc.hIconSm,eax
    invoke LoadCursor,NULL,IDC_ARROW
    mov  wc.hCursor,eax

    invoke RegisterClassEx, addr wc
    invoke CreateWindowEx,WS_EX_CLIENTEDGE,ADDR ClassName,\
        ADDR AppName, WS_OVERLAPPEDWINDOW,CW_USEDEFAULT,\
        CW_USEDEFAULT,300,200,NULL,NULL,\

    hInst,NULL
    mov  hwnd,eax
    invoke ShowWindow, hwnd,SW_SHOWNORMAL
    invoke UpdateWindow, hwnd

    .WHILE TRUE
        invoke GetMessage, ADDR msg,NULL,0,0
        .BREAK .IF (!eax)
        invoke TranslateMessage, ADDR msg

        invoke DispatchMessage, ADDR msg
    .ENDW
    mov  eax,msg.wParam
    ret

WinMain endp

WndProc proc hwnd:HWND, uMsg:UINT, wParam:WPARAM, lParam:LPARAM

    .IF uMsg==WM_CREATE
        invoke GetMenu,hwnd                ; Получаем хэндл меню
        mov  hMenu,eax
        mov  ofn.lStructSize,SIZEOF ofn

        push hwnd
        pop  ofn.hWndOwner
        push hInstance
        pop  ofn.hInstance

        mov  ofn.lpstrFilter, OFFSET FilterString
        mov  ofn.lpstrFile, OFFSET buffer
        mov  ofn.nMaxFile,MAXSIZE
    .ELSEIF uMsg==WM_DESTROY
        .if hMapFile!=0

```

```

        call CloseMapFile
    .endif
    invoke PostQuitMessage,NULL

.ELSEIF uMsg==WM_COMMAND
    mov eax,wParam
    .if lParam==0
        .if ax==IDM_OPEN

            mov ofn.Flags, OFN_FILEMUSTEXIST or \
                OFN_PATHMUSTEXIST or OFN_LONGNAMES or\
                OFN_EXPLORER or OFN_HIDEREADONLY
            invoke GetOpenFileName, ADDR ofn

            .if eax==TRUE
                invoke CreateFile,ADDR buffer,\
                    GENERIC_READ ,\
                    0,\
                    NULL,OPEN_EXISTING,FILE_ATTRIBUTE_ARCHIVE,\
                        NULL
                mov hFileRead,eax

                invoke
CreateFileMapping,hFileRead,NULL,PAGE_READONLY,0,0,NULL
                mov hMapFile,eax
                mov eax,OFFSET buffer

                movzx edx,ofn.nFileOffset
                add eax,edx
                invoke SetWindowText,hWnd,eax
                invoke EnableMenuItem,hMenu,IDM_OPEN,MF_GRAYED

                invoke EnableMenuItem,hMenu,IDM_SAVE,MF_ENABLED
            .endif
        .elseif ax==IDM_SAVE
            mov ofn.Flags,OFN_LONGNAMES or\
                OFN_EXPLORER or OFN_HIDEREADONLY
            invoke GetSaveFileName, ADDR ofn
            .if eax==TRUE
                invoke CreateFile,ADDR buffer,\
                    GENERIC_READ or GENERIC_WRITE ,\
                    FILE_SHARE_READ or FILE_SHARE_WRITE,\
                    NULL,CREATE_NEW,FILE_ATTRIBUTE_ARCHIVE,\
                        NULL
                mov hFileWrite,eax

                invoke MapViewOfFile,hMapFile,FILE_MAP_READ,0,0,0
                mov pMemory,eax
                invoke GetFileSize,hFileRead,NULL
                invoke WriteFile,hFileWrite,pMemory,eax,ADDR SizeWritten,NULL
                invoke UnmapViewOfFile,pMemory
                call CloseMapFile
                invoke CloseHandle,hFileWrite

                invoke SetWindowText,hWnd,ADDR AppName
                invoke EnableMenuItem,hMenu,IDM_OPEN,MF_ENABLED
                invoke EnableMenuItem,hMenu,IDM_SAVE,MF_GRAYED
            .endif
        .else
            invoke DestroyWindow, hWnd
        .endif
    .endif

.ELSE

```

```

        invoke DefWindowProc,hWnd,uMsg,wParam,lParam
        ret
    .ENDIF

    xor     eax,eax
    ret
WndProc endp

```

```

CloseMapFile PROC
    invoke CloseHandle,hMapFile
    mov     hMapFile,0

    invoke CloseHandle,hFileRead
    ret
CloseMapFile endp

```

end start

Анализ:

```

        invoke CreateFile,ADDR buffer,\
                                GENERIC_READ ,\
                                0,\
                                NULL,OPEN_EXISTING,FILE_ATTRIBUTE_ARCHIVE,\
                                NULL

```

Когда пользователь выбирает файл в окне открытия файла, мы вызываем CreateFile, чтобы открыть его. Заметьте, что мы указываем GENERIC_READ, чтобы открыть этот файл в режиме read-only, потому что мы не хотим, чтобы какие-либо другие процессы изменяли файл во время нашей работы с ним.

```

        invoke CreateFileMapping,hFileRead,NULL,PAGE_READONLY,0,0,NULL

```

Затем мы вызываем CreateFileMapping, чтобы создать MMF из открытого файла. CreateFileMapping имеет следующий синтаксис:

```

CreateFileMapping proto hFile:DWORD,\
                        lpFileMappingAttributes:DWORD,\
                        flProtect:DWORD,\
                        dwMaximumSizeHigh:DWORD,\
                        dwMaximumSizeLow:DWORD,\
                        lpName:DWORD

```

Вам следует знать, что CreateFileMapping не обязана мэппировать весь файл в память. Вы можете промэппировать только часть файла. Размер мэппируемого файла вы задаете параметрами dwMaximumSizeHigh и dwMaximumSizeLow. Если вы зададите размер больше, чем его действительный размер, файл будет увеличен до нового размера. Если вы хотите, чтобы MMF был такого же размера, как и исходный файл, сделайте оба параметра равными нулю.

Вы можете использовать NULL в lpFileMappingAttributes, чтобы Windows создали MMF со значениями безопасности по умолчанию.

flProtect определяет желаемую защиту для MMF. В нашем примере, мы используем PAGE_READONLY, чтобы разрешить только операции чтения над MMF. Заметьте, что этот атрибут не должен входить в противоречие с атрибутами, указанными в CreateFile, иначе CreateFileMapping возвратит ошибку.

lpName указывает на имя MMF. Если вы хотите разделять этот файл с другими процессами, вы должны присвоить ему имя. Но в нашем примере другие процессы не будут его использовать, поэтому мы игнорируем этот параметр.

```

        mov     eax,OFFSET buffer
        movzx   edx,ofn.nFileOffset
        add     eax,edx
        invoke SetWindowText,hWnd,eax

```

Если CreateFileMapping выполнена успешно, мы изменяем название окна на имя открытого файла. Имя файла с полным путем сохраняется в буфере, мы же хотим отобразить только собственно имя файла, поэтому мы должны добавить значение параметра nFileOffset структуры OPENFILENAME к адресу буфера.

```
invoke EnableMenuItem,hMenu,IDM_OPEN,MF_GRAYED
invoke EnableMenuItem,hMenu,IDM_SAVE,MF_ENABLED
```

В качестве предосторожности, мы не хотим чтобы пользователь мог открыть несколько файлов за раз, поэтому делаем пункт меню Open недоступным для выбора и делаем доступным пункт Save. EnableMenuItem используется для изменения атрибутов пункта меню. После этого, мы ждем, пока пользователь выберет File/Save или закроет программу.

```
.ELSEIF uMsg==WM_DESTROY
    .if hMapFile!=0
        call CloseMapFile
    .endif
    invoke PostQuitMessage,NULL
```

В выше приведенном коде, когда процедура окна получает сообщение WM_DESTROY, она сначала проверяет значение hMapFile - равно ли то нулю или нет. Если оно не равно нулю, она вызывает функцию CloseMapFile, которая содержит следующий код:

```
CloseMapFile PROC
    invoke CloseHandle,hMapFile
    mov     hMapFile,0
    invoke CloseHandle,hFileRead

    ret
CloseMapFile endp
```

CloseMapFile закрывает MMF и сам файл, так что наша программа не оставляет за собой следов при выходе из Windows. Если пользователь выберет сохранение информации в другой файл, программа покажет ему окно сохранения файла. После он сможет напечатать имя нового файла, который и будет создать функцией CreateFile.

```
invoke MapViewOfFile,hMapFile,FILE_MAP_READ,0,0,0
mov pMemory,eax
```

Сразу же после создания выходного файла, мы вызываем MapViewOfFile, чтобы промэппировать желаемую порцию MMF в память. Эта функция имеет следующий синтаксис:

```
MapViewOfFile proto hFileMappingObject:DWORD,\
                    dwDesiredAccess:DWORD,\
                    dwFileOffsetHigh:DWORD,\
                    dwFileOffsetLow:DWORD,\
                    dwNumberOfBytesToMap:DWORD
```

dwDesiredAccess specifies what operation we want to do to the file. In our example, we want to read the data only so we use FILE_MAP_READ. dwFileOffsetHigh and dwFileOffsetLow specify the starting file offset of the file portion that you want to map into memory. In our case, we want to read in the whole file so we start mapping from offset 0 onwards.

dwDesiredAccess определяет, какую операцию мы хотим совершить над файлом. В нашем примере мы хотим только прочитать данные, поэтому мы используем FILE_MAP_READ.

dwFileOffsetHigh и dwFileOffsetLow задают стартовый файловое смещение файловой порции, которую вы хотите загрузить в память. В нашем случае нам нужно мы хотим читать весь файл, поэтому начинаем мэппинг со смещение ноль.

dwNumberOfBytesToMap задает количество байтов, которое нужно промэппировать в память. Чтобы сделать это со всем файлом, передайте ноль MapViewOfFile.

После вызова MapViewOfFile, желаемое количество загружается в память. Вы получите указатель на блок памяти, который содержит данные из файла.

```
invoke GetFileSize,hFileRead,NULL
```

Теперь узнаем, какого размера наш файл. Размер файла возвращается в eax. Если файл больше, чем 4 GB, то верхнее двойное слово размера файла сохраняется в FileSizeHighWord. Так как мы не ожидаем встретить таких больших файлов, мы можем проигнорировать это.

```
invoke WriteFile,hFileWrite,pMemory,ebx,ADDR SizeWritten,NULL
```

Запишем данные в выходной файл.

```
invoke UnmapViewOfFile,pMemory
```

Когда мы заканчиваем со входным файлом, вызываем UnmapViewOfFile.

```
call CloseMapFile  
invoke CloseHandle,hFileWrite
```

И закрываем все файлы.

```
invoke SetWindowText,hWnd,ADDR AppName
```

Восстанавливаем оригинальное название окна.

```
invoke EnableMenuItem,hMenu,IDM_OPEN,MF_ENABLED  
invoke EnableMenuItem,hMenu,IDM_SAVE,MF_GRAYED
```

Разрешаем доступ к пункту меню Open и запрещаем к Save As.

Win32 API. Урок 14. Процесс

Автор: lczelion, пер. Aquila

Дата: 2002-05-14

Здесь мы изучим, что такое процесс и как его создать и прервать.

Скачайте пример здесь (находится в архиве wasm_files.zip: files/recovered/1001014/tut14.zip).

ВСТУПЛЕНИЕ

Что такое процесс? Я процитирую определение из справочника по Win32 API.

"Процесс - это выполняющееся приложение, которое состоит из личного виртуального адресного пространства, кода, данных и других ресурсов операционной системы, таких как файлы, пайпы и синхронизационные объекты, видимые для процесса."

Как вы можете видеть из вышеприведенного определения, у процесса есть несколько объектов: адресное пространство, выполняемый модуль (модули) и все, что эти модули создают или открывают. Как минимум, процесс должен состоять из выполняющегося модуля, личного адресного пространства и ветви. У каждого процесса по крайней мере одна ветвь. Что такое ветвь? Фактически, ветвь - это выполняющаяся очередь. Когда Windows впервые создает процесс, она делает только одну ветвь на процесс. Эта ветвь обычно начинает выполнение с первой инструкции в модуле. Если в дальнейшем понадобится больше ветвей, он может сам создать их.

Когда Windows получает команду для создания процесса, она создает личное адресное пространство для процесса, а затем она загружает исполняемый файл в пространство. После этого она создает основную ветвь для процесса.

Под Win32 вы также можете создать процессы из своих программ с помощью функции CreateProcess. Она имеет следующий синтаксис:

```
CreateProcess proto lpApplicationName:DWORD,\
                    lpCommandLine:DWORD,\
                    lpProcessAttributes:DWORD,\
                    lpThreadAttributes:DWORD,\
                    bInheritHandles:DWORD,\
                    dwCreationFlags:DWORD,\
                    lpEnvironment:DWORD,\
                    lpCurrentDirectory:DWORD,\
                    lpStartupInfo:DWORD,\
                    lpProcessInformation:DWORD
```

Не пугайтесь количества параметров. Большую их часть мы можем игнорировать.

- lpApplicationName --> Имя исполняемого файла с или без пути, который вы хотите запустить. Если параметр равен нулю, вы должны предоставить имя исполняемого файла в параметре lpCommandLine.
- lpCommandLine --> Аргументы командной строки к программе, которую вам требуется запустить. Заметьте, что если lpApplicationName равен нулю, этот параметр должен содержать также имя исполняемого файла. Например так: "notepad.exe readme.txt".
- lpProcessAttributes и lpThreadAttributes --> Укажите атрибуты безопасности для процесса и основной ветви. Если они равны NULL'ам, то используются атрибуты безопасности по умолчанию.
- bInheritHandles --> Флаг, который указывает, хотите ли вы, чтобы новый процесс наследовал все открытые хэндлы из вашего процесса.
- dwCreationFlags --> Несколько флагов, которые определяют поведение процесса, который вы хотите создать, например, хотите ли вы, чтобы процесс был создан, но тут же приостановлен,

чтобы вы могли проверить его или изменить, прежде, чем он запустится. Вы также можете указать класс приоритета ветви(ей) в новом процессе. Этот класс приоритета используется, чтобы определить планируемый приоритет ветвей внутри процесса. Обычно мы используем флаг `NORMAL_PRIORITY_CLASS`.

- `lpEnvironment` --> Указатель на блок памяти, который содержит несколько переменных окружения для нового процесса. Если этот параметр равен `NULL`, новый процесс наследует их от родительского процесса.
- `lpCurrentDirectory` --> Указатель на строку, которая указывает текущий диск и директорию для дочернего процесса. `NULL` - если вы хотите, чтобы дочерний процесс унаследовал их от родительского процесса.
- `lpStartupInfo` --> Указывает на структуру `STARTUPINFO`, которая определяет, как должно появиться основное окно нового процесса. Эта структура содержит много членов, которые определяют появление главного окна дочернего процесса. Если вы не хотите ничего особенного, вы можете заполнить данную структуру значениями родительского процесса, вызвав функцию `GetStartupInfo`.
- `lpProcessInformation` --> Указывает на структуру `PROCESS_INFORMATION`, которая получает идентификационную информацию о новом процессе. Структура `PROCESS_INFORMATION` имеет следующие параметры:

```
PROCESS_INFORMATION STRUCT
    hProcess          HANDLE ?           ; хэндл дочернего процесса

    process
        hThread       HANDLE ?           ; хэндл основной ветви дочернего процесса
        dwProcessId    DWORD ?           ; ID дочернего процесса
        dwThreadId     DWORD ?           ; ID основной ветви
PROCESS_INFORMATION ENDS
```

Хэндл процесса и ID процесса - это две разные вещи. ID процесса - это уникальный идентификатор процесса в системе. Хэндл процесса - это значение, возвращаемое Windows для использования другими API-функциями, связанными с процессами. Хэндл процесса не может использоваться для идентификации процесса, так как он не уникален.

После вызова функции `CreateProcess`, создается новый процесс и функция сразу же возвращается. Вы можете проверить, является ли еще процесс активным, вызвав функцию `GetExitCodeProcess`, которая имеет следующий синтаксис:

```
GetExitCodeProcess proto hProcess:DWORD, lpExitCode:DWORD
```

Если вызов этой функции успешен, `lpExitcode` будет содержать код выхода запрашиваемого процесса. Если значение в `lpExitCode` равно `STILL_ACTIVE`, тогда это означает, что процесс по-прежнему запущен.

Вы можете принудительно прервать процесс, вызвав функцию `TerminateProcess`. У нее следующий синтаксис:

```
TerminateProcess proto hProcess:DWORD, uExitCode:DWORD
```

Вы можете указать желаемый код выхода для процесса, любое значение, какое захотите. `TerminateProcess` - не лучший путь прервать процесс, так как любые используемые им dll не будут уведомлены о том, что процесс был прерван.

ПРИМЕР

Следующий пример создаст новый процесс, когда юзер выберет пункт меню "create process". Он попытается запустить "msgbox.exe". Если пользователь захочет прервать новый процесс, он может выбрать пункт меню "terminate process". Программа будет сначала проверять, уничтожен ли уже новый процесс, если нет, программ вызовет `TerminateProcess` для этого.

```

.model flat,stdcall
option casemap:none

WinMain proto :DWORD,:DWORD,:DWORD,:DWORD
include \masm32\include\windows.inc
include \masm32\include\user32.inc
include \masm32\include\kernel32.inc

includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib

.const
IDM_CREATE_PROCESS equ 1
IDM_TERMINATE equ 2
IDM_EXIT equ 3

.data
ClassName db "Win32ASMPProcessClass",0

AppName db "Win32 ASM Process Example",0
MenuName db "FirstMenu",0
processInfo PROCESS_INFORMATION <>
programname db "msgbox.exe",0

.data?
hInstance HINSTANCE ?

CommandLine LPSTR ?
hMenu HANDLE ?
ExitCode DWORD ? ; содержит код выхода процесса после
                  ; вызова функции GetExitCodeProcess

.code
start:

    invoke GetModuleHandle, NULL
    mov     hInstance,eax
    invoke GetCommandLine
    mov     CommandLine,eax

    invoke WinMain, hInstance,NULL,CommandLine, SW_SHOWDEFAULT
    invoke ExitProcess,eax

WinMain proc
hInst:HINSTANCE,hPrevInst:HINSTANCE,CmdLine:LPSTR,CmdShow:DWORD
LOCAL wc:WNDCLASSEX
LOCAL msg:MSG
LOCAL hwnd:HWND

    mov     wc.cbSize,SIZEOF WNDCLASSEX
    mov     wc.style, CS_HREDRAW or CS_VREDRAW
    mov     wc.lpfnWndProc, OFFSET WndProc

    mov     wc.cbClsExtra,NULL
    mov     wc.cbWndExtra,NULL
    push    hInst
    pop     wc.hInstance

    mov     wc.hbrBackground,COLOR_WINDOW+1
    mov     wc.lpszMenuName,OFFSET MenuName
    mov     wc.lpszClassName,OFFSET ClassName
    invoke LoadIcon,NULL,IDI_APPLICATION

    mov     wc.hIcon,eax
    mov     wc.hIconSm,eax
    invoke LoadCursor,NULL,IDC_ARROW

```

```

mov    wc.hCursor,eax

invoke RegisterClassEx, addr wc
invoke CreateWindowEx,WS_EX_CLIENTEDGE,ADDR ClassName,ADDR AppName,\
    WS_OVERLAPPEDWINDOW,CW_USEDEFAULT,\
    CW_USEDEFAULT,300,200,NULL,NULL,\
    hInst,NULL

mov    hwnd,eax
invoke ShowWindow, hwnd,SW_SHOWNORMAL
invoke UpdateWindow, hwnd

invoke GetMenu,hwnd
mov    hMenu,eax
.WHILE TRUE
    invoke GetMessage, ADDR msg,NULL,0,0
    .BREAK .IF (!eax)
    invoke TranslateMessage, ADDR msg
    invoke DispatchMessage, ADDR msg
.ENDW

mov    eax,msg.wParam
ret
WinMain endp

WndProc proc hwnd:HWND, uMsg:UINT, wParam:WPARAM, lParam:LPARAM
LOCAL startInfo:STARTUPINFO
.IF uMsg==WM_DESTROY
    invoke PostQuitMessage,NULL
.ELSEIF uMsg==WM_INITMENUPOPUP
    invoke GetExitCodeProcess,processInfo.hProcess,ADDR ExitCode
    .if eax==TRUE

        .if ExitCode==STILL_ACTIVE
            invoke EnableMenuItem,hMenu,IDM_CREATE_PROCESS,MF_GRAYED
            invoke EnableMenuItem,hMenu,IDM_TERMINATE,MF_ENABLED
        .else

            invoke EnableMenuItem,hMenu,IDM_CREATE_PROCESS,MF_ENABLED
            invoke EnableMenuItem,hMenu,IDM_TERMINATE,MF_GRAYED
        .endif
    .endif
.ELSEIF uMsg==WM_COMMAND

    mov    eax,wParam
    .if lParam==0
        .if ax==IDM_CREATE_PROCESS
            .if processInfo.hProcess!=0

                invoke CloseHandle,processInfo.hProcess
                mov    processInfo.hProcess,0
            .endif
            invoke GetStartupInfo,ADDR startInfo

            invoke CreateProcess,ADDR programname,NULL,NULL,NULL,FALSE,\
                NORMAL_PRIORITY_CLASS,\
                NULL,NULL,ADDR startInfo,ADDR processInfo
            invoke CloseHandle,processInfo.hThread
        .elseif ax==IDM_TERMINATE
            invoke GetExitCodeProcess,processInfo.hProcess,ADDR ExitCode
            .if ExitCode==STILL_ACTIVE
                invoke TerminateProcess,processInfo.hProcess,0
            .endif

            invoke CloseHandle,processInfo.hProcess
            mov    processInfo.hProcess,0
        .endif
    .endif
.endif

```

```

        .else
            invoke DestroyWindow,hWnd
        .endif
    .endif
.ENDIF
.ELSE
    invoke DefWindowProc,hWnd,uMsg,wParam,lParam

    ret
.ENDIF
xor    eax,eax
ret

WndProc endp
end start

```

АНАЛИЗ

Программа создает основное окно и получает хэндл меню для последующего использования. Затем она ждет, пока пользователь выберет команду в меню. Когда пользователь выберет "Process", мы обрабатываем сообщение WM_INITMENUPOPUP, чтобы изменить пункты меню.

```

.ELSEIF uMsg==WM_INITMENUPOPUP

    invoke GetExitCodeProcess,processInfo.hProcess,ADDR ExitCode
    .if eax==TRUE
        .if ExitCode==STILL_ACTIVE
            invoke EnableMenuItem,hMenu,IDM_CREATE_PROCESS,MF_GRAYED

            invoke EnableMenuItem,hMenu,IDM_TERMINATE,MF_ENABLED
        .else
            invoke EnableMenuItem,hMenu,IDM_CREATE_PROCESS,MF_ENABLED
            invoke EnableMenuItem,hMenu,IDM_TERMINATE,MF_GRAYED
        .endif
    .else
        invoke EnableMenuItem,hMenu,IDM_CREATE_PROCESS,MF_ENABLED
        invoke EnableMenuItem,hMenu,IDM_TERMINATE,MF_GRAYED
    .endif
.endif

```

Почему мы хотим обработать это сообщение? Потому что мы хотим пункты в выпадаемом меню прежде, чем пользователь увидит их. В нашем примере, если новый процесс еще не стартовал, мы хотим разрешить "start process" и запретить доступ к пункту "terminate process". Мы делаем обратное, если программа уже запущена.

Вначале мы проверяем, активен ли еще новый процесс, вызывая функцию GetExitCodeProcess и передавая ей хэндл процесса, полученный при вызове CreateProcess. Если GetExitCodeProcess возвращает FALSE, это значит, что процесс еще не был запущен, поэтому запрещаем пункт "terminate process". Если GetExitCodeProcess возвращает TRUE, мы знаем, что новый процесс уже стартовал, мы должны проверить, выполняется ли он еще. Поэтому мы сравниваем значение в ExitCode со значением STILL_ACTIVE, если они равны, процесс еще выполняется: мы должны запретить пункт меню "start process", так как мы не хотим, чтобы запустилось несколько совпадающих процессов.

```

    .if ax==IDM_CREATE_PROCESS

        .if processInfo.hProcess!=0
            invoke CloseHandle,processInfo.hProcess
            mov processInfo.hProcess,0
        .endif

        invoke GetStartupInfo,ADDR startInfo
        invoke CreateProcess,ADDR programname,NULL,NULL,NULL,FALSE,\
            NORMAL_PRIORITY_CLASS,\
            NULL,NULL,ADDR startInfo,ADDR processInfo
        invoke CloseHandle,processInfo.hThread
    .endif

```

Когда пользователь выбирает пункт "start process", мы вначале проверяем, закрыт ли уже параметр hProcess структуры PROCESS_INFORMATION. Если это в первый раз, значение hProcess будет всегда равно нулю, так как мы определяем структуру PROCESS_INFORMATION в секции .data. Если значение параметра hProcess не равно нулю, это означает, что дочерний процесс вышел, но мы не закрыли его хэндл. Поэтому пришло время сделать это.

Мы вызываем функцию GetStartupInfo, чтобы заполнить структуру startupinfo, которую передаем функции CreateProcess. После этого мы вызываем функцию CreateProcess. Заметьте, что я не проверил возвращаемое ей значение, потому что это усложнило бы пример. Вам следует проверять это значение. Сразу же после CreateProcess, мы закрываем хэндл основной ветви, возвращаемой в структуре processInfo. Закрытие хэндла не означает, что мы прерываем ветвь, только то, что мы не хотим использовать хэндл для обращения к ветви из нашей программы. Если мы не закроем его, это вызовет потерю ресурсов.

```
.elseif ax==IDM_TERMINATE
    invoke GetExitCodeProcess,processInfo.hProcess,ADDR ExitCode
    .if ExitCode==STILL_ACTIVE

        invoke TerminateProcess,processInfo.hProcess,0
    .endif
    invoke CloseHandle,processInfo.hProcess
    mov processInfo.hProcess,0B
```

Когда пользователь выберет пункт меню "terminate process", мы проверяем, активен ли еще новый процесс, вызвав функцию GetExitCodeProcess. Если он еще активен, мы вызываем функцию TerminateProcess, чтобы убить его. Также мы закрываем хэндл дочернего процесса, так как он больше нам не нужен.

Win32 API. Урок 15. Треды (ветви)

Автор: lczelion, пер. Aquila

Дата: Не указана

Мы узнаем, как создать мультитредную программу. Мы также изучим методы, с помощью которых треды могут общаться друг с другом.

Скачайте пример здесь (находится в архиве wasm_files.zip: files/recovered/1001015/tut15.zip).

ТЕОРИЯ

В предыдущем tutoriale, вы изучили процесс, состоящий по крайней мере из одного треда: основного. Тред - это цепь инструкций. Вы также можете создавать дополнительные треды в вашей программе. Вы можете считать мультитрединг как многозадачность внутри одной программы. Если говорить в терминах непосредственной реализации, тред - это функция, которая выполняется параллельно с основной программой. Вы можете запустить несколько экземпляров одной и той же функции или вы можете запустить несколько функций одновременно, в зависимости от ваших требований. Мультитрединг свойственен Win32, под Win16 аналогов не существует.

Треды выполняются в том же процесс, поэтому они имеют доступ ко всем ресурсам процесса: глобальным переменным, хэндлам и т.д. Тем не менее, каждый тред имеет свой собственный стек, так что локальные переменные в каждом треде приватны. Каждый тред также имеет свой собственный набор регистров, поэтому когда Windows переключается на другой тред, предыдущий "запоминает" свое состояние и может "восстановить" его, когда он снова получает контроль. Это обеспечивается внутренними средствами Windows. Мы можем поделить треды на две категории:

1. Тред интерфейса пользователя: тред такого типа создает свое собственное окно, поэтому он получает оконные сообщения. Он может отвечать пользователю с помощью своего окна. Этот тип тредов действуют согласно Win16 Mutex правилу, которое позволяет только один тред пользовательского интерфейса в 16-битном пользовательском и gdi-ядре. Пока один подобный тред выполняет код 16-битного пользовательского и gdi-ядра, другие UI треды не могут использовать сервисы этого ядра. Заметьте, что этот Win16 Mutex свойственен Windows 9x, так как его функции обращаются к 16-битному коду. В Windows NT нет Win16 Mutex'a, поэтому треды пользовательского интерфейса под NT работают более плавно, чем под Windows 95.
2. Рабочий тред: Этот тип тредов не создает окно, поэтому он не может принимать какие-либо windows-сообщения. Он существует только для того, чтобы делать предназначенную ему работу на заднем фоне (согласно своему названию).

Я советую следующую стратегию при использовании мультитредовых способностей Win32: позвольте основному треду делать все, что связано с пользовательским интерфейсом, а остальным делать тяжелую работу в фоновом режиме. В этом случае, основной тред - Правитель, другие треды - его помощники. Правитель поручает им определенные задания, в то время как сам общается с публикой. Его помощники послушно выполняют работу и докладывают об этом Правителю. Если бы Правитель делал всю работу сам, он бы не смог уделять достаточно внимания народу или прессе. Это похоже на окно, которое занято продолжительной работой в основном треде: оно не отвечает пользователю, пока работа не будет выполнена. Такая программа может быть улучшена созданием дополнительного треда, который возьмет часть работы на себя и позволит основной ветви отвечать на команды пользователя.

Мы можем создать тред с помощью вызова функции `CreateThread`, которая имеет следующий синтаксис:

```
CreateThread proto lpThreadAttributes:DWORD,\
```

```
dwStackSize:DWORD,\
lpStartAddress:DWORD,\
lpParameter:DWORD,\
dwCreationFlags:DWORD,\
lpThreadId:DWORD
```

Функция CreateThread похожа на CreateProcess.

- lpThreadAttributes --> Вы можете использовать NULL, если хотите, чтобы у треда были установки безопасности по умолчанию.
- dwStackSize --> укажите размер стека треда. Если вы хотите, чтобы тред имел такой же размер стека, как и у основного, используйте NULL в качестве параметра.
- lpStartAddress --> Адрес функции треда. Эта функция будет выполнять предназначенную для треда работу. Эта функция **должна** получать один и только один 32-битный параметр и возвращать 32-битное значение.
- lpParametr --> Параметр, который вы хотите передать функции треда.
- dwCreationFlags --> 0 означает, что тред начинает выполняться сразу же после его создания. Для обратного можно использовать флаг CREATE_SUSPEND.
- lpThreadId --> CreateThread поместит сюда ID созданного треда.

Если вызов CreateThread прошел успешно, она возвращает хэндл созданного треда, в противном случае она возвращает NULL.

Функция треда запускается так скоро, как только заканчивается вызов CreateThread, если только вы не указали флаг CREATE_SUSPENDED. В этом случае тред будет заморожен до вызова функции ResumeThread.

Когда функция треда возвращается (с помощью инструкции ret) Windows косвенно вызывает ExitThread для функции треда. Вы можете сами вызвать ExitThread, но в этом немного смысла. Вы можете получить код выхода треда с помощью функции GetExitCodeThread.

Если вы хотите прервать тред из другого треда, вы можете вызвать функцию TerminateThread. Но вы должны использовать эту функцию только в экстремальных условиях, так как эта функция немедленно прерывает тред, не давая ему шанса произвести необходимую чистку за собой.

Теперь давайте рассмотрим методы коммуникации между тредами. Вот три из них:

- Использование глобальных переменных
- Windows-сообщения
- События

Треды разделяют ресурсы процесса, включая глобальные переменные, поэтому треды могут использовать их для того, чтобы взаимодействовать друг с другом. Тем не менее, этот метод должен использоваться осторожно. Синхронизацию нужно внимательно спланировать. Например, если два треда используют одну и ту же структуру из 10 членов, что произойдет, если Windows вдруг передаст управление от одного треда другому, когда структура обновлена еще только наполовину. Другой тред получит неправильную информацию! Не сделайте никакой ошибки, мультитредовые программы тяжелее отлаживать и поддерживать. Этот тип багов случается непредсказуемо и их очень трудно отловить.

Вы также можете использовать windows-сообщения, чтобы осуществлять взаимодействие между тредами. Если все треды имеют юзерский интерфейс, то нет проблем: этот метод может использоваться для двухсторонней коммуникации. Все, что вам нужно сделать - это определить один или более дополнительных windows-сообщений, которые будут использоваться тредами. Вы определяете сообщение, используя значение WM_USER как базовое, например так:

```
WM_MYCUSTOMMSG equ WM_USER+100h
```


Windows не использует сообщения с номером выше WM_USER, поэтому мы можем использовать значение WM_USER и выше для наших собственных сообщений.

Если один из тредов имеет пользовательский интерфейс, а другой является рабочим, вы не можете использовать данный метод для двухстороннего общения, так как у рабочего треда нет своего окна, а следовательно и очереди сообщений. Вы можете использовать следующие схемы:

- Тред с пользовательским интерфейсом ----> глобальная переменная(ные) ----> Рабочий тред
- Рабочий тред ----> windows-сообщение ----> Тред с пользовательским интерфейсом

Фактически, мы будем использовать этот метод в нашем примере.

Последний метод, используемый для коммуникации - это объект события. Вы можете рассматривать его как своего рода флаг. Если объект события "не установлен", значит тред спит. Когда объект события "установлен", Windows "пробуждает" тред и он начинает выполнять свою работу.

ПРИМЕР

Вам следует скачать zip-файл с примером запустить thread1.exe. Нажмите на пункт меню "Savage Calculation". Это даст команду программе выполнить "add eax, eax" 600.000.000 раз. Заметьте, что во время этого времени вы не сможете ничего сделать с главным окном: вы не сможете его двигать, активировать меню и т.д. Когда вычисление закончится, появится окно с сообщением. После этого окно будет нормально реагировать на ваши команды.

Чтобы избежать подобного неудобства для пользователя, мы должны поместить процедуру вычисления в отдельный рабочий тред и позволить основному треду продолжать взаимодействие с пользователем. Вы можете видеть, что хотя основное окно отвечает медленнее, чем обычно, оно все же делает это.

```
.386

.model flat,stdcall
option casemap:none
WinMain proto :DWORD,:DWORD,:DWORD,:DWORD
include \masm32\include\windows.inc

include \masm32\include\user32.inc
include \masm32\include\kernel32.inc
includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib

.const
IDM_CREATE_THREAD equ 1

IDM_EXIT equ 2
WM_FINISH equ WM_USER+100h

.data
ClassName db "Win32ASMThreadClass",0
AppName db "Win32 ASM MultiThreading Example",0
MenuName db "FirstMenu",0

SuccessString db "The calculation is completed!",0

.data?

hInstance HINSTANCE ?
CommandLine LPSTR ?
hwnd HANDLE ?
ThreadID DWORD ?

.code
```

start:

```
    invoke GetModuleHandle, NULL
    mov     hInstance,eax
    invoke GetCommandLine
    mov     CommandLine,eax

    invoke WinMain, hInstance,NULL,CommandLine, SW_SHOWDEFAULT
    invoke ExitProcess,eax
```

WinMain proc

hInst:HINSTANCE,hPrevInst:HINSTANCE,CmdLine:LPSTR,CmdShow:DWORD

LOCAL wc:WNDCLASSEX

LOCAL msg:MSG

```
    mov     wc.cbSize,SIZEOF WNDCLASSEX
    mov     wc.style, CS_HREDRAW or CS_VREDRAW
    mov     wc.lpfnWndProc, OFFSET WndProc
    mov     wc.cbClsExtra,NULL
```

```
    mov     wc.cbWndExtra,NULL
    push    hInst
    pop      wc.hInstance
    mov     wc.hbrBackground,COLOR_WINDOW+1
```

```
    mov     wc.lpszMenuName,OFFSET MenuName
    mov     wc.lpszClassName,OFFSET ClassName
    invoke LoadIcon,NULL,IDI_APPLICATION
    mov     wc.hIcon,eax
```

```
    mov     wc.hIconSm,eax
    invoke LoadCursor,NULL,IDC_ARROW
    mov     wc.hCursor,eax
    invoke RegisterClassEx, addr wc
```

```
    invoke CreateWindowEx,WS_EX_CLIENTEDGE,ADDR ClassName,ADDR AppName,\
        WS_OVERLAPPEDWINDOW,CW_USEDEFAULT,\
        CW_USEDEFAULT,300,200,NULL,NULL,\
        hInst,NULL
```

```
    mov     hwnd,eax
    invoke ShowWindow, hwnd,SW_SHOWNORMAL
    invoke UpdateWindow, hwnd
    .WHILE TRUE
```

```
        invoke GetMessage, ADDR msg,NULL,0,0
        .BREAK .IF (!eax)
        invoke TranslateMessage, ADDR msg
        invoke DispatchMessage, ADDR msg
```

```
    .ENDW
    mov     eax,msg.wParam
    ret
```

WinMain endp

WndProc proc hWnd:HWND, uMsg:UINT, wParam:WPARAM, lParam:LPARAM

.IF uMsg==WM_DESTROY

invoke PostQuitMessage,NULL

.ELSEIF uMsg==WM_COMMAND

mov eax,wParam

.if lParam==0

.if ax==IDM_CREATE_THREAD

mov eax,OFFSET ThreadProc

invoke CreateThread,NULL,NULL,eax,\
 0,\

ADDR ThreadID

```

        invoke CloseHandle,eax
    .else
        invoke DestroyWindow,hWnd

    .endif
    .endif
    .ELSEIF uMsg==WM_FINISH
        invoke MessageBox,NULL,ADDR SuccessString,ADDR AppName,MB_OK

    .ELSE
        invoke DefWindowProc,hWnd,uMsg,wParam,lParam
        ret
    .ENDIF

    xor     eax,eax
    ret
WndProc endp

```

```

ThreadProc PROC USES ecx Param:DWORD
    mov     ecx,600000000
Loop1:

    add     eax,eax
    dec     ecx
    jz      Get_out
    jmp     Loop1

Get_out:
    invoke  PostMessage,hwnd,WM_FINISH,NULL,NULL
    ret

ThreadProc ENDP

end start

```

АНАЛИЗ

Основную программу пользователь воспринимает как обычное окно с меню. Если пользователь выбирает в последнем пункт "Создать тред", программа создает тред:

```

    .if ax==IDM_CREATE_THREAD
        mov     eax,OFFSET ThreadProc
        invoke  CreateThread,NULL,NULL,eax,\
                NULL,0,\
                ADDR ThreadID
        invoke  CloseHandle,eax
    .endif

```

Вышеприведенная функция создает тред, который запустит процедуру под названием ThreadProc параллельно с основным тредом. Если вызов функции прошел успешно, CreateThread немедленно возвращается и ThreadProc начинает выполняться. Так как мы не используем хэндл треда, нам следует закрыть его, чтобы не допустить бессмысленное расходование памяти. Заккрытие хэндла не прерывает сам тред. Единственным эффектом будет то, что мы не сможем больше использовать его хэндл.

```

ThreadProc PROC USES ecx Param:DWORD

    mov     ecx,600000000
Loop1:

    add     eax,eax
    dec     ecx

    jz      Get_out
    jmp     Loop1
Get_out:
    invoke  PostMessage,hwnd,WM_FINISH,NULL,NULL

    ret

```

ThreadProc ENDP

Как вы можете видеть ThreadProc выполняет подсчет, требующий некоторого времени, и когда она заканчивает его, она отправляет сообщение WM_FINISH основному окну. WM_FINISH - это наше собственное сообщение, определенное следующим образом:

```
WM_FINISH equ WM_USER+100h
```

Вам не обязательно добавлять к WM_USER 100h, но будет лучше сделать это. Сообщение WM_FINISH имеет значение только в пределах нашей программы. Когда основное окно получает WM_FINISH, оно реагирует на это показом окна с сообщением о том, что подсчет закончен.

Вы можете создать несколько тредов, выбрав "Create Thread" несколько раз. В этом примере применяется односторонняя коммуникация, то есть только тред может уведомлять основное окно о чем-либо. Если вы хотите, что основной тред слал команды рабочему, вы должны сделать следующее:

- добавить пункт меню "Kill Thread".
- добавить глобальную переменную, используемую в качестве флага. TRUE = остановить тред, FALSE = продолжить тред.
- Изменить ThreadProc так, чтобы та проверяла в цикле значение флага.

Когда пользователь выберет "Kill Thread", основная программа установит флаг в TRUE. Когда ThreadProc видит, что значение флага равно TRUE, она выходит из цикла и возвращается, что заканчивает действие треда.

Win32 API. Урок 16. Объект события

Автор: lczelion, пер. Aquila

Дата: 2002-05-16

Мы изучим, что такое объект события и как использовать его в мультитредной программе.

Скачайте пример здесь (находится в архиве wasm_files.zip: files/recovered/1001016/tut16.zip).

ТЕОРИЯ

В предыдущем tutorialе я продемонстрировал, как треды взаимодействуют друг с другом через собственные windows-сообщения. Я пропустил два других метода: глобальная переменная и объект события. В этом tutorialе мы используем оба.

Объект события - это что-то вроде переключателя: у него есть только два состояния: вкл и выкл. Вы создаете объект события и помещаете его в код соответствующего треда, где наблюдаете за состоянием объекта. Если объект события выключен, ждущие его треды "спать". В подобном состоянии треды мало загружают CPU.

Вы можете создать объект события, вызвав функцию `CreateEvent`, которая имеет следующий синтаксис:

```
CreateEvent proto lpEventAttributes:DWORD,\
                    bManualReset:DWORD,\
                    bInitialState:DWORD,\
                    lpName:DWORD
```

- `lpEventAttribute` --> Если вы укажете значение `NULL`, у создаваемого объекта будут установки безопасности по умолчанию.
- `bManualReset` --> Если вы хотите, чтобы Windows автоматически переключал объект события в "выключено", вы должны присвоить этому параметру значение `FALSE`. Иначе вам надо будет выключить объект вручную с помощью вызова `ResetEvent`.
- `bInitialStae` --> Если вы хотите, чтобы объект события при создании был установлен в положение "включено", укажите `TRUE` в качестве данного параметра, в противном случае объект события будет установлен в положение "выключен".

Указатель на ASCIIZ-строку, которая будет именем объекта события. Это имя будет использоваться, когда вы захотите вызвать `OpenEvent`.

Если вызов прошел успешно, `CreateEvent` возвратит хэндл на созданный объект события. В противном случае она возвратит `NULL`.

Вы можете изменять состояние объекта события с помощью двух API-функций: `SetEvent` и `ResetEvent`. Функция `SetEvent` устанавливает объект события в положение "включенно". `ResetEvent` делает обратное.

Когда объект события создан, вы должны поместить вызов функции `WaitForSingleObject` в тред, который должен следить за состоянием объекта события. Эта функция имеет следующий синтаксис:

```
WaitForSingleObject proto hObject:DWORD, dwTimeout:DWORD
```

- `hObject` --> Хэндл одного из синхронизационных объектов. Объект события - это вид синхронизационного события.
- `dwTimeout` --> Указывает в миллисекундах время, которое эта функция будет ждать, пока объект события не перейдет во включенное состояние. Если указанное время пройдет, а объект события все еще выключен, `WaitForSingleObject` вернет управление. Если вы хотите, чтобы функция

наблюдала за объектом бесконечно, вы должны указать значение INFINITE в качестве этого параметра.

ПРИМЕР

Нижеприведенный пример отображает окно, ожидающее пока пользователь не выберет какую-нибудь команду из меню. Если пользователь выберет "run thread", тред начнет подсчет. Когда он закончит, появится сообщение, информирующее пользователя о том, что работа выполнена. Во время того, как проводится подсчет, пользователь может выбрать команду "stop thread", чтобы остановить тред.

```
.386
.model flat,stdcall
option casemap:none
WinMain proto :DWORD,:DWORD,:DWORD,:DWORD

include \masm32\include\windows.inc
include \masm32\include\user32.inc
include \masm32\include\kernel32.inc
include lib \masm32\lib\user32.lib

include lib \masm32\lib\kernel32.lib

.const

IDM_START_THREAD equ 1
IDM_STOP_THREAD equ 2
IDM_EXIT equ 3
WM_FINISH equ WM_USER+100h

.data
ClassName db "Win32ASMEventClass",0

AppName db "Win32 ASM Event Example",0
MenuName db "FirstMenu",0
SuccessString db "The calculation is completed!",0
StopString db "The thread is stopped",0

EventStop BOOL FALSE

.data?

hInstance HINSTANCE ?
CommandLine LPSTR ?
hwnd HANDLE ?
hMenu HANDLE ?

ThreadID DWORD ?
ExitCode DWORD ?
hEventStart HANDLE ?

.code
start:
    invoke GetModuleHandle, NULL

    mov     hInstance,eax
    invoke GetCommandLine
    mov     CommandLine,eax
    invoke WinMain, hInstance,NULL,CommandLine, SW_SHOWDEFAULT

    invoke ExitProcess,eax

WinMain proc

hInst:HINSTANCE,hPrevInst:HINSTANCE,CmdLine:LPSTR,CmdShow:DWORD
LOCAL wc:WNDCLASSEX
LOCAL msg:MSG
```

```

mov     wc.cbSize,SIZEOF WNDCLASSEX

mov     wc.style, CS_HREDRAW or CS_VREDRAW
mov     wc.lpfnWndProc, OFFSET WndProc
mov     wc.cbClsExtra,NULL
mov     wc.cbWndExtra,NULL

push    hInst
pop      wc.hInstance
mov     wc.hbrBackground,COLOR_WINDOW+1
mov     wc.lpszMenuName,OFFSET MenuName

mov     wc.lpszClassName,OFFSET ClassName
invoke  LoadIcon,NULL,IDI_APPLICATION
mov     wc.hIcon,eax
mov     wc.hIconSm,eax

invoke  LoadCursor,NULL,IDC_ARROW
mov     wc.hCursor,eax
invoke  RegisterClassEx, addr wc
invoke  CreateWindowEx,WS_EX_CLIENTEDGE,ADDR ClassName,\

        ADDR AppName,\
        WS_OVERLAPPEDWINDOW,CW_USEDEFAULT,\
        CW_USEDEFAULT,300,200,NULL,NULL,\
        hInst,NULL

mov     hwnd,eax
invoke  ShowWindow, hwnd,SW_SHOWNORMAL
invoke  UpdateWindow, hwnd
invoke  GetMenu,hwnd

mov     hMenu,eax
.WHILE TRUE
    invoke GetMessage, ADDR msg,NULL,0,0
    .BREAK .IF (!eax)

    invoke TranslateMessage, ADDR msg
    invoke DispatchMessage, ADDR msg
.ENDW
mov     eax,msg.wParam

ret
WinMain endp

WndProc proc hwnd:HWND, uMsg:UINT, wParam:WPARAM, lParam:LPARAM
    .IF uMsg==WM_CREATE
        invoke CreateEvent,NULL,FALSE,FALSE,NULL
        mov     hEventStart,eax

        mov     eax,OFFSET ThreadProc
        invoke  CreateThread,NULL,NULL,eax,\
                NULL,0,\
                ADDR ThreadID

        invoke  CloseHandle,eax
    .ELSEIF uMsg==WM_DESTROY
        invoke  PostQuitMessage,NULL
    .ELSEIF uMsg==WM_COMMAND

        mov     eax,wParam
        .if lParam==0
            .if ax==IDM_START_THREAD
                invoke SetEvent,hEventStart

                invoke EnableMenuItem,hMenu,IDM_START_THREAD,MF_GRAYED
                invoke EnableMenuItem,hMenu,IDM_STOP_THREAD,MF_ENABLED
            .elseif ax==IDM_STOP_THREAD
                mov     EventStop,TRUE
                invoke EnableMenuItem,hMenu,IDM_START_THREAD,MF_ENABLED

```

```

        invoke EnableMenuItem,hMenu,IDM_STOP_THREAD,MF_GRAYED
    .else
        invoke DestroyWindow,hWnd

    .endif
    .endif
    .ELSEIF uMsg==WM_FINISH
        invoke MessageBox,NULL,ADDR SuccessString,ADDR AppName,MB_OK

    .ELSE
        invoke DefWindowProc,hWnd,uMsg,wParam,lParam
        ret
    .ENDIF

    xor     eax,eax
    ret
WndProc endp

ThreadProc PROC USES ecx Param:DWORD
    invoke WaitForSingleObject,hEventStart,INFINITE
    mov     ecx,600000000

    .WHILE ecx!=0
        .if EventStop!=TRUE
            add     eax,eax
            dec     ecx
        .else
            invoke MessageBox,hwnd,ADDR StopString,ADDR AppName,MB_OK
            mov     EventStop,FALSE
            jmp     ThreadProc
        .endif
    .ENDW
    invoke PostMessage,hwnd,WM_FINISH,NULL,NULL

    invoke EnableMenuItem,hMenu,IDM_START_THREAD,MF_ENABLED
    invoke EnableMenuItem,hMenu,IDM_STOP_THREAD,MF_GRAYED
    jmp     ThreadProc
    ret

ThreadProc ENDP
end start

```

АНАЛИЗ

В этом примере я демонстрирую другую технику работы с тредами.

```

    .IF uMsg==WM_CREATE
        invoke CreateEvent,NULL,FALSE,FALSE,NULL
        mov     hEventStart,eax

        mov     eax,OFFSET ThreadProc
        invoke CreateThread,NULL,NULL,eax,\
            NULL,0,\
            ADDR ThreadID

        invoke CloseHandle,eax
    .ENDIF

```

Вы можете видеть, что я создал объект события и тред во время обработки сообщения WM_CREATE. Я создаю объект события, установленного в состояние "выключенно" и обладающего свойством автоматического выключения. После того, как объект события создан, я создаю тред. Тем не менее, тред не начинает выполняться немедленно, так как он ждет, пока не включится объект события:

```

ThreadProc PROC USES ecx Param:DWORD

    invoke WaitForSingleObject,hEventStart,INFINITE
    mov     ecx,600000000

```


Первая линия процедуры треда - это вызов `WaitForSingleObject`. Она ждет, пока не включится объект события, а затем возвращается. Это означает, что даже если тред создан, мы помещаем его в спящее состояние. Когда пользователь выбирает в меню команду "run thread", мы включаем объект события:

```
.if ax==IDM_START_THREAD
    invoke SetEvent,hEventStart
```

Вызов `SetEvent` включает объект события, после чего `WaitForSingleObject` возвращается и тред начинает выполняться. Когда пользователь выбирает команду "stop thread", мы устанавливаем значение глобальной переменной в `TRUE`.

```
.if EventStop==FALSE
    add    eax,eax
    dec    ecx
.else
    invoke MessageBox,hwnd,ADDR StopString,ADDR AppName,MB_OK
    mov    EventStop,FALSE
    jmp    ThreadProc
.endif
```

Это останавливает тред и снова передает управление функции `WaitForSingleObject`. Заметьте, что мы не должны вручную выключать объект, так как мы указали при вызове функции `CreateEvent`, что значение `bManualReset` равно `FALSE`.

Win32 API. Урок 17. Динамические библиотеки

Автор: lczelion, пер. Aquila

Дата: 2002-05-17

В этом tutorialе мы узнаем о dll, что это такое и как их создавать.

Вы можете скачать пример [здесь](#) (находится в архиве `wasm_files.zip: files/recovered/1001017/tut17.zip`).

ТЕОРИЯ

Если вы программируете достаточно долго, вы заметите, что программы, которые вы пишете, зачастую используют один и те же общие процедуры. Из-за того, что вам приходится переписывать их снова и снова, вы теряете время. Во времена DOS'a программисты сохраняли эти общие процедуры в одной или более библиотеках. Когда они хотели использовать эти функции, они всего лишь прилинковывали библиотеку к объектному файлу и линкер извлекал функции прямо из библиотек и вставлял их в финальный файл. Этот процесс называется статической линковкой. Хорошим примером являются стандартные библиотеки в C. У этого метода есть изъян - то, что в каждой программе у вас находятся абсолютно одинаковые копии функций. Впрочем, для ДОСовских программ это не очень большой недостаток, так как только одна программа могла быть активной в памяти, поэтому не происходила трата драгоценной памяти.

Под Windows ситуация стала более критичной, так как у вас может быть несколько программ, выполняющихся одновременно. Память будет быстро пожираться, если ваша программа достаточно велика. У Windows есть решение этой проблемы: динамические библиотеки (dynamic link libraries). Динамическая библиотека - это что-то вроде сборника общих функций. Windows не будет загружать несколько копий DLL в память; даже если одновременно выполняются несколько экземпляров вашей программы, будет только одна копия DLL в памяти. Здесь я должен остановиться и разъяснить чуть поподробнее. В реальности, у всех процессов, использующих одну и ту же dll есть своя копия этой библиотеки, однако Windows делает так, чтобы все процессы разделяли один и тот же код этой dll. Впрочем, секция данных копируется для каждого процесса.

Программа линкуется к DLL во время выполнения в отличие от того, как это осуществлялось в старых статических библиотеках. Вы также можете выгрузить DLL во время выполнения, если она вам больше не нужна. Если программа одна использует эту DLL, тогда та будет выгружена немедленно. Но если ее еще используют какие-то другие программы, DLL останется в памяти, пока ее не выгрузит последняя из использующих ее программ.

Как бы то ни было, перед линкером стоит сложная задача, когда он проводит фиксирование адресов в конечном исполняемом файле. Так как он не может "извлечь" функции и вставить их в финальный исполняемый файл, он должен каким-то образом сохранить достаточно информации о DLL и используемых функциях в выходном файле, чтобы тот смог найти и загрузить верную DLL во время выполнения.

И тут в дело вступают библиотеки импорта. Библиотека импорта содержит информацию о DLL, которую она представляет. Линкер может получить из нее необходимую информацию и вставить ее в исполняемый файл.

Когда Windows загружает программу в память, она видит, что программа требует DLL, поэтому ищет библиотеку и мэппирует ту в адресное пространство процесса и выполняет фиксацию адресов для вызовов функций в DLL.

Вы можете загрузить DLL самостоятельно, не полагаясь на Windows-загрузчик.

- В этом случае вам не потребуется библиотека импорта, поэтому вы сможете загружать и использовать любую DLL, даже если к ней не прилагается библиотеки импорта. Тем не менее, вы все равно нужно знать какие функции находятся внутри нее, сколько параметров они принимают и тому подобную информацию.
- Когда вы поручаете Windows загружать DLL, если та отсутствует, Windows выдаст сообщение "Требуемый .DLL-файл, xxxxx.dll отсутствует" и все! Ваша программ не может сделать ничего, что изменить это, даже если ваша dll не является необходимой. Если же вы будете загружать DLL самостоятельно и библиотека не будет найдена, ваша программа может выдать пользователю сообщение, уведомляющее об этом, и продолжить работу.
- Вы можете вызывать *недокументированные* функции, которые не включены в библиотеки импорта, главное, чтобы у вас было достаточно информации об этих функциях.
- Если вы используете LoadLibrary, вам придется вызывать GetProcAddress для каждой функции, которую вы заходите вызвать. GetProcAddress получает адрес входной точки функции в определенной DLL. Поэтому ваш код будет чуть-чуть больше и медленнее, но не намного.

Теперь, рассмотрев преимущества и недостатки использования LoadLibrary, мы подробно рассмотрим как создать DLL.

Следующий код является каркасом DLL.

```
;-----
;                               DLLSkeleton.asm
;-----

.386
.model flat,stdcall

opt*on casemap:none
inc*ude \masm32\include\windows.inc
inc*ude \masm32\include\user32.inc
inc*ude \masm32\include\kernel32.inc

includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib

.data
.code
DllEntry proc hInstDLL:HINSTANCE, reason:DWORD, reserved1:DWORD
    mov     eax,TRUE
    *
    ret
Dll*ntry Endp

;-----
; Это функция-пустышка - она ничего не делает. Я поместил ее сюда, чтобы
; показать, как вставляют функции в DLL.
;-----
TestFunction proc
    ret
TestFunction endp

End DllEntry

;-----
;                               DLLSkeleton.def
;-----

LIBRARY     DLLSkeleton
EXPORTS     TestFunction
```

Вышеприведенная программа - это каркас DLL. Каждая DLL должна иметь стартовую функцию. Windows вызывает эту функцию каждый раз, когда: DLL загружена в первый раз DLL выгружена Создается тред в том же процессе Тред разрушен в том же процессе

```

DllEntry proc hInstDLL:HINSTANCE, reason:DWORD, reserved1:DWORD

    mov     eax,TRUE
    ret

DllEntry Endp

```

Вы можете назвать стартовую функцию как пожелаете, главное чтобы был END <имя_стартовой_функции>. Эта функция получает три параметра, только первые два из них важны.

hInstDLL - это хэндл модуля DLL. Это не тоже самое, что хэндл процесса. Вам следует сохранить это значение, так как оно понадобится вам позже. Вы не сможете ее получить в дальнейшем легко.

reason может иметь одно из следующих четырех значений:

- DLL_PROCESS_ATTACH - DLL получает это значение, когда впервые загружается в адресное пространство процесса. Вы можете использовать эту возможность для того, чтобы осуществить инициализацию.
- DLL_PROCESS_DETACH - DLL получает это значение, когда выгружается из адресного пространства процесса. Вы можете использовать эту возможность для того, чтобы "почистить" за собой: освободить память и так далее.
- DLL_THREAD_ATTACH - DLL получает это значение, когда процесс создает новую ветвь.
- DLL_THREAD_DETACH - DLL получает это значение, когда ветвь в процессе уничтожена.

Вы возвращаете TRUE в eax, если вы хотите, чтобы DLL продолжала выполняться. Если вы возвратите FALSE, DLL не будет загружена. Например, если ваш инициализационный код должен зарезервировать память и он не может это сделать, стартовой функции следует вернуть FALSE, чтобы показать, что DLL не может запуститься.

Вы можете поместить ваши функции в DLL следом за стартовой функцией или до нее. Но если вы хотите, чтобы их можно было вызвать из других программ, вы должны поместить их имена в списке экспортов в файле установок модуля.

DLL требуется данный файл на стадии разработки. Мы сейчас посмотрим, что это такое.

```

LIBRARY    DLLSkeleton
EXPORTS    TestFunction

```

Обычно у вас должна быть первая строка. Ключевое слово LIBRARY определяет внутреннее имя модуля DLL. Желательно, чтобы оно совпадало с именем файла.

EXPORTS говорит линкеру, какие функции в DLL экспортируются, то есть, могут вызываться из других программ. В прилагающемся примере нам нужно, чтобы другие модули могли вызывать TestFunction, поэтому мы указываем здесь ее имя.

Другое отличие заключается в параметрах, передаваемых линкеру. Вы должны указать /DLL и /DEF:<имя вашего def-файла>.

link/DLL /SUBSYSTEM:WINDOWS/DEF:DLLSkeleton.def/LIBPATH:c:\masm32\lib DLLSkeleton.obj

Параметры ассемблера те же самые, обычно /c /coff /Cp. После компиляции вы получите .dll и .lib. Последний файл - это библиотека импорта, которую вы можете использовать, чтобы прилинковать к другим программам функции из соответствующей .dll.

Далее я покажу вам как использовать LoadLibrary, чтобы загрузить DLL.

```

;-----
;                               UseDLL.asm
;-----

.386
.model flat,stdcall

```

```

option casemap:none
include \masm32\include\windows.inc

include \masm32\include\user32.inc
include \masm32\include\kernel32.inc
include lib \masm32\lib\kernel32.lib
include lib \masm32\lib\user32.lib

.data
LibName db "DLLSkeleton.dll",0
FunctionName db "TestHello",0
DllNotFound db "Cannot load library",0
AppName db "Load Library",0
FunctionNotFound db "TestHello function not found",0

.data?
hLib dd ? ; хэндл библиотеки (DLL)
TestHelloAddr dd ? ; адрес функции TestHello

.code
start:
    invoke LoadLibrary,addr LibName

;-----
; Вызываем LoadLibrary и передаем имя желаемой DLL. Если вызов проходит успешно,
; будет возвращен хэндл библиотеки (DLL). Если нет, то будет возвращен NULL.
; Вы можете передать хэндл библиотеки функции GetProcAddress или любой другой
; функции, которая требует его в качестве одного из параметров.
;-----

    .if eax==NULL
        invoke MessageBox,NULL,addr DllNotFound,addr AppName,MB_OK

    .else

        mov hLib,eax
        invoke GetProcAddress,hLib,addr FunctionName

;-----
; Когда вы получаете хэндл библиотеки, вы передаете его GetProcAddress вместе
; с именем функции в этой dll, которую вы хотите вызвать. Она возвратит адрес
; функции, если вызов пройдет успешно. В противном случае, она возвратит NULL.
; Адреса функций не изменятся, пока вы не перезагрузите библиотеку. Поэтому
; их можно поместить в глобальные переменные для будущего использования.
;-----

    .if eax==NULL
        invoke MessageBox,NULL,addr FunctionNotFound,addr AppName,MB_OK
    .else

        mov TestHelloAddr,eax
        call [TestHelloAddr]

;-----
; Затем мы вызываем функцию с помощью call и переменной, содержащей адрес
; функции в качестве операнда.
;-----

    .endif

    invoke FreeLibrary,hLib

;-----
; Когда вам больше не требуется библиотека, выгрузите ее с помощью FreeLibrary.
;-----

    .endif
    invoke ExitProcess,NULL
end start

```

Как вы можете видеть, использование LoadLibrary чуть сложнее, но гораздо гибче.

Win32 API. Урок 18. Common Control'ы

Автор: lczelion, пер. Aquila

Дата: 2002-05-18

Мы узнаем, что такое common control'ы и как их использовать. Этот tutorial является не более, чем поверхностным введением в данную тему.

Скачайте код примера [здесь](#) (находится в архиве `wasm_files.zip: files/recovered/1001018/tut18.zip`).

ТЕОРИЯ

Windows 95 принесла несколько новых элементов пользовательского интерфейса, сделавших GUI более разнообразным. Некоторые из них широко использовались и в Windows 3.1, но программисты должны были программировать их самостоятельно. Теперь Микрософт включил их в Windows 9x и NT. Мы изучим их в этом tutorialе.

Вот список новых контролов:

- Toolbar
- Tooltip
- Status bar
- Property sheet
- Property page
- Tree view
- List view
- Animation
- Drag list
- Header
- Hot-key
- Image list
- Progress bar
- Right edit
- Tab
- Trackbar
- Up-down

Так как новых контролов довольно много, их загрузка в память и регистрация была бы бессмысленной тратой ресурсов. Все эти элементы управления, за исключением rich edit'a, находятся в `comctl32.dll`, чтобы приложения могли загружать их, когда они им нужны. Rich edit находится в своей собственной dll, `richedXX.dll`, так как он слишком сложен и поэтому больше, чем остальные.

Вы можете вызвать `comctl32.dll`, поместив вызов функции `InitCommonControls` в вашу программу. `InitCommonControls` - это функция в `comctl32.dll`, поэтому ее вызов в любом месте вашего кода заставит PE-загрузчик загрузить `comctl32.dll`, когда ваша программа запустится. Вам не нужно выполнять эту функцию, просто поместите ее где-нибудь. Эта функция ничего не делает! Ее единственной инструкцией является "ret". Ее главная цель - это создание ссылки на `comctl32.dll` в секции импорта, чтобы PE-загрузчик загружал ее всегда, когда будет загружаться программа. Главным следствием будет являться то, что стартовая функция DLL регистрирует все классы common control'ов при загрузке dll. Common control'ы создаются на основе этих классов, как и другие дочерние элементы окон, например, edit control, listbox и так далее.

С rich edit'ом дел обстоит совершенно по другому. Если вы хотите использовать его, вы должны вызвать LoadLibrary, чтобы загрузить его и FreeLibrary, чтобы выгрузить. Теперь давайте научимся создавать common control'ы. Вы можете использовать редактор ресурсов, чтобы внедрить их в диалоговое окно, или создать их самостоятельно. Почти все common control'ы создаются с помощью вызова CreateWindowEx или CreateWindow, путем передачи имени класса контрола. У некоторых common control'ов есть специальные функции для создание, хотя, на самом деле, они являются функциями-обертками вокруг CreateWindowEx, чтобы сделать создание элемента управления легче. Такие функции перечислены ниже:

- CreateToolBarEx
- CreateStatusWindow
- CreatePropertySheetPage
- PropertySheet
- ImageList_Create

Чтобы создавать common control'ы, вы должны знать их имена. Они перечислены ниже:

Имя класса	Common Control'ы
-----	-----
ToolBarWindow32	Toolbar
tooltips_class32	Tooltip
msctls_statusbar32	Status bar
SysTreeView32	Tree view
SysListView32	List view
SysAnimate32	Animation
SysHeader32	Header
msctls_hotkey32	Hot-key
msctls_progress32	Progress bar
RICEDIT	Rich edit
msctls_updown32	Up-down
SysTabControl32	Tab

Property sheet'ы и property page'ы и контрол image list имеют собственные функции создания. Drag list control - это усовершенствованный listbox, поэтому у него нет своего собственного класса. Вышеприведенные имена проверены путем проверки скриптов ресурсов, генерируемых редактором ресурсов, входящего в Visual C++. Они отличаются от имен, приведенных в справочнике по Win32 API от Borland'a и тех, что указаны в книге Charles Petzold's "Programming Windows 95". Вышеприведенный список является точной версией.

Эти common control'ы могут использовать общие стили окна, такие как WS_CHILD и т.п. У них также есть специальные стили, такие как TVS_XXXXX для tree view control'a, LVS_XXXX для list view control'a и т.д. Справочник по Win32 API ваше лучшее руководство в данном случае.

Теперь, когда мы знаем, как создать common control'ы, мы можем перейти к тому, как взаимодействуют common control'ы и их родители. В отличие от дочерних элементов управления, common control'ы не взаимодействуют с родительским окном через WM_COMMAND. Вместо этого они используют сообщение WM_NOTIFY, посылаемое родительскому окну, когда происходит какое-то интересное событие. "Родитель" может контролировать "детей", посылая им определенные сообщения, которые введено достаточно много. Вам следует обратиться к справочнику по Win32 API за конкретными деталями.

Давайте посмотрим, как создать progress bar и status bar.

ПРИМЕР

```
.386
.model flat,stdcall
option casemap:none

include \masm32\include\windows.inc
include \masm32\include\user32.inc
include \masm32\include\kernel32.inc
```

```

include \masm32\include\comctl32.inc

includelib \masm32\lib\comctl32.lib
includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib

WinMain PROTO :DWORD,:DWORD,:DWORD,:DWORD

.const
IDC_PROGRESS equ 1          ; control IDs
IDC_STATUS equ 2
IDC_TIMER equ 3

.data
ClassName db "CommonControlWinClass",0

AppName db "Common Control Demo",0
ProgressClass db "msctls_progress32",0 ; the class name of the progress bar
Message db "Finished!",0

TimerID dd 0

.data?

hInstance HINSTANCE ?
hwndProgress dd ?
hwndStatus dd ?
CurrentStep dd ?

.code
start:
    invoke GetModuleHandle, NULL
    mov hInstance,eax

    invoke WinMain, hInstance,NULL,NULL, SW_SHOWDEFAULT
    invoke ExitProcess,eax
    invoke InitCommonControls

WinMain proc
hInst:HINSTANCE,hPrevInst:HINSTANCE,CmdLine:LPSTR,CmdShow:DWORD
LOCAL wc:WNDCLASSEX

LOCAL msg:MSG
LOCAL hwnd:HWND

    mov wc.cbSize,SIZEOF WNDCLASSEX
    mov wc.style, CS_HREDRAW or CS_VREDRAW
    mov wc.lpfnWndProc, OFFSET WndProc
    mov wc.cbClsExtra,NULL

    mov wc.cbWndExtra,NULL
    push hInst
    pop wc.hInstance
    mov wc.hbrBackground,COLOR_APPWORKSPACE

    mov wc.lpszMenuName,NULL
    mov wc.lpszClassName,OFFSET ClassName
    invoke LoadIcon,NULL,IDI_APPLICATION
    mov wc.hIcon,eax

    mov wc.hIconSm,eax
    invoke LoadCursor,NULL,IDC_ARROW
    mov wc.hCursor,eax
    invoke RegisterClassEx, addr wc

    invoke CreateWindowEx,WS_EX_CLIENTEDGE,ADDR ClassName,ADDR AppName,\

```



```

WS_OVERLAPPED+WS_CAPTION+WS_SYSMENU+WS_MINIMIZEBOX+WS_MAXIMIZEBOX+WS_VISIBLE, \
CW_USEDEFAULT,CW_USEDEFAULT,CW_USEDEFAULT,NULL,NULL,\
    hInst,NULL

    mov     hwnd,eax
    .while TRUE
        invoke GetMessage, ADDR msg,NULL,0,0
        .BREAK .IF (!eax)

        invoke TranslateMessage, ADDR msg
        invoke DispatchMessage, ADDR msg
    .endw
    mov     eax,msg.wParam

    ret
WinMain endp

WndProc proc hwnd:HWND, uMsg:UINT, wParam:WPARAM, lParam:LPARAM
    .if uMsg==WM_CREATE
        invoke CreateWindowEx,NULL,ADDR ProgressClass,NULL,\
            WS_CHILD+WS_VISIBLE,100,\
            200,300,20,hwnd,IDC_PROGRESS,\
            hInstance,NULL
        mov     hwndProgress,eax
        mov     eax,1000                ; the lParam of PBM_SETRANGE message

contains the range
        mov     CurrentStep,eax
        shl     eax,16                ; the high range is in the high word
        invoke  SendMessage,hwndProgress,PBM_SETRANGE,0,eax

        invoke  SendMessage,hwndProgress,PBM_SETSTEP,10,0
        invoke  CreateStatusWindow,WS_CHILD+WS_VISIBLE,NULL,hwnd,IDC_STATUS

        mov     hwndStatus,eax
        invoke  SetTimer,hwnd,IDC_TIMER,100,NULL        ; create a timer

        mov     TimerID,eax
    .elseif uMsg==WM_DESTROY
        invoke  PostQuitMessage,NULL
        .if TimerID!=0

            invoke KillTimer,hwnd,TimerID
        .endif
    .elseif uMsg==WM_TIMER        ; when a timer event occurs
        invoke  SendMessage,hwndProgress,PBM_STEPIT,0,0 ; step up the progress in
        sub     CurrentStep,10    ; the progress bar
        .if CurrentStep==0
            invoke KillTimer,hwnd,TimerID

            mov     TimerID,0
            invoke  SendMessage,hwndStatus,SB_SETTEXT,0,addr Message
            invoke  MessageBox,hwnd,addr Message,addr
AppName,MB_OK+MB_ICONINFORMATION

            invoke  SendMessage,hwndStatus,SB_SETTEXT,0,0
            invoke  SendMessage,hwndProgress,PBM_SETPOS,0,0
        .endif
    .else

        invoke  DefWindowProc,hwnd,uMsg,wParam,lParam
        ret
    .endif
    xor     eax,eax

    ret
WndProc endp
end start

```

АНАЛИЗ

```

invoke WinMain, hInstance, NULL, NULL, SW_SHOWDEFAULT
invoke ExitProcess, eax
invoke InitCommonControls

```

Я специально поместил InitCommonControls после ExitProcess, чтобы продемонстрировать то, что эта функция необходима только для создания ссылки на comctl32.dll в секции импорта. Как вы можете видеть, common control'ы работают, даже если функция InitCommonControls не запускалась.

```

.if uMsg==WM_CREATE

    invoke CreateWindowEx, NULL, ADDR ProgressClass, NULL, \
        WS_CHILD+WS_VISIBLE, 100, \
        200, 300, 20, hWnd, IDC_PROGRESS, \
        hInstance, NULL

    mov hWndProgress, eax

```

Здесь мы создаем common control. Заметьте, что вызов CreateWindowEx содержит hWnd в качестве хэндла родительского окна. Он также задает ID контрола, для идентификации последнего. Тем не менее, так как у нас есть хэндл окна контрола, этот ID не используется. Все дочерние окна должны иметь стиль WS_CHILD.

```

    mov eax, 1000
    mov CurrentStep, eax

    shl eax, 16
    invoke SendMessage, hWndProgress, PBM_SETRANGE, 0, eax
    invoke SendMessage, hWndProgress, PBM_SETSTEP, 10, 0

```

После того, как создан progress bar, мы можем установить его диапазон. Диапазон по умолчанию равен от 0 до 100. Если это вас не устраивает, вы можете указать ваш собственный диапазон с помощью сообщения PBM_SETRANGE. lParam этого сообщения содержит диапазон, максимальное значение в верхнем слове и минимальное в нижнем. Вы также можете указать шаг, используя сообщение PBM_SETSTEP. Этот пример устанавливает его в 10, что означает то, что когда вы посылаете сообщение PBM_STEPIT прогресс бару, индикатор прогресса будет повышаться на 10. Вы также можете установить положение индикатора, послав сообщение PBM_SETPOS. Это сообщение дает вам полный контроль над progress bar'ом.

```

    invoke CreateStatusWindow, WS_CHILD+WS_VISIBLE, NULL, hWnd, IDC_STATUS
    mov hWndStatus, eax
    invoke SetTimer, hWnd, IDC_TIMER, 100, NULL ; create a timer
    mov TimerID, eax

```

Затем мы создаем status bar, вызывая CreateStatusWindow. Этот вызов легко понять, поэтому я не буду комментировать его. После того, как status window создан, мы создаем таймер. В этом примере мы будем обновлять progress bar каждые 100 ms, поэтому нам нужно создать таймер.

```

SetTimer PROTO hWnd:DWORD, TimerID:DWORD, TimeInterval:DWORD, lpTimerProc:DWORD

```

hWnd : хэндл родительского окна

TimerID : не равный нулю идентификатор таймера. Вы можете создать свой собственный идентификатор.

TimeInterval : временной интервал в миллисекундах, который должен пройти, прежде чем таймер вызовет процедуру таймера или пошлет сообщение WM_TIMER.

lpTimeProc : адрес функции таймера, которая будет вызываться при истечении временного интервала. Если параметр равен нулю, таймер вместо этого будет посылать родительскому окну сообщение WM_TIMER.

Если вызов прошел успешно, функция возвратит TimerID. В противном случае, будет возвращен ноль. Вот почему идентификатор таймера не должен быть равен нулю.

```

.elseif uMsg==WM_TIMER

    invoke SendMessage,hwndProgress,PBM_STEPIT,0,0
    sub CurrentStep,10
    .if CurrentStep==0
        invoke KillTimer,hWnd,TimerID

        mov TimerID,0
        invoke SendMessage,hwndStatus,SB_SETTEXT,0,addr Message
        invoke MessageBox,hWnd,addr Message,addr AppName,\
            MB_OK+MB_ICONINFORMATION

        invoke SendMessage,hwndStatus,SB_SETTEXT,0,0
        invoke SendMessage,hwndProgress,PBM_SETPOS,0,0
    .endif

```

Когда истекает указанный временной интервал, таймер посылает сообщение WM_TIMER. Вы можете поместить здесь свой код, который будет выполнен. В данном примере, мы обновляем progress bar, а затем проверяем, было ли достигнуто максимальное значение. Если это так, мы убиваем таймер, после чего устанавливаем текст статус-окна с помощью сообщения SB_SETTEXT. Отображается message box, и когда юзер кликает ОК, мы очищаем текст в status bar'e и progress bar'e.

Win32 API. Урок 19. Tree View Control

Автор: lczelion, пер. Aquila

Дата: 2002-05-19

В этом tutorialе мы изучим как использовать контрол tree view. Более того, мы также узнаем как реализовать drag and drop для этого контрола и как использовать image list.

Скачайте пример здесь (находится в архиве wasm_files.zip: files/recovered/1001019/tut19.zip).

ТЕОРИЯ

Контрол tree view - это особый вид окна, который представляет объекты в иерархическом порядке. В качестве примера может случить левая панель Windows Explorer'a. Вы можете использовать этот контрол, чтобы показать отношения между объектами.

Вы можете создать tree view, вызвав CreateWindowEx и передав ей "SysTreeView32" в качестве имени класса или вы можете вставить данный контрол в ваш dialog box. Не забудьте поместить вызов InitCommonControls в ваш код.

Есть несколько стилей присущих только tree view. Вот наиболее часто используемые:

- TVS_HASBUTTONS - отображает кнопки плюс (+) и минус (-) перед родительским пунктом. Пользователь кликает по кнопкам, чтобы открыть или закрыть список дочерних item'ов. Чтобы вставить кнопки с пунктами в корень tree view, также должен быть указан TVS_LINESATROOT.
- TVS_HASLINES - используются линии для показа иерархии пунктов.
- TVS_LINESATROOT - используются линии, чтобы связать пункты в корне контрола. Этот стиль игнорируется, если не указан TVS_HASLINES.

Tree view, как и любой другой common control, взаимодействует с родительским окном с помощью сообщений. Родительское окно может посылать различные сообщения tree view, а тот может посылать "уведомительные" сообщения своему родительскому окну. В этом отношении tree view ничем не отличается от других окон.

Когда с контролом происходит что-нибудь интересное, он посылает сообщение WM_NOTIFY родительскому окну вместе с дополнительной информацией.

- wParam - ID контрола, но то, что оно будет уникальным не гарантируется, поэтому не используйте его. Вместо этого мы будем использовать hwndFrom или IDFrom из структуры NMHDR, на которую указывает lParam.
- lParam - указатель на структуру NMHDR. Некоторые контролы могут передавать указатель на большую структуру, но они должны иметь в качестве первого поля структуру NMHDR. Поэтому вы можете быть уверены, что lParam по крайней мере указывает на NMHDR.

Затем мы проанализируем структуру NMHDR.

```
NMHDR struct DWORD
    hwndFrom    DWORD ?
    idFrom      DWORD ?
    code        DWORD ?
NMHDR ends
```

hwndFrom - это хэндл окна контрола, который послал это сообщение.

idFrom - это ID этого контрола.

code - это настоящее сообщение, которое контрол хотел послать родительскому окну.

Уведомления от tree view начинаются с префикса TVN_.

Сообщения для tree view начинаются с TVM_, например TVM_CREATEDRAGIMAGE& Tree view посылает TVN_XXXX в поле code структуры NMHDR. Родительское окно может посылать TVM_XXXX контролю.

Добавление пунктов в tree view

После того, как вы создадите контрол tree view, вы можете добавить в него пункты. Вы можете сделать это, пошлав контролю TVM_INSERTITEM.

TVM_INSERTITEM

- wParam = 0;
- lParam = pointer to a TV_INSERTSTRUCT;

Вам следует знать кое-какую терминологию, касающуюся взаимоотношений между item'ами в tree view.

Item может быть родительским, дочерним или тем и другим одновременно. Родительский item - это такой item, с которым ассоциированы под-item'ы. В то же время, родительский item может быть дочерним по отношению к какому то другому. Item, у которого нет родителя, называется корнем (root). В tree view может быть много корневых элементов. Теперь мы проанализируем структуру TV_INSERTSTRUCT.

```
TV_INSERTSTRUCT STRUCT DWORD
    hParent      DWORD    ?
    hInsertAfter  DWORD    ?
    ITEMTYPE <>
TV_INSERTSTRUCT ENDS
```

hParent - хэндл родительского item'a. Если этот параметр равен TVI_ROOT или NULL, тогда item вставляется в корень tree view.

hInsertAfter - хэндл item'a, после которого будет вставляться новый item, или одно из следующих значений:

- TVI_FIRST - вставка элемента в начало списка.
- TVI_LAST - вставка элемента в конец списка.
- TVI_SORT - вставка элемента в список согласно алфавитному порядку.

```
ITEMTYPE UNION
    itemex TVITEMEX <>
    item TVITEM <>
ITEMTYPE ENDS
```

Мы будем использовать только TVITEM.

```
TV_ITEM STRUCT DWORD
    imask      DWORD    ?
    hItem      DWORD    ?
    state      DWORD    ?
    stateMask  DWORD    ?
    pszText    DWORD    ?
    cchTextMax  DWORD    ?
    iImage     DWORD    ?
    iSelectedImage  DWORD    ?
    cChildren  DWORD    ?
    lParam     DWORD    ?
TV_ITEM ENDS
```

Эта структура используется для отсылки и получения информации об элементе tree view (в зависимости от сообщений). Например, с помощью TVM_INSERTITEM, она используется для указания атрибутов item'a, который должен быть вставлен в tree view. С помощью TVM_GETITEM,

она будет заполнена информацией о выбранном элементе tree view.

imask используется для указания, какой член структуры TV_ITEM верен. Например, если значение в imask равно TVIF_TEXT, оно означает, что только pszText верно. Вы можете комбинировать несколько флагов вместе.

hItem - это хэндл элемента tree view. Каждый item имеет хэндл, как и в случае с окнами. Если вы хотите сделать что-нибудь с item'ом, вы должны выбрать его с помощью его хэнкла.

pszText - это указатель на строку, оканчивающуюся NULL'ом, которая является названием элемента tree view.

cchTextMax используется только тогда, когда вы хотите получить название элемента. Windows надо будет знать размер предоставленного вами буфера (pszText), поэтому этот элемент используется именно для этого.

image и iSelectedImage содержат индекс из image list'a, который содержит изображения, показываемые когда элемент выбран и не выбран. Если вспомните левую панель Windows Explorer'a, то изображения директорий задаются именно этими двумя параметрами.

Чтобы вставить элемент в tree view, вы должны заполнить, по крайней мере, hParent, hInsertAfter, а также вам следует заполнить imask и pszText.

Добавление изображений в tree view

Если вы хотите поместить изображение слева от названия элемента, вам следует создать image list и ассоциировать его с контролом tree view.

```
ImageList_Create PROTO cx:DWORD, cy:DWORD, flags:DWORD, \
                  cInitial:DWORD, cGrow:DWORD
```

Если вызов пройдет успешно, функция возвратит хэндл на пустой image list.

cx - ширина любого изображения в этом image list'e в пикселях.

cy - высота любого изображения в этом image list'e в пикселях. Все изображения в image list'e должно быть равны друг другу по размеру. Если вы укажете больший bitmap, Windows разрежет его на несколько кусков согласно значению в cx и cy. Поэтому вам следует тщательно подготовить необходимые изображения.

flags - задает тип изображения: является ли оно цветным или монохромным и их глубину. Проконсультируйтесь с вашим справочником по Win32 API.

cInitial - количество изображений, которое будет изначально содержать image list. Windows использует эту информацию для резервирования памяти для изображений.

cGrow - количество изображений, на которое должен увеличиваться image list, когда системе необходимо изменить размер списка, чтобы выделить место для новых изображений. Этот параметр представляет количество новых изображений, которое может содержать image list, изменивший размер.

Image list - это не окно! Это только хранилище изображений, которые будут использоваться другими окнами.

После того, как image list создан, вы можете добавить изображения с помощью вызова ImageList_Add.

```
ImageList_Add PROTO hIm1:DWORD, hbmImage:DWORD, hbmMask:DWORD
```

Если во время вызова произойдет какая-либо ошибка, будет возвращен -1.

hIm1 - хэндл image list'a, в который вы хотите добавить изображения. Это значение возвращается ImageList_Create.

hbmImage - хэнгл битмапа, который должен быть добавлен в image list. Обычно изображения задаются в ресурсах и вызываются с помощью LoadBitmap.

Заметьте, что вам не надо указывать количество изображений, содержащихся в этом bitmap'e, потому что это вытекает из параметров cx и cy, переданных ImageList_Create.

hbmMask - хэнгл битмапа, в котором содержится маска. Если маска в image list'e не используется, этот параметр игнорируется.

Обычно мы будем добавлять только два изображения в image list, который будет использоваться контролем tree view: одно для невыбранного элемента, а другое - для выбранного.

Когда image list готов, мы ассоциируем его с tree view, посылая тому сообщение TVM_SETIMAGELIST:

- wParam - тип image list'a. Есть две возможности:
- TMSIL_NORMAL - задает обычный image list, который содержит изображения выбранного и невыбранного элементов.
- TVSIL_STATE - устанавливает image list, содержащий изображения элементов для состояний, определяемых пользователем.
- lParam - хэнгл image list'a.

Получение информации о элементе tree view

Вы можете получить информацию об элементе tree view, послав ей сообщение TVM_GETITEM:

- wParam = 0
- lParam = pointer to the TV_ITEM structure to be filled with the information

Прежде, чем вы пошлете это сообщение, вы должны заполнить параметр imask флагами, которые укажут, какие из полей TV_ITEM должны быть заполнены Windows. А самое главное, вы должны заполнить hItem хэнглом элемента, о котором вы хотите получить информацию. И это порождает следующую проблему: где взять этот хэнгл? Надо ли вам сохранять все хэнглы tree view?

Ответ достаточно прост: вам не надо этого делать. Вы можете послать сообщение TVM_GETNEXTITEM контролю tree view, чтобы получить хэнгл элемента tree view, который имеет указанные вами атрибуты. Например, вы можете получить хэнгл первого дочернего элемента, корневого элемента, выбранного элемента и так далее.

TVM_GETNEXTITEM:

- wParam = флаг
- lParam - хэнгл на элемент tree view (не всегда необходим)

Значение wParam очень важно, поэтому я привожу ниже все возможные флаги:

- TVGN_CARET - получение хэнгла выбранного элемента.
- TVGN_CHILD - получение хэнгла первого дочернего элемента по отношению к item'у, чей хэнгл указан в параметре hItem.
- TVGN_DROPHILITE - получение хэнгла item'a, который является целью операции drag-and-drop.
- TVGN_FIRSTVISIBLE - получение хэнгла первого видимого item'a.
- TVGN_NEXT - получение хэнгла следующего родственного элемента.
- TVGN_NEXTVISIBLE - получение хэнгла следующего видимого элемента, который следует за указанным item'ом. Указанный элемент должен быть видимым. Используйте сообщение TVM_GETITEMRECT, чтобы определить, является ли item видимым.
- TVGN_PARENT - получение хэнгла указанного родительского элемента по отношению к указанному.
- TVGN_PREVIOUS - получение хэнгла предыдущего родственного элемента.

- TVGN_PREVIOUSVISIBLE - получение хэндла первого видимого элемента, который предшествует указанному item'у, который должен быть видимым. Используйте сообщение TVM_GETITEMRECT, чтобы определить, является ли item видимым.
- TVGN_ROOT - получает хэндл самого первого из корневых элементов tree view.

Вы можете видеть, что вы можете получить хэндл интересующего вас сообщения с помощью этого сообщения. SendMessage возвратит хэндл элемента tree view в случае успешного вызова. Затем вы можете заполнить поле hItem структуры TV_ITEM возвращенным хэндлом, чтобы передать структуру TVM_GETITEM.

Операции Drag-and-Drop над контролом tree view

Именно из-за этой части я написал этот tutorial. Когда я попытался следовать примеру из справочника по Win32 API (win32.hlp от Inprise), я был сильно обескуражен отсутствием жизненно важной информации. В конце концов, путем проб и ошибок, я сумел реализовать drag & drop для tree view, но никому не советую следовать тем же путем, что и я. Ниже изложены правильные действия.

Когда пользователь пытается перетащить элемент, tree view посылает уведомление TVN_BEGINDRAG родительскому окну. Вы можете использовать эту возможность для создания специального изображения, которое будет представлять элемент, когда его тащат. Вы можете послать tree view сообщение TVM_CREATEDRAGIMAGE, чтобы сказать тому создать такое изображение по умолчанию из изображения, используемое в настоящее время элементом, который будет перетащен. Tree view создаст image list с одним drag-изображением и возвратит хэндл этого image list'a вам.

После того, как drag-изображение создано, вы указываете его "горячую точку", вызывая ImageList_BeginDrag.

```
ImageList_BeginDrag PROTO himlTrack:DWORD, \
                        iTrack:DWORD, \
                        dxHotspot:DWORD, \
                        dyHotspot:DWORD
```

- himlTrack - это хэндл image list'a, который содержит drag-изображение.
- iTrack - это индекс элемента image list'a, который будет являться drag-изображением.
- dxHotspot указывает относительную горизонтальную координату "горячей точки" (которая нам необходима, так как мы будем использовать drag-изображение вместо курсора мыши. У стандартного курсора "горячая точка" находится на кончике стрелки).
- dyHotspot указывает относительную вертикальную координату "горячей точки".
- Как правило, iTrack будет равен 0, если вам нужно сказать tree view, чтобы тот создал для вас drag-изображение. dxHotspot и dyHotspot могут быть равными 0, если вы хотите, чтобы левый верхний угол drag-изображения был "горячей точкой".

Когда drag-изображение готово, мы вызываем ImageList_DragEnter, чтобы отобразить его в окне.

```
ImageList_DragEnter PROTO hwndLock:DWORD, x:DWORD, y:DWORD
```

hwndLock - это хэндл окна, которому принадлежит drag-изображение. Drag-изображение нельзя будет двигать за пределы этого окна.

x и y - это x- и y-координата места, где drag-изображение должно быть отображено сначала. Заметьте, что эти значения задаются по отношению к левому верхнему углу окна, а не клиентской области.

Теперь, когда drag-изображение отображено в окне, вам следует поддерживать операцию перетаскивания в контроле tree view. Тем не менее, здесь появляется небольшая проблема. Мы должны отслеживать путь перетаскивания с помощью WM_MOUSEMOVE и позицию сброса (drop) с помощью WM_LBUTTONUP. Однако, если drag-изображение находится над каким-нибудь

дочерним окном, родительское окно никогда не получит никаких сообщений от мыши. Решение состоит в том, чтобы взять контроль на сообщениями от мыши с помощью SetCapture. Эта функция позволяет направить мышинные сообщения напрямую определенному окну, вне зависимости от того, где находится курсор мыши.

Внутри обработчика WM_MOUSEMOVE, вы будете отслеживать drag-путь с помощью вызова ImageList_DragMove. Эта функция передвигает изображение относительно пути переноса. Более того, если вы захотите, вы можете подсвечивать элемент, над которым находится drag-изображение, посылая сообщение TVM_HITTEST, проверяя, находится ли изображение над каким-нибудь элементом. Если это так, вы можете послать TVM_SELECTITEM с флагом TVGN_DROPHILITE, чтобы подсветить элемент. Заметьте, что прежде, чем послать сообщение TVM_SELECTITEM, вы должны спрятать drag-изображение или оно будет оставлять уродливый след. Это можно сделать, вызвав ImageList_DragShowNolock, а после того, как элемент будет подсвечен, необходимо вызвать ImageList_DragShowNolock, чтобы снова отобразить drag-изображение.

Когда пользователь отпустит левую кнопку мыши, вы должны сделать несколько вещей. Если вы подсветили элемент, вам нужно перевести его в обычное состояние, снова послав TVM_SELECTITEM с флагом TVGN_DROPHILITE, но в этот раз IParam должен быть равен нулю. Затем вы должны вызвать ImageList_DragLeave, за которым должен следовать вызов ImageList_EndDrag. Вы должны освободить мышь с помощью ReleaseCapture. Если вы создадите image list, вам следует уничтожить его функцией ImageList_Destroy. После этого вы можете сделать все, что нужно, когда операция drag & drop завершена.

ПРИМЕР

```
.386
.model flat,stdcall
option casemap:none

include \masm32\include\windows.inc
include \masm32\include\user32.inc
include \masm32\include\kernel32.inc
include \masm32\include\comctl32.inc

include \masm32\include\gdi32.inc
includelib \masm32\lib\gdi32.lib
includelib \masm32\lib\comctl32.lib
includelib \masm32\lib\user32.lib

includelib \masm32\lib\kernel32.lib

WinMain PROTO :DWORD,:DWORD,:DWORD,:DWORD

.const
IDB_TREE equ 4006 ; ID битмапового ресурса
.data
ClassName db "TreeViewWinClass",0

AppName db "Tree View Demo",0
TreeViewClass db "SysTreeView32",0
Parent db "Parent Item",0
Child1 db "child1",0

Child2 db "child2",0
DragMode dd FALSE ; флаг, который определяет, находимся
; ли мы в режиме переноса

.data?
hInstance HINSTANCE ?
hwndTreeView dd ? ; хэндл контрола tree view

hParent dd ? ; хэндл корневого элемента
hImageList dd ? ; хэндл image list'a, который будет
```

```

                                ; использовать tree view
hDragImageList dd ?          ; хэндл image list'a, в котором будет
                                ; храниться drag-изображение

.code

start:
    invoke GetModuleHandle, NULL
    mov     hInstance,eax
    invoke WinMain, hInstance,NULL,NULL, SW_SHOWDEFAULT

    invoke ExitProcess,eax
    invoke InitCommonControls

WinMain proc
hInst:HINSTANCE,hPrevInst:HINSTANCE,CmdLine:LPSTR,CmdShow:DWORD
    LOCAL wc:WNDCLASSEX
    LOCAL msg:MSG

    LOCAL hwnd:HWND
    mov     wc.cbSize,SIZEOF WNDCLASSEX
    mov     wc.style, CS_HREDRAW or CS_VREDRAW
    mov     wc.lpfnWndProc, OFFSET WndProc

    mov     wc.cbClsExtra,NULL
    mov     wc.cbWndExtra,NULL
    push    hInst
    pop     wc.hInstance

    mov     wc.hbrBackground,COLOR_APPWORKSPACE
    mov     wc.lpszMenuName,NULL
    mov     wc.lpszClassName,OFFSET ClassName
    invoke LoadIcon,NULL,IDI_APPLICATION

    mov     wc.hIcon,eax
    mov     wc.hIconSm,eax
    invoke LoadCursor,NULL,IDC_ARROW
    mov     wc.hCursor,eax

    invoke RegisterClassEx, addr wc
    invoke CreateWindowEx,WS_EX_CLIENTEDGE,ADDR ClassName,ADDR AppName,\
        WS_OVERLAPPED+WS_CAPTION+WS_SYSMENU+WS_MINIMIZEBOX+\
        WS_MAXIMIZEBOX+WS_VISIBLE, \
        CW_USEDEFAULT,200,400,NULL,NULL,\
        hInst,NULL
    mov     hwnd,eax
    .while TRUE

        invoke GetMessage, ADDR msg,NULL,0,0
        .BREAK .IF (!eax)
        invoke TranslateMessage, ADDR msg
        invoke DispatchMessage, ADDR msg

    .endw
    mov     eax,msg.wParam
    ret
WinMain endp

WndProc proc uses edi hwnd:HWND, uMsg:UINT, wParam:WPARAM, lParam:LPARAM
    LOCAL tvinsert:TV_INSERTSTRUCT

    LOCAL hBitmap:DWORD
    LOCAL tvhit:TV_HITTESTINFO
    .if uMsg==WM_CREATE
        invoke CreateWindowEx,NULL,ADDR TreeViewClass,NULL,\
            WS_CHILD+WS_VISIBLE+TVS_HASLINES+TVS_HASBUTTONS+TVS_LINESATROOT,0,\
            0,200,400,hwnd,NULL,\
            hInstance,NULL          ; Создание tree view
        mov     hwndTreeView,eax
    
```

```

invoke ImageList_Create,16,16,ILC_COLOR16,2,10    ; Создание
                                                ; ассоциированного с ним image list'a
mov hImageList,eax

invoke LoadBitmap,hInstance,IDB_TREE ; загрузка bitmap'a из ресурса
mov hBitmap,eax
invoke ImageList_Add,hImageList,hBitmap,NULL ; Добавление bitmap'a
                                                ; в image list
invoke DeleteObject,hBitmap    ; всегда удаляйте ненужный bitmap

invoke SendMessage,hwndTreeView,TVM_SETIMAGELIST,0,hImageList
mov tvinsert.hParent,NULL

mov tvinsert.hInsertAfter,TVI_ROOT
mov tvinsert.item.imask,TVIF_TEXT+TVIF_IMAGE+TVIF_SELECTEDIMAGE
mov tvinsert.item.pszText,offset Parent
mov tvinsert.item.iImage,0

mov tvinsert.item.iSelectedImage,1
invoke SendMessage,hwndTreeView,TVM_INSERTITEM,0,addr tvinsert
mov hParent,eax
mov tvinsert.hParent,eax

mov tvinsert.hInsertAfter,TVI_LAST
mov tvinsert.item.pszText,offset Child1
invoke SendMessage,hwndTreeView,TVM_INSERTITEM,0,addr tvinsert
mov tvinsert.item.pszText,offset Child2

invoke SendMessage,hwndTreeView,TVM_INSERTITEM,0,addr tvinsert
.elseif uMsg==WM_MOUSEMOVE
    .if DragMode==TRUE
        mov eax,lParam

        and eax,0ffffh
        mov ecx,lParam
        shr ecx,16
        mov tvhit.pt.x,eax

        mov tvhit.pt.y,ecx
        invoke ImageList_DragMove,eax,ecx
        invoke ImageList_DragShowNolock,FALSE
        invoke SendMessage,hwndTreeView,TVM_HITTEST,NULL,addr tvhit

        .if eax!=NULL
            invoke SendMessage,hwndTreeView,TVM_SELECTITEM,\
                                TVGN_DROPHILITE,eax
        .endif

        invoke ImageList_DragShowNolock,TRUE
    .endif
.elseif uMsg==WM_LBUTTONDOWN
    .if DragMode==TRUE

        invoke ImageList_DragLeave,hwndTreeView
        invoke ImageList_EndDrag
        invoke ImageList_Destroy,hDragImageList
        invoke

SendMessage,hwndTreeView,TVM_GETNEXTITEM,TVGN_DROPHILITE,0
        invoke SendMessage,hwndTreeView,TVM_SELECTITEM,TVGN_CARET,eax
        invoke SendMessage,hwndTreeView,TVM_SELECTITEM,TVGN_DROPHILITE,0

        invoke ReleaseCapture
        mov DragMode,FALSE
    .endif
.elseif uMsg==WM_NOTIFY
    mov edi,lParam
    assume edi:ptr NM_TREEVIEW
    .if [edi].hdr.code==TVN_BEGINDRAG
        invoke

```

```

SendMessage, hwndTreeView, TVM_CREATEDRAGIMAGE, 0, [edi].itemNew.hItem
    mov hDragImageList, eax
    invoke ImageList_BeginDrag, hDragImageList, 0, 0, 0
    invoke

ImageList_DragEnter, hwndTreeView, [edi].ptDrag.x, [edi].ptDrag.y
    invoke SetCapture, hWnd
    mov DragMode, TRUE
    .endif

    assume edi:nothing
    .elseif uMsg==WM_DESTROY
        invoke PostQuitMessage, NULL
    .else

        invoke DefWindowProc, hWnd, uMsg, wParam, lParam
        ret
    .endif
    xor eax, eax

    ret
WndProc endp
end start

```

АНАЛИЗ

Внутри обработчика WM_CREATE вы создаете контрол tree view.

```

invoke CreateWindowEx, NULL, ADDR TreeViewClass, NULL, \
    WS_CHILD+WS_VISIBLE+TVS_HASLINES+TVS_HASBUTTONS+\
    TVS_LINESATROOT, 0, \
    0, 200, 400, hWnd, NULL, \
    hInstance, NULL

```

Обратите внимание на стили. TVS_XXXX - это стили, присущие tree view.

```

invoke ImageList_Create, 16, 16, ILC_COLOR16, 2, 10
mov hImageList, eax
invoke LoadBitmap, hInstance, IDB_TREE
mov hBitmap, eax
invoke ImageList_Add, hImageList, hBitmap, NULL
invoke DeleteObject, hBitmap
invoke SendMessage, hwndTreeView, TVM_SETIMAGELIST, 0, hImageList

```

Затем вы создаете пустой image list, который будет принимать изображения размером 16x16 пикселей и с глубиной цвета 16 бит. Вначале он будет содержать 2 изображения, но будет расширен до 10, если это потребуется. Далее мы загружаем bitmap из ресурса и добавляем его в только что созданный image list. После этого мы удаляем хэндл битмара, так как он больше нам не нужен. Как только image list готов, мы ассоциируем его с tree view, посылая ему TVM_SETIMAGELIST.

```

mov tvinsert.hParent, NULL
mov tvinsert.hInsertAfter, TVI_ROOT
mov tvinsert.u.item.imask, TVIF_TEXT+TVIF_IMAGE+TVIF_SELECTEDIMAGE
mov tvinsert.u.item.pszText, offset Parent
mov tvinsert.u.item.iImage, 0
mov tvinsert.u.item.iSelectedImage, 1
invoke SendMessage, hwndTreeView, TVM_INSERTITEM, 0, addr tvinsert

```

Мы вставляем элементы в контрол tree view, начиная с корневого элемента. Так как это будет корневой item, параметр hParent равен NULL, а hInsertAfter - TVI_ROOT. imask указывает, что pszText, iImage и iSelectedImage структуры TV_ITEM верны. Мы заполняем эти три параметра соответствующими значениями. pszText содержит название корневого элемента, iImage - это индекс изображения в image list'e, который будет отобразиться слева от невыбранного элемента, а iSelectedImage - индекс изображения выбранного элемента. Когда все требуемые параметры заполнены, мы посылаем сообщение TVM_INSERTITEM контролу tree view, чтобы добавить в него корневой элемент.

```

mov hParent,eax
mov tvinsert.hParent,eax
mov tvinsert.hInsertAfter,TVI_LAST
mov tvinsert.u.item.pszText,offset Child1
invoke SendMessage,hwndTreeView,TVM_INSERTITEM,0,addr tvinsert
mov tvinsert.u.item.pszText,offset Child2
invoke SendMessage,hwndTreeView,TVM_INSERTITEM,0,addr tvinsert

```

После этого мы добавляем дочерние элементы. hParent теперь заполнен хэндлом родительского элемента. Мы будем использовать те же изображения, поэтому не меняем ilImage и iSelectedImage.

```

.elseif uMsg==WM_NOTIFY
mov edi,lParam
assume edi:ptr NM_TREEVIEW
.if [edi].hdr.code==TVN_BEGINDRAG
invoke SendMessage,hwndTreeView,TVM_CREATEDRAGIMAGE,\
    0,[edi].itemNew.hItem
mov hDragImageList,eax
invoke ImageList_BeginDrag,hDragImageList,0,0,0

invoke ImageList_DragEnter,hwndTreeView,[edi].ptDrag.x,\
    [edi].ptDrag.y
invoke SetCapture,hWnd
mov DragMode,TRUE

.endif
assume edi:nothing

```

Теперь, когда юзер попытается перетащить item, tree view пошлет сообщение WM_NOTIFY с кодом TVN_BEGINDRAG. lParam - это указатель на структуру NM_TREEVIEW, которая содержит некоторую информацию, которая необходима нам, поэтому мы помещаем значение lParam в edi и используем edi как указатель на структуру NM_TREEVIEW. 'assume edi:ptr NM_TREEVIEW' указывает MASM'у, что edi - это указатель на структуру NM_TREEVIEW. Затем мы создаем drag-изображение, посылая TVM_CREATEDRAGIMAGE tree view. Сообщение возвращает хэндл на созданный image list, внутри которого содержится drag-изображение. Мы вызываем ImageList_BeginDrag, чтобы установить его "горячую точку". После этого начинаем операцию переноса с помощью ImageList_DragEnter. Эта функция отображает drag-изображение в указанном месте заданного окна.

Мы используем структуру ptDrag, которая является членом структуры NM_TREEVIEW в качестве точки, в которой должно быть показано drag-изображение. Затем перехватываем мышь и устанавливаем флаг, который показывает, что мы находимся в drag-режиме.

```

.elseif uMsg==WM_MOUSEMOVE
.if DragMode==TRUE
mov eax,lParam
and eax,0ffffh
mov ecx,lParam
shr ecx,16
mov tvhit.pt.x,eax
mov tvhit.pt.y,ecx
invoke ImageList_DragMove,eax,ecx
invoke ImageList_DragShowNolock,FALSE
invoke SendMessage,hwndTreeView,TVM_HITTEST,NULL,addr tvhit
.if eax!=NULL
invoke SendMessage,hwndTreeView,TVM_SELECTITEM,TVGN_DROPHILITE,eax
.endif
invoke ImageList_DragShowNolock,TRUE
.endif

```

Теперь мы концентрируемся на WM_MOUSEMOVE. Когда пользователь перетаскивает drag-изображение, наше родительское окно получает сообщения WM_MOUSEMOVE. В ответ на них мы обновляем позицию drag-изображения функцией ImageList_DragMove, после чего проверяем, не находится ли оно над каким-нибудь элементом с помощью сообщения

TVM_HITTEST с указанием координаты проверяемой точки. Если drag-изображение находится над каким-либо элементом, тот подсвечивается сообщением TVM_SELECTITEM с флагом TVGN_DROPHILITE. Во время операции подсветки мы прячем drag-изображение, чтобы не было лишних глюков.

```
.elseif uMsg==WM_LBUTTONUP
    .if DragMode==TRUE
        invoke ImageList_DragLeave,hwndTreeView
        invoke ImageList_EndDrag
        invoke ImageList_Destroy,hDragImageList
        invoke SendMessage,hwndTreeView,TVM_GETNEXTITEM,TVGN_DROPHILITE,0
        invoke SendMessage,hwndTreeView,TVM_SELECTITEM,TVGN_CARET,eax
        invoke SendMessage,hwndTreeView,TVM_SELECTITEM,TVGN_DROPHILITE,0
        invoke ReleaseCapture
        mov DragMode,FALSE
    .endif
```

Когда пользователь отпускает левую кнопку мыши, операция переноса закончена. Мы выходим из drag-режима, последовательно вызывая функции ImageList_DragLeave, ImageList_EndDrag и ImageList_Destroy. Также мы проверяем последний подсвеченный элемент и выбираем его. Мы также должны убрать его подсветку, иначе другие элементы не будут подсвечиваться, когда их будут выбирать. И наконец, мы убираем перехват сообщений от мыши.

Win32 API. Урок 20. Сабклассинг окна

Автор: lczelion, пер. Aquila

Дата: Не указана

В этом tutorialе мы изучим сабклассинг окна, что это такое, и как это использовать нам на пользу.

Скачайте пример здесь (находится в архиве wasm_files.zip: files/recovered/1001020/tut20.zip).

ТЕОРИЯ

Если вы уже некоторое время программируете в Windows, вы уже могли столкнуться с ситуацией, когда окно имеет почти все атрибуты, которые вам нужны, но не все. Сталкивались ли вы с ситуацией, когда вам требуется специальный вид edit control'a, который бы отфильтровывал ненужный текст? Первое, что может прийти в голову, это написать свое собственное окно. Но это действительно тяжелая работа, требующая значительного времени. Выходом является сабклассинг окна.

Вкратце, сабклассинг окна позволяет получить контроль над сабклассированным окном. У вас будет абсолютный контроль над ним. Давайте рассмотрим пример, что прояснит данное утверждение. Предположите, что вам нужен text box, в котором можно вводить только шестнадцатеричные числа. Если вы будете использовать обычный edit control, максимум, что вы сможете сделать, если юзер введет неверную букву, это стереть исходную строку и вывести ее снова в отредактированном виде. По меньшей мере, это непрофессионально. Фактически вам требуется получить возможность проверять каждый символ, который юзер набирает в text box'e, как раз в тот момент, когда он делает это.

Теперь мы изучим как это сделать. Когда пользователь печатает что-то в text box'e, Windows посылает сообщение WM_CHAR процедуре edit control'a. Эта процедура окна находится внутри Windows, поэтому мы не можем модифицировать ее. Но мы можем перенаправить поток сообщений к нашей оконной процедуре. Поэтому наша процедура окна первой получит возможность обработать сообщение, которое Windows пошлет edit control'у. Если наша процедура решит обработать сообщение, она так и сделает. Но если она не захочет его обрабатывать, она может передать его оригинальной оконной процедуре. Таким образом, наша функция будет стоять между Windows и edit control'ом. Посмотрите на условную схему внизу.

До сабклассинга

Windows ==> процедура edit control'a

После сабклассинга

Windows ==> наша оконная процедура ----> процедура edit control'a

Теперь мы можем рассмотреть то, каким образом происходит сабклассинг окна. Заметьте, что сабклассинг неограничивается контролами, он может использоваться с любым окном. Давайте подумаем о том, как Windows узнает, где находится процедура edit box'a. Ну?.. Поле lpfnWndProc в структуре WNDCLASSEX. Если мы сможем поменять значение этого поля на адрес собственной структуры, Windows пошлет сообщение нашей процедуре окна вместо этого. Мы можем сделать это, вызвав SetWindowLong.

SetWindowLong(hWnd:HWND, nIndex:DWORD, dwNewLong:DWORD)

hWnd = хэндл окна, чьи свойства мы хотим поменять.

nIndex = значение, которое нужно изменить.

GWL_EXSTYLE Установка нового расширенного стиля окна.

GWL_STYLE Установка нового стиля окна.
 GWL_WNDPROC Установка нового адреса для процедуры окна.
 GWL_HINSTANCE Установка нового хэндла приложения.
 GWL_ID Установка нового идентификатора окна.
 GWL_USERDATA Установка 32-битного значения, ассоциирующегося с окном.
 У каждого окна есть ассоциированное с ним 32-битное значение,
 предназначенное для использования приложением в своих целях.

dwNewLong = новое значение.

Таким образом, наша работа проста: мы создаем процедуру окна, которая будет обрабатывать сообщения для edit control'a и затем вызывать SetWindowLong с флагом GWL_WNDPROC, которому передается адрес нашего окна в качестве третьего параметра. В случае, если вызов функции прошел нормально, возвращаемым значением является прежнее значение замещаемого параметра, в нашем случае - это адрес оригинальной процедуры окна. Нам нужно сохранить это значение, чтобы использовать его внутри нашей процедуры.

Помните, что есть сообщения, которые нам не нужно будет обрабатывать. Их мы будем передавать оригинальной процедуре. Мы можем сделать это с помощью вызова функции CallWindowProc.

```

CallWindowProc Proto lpPrevWndFunc:DWORD, \

                                hWnd:DWORD, \
                                Msg:DWORD, \
                                wParam:DWORD, \
                                lParam:DWORD
  
```

lpPrevWndFunc = адрес оригинальной процедуры окна. Остальные четыре значения - это те, что передаются нашей процедуре окна. Мы передаем их CallWindowProc.

Пример:

```

.386
.model flat,stdcall
option casemap:none
include \masm32\include\windows.inc

include \masm32\include\user32.inc
include \masm32\include\kernel32.inc
include \masm32\include\comctl32.inc
includelib \masm32\lib\comctl32.lib

includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib

WinMain Proto :DWORD,:DWORD,:DWORD,:DWORD
EditWndProc Proto :DWORD,:DWORD,:DWORD,:DWORD

.data
ClassName db "SubclassWinClass",0
AppName db "Subclassing Demo",0
EditClass db "EDIT",0

Message db "You pressed Enter in the text box!",0

.data?

hInstance HINSTANCE ?
hwndEdit dd ?
OldWndProc dd ?

.code
start:
    invoke GetModuleHandle, NULL

    mov     hInstance, eax
  
```



```

    invoke WinMain, hInstance,NULL,NULL, SW_SHOWDEFAULT
    invoke ExitProcess,eax

WinMain proc
hInst:HINSTANCE,hPrevInst:HINSTANCE,CmdLine:LPSTR,CmdShow:DWORD
    LOCAL wc:WNDCLASSEX

    LOCAL msg:MSG
    LOCAL hwnd:HWND
    mov     wc.cbSize,SIZEOF WNDCLASSEX
    mov     wc.style, CS_HREDRAW or CS_VREDRAW

    mov     wc.lpfnWndProc, OFFSET WndProc
    mov     wc.cbClsExtra,NULL
    mov     wc.cbWndExtra,NULL
    push    hInst

    pop     wc.hInstance
    mov     wc.hbrBackground,COLOR_APPWORKSPACE
    mov     wc.lpszMenuName,NULL
    mov     wc.lpszClassName,OFFSET ClassName

    invoke LoadIcon,NULL,IDI_APPLICATION
    mov     wc.hIcon,eax
    mov     wc.hIconSm,eax
    invoke LoadCursor,NULL,IDC_ARROW

    mov     wc.hCursor,eax
    invoke RegisterClassEx, addr wc
    invoke CreateWindowEx,WS_EX_CLIENTEDGE,ADDR ClassName,ADDR AppName,\
WS_OVERLAPPED+WS_CAPTION+WS_SYSMENU+WS_MINIMIZEBOX+WS_MAXIMIZEBOX+WS_VISIBLE

    CW_USEDEFAULT,350,200,NULL,NULL,\
    hInst,NULL
    mov     hwnd,eax
    .while TRUE

        invoke GetMessage, ADDR msg,NULL,0,0
        .BREAK .IF (!eax)
        invoke TranslateMessage, ADDR msg
        invoke DispatchMessage, ADDR msg

    .endw
    mov     eax,msg.wParam
    ret
WinMain endp

WndProc proc hwnd:HWND, uMsg:UINT, wParam:WPARAM, lParam:LPARAM
    .if uMsg==WM_CREATE

        invoke CreateWindowEx,WS_EX_CLIENTEDGE,ADDR EditClass,NULL,\
        WS_CHILD+WS_VISIBLE+WS_BORDER,20,\
        20,300,25,hwnd,NULL,\
        hInstance,NULL

        mov     hwndEdit,eax
        invoke SetFocus,eax
        ;-----
        ; Subclass it!

        ;-----
        invoke SetWindowLong,hwndEdit,GWL_WNDPROC,addr EditWndProc
        mov     OldWndProc,eax
    .elseif uMsg==WM_DESTROY

        invoke PostQuitMessage,NULL
    .else
        invoke DefWindowProc,hwnd,uMsg,wParam,lParam
    .endif
    ret
WndProc endp

```

```

        ret

    .endif
    xor eax,eax
    ret
WndProc endp

EditWndProc PROC hEdit:DWORD,uMsg:DWORD,wParam:DWORD,lParam:DWORD
    .if uMsg==WM_CHAR

        mov eax,wParam
        .if (al>="0" && al<="9") || (al>="A" && al<="F") || (al>="a" && al<="f") || al==VK_BACK
            .if al>="a" && al<="f"

                sub al,20h
            .endif
            invoke CallWindowProc,OldWndProc,hEdit,uMsg,eax,lParam
            ret

        .endif
    .elseif uMsg==WM_KEYDOWN
        mov eax,wParam
        .if al==VK_RETURN

            invoke MessageBox,hEdit,addr Message,addr
AppName,MB_OK+MB_ICONINFORMATION
            invoke SetFocus,hEdit
        .else

            invoke CallWindowProc,OldWndProc,hEdit,uMsg,wParam,lParam
            ret
        .endif
    .else

        invoke CallWindowProc,OldWndProc,hEdit,uMsg,wParam,lParam
        ret
    .endif
    xor eax,eax

    ret
EditWndProc endp
end start

```

АНАЛИЗ

```

    invoke SetWindowLong,hwndEdit,GWL_WNDPROC,addr EditWndProc
    mov OldWndProc,eax

```

После того, как edit control создан, мы сабклассим его, вызывая SetWindowLong и замещая адрес оригинальной процедуры окна нашим собственным адресом. Заметьте, что мы сохраняем значение адреса оригинальной процедуры, чтобы впоследствии использовать его при вызове CallWindowProc. Заметьте, что EditWndProc - это обычная оконная процедура.

```

    .if uMsg==WM_CHAR
        mov eax,wParam
        .if (al>="0" && al<="9") || (al>="A" && al<="F") || (al>="a" && al<="f") || al==VK_BACK
            .if al>="a" && al<="f"
                sub al,20h
            .endif
            invoke CallWindowProc,OldWndProc,hEdit,uMsg,eax,lParam

            ret
        .endif
    .endif

```

Внутри EditWndProc, мы фильтруем сообщения WM_CHAR. Если введен символ в диапазоне 0-9 или a-f, мы передаем его оригинальной процедуре окна. Если это символ нижнего регистра, мы конвертируем его в верхний, добавляя 20h. Заметьте, что если символ не тот, который мы ожидали, мы пропускаем его. Мы не передаем его оригинальной процедуре окна. Поэтому, когда

пользователь печатает что-нибудь отличное от 0-9 или a-f, символ не появляется в edit control'e.

```
.elseif uMsg==WM_KEYDOWN
    mov eax,wParam
    .if al==VK_RETURN

        invoke MessageBox,hEdit,addr Message,addr
        AppName,MB_OK+MB_ICONINFORMATION
        invoke SetFocus,hEdit
    .else

        invoke CallWindowProc,OldWndProc,hEdit,uMsg,wParam,lParam
        ret
    .end
```

Я хочу продемонстрировать силу сабклассинга через перехват клавиши Enter. EditWndProc проверяет сообщение WM_KEYDOWN, не равно ли оно VK_RETURN (клавиша Enter). Если это так, она отображает окно с сообщением "You pressed the Enter key in the text box!". Если это не клавиша Enter, она передает сообщение оригинальной процедуре.

Вы можете использовать сабклассинг окна, чтобы получить контроль над другими окнами. Эту мощную технику вам следует иметь в своем арсенале.

Win32 API. Урок 21. Пайп

Автор: lczelion, пер. Aquila

Дата: 2002-05-21

В этом tutorialе мы исследуем пайп (pipe) , что это такое и для чего мы можем использовать его. Чтобы сделать этот процесс более интересным, я покажу, как можно изменить бэкграунд и цвет текста edit control'a.

Скачайте пример здесь (находится в архиве wasm_files.zip: files/recovered/1001021/tut21.zip).

ТЕОРИЯ

Пайп - канал или дорога с двумя концами. Вы можете использовать пайп, чтобы обмениваться данными между двумя различными процессами или внутри одного процесса. Это что-то вроде "уоки-токи". Вы даете другому участнику конец канала и он может использовать его для того, чтобы взаимодействовать с вами.

Есть два типа пайпов: анонимные и именованные. Анонимный пайп анонимен - вы можете использовать его не зная его имени. Для того, чтобы использовать именованный пайп, вам обязательно нужно знать его имя.

Вы можете разделить пайп по их свойствам: однонаправленные и двунаправленные. В однонаправленном пайпе данные могут течь только в одном направлении: от одного конца к другому, в то время как в двунаправленном данные могут передаваться между обоими концами.

Анонимный пайп всегда однонаправленный. Именованный может быть и таким, и таким. Именованные пайпы обычно используются в сетевом окружении, где сервер может коннектиться к нескольким клиентам.

В этом tutorialе мы подробно рассмотрим анонимные пайпы. Главная цель таких пайпов - служить каналом между родительским и дочерним процессом или между дочерними процессами.

Анонимный пайп действительно полезен, когда вы взаимодействуете с консольным приложением. Консольное приложение - это вид win32-программ, которые используют консоль для своего ввода и вывода. Консоль - это вроде DOS-box'a. Тем не менее, консольное приложение - это полноценное 32-битное приложение. Оно может использовать любую GUI-функцию, так же как и другие GUI-программы. Она отличается только тем, что у нее есть консоль.

У консольного приложения есть три хэндла, которые оно может использовать для ввода и вывода. Они называются стандартными хэндлами: стандартный ввод, стандартный вывод и стандартный вывод ошибок. Стандартный хэндл ввода используется для того, чтобы читать/получать информации из консоли и стандартный хэндл вывода используется для вывода/распечатки информации на консоль. Стандартный хэндл вывода ошибок используется для сообщения об ошибках.

Консольное приложение может получить эти три стандартных значения, вызвав функцию GetStdHandle, указав хэндл, который она хочет получить. GUI-приложение не имеет консоли. Если вы вызываете GetStdHandle, она возвратит ошибку. Если вы действительно хотите использовать консоль, вы можете вызвать AllocConsole, чтобы зарезервировать новую консоль. Тем не менее, не забудьте вызвать FreeConsole, когда вы уже не будете в ней нуждаться.

Анонимный пайп очень часто используется для перенаправления ввода и/или вывода дочернего консольного приложения. Родительский процесс может быть консолью или GUI-приложением, но дочернее приложение должно быть консольным, чтобы это сработало. Как вы знаете, консольное

приложение использует стандартные хэндлы для ввода и вывода. Если мы хотим перенаправить ввод/вывод консольного приложения, мы можем заменить один хэндл другим хэндлом одного конца пайпа. Консольное приложение не будет знать, что оно использует один конец пайпа. Оно будет считать, что это стандартный хэндл. Это вид полиморфизма на ООП-жаргоне. Это мощный подход, так как нам не нужно модифицировать родительский процесс ни каким образом.

Другая вещь, которую вы должны знать о консольном приложении - это откуда оно берет стандартный хэндл. Когда консольное приложение создано, у родительского приложения есть следующий выбор: оно может создать новую консоль для дочернего приложения или позволить тому наследовать собственную консоль. Чтобы второй метод работал, родительский процесс должен быть консольным, либо, если он GUI'евый, создать консоль с помощью AllocConsole.

Давайте начнем работу. Чтобы создать анонимный пайп, вам требуется вызывать CreatePipe. Эта функция имеет следующий прототип:

```
CreatePipe proto pReadHandle:DWORD, \
                pWriteHandle:DWORD, \
                pPipeAttributes:DWORD, \
                nBufferSize:DWORD
```

- pReadHandle - это указатель на переменную типа dword, которая получит хэндл конца чтения пайпа.
- pWriteHandle - это указатель на переменную типа dword, которая получит хэндл на конец записи пайпа.
- pPipeAttributes указывает на структуру SECURITY_ATTRIBUTES, которая определяет, наследуется ли каждый из концов дочерним процессом.
- nBufferSize - это предполагаемый размер буфера, который пайп зарезервирует для использования. Это всего лишь предполагаемый размер. Вы можете передать NULL, чтобы указать функции использовать размер по умолчанию.

Если вызов прошел успешно, возвращаемое значение не равно нулю, иначе оно будет нулевым.

После успешного вызова CreatePipe вы получите два хэндла, один к концу чтения, а другой к концу записи. Теперь я вкратце изложу шаги, необходимые для перенаправления стандартного вывода дочерней консольной программы в ваш процесс. Заметьте, что мой метод отличается от того, который изложен в справочнике по WinAPI от Borland. Тот метод предполагает, что родительский процесс - это консольное приложение, поэтому дочерний процесс должен наследовать стандартные хэндлы от него. Но большую часть времени нам будет требоваться перенаправить вывод из консольного приложения в GUI'евое.

- Создаем анонимный пайп с помощью CreatePipe. Не забудьте установить параметр bInheritable структуры SECURITY_ATTRIBUTES в TRUE, чтобы хэндлы могли наследоваться.
- Теперь мы должны подготовить параметры, которые передадим CreateProcess (мы используем эту функцию для загрузки консольного приложения). Среди аргументов этой функции есть важная структура STARTUPINFO. Эта структура определяет появление основного окна дочернего процесса, когда он запускается. Эта структура жизненно важна для нас. Вы можете спрятать основное окно и передать хэндл пайпа дочерней консоли вместе с этой структурой.
- Ниже находятся поля, которые вы должны заполнить:
- cb : размер структуры STARTUPINFO
- dwFlags : двоичные битовые флаги, которые определяют, какие члены структуры будут использоваться, также она управляет состоянием основного окна. Нам нужно указать комбинацию STARTF_USESHOWWINDOW and STARTF_USESTDHANDLES.
- hStdOutput и hStdError : хэндлы, которые будут использоваться в дочернем процессе в качестве хэндлов стандартного ввода/вывода. Для наших целей мы передадим хэндл пайпа в качестве стандартного вывода и вывода ошибок. Поэтому когда дочерний процесс выведет что-нибудь туда, он фактически передаст информацию через пайп родительскому процессу.

- wShowWindow управляет тем, как будет отображаться основное окно. Нам не нужно, что окно консоли отображалось на экран, поэтому мы приравняем этот параметр к SW_HIDE.
- Вызов CreateProcess, чтобы загрузить дочернее приложение. После того, как вызов прошел успешно, дочерний процесс все еще находится в спящем состоянии. Он загружается в память, но не запускается немедленно.
- Закройте конец хэндл конца записи пайпа. Это необходимо, так как родительский процессу нет нужды использовать этот хэндл, а пайп не будет работать, если открыть более чем один конец записи. Следовательно, мы должны закрыть его прежде, чем считывать данные из пайпа. тем не менее, не закрывайте этот конец до вызова CreateProcess, иначе ваш пайп будет сломан. Вам следует закрыть конец записи после того, как будет и вызвана функция CreateProcess, и до того, как вы считаете данные из конца чтения пайпа.
- Теперь вы можете читать данные из конца чтения с помощью ReadFile. С ее помощью вы запускаете дочерний процесс, который начнет выполняться, а когда он запишет что-нибудь в стандартный хэндл вывода, данные будут посланы на конец чтения пайпа. Вы должны последовательно вызывать ReadFile, пока она не возвратит ноль, что будет означать, что больше данных нет. С полученной информацией вы можете делать все, что хотите, в нашем случае я вывожу их в edit control.
- Закроем хэндл чтения пайпа.

ПРИМЕР

```
.386
.model flat,stdcall
option casemap:none
include \masm32\include\windows.inc

include \masm32\include\user32.inc
include \masm32\include\kernel32.inc
include \masm32\include\gdi32.inc
includelib \masm32\lib\gdi32.lib

includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib

WinMain PROTO :DWORD,:DWORD,:DWORD,:DWORD

.const
IDR_MAINMENU equ 101          ; the ID of the main menu

IDM_ASSEMBLE equ 40001

.data

ClassName      db "PipeWinClass",0
AppName        db "One-way Pipe Example",0
EditClass      db "EDIT",0
CreatePipeError db "Error during pipe creation",0

CreateProcessError db "Error during process creation",0
CommandLine    db "ml /c /coff /Cp test.asm",0

.data?
hInstance HINSTANCE ?
hwndEdit dd ?

.code
start:
    invoke GetModuleHandle, NULL

    mov hInstance,eax
    invoke WinMain, hInstance,NULL,NULL, SW_SHOWDEFAULT
```

```

    invoke ExitProcess,eax

WinMain proc hInst:DWORD,hPrevInst:DWORD,CmdLine:DWORD,CmdShow:DWORD
    LOCAL wc:WNDCLASSEX
    LOCAL msg:MSG

    LOCAL hwnd:HWND
    mov wc.cbSize,SIZEOF WNDCLASSEX
    mov wc.style, CS_HREDRAW or CS_VREDRAW mov wc.lpfnWndProc, OFFSET

WndProc

    mov wc.cbClsExtra,NULL
    mov wc.cbWndExtra,NULL
    push hInst
    pop wc.hInstance

    mov wc.hbrBackground,COLOR_APPWORKSPACE
    mov wc.lpszMenuName,IDR_MAINMENU
    mov wc.lpszClassName,OFFSET ClassName
    invoke LoadIcon,NULL,IDI_APPLICATION

    mov wc.hIcon,eax
    mov wc.hIconSm,eax
    invoke LoadCursor,NULL,IDC_ARROW
    mov wc.hCursor,eax

    invoke RegisterClassEx, addr wc
    invoke CreateWindowEx,WS_EX_CLIENTEDGE,ADDR ClassName,ADDR
AppName,\ WS_OVERLAPPEDWINDOW+WS_VISIBLE,CW_USEDEFAULT,\
CW_USEDEFAULT,400,200,NULL,NULL,\ hInst,NULL

    mov hwnd,eax
    .while TRUE
        invoke GetMessage, ADDR msg,NULL,0,0
        .BREAK .IF (!eax)

        invoke TranslateMessage, ADDR msg
        invoke DispatchMessage, ADDR msg
    .endw
    mov eax,msg.wParam

    ret
WinMain endp

WndProc proc hwnd:HWND, uMsg:UINT, wParam:WPARAM, lParam:LPARAM
    LOCAL rect:RECT
    LOCAL hRead:DWORD
    LOCAL hWrite:DWORD

    LOCAL startupinfo:STARTUPINFO
    LOCAL pinfo:PROCESS_INFORMATION
    LOCAL buffer[1024]:byte
    LOCAL bytesRead:DWORD

    LOCAL hdc:DWORD
    LOCAL sat:SECURITY_ATTRIBUTES
    .if uMsg==WM_CREATE
        invoke CreateWindowEx,NULL,addr EditClass, NULL, WS_CHILD+
WS_VISIBLE+ ES_MULTILINE+ ES_AUTOHSCROLL+ ES_AUTOVSCROLL, 0, 0, 0, 0,
hwnd, NULL, hInstance, NULL
        mov hwndEdit,eax
    .elseif uMsg==WM_CTLCOLOREDIT

        invoke SetTextColor,wParam,Yellow
        invoke SetBkColor,wParam,Black
        invoke GetStockObject,BLACK_BRUSH
        ret

```

```

.elseif uMsg==WM_SIZE
    mov edx,lParam
    mov ecx,edx
    shr ecx,16

    and edx,0ffffh
    invoke MoveWindow,hwndEdit,0,0,edx,ecx,TRUE
.elseif uMsg==WM_COMMAND
    .if lParam==0

        mov eax,wParam
        .if ax==IDM_ASSEMBLE
            mov sat.niLength,sizeof SECURITY_ATTRIBUTES
            mov sat.lpSecurityDescriptor,NULL

            mov sat.bInheritHandle,TRUE
            invoke CreatePipe,addr hRead,addr hWrite,addr sat,NULL
            .if eax==NULL

                invoke MessageBox, hWnd, addr CreatePipeError, \
                addr AppName, MB_ICONERROR+ MB_OK
            .else
                mov startupinfo.cb,sizeof STARTUPINFO

                invoke GetStartupInfo,addr startupinfo
                mov eax, hWrite
                mov startupinfo.hStdOutput,eax
                mov startupinfo.hStdError,eax

                mov startupinfo.dwFlags, STARTF_USESHOWWINDOW+ \
                STARTF_USESTDHANDLES
                mov startupinfo.wShowWindow,SW_HIDE
                invoke CreateProcess, NULL, addr CommandLine, \
                NULL, NULL, TRUE, NULL, NULL, NULL, addr startupinfo, \
                addr pinfo
                .if eax==NULL
                    invoke MessageBox,hWnd,addr CreateProcessError,\
                    addr AppName,MB_ICONERROR+MB_OK
                .else
                    invoke CloseHandle,hWrite
                    .while TRUE
                        invoke RtlZeroMemory,addr buffer,1024

                        invoke ReadFile,hRead,addr buffer,1023,addr bytesRead,NULL
                        .if eax==NULL
                            .break
                        .endif
                        invoke SendMessage,hwndEdit,EM_SETSEL,-1,0
                        invoke SendMessage,hwndEdit,EM_REPLACESEL,\
                        FALSE,addr buffer
                    .endw
                .endif
                invoke CloseHandle,hRead
            .endif
        .endif
    .endif
.elseif uMsg==WM_DESTROY
    invoke PostQuitMessage,NULL
.else
    invoke DefWindowProc,hWnd,uMsg,wParam,lParam ret
.endif

xor eax,eax
ret
WndProc endp
end start

```


АНАЛИЗ

Пример вызовет `ml.exe`, чтобы скомпилировать файл под названием `test.asm`, и перенаправит вывод в `edit control`. Когда программа загружена, она регистрирует класс окна и создает, как обычно, основное окно.

Теперь наступает самая интересная часть. Мы изменим цвет текста и бэкграунда `edit control'a`. Когда `edit control` подойдет к моменту отрисовки его клиентской области, он пошлет сообщение `WM_CTLCOLOREDIT` родительскому окну.

`wParam` содержит хэндл `device context'a`, который `edit control` будет использовать для отрисовки его клиентской области. Мы можем использовать эту возможность для изменения характеристик HDC.

```
.elseif uMsg==WM_CTLCOLOREDIT  
  
    invoke SetTextColor,wParam,Yellow  
    invoke SetTextColor,wParam,Black  
    invoke GetStockObject,BLACK_BRUSH  
    ret
```

`SetTextColor` изменяет цвет текста на желтый. `SetTextColor` изменяет цвет фона текста на черный. И, наконец, мы получаем хэндл черной кисти, которую мы возвратим Windows. Обработывая сообщение `WM_CTLCOLOREDIT`, мы должны вернуть хэндл кисти, которую Windows использует для отрисовки бэкграунда `edit control'a`. В нашем примере, я хочу, чтобы бэкграунд был черным, поэтому я возвращаю хэндл черной кисти Windows.

Когда пользователь выберет пункт меню 'Assemble', программа создаст анонимный пайп.

```
.if ax==IDM_ASSEMBLE  
  
    mov sat.niLength,sizeof SECURITY_ATTRIBUTES  
    mov sat.lpSecurityDescriptor,NULL  
    mov sat.bInheritHandle,TRUE
```

Перед вызовом `CreatePipe` мы должны заполнить структуру `SECURITY_ATTRIBUTES`. Заметьте, что мы можем передать `NULL`, если нас не интересуют настройки безопасности. И параметр `bInheritHandle` должен быть равен нулю, поэтому хэндл пайпа наследуется дочерним процессом.

```
    invoke CreatePipe,addr hRead,addr hWrite,addr sat,NULL
```

После этого мы вызываем `CreatePipe`, которая заполнит переменные `hRead` и `hWrite` хэндлами концов чтения и записи.

```
    mov startupinfo.cb,sizeof STARTUPINFO  
  
    invoke GetStartupInfo,addr startupinfo  
    mov eax, hWrite  
    mov startupinfo.hStdOutput,eax  
    mov startupinfo.hStdError,eax  
  
    mov startupinfo.dwFlags, STARTF_USESHOWWINDOW+STARTF_USESTDHANDLES  
    mov startupinfo.wShowWindow,SW_HIDE
```

Затем мы заполним структуру `STARTUPINFO`. Мы вызовем `GetStartupInfo`, чтобы заполнить ее значениями родительского процесса. Вы должны заполнить эту структуру, если хотите, чтобы ваш код работал и под `win9x` и под `NT`. После вы модифицируете члены структуры. Мы копируем хэндл конца записи в `hStdOutput` и `hStdError`, так как мы хотим, чтобы дочерний процесс использовал их вместо соответствующих стандартных хэндлов. Мы также хотим спрятать консольное окно дочернего процесса, поэтому в `wShowWindow` мы помещаем значение `SW_HIDE`. И, наконец, мы должны подтвердить, что модифицированные нами поля нужно использовать, поэтому мы указываем флаги `STARTF_USESHOWWINDOW` и `STARTF_USESTDHANDLES`.

```
    invoke CreateProcess, NULL, addr CommandLine, NULL, NULL, TRUE, NULL, NULL, NULL,\  
        addr startupinfo, addr pinfo
```

Теперь мы создаем дочерний процесс функцией `CreateProcess`. Заметьте, что параметр `blnheritHandles` должен быть установлен в `TRUE`, чтобы хэндл пайпа работал.

```
invoke CloseHandle,hWrite
```

После успешного создания дочернего процесса мы закрываем конец записи пайпа. Помните, что мы передали хэндл записи дочернему процессу через структуру `STURTUPINFO`. Если мы не закроем конец записи с нашей стороны, будет два конца записи, и тогда пайп не будет работать. Мы должны закрыть конец записи после `CreateProcess`, но до того, как начнем считывание данных.

```
.while TRUE

    invoke RtlZeroMemory,addr buffer,1024
    invoke ReadFile,hRead,addr buffer,1023,addr bytesRead,NULL
    .if eax==NULL
        .break

    .endif
    invoke SendMessage,hwndEdit,EM_SETSEL,-1,0
    invoke SendMessage,hwndEdit,EM_REPLACESEL,FALSE,addr buffer

.endw
```

Теперь мы готовы читать данные. Мы входим в бесконечный цикл, пока все данные не будут считанны. Мы вызываем `RtlZeroMemory`, чтобы заполнить буфер нулями, потом вызываем `ReadFile` и вместо хэндла файла передаем хэндл пайпа. Заметьте, что мы считываем максимум 1023 байта, так данные, которые мы получим, должны быть ASCIIZ-строкой, которую можно будет передать `edit control`'у.

Когда `ReadFile` вернет данные в буфере, мы выведем их в `edit control`. Тем не менее, здесь есть несколько проблем. Если мы используем `SetWindowText`, чтобы поместить данные в `edit control`, новые данные перезапишут уже считанные! Нам нужно, чтобы новые данные присоединялись к старым.

Для достижения цели мы сначала двигаем курсор к концу текста `edit control`'а, послав сообщение `EM_SETSEL` с `wParam`'ом равным -1. Затем мы присоединяем данные с помощью сообщения `EM_REPLACESEL`.

```
invoke CloseHandle,hRead
```

Когда `ReadFile` возвращает `NULL`, мы выходим из цикла и закрываем конец чтения.

Win32 API. Урок 22. Суперклассинг

Автор: lczelion, пер. Aquila

Дата: 2002-05-22

В этом tutorialе мы изучим суперклассинг, что это такое и для чего он служит. Вы также узнаете, как реализовать навигацию с помощью клавиши 'Tab' в вашем окне.

Скачайте пример здесь (находится в архиве wasm_files.zip: files/recovered/1001022/tut22.zip).

ТЕОРИЯ

Во время вашей программной карьеры, вы наверняка встретитесь с ситуацией, когда вам потребуется несколько контролов с *несколько* отличным поведением. Например, вам могут потребоваться 10 edit control'ов, которые принимают только число. Есть несколько путей достигнуть цели:

- Создать собственный класс и написать контролы с нуля
- Создать эти edit control'ы и сабклассировать каждый из них
- Суперклассировать edit control

Первый метод слишком сложен. Вам придется с нуля воплощать всю функциональность edit control'ов. Слишком трудоемкая задача, чтобы ее можно было быстро выполнить. Второй метод лучше, чем первый, но, тем не менее, также требует немало работы. Все нормально, пока вам надо сабклассировать несколько контролов, но сабклассинг дюжины или еще большего количества контролов может превратиться в ад. Суперклассинг - это техника, которой вы должны владеть.

Суперклассинг - это метод, с помощью которого вы сможете взять контроль над определенным классом окна. По взятием контроля я подразумеваю, что вы сможете изменить свойства класса, так чтобы они соответствовали вашим целям, после чего вы можете создать сколько угодно таких контролов.

Ниже приведены шаги для суперклассинга:

- вызвать функцию GetClassInfoEx, чтобы получить информацию о классе окна, который вы хотите суперклассировать. GetClassInfoEx требует указатель на структуру WNDCLASSEX, которая будет заполнена информацией, если вызов пройдет успешно.
- Изменяйте требуемые параметры WNDCLASSEX. Тем не менее, если два члена, которые вы должны обязательно изменить:
- hInstance - Вы должны поместить в это поле хэндл программы.
- lpzClassName - вы должны поместить сюда указатель на новое имя класса.
- Вы не обязаны изменять параметр lpfnWndProc, но обычно вам будет это нужно делать. Главное не забудьте сохранить старое значение lpfnWndProc, если вам надо будет его вызывать с помощью CallWindowProc.
- Зарегистрирует измененную структуру WNDCLASSEX. У вас будет новый класс окна, который будет обладать некоторыми характеристиками старого класса.
- Создайте окна с помощью нового класса.

Суперклассинг лучше, чем сабклассинг, если вы хотите создать много контролов с одинаковыми характеристиками.

ПРИМЕР

.386

```
.model flat,stdcall
option casemap:none
```

```

include \masm32\include\windows.inc
include \masm32\include\user32.inc

include \masm32\include\kernel32.inc
include \masm32\lib\user32.lib
include \masm32\lib\kernel32.lib

WM_SUPERCLASS equ WM_USER+5
WinMain PROTO :DWORD,:DWORD,:DWORD,:DWORD
EditWndProc PROTO :DWORD,:DWORD,:DWORD,:DWORD

.data
ClassName db "SuperclassWinClass",0

AppName db "Superclassing Demo",0
EditClass db "EDIT",0
OurClass db "SUPEREDITCLASS",0
Message db "You pressed the Enter key in the text box!",0

.data?
hInstance dd ?

hWndEdit dd 6 dup(?)
OldWndProc dd ?

.code
start:
    invoke GetModuleHandle, NULL
    mov hInstance,eax

    invoke WinMain, hInstance,NULL,NULL, SW_SHOWDEFAULT
    invoke ExitProcess,eax

WinMain proc
hInst:HINSTANCE,hPrevInst:HINSTANCE,CmdLine:LPSTR,CmdShow:DWORD
LOCAL wc:WNDCLASSEX
LOCAL msg:MSG

LOCAL hWnd:HWND

mov wc.cbSize,SIZEOF WNDCLASSEX

mov wc.style, CS_HREDRAW or CS_VREDRAW
mov wc.lpfnWndProc, OFFSET WndProc
mov wc.cbClsExtra,NULL
mov wc.cbWndExtra,NULL

push hInst
pop wc.hInstance
mov wc.hbrBackground,COLOR_APPWORKSPACE
mov wc.lpszMenuName,NULL

mov wc.lpszClassName,OFFSET ClassName
invoke LoadIcon,NULL,IDI_APPLICATION
mov wc.hIcon,eax
mov wc.hIconSm,eax

invoke LoadCursor,NULL,IDC_ARROW
mov wc.hCursor,eax
invoke RegisterClassEx, addr wc
invoke CreateWindowEx,WS_EX_CLIENTEDGE+WS_EX_CONTROLPARENT,ADDR ClassName,ADDR AppName,\
WS_OVERLAPPED+WS_CAPTION+WS_SYSMENU+WS_MINIMIZEBOX+WS_MAXIMIZEBOX+WS_VISIBLE, \
CW_USEDEFAULT,350,220,NULL,NULL,\
hInst,NULL

```

```

mov hwnd,eax

.while TRUE
    invoke GetMessage, ADDR msg,NULL,0,0
    .BREAK .IF (!eax)
    invoke TranslateMessage, ADDR msg

    invoke DispatchMessage, ADDR msg
.endw
mov eax,msg.wParam
ret

WinMain endp

WndProc proc uses ebx edi hwnd:HWND, uMsg:UINT, wParam:WPARAM,
lParam:LPARAM
    LOCAL wc:WNDCLASSEX
    .if uMsg==WM_CREATE
        mov wc.cbSize,sizeof WNDCLASSEX

        invoke GetClassInfoEx,NULL,addr EditClass,addr wc
        push wc.lpfnWndProc
        pop OldWndProc
        mov wc.lpfnWndProc, OFFSET EditWndProc

        push hInstance
        pop wc.hInstance
        mov wc.lpszClassName,OFFSET OurClass
        invoke RegisterClassEx, addr wc

        xor ebx,ebx
        mov edi,20
        .while ebx<6
            invoke CreateWindowEx,WS_EX_CLIENTEDGE,ADDR OurClass,NULL,\
                WS_CHILD+WS_VISIBLE+WS_BORDER,20,\
                edi,300,25,hwnd,ebx,\
                hInstance,NULL
            mov dword ptr [hwndEdit+4*ebx],eax

            add edi,25
            inc ebx
        .endw
        invoke SetFocus,hwndEdit

    .elseif uMsg==WM_DESTROY
        invoke PostQuitMessage,NULL
    .else
        invoke DefWindowProc,hwnd,uMsg,wParam,lParam

        ret
    .endif
    xor eax,eax
    ret

WndProc endp

EditWndProc PROC hEdit:DWORD,uMsg:DWORD,wParam:DWORD,lParam:DWORD

    .if uMsg==WM_CHAR
        mov eax,wParam
        .if (al>="0" && al<="9") || (al>="A" && al<="F") || (al>="a" && al<="f") || al==VK_BACK

            .if al>="a" && al<="f"
                sub al,20h
            .endif
            invoke CallWindowProc,OldWndProc,hEdit,uMsg,eax,lParam

            ret
        .endif
    .endif

```

```

        .elseif uMsg==WM_KEYDOWN
            mov eax,wParam

            .if al==VK_RETURN
                invoke MessageBox,hEdit,addr Message,addr
AppName,MB_OK+MB_ICONINFORMATION
                invoke SetFocus,hEdit

            .elseif al==VK_TAB
                invoke GetKeyState,VK_SHIFT
                test eax,80000000
                .if ZERO?

                    invoke GetWindow,hEdit,GW_HWNDNEXT
                    .if eax==NULL
                        invoke GetWindow,hEdit,GW_HWNDFIRST
                    .endif

                .else
                    invoke GetWindow,hEdit,GW_HWNDPREV
                    .if eax==NULL
                        invoke GetWindow,hEdit,GW_HWNDLAST
                    .endif
                .endif
                invoke SetFocus,eax
                xor eax,eax

            ret
        .else
            invoke CallWindowProc,OldWndProc,hEdit,uMsg,wParam,lParam
            ret
        .endif
    .endif
    xor eax,eax
    ret
EditWndProc endp

end start

```

АНАЛИЗ

Программа создаст простое окно с "измененными" edit control'ами в своей клиентской области. Edit control'ы будут принимать только шестнадцатиричные числа. Фактически, я адаптировал пример с сабклассингом. программа стартует как обычно, а самое интересное происходит, когда создается основное окно:

```

        .if uMsg==WM_CREATE
            mov wc.cbSize,sizeof WNDCLASSEX
            invoke GetClassInfoEx,NULL,addr EditClass,addr wc

```

Сначала мы заполним данными класса, который мы хотим суперклассировать, в нашем случае это класс edit'a. Помните, что вы должны установить параметр структуры WNDCLASSEX, перед тем, как вызвать GetClassInfoEx, в противном случае она будет заполнена неверно. После вызова GetClassInfoEx у нас будет иметься вся необходимая для создания нового класса информация.

```

        push wc.lpfnWndProc
        pop OldWndProc
        mov wc.lpfnWndProc, OFFSET EditWndProc
        push hInstance
        pop wc.hInstance
        mov wc.lpszClassName,OFFSET OurClass

```

Теперь мы можем изменить некоторые члены `wc`. Первый из них - это указатель на процедуру окна. Так как нам нужно будет соединить вызовы новой и старой процедуры в цепь, нам необходимо сохранить старое значение в переменную, чтобы потом воспользоваться функции `CallWindowProc`. Эта техника идентична с сабклассингом, не считая того, что вы напрямую изменяете структуру `WNDCLASSEX` не вызывая `SetWindowLong`. Следующие два поля должны быть изменены, иначе вам не удастся зарегистрировать ваш новый класс окна, `hInstance` и `lpszClassName`. Вы должны заменить старое значение `hInstance` на хэндл вашей программы, а также выбрать имя для нового класса.

```
invoke RegisterClassEx, addr wc
```

Когда все готово, регистрируйте новый класс. Вы получите новый класс, обладающий некоторыми характеристиками старого.

```
xor ebx,ebx
mov edi,20
.while ebx<6
    invoke CreateWindowEx,WS_EX_CLIENTEDGE,ADDR OurClass,NULL,\
        WS_CHILD+WS_VISIBLE+WS_BORDER,20,\
        edi,300,25,hWnd,ebx,\
        hInstance,NULL
    mov dword ptr [hWndEdit+4*ebx],eax

    add edi,25
    inc ebx
.endw
invoke SetFocus,hWndEdit
```

Теперь, когда мы зарегистрировали класс, мы можем создать основанные на нем окна. Вы вышеприведенном куске кода, я использовал `ebx` в качестве счетчика созданных окон. `edi` используется как у-координата левого верхнего угла окна. Когда окно создано, его хэндл сохраняется в массиве `dword`'ов. Когда все окна созданы, устанавливаем фокус на первое окно. К этому моменту у вас есть 6 `edit control`'ов, которые принимают только шестнадцатиричные числа. Новая процедура окна, заменившая старую, выполняет роль фильтра. Фактически, это работает точно также, как и в примере с сабклассингом, только вам не нужно выполнять лишнюю работу.

Я вставил кусок кода, который обрабатывает нажатия на `Tab`, чтобы сделать пример более полезным для вас. Обычно, если вы помещаете контролы на диалоговое окно, его внутренний менеджер сам обрабатывает нажатия на клавиши навигации. Увы, но это недоступно, когда вы помещаете контролы на обычное окно. Вам следует сабклассировать их, чтобы нажатия на `Tab` обрабатывались. В нашем примере нам нет нужны сабклассировать контролы по одному, так как мы уже суперклассировали, поэтому можем реализовать "центральный менеджер навигации контролов".

```
.elseif al==VK_TAB
    invoke GetKeyState,VK_SHIFT
    test eax,80000000
    .if ZERO?

        invoke GetWindow,hEdit,GW_HWNDNEXT
        .if eax==NULL
            invoke GetWindow,hEdit,GW_HWNDFIRST
        .endif

    .else
        invoke GetWindow,hEdit,GW_HWNDPREV
        .if eax==NULL
            invoke GetWindow,hEdit,GW_HWNDLAST
        .endif
    .endif
    invoke SetFocus,eax
xor eax,eax
ret
```

Вышеприведенный код взят из процедуры `EditWndClass`. Он проверяет, нажал ли пользователь клавишу `tab`, если да, он вызывает `GetKeyStat`, чтобы узнать, нажата ли также клавиша `Shift`. `GetKeyState` возвращает значение в `eax`, которое определяет, нажата ли указанная клавиша или нет. Если клавиша нажата, верхний бит `eax` будет установлен. Если нет, он будет очищен. Поэтому мы тестируем полученное значение `80000000h`. Если верхний бит установлен, это будет означать, что пользователь нажал `shift` и `tab` одновременно, и должны обработать это отдельно.

Если пользователь нажал клавишу `Tab`, мы вызываем `GetWindow`, чтобы получить хэндл следующего контрола. Мы используем флаг `GW_HWNDNEXT`, чтобы указать `GetWindow` получить хэндл следующего окна относительно текущего `hEdit`. Если эта функция возвращает `NULL`, то такого окна нет и мы устанавливаем фокус на первое окно, вызвав `GetWindow` с флагом `GW_HWNDFIRST`. `Shift-Tab` работает так же, как и обычно нажатие на `Tab`, только передвигает фокус окна назад.

Win32 API. Урок 23. Иконка в system tray

Автор: lczelion, пер. WD-40

Дата: 2002-05-23

На этом уроке мы узнаем, как помещать иконки в system tray и как создавать/использовать всплывающее меню.

Пример можете скачать [здесь](#) (находится в архиве `wasm_files.zip: files/recovered/1001023/tut23.zip`).

ТЕОРИЯ

System tray - это прямоугольная область панели задач, в которой располагаются несколько иконок. Скорее всего, вы обнаружите там как минимум цифровые часы. Вы можете самостоятельно помещать иконки в system tray. Далее приводятся шаги, которые нужно для этого выполнить:

- Заполните структуру NOTIFYICONDATA, содержащую следующие поля:
- `cbSize` - размер данной структуры.
- `hwnd` - хэндл окна, которое будет получать уведомление, когда над иконкой в tray'e произойдёт событие мыши.
- `uID` - константа, используемая в качестве идентификатора иконки. Вы сами выбираете значение этой константе. В случае, если вы поместили в system tray несколько иконок, вы сможете узнать, над какой именно из них произошло событие мыши.
- `uFlags` - указывает, какие поля данной структуры заполнены
- `NIF_ICON` Поле `hIcon` заполнено.
- `NIF_MESSAGE` Поле `uCallbackMessage` заполнено.
- `NIF_TIP` Поле `szTip` заполнено.
- `uCallbackMessage` - пользовательское сообщение, которое Windows отошлёт указанному в поле `hwnd` окну, в случае, когда над иконкой произойдёт событие мыши. Сообщение вы создаете сами.
- `hIcon` - хэндл иконки, которую вы хотите поместить в system tray.
- `szTip` - 64-байтовый массив, содержащий строку для использования в качестве всплывающей подсказки к иконке.
- Вызовите `Shell_NotifyIcon`, определённую в `shell32.inc`. Данная функция имеет следующий прототип:

```
Shell_NotifyIcon PROTO dwMessage:DWORD, pnid:DWORD
```

- `dwMessage` - это тип сообщения, которое нужно отправить оболочке.
- `NIM_ADD` Добавляет иконку в system tray.
- `NIM_DELETE` Удаляет иконку из system tray.
- `NIM_MODIFY` Изменяет иконку в system tray.
- `pnid` - это указатель на корректно заполненную структуру NOTIFYICONDATA.
- Если вы хотите добавить иконку в system tray, используйте сообщение `NIM_ADD`, если хотите удалить иконку, применяйте `NIM_DELETE`.

Вот, собственно, и всё. Но чаще всего просто поместить иконку в system tray недостаточно. Вам нужно как-то реагировать на события мыши, происходящие над этой иконкой. Это можно сделать, обрабатывая сообщение, указанное в поле `uCallbackMessage` структуры NOTIFYICONDATA. Это сообщение содержит следующие значения в `wParam` и `lParam` (отдельное спасибо `s__d` за эту информацию):

- `wParam` содержит ID иконки. Это то же самое значение, что вы поместили в поле `uID` структуры NOTIFYICONDATA.

- IParam Младшее слово содержит сообщение мыши. Например, если пользователь сделал правый щелчок по иконке, то IParam будет содержать WM_RBUTTONDOWN.

Обычно иконка в system tray показывает всплывающее меню при правом щелчке по ней. Этого можно добиться, если сначала создать само всплывающее меню, а затем вызывать TrackPopupMenu для его отображения. Шаги приведены ниже:

- Создайте всплывающее меню, вызвав CreatePopupMenu. Эта функция создаёт пустое меню, и при успешном создании возвращает его хэндл в eax.
- Добавьте пункты в меню с помощью AppendMenu, InsertMenu или InsertMenuItem.
- Когда вам будет нужно отобразить всплывающее меню на месте курсора мыши, вызовите GetCursorPos, чтобы узнать текущие координаты курсора, а затем вызовите TrackPopupMenu, чтобы вывести меню на экран.
- Когда пользователь щёлкнет на одном из пунктов меню, Windows отправит сообщение WM_COMMAND вашей оконной процедуре, точно так же, как и при работе с обычным меню.

Внимание: остерегайтесь следующих проблем, часто возникающих при работе со всплывающими меню.

- Когда меню отображено на экране, щелчок вне меню не приводит к его немедленному исчезновению. Это происходит потому, что окно, которое будет получать уведомления от меню, ДОЛЖНО быть на переднем плане. Просто вызовите SetForegroundWindow, чтобы исправить эту проблему.
- После вызова SetForegroundWindow вы обнаружите, что в первый раз всплывающее меню сработает нормально, но при последующем появлении оно будет отображаться, а затем тут же исчезать. Как написано в MSDN, это сделано "намеренно". Необходимо переключить задачу на программу, являющуюся владельцем иконки в system tray. Этого можно добиться, отправив любое сообщение окну вашей программы. Но только используйте PostMessage, а не SendMessage!

ПРИМЕР

```
.386
.model flat,stdcall
option casemap:none

include \masm32\include\windows.inc
include \masm32\include\user32.inc
include \masm32\include\kernel32.inc
include \masm32\include\shell32.inc

includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib
includelib \masm32\lib\shell32.lib

WM_SHELLNOTIFY equ WM_USER+5
IDI_TRAY equ 0
IDM_RESTORE equ 1000

IDM_EXIT equ 1010
WinMain PROTO :DWORD,:DWORD,:DWORD,:DWORD

.data
ClassName db "TrayIconWinClass",0
AppName db "TrayIcon Demo",0
RestoreString db "&Restore",0

ExitString db "E&xit Program",0

.data?

hInstance dd ?
note NOTIFYICONDATA <>
```

hPopupMenu dd ?

.code

start:

invoke GetModuleHandle, NULL

mov hInstance,eax

invoke WinMain, hInstance,NULL,NULL, SW_SHOWDEFAULT

invoke ExitProcess,eax

WinMain proc

hInst:HINSTANCE,hPrevInst:HINSTANCE,CmdLine:LPSTR,CmdShow:DWORD

LOCAL wc:WNDCLASSEX

LOCAL msg:MSG

LOCAL hwnd:HWND

mov wc.cbSize,SIZEOF WNDCLASSEX

mov wc.style, CS_HREDRAW or CS_VREDRAW or CS_DBLCLKS

mov wc.lpfnWndProc, OFFSET WndProc

mov wc.cbClsExtra,NULL

mov wc.cbWndExtra,NULL

push hInst

pop wc.hInstance

mov wc.hbrBackground,COLOR_APPWORKSPACE

mov wc.lpszMenuName,NULL

mov wc.lpszClassName,OFFSET ClassName

invoke LoadIcon,NULL,IDI_APPLICATION

mov* wc.hIcon,eax

mov* wc.hIconSm,eax

invoke LoadCursor,NULL,IDC_ARROW

mov* wc.hCursor,eax

invoke RegisterClassEx, addr wc

invoke CreateWindowEx,WS_EX_CLIENTEDGE,ADDR ClassName,ADDR AppName,\
WS_OVERLAPPED+WS_CAPTION+WS_SYSMENU+WS_MINIMIZEBOX+WS_MAXIMIZEBOX+WS_VISIBLE,\
CW_USEDEFAULT,350,200,NULL,NULL,\
hInst,NULL

mov hwnd,eax

.while TRUE

invoke GetMessage, ADDR msg,NULL,0,0

.BREAK .IF (!eax)

invoke TranslateMessage, ADDR msg

invoke DispatchMessage, ADDR msg

.endw

mov eax,msg.wParam

ret

WinMain endp

WndProc proc hwnd:HWND, uMsg:UINT, wParam:WPARAM, lParam:LPARAM

LOCAL pt:POINT

.if uMsg==WM_CREATE

invoke CreatePopupMenu

mov hPopupMenu,eax

invoke AppendMenu,hPopupMenu,MF_STRING,IDM_RESTORE,addr RestoreString

invoke AppendMenu,hPopupMenu,MF_STRING,IDM_EXIT,addr ExitString

.elseif uMsg==WM_DESTROY

invoke DestroyMenu,hPopupMenu

invoke PostQuitMessage,NULL

.elseif uMsg==WM_SIZE

.if wParam==SIZE_MINIMIZED

mov note.cbSize,sizeof NOTIFYICONDATA

```

        push hWnd
        pop note.hwnd
        mov note.uID,IDI_TRAY
        mov note.uFlags,NIF_ICON+NIF_MESSAGE+NIF_TIP

        mov note.uCallbackMessage,WM_SHELLNOTIFY
        invoke LoadIcon,NULL,IDI_WINLOGO
        mov note.hIcon,eax
        invoke lstrcpy,addr note.szTip,addr AppName

        invoke ShowWindow,hWnd,SW_HIDE
        invoke Shell_NotifyIcon,NIM_ADD,addr note
    .endif
.elseif uMsg==WM_COMMAND

    .if lParam==0
        invoke Shell_NotifyIcon,NIM_DELETE,addr note
        mov eax,wParam
        .if ax==IDM_RESTORE

            invoke ShowWindow,hWnd,SW_RESTORE
        .else
            invoke DestroyWindow,hWnd
        .endif

    .endif

.elseif uMsg==WM_SHELLNOTIFY
    .if wParam==IDI_TRAY
        .if lParam==WM_RBUTTONDOWN

            invoke GetCursorPos,addr pt
            invoke SetForegroundWindow,hWnd
            invoke TrackPopupMenu,hPopupMenu,TPM_RIGHTALIGN,pt.x,pt.y,NULL,hWnd,NULL

            invoke PostMessage,hWnd,WM_NULL,0,0
        .elseif lParam==WM_LBUTTONDBLCLK
            invoke SendMessage,hWnd,WM_COMMAND,IDM_RESTORE,0
        .endif

    .endif

.else
    invoke DefWindowProc,hWnd,uMsg,wParam,lParam
    ret

.endif
xor eax,eax
ret
WndProc endp

end start

```

АНАЛИЗ

Программа отобразит на экране обычное окно. По нажатию кнопки "Свернуть" оно свернётся до иконки в system tray. По двойному щелчку по иконке программа восстановит своё окно и удалит иконку из system tray. По правому щелчку будет выведено всплывающее меню, из которого можно восстановить программу или выйти из неё.

```

.if uMsg==WM_CREATE
    invoke CreatePopupMenu
    mov hPopupMenu,eax

    invoke AppendMenu,hPopupMenu,MF_STRING,IDM_RESTORE,addr RestoreString
    invoke AppendMenu,hPopupMenu,MF_STRING,IDM_EXIT,addr ExitString

```

Когда будет создано главное окно, также создастся всплывающее меню, к которому затем будут добавлены два пункта. Функция AppendMenu имеет следующий синтаксис:

```
AppendMenu PROTO hMenu:DWORD, uFlags:DWORD, uIDNewItem:DWORD, lpNewItem:DWORD
```

- `hMenu` это хэндл меню, к которому вы хотите добавить пункт
- `uFlags` информирует Windows о добавляемом пункте меню - изображение ли это, строка или отрисовываемый владельцем объект; включен ли он, неопределён или отключен, и т.д. Полный список есть в `win32 api reference`. В нашем случае мы используем флаг `MF_STRING`, который означает, что пункт меню - это строка.
- `uIDNewItem` это ID пункта меню. Это значение определяется пользователем, и используется для обращения к пункту меню.
- `lpNewItem` хранит содержание пункта меню, в зависимости от значения поля `uFlags`. Так как мы указали `MF_STRING` в поле `uFlags`, то `lpNewItem` должен содержать указатель на строку для отображения в пункте меню.

После того, как всплывающее меню создано, главное окно будет терпеливо ждать до тех пор, пока пользователь не нажмёт на кнопку "Свернуть".

Когда окно сворачивается, оно получает сообщение `WM_SIZE` со значением `SIZE_MINIMIZED` в `wParam`.

```
.elseif uMsg==WM_SIZE

    .if wParam==SIZE_MINIMIZED
        mov note.cbSize, sizeof NOTIFYICONDATA
        push hWnd
        pop note.hwnd

        mov note.uID, IDI_TRAY
        mov note.uFlags, NIF_ICON+NIF_MESSAGE+NIF_TIP
        mov note.uCallbackMessage, WM_SHELLNOTIFY
        invoke LoadIcon, NULL, IDI_WINLOGO

        mov note.hIcon, eax
        invoke lstrcpy, addr note.szTip, addr AppName
        invoke ShowWindow, hWnd, SW_HIDE
        invoke Shell_NotifyIcon, NIM_ADD, addr note
    .endif
```

Мы используем этот момент, чтобы заполнить структуру `NOTIFYICONDATA`. `IDI_TRAY` это просто константа, определённая в начале исходного кода. Ей можно задать любое значение. Это не очень важно, так как у нас только одна иконка в system tray. Но если вы захотите поместить туда сразу несколько иконок, то вам потребуется задать уникальный ID для каждой из них. Мы выставляем сразу все флаги в поле `uFlags`, так как мы указываем иконку (`NIF_ICON`), мы указываем пользовательское сообщение (`NIF_MESSAGE`), а также текст всплывающей подсказки (`NIF_TIP`). `WM_SHELLNOTIFY` это просто пользовательское сообщение, определённое как `WM_USER+5`. Само значение не так важно, пока оно сохраняет свою уникальность. Я использовал логотип Windows в качестве иконки для этой программы, но вы можете использовать и любую другую иконку ;) Просто загрузите её из файла ресурсов вызовом `LoadIcon` и сохраните возвращаемое значение в поле `hIcon`. После всего этого поместим в поле `szTip` текст, который мы хотим видеть в качестве всплывающей подсказки к иконке.

Мы скрываем главное окно, чтобы создать эффект "сворачивания в иконку". Затем мы вызываем `Shell_NotifyIcon` с сообщением `NIM_ADD`, чтобы добавить иконку в system tray.

Теперь наше главное окно скрыто, а иконка успешно помещена в system tray. Если вы наведёте на неё курсор, то увидите подсказку с текстом, который вы поместили в поле `szTip`. Далее, если вы дважды щелкните по иконке, восстановится главное окно, а сама иконка исчезнет.

```
.elseif uMsg==WM_SHELLNOTIFY

    .if wParam==IDI_TRAY
        .if lParam==WM_RBUTTONDOWN
            invoke GetCursorPos, addr pt
```

```

        invoke SetForegroundWindow,hWnd

        invoke TrackPopupMenu,hPopupMenu,TPM_RIGHTALIGN,pt.x,pt.y,NULL,hWnd,NULL
        invoke PostMessage,hWnd,WM_NULL,0,0
    .elseif lParam==WM_LBUTTONDOWNBLCLK
        invoke SendMessage,hWnd,WM_COMMAND,IDM_RESTORE,0
    .endif
.endif
.endif

```

Когда над иконкой происходит событие мыши, ваше окно получает сообщение WM_SHELLNOTIFY, то есть пользовательское сообщение, указанное в поле uCallbackMessage. Напомню, что по приёму этого сообщения wParam содержит ID иконки, а lParam содержит событие мыши. В вышеприведенном коде сначала проверяется, пришло ли сообщение от интересующей нас иконки. Если да, то тогда мы смотрим на событие мыши. Так как нам нужны только правый щелчок и левый двойной щелчок, то мы обрабатываем лишь сообщения WM_RBUTTONDOWN и WM_LBUTTONDOWNBLCLK.

Если сообщение от мыши это WM_RBUTTONDOWN, мы вызываем GetCursorPos, чтобы узнать текущие координаты курсора мыши. После возврата из функции, структура POINT содержит абсолютные координаты курсора. Под абсолютными координатами я подразумеваю координаты, привязанные ко всему экрану, не берущие во внимание границы окна. Например, если разрешение экрана 640*480, то правый нижний угол это x==639, y==479. Если вы желаете перевести абсолютные координаты в оконные, используйте функцию ScreenToClient.

Однако мы хотим отобразить всплывающее меню в точке, где сейчас расположен курсор мыши, с помощью функции TrackPopupMenu, которой требуются именно абсолютные координаты. Поэтому мы просто используем координаты, полученные от GetCursorPos.

TrackPopupMenu имеет следующий синтаксис: TrackPopupMenu PROTO hMenu:DWORD, uFlags:DWORD, x:DWORD, y:DWORD, nReserved:DWORD, hWnd:DWORD, prcRect:DWORD

- hMenu это хэндл всплывающего меню, которое нужно отобразить.
- uFlags указывает опции отображения. Например, как располагать меню относительно указанных ниже координат, и какая из кнопок мыши используется для отслеживания меню. В нашем примере мы используем флаг TPM_RIGHTALIGN, чтобы разместить меню слева от указанной точки.
- x и y указывают местоположение меню в абсолютных координатах.
- nReserved должно содержать NULL.
- hWnd это хэндл окна, которое будет получать сообщения от меню.
- prcRect это прямоугольная область экрана, щелчки в пределах которой НЕ будут приводить к исчезновению меню. Обычно сюда помещается NULL, чтобы меню исчезало при любом щелчке вне его.

Когда пользователь дважды щелкнёт по иконке, мы отправим нашему окну сообщение WM_COMMAND с указанием IDM_RESTORE, чтобы создать иллюзию выбора пользователем пункта "Восстановить" в меню, и таким образом восстановить окно, а также удалить иконку из system tray. Чтобы иметь возможность получать сообщения двойного щелчка, главное окно должно иметь стиль CS_DBLCLKS.

```

        invoke Shell_NotifyIcon,NIM_DELETE,addr note
        mov eax,wParam
        .if ax==IDM_RESTORE
            invoke ShowWindow,hWnd,SW_RESTORE
        .else
            invoke DestroyWindow,hWnd
        .endif

```

Когда пользователь выберет пункт "Восстановить" в меню, мы удаляем иконку повторным вызовом Shell_NotifyIcon, только на этот раз указывая NIM_DELETE в качестве сообщения. Затем мы возвращаем первоначальный вид главному окну. Если пользователь выберет пункт "Закрыть", мы

тоже удаляем иконку из system tray и уничтожаем главное окно вызовом DestroyWindow.

Win32 API. Урок 24. Windows-хуки

Автор: lczelion, пер. Aquila

Дата: Не указана

В этом tutorialе мы изучим хуки. Это очень мощная техника. С их помощью вы сможете вмешиваться в другие процессы и иногда менять их поведение.

Скачайте [пример](#) [здесь](#) (находится в архиве `wasm_files.zip: files/recovered/1001024/tut24.zip`).

ТЕОРИЯ

Хуки Windows можно считать одной из самых мощных техник. С их помощью вы можете перехватывать события, которые случаются внутри созданного вами или кем-то другим процесса. Перехватывая что-либо, вы сообщаете Windows о фильтрующей функции, также называемой функцией перехвата, которая будет вызываться каждый раз, когда будет происходить интересное вас событие. Есть два вида хуков: локальные и удаленные.

- Локальные хуки перехватывают события, которые случаются в процессе, созданном вам.
- Удаленные хуки перехватывают события, которые случаются в других процессах. Есть два вида удаленных хуков:
 - тредспециализированные перехватывают события, которые случаются в определенном треде другого процесса. То есть, такой хук нужен вам, когда необходимо наблюдать за процессами, происходящими в определенном треде какого-то процесса.
 - системные перехватывают все события, предназначенные для всех тредов всех процессов в системе.

При установке хуков, помните, что они оказывают отрицательное воздействие на быстродействие системы. Особенно в этом отличаются системные. Так как все требуемые события будут проходить через вашу функцию, ваша система может значительно потерять в быстродействии.

Поэтому, если вы используете системный хук, вам следует использовать их только тогда, когда вам это действительно нужно. Также, существует высокая вероятность того, что другие процессы могут зависнуть, если что-нибудь неправильно в вашей функции. Помните: вместе с силой приходит ответственность.

Вы должны понимать, как работают хуки, чтобы использовать их эффективно. Когда вы создаете хук, Windows создает в памяти структуры данных, которая содержит информацию о хуке, и добавляет ее в связанный список уже существующих хуков. Новый хук добавляется перед всеми старыми хуками. Когда случается событие, то если вы установили локальный хук, вызывается фильтрующая функция в вашем процессе, поэтому тут все просто. Но если вы установили удаленный хук, система должна вставить код хук-процедуры в адресное пространство другого процесса. Система может сделать это только, если функция находится в DLL. Таким образом, если вы хотите использовать удаленный хук, ваша хук-процедура должна находиться в DLL. Из этого правила есть два исключения:

журнально-записывающие и журнально-проигрывающие хуки. Хук-процедуры для этих типов хуков должны находиться в тред, который инсталлировал хуки. Причина этого кроется в том, что оба хука имеют дело с низкоуровневым перехватом хардварных входных событий. Эти события должны быть записаны/проиграны в том порядке, в котором они произошли. Если код такого хука находится в DLL, входные события могут быть "разбросаны" по нескольким тредам, что делает невозможным установления точной их последовательности. Решение: процедуры таких хуков должны быть в одном тред, то есть в том тред, который устанавливает хуки.

Существует 14 типов хуков:

- WH_CALLWNDPROC - хук вызывается при вызове SendMessage.
- WH_CALLWNDPROCRET - хук вызывается, когда возвращается SendMessage.
- WH_GETMESSAGE - хук вызывается, когда вызывается GetMessage или PeekMessage.
- WH_KEYBOARD - хук вызывается, когда GetMessage или PeekMessage получают WM_KEYUP или WM_KEYDOWN из очереди сообщений.
- WH_MOUSE - хук вызывается, когда GetMessage или PeekMessage получают сообщение от мыши из очереди сообщений.
- WH_HADRWARE - хук вызывается, когда GetMessage или PeekMessage получают хардварное сообщение, не относящееся к клавиатуре или мыши.
- WH_MSGFILTER - хук вызывается, когда диалоговое окно, меню или скролбар готовятся к обработке сообщения. Этот хук - локальный. Он создан специально для тех объектов, у которых свой внутренний цикл сообщений.
- WH_SYSMSGFILTER - то же самое WH_MSGFILTER, но системный.
- WH_JOURNALRECORD - хук вызывается, когда Windows получает сообщение из очереди хардварных сообщений.
- WH_JOURNALPLAYBACK - хук вызывается, когда событие затребовывается из очереди хардварных сообщений.
- WH_SHELL - хук вызывается, когда происходит что-то интересное и связанное с оболочкой, например, когда таскбару нужно перерисовать кнопку.
- WH_CBN - хук используется специально для CBT.
- WH_FOREGROUND - такие хуки используются Windows. Обычным приложениям от них пользы немного.
- WH_DEBUG - хук используется для отладки хук-процедуры.

Теперь, когда мы немного подучили теорию, мы можем перейти к тому, как, собственно, устанавливать/снимать хуки.

Чтобы установить хук, вам нужно вызвать функцию SetWindowsHookEx, имеющую следующий синтаксис:

```
SetWindowsHookEx proto HookType:DWORD, pHookProc:DWORD,  
                    hInstance:DWORD, ThreadID:DWORD
```

- HookType - это одно из значений, перечисленных выше (WH_MOUSE, WH_KEYBOARD и т.п.).
- pHookProc - это адрес хук-процедуры, которая будет вызвана для обработки сообщений от хука. Если хук является удаленным, он должен находиться в DLL. Если нет, то он должен быть внутри процесса.
- hInstance - это хэндл DLL, в которой находится хук-процедура. Если хук локальный, тогда это значения должно быть равно NULL.
- ThreadID - это ID треда, на который вы хотите поставить хук. Этот параметр определяет является ли хук локальным или удаленным. Если этот параметр равен NULL, Windows будет считать хук системным и удаленным, который затрагивает все треды в системе. Если вы укажете ID одного из тредов вашего собственного процесса, хук будет локальным. Если вы укажете ID треда из другого процесса, то хук будет тредоспециализированным и удаленным. Из этого правила есть два исключения: WH_JOURNALRECORD и WH_JOURNALPLAYBACK - это всегда локальные системные хуки, которым не нужно быть в DLL. Также WH_SYSMSGFILTER - это всегда системный удаленный хук. Фактически он идентичен хуку WH_MSGFILTER при ThreadID равным 0.

Если вызов успешен, он возвращает хэндл хука в eax. Если нет, возвращается NULL. Вы должны сохранить хэндл хука, чтобы снять его в дальнейшем.

Вы можете деинсталлировать хук, вызвав UnhookWindowsHookEx, которая принимает только один параметр - хэндл хука, который нужно деинсталлировать. Если вызов успешен, он возвращает

ненулевое значение в eax. Иначе он возвратит NULL.

Хук-процедура будет вызываться каждый раз, когда будет происходить событие, ассоциированное с установленным хуком. Например, если вы устанавливаете хук WH_MOUSE, когда происходит событие, связанное с мышью, ваша хук-процедура будет вызвана. Вне зависимости от типа установленного хука, хук-процедура всегда будет иметь один и тот же прототип:

```
HookProc proto nCode:DWORD, wParam:DWORD, lParam:DWORD
```

- nCode задает код хука.
- wParam и lParam содержат дополнительную информацию о событии.

Вместо HookProc будет имя вашей хук-процедуры. Вы можете назвать ее как угодно, главное чтобы ее прототип совпадал с вышеприведенным. Интерпретация nCode, wParam и lParam зависит от типа установленного хука, так же, как и возвращаемое хук-процедурой значение. Например:

WH_CALLWNDPROC

- nCode может иметь значение HC_ACTION - это означает, что окну было послано сообщение.
- wParam содержит посланное сообщение, если он не равен нулю, lParam указывает на структуру CWPSTRUCT.
- возвращаемое значение: не используется, возвращайте ноль.

WH_MOUSE

- nCode может быть равно HC_ACTION или HC_NOREMOVE.
- wParam содержит сообщение от мыши.
- lParam указывает на структуру MOUSEHOOKSTRUCT.
- возвращаемое значение: ноль, если сообщение должно быть обработано. 1, если сообщение должно быть пропущено.

Вы должны обратиться к вашему справочнику по Win32 API за подробным описанием значение параметров и возвращаемых значений хука, который вы хотите установить.

Теперь еще один нюанс относительно хук-процедуры. Помните, что хуки соединены в связанный список, причем в его начале стоит хук, установленный последним. Когда происходит событие, Windows вызовет только первый хук в цепи. Вызов следующего в цепи хука остается на вашей ответственности. Вы можете и не вызывать его, но вам лучше знать, что вы делаете. Как правило, стоит вызвать следующую процедуру, чтобы другие хуки также могли обработать событие. Вы можете вызвать следующий хук с помощью функции CallNextHookEx:

```
CallNextHookEx proto hHook:DWORD, nCode:DWORD, wParam:DWORD,  
lParam:DWORD
```

- hHook - хэндл вашего хука. Функция использует этот хук для того, чтобы определить, какой хук надо вызвать следующим.
- nCode, wParam и lParam - вы передаете соответствующие параметры, полученные от Windows.

Важная деталь относительно удаленных хуков: хук-процедура должна находиться в DLL, которая будет промэппирована в другой процесс. Когда Windows мэппирует DLL в другой процесс, секция данных мэппироваться не будет. То есть, все процессы разделяют одну копию секции кода, но у них будет своя личная копия секции кода DLL! Это может стать большим сюрпризом для непредупрежденного человека. Вы можете подумать, что при сохранении значения в переменную в секции данных DLL, это значение получают все процессы, загрузившие DLL в свое адресное пространство. На самом деле, это не так. В обычной ситуации, такое поведение правильно, потому что это создает иллюзию, что у каждого процесса есть отдельная копия DLL. Но не тогда, когда это касается хуков Windows. Нам нужно, чтобы DLL была идентична во всех процессах, включая данные. Решение: вы должны пометить секцию данных как разделяемую. Это можно сделать, указав атрибуты секции линкеру. Если речь идет о MASM'e, это делается так:

```
/SECTION: , S
```

Имя секции инициализированных данных '.data', а неинициализированных - '.bss'. Например, если вы хотите скомпилировать DLL, которая содержит хук-процедуру, и вам нужно, что секция неинициализированных данных разделялась между процессами, вы должны использовать следующую команду:

```
link /section:.bss,S /DLL /SUBSYSTEM:WINDOWS .....
```

Атрибут 'S' отмечает, что секция разделяемая.

ПРИМЕР

Есть два модуля: один - это основная программа с GUI'ем, а другая - это DLL, которая устанавливает/снимает хук.

```
;-----
; Исходный код основной программы
;-----
.386
.model flat,stdcall
option casemap:none

include \masm32\include\windows.inc
include \masm32\include\user32.inc
include \masm32\include\kernel32.inc
include mousehook.inc

includelib mousehook.lib
includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib

wsprintfA proto C :DWORD,:DWORD,:VARARG
wsprintf TEXTEQU

.const
IDD_MAINDLG          equ 101
IDC_CLASSNAME        equ 1000

IDC_HANDLE           equ 1001
IDC_WNDPROC          equ 1002
IDC_HOOK             equ 1004
IDC_EXIT             equ 1005

WM_MOUSEHOOK         equ WM_USER+6

DlgFunc PROTO :DWORD,:DWORD,:DWORD,:DWORD

.data
HookFlag dd FALSE

HookText db "&Hook",0
UnhookText db "&Unhook",0
template db "%lx",0

.data?
hInstance dd ?
hHook dd ?

.code
start:
    invoke GetModuleHandle,NULL
    mov hInstance,eax

    invoke DialogBoxParam,hInstance,IDD_MAINDLG,NULL,addr DlgFunc,NULL
    invoke ExitProcess,NULL
DlgFunc proc hDlg:DWORD,uMsg:DWORD,wParam:DWORD,lParam:DWORD
```

```

LOCAL hLib:DWORD
LOCAL buffer[128]:byte
LOCAL buffer1[128]:byte

LOCAL rect:RECT
.if uMsg==WM_CLOSE
    .if HookFlag==TRUE
        invoke UninstallHook

    .endif
    invoke EndDialog,hDlg,NULL
.elseif uMsg==WM_INITDIALOG
    invoke GetWindowRect,hDlg,addr rect

    invoke SetWindowPos, hDlg, HWND_TOPMOST, rect.left, rect.top,
rect.right, rect.bottom, SWP_SHOWWINDOW
.elseif uMsg==WM_MOUSEHOOK
    invoke GetDlgItemText,hDlg, IDC_HANDLE,addr buffer1,128

    invoke wsprintf,addr buffer,addr template,wParam
    invoke lstrcmpi,addr buffer,addr buffer1
    .if eax!=0
        invoke SetDlgItemText,hDlg, IDC_HANDLE,addr buffer

    .endif
    invoke GetDlgItemText,hDlg, IDC_CLASSNAME,addr buffer1,128
    invoke GetClassName,wParam,addr buffer,128
    invoke lstrcmpi,addr buffer,addr buffer1

    .if eax!=0
        invoke SetDlgItemText,hDlg, IDC_CLASSNAME,addr buffer
    .endif
    invoke GetDlgItemText,hDlg, IDC_WNDPROC,addr buffer1,128

    invoke GetClassLong,wParam,GCL_WNDPROC
    invoke wsprintf,addr buffer,addr template,eax
    invoke lstrcmpi,addr buffer,addr buffer1
    .if eax!=0

        invoke SetDlgItemText,hDlg, IDC_WNDPROC,addr buffer
    .endif
.elseif uMsg==WM_COMMAND
    .if lParam!=0

        mov eax,wParam
        mov edx,eax
        shr edx,16
        .if dx==BN_CLICKED

            .if ax==IDC_EXIT
                invoke SendMessage,hDlg,WM_CLOSE,0,0
            .else
                .if HookFlag==FALSE

                    invoke InstallHook,hDlg
                    .if eax!=NULL
                        mov HookFlag,TRUE
                        invoke SetDlgItemText,hDlg, IDC_HOOK,addr UnhookText
                    .endif
                .else
                    invoke UninstallHook

                    invoke SetDlgItemText,hDlg, IDC_HOOK,addr HookText
                    mov HookFlag,FALSE
                    invoke SetDlgItemText,hDlg, IDC_CLASSNAME,NULL
                    invoke SetDlgItemText,hDlg, IDC_HANDLE,NULL

                    invoke SetDlgItemText,hDlg, IDC_WNDPROC,NULL
                .endif
            .endif
        .endif
    .endif
.endif

```

```

        .endif
    .else
        mov eax,FALSE
        ret

    .endif
    mov eax,TRUE
    ret
DlgFunc endp

end start

;-----
; Это исходный код DLL
;-----
.386

.model flat,stdcall
option casemap:none
include \masm32\include\windows.inc
include \masm32\include\kernel32.inc

includelib \masm32\lib\kernel32.lib
include \masm32\include\user32.inc
includelib \masm32\lib\user32.lib

.const
WM_MOUSEHOOK equ WM_USER+6

.data
hInstance dd 0

.data?
hHook dd ?
hWnd dd ?

.code
DllEntry proc hInst:HINSTANCE, reason:DWORD, reserved1:DWORD

    .if reason==DLL_PROCESS_ATTACH
        push hInst
        pop hInstance
    .endif

    mov eax,TRUE
    ret
DllEntry Endp

MouseProc proc nCode:DWORD,wParam:DWORD,lParam:DWORD
    invoke CallNextHookEx,hHook,nCode,wParam,lParam
    mov edx,lParam

    assume edx:PTR MOUSEHOOKSTRUCT
    invoke WindowFromPoint,[edx].pt.x,[edx].pt.y
    invoke PostMessage,hWnd,WM_MOUSEHOOK,eax,0
    assume edx:nothing

    xor eax,eax
    ret
MouseProc endp

InstallHook proc hwnd:DWORD
    push hwnd

```

```

        pop hWndd

        invoke SetWindowsHookEx,WH_MOUSE,addr MouseProc,hInstance,NULL
        mov hHook,eax
        ret
InstallHook endp

UninstallHook proc
        invoke UnhookWindowsHookEx,hHook

        ret
UninstallHook endp

End DllEntry
;-----
; Это makefile DLL
;-----

NAME=mousehook

$(N*ME).dll: $(NAME).obj
        link /SECTION:.bss,S /DLL /DEF:$(NAME).def /SUBSYSTEM:WINDOWS
        /LIBPATH:c:\masm\lib $(NAME).obj
$(NAME).obj: $(NAME).asm

        ml /c /coff /Cp $(NAME).asm

```

АНАЛИЗ

Пример отобразит диалоговое окно с тремя edit control'ами, которые будут заполнены именем класса, хэндлом окна и адресом процедуры окна, ассоциированное с окном под курсором мыши. Есть две кнопки - Hook и Exit. Когда вы нажимаете кнопку Hook, программа перехватывает сообщения от мыши и текст на кнопке меняется на Unhook. Когда вы двигаете курсор мыши над каким-либо окном, информация о нем отобразится в окне программы. Когда вы нажмете кнопку Unhook, программа уберет установленный hook.

Основная программа использует диалоговое окно в качестве основного. Она определяет специальное сообщение - WM_MOUSEHOOK, которая будет использоваться между основной программой и DLL с хуком. Когда основная программа получает это сообщение, wParam содержит хэндл окна, над которым находится курсор мыши. Конечно, это было сделано произвольно. Я решил слать хэндл в wParam, чтобы было проще. Вы можете выбрать другой метод взаимодействия между основной программой и DLL с хуком.

```

        .if HookFlag==FALSE
            invoke InstallHook,hDlg
            .if eax!=NULL
                mov HookFlag,TRUE
                invoke SetDlgItemText,hDlg,IDC_HOOK,addr UnhookText
            .endif
        .endif

```

Программа пользуется флагом, HookFlag, чтобы отслеживать состояние хука.

Он равен FALSE, если хук не установлен, и TRUE, если установлен.

Когда пользователь нажимет кнопку hook, программа проверяет, установлен ли уже хук. Если это так, она вызывает функцию InstallHook из DLL. Заметьте, что мы передаем хэндл основного диалогового окна в качестве параметра функции, чтобы хук-DLL могла посылать сообщения WM_MOUSEHOOK верному окну, то есть нашему.

Когда программа загружена, DLL с хуком также загрузится. Фактически, DLL загружаются сразу после того, как программа оказывается в памяти. Входная функция DLL вызывается прежде, чем будет исполнена первая инструкция основной программы. Поэтому, когда основная программа запускается DLL и инициализируются. Мы помещаем следующий код во входную функцию хук-DLL:

```

        .if reason==DLL_PROCESS_ATTACH
            push hInst

            pop hInstance
        .endif

```

Данный код всего лишь сохраняет хэндл процесса DLL в глобальную переменную, названную hInstance для использования внутри функции InstallHook. Так как входная функция вызывается прежде, чем будут вызваны другие функции в DLL, hInstance будет всегда верен. Мы помещаем hInstance в секцию .data, поэтому это значение будет различаться от процесса к процессу. Когда курсор мыши проходит над окном, хук-DLL мэппируется в процес. Представьте, что уже есть DLL, которая занимает предполагаемый загрузочный адрес хук-DLL. Значение hInstance будет обновлено. Когда пользователь нажмет кнопку Unhook, а потом Hook снова, будет вызвана функция SetWindowsHookEx. Тем не менее, в этот раз, она будет использовать новое значение hInstance, которое будет неверным, потому что в данном процессе загрузочный адрес DLL не измениться. Хук будет локальным, что нам не нужно.

```

InstallHook proc hwnd:DWORD
    push hwnd
    pop hWnd
    invoke SetWindowsHookEx,WH_MOUSE,addr MouseProc,hInstance,NULL
    mov hHook,eax
    ret
InstallHook endp

```

Функция InstallHook сама по себе очень проста. Она сохраняет хэндл окна, переданный ей в качестве параметра, в глобальную переменную hWnd. Затем она вызывает SetWindowsHookEx, чтобы установить хук на мышь. Возвращенное значение сохраняется в глобальную переменную hHook, чтобы в будущем передать ее UnhookWindowsHookEx.

После того, как вызван SetWindowsHookEx, хук начинает работать. Всякий раз, когда в системе случается мышинное событие, вызывается MouseProc (ваша хук-процедура).

```

MouseProc proc nCode:DWORD,wParam:DWORD,lParam:DWORD
    invoke CallNextHookEx,hHook,nCode,wParam,lParam
    mov edx,lParam
    assume edx:PTR MOUSEHOOKSTRUCT
    invoke WindowFromPoint,[edx].pt.x,[edx].pt.y
    invoke PostMessage,hWnd,WM_MOUSEHOOK,eax,0
    assume edx:nothing
    xor eax,eax
    ret
MouseProc endp

```

Сначала вызывается CallNextHookEx, чтобы другие хуки также могли обработать событие мыши. После этого, она вызывает функцию WindowFromPoint, чтобы получить хэндл окна, находящегося в указанной координате экрана. Заметьте, что мы используем структуру POINT, являющуюся членом структуры MOUSEHOOKSTRUCT, на которую указывает lParam, то есть координату текущего местонахождения курсора. После этого, мы посылаем хэндл окна основной программы через сообщение WM_MOUSEHOOK. Вы должны помнить: вам не следует использовать SendMessage в хук-процедуре, так как это может вызвать "подвисы", поэтому рекомендуется использовать PostMessage. Структура MOUSEHOOKSTRUCT определена ниже:

```

MOUSEHOOKSTRUCT STRUCT DWORD
    pt          POINT <>
    hwnd        DWORD    ?
    wParam      DWORD    ?
    dwExtraInfo DWORD    ?
MOUSEHOOKSTRUCT ENDS

```

- pt - это текущая координата курсора мыши.

- hwnd - это хэндл окна, которое получает сообщения от мыши. Это обычно окно под курсором мыши, но не всегда. Если окно вызывает SetCapture, сообщения от мыши будут перенаправлены этому окну. По этой причине я не использую параметр hwnd этой структуры, а вызываю вместо этого WindowFromPoint.
- wParam дает дополнительную информацию о том, где находится курсор мыши. Полный список значений вы можете получить в вашем справочнике по Win32 API в разделе сообщения WM_NCHITTEST.
- lParam содержит дополнительную информацию, ассоциированную с сообщением. Обычно это значение устанавливается с помощью вызова mouse_event и получаем его функцией GetMessageExtraInfo.

Когда основное окно получает сообщение WM_MOUSEHOOK, оно использует хэндл окна в wParam'e, чтобы получить информацию об окне.

```
.elseif uMsg==WM_MOUSEHOOK

    invoke GetDlgItemText,hDlg,IDC_HANDLE,addr buffer1,128
    invoke wsprintf,addr buffer,addr template,wParam
    invoke lstrcmpi,addr buffer,addr buffer1
    .if eax!=0

        invoke SetDlgItemText,hDlg,IDC_HANDLE,addr buffer
    .endif
    invoke GetDlgItemText,hDlg,IDC_CLASSNAME,addr buffer1,128
    invoke GetClassName,wParam,addr buffer,128

    invoke lstrcmpi,addr buffer,addr buffer1
    .if eax!=0
        invoke SetDlgItemText,hDlg,IDC_CLASSNAME,addr buffer
    .endif

    invoke GetDlgItemText,hDlg,IDC_WNDPROC,addr buffer1,128
    invoke GetClassLong,wParam,GCL_WNDPROC
    invoke wsprintf,addr buffer,addr template,eax
    invoke lstrcmpi,addr buffer,addr buffer1

    .if eax!=0
        invoke SetDlgItemText,hDlg,IDC_WNDPROC,addr buffer
    .endif
```

Чтобы избежать мерцания, мы проверяем, не идентичны ли текст в edit control'ах с текстом, который мы собираемся ввести. Если это так, то мы пропускаем этот этап.

Мы получаем имя класса с помощью вызова GetClassName, адрес процедуры с помощью вызова GetClassLong со значением GCL_WNDPROC, а затем форматируем их в строки и помещаем в соответствующие edit control'ы.

```
    invoke UninstallHook
    invoke SetDlgItemText,hDlg,IDC_HOOK,addr HookText
    mov HookFlag,FALSE

    invoke SetDlgItemText,hDlg,IDC_CLASSNAME,NULL
    invoke SetDlgItemText,hDlg,IDC_HANDLE,NULL
    invoke SetDlgItemText,hDlg,IDC_WNDPROC,NULL
```

Когда юзер нажмет кнопку Unhook, программа вызовет функцию UninstallHook в хук-DLL. UninstallHook всего лишь вызывает UnhookWindowsHookEx. После этого, она меняет текст кнопки обратно на "Hook", HookFlag на FALSE и очищает содержимое edit control'ов.

Обратите внимание на опции линкера в makefile.

```
Link /SECTION:.bss,S /DLL /DEF:$(NAME).def /SUBSYSTEM:WINDOWS
```

Секции .bss помечается как разделяемая, чтобы все процессы разделяли секцию неинициализируемых данных хук-DLL. Без этой опции, ваша DLL функционировала бы

неправильно.

Win32 API. Урок 25. Простой битмэп

Автор: lczelion, пер. WD-40

Дата: Не указана

На этом уроке мы научимся использовать битмэпы в своих программах. Если быть более точным, мы научимся отображать битмэп в клиентской области нашей программы. Скачайте пример здесь (находится в архиве wasm_files.zip: files/recovered/1001025/tut25.zip).

ТЕОРИЯ

Битмэпы можно представлять себе как изображения, хранимые в компьютере. На компьютерах используется множество различных форматов изображений, но Windows естественным образом поддерживает только формат растровых изображений Windows (.bmp). На этом уроке, когда речь будет идти о битмэпах, будет подразумеваться именно этот формат. Самый простой способ использовать битмэп - это использовать его как ресурс. Есть два способа это выполнить. Можно включить битмэп в файл определения ресурсов (.rc) следующим образом:

```
#define IDB_MYBITMAP 100
IDB_MYBITMAP BITMAP "c:\project\example.bmp"
```

В этом методе для представления битмэпа используется константа. В первой строчке просто задаётся константа с именем IDB_MYBITMAP и значением 100. По этому имени мы и будем обращаться к битмэпу в нашей программе. В следующей строке объявляется битмэп-ресурс. Таким образом, компилятор узнаёт, где ему искать собственно сам .bmp файл.

В другом методе для представления битмэпа используется имя, делается это следующим образом:

```
MyBitMap BITMAP "c:\project\example.bmp"
```

При использовании этого метода, в вашей программе вам придётся ссылаться на битмэп по строке "MyBitMap", а не по значению.

Оба метода прекрасно работают, главное - определиться, какой именно вы будете использовать. После включения битмэпа в файл ресурсов, можно приступить к его отображению в клиентской области нашего окна:

- Вызовите LoadBitmap чтобы узнать хэндл битмэпа. Функция LoadBitmap имеет следующий прототип:

```
LoadBitmap proto hInstance:HINSTANCE, lpBitmapName:LPSTR
```

- Функция возвращает хэндл битмэпа. hInstance есть хэндл инстанции вашей программы. lpBitmapName - это указатель на строку, содержащую имя битмэпа (необходимо при использовании 2-го метода). Если для обращения к битмэпу вы используете константу (например, IDB_MYBITMAP), то её значение вы и должны сюда поместить (в нашем случае это было бы значение 100). Не помешает небольшой пример:

- Первый метод:

```
.386
.model flat, stdcall
.....
.const

IDB_MYBITMAP equ 100
.....
.data?
hInstance dd ?
.....
```

```

.code
.....
    invoke GetModuleHandle,NULL

    mov hInstance,eax
.....
    invoke LoadBitmap,hInstance,IDB_MYBITMAP
.....

```

- Второй метод:

```

.386
.model flat, stdcall
.....

.data
BitmapName db "MyBitMap",0
.....
.data?

hInstance dd ?
.....
.code
.....

    invoke GetModuleHandle,NULL
    mov hInstance,eax
.....
    invoke LoadBitmap,hInstance,addr BitmapName
.....

```

- Получите хэндл device context'a (DC). Это можно сделать либо вызовом функции BeginPaint в ответ на сообщение WM_PAINT, либо вызовом GetDC в любое время.

Создайте device context в памяти (memory DC) с теми же атрибутами, что и device context, полученный на предыдущем шаге. Идея в том, чтобы создать некоторую "невидимую" поверхность, на которой мы можем отрисовать битмэп. После этого мы просто копируем содержимое невидимой поверхности в текущий device context с помощью вызова одной-единственной функции. Этот приём называется двойной буферизацией (double buffering) и используется для быстрого вывода изображений на экран. Создать "невидимую" поверхность можно вызовом CreateCompatibleDC:

```
CreateCompatibleDC proto hdc:HDC
```

При успешном завершении функция возвращает через регистр eax хэндл device context'a в памяти. hdc - это хэндл device context'a, с которым должен быть совместим DC в памяти.

- После создания невидимой поверхности вы можете отобразить на ней битмэп с помощью вызова SelectObject, передав ей в качестве первого параметра хэндл DC в памяти, а в качестве второго - хэндл битмэпа. Прототип этой функции следующий:

```
SelectObject proto hdc:HDC, hGdiObject:DWORD
```

- Теперь битмэп отображен на device context'e в памяти. Единственное, что осталось сделать - это скопировать его на устройство вывода, то есть на настоящий device context. Этого можно добиться с помощью таких функций, как BitBlt и StretchBlt. BitBlt просто копирует содержимое одного DC в другой, поэтому она работает быстро; StretchBlt может сжимать или растягивать изображение по размерам того DC, куда копирует. Для простоты здесь мы будем использовать BitBlt, имеющую следующий прототип:

```
BitBlt proto hdcDest:DWORD, nxDest:DWORD, nyDest:DWORD, \
nWidth:DWORD, nHeight:DWORD, hdcSrc:DWORD, nxSrc:DWORD, \
nySrc:DWORD, dwROP:DWORD \
```

- hdcDest это хэндл device context'a, в который копируется битмэп
- nxDest, nyDest это координаты верхнего левого угла области вывода

- nWidth, nHeight это ширина и высота области вывода
- hdcSrc это хэндл device context'a, из которого копируется битмэп
- pxSrc, pySrc это координаты левого верхнего угла исходной области
- dwROP это код растровой операции (raster-operation code, ROP), указывающий, как совмещать пиксели исходного и конечного изображений. Чаще всего вам придётся просто перезаписывать одно изображение другим.
- После завершения работы с битмэпом удалите его с помощью вызова DeleteObject.

Вот и всё! Если коротко, то сначала добавьте битмэп в файл ресурсов. После этого загрузите его из ресурсов с помощью LoadBitmap. Вы получите хэндл битмэпа. Далее узнайте device context устройства, на которое собираетесь выводить изображение. Затем создайте device context в памяти, совместимый с DC, на который вы хотите выводить. "Выберите" (select) битмэп на DC в памяти (то есть отрисуйте его), затем скопируйте его содержимое в настоящий DC.

ПРИМЕР

```
.386
.model flat,stdcall
option casemap:none
include \masm32\include\windows.inc
include \masm32\include\user32.inc
include \masm32\include\kernel32.inc
include \masm32\include\gdi32.inc
includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib
includelib \masm32\lib\gdi32.lib

WinMain proto :DWORD,:DWORD,:DWORD,:DWORD
IDB_MAIN equ 1

.data
ClassName db "SimpleWin32ASMBitmapClass",0
AppName db "Win32ASM Simple Bitmap Example",0

.data?
hInstance HINSTANCE ?
CommandLine LPSTR ?
hBitmap dd ?

.code
start:
invoke GetModuleHandle, NULL
mov hInstance,eax
invoke GetCommandLine
mov CommandLine,eax
invoke WinMain, hInstance,NULL,CommandLine, SW_SHOWDEFAULT
invoke ExitProcess,eax

WinMain proc
hInst:HINSTANCE,hPrevInst:HINSTANCE,CmdLine:LPSTR,CmdShow:DWORD
LOCAL wc:WNDCLASSEX
LOCAL msg:MSG
LOCAL hwnd:HWND
mov wc.cbSize,SIZEOF WNDCLASSEX
mov wc.style, CS_HREDRAW or CS_VREDRAW
mov wc.lpfnWndProc, OFFSET WndProc
mov wc.cbClsExtra,NULL
mov wc.cbWndExtra,NULL
push hInstance
pop wc.hInstance
mov wc.hbrBackground,COLOR_WINDOW+1
mov wc.lpszMenuName,NULL
mov wc.lpszClassName,OFFSET ClassName
invoke LoadIcon,NULL,IDI_APPLICATION
mov wc.hIcon,eax
```

```

mov     wc.hIconSm,eax
invoke  LoadCursor,NULL,IDC_ARROW
mov     wc.hCursor,eax
invoke  RegisterClassEx, addr wc
INVOKE  CreateWindowEx,NULL,ADDR  ClassName,ADDR  AppName,\
        WS_OVERLAPPEDWINDOW,CW_USEDEFAULT,\
        CW_USEDEFAULT,CW_USEDEFAULT,CW_USEDEFAULT,NULL,NULL,\
        hInst,NULL
mov     hwnd,eax
invoke  ShowWindow, hwnd,SW_SHOWNORMAL
invoke  UpdateWindow, hwnd
.while  TRUE
    invoke  GetMessage, ADDR msg,NULL,0,0
    .break .if (!eax)
    invoke  TranslateMessage, ADDR msg
    invoke  DispatchMessage, ADDR msg
.endw
mov     eax,msg.wParam
ret
WinMain endp

WndProc proc hwnd:HWND, uMsg:UINT, wParam:WPARAM, lParam:LPARAM
    LOCAL ps:PAINTSTRUCT
    LOCAL hdc:HDC
    LOCAL hMemDC:HDC
    LOCAL rect:RECT
    .if uMsg==WM_CREATE
        invoke  LoadBitmap,hInstance,IDB_MAIN
        mov     hBitmap,eax
    .elseif uMsg==WM_PAINT
        invoke  BeginPaint,hwnd,addr ps
        mov     hdc,eax
        invoke  CreateCompatibleDC,hdc
        mov     hMemDC,eax
        invoke  SelectObject,hMemDC,hBitmap
        invoke  GetClientRect,hwnd,addr rect
        invoke  BitBlt,hdc,0,0,rect.right,rect.bottom,hMemDC,0,0,SRCCOPY
        invoke  DeleteDC,hMemDC
        invoke  EndPaint,hwnd,addr ps
    .elseif uMsg==WM_DESTROY
        invoke  DeleteObject,hBitmap
        invoke  PostQuitMessage,NULL
    .ELSE
        invoke  DefWindowProc,hwnd,uMsg,wParam,lParam
        ret
    .ENDIF
    xor     eax,eax
    ret
WndProc endp
end start

;-----
;                                     Файл ресурсов
;-----
#define IDB_MAIN 1
IDB_MAIN BITMAP "tweety78.bmp"

```

АНАЛИЗ

Собственно, на этом уроке и анализировать нечего :)

```

#define IDB_MAIN 1
IDB_MAIN BITMAP "tweety78.bmp"

```

Определите константу IDB_MAIN, присвоив ей значение 1. Затем используйте эту константу при определении битмэп-ресурса. Файл, который будет включен в ресурсы, называется "tweety78.bmp" и располагается в той же папке, что и файл ресурсов.

```

.if uMsg==WM_CREATE
    invoke  LoadBitmap,hInstance,IDB_MAIN

```

```
mov hBitmap,eax
```

После получения сообщения WM_CREATE мы вызываем LoadBitmap для загрузки битмэпа из файла ресурсов, передавая идентификатор битмэпа в качестве второго параметра. По завершению работы функции мы получим хэндл битмэпа. Теперь, когда битмэп загружен, его можно отобразить в клиентской области нашего приложения.

```
.elseif uMsg==WM_PAINT
    invoke BeginPaint,hWnd,addr ps
    mov    hdc,eax
    invoke CreateCompatibleDC,hdc
    mov    hMemDC,eax
    invoke SelectObject,hMemDC,hBitmap
    invoke GetClientRect,hWnd,addr rect
    invoke BitBlt,hdc,0,0,rect.right,rect.bottom,hMemDC,0,0,SRCCOPY
    invoke DeleteDC,hMemDC
    invoke EndPaint,hWnd,addr ps
```

Мы решили отрисовывать битмэп в ответ на сообщение WM_PAINT. Для этого мы сначала вызываем BeginPaint и получаем хэндл DC. Затем создаем совместимый memory DC вызовом CreateCompatibleDC. Далее "выбираем" битмэп в память с помощью SelectObject. Определяем размеры клиентской области окна через GetClientRect. Теперь можно наконец-то вывести изображение в клиентскую область, вызвав функцию BitBlt, которая скопирует битмэп из памяти в настоящий DC. По завершению рисования мы удаляем DC в памяти вызовом DeleteDC, так как он нам больше не нужен. Подаём сигнал о завершении отрисовки окна с помощью EndPaint.

```
.elseif uMsg==WM_DESTROY
    invoke DeleteObject,hBitmap
    invoke PostQuitMessage,NULL
```

По окончании работы удаляем битмэп посредством DeleteObject.

Win32 API. Урок 26. Сплэш-экран

Автор: lczelion, пер. Aquila

Дата: 2002-05-26

Теперь, когда мы знаем, как использовать битмап, мы можем применить его более творчески. Сплэш-экран. Скачайте пример (находится в архиве `wasm_files.zip:files/recovered/1001026/tut26.zip`).

ТЕОРИЯ

Сплэш-экран - это окно, у которого нет заголовка, нет системных кнопок, нет `border'a`, которое отображает битмап на некоторое время и затем исчезает. Обычно оно используется во время загрузки программы, чтобы отображать лого программы или отвлечь внимание пользователя, пока программа делает объемную инициализацию. В этом tutorialе мы создадим сплэш-экран.

Первый шаг - это прописать битмап в файле ресурсов. Тем не менее, если это важно для вас, то загружать битмап, который будет использоваться только один раз, и держать его в памяти, пока программа не будет закрыта, пустая трата ресурсов. Лучшим решением является ресурсовую DLL, которая будет содержать битмап, и чьей целью является отображение сплэш-экрана. В этом случае вы сможете загрузить DLL, когда вам нужно отобразить сплэш-экран, и выгрузить ее, как только нужна в ней отпадает. Поэтому у нас будет два модуля: основная программа и сплэш-экран. Мы поместим битмап в файл ресурсов DLL.

Общая схема такова:

- Поместить битмап в DLL как ресурс.
- Основная программа вызывает `LoadLibrary`, чтобы загрузить `dll` в память.
- Запускается входная функция DLL. Она создаст таймер и установит время, в течении которого будет отображаться сплэш-экран. Затем она регистрирует и создаст окно без заголовка и бордера, после чего отобразит битмап в клиентской области.
- Когда закончится указанный период времени, сплэш-экран будет убран с экрана и контроль будет передан главной программе.
- Основная программа вызовет `FreeLibrary`, чтобы выгрузить DLL из памяти, а затем перейдет к выполнению того, к чему она предназначена.

Мы детально проанализируем описанную последовательность действий.

Загрузка/выгрузка DLL

Вы можете динамически загрузить DLL с помощью функции `LoadLibrary`, которая имеет следующий синтаксис:

```
LoadLibrary proto lpDLLName:DWORD
```

Она принимает только один параметр: адрес имени DLL, который вы хотите загрузить в память. Если вызов пройдет успешно, он возвратит хэндл модуля DLL, в противном случае `NULL`.

Чтобы выгрузить DLL, вызовите `FreeLibrary`:

```
FreeLibrary proto hLib:DWORD
```

Она получает один параметр: хэндл модуля DLL, которую вы хотите выгрузить.

Как использовать таймер

Во-первых, мы должны создать таймер с помощью функции `SetTimer`:

```
SetTimer proto hWnd:DWORD, TimerID:DWORD, uElapse:DWORD,  
lpTimerFunc:DWORD
```

- `hWnd` - хэндл окна, которое будет получать уведомительные сообщения от таймера. Этот параметр может быть равным `NULL`, если никакое окно не ассоциируется с таймером.
- `TimerID` - заданное пользователем значение, которое будет использоваться в качестве ID таймера.
- `uElapsed` - временной интервал в миллисекундах.
- `lpTimerFunc` - адрес функции, которая будет обрабатывать уведомительные сообщения от таймера. Если вы передаете `NULL`, сообщения от таймера будут посылаться окну, указанному в параметре `hWnd`.
- `SetTimer` возвращает ID таймера, если вызов прошел успешно, иначе она возвратит `NULL`. Поэтому лучше не использовать ноль в качестве ID таймера.

Вы можете создать таймер двумя путями:

- Если у вас есть окно и вы хотите, чтобы сообщения от таймера посылались окну, вы должны передать все четыре параметра `SetTimer` (`lpTimerFunc` должен быть равен `NULL`).
- Если у вас нет окна или вы не хотите обрабатывать сообщения таймера в процедуре окна, вы должны передать `NULL` функции вместо хэндла окна. Вы также должны указать адрес функции таймера, которая будет обрабатывать его сообщения.

В этом tutorialе мы используем первый подход.

Каждый раз за указанный вами временной интервал окну, ассоциированному с таймером, будет посылаться сообщение `WM_TIMER`. Например, если вы укажете 1000: ваше окно будет получать `WM_TIMER` каждую секунду.

Когда вам больше не нужен таймер, уничтожьте его с помощью `KillTimer`:

```
KillTimer proto hWnd:DWORD, TimerID:DWORD
```

ПРИМЕР

```
;-----
;                               Основная программа
;-----
.386
.model flat,stdcall
option casemap:none
include \masm32\include\windows.inc
include \masm32\include\user32.inc
include \masm32\include\kernel32.inc
includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib

WinMain proto :DWORD,:DWORD,:DWORD,:DWORD

.data
ClassName db "SplashDemoWinClass",0
AppName db "Splash Screen Example",0
Libname db "splash.dll",0

.data?
hInstance HINSTANCE ?
CommandLine LPSTR ?
.code
start:
    invoke LoadLibrary,addr Libname
    .if eax!=NULL
        invoke FreeLibrary,eax
    .endif
    invoke GetModuleHandle, NULL
    mov     hInstance,eax
    invoke GetCommandLine
    mov     CommandLine,eax
    invoke WinMain, hInstance,NULL,CommandLine, SW_SHOWDEFAULT
```

```
invoke ExitProcess,eax
```

```
WinMain proc
hInst:HINSTANCE,hPrevInst:HINSTANCE,CmdLine:LPSTR,CmdShow:DWORD
LOCAL wc:WNDCLASSEX
LOCAL msg:MSG
LOCAL hwnd:HWND
mov wc.cbSize,SIZEOF WNDCLASSEX
mov wc.style, CS_HREDRAW or CS_VREDRAW
mov wc.lpfWndProc, OFFSET WndProc
mov wc.cbClsExtra,NULL
mov wc.cbWndExtra,NULL
push hInstance
pop wc.hInstance
mov wc.hbrBackground,COLOR_WINDOW+1
mov wc.lpszMenuName,NULL
mov wc.lpszClassName,OFFSET ClassName
invoke LoadIcon,NULL,IDI_APPLICATION
mov wc.hIcon,eax
mov wc.hIconSm,eax
invoke LoadCursor,NULL,IDC_ARROW
mov wc.hCursor,eax
invoke RegisterClassEx, addr wc
INVOKE CreateWindowEx,NULL,ADDR ClassName,ADDR AppName,\
        WS_OVERLAPPEDWINDOW,CW_USEDEFAULT,\
        CW_USEDEFAULT,CW_USEDEFAULT,CW_USEDEFAULT,NULL,NULL,\
        hInst,NULL
mov hwnd,eax
invoke ShowWindow, hwnd,SW_SHOWNORMAL
invoke UpdateWindow, hwnd
.while TRUE
    invoke GetMessage, ADDR msg,NULL,0,0
    .break .if (!eax)
    invoke TranslateMessage, ADDR msg
    invoke DispatchMessage, ADDR msg
.endw
mov eax,msg.wParam
ret
WinMain endp

WndProc proc hwnd:HWND, uMsg:UINT, wParam:WPARAM, lParam:LPARAM
    .IF uMsg==WM_DESTROY
        invoke PostQuitMessage,NULL
    .ELSE
        invoke DefWindowProc,hwnd,uMsg,wParam,lParam
    .ENDIF
    xor eax,eax
    ret
WndProc endp
end start
```

```
;-----
;                               DLL с битмапом
;-----
.386
.model flat, stdcall
include \masm32\include\windows.inc
include \masm32\include\user32.inc
include \masm32\include\kernel32.inc
include \masm32\include\gdi32.inc
includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib
includelib \masm32\lib\gdi32.lib
.data
BitmapName db "MySplashBMP",0
ClassName db "SplashWndClass",0
hBitmap dd 0
TimerID dd 0
.data
```



```

hInstance dd ?

.code

DllEntry proc hInst:DWORD, reason:DWORD, reserved1:DWORD
    .if reason==DLL_PROCESS_ATTACH ; When the dll is loaded
        push hInst
        pop hInstance
        call ShowBitMap
    .endif
    mov eax,TRUE
    ret
DllEntry Endp
ShowBitMap proc
    LOCAL wc:WNDCLASSEX
    LOCAL msg:MSG
    LOCAL hwnd:HWND
    mov wc.cbSize,SIZEOF WNDCLASSEX
    mov wc.style, CS_HREDRAW or CS_VREDRAW
    mov wc.lpfnWndProc, OFFSET WndProc
    mov wc.cbClsExtra,NULL
    mov wc.cbWndExtra,NULL
    push hInstance
    pop wc.hInstance
    mov wc.hbrBackground,COLOR_WINDOW+1
    mov wc.lpszMenuName,NULL
    mov wc.lpszClassName,OFFSET ClassName
    invoke LoadIcon,NULL,IDI_APPLICATION
    mov wc.hIcon,eax
    mov wc.hIconSm,0
    invoke LoadCursor,NULL,IDC_ARROW
    mov wc.hCursor,eax
    invoke RegisterClassEx, addr wc
    INVOKE CreateWindowEx,NULL,ADDR ClassName,NULL,\
        WS_POPUP,CW_USEDEFAULT,\
        CW_USEDEFAULT,250,250,NULL,NULL,\
        hInstance,NULL
    mov hwnd,eax
    INVOKE ShowWindow, hwnd,SW_SHOWNORMAL
    .WHILE TRUE
        INVOKE GetMessage, ADDR msg,NULL,0,0
        .BREAK .IF (!eax)
        INVOKE TranslateMessage, ADDR msg
        INVOKE DispatchMessage, ADDR msg
    .ENDW
    mov eax,msg.wParam
    ret
ShowBitMap endp
WndProc proc hwnd:DWORD,uMsg:DWORD,wParam:DWORD,lParam:DWORD
    LOCAL ps:PAINTSTRUCT
    LOCAL hdc:HDC
    LOCAL hMemoryDC:HDC
    LOCAL hOldBmp:DWORD
    LOCAL bitmap:BITMAP
    LOCAL DlgHeight:DWORD
    LOCAL DlgWidth:DWORD
    LOCAL DlgRect:RECT
    LOCAL DesktopRect:RECT

    .if uMsg==WM_DESTROY
        .if hBitMap!=0
            invoke DeleteObject,hBitMap
        .endif
        invoke PostQuitMessage,NULL
    .elseif uMsg==WM_CREATE
        invoke GetWindowRect,hwnd,addr DlgRect
        invoke GetDesktopWindow
        mov ecx,eax
        invoke GetWindowRect,ecx,addr DesktopRect

```

```

        push 0
        mov eax,DlgRect.bottom
        sub eax,DlgRect.top
        mov DlgHeight,eax
        push eax
        mov eax,DlgRect.right
        sub eax,DlgRect.left
        mov DlgWidth,eax
        push eax
        mov eax,DesktopRect.bottom
        sub eax,DlgHeight
        shr eax,1
        push eax
        mov eax,DesktopRect.right
        sub eax,DlgWidth
        shr eax,1
        push eax
        push hWnd
        call MoveWindow
        invoke LoadBitmap,hInstance,addr BitmapName
        mov hBitMap,eax
        invoke SetTimer,hWnd,1,2000,NULL
        mov TimerID,eax
    .elseif uMsg==WM_TIMER
        invoke SendMessage,hWnd,WM_LBUTTONDOWN,NULL,NULL
        invoke KillTimer,hWnd,TimerID
    .elseif uMsg==WM_PAINT
        invoke BeginPaint,hWnd,addr ps
        mov hdc,eax
        invoke CreateCompatibleDC,hdc
        mov hMemoryDC,eax
        invoke SelectObject,eax,hBitMap
        mov hOldBmp,eax
        invoke GetObject,hBitMap,sizeof BITMAP,addr bitmap
        invoke StretchBlt,hdc,0,0,250,250,\
            hMemoryDC,0,0,bitmap.bmWidth,bitmap.bmHeight,SRCCOPY
        invoke SelectObject,hMemoryDC,hOldBmp
        invoke DeleteDC,hMemoryDC
        invoke EndPaint,hWnd,addr ps
    .elseif uMsg==WM_LBUTTONDOWN
        invoke DestroyWindow,hWnd
    .else
        invoke DefWindowProc,hWnd,uMsg,wParam,lParam
        ret
    .endif
    xor eax,eax
    ret
WndProc endp

End DllEntry

```

АНАЛИЗ

Сначала мы проанализируем код основной программы.

```

        invoke LoadLibrary,addr Libname
    .if eax!=NULL
        invoke FreeLibrary,eax
    .endif

```

Мы вызовем LoadLibrary, чтобы загрузить DLL "splash.dll". После этого выгружаем ее из памяти функцией FreeLibrary. LoadLibrary не возвратится, пока DLL не закончит свою инициализацию.

Это все, что делает основная программа. Интересующая нас часть находится в DLL.

```

    .if reason==DLL_PROCESS_ATTACH ; When the dll is loaded
        push hInst
        pop hInstance
        call ShowBitMap
    .endif

```

После загрузки DLL в память, Windows вызывает ее входную функцию с флагом DLL_PROCESS_ATTACH. Мы пользуемся этой возможностью, чтобы отобразить сплэш-экран. Во-первых, мы сохраняем хэндл DLL на будущее. Потом вызываем функцию ShowBitmap, которая выполняет главную работу. ShowBitmap регистрирует класс окна, создает окно и входит в цикл обработки сообщений. Следует обратить внимание на вызов CreateWindowEx:

```
INVOKE CreateWindowEx,NULL,ADDR ClassName,NULL,\
WS_POPUP,CW_USEDEFAULT,\
CW_USEDEFAULT,250,250,NULL,NULL,\
hInstance,NULL
```

Обратите внимание, что стиль окна WS_POPUP, что делает окно без бордюра и без заголовка. Мы также ограничиваем размер окна - 250x250.

Теперь, когда окно создано, в обработчике WM_CREATE мы передвигаем окно в центр экрана следующим кодом.

```
invoke GetWindowRect,hWnd,addr DlgRect
invoke GetDesktopWindow
mov ecx,eax
invoke GetWindowRect,ecx,addr DesktopRect
push 0
mov eax,DlgRect.bottom
sub eax,DlgRect.top
mov DlgHeight,eax
push eax
mov eax,DlgRect.right
sub eax,DlgRect.left
mov DlgWidth,eax
push eax
mov eax,DesktopRect.bottom
sub eax,DlgHeight
shr eax,1
push eax
mov eax,DesktopRect.right
sub eax,DlgWidth
shr eax,1
push eax
push hWnd
call MoveWindow
```

Мы получаем размеры десктопа и окон, а затем вычисляем координаты левого верхнего угла окна, чтобы оно было в центре.

```
invoke LoadBitmap,hInstance,addr BitmapName
mov hBitMap,eax
invoke SetTimer,hWnd,1,2000,NULL
mov TimerID,eax
```

Затем мы загружаем битмап из ресурса функцией LoadBitmap и создаем таймер, указывая в качестве его ID 1, а в качестве временного интервала 2 секунды. Таймер будет посылать сообщения WM_TIMER окну каждый две секунды.

```
.elseif uMsg==WM_PAINT
invoke BeginPaint,hWnd,addr ps
mov hdc,eax
invoke CreateCompatibleDC,hdc
mov hMemoryDC,eax
invoke SelectObject,eax,hBitMap
mov hOldBmp,eax
invoke GetObject,hBitMap,sizeof BITMAP,addr bitmap
invoke StretchBlt,hdc,0,0,250,250,\
hMemoryDC,0,0,bitmap.bmWidth,bitmap.bmHeight,SRCCOPY
invoke SelectObject,hMemoryDC,hOldBmp
invoke DeleteDC,hMemoryDC
invoke EndPaint,hWnd,addr ps
```

Когда окно получит сообщение WM_PAINT, она создаст DC в памяти, выберет в него битмап, получит размер битмапа функцией GetObject, а затем поместит битмап на окно, вызвав StretchBlt, которая действует как BitBlt, но адаптирует битмап к желаемым размерам. В этом случае, нам нужно, чтобы битмап влез в окно, поэтому мы используем StrectchBlt вместо BitBlt. Мы удаляем созданный в памяти DC.

```
.elseif uMsg==WM_LBUTTONDOWN  
    invoke DestroyWindow,hWnd
```

Пользователя бы раздражало, если бы ему пришлось бы ждать, пока сплэш-экран не исчез. Мы можем предоставить пользователю выбор. Когда он кликнет на сплэш-экране, тот исчезнет. Вот почему нам нужно обрабатывать сообщение WM_LBUTTONDOWN. Когда мы получим это сообщение, окно будет уничтожено вызовом DestroyWindow.

```
.elseif uMsg==WM_TIMER  
    invoke SendMessage,hWnd,WM_LBUTTONDOWN,NULL,NULL  
    invoke KillTimer,hWnd,TimerID
```

Если пользователь решит подождать, сплэш-экран исчезнет, когда пройдет заданный период времени (в нашем примере, это две секунды). Мы можем сделать это обработкой сообщения WM_TIMER. После получения этого сообщения, мы закрываем окно, послав ему сообщение WM_LBUTTONDOWN, чтобы избежать повторения кода. Таймер нам больше не нужен, поэтому мы уничтожаем его KillTimer.

Когда окно будет закрыто, DLL возвратит контроль основной программе.

Win32 API. Урок 27. Тултип-контрол

Автор: lczelion, пер. Aquila

Дата: 2002-05-27

Мы изучим контроль tooltip. Что это такое, как его создать и как им пользоваться.

Сырец довнлоад здесь (находится в архиве wasm_files.zip: files/recovered/1001027/tut27.zip).

ТЕОРИЯ

Тултип - это маленькая прямоугольное окно, которое отображается, когда курсор мыши находится над какой-то определенной областью. Окно тултипа содержит текст, заданный программистом. В этом отношении тултип играет ту же роль, что и окно статуса, но оно исчезает, когда пользователь кликает или убирает курсор мыши из заданной области. Вы, вероятно, знакомы с тултипами, ассоциированные с кнопками тулбара. Эти "тултипы" - одно из удобств, предоставляемых тулбаром. Если вам нужны тултипы для других окон/контролов, вам необходимо создать собственный тултип контрол.

Теперь, когда вы знаете, что такое тултип, давайте перейдем к тому, как мы можем создать и использовать его. Ниже расписаны шаги:

- Создать тултип-контрол функцией `CreateWindowEx`.
- Определить регион, в котором он будет отслеживать передвижения мыши.
- Передать регион тултип-контролю.
- Передавать сообщения от мыши в указанном регионе тултип-контролю (этот шаг зависит от заданных флагов).

Ниже мы детально проанализируем каждый шаг.

Создание тултипа

Тултип - это common control. Поэтому вам необходимо где-нибудь в программе вызвать функцию `InitCommonControls`, чтобы MASM подлинковал к выходному экзешнику `comctl32.dll`. Вы создаете тултип с помощью `CreateWindowEx`. Это будет выглядеть примерно так:

```
.data
TooltipClassName db "Tooltips_class32",0
.code
.....
invoke InitCommonControls
invoke CreateWindowEx, NULL, addr TooltipClassName, NULL,
TIS_ALWAYSSTIP, CW_USEDEFAULT, CW_USEDEFAULT, CW_USEDEFAULT,
CW_USEDEFAULT, NULL, NULL, hInstance, NULL
```

Обратите внимание на стиль окна: `TIS_ALWAYSSTIP`. Этот стиль указывает, что тултип будет показываться, когда курсор мыши будет находиться над заданной областью вне зависимости от статуса окна. То есть, если вы будете использовать этот фла, тултип будет появляться (когда курсор мыши будет находиться над определенной областью), даже если окно, с которым ассоциирован тултип, неактивно.

Вам не нужно задавать стили `WS_POPUP` и `WS_EX_TOOLWINDOW`, потому что тултип определяет их автоматически. Вам также не нужно указывать координаты, ширину и высоту тултипа: он сам рассчитывает свои характеристики, поэтому в качестве всех четырех параметров мы указываем `CW_USEDEFAULT`. Оставшиеся параметры не играют роли.

Определение tool'ов

Тултип создается, но не отображается сразу. Нам нужно, чтобы он отображался только над определенной областью. Теперь пришло время задать ее. Мы называем такую область 'tool'. Tool - это прямоугольная область клиентской части окна, в пределах которой тултип будет отслеживать передвижение мыши. Прямоугольная область может покрывать всю клиентскую часть окна или только некоторую долю от нее. Поэтому мы можем поделить 'tool' на два типа: один - это, когда в качестве tool'a выступает целая клиентская область окна, а другой - прямоугольная часть клиентской области окна. Оба типа находят свое применение. Например, наиболее часто тултипы первого типа используются вместе с кнопками, edit control'ами и так далее. Вам не нужно указывать координаты и размерность tool'a: предполагается, что будет задействована вся клиентская область. Tool'ы второго типа полезны, когда вы хотите поделить клиентскую часть окна на несколько регионов без использования дочерних окон. В этом случае вам будет необходимо задать координату верхнего левого угла, ширину и высоту tool'a.

Вы определяете характеристики tool'a в структуре TOOLINFO, которая имеет следующее определение:

```

TOOLINFO STRUCT
    cbSize          DWORD    ?
    uFlags          DWORD    ?
    hWnd *         DWORD    ?
    uId             DWORD    ?
    rect           RECT      <>
    hInst *        DWORD    ?
    lpszText       DWORD    ?
    lParam         LPARAM    ?
TOOLINFO*ENDS

```

- cbSize - размер структуры TOOLINFO. Вы должны заполнить этот параметр. Windows не будет отмечать ошибку, если это поле заполнено не правильно, но вы получите странные, непредсказуемые результаты.
- uFlags - битовые флаги определяют характеристики tool'a.
- TTF_IDISHWND - "ID - это hWnd". Если вы укажете этот флаг, вы должны заполнить параметр uld хэндлом окна, который вы хотите использовать. Если вы не укажете этот флаг, это будет означать, что вы хотите использовать второй тип tool'a. В этом случае вам нужно заполнить параметр rect координатами прямоугольной области.
- TTF_CENTERTIP - обычно окно тултипа появляется справа и ниже курсора мыши. Если вы укажете этот флаг, окно тултипа появится ниже tool'a и отцентрируется независимо от позиции мышиного курсора.
- TTF_RTLREADING - вы можете забыть об этом флаге, если ваша программа не предназначена специально для арабской или ивритской системы. Этот флаг отображает текст тултипа справа налево. Не работает под другими системами.
- TTF_SUBCLASS - если вы используете флаг, это означает, что вы указываете тултип-контролю subclassировать окно tool'a, чтобы тултип мог интерпретировать сообщения от мыши, которые посылаются окну. Этот флаг очень удобен. Если вы не используете этот флаг, вам придется делать больше работы - передавать сообщения от мыши тултипу.
- hWnd - Хэндл окна, который содержит tool. Если вы указали флаг TTF_IDISHWND, это поле игнорируется, так как Windows будет использовать значение uld в качестве хэндла окна. Вам нужно заполнить это поле, если:
 - Вы не устанавливали флаг TTF_IDISHWND
 - Вы указываете значение LPSTR_TEXTCALLBACK в параметре lpszText. Это значение указывает тултипу, что когда ему необходимо отобразить свое окно, он должен уведомить об этом окно, которое содержит tool. Это вид динамического обновления текста тултипа. Если вы хотите изменять динамически текст тултипа, вам следует LPSTR_TEXTCALLBACK в качестве значения LPSTR_TEXTCALLBACK. Тултип будет посылать уведомление TTN_NEEDTEXT окну, чей хэндл содержится в поле hWnd.

- `uld` - это поле может иметь одно из двух значений, в зависимости от того, содержит ли `uFlags` флаг `IIF_IDISHWND`.
- Определяемое приложением ID tool'a, если флаг `TTF_IDISHWND`. Так как это означает, что вы используете tool, покрывающее только часть клиентской области, то логично, что вы можете иметь несколько tool'ов на одной клиентской области (без пересечений). Тултип-контрол должен иметь какой-то путь отличать их друг от друга. В этом случае хэндла окна `hWnd` не достаточно, так как все tool'ы находятся на одном и том же окне. Определяемые приложением ID служат именно для этой цели. ID может быть любым значением, главное, чтобы оно было уникально по отношению к другим ID.
- Хэндл окна, чья клиентская область полностью используется в качестве tool'a, если указан флаг `TTF_IDISHWND`. Вы можете удивиться, почему это поле используется для хранения хэндла окна, если есть `hWnd`? Ответ следующий: поле `hWnd` уже может быть заполнено, если в параметре `lpszText` указано значение `LPSTR_TEXTCALLBACK`. Окно, которое ответственно за предоставление текста тултипа, и окно, которое содержит tool, могут быть не одним и тем же.
- `rect` - структура `RECT`, которая указывает размерность tool'a. Эта структура определяет прямоугольник относительно верхнего левого угла клиентской области окна, указанного в параметре `hWnd`. То есть, вы должны заполнить эту структуру, если вы хотите указать tool, который покрывает только часть клиентской области. Тултип-контрол проигнорирует это поле, если вы укажете флаг `TTF_IDISHWND` (вы хотите использовать в качестве tool'a целое окно).
- `hInst` - это хэндл процесса, содержащий ресурс строки, которая будет использована в качестве текста, если значение `lpszText` равно ID строкового ресурса. Это может вас несколько смутить. Прочтите сначала описание параметра `lpszText`, и вы поймете, для чего используется это поле. Тултип-контрол игнорирует это поле, если `lpszText` не содержит ID ресурса.
- `lpszText` - это поле имеет несколько значений:
 - Если вы укажете в этом поле значение `LPSTR_TEXTCALLBACK`, тултип будет посылать уведомительное сообщение `TTN_NEEDTEXT` окну, которое идентифицируется хэндлом поля `hWnd`, чтобы то предоставило тултипу текстовую строку. Это наиболее динамичный метод обновления текста тултипа: вы можете менять его каждый раз, когда отображается окно тултипа.
 - Если вы укажете в этом поле ID строкового ресурса, тултип, когда ему потребуется отобразить текст в своем окне, будет искать строку в таблице строк процесса, заданного параметром `hInst`. Тултип-контрол идентифицирует ID ресурса следующим образом: так как ID ресурса - это 16-битное значение, верхнее слово этого поля всегда будет равно нулю. Этот метод полезен, если вы хотите портировать вашу программу на другие языки. Так как строковый ресурс определен в файле определения ресурсов, вам не нужно модифицировать исходный код. Вам только нужно изменить таблицу строк и текст тултипа изменится без риска внесения ошибок в программу.
 - Если значение в этом поле не равно `LPSTR_TEXTCALLBACK` и верхнее слово не равно нулю, тултип-контрол интерпретирует значение как указатель на текстовую строку, которая будет использована в качестве текста тултипа. Этот метод самый простой, но наименее гибкий.

Резюме: вы должны заполнить структуру `TOOLINFO` и передать ее тултипу. Эта структура задаст характеристики tool'a.

Регистрация tool'a

После того, как вы заполнили структуру `TOOLINFO`, вы должны передать ее тултипу. Тултип может обслуживать много tool'ов, поэтому обычно одно тултипа хватает на все окно. Чтобы зарегистрировать tool, вы посылаете тултипу сообщение `TTM_ADDTOOL`. `wParam` не используется, а `lParam` должен содержать адрес структуры `TOOLINFO`.

```
.data?
ti TOOLINFO <>
.....
.code
.....
.....
```

```
invoke SendMessage, hwndTooltip, TTM_ADDTOOL, NULL, addr ti
```

SendMessage возвратит TRUE, если tool был успешно зарегистрирован тултипом или FALSE в обратном случае. Вы можете удалить tool сообщением TTM_DELTOOL.

Передача сообщений от мыши тултипу

Когда вышеописанные шаги выполнены, тултип имеет всю необходимую информацию о том, в какой области он должен отслеживать сообщения мыши и какой текст он должен отображать. Единственное, что отсутствует - это триггер. Подумайте: область, указанная в качестве tool'a находится на клиентской части другого окна. Как может тултип перехватить сообщения от мыши для этого окна? Необходимо, чтобы он мог измерить количество времени, которое курсор мыши находится над tool'ом, чтобы вовремя отобразить окно тултипа. Есть два метода, чтобы достичь этой цели, один требует помощи со стороны окна, которое tool, а другой этого не требует.

Окно, которое содержит tool, должно переправлять сообщения от мыши тултипу с помощью сообщения TTM_RELAYEVENT. IParam должен содержать адрес структуры MSG, содержащую сообщение от мыши. Тултип обрабатывает только следующие сообщения от мыши:

- WM_LBUTTONDOWN
- WM_MOUSEMOVE
- WM_LBUTTONUP
- WM_RBUTTONDOWN
- WM_MBUTTONDOWN
- WM_RBUTTONUP
- WM_MBUTTONUP

Все другие сообщения игнорируются. Таким образом, в процедуре окна, содержащего tool, должен быть обработчик вроде следующего:

```
WndProc proc hwnd:DWORD, uMsg:DWORD, wParam:DWORD, lParam:DWORD
.....
if uMsg==WM_CREATE
.....
elseif uMsg==WM_LBUTTONDOWN || uMsg==WM_MOUSEMOVE || \
uMsg==WM_LBUTTONUP || uMsg==WM_RBUTTONDOWN || \
uMsg==WM_MBUTTONDOWN || uMsg==WM_RBUTTONUP || \
uMsg==WM_MBUTTONUP
invoke SendMessage, hwndTooltip, TTM_RELAYEVENT, NULL, addr msg
.....
```

Вы можете указать флаг TTF_SUBCLASS в параметре uFlags структуры TOOLINFO. Этот флаг указывает тултипу сабклассировать окно, которое содержит tool, чтобы перехватывать сообщения от мыши без участия сабклассированного окна. Этот метод проще использовать и он требует меньше усилий, так как тултип берет всю обработку сообщений на себя.

Вот и все. Теперь ваш тултип полностью функционален. Есть несколько полезных тултиповых сообщений, о которых вас следует знать.

- TTM_ACTIVATE - если вы хотите включать/выключать контрол динамически, это сообщение для вас. Если wParam равен TRUE, тултип включается, если wParam равен FALSE, тултип выключается. Тултип включается автоматически, как только он был создан, поэтому вам не надо посылать сообщение, чтобы активировать его.
- TTM_GETTOOLINFO и TTM_SETTOOLINFO. Если вы хотите получить/изменить значения в структуре TOOLINFO после того, как она была отправлена тултипу, используйте данное сообщение. Вам потребуется указать tool, чьи характеристики вы хотите изменить, с помощью верных uld и hwnd. Если вы хотите изменить только параметр rect, используйте сообщение TTM_NEWTOOLRECT. Если вам нужно только изменить текст тултипа, используйте TTM_UPDATATIPTEXT.

- TTM_SETDELAYTIME. С помощью этого сообщения вы можете задать временную задержку, которую будет использовать тултип.

ПРИМЕР

Следующий пример - это простое диалоговое окно с двумя кнопками. Клиентская область диалогового окна поделена на 4 области: верхняя левая, верхняя правая, нижняя левая и нижняя правая. Каждая область указана как tool с собственным текстом. Две кнопки также имеют свои собственные тексты подсказок.

```
.386
.model flat,stdcall
option casemap:none
include \masm32\include\windows.inc
include \masm32\include\kernel32.inc
include \masm32\include\user32.inc
include \masm32\include\comctl32.inc
includelib \masm32\lib\comctl32.lib
includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib
DlgProc proto :DWORD,:DWORD,:DWORD,:DWORD
EnumChild proto :DWORD,:DWORD
SetDlgToolArea proto :DWORD,:DWORD,:DWORD,:DWORD,:DWORD
.const
IDD_MAINDIALOG equ 101
.data
ToolTipsClassName db "Tooltips_class32",0
MainDialogText1 db "This is the upper left area of the dialog",0
MainDialogText2 db "This is the upper right area of the dialog",0
MainDialogText3 db "This is the lower left area of the dialog",0
MainDialogText4 db "This is the lower right area of the dialog",0
.data?
hwndTool dd ?
hInstance dd ?
.code
start:
    invoke GetModuleHandle,NULL
    mov hInstance,eax
    invoke DialogBoxParam,hInstance,IDD_MAINDIALOG,NULL,addr
    DlgProc,NULL
    invoke ExitProcess,eax

DlgProc proc hDlg:DWORD,uMsg:DWORD,wParam:DWORD,lParam:DWORD
    LOCAL ti:TOOLINFO
    LOCAL id:DWORD
    LOCAL rect:RECT
    .if uMsg==WM_INITDIALOG
        invoke InitCommonControls
        invoke CreateWindowEx,NULL,ADDR ToolTipsClassName,NULL,\
            TTS_ALWAYSTIP,CW_USEDEFAULT,\
            CW_USEDEFAULT,CW_USEDEFAULT,CW_USEDEFAULT,NULL,NULL,\
            hInstance,NULL
        mov hwndTool,eax
        mov id,0
        mov ti.cbSize,sizeof TOOLINFO
        mov ti.uFlags,TTF_SUBCLASS
        push hDlg
        pop ti.hWnd
        invoke GetWindowRect,hDlg,addr rect
        invoke SetDlgToolArea,hDlg,addr ti,addr
MainDialogText1,id,addr rect
        inc id
        invoke SetDlgToolArea,hDlg,addr ti,addr
MainDialogText2,id,addr rect
        inc id
        invoke SetDlgToolArea,hDlg,addr ti,addr
MainDialogText3,id,addr rect
        inc id
        invoke SetDlgToolArea,hDlg,addr ti,addr
```

```

MainDialogText4,id,addr rect
    invoke EnumChildWindows,hDlg,addr EnumChild,addr ti
    .elseif uMsg==WM_CLOSE
        invoke EndDialog,hDlg,NULL
    .else
        mov eax,FALSE
        ret
    .endif
    mov eax,TRUE
    ret
DlgProc endp

EnumChild proc uses edi hwndChild:DWORD,lParam:DWORD
    LOCAL buffer[256]:BYTE
    mov edi,lParam
    assume edi:ptr TOOLINFO
    push hwndChild
    pop [edi].uId
    or [edi].uFlags,TF_IDISHWND
    invoke GetWindowText,hwndChild,addr buffer,255
    lea eax,buffer
    mov [edi].lpszText,eax
    invoke SendMessage,hwndTool,TTM_ADDTOOL,NULL,edi
    assume edi:nothing
    ret
EnumChild endp

SetDlgToolArea proc uses edi esi
hDlg:DWORD,lpti:DWORD,lpText:DWORD,id:DWORD,lprect:DWORD
    mov edi,lpti
    mov esi,lprect
    assume esi:ptr RECT
    assume edi:ptr TOOLINFO
    .if id==0
        mov [edi].rect.left,0
        mov [edi].rect.top,0
        mov eax,[esi].right
        sub eax,[esi].left
        shr eax,1
        mov [edi].rect.right,eax
        mov eax,[esi].bottom
        sub eax,[esi].top
        shr eax,1
        mov [edi].rect.bottom,eax
    .elseif id==1
        mov eax,[esi].right
        sub eax,[esi].left
        shr eax,1
        inc eax
        mov [edi].rect.left,eax
        mov [edi].rect.top,0
        mov eax,[esi].right
        sub eax,[esi].left
        mov [edi].rect.right,eax
        mov eax,[esi].bottom
        sub eax,[esi].top
        mov [edi].rect.bottom,eax
    .elseif id==2
        mov [edi].rect.left,0
        mov eax,[esi].bottom
        sub eax,[esi].top
        shr eax,1
        inc eax
        mov [edi].rect.top,eax
        mov eax,[esi].right
        sub eax,[esi].left
        shr eax,1
        mov [edi].rect.right,eax
        mov eax,[esi].bottom
        sub eax,[esi].top

```

```

        mov [edi].rect.bottom,eax
    .else
        mov eax,[esi].right
        sub eax,[esi].left
        shr eax,1
        inc eax
        mov [edi].rect.left,eax
        mov eax,[esi].bottom
        sub eax,[esi].top
        shr eax,1
        inc eax
        mov [edi].rect.top,eax
        mov eax,[esi].right
        sub eax,[esi].left
        mov [edi].rect.right,eax
        mov eax,[esi].bottom
        sub eax,[esi].top
        mov [edi].rect.bottom,eax
    .endif
    push lpText
    pop [edi].lpSzText
    invoke SendMessage,hwndTool,TTM_ADDTOOL,NULL,lpti
    assume edi:nothing
    assume esi:nothing
    ret
SetDlgToolArea endp
end start

```

АНАЛИЗ

После того, как создано основное диалоговое окно, мы создаем тултип-контрол функцией `CreateWindowEx`.

```

    invoke InitCommonControls
    invoke CreateWindowEx,NULL,ADDR ToolTipsClassName,NULL,\
        TTS_ALWAYSTIP,CW_USEDEFAULT,\
        CW_USEDEFAULT,CW_USEDEFAULT,CW_USEDEFAULT,NULL,NULL,\
        hInstance,NULL
    mov hwndTool,eax

```

После этого мы переходим к определению четырех tool'ов для каждого угла диалогового окна.

```

    mov id,0          ; used as the tool ID
    mov ti.cbSize,sizeof TOOLINFO
    mov ti.uFlags,TTF_SUBCLASS ; tell the tooltip control to subclass the dialog window.
    push hDlg
    pop ti.hwnd        ; handle to the window that contains the tool
    invoke GetWindowRect,hDlg,addr rect ; получаем размерность клиентской
                                        ; области
    invoke SetDlgToolArea,hDlg,addr ti,addr MainDialogText1,id,addr rect

```

Мы инициализируем члены структуры `TOOLINFO`. Заметьте, что мы хотим поделить клиентскую область на 4 tool'a, поэтому нам нужно знать размерность клиентской области. Это то, для чего мы вызываем `GetWindowRect`. Мы не хотим передавать сообщения мыши тултипу, поэтому мы указываем флаг `TTF_SUBCLASS`.

`SetDlgToolArea` - это функция, которая высчитывает координаты прямоугольной области каждого tool'a и регистрирует tool в тултипе. Я не хочу вдаваться в подробности относительно этого, достаточно сказать, что она делит клиентскую область на 4 области с одними и теми же размерами. Затем она посылает сообщение `TTM_ADDTOOL` тултипу, передавая адрес структуры `TOOLINFO` в `lParam`.

```

    invoke SendMessage,hwndTool,TTM_ADDTOOL,NULL,lpti

```

После того, как все 4 tool'a зарегистрированы, мы можем перейти к кнопкам на диалоговом окне. Мы можем обрабатывать каждую кнопку с помощью ее ID, но это утомительно. Вместо этого мы используем `EnumChildWindows` API, чтобы перечислить все контролы на диалоговом окне и затем

зарегистрировать для каждого из них подсказку. EnumChildWindows имеет следующий синтаксис:

```
EnumChildWindows proto hWnd:DWORD, lpEnumFunc:DWORD, lParam:DWORD
```

hWnd - хэндл родительского окна. lpEnumFunc - адрес функции EnumChildProc, которая будет вызываться для каждого перечисленного контрола. lParam - заданное приложением значение, которое будет передано EnumChildProc. У этой функции следующее определение:

```
EnumChildProc proto hWndChild:DWORD, lParam:DWORD
```

hWndChild - хэндл контрола, найденного EnumChildWindows. lParam - это тоже значение, что вы передали EnumChildWindow. В нашем примере мы вызываем EnumChildWindows следующим образом:

```
invoke EnumChildWindows,hDlg,addr EnumChild,addr ti
```

Мы передаем адрес структуры TOOLINFO в параметре lParam, потому что мы будем регистрировать подсказки для каждого дочерний контрол в функции EnumChild. Если мы не будем использовать данный метода, нам придется объявить глобальную переменную, чтобы предотвратить баги.

Когда мы вызываем EnumchildWindows, Windows перечислит дочерние контролы нашего диалогового окна и вызовет для каждого из них функцию Enumchild. То есть, если наше диалоговое окно имеет два контрола, EnumChild будет вызван дважды.

Функция EnumChild заполнит соответствующие поля структуры TOOLINFO и зарегистрирует tool.

```
EnumChild proc uses edi hWndChild:DWORD,lParam:DWORD
    LOCAL buffer[256]:BYTE
    mov edi,lParam
    assume edi:ptr TOOLINFO
    push hWndChild
    pop [edi].uId    ; we use the whole client area of the control as the tool
    or [edi].uFlags,TTF_IDISHWND
    invoke GetWindowText,hWndChild,addr buffer,255
    lea eax,buffer    ; use the window text as the tooltip text
    mov [edi].lpszText,eax
    invoke SendMessage,hWndTool,TTM_ADDTOOL,NULL,edi
    assume edi:nothing
    ret
EnumChild endp
```

Заметьте, что в этом случае мы используем другой тип tool'ов, покрывающий всю клиентскую область окна. Поэтому нам нужно заполнить поле uID хэндлом окна, которое содержит tool. Также мы указываем флаг TTF_IDISHWND в параметре uFlags.

Win32 API. Урок 28. Win32 Debug API I

Автор: lczelion, пер. Aquila

Дата: Не указана

В этом tutorialе вы изучите, какие примитивные отладочные средства предлагает разработчику Win32. Вы узнаете, как отладить процесс, когда вы закончите читать этот tutorial.

Скачайте пример здесь (находится в архиве wasm_files.zip: files/recovered/1001028/tut28.zip).

ТЕОРИЯ

Win32 имеет несколько функций API, которые позволяют программисту использовать некоторые возможности отладчика. Они называются Win32 Debug API. С помощью них вы можете:

- Загрузить программу и подсоединиться к запущенной программе для отладки
- Получить низкоуровневую информацию о программе, которую вы отлаживаете, например, ID процесса, адрес входной точки, image base и так далее.
- Быть уведомленным о событиях, связанных с отладкой, например, когда процесс запускается/заканчивает выполнение
- Изменять отлаживаемый процесс/ветвь

Короче говоря, с помощью этих API вы можете написать простой отладчик. Так как это объемный предмет, я поделю его на несколько частей: этот tutorial будет первой частью. Я объясню основные концепции, касающиеся Win32 Debug API, здесь.

Этапы использования Win32 Debug API следующие:

- Создаем или присоединяемся к запущенному процессу. Это первый шаг. Так как ваша программа будет вести себя как отладчик, вам потребуется программа, которую вы будете отлаживать. Вы можете сделать следующее:
- Создать специальный процесс для отладки с помощью CreateProcess. Чтобы создать процесс для отладки, вы можете указать флаг DEBUG_PROCESS. Этот флаг говорит Windows, что мы хотим отлаживать процесс. Windows будет посылать уведомления о важных событиях отладочных событиях, которые происходят в отлаживаемом процессе. Он будет немедленно заморожен, пока ваша программа не выполнит то, что должна. Если отлаживаемый процесс создаст дочерние процессы, Windows также будет посылать уведомления о происходящих в них отладочных событиях. Обычно это нежелательно, поэтому это можно отключить, указав кроме флага DEBUG_PROCESS флаг DEBUG_ONLY_THIS_PROCESS.
- Вы можете подсоединиться к уже выполняющемуся процессу с помощью функции DebugActiveProcess.
- Ждем отладочные события. Когда вы создаете отлаживаемый процесс или присоединяетесь к нему, он замораживается, пока ваша программа не вызовет WaitForDebugEvent. Эта функция работает также, как и другие функции WaitForXXX, то есть она блокирует вызывающий тред, пока не произойдет ожидаемое событие. В данном случае она ожидает отладочных событий, которые должны посылаются Windows. Давайте посмотрим ее определение:

```
WaitForDebugEvent proto lpDebugEvent:DWORD, dwMilliseconds:DWORD
```

- lpDebugEvent - это адрес структуры DEBUG_EVENT, которая должна быть заполнена информацией об отладочном событии, которое происходит внутри отлаживаемого процесса.
- dwMilliseconds - это временной интервал в миллисекундах, в течении которого эта функция будет ожидать отладочного события. Если этот период истечет и не произойдет никакого отладочного события, WaitForDebugEvent возвратит управления вызвавшему ее треду. С другой стороны, если

- вы укажете константу INFINITE, функция не возвратится, пока не произойдет отладочное событие.
- Теперь давайте проанализируем структуру DEBUG_EVENT более подробно.

```
DEBUG_EVENT STRUCT

    dwDebugEventCode dd ?
    dwProcessId dd ?
    dwThreadId dd ?
    u DEBUGSTRUCT <>

DEBUG_EVENT ENDS
```

dwDebugEventCode содержит значение, которое указывает тип произошедшего отладочного события. Кратко говоря, есть много типов событий, ваша программа должна проверять значение в этом поле, чтобы знать, какого типа произошедшее событие и адекватно реагировать. Возможные значения следующие:

- CREATE_PROCESS_DEBUG_EVENT - процесс создан. Это событие будет послано, когда отлаживаемый процесс только что создан (и еще не запущен), или когда ваша программа присоединяет себя к запущенному процессу с помощью DebugActiveProcess. Это первое событие, которое получит ваша программа.
- EXIT_PROCESS_DEBUG_EVENT - процесс прекращает выполнение.
- CREATE_THREAD_DEBUG_EVENT - в отлаживаемом процессе создан новый тред. Заметьте, что вы не получите это уведомление, когда будет создан основной тред отлаживаемой программы.
- EXIT_THREAD_DEBUG_EVENT - тред в отлаживаемом процессе прекращает выполнение. Ваша программа не получит это сообщение, если прекратит выполняться основная ветвь отлаживаемого процесса. Вы можете считать, что основная ветвь отлаживаемого процесса эквивалентна самому процессу. Таким образом, когда ваша программа видит CREATE_PROCESS_DEBUG_EVENT, это все равно, что CREATE_THREAD_DEBUG_EVENT по отношению к основному треду.
- LOAD_DLL_DEBUG_EVENT - отлаживаемый процесс загружает DLL. Вы получите это событие, когда PE-загрузчик установит связь с DLL'ями и когда отлаживаемый процесс вызовет LoadLibrary.
- UNLOAD_DLL_DEBUG_EVENT - в отлаживаемом процессе выгружена DLL.
- EXCEPTION_DEBUG_EVENT - в отлаживаемом процессе возникло исключение. Важно: это событие будет случиться, как только отлаживаемый процесс выполнит свою первую инструкцию. Этим исключением является отладочный 'break' (int 3h). Когда вы хотите, чтобы отлаживаемый процесс продолжил выполнение, вызовите ContinueDebugEvent с флагом DBG_CONTINUE. Не используйте DBG_EXCEPTION. Также не используйте DBG_EXCEPTION_NOT_HANDLED, иначе отлаживаемый процесс откажется выполняться дальше под NT (под Win98 все работает прекрасно).
- OUTPUT_DEBUG_STRING_EVENT - это событие генерируется, когда отлаживаемый процесс вызывает функцию DebugOutputString, чтобы послать строку с сообщением вашей программе.
- RIP_EVENT - произошла системная ошибка отладки.

dwProcessId и dwThreadId - это ID процесса и треда в этом процессе, где произошло отладочное событие. Помните, что если вы использовали CreateProcess для загрузки отлаживаемого процесса, эти ID вы получите через структуру PROCESS_INFO. Вы можете использовать эти значения, чтобы отличить отладочные события, произошедшие в отлаживаемом процессе, от событий, произошедших в дочерних процессах.

u - это объединение, которое содержит дополнительную информацию об отладочном событии. Это может быть одна из следующих структур, в зависимости от dwDebugEventCode.

- CREATE_PROCESS_DEBUG_EVENT - CREATE_PROCESS_DEBUG_INFO-структура под названием CreateProcessInfo
- EXIT_PROCESS_DEBUG_EVENT - EXIT_PROCESS_DEBUG_INFO-структура под названием ExitProcess

- `CREATE_THREAD_DEBUG_EVENT` - `CREATE_THREAD_DEBUG_INFO`-структура под названием `CreateThread`
- `EXIT_THREAD_DEBUG_EVENT` - `EXIT_THREAD_DEBUG_EVENT`-структура под названием `ExitThread`
- `LOAD_DLL_DEBUG_EVENT` - `LOAD_DLL_DEBUG_INFO`-структура под названием `LoadDll`
- `UNLOAD_DLL_DEBUG_EVENT` - `UNLOAD_DLL_DEBUG_INFO`-структура под названием `UnloadDll`
- `EXCEPTION_DEBUG_EVENT` - `EXCEPTION_DEBUG_INFO`-структура под названием `Exception`
- `OUTPUT_DEBUG_STRING_EVENT` - `OUTPUT_DEBUG_STRING_INFO`-структура под названием `DebugString`
- `RIP_EVENT` - `RIP_INFO`-структура под названием `RipInfo`
- В этом tutorialе я не буду вдаваться в детали относительно всех структур, здесь будет рассказано только о `CREATE_PROCESS_DEBUG_INFO`. Предполагается, что наша программа вызывает `WaitForDebugEvent` и возвращает управление. Первая вещь, которую мы должны сделать, это проверить значение `dwDebugEventCode`, чтобы узнать тип отлаживаемого события в отлаживаемом процессе. Например, если значение `dwDebugEventCode` равно `CREATE_PROCESS_DEBUG_EVENT`, вы можете проинтерпретировать значение и как `CreateProcessInfo` и получить к ней доступ через `u.CreateProcessInfo`.
- Делайте все, что нужно сделать в ответ на это событие. Когда `WaitForDebugEvent` возвратит управление, это будет означать, что произошло отлаживаемое событие или истек заданный временной интервал. Ваша программа должна проверить значение `dwDebugEventCode`, чтобы отреагировать на него соответствующим образом. В этом отношении это напоминает обработку Windows-сообщений: вы выбираете какие обрабатывать, а какие игнорировать.
- Пусть отлаживаемый процесс продолжит выполнение. Когда вы закончите обработку события, вам нужно пнуть процесс, чтобы он продолжил выполнение. Вы можете сделать это с помощью `ContinueDebugEvent`.

```
ContinueDebugEvent proto dwProcessId:DWORD, dwThreadId:DWORD, dwContinueStatus:DWORD
```

Эта функция продолжает выполнение треда, который был заморожен произошедшим отладочным событием.

`dwProcessId` и `dwThreadId` - это процесса и треда в нем, который должен быть продолжен. Обычно эти значения вы получаете из структуры `DEBUG_EVENT`.

`dwContinueStatus` каким образом продолжить тред, который сообщил об отлаживаемом событии. Есть два возможных значения: `DBG_CONTINUE` и `DBG_EXCEPTION_NOT_HANDLED`. Практически для всех отладочных событий они значат одно: продолжить выполнение треда. Исключение составляет событие `EXCEPTION_DEBUG_EVENT`. Если тред сообщает об этом событии, значит в нем случилось исключение. Если вы указали `DBG_CONTINUE`, тред проигнорирует собственный обработчик исключения и продолжит выполнение. В этом случае ваша программа должна сама определить и ответить на исключение, прежде, чем позволить треду продолжить выполнение, иначе исключение произойдет еще раз и еще и еще... Если вы указали `DBG_EXCEPTION_NOT_HANDLED`, ваша программа указывает Windows, что она не будет обрабатывать исключения: Windows должна использовать обработчик исключений по умолчанию.

- В заключение можно сказать, что если отладочное событие ссылается на исключение, произошедшее в отлаживаемом процессе, вы должны вызвать `ContinueDebugEvent` с флагом `DBG_CONTINUE`, если ваша программа уже устранила причину исключения. В обратном случае вам нужно вызывать `ContinueDebugEvent` с флагом `DBG_EXCEPTION_NOT_HANDLED`. Только в одном случае вы должны всегда использовать флаг `DBG_CONTINUE`: первое событие `EXCEPTION_DEBUG_EVENT`, в параметре `ExceptionCode` которого содержится значение `EXCEPTION_BREAKPOINT`. Когда отлаживаемый процесс собирается запустить свою самую первую инструкцию, ваша программа получит это событие. Фактически это отладочный останов (int 3h). Если вы сделаете вызов `ContinueDebugEvent` с `DBG_EXCEPTION_NOT_HANDLED`, Windows NT

откажется запускать отлаживаемый процесс. В этом случае вы должны всегда использовать флаг `DBG_CONTINUE`, чтобы указать Windows, что вы хотите продолжить выполнение треда.

- Делается бесконечный цикл, пока отлаживаемый процесс не завершится. Цикл выглядит примерно так:

```
.while TRUE
    invoke WaitForDebugEvent, addr DebugEvent, INFINITE
    .break .if DebugEvent.dwDebugEventCode==EXIT_PROCESS_DEBUG_EVENT

    invoke ContinueDebugEvent, DebugEvent.dwProcessId, \
        DebugEvent.dwThreadId, DBG_EXCEPTION_NOT_HANDLED
.endw
```

- Вот хинт: как только вы начинаете отладку программы, вы не можете отсоединиться от отлаживаемого процесса, пока тот не завершится.

Давайте кратко прорезюмируем шаги:

- Создаем процесс или присоединяемся к уже выполняющемуся процессу.
- Ожидаем отладочных событий
- Ваша программа реагирует на отладочное событие
- Продолжаем выполнение отлаживаемого процесса
- Продолжаем этот бесконечный цикл, пока существует отлаживаемый процесс

ПРИМЕР

Этот пример отлаживает win32-программу и показывает важную информацию, такую как хэндл процесса, его ID, image base и так далее.

```
.386

.model flat,stdcall
option casemap:none
include \masm32\include\windows.inc
include \masm32\include\kernel32.inc

include \masm32\include\comdlg32.inc
include \masm32\include\user32.inc
includelib \masm32\lib\kernel32.lib
includelib \masm32\lib\comdlg32.lib

includelib \masm32\lib\user32.lib
.data
AppName db "Win32 Debug Example no.1",0
ofn OPENFILENAME <>

FilterString db "Executable Files",0,"*.exe",0
             db "All Files",0,"*.*",0,0
ExitProc db "The debuggee exits",0
NewThread db "A new thread is created",0

EndThread db "A thread is destroyed",0
ProcessInfo db "File Handle: %lx ",0dh,0Ah
             db "Process Handle: %lx",0dh,0Ah
             db "Thread Handle: %lx",0dh,0Ah

             db "Image Base: %lx",0dh,0Ah
             db "Start Address: %lx",0

.data?
buffer db 512 dup(?)

startinfo STARTUPINFO <>
pi PROCESS_INFORMATION <>
DBEvent DEBUG_EVENT <>
.code

start:
mov ofn.lStructSize, sizeof ofn
```



```

mov ofn.lpstrFilter, offset FilterString
mov ofn.lpstrFile, offset buffer

mov ofn.nMaxFile,512
mov ofn.Flags, OFN_FILEMUSTEXIST or OFN_PATHMUSTEXIST or OFN_LONGNAMES or \
OFN_EXPLORER or OFN_HIDEREADONLY
invoke GetOpenFileName, ADDR ofn

.if eax==TRUE
invoke GetStartupInfo,addr startinfo
invoke CreateProcess, addr buffer, NULL, NULL, NULL, FALSE, DEBUG_PROCESS+ \
DEBUG_ONLY_THIS_PROCESS, NULL, NULL, addr startinfo, addr pi

.while TRUE
    invoke WaitForDebugEvent, addr DBEvent, INFINITE
    .if DBEvent.dwDebugEventCode==EXIT_PROCESS_DEBUG_EVENT
        invoke MessageBox, 0, addr ExitProc, addr AppName, MB_OK+MB_ICONINFORMATION
        .break
    .elseif DBEvent.dwDebugEventCode==CREATE_PROCESS_DEBUG_EVENT
        invoke wsprintf, addr buffer, addr ProcessInfo, \

DBEvent.u.CreateProcessInfo.hFile, DBEvent.u.CreateProcessInfo.hProcess, \
DBEvent.u.CreateProcessInfo.hThread, \
DBEvent.u.CreateProcessInfo.lpBaseOfImage, \
DBEvent.u.CreateProcessInfo.lpStartAddress

        invoke MessageBox,0, addr buffer, addr AppName, MB_OK+MB_ICONINFORMATION
    .elseif DBEvent.dwDebugEventCode==EXCEPTION_DEBUG_EVENT
        .if DBEvent.u.Exception.pExceptionRecord.ExceptionCode==EXCEPTION_BREAKPOINT
            invoke ContinueDebugEvent, DBEvent.dwProcessId, DBEvent.dwThreadId, DBG_CONTINUE
            .continue
        .endif
    .elseif DBEvent.dwDebugEventCode==CREATE_THREAD_DEBUG_EVENT
        invoke MessageBox,0, addr NewThread, addr AppName, MB_OK+MB_ICONINFORMATION
    .elseif DBEvent.dwDebugEventCode==EXIT_THREAD_DEBUG_EVENT
        invoke MessageBox,0, addr EndThread, addr AppName, MB_OK+MB_ICONINFORMATION
    .endif
    invoke ContinueDebugEvent, DBEvent.dwProcessId, DBEvent.dwThreadId, DBG_EXCEPTION_NOT_HANDLED
.endw
invoke CloseHandle,pi.hProcess
invoke CloseHandle,pi.hThread
.endif
invoke ExitProcess, 0
end start

```

АНАЛИЗ

Программа заполняет структуру OPENFILENAME, а затем вызывает GetOpenFileName, чтобы юзер выбрал программу, которую нужно отладить.

```

invoke GetStartupInfo,addr startinfo
invoke CreateProcess, addr buffer, NULL, NULL, NULL, FALSE, \
    DEBUG_PROCESS+ DEBUG_ONLY_THIS_PROCESS, NULL, \
    NULL, addr startinfo, addr pi

```

Когда пользователь выберет процесс, программа вызовет CreateProcess, чтобы загрузить его. Она вызывает GetStartupInfo, чтобы заполнить структуру STARTUPINFO значениями по умолчанию. Обратите внимание, что мы комбинируем флаги DEBUG_PROCESS и DEBUG_ONLY_THIS_PROCESS, чтобы отладить только этот процесс, не включая его дочерние процессы.

```

.while TRUE
    invoke WaitForDebugEvent, addr DBEvent, INFINITE

```

Когда отлаживаемый процесс загружен, мы входим в бесконечный цикл, вызывая WaitForDebugEvent. Эта функция не возвратит управление, пока не произойдет отладочное событие в отлаживаемом процессе, потому что мы указали INFINITE в качестве второго параметра. Когда происходит отладочное событие, WaitForDebugEvent возвращает управление и DBEvent

заполняется информацией о произошедшем событии.

```
.if DBEvent.dwDebugEventCode==EXIT_PROCESS_DEBUG_EVENT
    invoke MessageBox, 0, addr ExitProc, addr AppName, MB_OK+MB_ICONINFORMATION
.break
```

Сначала мы проверяем значение dwDebugEventCode. Если это EXIT_PROCESS_DEBUG_EVENT, мы отображаем message box с надписью "The debuggee exits", а затем выходим из бесконечного цикла.

```
.elseif DBEvent.dwDebugEventCode==CREATE_PROCESS_DEBUG_EVENT
    invoke wsprintf, addr buffer, addr ProcessInfo, \
        DBEvent.u.CreateProcessInfo.hFile, DBEvent.u.CreateProcessInfo.hProcess, \
        DBEvent.u.CreateProcessInfo.hThread, \
        DBEvent.u.CreateProcessInfo.lpBaseOfImage, \
        DBEvent.u.CreateProcessInfo.lpStartAddress
    invoke MessageBox,0, addr buffer, addr AppName, MB_OK+MB_ICONINFORMATION
```

Если значение в dwDebugEventCode равно CREATE_PROCESS_DEBUG_EVENT, мы отображаем некоторую интересную информацию об отлаживаемом процесс в message box'e. Мы получаем эту информацию из u.CreateProcessInfo. CreateProcessInfo - это структура типа CREATE_PROCESS_DEBUG_INFO. Вы можете узнать об этой структуре более подробно из справочника по Win32 API.

```
.elseif DBEvent.dwDebugEventCode==EXCEPTION_DEBUG_EVENT
    .if DBEvent.u.Exception.pExceptionRecord.ExceptionCode==EXCEPTION_BREAKPOINT
        invoke ContinueDebugEvent, DBEvent.dwProcessId, DBEvent.dwThreadId, DBG_CONTINUE
    .continue
    .endif
```

Если значение dwDebugEventCode равно EXCEPTION_DEBUG_EVENT, мы также должны определить точный тип исключения из параметра ExceptionCode. Если значение в ExceptionCode равно EXCEPTION_BREAKPOINT, и это случилось в первый раз, мы можем считать, что это исключение возникло при запуске отлаживаемым процессом своей первой инструкции. После обработки сообщения мы должны вызвать ContinueDebugEvent с флагом DBG_CONTINUE, чтобы позволить отлаживаемому процессу продолжать выполнение. Затем мы снова ждем следующего отлаживаемого события.

```
.elseif DBEvent.dwDebugEventCode==CREATE_THREAD_DEBUG_EVENT
    invoke MessageBox,0, addr NewThread, addr AppName, MB_OK+MB_ICONINFORMATION
.elseif DBEvent.dwDebugEventCode==EXIT_THREAD_DEBUG_EVENT
    invoke MessageBox,0, addr EndThread, addr AppName, MB_OK+MB_ICONINFORMATION
.endif
```

Если значение в dwDebugEventCode равно CREATE_THREAD_DEBUG_EVENT или EXIT_THREAD_DEBUG_EVENT, мы отображаем соответствующий message box.

```
    invoke ContinueDebugEvent, DBEvent.dwProcessId, DBEvent.dwThreadId, DBG_EXCEPTION_NOT_HANDLED
.endw
```

Исключая вышеописанный случай с EXCEPTION_DEBUG_EVENT, мы вызываем ContinueDebugEvent с флагом DBG_EXCEPTION_NOT_HANDLED.

```
    invoke CloseHandle,pi.hProcess
    invoke CloseHandle,pi.hThread
```

Когда отлаживаемый процесс завершает выполнение, мы выходим из цикла отладки и должны закрыть хэндлы отлаживаемого процесса и треда. Закрытие хэндлов не означает, что мы их прерываем. Это только значит, что мы больше не хотим использовать эти хэндлы для ссылки на соответствующий процесс/тред.

Win32 API. Урок 29. Win32 Debug API II

Автор: lczelion, пер. Aquila

Дата: 2002-05-29

Мы продолжаем изучать отладочный win32-API. В этом tutorialе мы изучим, как модифицировать отлаживаемый процесс.

Скачайте пример (находится в архиве wasm_files.zip: files/recovered/1001029/tut29.zip).

ТЕОРИЯ

В предыдущем tutorialе мы узнали как загрузить процесс для отладки и обрабатывать отладочные сообщения, которые происходят в нем. Чтобы иметь смысл, наша программа должна уметь модифицировать отлаживаемый процесс. Есть несколько функций API, которые вы можете использовать в этих целях.

- `ReadProcessMemory` - эта функция позволяет вам читать память в указанном процессе. Прототип функции следующий:
`ReadProcessMemory` proto `hProcess:DWORD`, `lpBaseAddress:DWORD`, `lpBuffer:DWORD`, `nSize:DWORD`, `lpNumberOfBytesRead:DWORD`
- `hProcess` - хэндл процесса
- `lpBaseAddress` - адрес в процессе-цели, с которого вы хотите начать чтение. Например, если вы хотите прочитать 4 байта из отлаживаемого процесса начиная с `401000h`, значение этого параметра должно быть равно `401000h`.
- `lpBuffer` - адрес буфера, в который будут записаны прочитанные из процесса байты.
- `nSize` - количество байтов, которое вы хотите прочитать.
- `lpNumberOfBytesRead` - адрес переменной размером в двойное слово, которая получает количество байт, которое было прочитано в действительности. Если вам не важно это значение, вы можете использовать `NULL`.
- `WriteProcessMemory` - функция, обратная `ReadProcessMemory`. Она позволяет вам писать в память процесса. У нее точно такие же параметры, как и у `ReadProcessMemory`.

Прежде, чем начать описание двух следующих функций, необходимо сделать небольшое отступление. Под мультизадачной операционной системой, такой как Windows, может быть несколько программ, работающих в одно и то же время. Windows дает каждому треду небольшой временной интервал, по истечении которого ОС замораживает этот тред и переключается на следующий (согласно приоритету). Но перед тем, как переключиться на другой тред, Windows сохраняет значения регистров текущего треда, чтобы когда тот продолжил выполнение, Windows могла восстановить его рабочую среду. Все вместе сохраненные значения называют контекстом.

Вернемся к рассматриваемой теме. Когда случается отладочное событие, Windows замораживает отлаживаемый процесс. Его контекст сохраняется. Так как он заморожен, мы можем быть уверены, что значения контекста останутся неизменными. Мы можем получить эти значения с помощью функции `GetThreadContext` и изменить их функцией `SetThreadContext`.

Это две очень мощные API-функции. С их помощью у вас есть власть над отлаживаемым процессом, обладающая возможностями `VxD`: вы можете изменять сохраненные регистры и как только процесс продолжит выполнение, значения контекста будут записаны обратно в регистры. Любое изменение контекста отразится над отлаживаемым процессом.

Подумайте об этом: вы даже можете изменить значение регистра `esp` и повернуть ход исполнения программы так, как вам это надо! В обычных обстоятельствах вы бы не смогли этого сделать.

`GetThreadContext` proto `hThread:DWORD`, `lpContext:DWORD`

- hThread - хэндл треда, чей контекст вы хотите получить
- lpContext - адрес структуры CONTEXT, которая будет заполнена соответствующими значениями, когда функция передаст управление обратно

У функции SetThreadContext точно такие же параметры. Давайте посмотрим, как выглядит структура CONTEXT:

```
CONTEXT STRUCT

ContextFlags dd ?
;-----
; Эта секция возвращается, если ContextFlags содержит значение
; CONTEXT_DEBUG_REGISTERS
;-----

iDr0 dd ?
iDr1 dd ?
iDr2 dd ?
iDr3 dd ?
iDr6 dd ?
iDr7 dd ?

;-----
; Эта секция возвращается, если ContextFlags содержит значение
; CONTEXT_FLOATING_POINT
;-----

FloatSave FLOATING_SAVE_AREA <>

;-----
; Эта секция возвращается, если ContextFlags содержит значение
; CONTEXT_SEGMENTS
;-----

regGs dd ?
regFs dd ?
regEs dd ?
regDs dd ?

;-----
; Эта секция возвращается, если ContextFlags содержит значение
; CONTEXT_INTEGER
;-----

regEdi dd ?
regEsi dd ?
regEbx dd ?
regEdx dd ?
regEcx dd ?
regEax dd ?

;-----
; Эта секция возвращается, если ContextFlags содержит значение
; CONTEXT_CONTROL
;-----

regEbp dd ?
regEip dd ?
regCs dd ?
regFlag dd ?
regEsp dd ?
regSs dd ?

;-----
; Эта секция возвращается, если ContextFlags содержит значение
; CONTEXT_EXTENDED_REGISTERS
;-----

ExtendedRegisters db MAXIMUM_SUPPORTED_EXTENSION dup(?) CONTEXT ENDS
```

Как вы можете видеть, члены этих структур - это образы настоящих регистров процессора. Прежде, чем вы сможете использовать эту структуру, вам нужно указать, какую группу регистров вы хотите прочитать/записать, в параметре ContextFlags. Например, если вы хотите прочитать/записать все регистры, вы должны указать CONTEXT_FULL в ContextFlags. Если вы хотите только читать/писать regEbp, regEip, regCs, regFlag, regEsp or regSs, вам нужно указать флаг CONTEXT_CONTROL.

Используя структуру CONTEXT, вы должны помнить, что она должна быть выравнена по двойному слову, иначе под NT вы получите весьма странные результаты. Вы должны поместить "align dword" над строкой, объявляющей эту переменную:

```
align dword
MyContext CONTEXT <>
```

ПРИМЕР

Первый пример демонстрирует использование DebugActiveProcess. Сначала вам нужно запустить цель под названием win.exe, которая входит в бесконечный цикл перед показом окна. Затем вы запускаете пример, он подключится к win.exe и модифицирует код win.exe таким образом, чтобы он вышел из бесконечного цикла и показал свое окно.

```
.386
.model flat,stdcall
option casemap:none
include \masm32\include\windows.inc

include \masm32\include\kernel32.inc
include \masm32\include\comdlg32.inc
include \masm32\include\user32.inc
includelib \masm32\lib\kernel32.lib

includelib \masm32\lib\comdlg32.lib
includelib \masm32\lib\user32.lib

.data
AppName db "Win32 Debug Example no.2",0

ClassName db "SimpleWinClass",0
SearchFail db "Cannot find the target process",0
TargetPatched db "Target patched!",0
buffer dw 9090h

.data?
DBEvent DEBUG_EVENT <>
ProcessId dd ?
ThreadId dd ?

align dword
context CONTEXT <>

.code
start:

invoke FindWindow, addr ClassName, NULL
.if eax!=NULL
    invoke GetWindowThreadProcessId, eax, addr ProcessId
    mov ThreadId, eax

    invoke DebugActiveProcess, ProcessId
    .while TRUE
        invoke WaitForDebugEvent, addr DBEvent, INFINITE
        .break .if DBEvent.dwDebugEventCode==EXIT_PROCESS_DEBUG_EVENT

    .if DBEvent.dwDebugEventCode==CREATE_PROCESS_DEBUG_EVENT
        mov context.ContextFlags, CONTEXT_CONTROL
        invoke GetThreadContext,DBEvent.u.CreateProcessInfo.hThread, addr context

        invoke WriteProcessMemory, DBEvent.u.CreateProcessInfo.hProcess, \
```

```

context.regEip ,addr buffer, 2, NULL

invoke MessageBox, 0, addr TargetPatched, addr AppName, \
    MB_OK+MB_ICONINFORMATION

.elseif DBEvent.dwDebugEventCode==EXCEPTION_DEBUG_EVENT
    .if DBEvent.u.Exception.pExceptionRecord.ExceptionCode==EXCEPTION_BREAKPOINT
        invoke ContinueDebugEvent, DBEvent.dwProcessId, \
            DBEvent.dwThreadId, DBG_CONTINUE
    .continue
    .endif
.endif

invoke ContinueDebugEvent, DBEvent.dwProcessId, DBEvent.dwThreadId, \
    DBG_EXCEPTION_NOT_HANDLED

.endw
.else
    invoke MessageBox, 0, addr SearchFail, addr AppName,MB_OK+MB_ICONERROR
.endif
invoke ExitProcess, 0
end start

;-----
; Частичный исходный код win.asm, отлаживаемого нами процесса. Это
; копия примера простого окна из 2-го tutorials с добавленным бесконечным
; циклом перед циклом обработки сообщений.
;-----

.....
mov wc.hIconSm,eax
invoke LoadCursor,NULL,IDC_ARROW
mov wc.hCursor,eax
invoke RegisterClassEx, addr wc
INVOKE CreateWindowEx,NULL,ADDR ClassName,ADDR AppName,\
WS_OVERLAPPEDWINDOW,CW_USEDEFAULT,\
CW_USEDEFAULT,CW_USEDEFAULT,CW_USEDEFAULT,NULL,NULL,\ hInst,NULL
mov hwnd,eax
jmp $ ; <---- Here's our infinite loop. It assembles to EB FE
invoke ShowWindow, hwnd,SW_SHOWNORMAL
invoke UpdateWindow, hwnd
.while TRUE
    invoke GetMessage, ADDR msg,NULL,0,0
    .break .if (!eax)
    invoke TranslateMessage, ADDR msg
    invoke DispatchMessage, ADDR msg
.endw
mov eax,msg.wParam
ret
WinMain endp

```

АНАЛИЗ

```
invoke FindWindow, addr ClassName, NULL
```

Наша программа должна подсоединиться к отлаживаемому процессу с помощью DebugActiveProcess, который требует ID процесса, который будет отлаживаться. Мы можем получить этот ID с помощью GetWindowThreadProcessId, которая, в свою очередь, требует хэндл окна. Поэтому мы сначала должны получить хэндл окна.

Мы указываем функции FindWindow имя класса окна, которое нам нужно. Она возвращает хэндл окна этого класса. Если возвращен NULL, окон такого класса нет.

```

.if eax!=NULL
    invoke GetWindowThreadProcessId, eax, addr ProcessId
    mov ThreadId, eax
    invoke DebugActiveProcess, ProcessId

```

После получения ID процесса, мы вызываем DebugActiveProcess, а затем входим в отладочный цикл, в котором ждем отладочных событий.

```

.if DBEvent.dwDebugEventCode==CREATE_PROCESS_DEBUG_EVENT

mov context.ContextFlags, CONTEXT_CONTROL
invoke GetThreadContext, DBEvent.u.CreateProcessInfo.hThread, addr context

```

Когда мы получаем CREATE_PROCESS_DEBUG_INFO, это означает, что отлаживаемый процесс заморожен и мы можем произвести над ним нужное нам действие. В этом примере мы перепишем инструкцию бесконечного цикла (0EBh 0FEh) NOP'ами (90h 90h).

Сначала мы должны получить адрес инструкции. Так как отлаживаемый процесс уже будет в цикле к тому времени, как к нему присоединится наша программа, еip будет всегда указывать на эту инструкцию. Все, что мы должны сделать, это получить значение еip. Мы используем GetThreadContext, чтобы достичь этой цели. Мы устанавливаем поле ContextFlags в CONTEXT_CONTROL.

```

invoke WriteProcessMemory, DBEvent.u.CreateProcessInfo.hProcess, \
context.regEip, addr buffer, 2, NULL

```

Получив значение еip, мы можем вызвать WriteProcessMemory, чтобы переписать инструкцию "jmp \$" NOP'ами. После этого мы отображаем сообщение пользователю, а затем вызываем ContinueDebugEvent, чтобы продолжить выполнение отлаживаемого процесса. Так как инструкция "jmp \$" будет перезаписана NOP'ами, отлаживаемый процесс сможет продолжить выполнение, показав свое окно и войдя в цикл обработки сообщений. В качестве доказательства мы увидим его окно на экране.

Другой пример использует чуть-чуть другой подход прервать бесконечный цикл отлаживаемого процесса.

```

.....
.....
.if DBEvent.dwDebugEventCode==CREATE_PROCESS_DEBUG_EVENT
mov context.ContextFlags, CONTEXT_CONTROL
invoke GetThreadContext,DBEvent.u.CreateProcessInfo.hThread, addr context
add context.regEip,2
invoke SetThreadContext,DBEvent.u.CreateProcessInfo.hThread, addr context
invoke MessageBox, 0, addr LoopSkipped, addr AppName, MB_OK+MB_ICONINFORMATION
.....
.....

```

Вызывается GetThreadContext, чтобы получить текущее значение еip, но вместо перезаписывания инструкции "jmp \$", меняется значение regEip на +2, чтобы "пропустить" инструкцию. Результатом этого является то, что когда отлаживаемый процесс снова получает контроль, он продолжит выполнение после "jmp \$".

Теперь вы можете видеть силу Get/SetThreadContext. Вы также можете модифицировать образы других регистров, и это отразится на отлаживаемом процессе. Вы даже можете вставить инструкцию int 3h, чтобы поместить breakpoint'ы в отлаживаемый процесс.

Win32 API. Урок 30. Win32 Debug API III

Автор: lczelion, пер. Aquila

Дата: 2002-05-30

В этом tutorialе мы продолжим исследование Win32 debug API. В частности, мы узнаем, как трассировать отлаживаемый процесс.

Скачайте пример (находится в архиве wasm_files.zip: files/recovered/1001030/tut30.zip).

ТЕОРИЯ

Если вы использовали дебаггер раньше, вам должно быть знаком понятие трассировки.

Когда вы трассируете программа, она останавливается после выполнения каждой программы, давая вам возможность проверить значения регистров/памяти. Пошаговая отладка - это официальное название трассировки. Эта возможность предоставлена самим процессором. Восьмой бит регистра флагов называется trap-флаг. Если этот флаг (бит) установлен, процессор работает в пошаговом режиме. Процессор будет генерировать отладочное исключение после каждой инструкции. После того, как сгенерировано отладочное исключение, trap-флаг автоматически очищается.

Мы тоже можем пошагово отлаживать процесс, используя Win32 Debug API. Шаги следующие:

- Вызываем `GetThreadContext`, указав `CONTEXT_CONTROL` в `ContextFlags`, чтобы получить значение флагового регистра.
- Устанавливаем trap-бит в поле `regFlag` структуры `CONTEXT`.
- Вызываем `SetThreadContext`.
- Как обычно ждем отладочного события. Отлаживаемый процесс будет запущен в пошаговом режиме. После выполнения каждой инструкции мы будем получать значение `EXCEPTION_DEBUG_EVENT` + `EXCEPTION_SINGLE_STEP` в `u.Exception.pExceptionRecord.ExceptionCode`.
- Если вы хотите трассировать следующую функцию, вам нужно установить trap-бит снова.

ПРИМЕР

```
.386
.model flat,stdcall
option casemap:none
include \masm32\include\windows.inc
include \masm32\include\kernel32.inc
include \masm32\include\comdlg32.inc
include \masm32\include\user32.inc
includelib \masm32\lib\kernel32.lib
includelib \masm32\lib\comdlg32.lib
includelib \masm32\lib\user32.lib

.data
AppName db "Win32 Debug Example no.4",0
ofn OPENFILENAME <>
FilterString db "Executable Files",0,"*.exe",0
             db "All Files",0,"*.*",0,0
ExitProc db "The debuggee exits",0Dh,0Ah
          db "Total Instructions executed : %lu",0
TotalInstruction dd 0

.data?
buffer db 512 dup(?)
startinfo STARTUPINFO <>
pi PROCESS_INFORMATION <>
DBEvent DEBUG_EVENT <>
context CONTEXT <>
```



```

.code
start:
mov ofn.lStructSize,SIZEOF ofn
mov ofn.lpstrFilter, OFFSET FilterString
mov ofn.lpstrFile, OFFSET buffer
mov ofn.nMaxFile,512
mov ofn.Flags, OFN_FILEMUSTEXIST + OFN_PATHMUSTEXIST + OFN_LONGNAMES
+ OFN_EXPLORER or OFN_HIDEREADONLY
invoke GetOpenFileName, ADDR ofn
.if eax==TRUE
    invoke GetStartupInfo,addr startinfo
    invoke CreateProcess, addr buffer, NULL, NULL, NULL, FALSE, \
        DEBUG_PROCESS+ DEBUG_ONLY_THIS_PROCESS, NULL, NULL, \
        addr startinfo, addr pi
    .while TRUE
        invoke WaitForDebugEvent, addr DBEvent, INFINITE
        .if DBEvent.dwDebugEventCode==EXIT_PROCESS_DEBUG_EVENT
            invoke wsprintf, addr buffer, addr ExitProc, TotalInstruction
            invoke MessageBox, 0, addr buffer, addr AppName, MB_OK+MB_ICONINFORMATION
            .break
        .elseif DBEvent.dwDebugEventCode==EXCEPTION_DEBUG_EVENT
            .if DBEvent.u.Exception.pExceptionRecord.ExceptionCode==EXCEPTION_BREAKPOINT
                mov context.ContextFlags, CONTEXT_CONTROL
                invoke GetThreadContext, pi.hThread, addr context
                or context.regFlag,100h
                invoke SetThreadContext,pi.hThread, addr context
                invoke ContinueDebugEvent, DBEvent.dwProcessId, \
                    DBEvent.dwThreadId, DBG_CONTINUE
                .continue
            .elseif DBEvent.u.Exception.pExceptionRecord.ExceptionCode==EXCEPTION_SINGLE_STEP
                inc TotalInstruction
                invoke GetThreadContext,pi.hThread, \
                    addr context or context.regFlag,100h
                invoke SetThreadContext,pi.hThread, addr context
                invoke ContinueDebugEvent, DBEvent.dwProcessId, \
                    DBEvent.dwThreadId,DBG_CONTINUE
                .continue
            .endif
        .endif
        invoke ContinueDebugEvent, DBEvent.dwProcessId, DBEvent.dwThreadId, \
            DBG_EXCEPTION_NOT_HANDLED
    .endw
.endif
invoke CloseHandle,pi.hProcess
invoke CloseHandle,pi.hThread
invoke ExitProcess, 0
end start

```

АНАЛИЗ

Программа показывает окно выбора файла. Когда пользователь выбирает исполняемый файл, она запускает программу в пошаговом режиме, подсчитывая количество выполненных инструкций пока отлаживаемый процесс не завершится.

```

.elseif DBEvent.dwDebugEventCode==EXCEPTION_DEBUG_EVENT
    .if DBEvent.u.Exception.pExceptionRecord.ExceptionCode==EXCEPTION_BREAKPOINT

```

Мы используем эту возможность, чтобы установить отлаживаемый процесс в пошаговый режим. Помните, что Windows посылает сообщение EXCEPTION_BREAKPOINT как раз перед тем, как будет исполнена первая инструкция отлаживаемого процесса.

```

    mov context.ContextFlags, CONTEXT_CONTROL
    invoke GetThreadContext, pi.hThread, addr context

```

Мы вызываем GetThreadContext, чтобы заполнить структуру CONTEXT текущими значениями регистров отлаживаемого процесса. Конкретно нам нужно текущее значение регистра флагов.

```

    or context.regFlag,100h

```

Мы устанавливаем trap-бит (8-ой бит) в образе регистра флагов.

```
invoke SetThreadContext, pi.hThread, addr context
invoke ContinueDebugEvent, DBEvent.dwProcessId, \
    DBEvent.dwThreadId, DBG_CONTINUE
.continue
```

Затем мы вызываем SetThreadContext для перезаписи контекста новыми значениями и вызываем ContinueDebugEvent с флагом DBG_CONTINUE, чтобы продолжить выполнение отлаживаемого процесса.

```
.elseif DBEvent.u.Exception.pExceptionRecord.ExceptionCode==EXCEPTION_SINGLE_STEP
    inc TotalInstruction
```

Когда в отлаживаемом процессе выполняется инструкция, мы получаем EXCEPTION_DEBUG_EVENT. Мы должны проверить значение u.Exception.pExceptionRecord.ExceptionCode. Если значение равно EXCEPTION_SINGLE_STEP, значит это отладочное событие было сгенерировано из-за пошагового режима. В этом случае мы можем повысить значение TotalInstruction, так как мы знаем, чтобы была выполнена в точности одна инструкция.

```
invoke GetThreadContext, pi.hThread, \
    addr context or context.regFlag, 100h
invoke SetThreadContext, pi.hThread, addr context
invoke ContinueDebugEvent, DBEvent.dwProcessId, \
    DBEvent.dwThreadId, DBG_CONTINUE
.continue
```

Так как trap-флаг очищается после генерации отладочного исключения, мы должны установить trap-флаг снова, если мы хотим продолжить выполнение в пошаговом режиме. Предупреждение: не используйте пример в этом tutorialе с большими программами: трассировка - это медленный процесс. Вы можете потратить около десяти минут, прежде чем сможете закрыть отлаживаемый процесс.

Win32 API. Урок 31. Контроль ListView

Автор: lczelion, пер. Aquila

Дата: 2002-06-01

В этом tutorialе мы изучим как создать и использовать контроль listview.

Скачайте пример (находится в архиве wasm_files.zip: files/recovered/1001031/tut31.zip).

ТЕОРИЯ

Listview - это один из common control'ов, таких как treeview, richedit и так далее. Вы знакомы с ними, даже если не знаете их имен. Например, правая панель Windows Explorer'a - это контроль listview. Этот контроль подходит для отображения item'ов. В этом отношении его можно рассматривать как усовершенствованный listbox.

Вы можете создать listview двумя путями. Первый метод самый простой: создайте его с помощью редактора ресурсов, главное не забудьте поместить вызов InitCommonControls. Другой метод заключается в вызове CreateWindowEx. Вы должны указать правильное имя класса окна, то есть SysListView32.

Существует четыре метода отображения item'ов в listview: иконки, маленькие иконки, список и отчет. Вы можете увидеть чем отличаются виды отображения друг от друга, выбрав View->Large Icons (иконки), Small Icons (маленькие иконки), List (список) and Details (отчет)

Теперь, когда мы знаем, как создать listview, мы рассмотрим, как его можно применять. Я сосредоточусь на отчете, как методе отображения, который может продемонстрировать многие свойства listview. Шаги использования listview следующие:

- Создаем listview с помощью CreateWindowEx, указав SysListView32 как имя класса. Вы должны указать начальный тип отображения.
- (если предусматривается) Создаем и инициализируем списки изображений, которые будут использованы при отображении item'ов listview.
- Вставляем колонки в listview. Этот шаг необходим, если listview будет использовать тип отображения 'отчет'.
- Вставьте item'ы и подitem'ы в listview.

Колонки

При отчете в listview может быть одна или более колонок. Вы можете считать тип организации данных в этом режиме таблицей: данные организованы в ряды и колонки. В режиме отчета в listview должна быть по крайней мере одна колонка. В других режимах вам не надо вставлять колонку, так как в контроле будет одна и только одна колонка.

Вы можете вставить колонку, пошлав сообщение LVM_INSERTCOLUMN контролю listview.

```
LVM_INSERTCOLUMN
wParam = iCol
lParam = pointer to a LV_COLUMN structure
```

iCol - это номер колонки, начиная с нуля.

LV_COLUMN содержит информацию о колонке, которая должна быть вставлена. У нее следующее определение:

```
LV_COLUMN STRUCT
    iMask dd ?
    fmt dd ?
    iX dd ?
    pszText dd ?
    cchTextMax dd ?
```

```

    iSubItem dd ?
    iImage dd ?
    iOrder dd ?
LV_COLUMN ENDS

```

• `imask` - коллекция флагов, задающие, какие члены структуры верны. Этот параметр был введен, потому что не все члены этой структуры используются одновременно. Некоторые из них используются в особых ситуациях. Эта структура используется и для ввода и для вывода, поэтому важно, чтобы вы поместили, какие параметры верны. Существуют следующие флаги:

`LVCF_FMT` = The `fmt` member is valid.
`LVCF_SUBITEM` = The `iSubItem` member is valid.
`LVCF_TEXT` = The `pszText` member is valid.
`LVCF_WIDTH` = The `lx` member is valid.

`LVCF_FMT` = Параметр `fmt` верен.
`LVCF_SUBITEM` = Параметр `iSubItem` верен.
`LVCF_TEXT` = Параметр `pszText` верен.
`LVCF_WIDTH` = Параметр `lx` верен.

Вы можете комбинировать вышеприведенные флаги. Например, если вы хотите указать текстовое имя колонки, вам нужно предоставить указатель на строку в параметре `pszText`. Также вы должны указать Windows, что параметр `pszText` содержит данные, указав флаг `LVCF_TEXT` в этом поле, иначе Windows будет игнорировать значение `pszText`.

• `fmt` - указывает выравнивание элементов/подэлементов в колонке. Доступны следующие значения:

`LVCFMT_CENTER` = Text is centered.
`LVCFMT_LEFT` = Text is left-aligned.
`LVCFMT_RIGHT` = Text is right-aligned.

`LVCFMT_CENTER` = текст отцентрированы.
`LVCFMT_LEFT` = текст выравнивается слева.
`LVCFMT_RIGHT` = текст выравнивается справа.

• `lx` - ширина колонки в пикселях. В дальнейшем вы можете изменить ширину колонки `LVM_SETCOLUMNWIDTH`.

• `pszText` - содержит указатель на имя колонки, если эта структура используется для установки свойств колонки. Если эта структура используется для получения свойств колонки, это поле содержит указатель на буфер, достаточно большой для получения имени колонки, которая будет возвращена. В этом случае вы должны указать размер буфера в поле `cchTextMax`. Вы должны игнорировать `cchTextMax`, если вы хотите установить имя колонки, потому что имя должно быть ASCII-строкой, длину которой Windows сможет определить.

• `cchTextMax` - размер в байтах буфера, указанного в поле `pszText`. Этот параметр используется только когда вы используете структуру для получения информации о колонке. Если вы используете эту структуру, чтобы установить свойства колонки, это поле будет игнорироваться.

• `iSubItem` - указывает индекс подэлемента, ассоциированного с этой колонкой. Это значение используется в качестве маркера подэлемента, с которым ассоциирована эта колонка. Если хотите, вы можете указать бессмысленный номер и ваш `listview` будет прекрасно работать. Использование этого поля лучше всего демонстрируется, когда у вас есть номер колонки и вам нужно узнать с каким подэлементом ассоциирована эта колонка. Чтобы сделать это, вы можете послать сообщение `LVM_GETCOLUMN`, указав в параметре `imask` флаг `LVCF_SUBITEM`. `Listview` заполнит параметр `iSubItem` значением, которое вы укажете в этом поле, поэтому для работоспособности данного метода вам нужно указывать корректные подэлементы в этом поле.

• `iImage` и `iOrder` - используется начиная с Internet Explorer 3.0. У меня нет информации относительно этих полей.

Когда listview создан, вам нужно вставить в него одну или более колонок. Если не предполагается переключение в режим отчета, то это не нужно. Чтобы вставить колонку, вам нужно создать структуру LV_COLUMN, заполнить ее необходимой информацией, указать номер колонки, а затем послать структуру listview с помощью сообщения LVM_INSERTCOLUMN.

```
LOCAL lvc:LV_COLUMN
mov lvc.imask,LVCF_TEXT+LVCF_WIDTH
mov lvc.pszText,offset Heading1

mov lvc.lx,150
invoke SendMessage,hList, LVM_INSERTCOLUMN,0,addr lvc
```

Вышеприведенный кусок кода демонстрирует процесс. Он указывает текста заголовка и его ширину, а затем посылает сообщение LVM_INSERTCOLUMN listview. Это просто.

Item'ы и под-item'ы

Item'ы - это основные элементы listview. В режимах отображения, отличных от отчета, вы будете видеть только item'ы. Под-item'ы - это детали item'ов. Например, если item - это имя файла, тогда вы можете считать атрибуты файла, его размер, дату создания файла как под-item'ы. В режиме отчета самая левая колонка содержит item'ы, а остальные - под-item'ы. Вы можете думать о item'е и его под-item'ах как о записи базы данных. Item - это основной ключ записи и его под-item'ы - это поля записи.

Минимум, что вам нужно иметь в listview - это item'ы, под-item'ы необязательны. Тем не менее, если вы хотите дать пользователю больше информации об элементах, вы можете ассоциировать item'ы с под-item'ами, чтобы пользователь мог видеть детали в режиме отчета.

Вставить item в listview можно посылая сообщение LVM_INSERTITEM. Вам также нужно передать адрес структуры LV_ITEM в lParam. LV_ITEM имеет следующее определение:

```
LV_ITEM STRUCT
    imask dd ?
    iItem dd ?
    iSubItem dd ?
    state dd ?
    stateMask dd ?
    pszText dd ?
    cchTextMax dd ?
    iImage dd ?
    lParam dd ?
    iIndent dd ?
LV_ITEM ENDS
```

- imask - множество флагов, которые задают, какие из параметров данной структуры будут верны. В сущности, это поле идентично параметру imask LV_COLUMN. Чтобы получить детали относительно флагов, обратитесь к вашему справочнику по API.
- iItem - индекс item'а, на который ссылается эта структура. Индексы начинаются с нуля. Вы можете считать, что это поле содержит значение "ряда" таблицы.
- iSubItem - индекс под-item'а, ассоциированный с item'ом, заданном в iItem. Вы можете считать, что это поле содержит "колонку" таблицы. Например, если вы хотите вставить item в только что созданный listview, значение в iItem будет равно 0 (потому что этот item первый), а значение в iSubItem также будет равно нулю (нам нужно вставить item в первую колонку). Если вы хотите указать под-item, ассоциированный с этим item'ом, iItem будет являться индексом item'а, с которым будет происходить ассоциирование (в выше приведенном примере это 0). iSubItem будет равен 1 или более, в зависимости от того, в какую колонку вы хотите вставить под-item. Например, если у вашего listview 4 колонки, первая колонка будет содержать item'ы. Остальные 3 колонки предназначены для под-item'ов. Если вы хотите вставить под-item в 4-ую колонку, вам нужно указать в iSubItem значение 3.

- `state` - параметр, содержащий флаги, отражающие состояние `item'a`. Оно может изменяться из-за действий юзера или другой программы. Термин 'состояние' включает в себя, находится ли `item` в фокусе, подсвечен ли он, выделен для операции вырезания, выбран ли он. В дополнение к флагам состояния он также содержит основанный на единице индекс изображения состояния данного `item'a`.
- `stateMask` - так как параметр `state` может содержать флаги состояния, индекс изображения, нам требуется сообщить Windows, какое значение мы хотим установить или получить. Это поле создано именно для этого.
- `pszText` - адрес ASCIIZ-строки, которая будет использоваться в качестве названия элемента в случае, если мы хотим установить или вставить элемент. Если мы используем эту структуру для того, чтобы получить свойства элемента, этот параметр должен содержать адрес буфера, который будет заполнен названием элемента.
- `cchTextMax` - это поле используется только тогда, когда вы используете данную структуру, чтобы получать информацию об элементе. В этом случае это поле содержит размер в байтах буфера, указанного параметром `pszText`.
- `image` - индекс `image list'a`, содержащего иконки для `listview`. Индекс указывает на иконку, которая будет использоваться для этого элемента.
- `IParam` - определяемое пользователем значение, которое будет использоваться, когда вы будете сортировать элементы в `listview`. Кратко говоря, когда вы будете указывать `listview` отсортировать `item'ы`, `listview` будет сравнивать `item'ы` попарно. Он будет посылать значение `IParam` обоим элементам вам, чтобы вы могли решить, какое из этих двух должно быть в списке идти раньше. Если вы пока не можете этого понять, не беспокойтесь. Вы изучите сортировку позже.

Давайте кратко изложим шаги вставки элемента/подэлемента в `listview`.

- Создаем переменную типа структуры `LV_ITEM`.
- Заполняем ее необходимой информацией.
- Посылаем сообщение `LVM_INSERTITEM listview`, если вам нужно вставить элемент. Или, если вы хотите вставить подэлемент, посылаем сообщение `LVM_SETITEM`. Это может смущать вас, если вы не понимаете взаимоотношений между элементом и его подэлементами. Подэлементы считаются свойствами элемента. Поэтому вы можете вставить `item'ы`, но не под-`item'ы`, а также у вас не может быть подэлемента без ассоциированного с ним элемента. Вот почему вам нужно послать сообщение `LVM_SETITEM`, чтобы добавить подэлемент вместо `LVM_INSERTITEM`.

Сообщения/уведомления `listview` Теперь, когда вы знаете, как создавать и заполнять элементами `listview`, следующим шагом является общение с ним. `Listview` общается с родительским окном через сообщения и уведомления. Родительское окно может контролировать `listview`, посылая ему сообщения. `Listview` уведомляет родительское окно о важных/интересных сообщениях через сообщение `WM_NOTIFY`, как и другие `common control'ы`. **Сортировка элементов/подэлементов**

Вы можете указать порядок сортировки контрола `listview` по умолчанию указав стили `LVS_SORTASCENDING` или `LVS_SORTDESCENDING` в `CreateWindowEx`. Эти два стиля упорядочивают элементы только по элементам. Если вы хотите отсортировать элементы другим путем, вы должны послать сообщение `LVM_SORTITEMS listview`.

```
LVM_SORTITEMS
wParam = lParamSort
lParam = pCompareFunction
```

`lParamSort` - это определяемое пользователем значение, которое будет передаваться функции сравнения. Вы можете использовать это значение любым путем, которым хотите.

`pCompareFunction` - это адрес задаваемой пользователем функции, которая будет определять результат сравнения `item'ов` в `listview`. Функция имеет следующий прототип:

```
CompareFunc proto lParam1:DWORD, lParam2:DWORD, lParamSort:DWORD
```

IParam1 или IParam2 - это значения параметра IParam LV_ITEM, который вы указали, когда вставляли элементы в listview.

IParamSort - это значение wParam, посланное вместе с сообщением LVM_SORTITEMS.

Когда listview получает сообщение LVM_SORTITEMS, она вызывает сортирующую функцию, указанную в параметре IParam, когда ей нужно узнать результат сравнения двух элементов. Кратко говоря, функция сравнения будет решать, какой из двух элементов, посланных ей, будет предшествовать другому. Правило простое: если функция возвращается отрицательное значение, тогда первый элемент (указанный в IParam1) будет предшествовать другому.

Если функция возвращает положительное значение, второй элемент (заданный параметром IParam2) должен предшествовать первому. Если оба равны, тогда функция должна вернуть ноль.

Что заставляет этот метод работать, так это значение IParam структуры LV_ITEM. Если вам нужно отсортировать item'ы (например, когда пользователь кликает по заголовку колонки), вам нужно подумать о схеме сортировки, в которой будет использоваться значения параметра IParam. В данном примере я помещаю это поле индекс элемента, чтобы получить другую информация о нем, пошлав сообщение LVM_GETITEM. Заметьте, что когда элементы перегруппированы, их индексы также меняются. Поэтому когда сортировка в моем примере выполнена, мне необходимо обновить значения в IParam, чтобы учесть новые значения индексов. Если вы хотите отсортировать элементы, когда пользователь кликает по заголовку колонки, вам нужно обработать уведомительное сообщение LVN_COLUMNCLICK в вашей оконной процедуре. LVN_COLUMNCLICK передается вашему окну через сообщение WM_NOTIFY.

ПРИМЕР

Этот пример создает listview и заполняем его именами и размерами полей текущей папки. Режим отображения элементов по умолчанию поставлен в 'отчет'. В этом режиме вы можете кликать по заголовку колонок и элементы будут отсортированы согласно восходящему/нисходящему порядку. Вы можете выбрать режим отображения в меню. Когда вы делаете двойной клик по элементу, показывается окно с названием элемента.

```
.386
.model flat,stdcall
option casemap:none
include \masm32\include\windows.inc
include \masm32\include\user32.inc
include \masm32\include\kernel32.inc
include \masm32\include\comctl32.inc
includelib \masm32\lib\comctl32.lib
includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib

WinMain proto :DWORD,:DWORD,:DWORD,:DWORD

IDM_MAINMENU equ 10000
IDM_ICON equ LVS_ICON
IDM_SMALLICON equ LVS_SMALLICON
IDM_LIST equ LVS_LIST
IDM_REPORT equ LVS_REPORT

RGB macro red,green,blue
    xor eax,eax
    mov ah,blue
    shl eax,8
    mov ah,green
    mov al,red
endm

.data
ClassName db "ListViewWinClass",0
```

```

AppName db "Testing a ListView Control",0
ListViewClassName db "SysListView32",0
Heading1 db "Filename",0
Heading2 db "Size",0
FileNamePattern db "**.*",0
FileNameSortOrder dd 0
SizeSortOrder dd 0
template db "%lu",0

.data?
hInstance HINSTANCE ?
hList dd ?
hMenu dd ?

.code
start:
    invoke GetModuleHandle, NULL
    mov hInstance,eax
    invoke WinMain, hInstance,NULL, NULL, SW_SHOWDEFAULT
    invoke ExitProcess,eax
    invoke InitCommonControls

WinMain proc
hInst:HINSTANCE,hPrevInst:HINSTANCE,CmdLine:LPSTR,CmdShow:DWORD
LOCAL wc:WNDCLASSEX
LOCAL msg:MSG
LOCAL hwnd:HWND

    mov wc.cbSize,SIZEOF WNDCLASSEX
    mov wc.style, NULL
    mov wc.lpfnWndProc, OFFSET WndProc
    mov wc.cbClsExtra,NULL
    mov wc.cbWndExtra,NULL
    push hInstance
    pop wc.hInstance
    mov wc.hbrBackground,COLOR_WINDOW+1
    mov wc.lpszMenuName,IDM_MAINMENU
    mov wc.lpszClassName,OFFSET ClassName
    invoke LoadIcon,NULL,IDI_APPLICATION
    mov wc.hIcon,eax
    mov wc.hIconSm,eax
    invoke LoadCursor,NULL,IDC_ARROW
    mov wc.hCursor,eax
    invoke RegisterClassEx, addr wc
    invoke CreateWindowEx,NULL,ADDR ClassName,ADDR AppName, WS_OVERLAPPEDWINDOW, \
        CW_USEDEFAULT, CW_USEDEFAULT, CW_USEDEFAULT, CW_USEDEFAULT, \
        NULL, NULL, hInst, NULL
    mov hwnd,eax
    invoke ShowWindow, hwnd,SW_SHOWNORMAL
    invoke UpdateWindow, hwnd
    .while TRUE
        invoke GetMessage, ADDR msg,NULL,0,0
        .break .if (!eax)
        invoke TranslateMessage, ADDR msg
        invoke DispatchMessage, ADDR msg
    .endw
    mov eax,msg.wParam
    ret
WinMain endp

InsertColumn proc
LOCAL lvc:LV_COLUMN

    mov lvc.imask,LVCF_TEXT+LVCF_WIDTH
    mov lvc.pszText,offset Heading1
    mov lvc.lx,150
    invoke SendMessage,hList, LVM_INSERTCOLUMN, 0, addr lvc
    or lvc.imask,LVCF_FMT
    mov lvc.fmt,LVCFMT_RIGHT
    mov lvc.pszText,offset Heading2
    mov lvc.lx,100
    invoke SendMessage,hList, LVM_INSERTCOLUMN, 1 ,addr lvc

```



```

    ret
InsertColumn endp

ShowFileInfo proc uses edi row:DWORD, lpFind:DWORD
    LOCAL lvi:LV_ITEM
    LOCAL buffer[20]:BYTE
    mov edi,lpFind
    assume edi:ptr WIN32_FIND_DATA
    mov lvi.imask,LVIF_TEXT+LVIF_PARAM
    push row
    pop lvi.iItem
    mov lvi.iSubItem,0
    lea eax,[edi].cFileName
    mov lvi.pszText,eax
    push row
    pop lvi.lParam
    invoke SendMessage,hList, LVM_INSERTITEM,0, addr lvi
    mov lvi.imask,LVIF_TEXT
    inc lvi.iSubItem
    invoke wsprintf,addr buffer, addr template,[edi].nFileSizeLow
    lea eax,buffer
    mov lvi.pszText,eax
    invoke SendMessage,hList,LVM_SETITEM, 0,addr lvi
    assume edi:nothing
    ret
ShowFileInfo endp

FillFileInfo proc uses edi
    LOCAL finddata:WIN32_FIND_DATA
    LOCAL FHandle:DWORD

    invoke FindFirstFile,addr FileNamePattern,addr finddata
    .if eax!=INVALID_HANDLE_VALUE
        mov FHandle,eax
        xor edi,edi
        .while eax!=0
            test finddata.dwFileAttributes,FILE_ATTRIBUTE_DIRECTORY
            .if ZERO?
                invoke ShowFileInfo,edi, addr finddata
                inc edi
            .endif
            invoke FindNextFile,FHandle,addr finddata
        .endw
        invoke FindClose,FHandle
    .endif
    ret
FillFileInfo endp

String2Dword proc uses ecx edi edx esi String:DWORD
    LOCAL Result:DWORD

    mov Result,0
    mov edi,String
    invoke strlen,String
    .while eax!=0
        xor edx,edx
        mov dl,byte ptr [edi]
        sub dl,"0"
        mov esi,eax
        dec esi
        push eax
        mov eax,edx
        push ebx
        mov ebx,10
        .while esi > 0
            mul ebx
            dec esi
        .endw
        pop ebx
        add Result,eax
        pop eax
    .endw

```

```

        inc edi
        dec eax
    .endw
    mov eax,Result
    ret
String2Dword endp

CompareFunc proc uses edi lParam1:DWORD, lParam2:DWORD, SortType:DWORD
    LOCAL buffer[256]:BYTE
    LOCAL buffer1[256]:BYTE
    LOCAL lvi:LV_ITEM

    mov lvi.imask,LVIF_TEXT
    lea eax,buffer
    mov lvi.pszText,eax
    mov lvi.cchTextMax,256
    .if SortType==1
        mov lvi.iSubItem,1
        invoke SendMessage,hList,LVM_GETITEMTEXT,lParam1,addr lvi
        invoke String2Dword,addr buffer
        mov edi,eax
        invoke SendMessage,hList,LVM_GETITEMTEXT,lParam2,addr lvi
        invoke String2Dword,addr buffer
        sub edi,eax
        mov eax,edi
    .elseif SortType==2
        mov lvi.iSubItem,1
        invoke SendMessage,hList,LVM_GETITEMTEXT,lParam1,addr lvi
        invoke String2Dword,addr buffer
        mov edi,eax
        invoke SendMessage,hList,LVM_GETITEMTEXT,lParam2,addr lvi
        invoke String2Dword,addr buffer
        sub eax,edi
    .elseif SortType==3
        mov lvi.iSubItem,0
        invoke SendMessage,hList,LVM_GETITEMTEXT,lParam1,addr lvi
        invoke lstrcpy,addr buffer1,addr buffer
        invoke SendMessage,hList,LVM_GETITEMTEXT,lParam2,addr lvi
        invoke lstrcmpi,addr buffer1,addr buffer
    .else
        mov lvi.iSubItem,0
        invoke SendMessage,hList,LVM_GETITEMTEXT,lParam1,addr lvi
        invoke lstrcpy,addr buffer1,addr buffer
        invoke SendMessage,hList,LVM_GETITEMTEXT,lParam2,addr lvi
        invoke lstrcmpi,addr buffer,addr buffer1
    .endif
    ret
CompareFunc endp

UpdateiParam proc uses edi
    LOCAL lvi:LV_ITEM

    invoke SendMessage,hList, LVM_GETITEMCOUNT,0,0
    mov edi,eax
    mov lvi.imask,LVIF_PARAM
    mov lvi.iSubItem,0
    mov lvi.iItem,0
    .while edi>0
        push lvi.iItem
        pop lvi.lParam
        invoke SendMessage,hList, LVM_SETITEM,0,addr lvi
        inc lvi.iItem
        dec edi
    .endw
    ret
UpdateiParam endp

ShowCurrentFocus proc
    LOCAL lvi:LV_ITEM
    LOCAL buffer[256]:BYTE
    invoke SendMessage,hList,LVM_GETNEXTITEM,-1, LVNI_FOCUSED

```

```

    mov lvi.iItem,eax
    mov lvi.iSubItem,0
    mov lvi.imask,LVIF_TEXT
    lea eax,buffer
    mov lvi.pszText,eax
    mov lvi.cchTextMax,256
    invoke SendMessage,hList,LVM_GETITEM,0,addr lvi
    invoke MessageBox,0, addr buffer,addr AppName,MB_OK
    ret
ShowCurrentFocus endp

WndProc proc hWnd:HWND, uMsg:UINT, wParam:WPARAM, lParam:LPARAM
    .if uMsg==WM_CREATE
        invoke CreateWindowEx, NULL, addr ListViewClassName, NULL, \
            LVS_REPORT+WS_CHILD+WS_VISIBLE, 0,0,0,0,hWnd, NULL, hInstance, NULL
        mov hList, eax
        invoke InsertColumn
        invoke FillFileInfo
        RGB 255,255,255
        invoke SendMessage,hList,LVM_SETTEXTCOLOR,0,eax
        RGB 0,0,0
        invoke SendMessage,hList,LVM_SETBKCOLOR,0,eax
        RGB 0,0,0
        invoke SendMessage,hList,LVM_SETTEXTBKCOLOR,0,eax
        invoke GetMenu,hWnd
        mov hMenu,eax
        invoke CheckMenuRadioItem,hMenu,IDM_ICON,IDM_LIST, IDM_REPORT,MF_CHECKED
    .elseif uMsg==WM_COMMAND
        .if lParam==0
            invoke GetWindowLong,hList,GWL_STYLE
            and eax,not LVS_TYPEMASK
            mov edx,wParam
            and edx,0FFFFh
            push edx
            or eax,edx
            invoke SetWindowLong,hList,GWL_STYLE,eax
            pop edx
            invoke CheckMenuRadioItem,hMenu,IDM_ICON,IDM_LIST, edx,MF_CHECKED
        .endif
    .elseif uMsg==WM_NOTIFY
        push edi
        mov edi,lParam
        assume edi:ptr NMHDR
        mov eax,[edi].hwndFrom
        .if eax==hList
            .if [edi].code==LVN_COLUMNCLICK
                assume edi:ptr NM_LISTVIEW
                .if [edi].iSubItem==1
                    .if SizeSortOrder==0 || SizeSortOrder==2
                        invoke SendMessage,hList,LVM_SORTITEMS,1,addr CompareFunc
                        invoke UpdateParam
                        mov SizeSortOrder,1
                    .else
                        invoke SendMessage,hList,LVM_SORTITEMS,2,addr CompareFunc
                        invoke UpdateParam
                        mov SizeSortOrder,2
                    .endif
                .endif
            .else
                .if FileNameSortOrder==0 || FileNameSortOrder==4
                    invoke SendMessage,hList,LVM_SORTITEMS,3,addr CompareFunc
                    invoke UpdateParam
                    mov FileNameSortOrder,3
                .else
                    invoke SendMessage,hList,LVM_SORTITEMS,4,addr CompareFunc
                    invoke UpdateParam
                    mov FileNameSortOrder,4
                .endif
            .endif
        .endif
        assume edi:ptr NMHDR
    .elseif [edi].code==NM_DBLCLK
        invoke ShowCurrentFocus
    
```

```

        .endif
    .endif
    pop edi
.elseif uMsg==WM_SIZE
    mov eax,lParam
    mov edx,eax
    and eax,0ffffh
    shr edx,16
    invoke MoveWindow,hList, 0, 0, eax,edx,TRUE

.elseif uMsg==WM_DESTROY
    invoke PostQuitMessage,NULL
.else
    invoke DefWindowProc,hWnd,uMsg,wParam,lParam
    ret
.endif
xor eax,eax
ret
WndProc endp
end start

```

АНАЛИЗ

Первое, что должна сделать программа после того, как создано основное окно - это создать listview.

```

.if uMsg==WM_CREATE
    invoke CreateWindowEx, NULL, addr ListViewClassName, NULL, \
        LVS_REPORT+WS_CHILD+WS_VISIBLE, 0,0,0,0,hWnd, NULL, hInstance, NULL
    mov hList, eax

```

Мы вызываем CreateWindowEx, передавая ей имя класса окна "SysListView32". Режим отображения по умолчанию задан стилем LVS_REPORT.

```

    invoke InsertColumn

```

После того, как создан listview, мы вставляем в него колонку.

```

LOCAL lvc:LV_COLUMN

mov lvc.imask,LVCF_TEXT+LVCF_WIDTH
mov lvc.pszText,offset Heading1
mov lvc.lx,150
invoke SendMessage,hList, LVM_INSERTCOLUMN, 0, addr lvc

```

Мы указываем название и ширину первой колонки, в которой будут отображаться имена файлов, в структуре LV_COLUMN, поэтому нам нужно установить в imask флаги LVCF_TEXT и LVCF_WIDTH. Мы заполняем pszText адресом названия и lx - шириной колонки в пикселях. Когда все сделано, мы посылаем сообщение LVM_INSERTCOLUMN listview, передавая ей структуру.

```

or lvc.imask,LVCF_FMT
mov lvc.fmt,LVCFMT_RIGHT

```

После вставления первой колонки, мы вставляем следующую, в которой будут отображаться размеры файлов. Так как нам нужно, чтобы размеры файлов выравнивались по правой стороне, нам необходимо указать флаг в параметре fmt, LVCFMT_RIGHT. Мы также указываем флаг LVCF_FMT в imask, в добавление к LVCF_TEXT и LVCF_WIDTH.

```

mov lvc.pszText,offset Heading2
mov lvc.lx,100
invoke SendMessage,hList, LVM_INSERTCOLUMN, 1,addr lvc

```

Оставшийся код прост. Помещаем адреса названия в pszText и ширину в lx. Затем посылаем сообщение LVM_INSERTCOLUMN listview, указывая номер колонки и адрес структуры.

Когда колонки вставлены, мы можем заполнить listview элементами.

```

    invoke FillFileInfo

```

В FillFileInfo содержится следующий код.

```
FillFileInfo proc uses edi
    LOCAL finddata:WIN32_FIND_DATA
    LOCAL FHandle:DWORD
```

```
    invoke FindFirstFile,addr FileNamePattern,addr finddata
```

Мы вызываем FindFirstFile, чтобы получить информацию о первом файле, который отвечает заданным условиям. У FindFirstFile следующий прототип:

```
FindFirstFile proto pFileName:DWORD, pWin32_Find_Data:DWORD
```

pFileName - это адрес имени файла, который надо искать. Эта строка может содержать "дикие" символы. В нашем примере мы используем ., чтобы искать все файлы в данной папке.

pWin32_Find_Data - это адрес структуры WIN32_FIND_DATA, которая будет заполнена информацией о файле (если что-нибудь будет найдено).

Эта функция возвращает INVALID_HANDLE_VALUE в eax, если не было найдено соответствующих заданным критериям файлов. Иначе она возвратит хэндл поиска, который будет использован в последующих вызовах FindNextFile.

```
.if eax!=INVALID_HANDLE_VALUE
    mov FHandle,eax
    xor edi,edi
```

Если файл будет найден, мы сохраним хэндл поиска в переменную, а потом обнулим edi, который будет использован в качестве индекса элемента (номер ряда).

```
.while eax!=0
    test finddata.dwFileAttributes,FILE_ATTRIBUTE_DIRECTORY
    .if ZERO?
```

В этом tutorialе я не хочу иметь дело с папками, поэтому отфильтровываю их проверяя параметр dwFileAttributes на предмет наличия установленного флага FILE_ATTRIBUTE_DIRECTORY. Если он есть, я сразу перехожу к вызову FindNextFile.

```
        invoke ShowFileInfo,edi, addr finddata
        inc edi
    .endif
    invoke FindNextFile,FHandle,addr finddata
.endif
.endw
```

Мы вставляем имя и размер файла в listview вызывая функцию ShowFileInfo. Затем мы повышаем значение edi (текущий номер столбца). И, наконец, мы делаем вызов FindNextFile, чтобы найти следующий файл в нашей папке, пока FindNextFile не возвратит 0, что означает то, что больше файлов найдено не было.

```
        invoke FindClose,FHandle
    .endif
    ret
FillFileInfo endp
```

Когда все файлы в текущей папке найдены, мы должны закрыть хэндл поиска.

Теперь давайте взглянем на функцию ShowFileInfo. Эта функция принимает два параметра, индекс элемента (номер ряда) и адрес структуры WIN32_FIND_DATA.

```
ShowFileInfo proc uses edi row:DWORD, lpFind:DWORD
    LOCAL lvi:LV_ITEM
    LOCAL buffer[20]:BYTE
    mov edi,lpFind
    assume edi:ptr WIN32_FIND_DATA
```

Сохраняем адрес структуры WIN32_FIND_DATA в edi.

```
    mov lvi.imask,LVIF_TEXT+LVIF_PARAM
    push row
```

```

pop lvi.iItem
mov lvi.iSubItem,0

```

Мы предоставляем название элемента и значение IParam, поэтому мы помещаем флаги LVIF_TEXT и LVIF_PARAM в imask. Затем мы устанавливаем приравниваем item номер ряда, переданный функции и, так как это главный элемент, мы должны приравнять iSubItem нулю (колонка 0).

```

lea eax,[edi].cFileName
mov lvi.pszText,eax
push row
pop lvi.lParam

```

Затем мы помещаем адрес названия, в данном случае это имя файла в структуре WIN32_FIND_DATA, в pszText. Так как мы реализуем свою сортировку, мы должны заполнить IParam определенным значением. Я решил помещать номер ряда в этот параметр, чтобы я мог получать информацию об элементе по его индексу.

```

invoke SendMessage,hList, LVM_INSERTITEM,0, addr lvi

```

Когда все необходимые поля в LV_ITEM заполнены, мы посылаем сообщение LVM_INSERTITEM listview, чтобы вставить в него элемент.

```

mov lvi.imask,LVIF_TEXT
inc lvi.iSubItem
invoke wsprintf,addr buffer, addr template,[edi].nFileSizeLow
lea eax,buffer
mov lvi.pszText,eax

```

Мы установим подэлементы, ассоциированные с элементом. Подэлемент может иметь только название. Поэтому мы указываем в imask LVIF_TEXT. Затем мы указываем в iSubItem колонку, в которой должен находиться подэлемент. В этом случае мы устанавливаем его в 1. Названием этого элемента будет являться размер файла. Тем не менее, мы сначала должны конвертировать его в строку, вызвать wsprintf. Затем мы помещаем адрес строки в pszText.

```

invoke SendMessage,hList,LVM_SETITEM, 0,addr lvi
assume edi:nothing
ret
ShowFileInfo endp

```

Когда все требуемые поля в LV_ITEM заполнены, мы посылаем сообщение LVM_SETITEM listview, передавая ему адрес структуры LV_ITEM. Обратите внимание, что мы используем LVM_SETITEM, а не LVM_INSERTITEM, потому что подэлемент считается свойством элемента. Поэтому устанавливаем свойство элемента, а не вставляем новый элемент.

Когда все элементы вставлены в listview, мы устанавливаем текст и цвет бэкграунда контрола listview.

```

RGB 255,255,255
invoke SendMessage,hList,LVM_SETTEXTCOLOR,0,eax
RGB 0,0,0
invoke SendMessage,hList,LVM_SETBKCOLOR,0,eax
RGB 0,0,0
invoke SendMessage,hList,LVM_SETTEXTBKCOLOR,0,eax

```

Я использую макро RGB, чтобы конвертировать значения red, green, blue в eax и использую его для того, что указать нужное нам значение. Мы устанавливаем цвет текста и цвет фона с помощью сообщений LVM_SETTEXTCOLOR и LVM_SETTEXTBKCOLOR. Мы устанавливаем цвет фона listview сообщением LVM_SETBKCOLOR.

```

invoke GetMenu,hWnd
mov hMenu,eax
invoke CheckMenuRadioItem,hMenu,IDM_ICON,IDM_LIST, IDM_REPORT,MF_CHECKED

```

Мы позволим пользователю выбирать режимы отображения через меню. Поэтому мы должны получить сначала хэндл меню. Чтобы помочь юзеру переключать режимы отображения, мы помещаем в меню систему radio button'ов. Для этого нам понадобится функция CheckMenuItem. Эта функция поместит radio button перед пунктом меню.

Заметьте, что мы создаем окно listview с шириной и высотой равной нулю. Оно будет менять размер каждый раз, когда будет менять размер родительское окно. В этом случае мы можем быть уверены, что размер listview всегда будет соответствовать родительскому окну. В нашем примере нам требуется, чтобы listview занимал всю клиентскую область родительского окна.

```
.elseif uMsg==WM_SIZE
    mov eax,lParam
    mov edx,eax
    and eax,0ffffh
    shr edx,16
    invoke MoveWindow,hList, 0, 0, eax,edx,TRUE
```

Когда родительское окно получает сообщение WM_SIZE, нижнее слово lParam содержит новую ширину клиентской области и верхнее слово новой высоты. Тогда мы вызываем MoveWindow, чтобы изменить размер listview, чтобы тот покрывал всю клиентскую область родительского окна.

Когда пользователь выберет режим отображения в меню, мы должны соответственно отреагировать. Мы устанавливаем новый стиль контроля listview функцией SetWindowLong.

```
.elseif uMsg==WM_COMMAND
    .if lParam==0
        invoke GetWindowLong,hList,GWL_STYLE
        and eax,not LVS_TYPEMASK
```

Сначала мы получаем текущие стили listview. Затем мы стираем старый стиль отображения. LVS_TYPEMASK - это комбинированное значение всех четырех стилей отображения. Поэтому когда мы выполняем логическое умножение текущих флагов стилей со значением "not LVS_TYPEMASK", стиль текущего отображения стирается.

Во время проектирования меню я немного сжульничал. Я использовал в качестве ID пунктов меню константы стилей отображения.

```
IDM_ICON equ LVS_ICON
IDM_SMALLICON equ LVS_SMALLICON
IDM_LIST equ LVS_LIST
IDM_REPORT equ LVS_REPORT
```

Поэтому, когда родительское окно получает сообщение WM_COMMAND, нужный стиль отображения находится в нижнем слове wParam'a (как ID пункта меню).

```
mov edx,wParam
and edx,0FFFFh
```

Мы получили стиль отображения в нижнем слове wParam. Все, что нам теперь нужно, это обнулить верхнее слово.

```
push edx
or eax,edx
```

И добавить стиль отображения к уже существующим стилям (текущий стиль отображения мы ранее оттуда убрали).

```
invoke SetWindowLong,hList,GWL_STYLE,eax
```

И установить новые стили функцией SetWindowLong.

```
pop edx
invoke CheckMenuItem,hMenu,IDM_ICON,IDM_LIST, edx,MF_CHECKED

.endif
```

Нам также требуется поместить radio button перед выбранным пунктом меню. Поэтому мы вызываем CheckMenuRadioItem, передавая ей текущий стиль отображения (а также ID пункта меню).

Когда пользователь кликает по заголовку колонки в режиме отчета, нам нужно отсортировать элементы в listview. Мы должны отреагировать на сообщение WM_NOTIFY.

```
.elseif uMsg==WM_NOTIFY
    push edi
    mov edi,IParam
    assume edi:ptr NMHDR
    mov eax,[edi].hwndFrom
    .if eax==hList
```

Когда мы получаем сообщение WM_NOTIFY, IParam содержит указатель на структуру NMHDR. Мы можем проверить, пришло ли это сообщение от listview, сравнив параметр hwndFrom структуры NMHDR с хэндлом контрола listview. Если они совпадают, мы можем заключить, что уведомление пришло от listview.

```
.if [edi].code==LVN_COLUMNCLICK
    assume edi:ptr NM_LISTVIEW
```

Если уведомление пришло от listview, мы проверяем, равен ли код LVN_COLUMNCLICK. Если это так, это означает, что пользователь кликает на заголовке колонки. В случае, что код равен LVN_COLUMNCLICK, мы считаем, что IParam содержит указатель на структуру NM_LISTVIEW, которая является супермножеством по отношению к структуре NMHDR (т.е. включает ее). Затем нам нужно узнать, по какому заголовку колонки кликнул пользователь. Эту информацию мы получаем из параметра iSubItem. Его значение можно считать номером колонки (отсчет начинается с нуля).

```
.if [edi].iSubItem==1
    .if SizeSortOrder==0 || SizeSortOrder==2
```

Если iSubItem равен 1, это означает, что пользователь кликнул по второй колонке. Мы используем глобальные переменные, чтобы сохранять текущий статус порядка сортировки. 0 означает "еще не отсортировано", 1 значит "восходящая сортировка", а 2 - "нисходящая сортировка". Если элементы/подэлементы в колонке ранее не были отсортированы или отсортированы по нисходящей, то мы устанавливаем сортировку по восходящей.

```
    invoke SendMessage,hList,LVM_SORTITEMS,1,addr CompareFunc
```

Мы посылаем сообщение LVM_SORTITEMS listview, передавая 1 через wParam и адрес нашей сравнивающей функции через lParam. Заметьте, что значение в wParam задается пользователем, вы можете использовать его как хотите. Я использовал его в нашем примере как метод сортировки. Сначала мы взглянем на сравнивающую функцию.

```
CompareFunc proc uses edi lParam1:DWORD, lParam2:DWORD, SortType:DWORD
    LOCAL buffer[256]:BYTE
    LOCAL buffer1[256]:BYTE
    LOCAL lvi:LV_ITEM

    mov lvi.imask,LVIF_TEXT
    lea eax,buffer
    mov lvi.pszText,eax
    mov lvi.cchTextMax,256
```

В сравнивающей функции контрол listview будет передавать lParam'ы (через LV_ITEM) двух элементов, которые нужно сравнить, через lParam1 и lParam2. Вспомните, что мы помещаем индекс элемента в lParam. Таким образом мы можем получить информацию об элементах, используя эти индексы. Информация, которая нам нужна - это названия сортирующихся элементов/подэлементов. Мы подготавливаем структуру LV_ITEM для этого, указывая в imask LVIF_TEXT и адрес буфера в pszText и размер буфера в cchTextMax.


```

.if SortType==1
    mov lvi.iSubItem,1
    invoke SendMessage,hList,LVM_GETITEMTEXT,lParam1,addr lvi

```

Если значение SortType равно 1 или 2, мы знаем, что кликнута колонка размера файла. 1 означает, что необходимо отсортировать элементы в нисходящем порядке. 2 значит обратное. Таким образом мы указываем iSubItem равным 1 (чтобы задать колонку размера) и посылаем сообщение LVM_GETITEMTEXT контролю listview, чтобы получить название (строку с размером файла) подэлемента.

```

    invoke String2Dword,addr buffer
    mov edi,eax

```

Конвертируем строку в двойное слово с помощью функции String2Dword, написанную мной. Она возвращает dword-значение в eax. Мы сохраняем ее в edi для последующего сравнения.

```

    invoke SendMessage,hList,LVM_GETITEMTEXT,lParam2,addr lvi
    invoke String2Dword,addr buffer
    sub edi,eax
    mov eax,edi

```

То же самое мы делаем и с lParam2. После получения размеров обоих файлов, мы можем сравнить их.

Правила, которых придерживается функция сравнения, следующие:

- Если первый элемент должен предшествовать другому, вы должны вернуть отрицательное значение через eax.
- Если второй элемента должен предшествовать первому, вы должны вернуть через eax положительное значение.
- Если оба элемента равны, вы должны вернуть ноль.

В нашем случае нам нужно отсортировать элементы согласно их размерам в восходящем порядке. Поэтому мы просто можем вычесть размер первого элемента из второго и вернуть результат в eax.

```

.elseif SortType==3
    mov lvi.iSubItem,0
    invoke SendMessage,hList,LVM_GETITEMTEXT,lParam1,addr lvi
    invoke lstrcpy,addr buffer1,addr buffer
    invoke SendMessage,hList,LVM_GETITEMTEXT,lParam2,addr lvi
    invoke lstrcmpi,addr buffer1,addr buffer

```

В случае, если пользователь кликнет по колонке с именем файла, мы должны сравнивать имена файлов. Мы должны получить имена файлов, а затем сравнить их с помощью функции lstrcmpi. Мы можем вернуть значение, возвращаемое этой функцией, так как оно использует те же правила сравнения.

После того, как элементы отсортированы, нам нужно обновить значения lParam'ов для всех элементов, чтобы учесть изменившиеся индексы элементов, поэтому мы вызываем функцию UpdateLParam.

```

    invoke UpdateLParam
    mov SizeSortOrder,1

```

Эта функция просто-напросто перечисляет все элементы в listview и обновляет значения lParam. Нам требуется это делать, иначе следующая сортировка не будет работать как ожидается, потому что мы исходим из того, что значение lParam - это индекс элемента.

```

.elseif [edi].code==NM_DBLCLK
    invoke ShowCurrentFocus
.endif

```

Когда пользователь делает двойной клик на элементе, нам нужно отобразить окно с сообщением с названием элемента. Мы должны проверить, равно ли поле code в NMHDR NM_DBLCLK. Если это так, мы можем перейти к получению названия и отображению его в окне с сообщением.

```
ShowCurrentFocus proc
    LOCAL lvi:LV_ITEM
    LOCAL buffer[256]:BYTE

    invoke SendMessage,hList,LVM_GETNEXTITEM,-1, LVNI_FOCUSED
```

Как мы можем узнать, по какому элементу кликнули два раза? Когда элемент кликнут (одинарным или двойным нажатием), он получает фокус. Даже если выбрано несколько элементов, фокус будет только у одного. Наша задача заключается в том, чтобы найти элемент у которого находится фокус. Мы делаем это, посылая сообщение LVM_GETNEXTITEM контролу listview, указав желаемое состояние элемента в lParam. -1 в wParam означает поиск по всем элементам. Индекс элемента возвращается в eax.

```
mov lvi.iItem,eax
mov lvi.iSubItem,0
mov lvi.imask,LVIF_TEXT
lea eax,buffer
mov lvi.pszText,eax
mov lvi.cchTextMax,256
invoke SendMessage,hList,LVM_GETITEM,0,addr lvi
```

Затем мы получаем название элемента с помощью сообщения LVM_GETITEM.

```
invoke MessageBox,0, addr buffer,addr AppName,MB_OK
```

И наконец, мы отображаем название элемента в окне сообщения.

Если вы хотите узнать, как использовать в контроле listview иконки, вы можете прочитать об этом в моем tutorialе о treeview. В случае с listview надо будет сделать примерно то же самое.

Win32 API. Урок 32. MDI-интерфейс

Автор: lczelion, пер. Aquila

Дата: Не указана

Этот tutorial расскажет, как создать MDI-приложение. Это не так сложно. Скачайте пример (находится в архиве wasm_files.zip: files/recovered/1001032/tut32.zip).

ТЕОРИЯ

Мультидокументный интерфейс - это спецификация для приложений, которые обрабатывают несколько документов в одно и то же время. Вы знакомы с Notepad'ом: это пример однодокументного интерфейса (SDI). Notepad может обрабатывать только один документ за раз. Если вы хотите открыть другой документ, вам нужно закрыть предыдущий. Как вы можете себе представить, это довольно неудобно. Сравните его с Microsoft Word: тот может держать открытыми различные документы в одно и то же время и позволяет пользователю выбирать, какой документ использовать.

У MDI-приложений есть несколько характеристик, присущих только им. Я перечислю некоторые из них:

- Внутри основного окна может быть несколько дочерних окон в пределах клиентской области.
- Когда вы сворачиваете окно, он сворачивается к нижнему левому углу клиентской области основного окна.
- Когда вы разворачиваете окно, его заголовок сливается с заголовком главного окна.
- Вы можете закрыть дочернее окно, нажав Ctrl+F4 и переключаться между дочерними окнами, нажав на Ctrl+Tab.

Главное окно, которое содержит дочерние окна называется фреймовым окном. Его клиентская область - это место, где находятся дочерние окна, поэтому оно и называется фреймовым (на английском 'frame' означает "рамка, рама"). Его работа чуть более сложна, чем задачи обычного окна, так как оно обеспечивает работу MDI.

Чтобы контролировать дочерние окна в клиентской области, вам нужно специальное окно, которое называется клиентским окном. Вы можете считать это клиентское окно прозрачным окном, покрывающим всю клиентскую область фреймового окна.

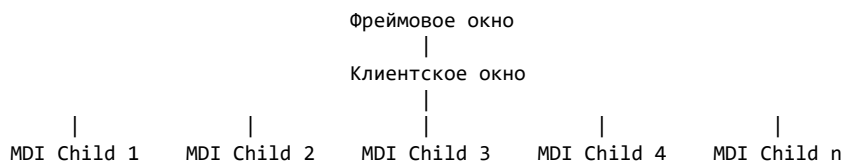


Рисунок 1. Иерархия MDI-приложения

Создание фреймового окна

Теперь мы переключим наше внимание на детали. Прежде всего вам нужно создать фреймовое окно. Оно создается примерно таким же образом, как и обычное окно: с помощью вызова CreateWindowEx. Есть два основных отличия от создания обычного окна.

Первое различие состоит в том, что вы ДОЛЖНЫ вызывать DefFrameProc вместо DefWindowProc для обработки Windows-сообщений вашему окну, которые вы не хотите обрабатывать самостоятельно. Это единственный путь заставить Windows делать за вас грязную работу по управлению MDI-приложением. Если вы забудете использовать DefFrameProc, ваше приложение не будет иметь MDI-свойств. DefFrameProc имеет следующий синтаксис:

```
DefFrameProc proc hwndFrame:DWORD,
```

```

hWndClient:DWORD,
uMsg:DWORD,
wParam:DWORD,
lParam:DWORD

```

Если вы сравните DefFrameProc с DefWindowProc, вы заметите, что разница между ними состоит в том, что у DefFrameProc пять параметров, в то время как у DefWindowProc только четыре. Дополнительный параметр - это хэндл клиентского окна. Это хэндл необходим для того, чтобы Windows могла посылать MDI-сообщения клиентскому окну.

Второе различие заключается в том, что вы должны вызывать TranslateMDISysAccel в цикле обработки сообщений вашего фреймового окна. Это необходимо, если вы хотите, чтобы Windows обрабатывала нажатия на комбинации клавиш, связанных с MDI, такие как Ctrl+F4, Ctrl+Tab. У этой функции следующий прототип:

```

TranslateMDISysAccel proc hWndClient:DWORD, lpMsg:DWORD

```

Первый параметр - это хэндл клиентского окна. Для вас не должно быть сюрпризом, что клиентское окно будет родителем окном для все дочерних MDI-окон. Вторым параметром - это адрес MSG-структуры, которую вы заполните с помощью функции GetMessage. Идея состоит в том, чтобы передать MSG-структуру клиентскому окну, чтобы оно могло проверить, содержит ли эта структура искомые комбинации клавиш. Если это так, она обрабатывает само сообщение и возвращает ненулевое значение, или, в противном случае, FALSE.

Этапы создания фреймового окна могут быть кратко просуммированы:

- Заполняем структуру WNDCLASSEX как обычно.
- Регистрируем класс фреймового окна, вызывая RegisterClassEx.
- Создаем фреймовое окно с помощью CreateWindowEx.
- Внутри цикла обработки сообщений вызываем TranslateMDISysAccel.
- Внутри процедуры окна передаем необработанные сообщения DefFrameProc вместо DefWindowProc.

Создание клиентского окна

Теперь, когда у нас есть фреймовое окно, мы можем создать клиентское окно. Класс клиентского окна зарегистрирован Windows. Имя этого класса - "MDICLIENT". Вам также нужно передать адрес структуры CLIENTCREATESTRUCT функции CreateWindowEx. Эта структура имеет следующее определение:

```

CLIENTCREATESTRUCT struct
    hWndMenu      dd ?
    idFirstChild   dd ?
CLIENTCREATESTRUCT ends

```

hWndMenu - это хэндл подменю, к которому Windows присоединит список имен дочерних MDI-окон. Здесь требуется некоторое пояснение. Если вы когда-нибудь использовали раньше MDI-приложение вроде Microsoft Word, вы могли заметить, что у него есть подменю под названием "window", которое при активации отображает различные пункты меню, связанные с управлением дочерними окнами, а также список открытых дочерних окон. Этот список создается самими Windows: вам не нужно прилагать специальных усилий. Всего лишь передайте хэндл подменю, к которому должен быть присоединен список, а Windows возьмет на себя все остальное. Обратите внимание, что подменю может быть любым: не обязательно тем, которое названо "window". Если вам не нужен список окон, просто передайте NULL в hWndMenu. Получить хэндл подменю можно с помощью GetSubMenu.

idFirstChild - ID первого дочернего MDI-окна. Windows увеличивает ID на 1 для каждого нового MDI-окна, которое создает приложение. Например, если вы передаете 100 через это поле, первое MDI-окно будет иметь ID 100, второе - 101 и так далее. Это ID посылается фреймовому окну через

WM_COMMAND, когда дочернее MDI-окно выбрано из списка окон. Обычно вы будете передавать эти "необрабатываемые" сообщения процедуре DefFrameProc. Я использую слово "необрабатываемые", потому что пункты меню списка окон не создаются вашим приложением, поэтому ваше приложение не знает их ID и не имеет обработчика для них. Поэтому для фреймового окна есть специальное правило: если у вас есть список окон, вы должны модифицировать ваш обработчик WM_COMMAND так:

```
.elseif uMsg==WM_COMMAND

    .if lParam==0
        mov eax,wParam
        .if ax==IDM_CASCADE
            .....

        .elseif ax==IDM_TILEVERT
            .....
        .else
            invoke DefFrameProc, hwndFrame, hwndClient, uMsg,wParam,
            ret
        .endif
```

Обычно вам следует игнорировать сообщения о необрабатываемых событиях, но в случае с MDI, если вы просто проигнорируете их, то когда пользователь кликнет на имени дочернего MDI-окна, это окно не станет активным. Вам следует передавать управление DefFrameProc, чтобы они были правильно обработаны.

Я хочу предупредить вас относительно возможного значения idFirstChild: вам не следует использовать 0. Ваш список окон будет себя вести неправильно, то есть напротив пункта меню, обозначающего активное MDI-окно, не будет галочки. Лучше выберите какое-нибудь безопасное значение вроде 100 или выше.

Заполнив структуру CLIENTCREATESTRUCT, вы можете создать клиентское окно, вызвав CreateWindowEx, указав предопределенный класс "MDICLIENT" и передав адрес структуры CLIENTCREATESTRUCT через lParam. Вы должны также указать хэндл на фреймовое окно в параметре hWndParent, чтобы Windows знала об отношениях родитель-ребенок между фреймовым окном и клиентским окном. Вам следует использовать следующие стили окна: WS_CHILD, WS_VISIBLE и WS_CLIPCHILDREN. Если вы забудете указать стиль WS_VISIBLE, то не увидите дочерних MDI-окон, даже если они будут созданы успешно.

Этапы создания клиентского окна следующие:

- Получить хэндл на подменю, к которому вы хотите присоединить список окон.
- Поместите значение хэндла меню и значения, которое вы хотите использовать как ID первого дочернего MDI-окна в структуру CLIENTCREATESTRUCT.
- Вызовите CreateWindowEx, передав имя класса "MDICLIENT" и адрес структуры CLIENTCREATESTRUCT, которую вы только что заполнили, через lParam.

Создание дочернего MDI-окна.

Теперь у вас есть и фреймовое и клиентское окно. Теперь все готово для создания дочернего MDI-окна. Есть два пути сделать это.

- Вы можете послать сообщение WM_MDICREATE клиентскому окну, передав тому адрес структуры типа MDICREATESTRUCT через wParam. Это простейший и наиболее часто используемый способ создания дочерних MDI-окон.

```
.data?
    mdicreate MDICREATESTRUCT <>
    ....
.code
    .....
    [fill the members of mdicreate]
```

```
[ заполняем поля структуры mdicreate ]
.....
invoke SendMessage, hwndClient, WM_MDICREATE, addr mdicreate, 0
```

Функция `SendMessage` возвратит хэндл только что созданного дочернего MDI-оанк, если все пройдет успешно. Вам не нужно сохранять хэндл. Вы можете получить его каким-либо другим образом, если захотит. У структуры `MDICREATESTRUCT` следующее определение:

```
MDICREATESTRUCT STRUCT
szClass    DWORD ?
szTitle    DWORD ?
hOwner     DWORD ?
x          DWORD ?
y          DWORD ?
lx         DWORD ?
ly         DWORD ?
style      DWORD ?
lParam     DWORD ?
MDICREATESTRUCT ENDS
```

- `szClass` - адрес класса окна, который вы хотите использовать в качестве шаблона для дочернего MDI-окна.
- `szTitle` - адрес строки, которая должна появиться в заголовке дочернего окна.
- `hOwner` - хэндл приложения.
- `x`, `y`, `lx`, `ly` - верхняя левая координата, ширина и высота дочернего окна.
- `style` - стиль дочернего окна. Если вы создали клиентское окно со стилем `MDIS_ALLCHILDSTYLES`, то вы можете использовать все стили.
- `lParam` - определяемое программистом 32-х битное значение. Используется для передачи значение между MDI-окнами. Если у вас нет подобной нужды, просто поставьте их в `NULL`.
- Вы можете вызвать `CreateMDIWindow`. Эта функция имеет следующий синтаксис:

```
CreateMDIWindow proto lpClassName:DWORD
lpWindowName:DWORD
dwStyle:DWORD
x:DWORD
y:DWORD
nWidth:DWORD
nHeight:DWORD
hWndParent:DWORD
hInstance:DWORD
lParam:DWORD
```

Если вы внимательно посмотрите на параметры, вы увидите, что они идентичны параметрам структуры `MDICREATESTRUCT` не считая `hWndParent`. Очевидно, что точно такое количество параметров вы передавали вместе с сообщением `WM_MDICREATE`. У структуры `MDICREATESTRUCT` нет поля `hWndParent`, потому что вы все равно должны передавать структуру клиентскому окну с помощью функции `SendMessage`.

Сейчас вы можете спросить: какой метод я должен выбрать? Какая разница между этими двумя методами? Вот ответ:

Метод `WM_MDCREATE` создает дочернее MDI-окно в том же треде, что и вызывающий код. Это означает, что если у приложения есть только одна ветвь, все дочерние MDI-окна будут выполняться в контексте основной ветви. Это не слишком большая проблема, если одно или более из дочерних MDI-окон не выполняет какие-либо продолжительные операции, что может стать проблемой. Подумайте об этом, иначе в какой-то момент все ваше приложение внезапно зависнет, пока операция не будет выполнена.

Эта проблема как раз то, что призвана решить функция `CreateMDIWindow`. Она создает отдельный тред для каждого из дочерних MDI-окон, поэтому если одно из них занято, оно не приводит к зависанию всего приложения.

Необходимо сказать несколько слов относительно оконной процедуры дочернего MDI-окна. Как и в случае с фреймовым окном, вы не должны вызывать DefWindowProc, чтобы обработать необрабатываемые вашим приложением сообщения. Вместо этого вы должны использовать DefMDIChildProc. У этой функции точно такие же параметры, как и у DefWindowProc.

Кроме WM_MDICREATE, есть еще несколько сообщений, относящихся к MDI-окнам.

Я приведу их описание:

- WM_MDIACTIVATE - это сообщение может быть послано приложением клиентскому окну, чтобы последнее активировало выбранное дочернее MDI-окно. Когда клиентское окно получает сообщение, оно активирует выбранное дочернее MDI-окно, а также посылает это же сообщение окну, которое было активировано или деактивировано. Таким образом, данное сообщение имеет двойное назначение: с его помощью приложение может активировать выбранное дочернее окно, а также оно может быть использовано дочерним MDI-приложением для определения того, активировано оно или нет. Например, если каждое дочернее MDI-окно имеет различное меню, оно может использовать эту возможность для изменения меню фреймового окна при активации/деактивации.
- WM_MDICASCADE, WM_MDITILE, WM_MDICONARRANGE - эти сообщения отвечают за расположение дочерних MDI-окон. Например, если вы хотите, чтобы дочерние MDI-окна расположились каскадом, пошлите сообщение WM_MDICASCADE клиентскому окну.
- WM_MDIDESTROY - пошлите это сообщение клиентскому окну, если хотите уничтожить дочернее MDI-окно. Вам следует использовать это сообщение вместо DestroyWindow, потому что если дочернее MDI-приложение максимизировано, это сообщение восстановит заголовок фреймового окна, что не будет сделано в случае применения функции DestroyWindow.
- WM_MDIGETACTIVE - используйте это сообщение, чтобы получить хэндл активного в настоящий момент дочернего MDI-окна.
- WM_MDIMAXIMIZE, WM_MDIRESTORE - используйте WM_MDIMAXIMIZE для разворачивания дочернего MDI-окна и WM_MDIRESTORE для его восстановления. Всегда используйте эти сообщения для данных операций. Если вы используете ShowWindow с SW_MAXIMIZE, дочернее MDI-окно будет развернуто, но у вас появятся проблемы, когда вы захотите восстановить его до прежнего размера. Вы можете минимизировать окно с помощью ShowWindow без всяких проблем.
- WM_MDINEXT - посылайте это сообщение клиентскому окну, чтобы активировать следующее или предыдущее дочернее MDI-окно, согласно значению wParam и lParam.
- WM_MDIREFRESHMENU - это сообщение посылается клиентскому окну, чтобы обновить меню фреймового окна. Обратите внимание, что вы должны вызвать DrawMenuBar для обновления меню бар после отсылки данного сообщения.
- WM_MDISETMENU - посылайте это сообщение клиентскому окну, чтобы полностью заменить меню фреймового окна или только подменю окон. Вы должны использовать данное сообщение вместо SetMenu. После того, как вы отослали данное сообщение, вы должны вызвать DrawMenuBar. Обычно вы будете использовать это сообщение, когда у активного дочернего MDI-окна есть свое меню и вы хотите, чтобы оно заменяло меню фреймового окна, пока это дочернее MDI-окно активно.

Я сделаю небольшое обзрение создания MDI-приложения еще раз:

- Регистрируем классы окна, фреймового класса и дочернего MDI-окна.
- Создаем фреймовое окно с помощью CreateWindowEx.
- Внутри цикла обработки сообщений вызываем TranslateMDISysAccel, чтобы обработать "горячие клавиши", относящиеся к MDI.
- Внутри оконной процедуры фреймового окна вызываем DefFramProc, чтобы обрабатывать все сообщения, необрабатываемые приложением.
- Создаем клиентское окно, вызвав CreateWindowEx, которой передаем имя предопределенного класса окна, "MDICLIENT", передавая адрес структуры CLIENTCREATESTRUCT через lParam.

Обычно вы будете создавать клиентское окно внутри обработчика сообщения WM_CREATE фреймового окна.

- Вы можете создать дочернее MDI-окно, послав клиентскому окну сообщение WM_MDICREATE или вызвав функцию CreateMDIWindow.
- Внутри оконной процедуры дочернего MDI-ОКНА, передаем все необработанные сообщения DefMDIChildProc.
- Используем MDI-версии сообщений, если таковые существуют. Например, вместо WM_MDIDESTROY вместо DestroyWindow.

ПРИМЕР

```
.386
.model flat,stdcall
option casemap:none
include \masm32\include\windows.inc
include \masm32\include\user32.inc
include \masm32\include\kernel32.inc
includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib
WinMain proto :DWORD,:DWORD,:DWORD,:DWORD

.const
IDR_MAINMENU      equ 101
IDR_CHILDMENU     equ 102
IDM_EXIT          equ 40001
IDM_TILEHORZ      equ 40002
IDM_TILEVERT      equ 40003
IDM_CASCADE       equ 40004
IDM_NEW           equ 40005
IDM_CLOSE         equ 40006

.data
ClassName         db "MDIASMClass",0
MDIClientName     db "MDICLIENT",0
MDIChildClassName db "Win32asmMDIChild",0
MDIChildTitle     db "MDI Child",0
AppName           db "Win32asm MDI Demo",0
ClosePromptMessage db "Are you sure you want to close this window?",0

.data?
hInstance         dd ?
hMainMenu         dd ?
hwndClient        dd ?
hChildMenu        dd ?
mdicreate         dd MDICREATESTRUCT <>
hwndFrame         dd ?

.code
start:
    invoke GetModuleHandle, NULL
    mov hInstance,eax
    invoke WinMain, hInstance,NULL,NULL, SW_SHOWDEFAULT
    invoke ExitProcess,eax

WinMain proc hInst:HINSTANCE,hPrevInst:HINSTANCE,CmdLine:LPSTR,CmdShow:DWORD
    LOCAL wc:WNDCLASSEX
    LOCAL msg:MSG
    ;=====
    ; Register the frame window class
    ;=====
    mov wc.cbSize,SIZEOF WNDCLASSEX
    mov wc.style, CS_HREDRAW or CS_VREDRAW
    mov wc.lpfnWndProc,OFFSET WndProc
    mov wc.cbClsExtra,NULL
    mov wc.cbWndExtra,NULL
    push hInstance
    pop wc.hInstance
    mov wc.hbrBackground,COLOR_APPWORKSPACE
```



```

mov wc.lpszMenuName, IDR_MAINMENU
mov wc.lpszClassName, OFFSET ClassName
invoke LoadIcon, NULL, IDI_APPLICATION
mov wc.hIcon, eax
mov wc.hIconSm, eax
invoke LoadCursor, NULL, IDC_ARROW
mov wc.hCursor, eax
invoke RegisterClassEx, addr wc
;=====
; Register the MDI child window class
;=====
mov wc.lpfnWndProc, offset ChildProc
mov wc.hbrBackground, COLOR_WINDOW+1
mov wc.lpszClassName, offset MDIChildClassName
invoke RegisterClassEx, addr wc
invoke CreateWindowEx, NULL, ADDR ClassName, ADDR AppName, \
    WS_OVERLAPPEDWINDOW or WS_CLIPCHILDREN, CW_USEDEFAULT,
    CW_USEDEFAULT, CW_USEDEFAULT, CW_USEDEFAULT, NULL, 0, \
    hInst, NULL
mov hwndFrame, eax
invoke LoadMenu, hInstance, IDR_CHILDMENU
mov hChildMenu, eax
invoke ShowWindow, hwndFrame, SW_SHOWNORMAL
invoke UpdateWindow, hwndFrame
.while TRUE
    invoke GetMessage, ADDR msg, NULL, 0, 0
    .break .if (!eax)
    invoke TranslateMDISysAccel, hwndClient, addr msg
    .if !eax
        invoke TranslateMessage, ADDR msg
        invoke DispatchMessage, ADDR msg
    .endif
.endw
invoke DestroyMenu, hChildMenu
mov eax, msg.wParam
ret
WinMain endp

WndProc proc hwnd:HWND, uMsg:UINT, wParam:WPARAM, lParam:LPARAM
LOCAL ClientStruct:CLIENTCREATESTRUCT
.if uMsg==WM_CREATE
    invoke GetMenu, hwnd
    mov hMainMenu, eax
    invoke GetSubMenu, hMainMenu, 1
    mov ClientStruct.hWindowMenu, eax
    mov ClientStruct.idFirstChild, 100
    INVOKE CreateWindowEx, NULL, ADDR MDIClientName, NULL, \
        WS_CHILD or WS_VISIBLE or WS_CLIPCHILDREN, CW_
        CW_USEDEFAULT, CW_USEDEFAULT, CW_USEDEFAULT, hwn
        hInstance, addr ClientStruct
    mov hwndClient, eax
    ;=====
    ; Initialize the MDICREATESTRUCT
    ;=====
    mov mdicreate.szClass, offset MDIChildClassName
    mov mdicreate.szTitle, offset MDIChildTitle
    push hInstance
    pop mdicreate.hOwner
    mov mdicreate.x, CW_USEDEFAULT
    mov mdicreate.y, CW_USEDEFAULT
    mov mdicreate.lx, CW_USEDEFAULT
    mov mdicreate.ly, CW_USEDEFAULT
.elseif uMsg==WM_COMMAND
    .if lParam==0
        mov eax, wParam
        .if ax==IDM_EXIT
            invoke SendMessage, hwnd, WM_CLOSE, 0, 0
        .elseif ax==IDM_TILEHORZ
            invoke SendMessage, hwndClient, WM_MDITILE, MDIT
        .elseif ax==IDM_TILEVERT
            invoke SendMessage, hwndClient, WM_MDITILE, MDIT

```

```

        .elseif ax==IDM_CASCADE
            invoke SendMessage,hwndClient,WM_MDICASCADE,M
        .elseif ax==IDM_NEW
            invoke SendMessage,hwndClient,WM_MDICREATE,0,
        .elseif ax==IDM_CLOSE
            invoke SendMessage,hwndClient,WM_MDIGETACTIVE
            invoke SendMessage,eax,WM_CLOSE,0,0
        .else
            invoke DefFrameProc,hWnd,hwndClient,uMsg,wPar
            ret
        .endif
    .endif
.elseif uMsg==WM_DESTROY
    invoke PostQuitMessage,NULL
.else
    invoke DefFrameProc,hWnd,hwndClient,uMsg,wParam,lParam
    ret
.endif
xor eax,eax
ret
WndProc endp

ChildProc proc hChild:DWORD,uMsg:DWORD,wParam:DWORD,lParam:DWORD
    .if uMsg==WM_MDIACTIVATE
        mov eax,lParam
        .if eax==hChild
            invoke GetSubMenu,hChildMenu,1
            mov edx,eax
            invoke SendMessage,hwndClient,WM_MDISETMENU,hChildMen
        .else
            invoke GetSubMenu,hMainMenu,1
            mov edx,eax
            invoke SendMessage,hwndClient,WM_MDISETMENU,hMainMenu
        .endif
        invoke DrawMenuBar,hwndFrame
    .elseif uMsg==WM_CLOSE
        invoke MessageBox,hChild,addr ClosePromptMessage,addr AppName
        .if eax==IDYES
            invoke SendMessage,hwndClient,WM_MDIDESTROY,hChild,0
        .endif
    .else
        invoke DefMDIChildProc,hChild,uMsg,wParam,lParam
        ret
    .endif
    xor eax,eax
    ret
ChildProc endp
end start

```

АНАЛИЗ

Первое, что должна сделать программа - это зарегистрировать классы фреймового и дочернего MDI-окна. После этого она вызывает функцию CreateWindowEx, чтобы создать фреймовое окно. Внутри обработчика WM_CREATE фреймового окна мы создаем клиентское окно:

```

LOCAL ClientStruct:CLIENTCREATESTRUCT
.if uMsg==WM_CREATE
    invoke GetMenu,hWnd
    mov hMainMenu,eax
    invoke GetSubMenu,hMainMenu,1
    mov ClientStruct.hWindowMenu,eax
    mov ClientStruct.idFirstChild,100
    invoke CreateWindowEx,NULL,ADDR MDIClientName,NULL,\
        WS_CHILD or WS_VISIBLE or WS_CLIPCHILDREN,CW_USEDEFAU
        CW_USEDEFAULT,CW_USEDEFAULT,CW_USEDEFAULT,hWnd,NULL,\
        hInstance,addr ClientStruct
    mov hwndClient,eax

```

Здесь мы вызываем GetMenu, чтобы получить хэндл меню фреймового окна, который будем использовать в GetSubMenu. Обратите внимание, что мы передаем 1 функции GetSubMenu, потому

что подменю, к которому мы будем присоединять список окон, является вторым подменю. Затем мы заполняем параметры структуры CLIENTCREATESTRUCT.

Затем мы инициализируем структуру MDCLIENTSTRUCT. Обратите внимание, что мы не обязаны делать это здесь. Просто это удобнее осуществлять в обработчике WM_CREATE.

```
mov mdicreate.szClass,offset MDIChildClassName
mov mdicreate.szTitle,offset MDIChildTitle
push hInstance
pop mdicreate.hOwner
mov mdicreate.x,CW_USEDEFAULT
mov mdicreate.y,CW_USEDEFAULT
mov mdicreate.lx,CW_USEDEFAULT
mov mdicreate.ly,CW_USEDEFAULT
```

После того, как фреймовое окно создано (так же как клиентское окно), мы вызываем LoadMenu, чтобы загрузить меню дочернего окна из ресурса. Нам нужно получить хэндл этого меню, чтобы мы могли заменить меню фреймового окна, когда дочернее MDI-окно становится активным. Не забудьте вызвать DestroyMenu, прежде чем приложение завершит работу. Обычно Windows сама освобождает память, занятую меню, но в данном случае этого не произойдет, так как меню дочернего окна не ассоциировано ни с каким окном, поэтому оно все еще будет занимать ценную память, хотя приложение уже прекратило свое выполнение.

```
invoke LoadMenu,hInstance, IDR_CHILDMENU
mov hChildMenu,eax
.....
invoke DestroyMenu, hChildMenu
```

Внутри цикла обработки сообщений, мы вызываем TranslateMDISysAccel.

```
.while TRUE
    invoke GetMessage,ADDR msg,NULL,0,0
    .break .if (!eax)
    invoke TranslateMDISysAccel,hwndClient,addr msg
    .if !eax
        invoke TranslateMessage, ADDR msg
        invoke DispatchMessage, ADDR msg
    .endif
.endw
```

Если TranslateMDISysAccel возвращает ненулевое значение, это означает, что сообщение уже было обработано Windows, поэтому вам не нужно делать что-либо с ним. Если был возвращен 0, сообщение не относится к MDI и поэтому должно обрабатываться как обычно.

```
WndProc proc hWnd:HWND, uMsg:UINT, wParam:WPARAM, lParam:LPARAM
    ....
    .else
        invoke DefFrameProc,hWnd,hwndClient,uMsg,wParam,lParam
        ret
    .endif
    xor eax,eax
    ret
WndProc endp
```

Обратите внимание, что внутри оконной процедуры фреймового окна мы вызываем DefFrameProc для обработки сообщений, которые не представляют для нас интереса.

Основной частью процедуры окна является обработчик сообщения WM_COMMAND. Когда пользователь выбирает в меню пункт "New", мы создаем новое дочернее MDI-окно.

```
.elseif ax==IDM_NEW
    invoke SendMessage,hwndClient,WM_MDICREATE,0,addr mdicreate
```

В нашем примере мы создаем дочернее MDI-окно, посылая WM_MDICREATE клиентскому окну, передавая адрес структуры MDICREATESTRUCT через lParam.

```
ChildProc proc hChild:DWORD,uMsg:DWORD,wParam:DWORD,lParam:DWORD
```

```

.if uMsg==WM_MDIACTIVATE
    mov eax,lParam
    .if eax==hChild
        invoke GetSubMenu,hChildMenu,1
        mov edx,eax
        invoke SendMessage,hwndClient,WM_MDISETMENU,hChildMen
    .else
        invoke GetSubMenu,hMainMenu,1
        mov edx,eax
        invoke SendMessage,hwndClient,WM_MDISETMENU,hMainMenu
    .endif
    invoke DrawMenuBar,hwndFrame

```

Когда создано дочернее MDI-окно, оно отслеживает сообщение WM_MDIACTIVATE, чтобы определить, является ли оно в данный момент активным. Оно делает это сравнивая значение lParam, которое содержит хэндл активного дочернего окна со своим собственным хэндлом.

Если они совпадают, значит оно является активным и следующим шагом будет замена меню фреймового окна на свое собственное. Так как изначально меню будет заменено, вам надо будет указать Windows снова в каком подменю должен появиться список окон. Поэтому мы должны снова вызвать функцию GetSubMenu, чтобы получить хэндл подменю. Мы посылаем сообщение WM_MDISETMENU клиентскому окну, достигая, таким образом, желаемого результата. Параметр wParam сообщения WM_MDISETMENU содержит хэндл меню, которое заменит оригинальное. lParam содержит хэндл подменю, к которому будет присоединен список окон. Сразу после отсылки сообщения WM_MDISETMENU, мы вызываем DrawMenuBar, чтобы обновить меню, иначе произойдет большая путаница.

```

    .else
        invoke DefMDIChildProc,hChild,uMsg,wParam,lParam
        ret
    .endif

```

Внутри оконной процедуры дочернего MDI-окна вы должны передать все необработанные сообщения функции DefMDIChildProc.

```

    .elseif ax==IDM_TILEHORZ
        invoke SendMessage,hwndClient,WM_MDITILE,MDITILE_HORIZONTAL,0
    .elseif ax==IDM_TILEVERT
        invoke SendMessage,hwndClient,WM_MDITILE,MDITILE_VERTICAL,0
    .elseif ax==IDM_CASCADE
        invoke SendMessage,hwndClient,WM_MDICASCADE,MDITILE_SKIPDISAB

```

Когда пользователь выбирает один из пунктов меню в подменю окон, мы посылаем соответствующее сообщение клиентскому окну. Если пользователь выбирает один из методов расположения окон, мы посылаем WM_MDITILE или WM_CASCADE.

```

    .elseif ax==IDM_CLOSE
        invoke SendMessage,hwndClient,WM_MDIGETACTIVE,0,0
        invoke SendMessage,eax,WM_CLOSE,0,0

```

Если пользователь выбирает пункт меню "Close", мы должны получить хэндл текущего активного MDI-окна.

```

    .elseif uMsg==WM_CLOSE
        invoke MessageBox,hChild,addr ClosePromptMessage,addr AppName
        .if eax==IDYES
            invoke SendMessage,hwndClient,WM_MDIDESTROY,hChild,0
        .endif

```

Когда процедура дочернего MDI-окна получает сообщение WM_CLOSE, его обработчик отображает окно, которое спрашивает пользователя, действительно ли он хочет закрыть окно. Если ответ - "Да", то мы посылаем клиентскому окну сообщение WM_MDIDESTROY, которое закрывает дочернее MDI-окно и восстанавливает заголовок фреймового окна.

Win32 API. Урок 33. RichEdit Control: основы

Автор: lczelion, пер. UniSoft

Дата: Не указана

Загрузите пример (находится в архиве wasm_files.zip: files/recovered/site-1001033/tut33.zip).

ТЕОРИЯ

О richedit контроле можно думать, как о функционально-расширенном средстве редактирования. Он обеспечивает множество полезных особенностей, которых нет в простых средствах редактирования, например, возможность использовать множество видов и размеров шрифта, глубокий уровень отмены/восстановления, операцией поиска по тексту, встроенные OLE-объекты, поддержка редактирования методом перетаскивания (drag-and-drop), и т.д. Так как richedit контрол имеет так много особенностей, он сохранен в отдельной DLL-библиотеке. Это также означает что, чтобы его использовать, вам недостаточно просто вызывать **InitCommonControls**, как в других Common-контролах. Вы должны вызвать **LoadLibrary**, чтобы загрузить richedit DLL.

Проблема состоит в том, что в настоящее время есть уже три версии richedit контрола. Версия 1,2, и 3. В таблице ниже показаны имена DLL для каждой версии.

имя DLL	версия RichEdit'a	Имя класса Richedit'a
Riched32.dll	1.0	RICHEDIT
RichEd20.dll	2.0	RICHEDIT20A
RichEd20.dll	3.0	RICHEDIT20A

Обратите внимание, что richedit версия 2 и 3 использует то же самое имя DLL. Они также используют одно и то же имя класса! Это может вызвать проблему, если Вы захотите использовать определенные особенности richedit'a версии 3.0. До сих пор, я не нашел официального метода различия между версиями 2.0 и 3.0. Однако, есть рабочий пример, который хорошо работает, я покажу Вам позже.

```
.data
    RichEditDLL db "RichEd20.dll", 0
    ....
.data?
    hRichEditDLL dd?
.code
    invoke LoadLibrary, addr RichEditDLL
    mov hRichEditDLL, eax
    .....
    invoke FreeLibrary, hRichEditDLL
```

Когда richedit dll загружена, она регистрирует класс окна RichEdit. Следовательно вам необходимо загрузить DLL прежде, чем Вы создадите контрол. Имена классов richedit контрола также различны. Теперь Вы можете задать вопрос: а какую версию richedit контрола мне следует использовать? Использование самой последней версии не всегда подходит, если Вы не требуете дополнительных особенностей. В таблице ниже, показаны особенности каждой версии richedit контрола.

Особенность	Версия 1.0	Версия 2.0	Версия 3.0
Выделение области	x	x	x
Редактирование уникада		x	x
Форматирование символа/абзаца	x	x	x

Особенность	Версия 1.0	Версия 2.0	Версия 3.0
Поиск по тексту	Вперед	Вперед/назад	Вперед/назад
Внедрение OLE	x	x	x
Редактирование перетаскать и отпустить(drag-and-drop)	x	x	x
Отменить/Повторить	Одноуровневый	Многоуровневый	Многоуровневый
Автоматическое распознавание URL		x	x
Поддержка клавиш быстрого доступа(hot key)		x	x
операция уменьшения окна		x	x
Конец строки	CRLF	только CR	только CR(может подражать версии 1.0)
Увеличение			x
Нумерация абзацев			x
Простая таблица			x
Нормальный и заголовочный стили			x
Цветное подчеркивание			x
Скрытый текст			x
связывание шрифта			x

Вышеуказанная таблица ни в коем случае не полная: я только перечислил важные особенности.

Создание richedit контрола

После загрузки richedit dll, Вы можете вызывать **CreateWindowEx** , для создания контрола. Вы можете использовать стили средств редактирования и common windows стили в **CreateWindowEx** кроме **ES_LOWERCASE**, **ES_UPPERCASE** и **ES_OEMCONVERT**.

```
.const
    RichEditID equ 300
.data
    RichEditDLL db "RichEd20.dll", 0
    RichEditClass db "RichEdit20A", 0
    ....
.data?
    HRichEditDLL dd ?
    HwndRichEdit dd ?
.code
    ....
    invoke LoadLibrary, addr RichEditDLL
    mov hRichEditDLL, eax
    invoke CreateWindowEx, 0, addr RichEditClass, \
        WS_VISIBLE or ES_MULTILINE or WS_CHILD or WS_VSCROLL or WS_HSCROLL, \
        CW_USEDEFAULT, CW_USEDEFAULT, CW_USEDEFAULT, CW_USEDEFAULT, \
        hwnd, RichEditID, hInstance, 0
    mov hwndRichEdit, eax
```

Установка по умолчанию цвета текста и фона

У вас может возникнуть проблема с установкой цвета текста и фона в средствах редактирования. Но эта проблема была исправлена в richedit контроле. Чтобы установить цвет фона richedit контрола, вам нужно послать ему **EM_SETBKGNDCOLOR**. Это сообщение имеет следующий синтаксис:

WParam == цвет. Значение 0 в этом параметре означает, что Windows использует значение цвета из **IParam** как цвет фона. Если это значение отличное от нуля, Windows использует цвет фона системы Windows. Так как мы посылаем это сообщение, чтобы изменить цвет фона, мы должны поместить 0 в wParam.

LParam == определяет структуру COLORREF цвета, который Вы хотите установить, если wParam - 0.

Например, если бы я захотел установить цвет фона в синий, я бы поместил этот код:

```
invoke SendMessage, hwndRichEdit, EM_SETBKGDNDCOLOR, 0,0FF0000h
```

Устанавливая цвет текста, richedit контрол создает новое сообщение, **EM_SETCHARFORMAT**. Это сообщение управляет форматированием текста в диапазоне от символа в выделении до всего текста. Это сообщение имеет следующий синтаксис:

WParam == опции форматирования:

SCF_ALL	Операция затрагивает весь текст в контроле.
SCF_SELECTION	Операция затрагивает только выделенный текст
SCF_WORD или SCF_SELECTION	Затрагивает слово в выделении. Если ничего не выделенно, то операция затронет то слово на котором находится курсор. Флаг SCF_WORD должен использоваться с SCF_SELECTION .

LParam == указатель на структуру **CHARFORMAT** ИЛИ **CHARFORMAT2**, которая определяет форматирование текста, которое нужно применить. **CHARFORMAT2** доступен только для richedit 2.0 и выше. Это не подразумевает, что Вы должны использовать **CHARFORMAT2** с RichEdit 2.0 и выше. Вы все еще можете использовать **CHARFORMAT**, если добавленные в **CHARFORMAT2** особенности вам не нужны.

```
CHARFORMAT2A STRUCT
    CbSize DWORD ?
    DwMask DWORD ?
    DweEffects DWORD ?
    YHeight DWORD ?
    YOffset DWORD ?
    CrTextColor COLORREF ?
    BCharSet BYTE ?
    BPitchAndFamily BYTE ?
    SzFaceName BYTE LF_FACESIZE dup(?)
    _wPad2 WORD ?
CHARFORMAT2A ENDS
```

Имя поля	Описание
CbSize	Размер структуры. RichEdit контрол использует это поле, чтобы определить версию структуры, является ли это CHARFORMAT или CHARFORMAT2

Имя поля	Описание
DwMask	Разряды флагов, которые определяют, какие из следующих членов являются правильными. CFM_BOLD — CFE_BOLD член dwEffects правильный CFM_CHARSET — член BCharSet- правильный. CFM_COLOR — член CrTextColor и значение CFE_AUTOCOLOR члена dwEffects - правильные CFM_FACE — член SzFaceName правильный. CFM_ITALIC — Значение CFE_ITALIC члена dwEffects правильное CFM_OFFSET — член YOffset правильный CFM_PROTECTED — Значение CFE_PROTECTED члена dwEffects правильное CFM_SIZE — член YHeight правильный CFM_STRIKEOUT — Значение CFE_STRIKEOUT члена dwEffects правильное. CFM_UNDERLINE — Значение CFE_UNDERLINE члена dwEffects правильное
DwEffects	Символьные эффекты. Может быть комбинация следующих значений CFE_AUTOCOLOR — Использует системный цвет текста CFE_BOLD — Полужирный CFE_ITALIC — Курсивный CFE_STRIKEOUT — Перечеркнутый. CFE_UNDERLINE — Подчеркнутый. CFE_PROTECTED — Символы защищены; попытка изменять их вызовет уведомительное сообщение EN_PROTECTED.
YHeight	Высота символов, в twips (1/1440 дюйма или 1/20 точки принтера).
YOffset	Смещение Символа, в twips (1/1440 дюйма или 1/20 точки принтера), от основной линии. Если значение этого члена положительное, символ - верхний индекс; если - отрицательное - нижний индекс.
CrTextColor	Цвет текста. Этот член игнорируется, если эффект символа CFE_AUTOCOLOR определен.
BCharSet	Набор символов
BPitchAndFamily	Семейство Шрифта и шаг.
SzFaceName	Символьный массив с нулевым символом в конце, определяющий имя шрифта
_wPad2	Дополнение

Из исследования структуры, Вы увидите, что мы можем изменять эффекты текста (полужирный, курсивный, зачеркнутый, подчеркнутый), цвет текста (**crTextColor**) и шрифт (вид/размер/набор символов). Текст с флагом **CFE_PROTECTED**, отмечен как защищенный, это означает, что, когда пользователь попытается изменить его, то родительскому окну будет послано уведомительное сообщение **EN_PROTECTED**. И Вы можете либо разрешить, либо запретить изменения.

CHARFORMAT2 добавляет большее количество текстовых стилей, таких как weight шрифта, интервала, цвета фона текста, кернинга, и т.д. Если Вы не нуждаетесь в этих дополнительных особенностях, просто используйте **CHARFORMAT**.

Чтобы установить формат текста, Вы должны указать диапазон текста, к которому хотите применить формат. Richedit контрол присваивает каждому символу свой номер (идентификатор), начинающийся с 0: первый символ имеет идентификатор 0, второй - 1 и так далее. Чтобы указать диапазон текста, вы должны дать richedit контролу, два числа: ИДЕНТИФИКАТОРЫ первого и последнего символа диапазона. Чтобы применить форматирование к тексту с **EM_SETCHARFORMAT**, у вас есть 3 выбора:

1. Применить ко всему тексту (**SCF_ALL**)
2. Применить к выделенному тексту (**SCF_SELECTION**)
3. Применить к целому слову в выделенной области (**SCF_WORD** or **SCF_SELECTION**)

Первый и второй выборы прямые, последний требует некоторых объяснений. Если текущее выделение охватывает только один или большее количество символов в слове, но не, целое слово, определяя флаг **SCF_WORD + SCF_SELECTION**, применяется форматирование текста к целому слову. Даже если ничего не выделено, форматирование применяется к целому слову, над которым находится курсор вставки.

Чтобы использовать **EM_SETCHARFORMAT**, Вы должны заполнить некоторые члены структуры **CHARFORMAT** (или **CHARFORMAT2**). Например, если мы хотим установить цвет текста, мы заполним структуру **CHARFORMAT** следующим образом:

```
.data?
    Cf CHARFORMAT < >
..
.code
    Mov cf.cbSize, sizeof cf
    Mov cf.dwMask, CFM_COLOR
    Mov cf.crTextColor, 0xFF0000h
    invoke SendMessage, hwndRichEdit, EM_SETCHARFORMAT, \
        SCF_ALL, addr cf
```

Этот фрагмент кода устанавливает цвет текста richedit контрола в синий. Обратите внимание, что, если в richedit контроле нет никакого текста, когда мы посылаем сообщение **EM_SETCHARFORMAT**, текст, введенный в richedit контроле после сообщения будет использовать формат текста, указанный с сообщением **EM_SETCHARFORMAT**.

Установка текста/сохранение текста

Те, кто уже использовали Edit контролы, наверняка знакомы с **WM_GETTEXT / WM_SETTEXT**, для установки/получения текста в/из контрола. Этот метод работает и с richedit контролом, но не может работать с большими файлами. Edit контрол ограничивает текст, который может быть введен в него 64КБ, но richedit контрол может принимать текст намного больше. Это было бы не правильное решение, выделить очень большой блок памяти (типа 10 мб) чтобы получить текст с помощью **WM_GETTEXT**. Richedit контрол предлагает новый подход к этому методу, такой как текстовый поток.

Зделать это просто, вы передаете адрес функции richedit контролу. И richedit контрол вызовет эту функцию, передавая ей адрес буфера, когда он готов. Функция заполнит буфер данными, которые требуется послать контролу или считает данные из буфера и будет ждать следующего запроса, пока операция не закончена. Эта парадигма используется для обоих операций: потоковый ввод (установка текста) и потоковый вывод (получение текста из контрола). Вы увидите, что этот метод более эффективен: буфер обеспечивается richedit контролом непосредственно, так что данные разделены на куски. Операции включают два сообщения: **EM_STREAMIN** и **EM_STREAMOUT**

Оба сообщения **EM_STREAMIN** и **EM_STREAMOUT** используют одинаковый синтаксис:

WParam == опции.

SF_RTF	Данные в формате RTF (rich-text format)
SF_TEXT	Данные в формате открытого текста
SFF_PLAINRTF	Только ключевые слова, общие ко всем языкам в потоке.
SFF_SELECTION	Если цель операции - выделенный текст. Если вводите текст, он заменяет текущее выделение. Если вы выводите текст, то будет выведен только выделенный в настоящее время текст. Если этот флаг не определен, операция работает со всем текстом в контроле.

SF_RTF	Данные в формате RTF (rich-text format)
SF_UNICODE	(Доступна для RichEdit 2.0 и выше) Определяют текст уникада.

LParam == указывают на структуру EDITSTREAM, которая имеет следующее определение:

```

EDITSTREAM STRUCT
    DwCookie DWORD ?
    DwError  DWORD ?
    PfnCallback DWORD ?
EDITSTREAM ENDS

```

DwCookie	Определенное приложением значение, которое будет передаваться к функции определенной в члене pfnCallback ниже. Мы обычно передаем функции некоторые важные значения, такие как хэндл файла, для использования в stream-in/out процедуре.
DwError	Отображает результат операции stream-in (чтения) или stream-out (записи). Значение 0 - нет ошибок. Значение отличное от нуля может быть результатом функции EditStreamCallback или кода, указывающего, что произошла ошибка.
PfnCallback	Указатель на функцию EditStreamCallback, которая является определенной приложением функцией, которая управляет передачей данных. Управление вызывает функцию неоднократно, передавая часть данных с каждым запросом

Editstream функция имеет следующее определение:

```

EditStreamCallback proto dwCookie:DWORD,
    PBuffer:DWORD,
    NumBytes:DWORD,
    PBytesTransferred:DWORD

```

Вы должны создать функцию с вышеупомянутым прототипом в вашей программе. И затем передайте ее адрес с помощью **EM_STREAMIN** или **EM_STREAMOUT** через структуру EDITSTREAM.

Для операции stream-in (загрузка текста в richedit контрол):

DwCookie : определенное приложением значение вы передаете с **EM_STREAMIN** через структуру EDITSTREAM. Мы почти всегда передаем хэндл файла, содержание которого мы хотим установить в контрол.

PBuffer : указывает на буфер, переданный richedit контролом, который получит текст от вашей функции.

NumBytes : максимальное количество байт, которые вы может записать в буфер (pBuffer) в этом запросе. Вы всегда **ДОЛЖНЫ** придерживаться этого предела, т.е., вы можете посылать меньшее количество данных чем значение в NumBytes, и не должны послать большее количество данных чем это значение. Вы можете считать это значение, как размер буфера pBuffer.

pBytesTransferred : указывает на переменную (dword), указывающую число байтов, которые вы фактически передали в буфер. Это значение обычно идентично значению в **NumBytes**. Исключение когда данных послано меньше, чем размер буфера, обычно когда достигнут конец файла.

Для операции stream-out (получение текста из richedit контрола):

DwCookie : Такой же, как в операции stream-in. Мы обычно передаем хэндл файла, в который мы хотим записать данные.

PBuffer : указывает на буфер, обеспеченный richedit контролом, который заполнен данными из richedit контрола. Чтобы получить его размер, Вы должны посмотреть значение **NumBytes**.

NumBytes : размер данных в буфере, указанном в pBuffer.

PBytesTransferred : указывает на переменную (dword), в которую вы должны установить значение, индицирующее число байт, которые Вы фактически считали из буфера.

Функция возвращает 0, если не было ошибок, и richedit контрол продолжит вызывать функцию, если данные все еще есть. Если произошла ошибка в течение процесса, и вы хотите остановить операцию, верните ненулевое значение, и richedit контрол откажется от данных, указанных в pBuffer. Значение ошибки/успеха будет заполнено в поле **dwError** структуры **EDITSTREAM**, так что вы можете проверить результат операции после возврата из **SendMessage**.

ПРИМЕР

Пример ниже это - простой редактор, которым Вы можете открывать файлы исходного текста ассемблера, редактировать, и сохранять их. Он использует RichEdit версии 2.0 или выше.

```
.386.model flat,stdcall
option casemap:none
include \masm32\include\windows.inc
include \masm32\include\user32.inc
include \masm32\include\comdlg32.inc
include \masm32\include\gdi32.inc
include \masm32\include\kernel32.inc
includelib \masm32\lib\gdi32.lib
includelib \masm32\lib\comdlg32.lib
includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib

WinMain proto :DWORD,:DWORD,:DWORD,:DWORD

.const
IDR_MAINMENU      equ 101
IDM_OPEN          equ 40001
IDM_SAVE          equ 40002
IDM_CLOSE         equ 40003
IDM_SAV           equ 40004
IDM_EXIT          equ 40005
IDM_COPY          equ 40006
IDM_CUT           equ 40007
IDM_PASTE         equ 40008
IDM_DELETE        equ 40009
IDM_SELECTALL     equ 40010
IDM_OPTION        equ 40011
IDM_UNDO          equ 40012
IDM_REDO          equ 40013
IDD_OPTIONDLG     equ 101
IDC_BACKCOLORBOX equ 1000
IDC_TEXTCOLORBOX equ 1001

RichEditID        equ 300

.data
ClassName db "IczEditClass",0
AppName db "IczEdit версия 1.0",0
RichEditDLL db "riched20.dll",0
RichEditClass db "RichEdit20A",0
NoRichEdit db "
ASMFilterString db "ASM Исходники (*.asm)",0,"*.asm",0
               db "Все файлы (*.*)",0,"*.*",0,0
OpenFileFail db "Не могу открыть файл",0
WannaSave db "Данные были изменены. Вы хотите сохранить изменения?",0
FileOpened dd FALSE
BackgroundColor dd 0FFFFFFh ; по умолчанию - белый
TextColor dd 0               ; по умолчанию - черный

.data?
hInstance dd ?
hRichEdit dd ?
hwndRichEdit dd ?
FileName db 256 dup(?)
AlternateFileName db 256 dup(?)
```

```

CustomColors dd 16 dup(?)

.code
start:
    invoke GetModuleHandle, NULL
    mov     hInstance,eax
    invoke LoadLibrary,addr RichEditDLL
    .if eax!=0
        mov hRichEdit,eax
        invoke WinMain, hInstance,0,0, SW_SHOWDEFAULT
        invoke FreeLibrary,hRichEdit
    .else
        invoke MessageBox,0,addr NoRichEdit,addr AppName, \
            MB_OK or MB_ICONERROR
    .endif
    invoke ExitProcess,eax

WinMain proc hInst:DWORD,hPrevInst:DWORD,CmdLine:DWORD,CmdShow:DWORD
    LOCAL wc:WNDCLASSEX
    LOCAL msg:MSG
    LOCAL hwnd:DWORD
    mov     wc.cbSize,SIZEOF WNDCLASSEX
    mov     wc.style, CS_HREDRAW or CS_VREDRAW
    mov     wc.lpfnWndProc, OFFSET WndProc
    mov     wc.cbClsExtra,NULL
    mov     wc.cbWndExtra,NULL
    push    hInst
    pop     wc.hInstance
    mov     wc.hbrBackground,COLOR_WINDOW+1
    mov     wc.lpszMenuName,IDR_MAINMENU
    mov     wc.lpszClassName,OFFSET ClassName
    invoke LoadIcon,NULL,IDI_APPLICATION
    mov     wc.hIcon,eax
    mov     wc.hIconSm,eax
    invoke LoadCursor,NULL,IDC_ARROW
    mov     wc.hCursor,eax
    invoke RegisterClassEx, addr wc
    INVOKE CreateWindowEx,NULL,ADDR ClassName,ADDR AppName,\
        WS_OVERLAPPEDWINDOW,CW_USEDEFAULT,\
        CW_USEDEFAULT,CW_USEDEFAULT,CW_USEDEFAULT,NULL,NULL,\
        hInst,NULL
    mov     hwnd,eax
    invoke ShowWindow, hwnd,SW_SHOWNORMAL
    invoke UpdateWindow, hwnd
    .while TRUE
        invoke GetMessage, ADDR msg,0,0,0
        .break .if (!eax)
        invoke TranslateMessage, ADDR msg
        invoke DispatchMessage, ADDR msg
    .endw
    mov     eax,msg.wParam
    ret
WinMain endp

StreamInProc proc hFile:DWORD,pBuffer:DWORD, \
    NumBytes:DWORD, pBytesRead:DWORD
    invoke ReadFile,hFile,pBuffer,NumBytes,pBytesRead,0
    xor     eax,1
    ret
StreamInProc endp

StreamOutProc proc hFile:DWORD,pBuffer:DWORD, \
    NumBytes:DWORD, pBytesWritten:DWORD
    invoke WriteFile,hFile,pBuffer,NumBytes,pBytesWritten,0
    xor     eax,1
    ret
StreamOutProc endp

CheckModifyState proc hwnd:DWORD
    invoke SendMessage,hwndRichEdit,EM_GETMODIFY,0,0
    .if eax!=0

```

```

        invoke MessageBox,hWnd,addr WannaSave, \
            addr AppName,MB_YESNOCANCEL
        .if eax==IDYES
            invoke SendMessage,hWnd,WM_COMMAND,IDM_SAVE,0
        .elseif eax==IDCANCEL
            mov eax,FALSE
            ret
        .endif
    .endif
    mov eax,TRUE
    ret
CheckModifyState endp

SetColor proc
    LOCAL cfm:CHARFORMAT
    invoke SendMessage,hwndRichEdit,EM_SETBKGNDCOLOR, \
        0,BackgroundColor
    invoke RtlZeroMemory,addr cfm,sizeof cfm
    mov cfm.cbSize,sizeof cfm
    mov cfm.dwMask,CFM_COLOR
    push TextColor
    pop cfm.crTextColor
    invoke SendMessage,hwndRichEdit,EM_SETCHARFORMAT, \
        SCF_ALL,addr cfm
    ret
SetColor endp

OptionProc proc hWnd:DWORD, uMsg:DWORD, \
    wParam:DWORD, lParam:DWORD
    LOCAL clr:CHOOSECOLOR
    .if uMsg==WM_INITDIALOG
    .elseif uMsg==WM_COMMAND
        mov eax,wParam
        shr eax,16
        .if ax==BN_CLICKED
            mov eax,wParam
            .if ax==IDCANCEL
                invoke SendMessage,hWnd,WM_CLOSE,0,0
            .elseif ax==IDC_BACKCOLORBOX
                invoke RtlZeroMemory,addr clr,sizeof clr
                mov clr.lStructSize,sizeof clr
                push hWnd
                pop clr.hwndOwner
                push hInstance
                pop clr.hInstance
                push BackgroundColor
                pop clr.rgbResult
                mov clr.lpCustColors,offset CustomColors
                mov clr.Flags,CC_ANYCOLOR or CC_RGBINIT
                invoke ChooseColor,addr clr
                .if eax!=0
                    push clr.rgbResult
                    pop BackgroundColor
                    invoke GetDlgItem,hWnd,IDC_BACKCOLORBOX
                    invoke InvalidateRect,eax,0,TRUE
                .endif
            .elseif ax==IDC_TEXTCOLORBOX
                invoke RtlZeroMemory,addr clr,sizeof clr
                mov clr.lStructSize,sizeof clr
                push hWnd
                pop clr.hwndOwner
                push hInstance
                pop clr.hInstance
                push TextColor
                pop clr.rgbResult
                mov clr.lpCustColors,offset CustomColors
                mov clr.Flags,CC_ANYCOLOR or CC_RGBINIT
                invoke ChooseColor,addr clr
                .if eax!=0
                    push clr.rgbResult
                    pop TextColor
                .endif
            .endif
        .endif
    .endif

```

```

        invoke GetDlgItem,hWnd,IDC_TEXTCOLORBOX
        invoke InvalidateRect,eax,0,TRUE
    .endif
    .elseif ax==IDOK
        ;=====
        ; Сохраните состояние richedit контроля потому,
        ; что изменение цвета текста изменяет это состояние.
        ;=====
        invoke SendMessage,hwndRichEdit,EM_GETMODIFY,0,0
        push eax
        invoke SetColor
        pop eax
        invoke SendMessage,hwndRichEdit,EM_SETMODIFY,eax,0
        invoke EndDialog,hWnd,0
    .endif
    .endif
    .elseif uMsg==WM_CTLCOLORSTATIC
        invoke GetDlgItem,hWnd,IDC_BACKCOLORBOX
        .if eax==lParam
            invoke CreateSolidBrush,BackgroundColor
            ret
        .else
            invoke GetDlgItem,hWnd,IDC_TEXTCOLORBOX
            .if eax==lParam
                invoke CreateSolidBrush,TextColor
                ret
            .endif
        .endif
    .endif
    mov eax,FALSE
    ret
    .elseif uMsg==WM_CLOSE
        invoke EndDialog,hWnd,0
    .else
        mov eax,FALSE
        ret
    .endif
    mov eax,TRUE
    ret
OptionProc endp

WndProc proc hWnd:DWORD, uMsg:DWORD, \
        wParam:DWORD, lParam:DWORD
    LOCAL chrg:CHARRANGE
    LOCAL ofn:OPENFILENAME
    LOCAL buffer[256]:BYTE
    LOCAL editstream:EDITSTREAM
    LOCAL hFile:DWORD
    .if uMsg==WM_CREATE
        invoke CreateWindowEx,WS_EX_CLIENTEDGE,addr RichEditClass,0,\
            WS_CHILD or WS_VISIBLE or ES_MULTILINE or\
            WS_VSCROLL or WS_HSCROLL or ES_NOHIDESEL,\
            CW_USEDEFAULT,CW_USEDEFAULT, CW_USEDEFAULT,\
            CW_USEDEFAULT,hWnd,RichEditID,hInstance,0
        mov hwndRichEdit,eax
        ;=====
        ; Установка предела текста. По умолчанию - 64K
        ;=====
        invoke SendMessage,hwndRichEdit,EM_LIMITTEXT,-1,0
        ;=====
        ; установка цвета текста/фона
        ;=====
        invoke SetColor
        invoke SendMessage,hwndRichEdit,EM_SETMODIFY,FALSE,0
        invoke SendMessage,hwndRichEdit,EM_EMPTYUNDOBUFFER,0,0
    .elseif uMsg==WM_INITMENUPOPUP
        mov eax,lParam
        .if ax==0 ; меню Файл
            .if FileOpened==TRUE ; a file is already opened
                invoke EnableMenuItem,wParam,IDM_OPEN,MF_GRAYED
                invoke EnableMenuItem,wParam,IDM_CLOSE,MF_ENABLED
                invoke EnableMenuItem,wParam,IDM_SAVE,MF_ENABLED
            .endif
        .endif
    .endif

```

```

        invoke EnableMenuItem,wParam,IDM_SAVEAS,MF_ENABLED
    .else
        invoke EnableMenuItem,wParam,IDM_OPEN,MF_ENABLED
        invoke EnableMenuItem,wParam,IDM_CLOSE,MF_GRAYED
        invoke EnableMenuItem,wParam,IDM_SAVE,MF_GRAYED
        invoke EnableMenuItem,wParam,IDM_SAVEAS,MF_GRAYED
    .endif
.elseif ax==1 ; edit menu
;=====
; проверьте есть ли текст в буфере обмена, если есть
; мы активизируем в меню пункт "Вставить".
;=====
invoke SendMessage,hwndRichEdit,EM_CANPASTE,CF_TEXT,0
.if eax==0 ; no text in the clipboard
    invoke EnableMenuItem,wParam,IDM_PASTE,MF_GRAYED
.else
    invoke EnableMenuItem,wParam,IDM_PASTE,MF_ENABLED
.endif
;=====
; Проверьте, является ли очередь отмены пустой
;=====
invoke SendMessage,hwndRichEdit,EM_CANUNDO,0,0
.if eax==0
    invoke EnableMenuItem,wParam,IDM_UNDO,MF_GRAYED
.else
    invoke EnableMenuItem,wParam,IDM_UNDO,MF_ENABLED
.endif
;=====
; Проверьте, является ли очередь повтора пустой
;=====
invoke SendMessage,hwndRichEdit,EM_CANREDO,0,0
.if eax==0
    invoke EnableMenuItem,wParam,IDM_REDO,MF_GRAYED
.else
    invoke EnableMenuItem,wParam,IDM_REDO,MF_ENABLED
.endif
;=====
; Проверьте, выделено ли что-нибудь в richedit контроле.
; Если да, мы активизируем пункты меню вырезать/
; копировать/удалить
;=====
invoke SendMessage,hwndRichEdit,EM_EXGETSEL,0,addr chrg
mov eax,chrg.cpMin
.if eax==chrg.cpMax ; no current selection
    invoke EnableMenuItem,wParam,IDM_COPY,MF_GRAYED
    invoke EnableMenuItem,wParam,IDM_CUT,MF_GRAYED
    invoke EnableMenuItem,wParam,IDM_DELETE,MF_GRAYED
.else
    invoke EnableMenuItem,wParam,IDM_COPY,MF_ENABLED
    invoke EnableMenuItem,wParam,IDM_CUT,MF_ENABLED
    invoke EnableMenuItem,wParam,IDM_DELETE,MF_ENABLED
.endif
.endif
.elseif wParam==WM_COMMAND
.if lParam==0 ; menu commands
    mov eax,wParam
    .if ax==IDM_OPEN
        invoke RtlZeroMemory,addr ofn,sizeof ofn
        mov ofn.lStructSize,sizeof ofn
        push hWnd
        pop ofn.hwndOwner
        push hInstance
        pop ofn.hInstance
        mov ofn.lpstrFilter,offset ASMFilterString
        mov ofn.lpstrFile,offset FileName
        mov byte ptr [FileName],0
        mov ofn.nMaxFile,sizeof FileName
        mov ofn.Flags,OFN_FILEMUSTEXIST or OFN_HIDEREADONLY or OFN_PATHMUSTEXIST
        invoke GetOpenFileName,addr ofn
        .if eax!=0
            invoke CreateFile,addr FileName,GENERIC_READ,\

```

```

        FILE_SHARE_READ,NULL,OPEN_EXISTING,\
        FILE_ATTRIBUTE_NORMAL,0
.if eax!=INVALID_HANDLE_VALUE
    mov hFile,eax
    ;=====
    ; stream the text into the richedit control
    ;=====
    mov editstream.dwCookie,eax
    mov editstream.pfnCallback,offset StreamInProc
    invoke SendMessage,hwndRichEdit,EM_STREAMIN,\
        SF_TEXT,addr editstream
    ;=====
    ; Initialize the modify state to false
    ;=====
    invoke SendMessage,hwndRichEdit,EM_SETMODIFY,FALSE,0
    invoke CloseHandle,hFile
    mov FileOpened,TRUE
.else
    invoke MessageBox,hWnd,addr OpenFileFail,\
        addr AppName,MB_OK or MB_ICONERROR
.endif
.endif
.elseif ax==IDM_CLOSE
    invoke CheckModifyState,hWnd
    .if eax==TRUE
        invoke SetWindowText,hwndRichEdit,0
        mov FileOpened,FALSE
    .endif
.elseif ax==IDM_SAVE
    invoke CreateFile,addr FileName,GENERIC_WRITE, \
        FILE_SHARE_READ,NULL,\
        CREATE_ALWAYS,FILE_ATTRIBUTE_NORMAL,0
    .if eax!=INVALID_HANDLE_VALUE
@@:
        mov hFile,eax
        ;=====
        ; stream the text to the file
        ;=====
        mov editstream.dwCookie,eax
        mov editstream.pfnCallback,offset StreamOutProc
        invoke SendMessage,hwndRichEdit,EM_STREAMOUT, \
            SF_TEXT, addr editstream
        ;=====
        ; Initialize the modify state to false
        ;=====
        invoke SendMessage,hwndRichEdit,EM_SETMODIFY,FALSE,0
        invoke CloseHandle,hFile
    .else
        invoke MessageBox,hWnd,addr OpenFileFail,addr AppName,\
            MB_OK or MB_ICONERROR
    .endif
.elseif ax==IDM_COPY
    invoke SendMessage,hwndRichEdit,WM_COPY,0,0
.elseif ax==IDM_CUT
    invoke SendMessage,hwndRichEdit,WM_CUT,0,0
.elseif ax==IDM_PASTE
    invoke SendMessage,hwndRichEdit,WM_PASTE,0,0
.elseif ax==IDM_DELETE
    invoke SendMessage,hwndRichEdit,EM_REPLACESEL,TRUE,0
.elseif ax==IDM_SELECTALL
    mov chrg.cpMin,0
    mov chrg.cpMax,-1
    invoke SendMessage,hwndRichEdit,EM_EXSETSEL,0,addr chrg
.elseif ax==IDM_UNDO
    invoke SendMessage,hwndRichEdit,EM_UNDO,0,0
.elseif ax==IDM_REDO
    invoke SendMessage,hwndRichEdit,EM_REDO,0,0
.elseif ax==IDM_OPTION
    invoke DialogBoxParam,hInstance,IDD_OPTIONDLG,hWnd,addr OptionProc,0
.elseif ax==IDM_SAVEAS
    invoke RtlZeroMemory,addr ofn,sizeof ofn

```



```

mov ofn.lStructSize, sizeof ofn
push hWnd
pop ofn.hwndOwner
push hInstance
pop ofn.hInstance
mov ofn.lpstrFilter, offset ASMFilterString
mov ofn.lpstrFile, offset AlternateFileName
mov byte ptr [AlternateFileName], 0
mov ofn.nMaxFile, sizeof AlternateFileName
mov ofn.Flags, OFN_FILEMUSTEXIST or OFN_HIDEREADONLY
or OFN_PATHMUSTEXIST
invoke GetSaveFileName, addr ofn
.if eax!=0
    invoke CreateFile, addr AlternateFileName, GENERIC_WRITE, \
        FILE_SHARE_READ, NULL, CREATE_ALWAYS, \
        FILE_ATTRIBUTE_NORMAL, 0
    .if eax!=INVALID_HANDLE_VALUE
        jmp @B
    .endif
.endif
.elseif ax==IDM_EXIT
    invoke SendMessage, hWnd, WM_CLOSE, 0, 0
.endif
.endif
.elseif uMsg==WM_CLOSE
    invoke CheckModifyState, hWnd
    .if eax==TRUE
        invoke DestroyWindow, hWnd
    .endif
.elseif uMsg==WM_SIZE
    mov eax, lParam
    mov edx, eax
    and eax, 0FFFFh
    shr edx, 16
    invoke MoveWindow, hWndRichEdit, 0, 0, eax, edx, TRUE
.elseif uMsg==WM_DESTROY
    invoke PostQuitMessage, NULL
.else
    invoke DefWindowProc, hWnd, uMsg, wParam, lParam
    ret
.endif
xor eax, eax
ret
WndProc endp
end start

```

```

;=====
; Файл ресурсов
;=====
#include "resource.h"
#define IDR_MAINMENU          101
#define IDD_OPTIONDLG         101
#define IDC_BACKCOLORBOX     1000
#define IDC_TEXTCOLORBOX     1001
#define IDM_OPEN              40001
#define IDM_SAVE              40002
#define IDM_CLOSE            40003
#define IDM_SAVEAS           40004
#define IDM_EXIT             40005
#define IDM_COPY             40006
#define IDM_CUT              40007
#define IDM_PASTE            40008
#define IDM_DELETE           40009
#define IDM_SELECTALL        40010
#define IDM_OPTION           40011
#define IDM_UNDO             40012
#define IDM_REDO             40013

```

```

IDR_MAINMENU MENU DISCARDABLE
BEGIN
    POPUP "&Файл"

```

```

BEGIN
    MENUITEM "&Открыть...",      IDM_OPEN
    MENUITEM "&Закрыть",         IDM_CLOSE
    MENUITEM "&Сохранить",      IDM_SAVE
    MENUITEM "Сохранить &как...", IDM_SAVEAS
    MENUITEM SEPARATOR
    MENUITEM "В&ыход",           IDM_EXIT
END
POPUP "&Правка"
BEGIN
    MENUITEM "&Отменить",        IDM_UNDO
    MENUITEM "&Повторить",       IDM_REDO
    MENUITEM "&Копировать",      IDM_COPY
    MENUITEM "&Вырезать",        IDM_CUT
    MENUITEM "Вст&авить",        IDM_PASTE
    MENUITEM SEPARATOR
    MENUITEM "&Удалить",          IDM_DELETE
    MENUITEM SEPARATOR
    MENUITEM "В&ыделить все",    IDM_SELECTALL
END
MENUITEM "П&араметры",          IDM_OPTION
END

IDD_OPTIONDLG DIALOG DISCARDABLE 0, 0, 183, 54
STYLE DS_MODALFRAME | WS_POPUP | WS_VISIBLE | WS_CAPTION
    | WS_SYSMENU | DS_CENTER
CAPTION "Options"
FONT 8, "MS Sans Serif"
BEGIN
    DEFPUSHBUTTON "OK",IDOK,137,7,39,14
    PUSHBUTTON "Отмена",IDCANCEL,137,25,39,14
    GROUPBOX "", IDC_STATIC,5,0,124,49
    LTEXT "Цвет фона:",IDC_STATIC,20,14,60,8
    LTEXT "",IDC_BACKCOLORBOX,85,11,28,14,SS_NOTIFY | WS_BORDER
    LTEXT "Цвет текста:",IDC_STATIC,20,33,35,8
    LTEXT "",IDC_TEXTCOLORBOX,85,29,28,14,SS_NOTIFY | WS_BORDER
END

```

АНАЛИЗ

Программа сначала загружает richedit dll, в нашем случае riched20.dll. Если dll не может быть загружена, то выходим из программы.

```

invoke LoadLibrary, addr RichEditDLL
.if eax!=0
    mov hRichEdit, eax
    invoke WinMain, hInstance, 0,0, SW_SHOWDEFAULT
    invoke FreeLibrary, hRichEdit
.else
    invoke MessageBox, 0, addr NoRichEdit, addr AppName, \
        MB_OK or MB_ICONERROR
.endif
invoke ExitProcess, eax

```

После того, как dll успешно загружена, мы переходим к созданию нормального окна, которое будет родительским richedit контрола. Внутри обработчика **WM_CREATE**, мы создаем richedit контрол:

```

invoke CreateWindowEx, WS_EX_CLIENTEDGE, addr RichEditClass, 0,\
    WS_CHILD or WS_VISIBLE or ES_MULTILINE or WS_VSCROLL or
    WS_HSCROLL or ES_NOHIDESEL,\
    CW_USEDEFAULT, CW_USEDEFAULT, CW_USEDEFAULT,
    CW_USEDEFAULT, hWnd, RichEditID,\
    hInstance, 0
mov hwndRichEdit, eax

```

Обратите внимание, что мы определяем стиль **ES_MULTILINE**, иначе контрол будет одиночно-выровненный.

```

invoke SendMessage, hwndRichEdit, EM_LIMITTEXT,-1,0

```

После того, как richedit контрол создан, мы должны установить в нем новый текстовый предел. По умолчанию, richedit контрол имеет предел текста 64КБ, такой же как в простых многострочных Edit контролах. Мы должны расширить этот предел, чтобы оперировать с большими файлами. В вышеупомянутой строке, я определяю -1, которая составляет 0FFFFFFFFh, очень большое значение.

```
invoke SetColor
```

Затем, мы устанавливаем цвет текста и фона. Так как эта операция может быть вызвана и из другой части программы, я поместил код в функцию SetColor.

```
SetColor proc
LOCAL cfm:CHARFORMAT
invoke SendMessage, hwndRichEdit, EM_SETBKGDNCOLOR, \
0, BackgroundColor
```

Установка цвета фона richedit контрола это прямая операция: просто пошлите сообщение **EM_SETBKGDNCOLOR** richedit контролу. (Если Вы используете многострочные Edit контролы, Вы должны обрабатывать **WM_CTLCOLOREDIT**). Заданный по умолчанию цвет фона белый.

```
invoke RtlZeroMemory, addr cfm, sizeof cfm
Mov cfm.cbSize, sizeof cfm
Mov cfm.dwMask, CFM_COLOR
push TextColor
pop cfm.crTextColor
```

После того, как цвет фона установлен, мы заполняем члены структуры **CHARFORMAT**, чтобы установить цвет текста. Обратите внимание, что мы заполняем **cbSize** размером структуры, так что richedit контрол знает, что мы посылаем ему **CHARFORMAT**, а не **CHARFORMAT2**. **DwMask** имеет только один флаг, **CFM_COLOR**, потому что мы только хотим установить цвет текста, и **crTextColor** заполнен значением желаемого цвета текста.

```
invoke SendMessage, hwndRichEdit, EM_SETCHARFORMAT, \
SCF_ALL, addr cfm
Ret
SetColor endp
```

После установки цвета, Вы должны освободить буфер отмены, потому что действие изменения текста/цвета фона возможно отменить. Мы посылаем сообщение **EM_EMPTYUNDOBUFFER**, чтобы сделать этого.

```
invoke SendMessage, hwndRichEdit, EM_EMPTYUNDOBUFFER, 0,0
```

После заполнения структуры CHARFORMAT, мы посылаем **EM_SETCHARFORMAT** richedit контролу, определяя **SCF_ALL** флаг в **wParam**, чтобы указать, что мы хотим, чтобы форматирование текста применилось ко всему тексту.

Обратите внимание, что, когда мы создавали richedit контрол, мы не определили его размер и позицию. Дело в том, что мы хотим, чтобы он закрыл всю клиентскую область родительского окна. Мы изменяем его размеры всякий раз, когда изменяется размер родительского окна.

```
.elseif uMsg== WM_SIZE
Mov eax, lParam
Mov edx, eax
and eax, 0FFFFh
Shr edx, 16
invoke MoveWindow, hwndRichEdit, 0,0, eax, edx, TRUE
```

В вышеупомянутом фрагменте кода, мы используем новые размеры клиентской области в **IParam**, чтобы изменить размеры richedit контрола с помощью **MoveWindow**.

Когда пользователь кликает на строке меню Файл/Правка, мы обрабатываем сообщение **WM_INITPOPUPMENU** так, чтобы мы могли установить состояние некоторых пунктов в подменю перед отображением эго пользователю. Например, если файл уже открыт в richedit контроле, мы

хотим отключить пункт открыть в подменю и включить пункт сохранить, сохранить как... и т.д.

В случае сострокой меню Файл, мы используем переменную **FileOpened** как флаг, чтобы определить, открыт ли уже файл. Если значение в этой переменной TRUE, то мы знаем, что файл уже открыт.

```
.elseif uMsg==WM_INITMENUPOPUP
    mov eax, lParam
    .if ax==0 ; меню Файл
        .if FileOpened==TRUE ; файл уже открыт
            invoke EnableMenuItem, wParam, IDM_OPEN, MF_GRAYED
            invoke EnableMenuItem, wParam, IDM_CLOSE, MF_ENABLED
            invoke EnableMenuItem, wParam, IDM_SAVE, MF_ENABLED
            invoke EnableMenuItem, wParam, IDM_SAVEAS, MF_ENABLED
        .else
            invoke EnableMenuItem, wParam, IDM_OPEN, MF_ENABLED
            invoke EnableMenuItem, wParam, IDM_CLOSE, MF_GRAYED
            invoke EnableMenuItem, wParam, IDM_SAVE, MF_GRAYED
            invoke EnableMenuItem, wParam, IDM_SAVEAS, MF_GRAYED
        .endif
    .endif
```

Как вы можете заметить, если файл уже открыт, мы запрещаем пункты меню открытие и разрешаем пункты меню сохранение, и наоборот если **FileOpened** - FALSE.

В случае со строкой меню правка, мы сначала должны проверить состояние richedit контрола и буфера обмена.

```
invoke SendMessage, hwndRichEdit, EM_CANPASTE, CF_TEXT, 0
.if eax==0 ; нет текста в буфере обмена
    invoke EnableMenuItem, wParam, IDM_PASTE, MF_GRAYED
.else
    invoke EnableMenuItem, wParam, IDM_PASTE, MF_ENABLED
.endif
```

Мы сначала проверяем, является ли текст в буфере обмена доступным, посылая сообщение **EM_CANPASTE**. Если текст доступен, **SendMessage** возвращает TRUE, и мы разрешаем пункт меню вставка, а если FALSE, то запрещаем.

```
invoke SendMessage, hwndRichEdit, EM_CANUNDO, 0, 0
.if eax==0
    invoke EnableMenuItem, wParam, IDM_UNDO, MF_GRAYED
.else
    invoke EnableMenuItem, wParam, IDM_UNDO, MF_ENABLED
.endif
```

Затем, мы проверяем, является ли буфер отмены пустым, посылая сообщение **EM_CANUNDO**. Если - не пустой, **SendMessage** возвращает TRUE, и мы разрешаем пункт меню отмена.

```
invoke SendMessage, hwndRichEdit, EM_CANREDO, 0, 0
.if eax==0
    invoke EnableMenuItem, wParam, IDM_REDO, MF_GRAYED
.else
    invoke EnableMenuItem, wParam, IDM_REDO, MF_ENABLED
.endif
```

Мы проверяем буфер Повтора, посылая richedit контролу сообщение **EM_CANREDO**. Если - не пустой, **SendMessage** возвращает TRUE, и мы разрешаем пункт меню повторить.

```
invoke SendMessage, hwndRichEdit, EM_EXGETSEL, 0, addr chrg
mov eax, chrg.cpmIn
.if eax==chrg.cpmMax ; ничего не выделенно
    invoke EnableMenuItem, wParam, IDM_COPY, MF_GRAYED
    invoke EnableMenuItem, wParam, IDM_CUT, MF_GRAYED
    invoke EnableMenuItem, wParam, IDM_DELETE, MF_GRAYED
.else
    invoke EnableMenuItem, wParam, IDM_COPY, MF_ENABLED
    invoke EnableMenuItem, wParam, IDM_CUT, MF_ENABLED
    invoke EnableMenuItem, wParam, IDM_DELETE, MF_ENABLED
.endif
```

Наконец, мы проверяем, выделенно ли что-нибудь в richedit контроле, посылая сообщение **EM_EXGETSEL**. Это сообщение использует структуру CHARRANGE, которая определена следующим образом:

```
CHARRANGE STRUCT
    cpMin DWORD ?
    CpMax DWORD ?
CHARRANGE ENDS
```

CpMin содержит индекс позиции символа предшествующего первому символу в диапазоне.

CpMax содержит индекс позиции символа идущего после последнего символа в диапазоне.

После возвращения **EM_EXGETSEL**, структура CHARRANGE заполнена индексами позиции стартовой и конечной метками выделенного диапазона. Если ничего не выделенно, **cpMin** и **cpMax** идентичны и мы, запрещаем пункты меню вырезать/копировать/удалить.

Когда пользователь нажимает пункт Открыть, мы отображаем диалоговое окно открытия файла и если пользователь выбирает файл, мы открываем файл и потоком загружаем его содержание richedit контрол.

```
invoke CreateFile,addr FileName,GENERIC_READ,
    FILE_SHARE_READ,NULL,OPEN_EXISTING,
    FILE_ATTRIBUTE_NORMAL,0
.if eax!=INVALID_HANDLE_VALUE
    mov hFile,eax
    mov editstream.dwCookie,eax
    mov editstream.pfnCallback,offset StreamInProc
    invoke SendMessage,hwndRichEdit,EM_STREAMIN,
        SF_TEXT,addr editstream
```

После того, как файл успешно открыт с **CreateFile**, мы заполняем структуру EDITSTREAM, подготавливаем к сообщению **EM_STREAMIN**. Мы выбираем послать хэндл открытого файла через член **dwCookie** и передать адрес потоковой функции в **pfnCallback**.

Потоковая процедура сама по себе простая.

```
StreamInProc proc hFile:DWORD, pBuffer:DWORD,
    NumBytes:DWORD, pBytesRead:DWORD
    invoke ReadFile, hFile, pBuffer, NumBytes, pBytesRead, 0
    Xor eax, 1
    Ret
StreamInProc endp
```

Вы можете заметить, что все параметры потоковой процедуры совершенно соответствуют **ReadFile**. И возвращаемое значение **ReadFile** - хог с 1 для, того чтобы, если она возвратит 1 (в случае успеха), фактическое значение, возвращенное в **eax** - 0 и наоборот.

```
invoke SendMessage,hwndRichEdit,EM_SETMODIFY,FALSE,0
invoke CloseHandle,hFile
mov FileOpened,TRUE
```

После возврата **EM_STREAMIN**, это означает, что потоковая операция завершена. В действительности, мы должны проверить значение **dwError** члена структуры EDITSTREAM.

Richedit (и Edit) контролы поддерживают флаг, чтобы указать, изменялось ли их содержание. Мы можем получить значение этого флага, посылая им сообщение **EM_GETMODIFY**. **SendMessage** возвращает TRUE, если содержание изменялось. Так как загрузка текста в контрол, это - своего рода модификация. Мы должны установить флаг модификации в FALSE, посылая контролу сообщение **EM_SETMODIFY** с **wParam == FALSE** после того, как операция stream-in завершится. Мы немедленно закрываем файл и устанавливаем **FileOpened** в TRUE чтобы указывать, что файл был открыт.

Когда пользователь кликнет на пункт меню сохранить/сохранить как..., мы используем сообщение **EM_STREAMOUT**, чтобы вывести содержимое richedit контрола в файл. Как и функция stream-in, функции stream-out - простота сама по себе. Это совершенно соответствует **WriteFile**.

Текстовые операции такие как вырезать/копировать/вставить/восстановить/отменить, легко осуществимы, посылая richedit контролу сообщение **WM_CUT /WM_COPY /WM_PASTE /WM_REDO /WM_UNDO** соответственно.

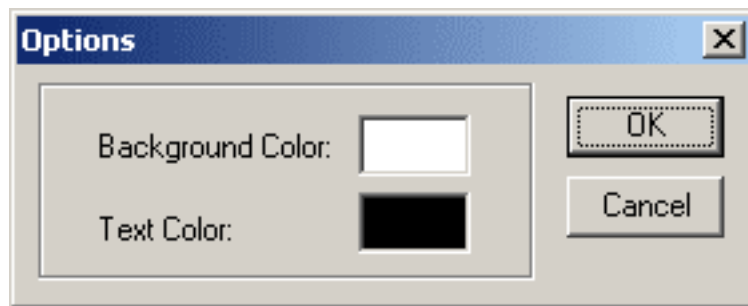
Операции удаление/выделить все сделаны следующим образом:

```
.elseif ax==IDM_DELETE
    invoke SendMessage,hwndRichEdit,EM_REPLACESEL,TRUE,0
.elseif ax==IDM_SELECTALL
    mov chrg.cpMin,0
    mov chrg.cpMax,-1
    invoke SendMessage,hwndRichEdit,EM_EXSETSEL,0,addr chrg
```

Операция удаление работает с выделением. Я посылаю сообщение **EM_REPLACESEL** с NULL строкой, чтобы richedit контрол заменил выделенный текст пустой строкой.

Операция выделить всё сделана, посылая сообщение **EM_EXSETSEL**, установив **cpMin == 0** и **cpMax == -1**, что равносильно выделению всего текста.

Когда пользователь выбирает строку меню Параметры, мы отображаем диалоговое окно, представляющее текущие цвета фона/текста.



Когда пользователь кликает на одной из палитры цветов, вызывается диалоговое окно выбора цвета. "Палитра цветов" - фактически статический элемент управления с флагом **WS_BORDER** и **SS_NOTIFY**. Статический элемент управления с флагом **SS_NOTIFY** уведомит его родительское окно с действиями мыши на нем, типа **BN_CLICKED (STN_CLICKED)**. Это - уловка.

```
.elseif ax==IDC_BACKCOLORBOX
    invoke RtlZeroMemory,addr clr,sizeof clr
    mov clr.lStructSize,sizeof clr
    push hWnd
    pop clr.hwndOwner
    push hInstance
    pop clr.hInstance
    push BackgroundColor
    pop clr.rgbResult
    mov clr.lpCustColors,offset CustomColors
    mov clr.Flags,CC_ANYCOLOR or CC_RGBINIT
    invoke ChooseColor,addr clr
.iff eax!=0
    push clr.rgbResult
    pop BackgroundColor
    invoke GetDlgItem,hWnd,IDC_BACKCOLORBOX
    invoke InvalidateRect,eax,0,TRUE
.endif
```

Когда пользователь кликает на одной из палитры цветов, мы заполняем члены структуры **CHOOSECOLOR** и вызываем диалоговое окно выбора цвета **ChooseColor**. Если пользователь выбирает цвет, то это значение colorref возвращается в члене **rgbResult**, и мы сохраняем это значение в переменной **BackgroundColor**. После этого, мы вынуждаем перекрашивание на

палитре цветов, вызывая **InvalidateRect** на хэндл палитры цветов. Палитра цветов посылает **WM_CTLCOLORSTATIC** сообщение своему родительскому окну.

```
invoke GetDlgItem,hWnd,IDC_BACKCOLORBOX  
.if eax==1Param  
    invoke CreateSolidBrush,BackgroundColor  
ret
```

Внутри обработчика **WM_CTLCOLORSTATIC** , мы сравниваем, хэндл статического элемента управления переданного в **1Param** > с обоими палитрами цветов. Если значение соответствует, мы создаем новую кисть, используя цвет из переменной и немедленно возвращаемся. Статический элемент управления будет использовать недавно созданную кисть, чтобы окрасить ее фон.

Win32 API. Урок 34. RichEdit Control: больше об операциях над текстом

Автор: Iczelion, пер. Aquila

Дата: 2002-06-04

В этом tutorialе вы узнаете больше о операциях над текстом, доступных в RichEdit, например о том, как искать/заменять текст и переходить к определенной строке.

Скачайте пример (находится в архиве wasm_files.zip: files/recovered/1001034/tut34.zip).

ТЕОРИЯ

Поиск текста

В RichEdit есть несколько текстовых операций. Поиск определенного текста - одна из них. Поиск текста осуществляется с помощью сообщений EM_FINDTEXT или EM_FINDTEXTX. Эти сообщения мало отличаются друг от друга.

```
EM_FINDTEXT
```

wParam == опции поиска.

Может быть комбинацией значений, приведенных ниже. Эти опции идентичны как для EM_FINDTEXT, так и для EM_FINDTEXTX.

- FR_DOWN - если этот флаг указан, поиск начинается с конца текущего выделенного текста и до самого конца (вперед). Этот флаг действует только в RichEdit 2.0 или выше: этот способ применяется по умолчанию в RichEdit 1.0. В RichEdit 2.0 по умолчанию поиск осуществляется от конца выделенного текста до начала (назад). Кратко говоря, если вы используете RichEdit 1.0, вы ничего не можете сделать, чтобы изменить направление поиска: он всегда ищет вперед. Если вы используете RichEdit 2.0 и хотите искать вперед, вам нужно указать этот флаг, иначе поиск будет производиться назад.
- FR_MATCHCASE - если этот флаг указан, будет учитываться регистр.
- FR_WHOLEWORD - если этот флаг указан, поиск будет искать место в тексте, которое будет удовлетворять указанной поисковой строке.

Вообще-то, есть еще несколько флагов, но они относятся к языкам, отличным от английского.

lParam == указатель на структуру FINDTEXT.

```
FINDTEXT STRUCT
    chrg      CHARRANGE  <>
    lpstrText DWORD      ?
FINDTEXT ENDS
```

chrg - это структура CHARRANGE, которая определена следующим образом:

```
CHARRANGE STRUCT
    cpMin  DWORD  ?
    cpMax  DWORD  ?
CHARRANGE ENDS
```

cpMin содержит индекс первого символа в массиве символов (диапазон).

cpMax содержит индекс символа, который следует непосредственно за последним символом в массиве символов.

Фактически, чтобы найти строку текста, вам нужно указать диапазон символом, в котором нужно искать. Значение cpMin и cpMax будут зависеть от того, проводится ли поиск назад или вперед. Если поиск идет вперед, cpMin задает начальный индекс, а cpMax - конечный. Если поиск идет

назад, тогда `srMin` содержит конечное значение индекса, в то время как `srMax` задает начальный индекс.

`lpstrText` - это указатель на текстовую строку, которую нужно искать.

`EM_FINDTEXT` возвращает индекс первого символа в заданной текстовой строке в `RichEdit`. Оно возвращает -1, если указанный текст не был найден.

`EM_FINDTEXTEX`

`wParam` == опции поиска. То же самое, что и `EM_FINDTEXT`. `lParam` == указатель на структуру `FINDTEXTEX`.

```
FINDTEXTEX STRUCT
    chrg          CHARRANGE <>
    lpstrText     DWORD      ?
    chrgText      CHARRANGE <>
FINDTEXTEX ENDS
```

Первые два члена `FINDTEXTEX` идентичны соответствующим полям в структуре `FINDTEXT`. `chrgText` - это структура `CHARRANGE`, которая будет заполнена начальным и конечным индексами, если будут найдены какие-либо совпадения.

Возвращаемое значение `EM_FINDTEXTEX` - то же самое, что и у `EM_FINDTEXT`.

Разница между `EM_FINDTEXT` и `EM_FINDTEXTEX` - это то, что у структуры `FINDTEXTEX` есть дополнительное поле, `chrgText`, которое будет заполнено начальным и конечным индексами, если будут найдены совпадения. Это удобно, если мы хотим иметь возможность осуществлять больше текстовых операций над строкой.

Замещение/вставка текста

Контроль `RichEdit` предоставляет `EM_SETTEXTEX` для замещения/вставки текста. Это сообщение комбинирует функциональность `WM_SETTEXT` и `WM_REPLACESEL`. У него следующий синтаксис:

`EM_SETTEXTEX` `wParam` == указатель на структуру `SETTEXTEX`.

```
SETTEXTEX STRUCT
    flags         DWORD      ?
    codepage      DWORD      ?
SETTEXTEX ENDS
```

Поле 'flags' может быть комбинацией следующих значений:

- `ST_DEFAULT` - удаляет стек совершенных операций, сбрасывает форматирование `RichEdit`.
- `ST_KEEPUndo` - сохраняет стек совершенных операций.
- `ST_SELECTION` - замещает выделенный текст и сохраняет форматирование.

Поле 'codepage' - это константа, которая указывает кодировку. Обычно мы указываем `CP_ACP`.

Выделение текста

Мы можем выделить текст программно с помощью сообщений `EM_SETSEL` или `EM_EXSETSEL`. Обе прекрасно работают. Выбирать, какое из сообщений необходимо использовать, зависит от доступного формата индексов символов. Если они уже сохранены в структуры `CHARRANGE`, проще использовать `EM_EXSETSEL`.

`EM_EXSETSEL`

`wParam` == не используется. Должен быть равен 0. `lParam` == указатель на структуру `CHARRANGE`, который содержит диапазон символов, который необходимо выделить.

Уведомительные события

В случае с многолинейным edit control'ом вам необходимо сабклассировать его, чтобы получить входные сообщения, такие как события мыши/клавиатуры. RichEdit предоставляет лучший способ: он будет уведомлять родительское окно о таких событиях. Чтобы указать RichEdit, какие события нужно посылать, родительское окно посылает сообщение EM_SETEVENTMASK контролю RichEdit, указывая, в каких событиях он заинтересован. У EM_SETEVENTMASK следующий синтаксис:

EM_SETEVENTMASK

wParam == не используется. Должен быть равен нулю. lParam == маска событий. Это должна быть комбинация флагов, указанных ниже.

- ENM_CHANGE - прием уведомлений EN_CHANGE.
- ENM_CORRECTTEXT - прием уведомлений EN_CORRECTTEXT.
- ENM_DRAGDROPDONE - прием уведомлений EN_DRAGDROPDONE.
- ENM_DROPFILES - прием уведомлений EN_DROPFILES.
- ENM_KEYEVENTS - прием уведомлений EN_MSGFILTER, относящихся к событиям от клавиатуры.
- ENM_LINK - RichEdit 2.0 и выше: прием уведомлений EN_LINK, когда курсор мыши находится над текстом, который принимает CFE_LINK и/или другие события от мыши.
- ENM_MOUSEEVENTS - прием уведомлений EN_MSGFILTER, относящихся к событиям от мыши.
- ENM_OBJECTPOSITIONS - прием уведомлений EN_OBJECTPOSITIONS.
- ENM_PROTECTED - прием уведомлений EN_PROTECTED.
- ENM_REQUESTRESIZE - прием уведомлений EN_REQUESTRESIZE.
- ENM_SCROLL - прием уведомлений EN_HSCROLL и EN_VSCROLL.
- ENM_SCROLLEVENTS - прием уведомлений EN_MSGFILTER от колесика мыши.
- ENM_SELCHANGE - прием уведомлений EN_SELCHANGE.
- ENM_UPDATE - прием уведомлений EN_UPDATE. RichEdit 2.0 и выше: этот флаг игнорирует, а сообщения EN_UPDATE отсылаются всегда. Тем не менее, если Rich Edit 3.0 эмулирует Rich Edit 1.0, вы должны указать этот флаг для того, чтобы родительское окно принимало уведомления EN_UPDATE.

Все вышеуказанные уведомления будут отсылаться через сообщение WM_NOTIFY: вы должны проверить поле 'code' структуры NMHDR, чтобы узнать, какое уведомление вы получили. Например, если вы хотите зарегистрировать сообщения от мыши (скажем, чтобы отображать контекстное меню по нажатию на правую кнопку мыши), вы должны сделать что-то вроде следующего:

```
invoke SendMessage, hwndRichEdit, EM_SETEVENTMASK,
                                0, ENM_MOUSEEVENTS

.....
.....
WndProc proc hwnd:DWORD, uMsg:DWORD,
                                wParam:DWORD, lParam:DWORD
.....
.....
    .elseif uMsg==WM_NOTIFY
        push esi
        mov esi, lParam
        assume esi:ptr NMHDR
    .if [esi].code==EN_MSGFILTER
        ....
        [ do something here]
        ....
    .endif
    pop esi
```

ПРИМЕР

Следующий пример является обновлением IczEdit, шедшего вместе с tutorialом 33. Были добавлены возможности поиска/замещения и акселераторы. Также обрабатываются события от

мышь и отображается контекстное меню.

```
.386
.model flat,stdcall
option casemap:none
include \masm32\include\windows.inc
include \masm32\include\user32.inc
include \masm32\include\comdlg32.inc
include \masm32\include\gdi32.inc
include \masm32\include\kernel32.inc
includelib \masm32\lib\gdi32.lib
includelib \masm32\lib\comdlg32.lib
includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib

WinMain proto :DWORD,:DWORD,:DWORD,:DWORD

.const
IDR_MAINMENU          equ 101
IDM_OPEN              equ 40001
IDM_SAVE              equ 40002
IDM_CLOSE             equ 40003
IDM_SAVEAS            equ 40004
IDM_EXIT              equ 40005
IDM_COPY              equ 40006
IDM_CUT               equ 40007
IDM_PASTE             equ 40008
IDM_DELETE            equ 40009
IDM_SELECTALL         equ 40010
IDM_OPTION            equ 40011
IDM_UNDO              equ 40012
IDM_REDO              equ 40013
IDD_OPTIONDLG         equ 101
IDC_BACKCOLORBOX      equ 1000
IDC_TEXTCOLORBOX      equ 1001
IDR_MAINACCEL         equ 105
IDD_FINDDLG           equ 102
IDD_GOTODLG           equ 103
IDD_REPLACEDLG        equ 104
IDC_FINDEEDIT         equ 1000
IDC_MATCHCASE         equ 1001
IDC_REPLACEEDIT       equ 1001
IDC_WHOLEWORD         equ 1002
IDC_DOWN              equ 1003
IDC_UP                equ 1004
IDC_LINENO            equ 1005
IDM_FIND              equ 40014
IDM_FINDNEXT          equ 40015
IDM_REPLACE           equ 40016
IDM_GOTOLINE          equ 40017
IDM_FINDPREV          equ 40018
RichEditID            equ 300

.data
ClassName db "IczEditClass",0
AppName   db "IczEdit version 2.0",0
RichEditDLL db "riched20.dll",0
RichEditClass db "RichEdit20A",0
NoRichEdit db "Cannot find riched20.dll",0
ASMFilterString db "ASM Source code (*.asm)",0,"*.asm",0
               db "All Files (*.*)",0,"*.*",0,0
OpenFileFail db "Cannot open the file",0
WannaSave db "The data in the control is modified. Want to save it?",0
FileOpened dd FALSE
BackgroundColor dd 0FFFFFFh ; default to white
TextColor dd 0 ; default to black
hSearch dd ? ; handle to the search/replace dialog box
hAccel dd ?

.data?
hInstance dd ?
```

```

hRichEdit dd ?
hwndRichEdit dd ?
FileName db 256 dup(?)
AlternateFileName db 256 dup(?)
CustomColors dd 16 dup(?)
FindBuffer db 256 dup(?)
ReplaceBuffer db 256 dup(?)
uFlags dd ?
findtext FINDTEXT <>

.code
start:
    mov byte ptr [FindBuffer],0
    mov byte ptr [ReplaceBuffer],0
    invoke GetModuleHandle, NULL
    mov     hInstance,eax
    invoke LoadLibrary,addr RichEditDLL
    .if eax!=0
        mov hRichEdit,eax
        invoke WinMain, hInstance,0,0, SW_SHOWDEFAULT
        invoke FreeLibrary,hRichEdit
    .else
        invoke MessageBox,0,addr NoRichEdit,addr AppName,MB_OK or MB_ICONERROR
    .endif
    invoke ExitProcess,eax

WinMain proc hInst:DWORD,hPrevInst:DWORD,CmdLine:DWORD,CmdShow:DWORD
    LOCAL wc:WNDCLASSEX
    LOCAL msg:MSG
    LOCAL hwnd:DWORD
    mov     wc.cbSize,SIZEOF WNDCLASSEX
    mov     wc.style, CS_HREDRAW or CS_VREDRAW
    mov     wc.lpfnWndProc, OFFSET WndProc
    mov     wc.cbClsExtra,NULL
    mov     wc.cbWndExtra,NULL
    push    hInst
    pop     wc.hInstance
    mov     wc.hbrBackground,COLOR_WINDOW+1
    mov     wc.lpszMenuName,IDR_MAINMENU
    mov     wc.lpszClassName,OFFSET ClassName
    invoke LoadIcon,NULL,IDI_APPLICATION
    mov     wc.hIcon,eax
    mov     wc.hIconSm,eax
    invoke LoadCursor,NULL,IDC_ARROW
    mov     wc.hCursor,eax
    invoke RegisterClassEx, addr wc
    INVOKE CreateWindowEx,NULL,ADDR ClassName,ADDR AppName,\
        WS_OVERLAPPEDWINDOW,CW_USEDEFAULT,\
        CW_USEDEFAULT,CW_USEDEFAULT,CW_USEDEFAULT,NULL,NULL,\
        hInst,NULL
    mov     hwnd,eax
    invoke ShowWindow, hwnd,SW_SHOWNORMAL
    invoke UpdateWindow, hwnd
    invoke LoadAccelerators,hInstance,IDR_MAINACCEL
    mov     hAccel,eax
    .while TRUE
        invoke GetMessage, ADDR msg,0,0,0
        .break .if (!eax)
        invoke IsDialogMessage,hSearch,addr msg
        .if eax==FALSE
            invoke TranslateAccelerator,hwnd,hAccel,addr msg
            .if eax==0
                invoke TranslateMessage, ADDR msg
                invoke DispatchMessage, ADDR msg
            .endif
        .endif
    .endw
    mov     eax,msg.wParam
    ret
WinMain endp
StreamInProc proc hFile:DWORD,pBuffer:DWORD,

```

```

        NumBytes:DWORD, pBytesRead:DWORD
    invoke ReadFile,hFile,pBuffer,NumBytes,pBytesRead,0
    xor eax,1
    ret
StreamInProc endp

StreamOutProc proc hFile:DWORD,pBuffer:DWORD, NumBytes:DWORD,
    pBytesWritten:DWORD
    invoke WriteFile,hFile,pBuffer,NumBytes,pBytesWritten,0
    xor eax,1
    ret
StreamOutProc endp

CheckModifyState proc hWnd:DWORD
    invoke SendMessage,hwndRichEdit,EM_GETMODIFY,0,0
    .if eax!=0
        invoke MessageBox,hWnd,addr WannaSave,addr AppName,MB_YESNOCANCEL
        .if eax==IDYES
            invoke SendMessage,hWnd,WM_COMMAND,IDM_SAVE,0
        .elseif eax==IDCANCEL
            mov eax,FALSE
            ret
        .endif
    .endif
    mov eax,TRUE
    ret
CheckModifyState endp

SetColor proc
    LOCAL cfm:CHARFORMAT
    invoke SendMessage,hwndRichEdit,EM_SETBKGNDCOLOR,0,BackgroundColor
    invoke RtlZeroMemory,addr cfm,sizeof cfm
    mov cfm.cbSize,sizeof cfm
    mov cfm.dwMask,CFM_COLOR
    push TextColor
    pop cfm.crTextColor
    invoke SendMessage,hwndRichEdit,EM_SETCHARFORMAT,SCF_ALL,addr cfm
    ret
SetColor endp

OptionProc proc hWnd:DWORD, uMsg:DWORD, wParam:DWORD, lParam:DWORD
    LOCAL clr:CHOOSECOLOR
    .if uMsg==WM_INITDIALOG
    .elseif uMsg==WM_COMMAND
        mov eax,wParam
        shr eax,16
        .if ax==BN_CLICKED
            mov eax,wParam
            .if ax==IDCANCEL
                invoke SendMessage,hWnd,WM_CLOSE,0,0
            .elseif ax==IDC_BACKCOLORBOX
                invoke RtlZeroMemory,addr clr,sizeof clr
                mov clr.lStructSize,sizeof clr
                push hWnd
                pop clr.hwndOwner
                push hInstance
                pop clr.hInstance
                push BackgroundColor
                pop clr.rgbResult
                mov clr.lpCustColors,offset CustomColors
                mov clr.Flags,CC_ANYCOLOR or CC_RGBINIT
                invoke ChooseColor,addr clr
                .if eax!=0
                    push clr.rgbResult
                    pop BackgroundColor
                    invoke GetDlgItem,hWnd,IDC_BACKCOLORBOX
                    invoke InvalidateRect,eax,0,TRUE
                .endif
            .elseif ax==IDC_TEXTCOLORBOX
                invoke RtlZeroMemory,addr clr,sizeof clr
                mov clr.lStructSize,sizeof clr

```

```

        push hWnd
        pop clr.hwndOwner
        push hInstance
        pop clr.hInstance
        push TextColor
        pop clr.rgbResult
        mov clr.lpCustColors,offset CustomColors
        mov clr.Flags,CC_ANYCOLOR or CC_RGBINIT
        invoke ChooseColor,addr clr
        .if eax!=0
            push clr.rgbResult
            pop TextColor
            invoke GetDlgItem,hWnd,IDC_TEXTCOLORBOX
            invoke InvalidateRect,eax,0,TRUE
        .endif
    .elseif ax==IDOK
        invoke SendMessage,hwndRichEdit,EM_GETMODIFY,0,0
        push eax
        invoke SetColor
        pop eax
        invoke SendMessage,hwndRichEdit,EM_SETMODIFY,eax,0
        invoke EndDialog,hWnd,0
    .endif
.endif
.elseif uMsg==WM_CTLCOLORSTATIC
    invoke GetDlgItem,hWnd,IDC_BACKCOLORBOX
    .if eax==1Param
        invoke CreateSolidBrush,BackgroundColor
        ret
    .else
        invoke GetDlgItem,hWnd,IDC_TEXTCOLORBOX
        .if eax==1Param
            invoke CreateSolidBrush,TextColor
            ret
        .endif
    .endif
    mov eax,FALSE
    ret
.elseif uMsg==WM_CLOSE
    invoke EndDialog,hWnd,0
.else
    mov eax,FALSE
    ret
.endif
mov eax,TRUE
ret
OptionProc endp

SearchProc proc hWnd:DWORD, uMsg:DWORD, wParam:DWORD, lParam:DWORD
    .if uMsg==WM_INITDIALOG
        push hWnd
        pop hSearch
        invoke CheckRadioButton,hWnd,IDC_DOWN,IDC_UP,IDC_DOWN
        invoke SendDlgItemMessage,hWnd,IDC_FINDEDIT,
            WM_SETTEXT,0,addr FindBuffer
    .elseif uMsg==WM_COMMAND
        mov eax,wParam
        shr eax,16
        .if ax==BN_CLICKED
            mov eax,wParam
            .if ax==IDOK
                mov uFlags,0
                invoke SendMessage,hwndRichEdit,
                    EM_EXGETSEL,0,addr findtext.chrg
                invoke GetDlgItemText,hWnd,IDC_FINDEDIT,
                    addr FindBuffer,sizeof FindBuffer
                .if eax!=0
                    invoke IsDlgButtonChecked,hWnd,IDC_DOWN
                    .if eax==BST_CHECKED
                        or uFlags,FR_DOWN
                    .endif
                    mov eax,findtext.chrg.cpMin
                .endif
            .endif
        .endif
    .endif

```

```

        .if eax!=findtext.chrg.cpMax
            push findtext.chrg.cpMax
            pop findtext.chrg.cpMin
        .endif
        mov findtext.chrg.cpMax,-1
    .else
        mov findtext.chrg.cpMax,0
    .endif
    invoke IsDlgButtonChecked,hWnd,IDC_MATCHCASE
    .if eax==BST_CHECKED
        or uFlags,FR_MATCHCASE
    .endif
    invoke IsDlgButtonChecked,hWnd,IDC_WHOLEWORD
    .if eax==BST_CHECKED
        or uFlags,FR_WHOLEWORD
    .endif
    mov findtext.lpstrText,offset FindBuffer
    invoke SendMessage,hwndRichEdit,EM_FINDTEXT,
        uFlags,addr findtext
    .if eax!=-1
        invoke SendMessage,hwndRichEdit,EM_EXSETSEL,0,
            addr findtext.chrgText
    .endif
    .endif
.elseif ax==IDCANCEL
    invoke SendMessage,hWnd,WM_CLOSE,0,0
.else
    mov eax,FALSE
    ret
.endif
.endif
.elseif uMsg==WM_CLOSE
    mov hSearch,0
    invoke EndDialog,hWnd,0
.else
    mov eax,FALSE
    ret
.endif
mov eax,TRUE
ret
SearchProc endp

ReplaceProc proc hWnd:DWORD, uMsg:DWORD, wParam:DWORD, lParam:DWORD
LOCAL settext:SETTEXT
.if uMsg==WM_INITDIALOG
    push hWnd
    pop hSearch
    invoke SetDlgItemText,hWnd,IDC_FINDEDIT,addr FindBuffer
    invoke SetDlgItemText,hWnd,IDC_REPLACEEDIT,addr ReplaceBuffer
.elseif uMsg==WM_COMMAND
    mov eax,wParam
    shr eax,16
    .if ax==BN_CLICKED
        mov eax,wParam
        .if ax==IDCANCEL
            invoke SendMessage,hWnd,WM_CLOSE,0,0
        .elseif ax==IDOK
            invoke GetDlgItemText,hWnd,IDC_FINDEDIT,
                addr FindBuffer,sizeof FindBuffer
            invoke GetDlgItemText,hWnd,IDC_REPLACEEDIT,
                addr ReplaceBuffer,sizeof ReplaceBuffer
            mov findtext.chrg.cpMin,0
            mov findtext.chrg.cpMax,-1
            mov findtext.lpstrText,offset FindBuffer
            mov settext.flags,ST_SELECTION
            mov settext.codepage,CP_ACP
            .while TRUE
                invoke SendMessage,hwndRichEdit,EM_FINDTEXT,
                    FR_DOWN,addr findtext
                .if eax==-1
                    .break
            .endwhile
        .endif
    .endif

```

```

        .else
            invoke SendMessage,hwndRichEdit,EM_EXSETSEL,0,
                addr findtext.chrgText
            invoke SendMessage,hwndRichEdit,EM_SETTEXTTEX,
                addr setttext,addr ReplaceBuffer
        .endif
    .endw
.endif
.endif
.elseif uMsg==WM_CLOSE
    mov hSearch,0
    invoke EndDialog,hwnd,0
.else
    mov eax,FALSE
    ret
.endif
mov eax,TRUE
ret
ReplaceProc endp

GoToProc proc hwnd:DWORD, uMsg:DWORD, wParam:DWORD, lParam:DWORD
LOCAL LineNo:DWORD
LOCAL chrg:CHARRANGE
.if uMsg==WM_INITDIALOG
    push hwnd
    pop hSearch
.elseif uMsg==WM_COMMAND
    mov eax,wParam
    shr eax,16
    .if ax==BN_CLICKED
        mov eax,wParam
        .if ax==IDCANCEL
            invoke SendMessage,hwnd,WM_CLOSE,0,0
        .elseif ax==IDOK
            invoke GetDlgItemInt,hwnd,IDC_LINENO,NULL,FALSE
            mov LineNo,eax
            invoke SendMessage,hwndRichEdit,EM_GETLINECOUNT,0,0
            .if eax>LineNo
                invoke SendMessage,hwndRichEdit,EM_LINEINDEX,LineNo,0
                mov chrg.cpMin,eax
                mov chrg.cpMax,eax
                invoke SendMessage,hwndRichEdit,EM_EXSETSEL,0,addr chrg
                invoke SetFocus,hwndRichEdit
            .endif
        .endif
    .endif
.elseif uMsg==WM_CLOSE
    mov hSearch,0
    invoke EndDialog,hwnd,0
.else
    mov eax,FALSE
    ret
.endif
mov eax,TRUE
ret
GoToProc endp

PrepareEditMenu proc hSubMenu:DWORD
LOCAL chrg:CHARRANGE
invoke SendMessage,hwndRichEdit,EM_CANPASTE,CF_TEXT,0
.if eax==0 ; no text in the clipboard
    invoke EnableMenuItem,hSubMenu,IDM_PASTE,MF_GRAYED
.else
    invoke EnableMenuItem,hSubMenu,IDM_PASTE,MF_ENABLED
.endif
invoke SendMessage,hwndRichEdit,EM_CANUNDO,0,0
.if eax==0
    invoke EnableMenuItem,hSubMenu,IDM_UNDO,MF_GRAYED
.else
    invoke EnableMenuItem,hSubMenu,IDM_UNDO,MF_ENABLED
.endif

```



```

invoke SendMessage,hwndRichEdit,EM_CANREDO,0,0
.if eax==0
    invoke EnableMenuItem,hSubMenu,IDM_REDO,MF_GRAYED
.else
    invoke EnableMenuItem,hSubMenu,IDM_REDO,MF_ENABLED
.endif
invoke SendMessage,hwndRichEdit,EM_EXGETSEL,0,addr chrg
mov eax,chrg.cpMin
.if eax==chrg.cpMax    ; no current selection
    invoke EnableMenuItem,hSubMenu,IDM_COPY,MF_GRAYED
    invoke EnableMenuItem,hSubMenu,IDM_CUT,MF_GRAYED
    invoke EnableMenuItem,hSubMenu,IDM_DELETE,MF_GRAYED
.else
    invoke EnableMenuItem,hSubMenu,IDM_COPY,MF_ENABLED
    invoke EnableMenuItem,hSubMenu,IDM_CUT,MF_ENABLED
    invoke EnableMenuItem,hSubMenu,IDM_DELETE,MF_ENABLED
.endif
ret
PrepareEditMenu endp

WndProc proc hWnd:DWORD, uMsg:DWORD, wParam:DWORD, lParam:DWORD
    LOCAL ofn:OPENFILENAME
    LOCAL buffer[256]:BYTE
    LOCAL editstream:EDITSTREAM
    LOCAL hFile:DWORD
    LOCAL hPopup:DWORD
    LOCAL pt:POINT
    LOCAL chrg:CHARRANGE
    .if uMsg==WM_CREATE
        invoke CreateWindowEx,WS_EX_CLIENTEDGE,addr RichEditClass,0, \
            WS_CHILD or WS_VISIBLE or ES_MULTILINE or
            WS_VSCROLL or WS_HSCROLL or ES_NOHIDESEL,\
            CW_USEDEFAULT,CW_USEDEFAULT, CW_USEDEFAULT,
            CW_USEDEFAULT,hWnd,RichEditID,hInstance,0
        mov hwndRichEdit,eax
        invoke SendMessage,hwndRichEdit,EM_LIMITTEXT,-1,0
        invoke SetColor
        invoke SendMessage,hwndRichEdit,EM_SETMODIFY,FALSE,0
        invoke SendMessage,hwndRichEdit,EM_SETEVENTMASK,0,ENM_MOUSEEVENTS
        invoke SendMessage,hwndRichEdit,EM_EMPTYUNDOBUFFER,0,0
    .elseif uMsg==WM_NOTIFY
        push esi
        mov esi,lParam
        assume esi:ptr NMHDR
        .if [esi].code==EN_MSGFILTER
            assume esi:ptr MSGFILTER
            .if [esi].msg==WM_RBUTTONDOWN
                invoke GetMenu,hWnd
                invoke GetSubMenu,eax,1
                mov hPopup,eax
                invoke PrepareEditMenu,hPopup
                mov edx,[esi].lParam
                mov ecx,edx
                and edx,0FFFFh
                shr ecx,16
                mov pt.x,edx
                mov pt.y,ecx
                invoke ClientToScreen,hWnd,addr pt
                invoke TrackPopupMenu,hPopup,TPM_LEFTALIGN or TPM_BOTTOMALIGN,
                    pt.x,pt.y,NULL,hWnd,NULL
            .endif
        .endif
        pop esi
    .elseif uMsg==WM_INITMENUPOPUP
        mov eax,lParam
        .if ax==0    ; file menu
            .if FileOpened==TRUE    ; a file is already opened
                invoke EnableMenuItem,wParam,IDM_OPEN,MF_GRAYED
                invoke EnableMenuItem,wParam,IDM_CLOSE,MF_ENABLED
                invoke EnableMenuItem,wParam,IDM_SAVE,MF_ENABLED
                invoke EnableMenuItem,wParam,IDM_SAVEAS,MF_ENABLED
            .endif
        .endif
    .endif
endp

```

```

        .else
            invoke EnableMenuItem,wParam,IDM_OPEN,MF_ENABLED
            invoke EnableMenuItem,wParam,IDM_CLOSE,MF_GRAYED
            invoke EnableMenuItem,wParam,IDM_SAVE,MF_GRAYED
            invoke EnableMenuItem,wParam,IDM_SAVEAS,MF_GRAYED
        .endif
    .elseif ax==1 ; edit menu
        invoke PrepareEditMenu,wParam
    .elseif ax==2 ; search menu bar
        .if FileOpened==TRUE
            invoke EnableMenuItem,wParam,IDM_FIND,MF_ENABLED
            invoke EnableMenuItem,wParam,IDM_FINDNEXT,MF_ENABLED
            invoke EnableMenuItem,wParam,IDM_FINDPREV,MF_ENABLED
            invoke EnableMenuItem,wParam,IDM_REPLACE,MF_ENABLED
            invoke EnableMenuItem,wParam,IDM_GOTOLINE,MF_ENABLED
        .else
            invoke EnableMenuItem,wParam,IDM_FIND,MF_GRAYED
            invoke EnableMenuItem,wParam,IDM_FINDNEXT,MF_GRAYED
            invoke EnableMenuItem,wParam,IDM_FINDPREV,MF_GRAYED
            invoke EnableMenuItem,wParam,IDM_REPLACE,MF_GRAYED
            invoke EnableMenuItem,wParam,IDM_GOTOLINE,MF_GRAYED
        .endif
    .endif
    .elseif uMsg==WM_COMMAND
        .if lParam==0 ; menu commands
            mov eax,wParam
            .if ax==IDM_OPEN
                invoke RtlZeroMemory,addr ofn,sizeof ofn
                mov ofn.lStructSize,sizeof ofn
                push hWnd
                pop ofn.hwndOwner
                push hInstance
                pop ofn.hInstance
                mov ofn.lpstrFilter,offset ASMFilterString
                mov ofn.lpstrFile,offset FileName
                mov byte ptr [FileName],0
                mov ofn.nMaxFile,sizeof FileName
                mov ofn.Flags,OFN_FILEMUSTEXIST or OFN_HIDEREADONLY
                    or OFN_PATHMUSTEXIST
                invoke GetOpenFileName,addr ofn
                .if eax!=0
                    invoke CreateFile,addr FileName,GENERIC_READ,
                        FILE_SHARE_READ,NULL,OPEN_EXISTING, \
                        FILE_ATTRIBUTE_NORMAL,0
                    .if eax!=INVALID_HANDLE_VALUE
                        mov hFile,eax
                        ;=====
                        ; записываем текст в контрол richedit
                        ;=====
                        mov editstream.dwCookie,eax
                        mov editstream.pfnCallback,offset StreamInProc
                        invoke SendMessage,hwndRichEdit,EM_STREAMIN,
                            SF_TEXT,addr editstream
                        ;=====
                        ; Устанавливаем флаг модификации в false
                        ;=====
                        invoke SendMessage,hwndRichEdit,EM_SETMODIFY,FALSE,0
                        invoke CloseHandle,hFile
                        mov FileOpened,TRUE
                    .else
                        invoke MessageBox,hWnd,addr OpenFileFail,
                            addr AppName,MB_OK or MB_ICONERROR
                    .endif
                .endif
            .elseif ax==IDM_CLOSE
                invoke CheckModifyState,hWnd
                .if eax==TRUE
                    invoke SetWindowText,hwndRichEdit,0
                    mov FileOpened,FALSE
                .endif
            .elseif ax==IDM_SAVE

```

```

        invoke CreateFile,addr FileName,GENERIC_WRITE,
            FILE_SHARE_READ,NULL,CREATE_ALWAYS, \
FILE_ATTRIBUTE_NORMAL,0
        .if eax!=INVALID_HANDLE_VALUE

@@:
        mov hFile,eax
        ;=====
        ; записываем текст в файл
        ;=====
        mov editstream.dwCookie,eax
        mov editstream.pfnCallback,offset StreamOutProc
        invoke SendMessage,hwndRichEdit,EM_STREAMOUT,SF_TEXT,
            addr editstream
        ;=====
        ; Initialize the modify state to false
        ; Устанавливаем флаг модификации в false
        ;=====
        invoke SendMessage,hwndRichEdit,EM_SETMODIFY,FALSE,0
        invoke CloseHandle,hFile
    .else
        invoke MessageBox,hWnd,addr OpenFileFail,
            addr AppName,MB_OK or MB_ICONERROR
    .endif
.elseif ax==IDM_COPY
    invoke SendMessage,hwndRichEdit,WM_COPY,0,0
.elseif ax==IDM_CUT
    invoke SendMessage,hwndRichEdit,WM_CUT,0,0
.elseif ax==IDM_PASTE
    invoke SendMessage,hwndRichEdit,WM_PASTE,0,0
.elseif ax==IDM_DELETE
    invoke SendMessage,hwndRichEdit,EM_REPLACESEL,TRUE,0
.elseif ax==IDM_SELECTALL
    mov chrg.cpMin,0
    mov chrg.cpMax,-1
    invoke SendMessage,hwndRichEdit,EM_EXSETSEL,0,addr chrg
.elseif ax==IDM_UNDO
    invoke SendMessage,hwndRichEdit,EM_UNDO,0,0
.elseif ax==IDM_REDO
    invoke SendMessage,hwndRichEdit,EM_REDO,0,0
.elseif ax==IDM_OPTION
    invoke DialogBoxParam,hInstance,IDD_OPTIONDLG,
        hWnd,addr OptionProc,0
.elseif ax==IDM_SAVEAS
    invoke RtlZeroMemory,addr ofn,sizeof ofn
    mov ofn.lStructSize,sizeof ofn
    push hWnd
    pop ofn.hwndOwner
    push hInstance
    pop ofn.hInstance
    mov ofn.lpstrFilter,offset ASMFilterString
    mov ofn.lpstrFile,offset AlternateFileName
    mov byte ptr [AlternateFileName],0
    mov ofn.nMaxFile,sizeof AlternateFileName
    mov ofn.Flags,OFN_FILEMUSTEXIST or OFN_HIDEREADONLY
        or OFN_PATHMUSTEXIST
    invoke GetSaveFileName,addr ofn
    .if eax!=0
        invoke CreateFile,addr AlternateFileName,
            GENERIC_WRITE,FILE_SHARE_READ, \
            NULL,CREATE_ALWAYS,FILE_ATTRIBUTE_NORMAL,0
        .if eax!=INVALID_HANDLE_VALUE
            jmp @B
        .endif
    .endif
.elseif ax==IDM_FIND
    .if hSearch==0
        invoke CreateDialogParam,hInstance,IDD_FINDDLG,
            hWnd,addr SearchProc,0
    .endif
.elseif ax==IDM_REPLACE
    .if hSearch==0

```

```

        invoke CreateDialogParam,hInstance,IDD_REPLACEDLG,
            hWnd,addr ReplaceProc,0
    .endif
.elseif ax==IDM_GOTOLINE
    .if hSearch==0
        invoke CreateDialogParam,hInstance,IDD_GOTODLG,
            hWnd,addr GoToProc,0
    .endif
.elseif ax==IDM_FINDNEXT
    invoke lstrlen,addr FindBuffer
    .if eax!=0
        invoke SendMessage,hwndRichEdit,EM_EXGETSEL,
            0,addr findtext.chrg
        mov eax,findtext.chrg.cpMin
        .if eax!=findtext.chrg.cpMax
            push findtext.chrg.cpMax
            pop findtext.chrg.cpMin
        .endif
        mov findtext.chrg.cpMax,-1
        mov findtext.lpstrText,offset FindBuffer
        invoke SendMessage,hwndRichEdit,EM_FINDTEXT,
            FR_DOWN,addr findtext
        .if eax!=-1
            invoke SendMessage,hwndRichEdit,EM_EXSETSEL,
                0,addr findtext.chrgText
        .endif
    .endif
.elseif ax==IDM_FINDPREV
    invoke lstrlen,addr FindBuffer
    .if eax!=0
        invoke SendMessage,hwndRichEdit,EM_EXGETSEL,0,
            addr findtext.chrg
        mov findtext.chrg.cpMax,0
        mov findtext.lpstrText,offset FindBuffer
        invoke SendMessage,hwndRichEdit,EM_FINDTEXT,
            0,addr findtext
        .if eax!=-1
            invoke SendMessage,hwndRichEdit,
                EM_EXSETSEL,0,addr findtext.chrgText
        .endif
    .endif
.elseif ax==IDM_EXIT
    invoke SendMessage,hWnd,WM_CLOSE,0,0
.endif
.endif
.elseif uMsg==WM_CLOSE
    invoke CheckModifyState,hWnd
    .if eax==TRUE
        invoke DestroyWindow,hWnd
    .endif
.elseif uMsg==WM_SIZE
    mov eax,lParam
    mov edx,eax
    and eax,0FFFFh
    shr edx,16
    invoke MoveWindow,hwndRichEdit,0,0,eax,edx,TRUE
.elseif uMsg==WM_DESTROY
    invoke PostQuitMessage,NULL
.else
    invoke DefWindowProc,hWnd,uMsg,wParam,lParam
    ret
.endif
xor eax,eax
ret
WndProc endp
end start

```

АНАЛИЗ

Поиск текста реализуется с помощью EM_FINDTEXT. Когда пользователь кликает на пункте меню 'Find', отсылается сообщение IDM_FIND и отображается соответствующее диалоговое окно.

```

invoke GetDlgItemText,hWnd,IDC_FINDEDIT,addr FindBuffer,sizeof FindBuffer
.if eax!=0

```

Когда пользователь задает текст, который нужно найти, а затем нажимает кнопку 'OK', текст, который мы собираемся искать, оказывается в FindBuffer.

```

mov uFlags,0
invoke SendMessage,hwndRichEdit,EM_EXGETSEL,0,addr findtext.chrg

```

Если текстовая строка не равняется NULL, мы устанавливаем значение переменной uFlags равной 0. Эта переменная используется для хранения флагов поиска, отсылаемых вместе с сообщением EM_FINDTEXT. После этого мы получаем текущий выделенный текст с помощью EM_EXGETSEL, потому что нам нужно знать, откуда будет производиться поиск.

```

invoke IsDlgButtonChecked,hWnd,IDC_DOWN
.if eax==BST_CHECKED
    or uFlags,FR_DOWN
    mov eax,findtext.chrg.cpMin
    .if eax!=findtext.chrg.cpMax
        push findtext.chrg.cpMax
        pop findtext.chrg.cpMin
    .endif
    mov findtext.chrg.cpMax,-1
.else
    mov findtext.chrg.cpMax,0
.endif

```

Следующая часть несколько сложнее. Мы проверяем значение кнопки, задающей направление поиска. Если указан поиск вперед, мы задаем флаг FR_DOWN в uFlags. После этого мы проверяем, выделен ли сейчас какой-нибудь текст, проверяя значения cpMin и cpMax. Если оба значения не равны, это означает, что выделение существует, и мы должны продолжать поиск от конца текущего выделения до конца текста в контроле. Поэтому нам требуется заменить значение cpMax на значение cpMin, а cpMax приравнять к -1 (0FFFFFFFh). если выделения нет, поиск будет производиться от текущей позиции курсора до конца текста.

Если пользователь выберет поиск назад, мы используем диапазон от начала выделения до начала текста в контроле. Поэтому на нужно только изменить значение cpMax на 0. В случае с поиском назад cpMin содержит индекс последнего символа в диапазоне поиска, а cpMax - индекс первого символа. Это инверсия поиска вперед.

```

invoke IsDlgButtonChecked,hWnd,IDC_MATCHCASE
.if eax==BST_CHECKED
    or uFlags,FR_MATCHCASE
.endif
invoke IsDlgButtonChecked,hWnd,IDC_WHOLEWORD
.if eax==BST_CHECKED
    or uFlags,FR_WHOLEWORD
.endif
mov findtext.lpstrText,offset FindBuffer

```

Мы продолжаем проверку checkbox'ов, чтобы задать значения FR_MATCHCASE и FR_WHOLEWORD. Наконец, мы помещаем смещение текста, который нужно найти в поле lpstrText.

```

invoke SendMessage,hwndRichEdit,EM_FINDTEXT,uFlags,addr findtext
.if eax!=-1
    invoke SendMessage,hwndRichEdit,EM_EXSETSEL,0,addr findtext.chrgText
.endif

```

Мы сейчас готовы послать сообщение EM_FINDTEXT. После этого мы проверяем результаты поиска, возвращенные SendMessage. Если возвращаемое значение равно -1, значит никаких совпадений не было найдено. В противном случае поле chrgText структуры FINDTEXT заполняется индексами совпавшего текста. Мы выделяем найденный текст с помощью EM_EXSETSEL.

Операция замещения делается подобным же образом.

```
invoke GetDlgItemText,hWnd,IDC_FINDEDIT,
    addr FindBuffer,sizeof FindBuffer
invoke GetDlgItemText,hWnd,IDC_REPLACEEDIT,
    addr ReplaceBuffer,sizeof ReplaceBuffer
```

Мы получаем текст, который нужно найти, и текст, который нужно заменить.

```
mov findtext.chrg.cpMin,0
mov findtext.chrg.cpMax,-1
mov findtext.lpstrText,offset FindBuffer
```

Чтобы сделать проще, операция замещения действует на весь текст в контроле. Поэтому начальный индекс равен 0, а конечный - -1.

```
mov setttext.flags,ST_SELECTION
mov setttext.codepage,CP_ACP
```

Мы инициализируем структуру SETTEXTEX, чтобы задать то, что мы хотим заменить текущее выделение и использовать системную страницу кодировки по умолчанию.

```
.while TRUE
    invoke SendMessage,hwndRichEdit,EM_FINDTEXTEX,
        FR_DOWN,addr findtext
    .if eax== -1
        .break
    .else
        invoke SendMessage,hwndRichEdit,EM_EXSETSEL,
            0,addr findtext.chrgText
        invoke SendMessage,hwndRichEdit,EM_SETTEXTEX,
            addr setttext,addr ReplaceBuffer
    .endif
.endw
```

Мы входим в бесконечный цикл и ищем совпадающий текст. Если он был найден, мы выбираем его с помощью EM_EXSETSEL и заменяем его EM_SETTEXTEX. Если больше совпадений не было найдено, мы выходим из цикла.

'Find Next' и 'Find Prev.' используют сообщение EM_FINDTEXTEX похожим образом.

Теперь мы рассмотрим 'Go to Line'. Когда пользователь кликает по этому пункту меню, мы отображаем диалоговое окно.

Когда пользователь набирает номер линии и нажимет кнопку 'Ok', мы начинаем операцию.

```
invoke GetDlgItemInt,hWnd,IDC_LINENO,NULL,FALSE
mov LineNo,eax
```

Получаем номер линии из edit control'a.

```
invoke SendMessage,hwndRichEdit,EM_GETLINECOUNT,0,0
.if eax>LineNo
```

Получаем количество линий в контроле. Проверяем, не указал ли пользователь номер линии, который выходит за рамки допустимых значений.

```
invoke SendMessage,hwndRichEdit,EM_LINEINDEX,LineNo,0
```

Если номер линии верен, мы передвигаем курсор к первому символу в этой строке. Поэтому мы посылаем сообщение EM_LINEINDEX контролю richedit. Это сообщение возвращает индекс первого символа в заданной строке. Мы посылаем номер строки через wParam, а получаем индекс символа.

```
invoke SendMessage,hwndRichEdit,EM_SETSEL,eax,eax
```

Чтобы установить текущее выделение, в этот раз мы используем EM_SETSEL, потому что индексы символов уже не находятся в структуре CHARRANGE, что сохраняет нам две инструкции (помещение этих индексов в структуру CHARRANGE).

```
        invoke SetFocus,hwndRichEdit  
    .endif
```

Курсор не будет отображен, пока контрол RichEdit не получит фокус. Поэтому мы вызываем SetFocus.

Win32 API. Урок 35. RichEdit Control: подсветка синтаксиса

Автор: Iczelion, пер. Aquila

Дата: 2002-06-05

Вначале я хочу вас предупредить, что это сложная тема, не подходящая для начинающего. Это последний tutorial из серии о RichEdit.

Скачайте пример (находится в архиве `wasm_files.zip: files/recovered/1001035/tut35.zip`).

ТЕОРИЯ

Подсветка синтаксиса - это предмет жарких дискуссий между создателями текстовых редакторов. Лучший метод (на мой взгляд) - это создать собственный edit control. Именно этот метод применяется во многих коммерческих приложениях. Тем не менее для тех из нас, у кого нет времени на создание подобного контрола, лучшим вариантом будет приспособить существующий контрол к нашим нуждам.

Давайте посмотрим, как может нам помочь RichEdit в реализации цветовой подсветки. Я должен сказать, что следующий метод "неверен": я всего лишь продемонстрирую ловушку, в которую угодили многие. RichEdit предоставляет сообщение `EM_SETCHARFORMAT`, которое позволяет вам менять цвет текста. На первый взгляд это именно то, что нам нужно (я знаю, потому что я стал одной из жертв этого подхода). Тем не менее, более детальное исследование покажет вам, что у данного сообщения есть несколько недостатков:

`EM_SETCHARFORMAT` работает либо со всем текстом сразу, либо только с тем, который сейчас выделен. Если вам потребуется изменить цвет определенного слова, вам сначала придется выделить его.

`EM_SETCHARFORMAT` очень медленна.

У нее есть проблемы с позицией курсора в контроле RichEdit.

Из всего вышеизложенного вы можете сделать вывод, что использование `EM_SETCHARFORMAT` - неправильный выбор. Я покажу вам "относительно верный" выбор.

Мой метод заключается только в том, чтобы подсвечивать только видимую часть текста. В этом случае скорость подсветки не будет зависеть от размера файла. Вне зависимости от того, как велик файл, только маленькая его часть видна на экране.

Как это сделать? Ответ прост:

Субклассируйте контрол RichEdit и обрабатывайте сообщение `WM_PAINT` внутри вашей оконной процедуры.

Когда она встречает сообщение `WM_PAINT`, вызывается оригинальная оконная функция, которая обновляет экран как обычно.

После этого мы перерисовываем слова, которые нужно отобразить другим цветом.

Конечно, этот путь не так прост: есть несколько вещей, которые нужно пофиксить, но вышеизложенный метод работает достаточно хорошо. Скорость отображения на экране более чем удовлетворительна.

Теперь давайте сконцентрируемся на деталях. Процедура сабклассинга проста и не требует внимательного рассмотрения. Действительно сложная часть - это когда мы находим быстрый способ поиска слов, которые нужно подсветить. Это усложняется тем, что не надо подсвечивать

слова внутри комментариев.

Метод, который я использовал, возможно, не самый лучший, но он работает хорошо. Я уверен, что вы сможете найти более быстрый путь. Как бы то ни было, вот он:

Я создал массив на 256 двойных слов, заполненный нулями. Каждый dword соответствует возможному ASCII-символу. Я назвал массив ASMSyntaxArray. Например, 21-ый элемент представляет собой 20h (пробел). Я использовал массив для быстрого просмотра таблицы: например, если у меня есть слово "include", я извлекаю первый символ из слова и смотрю значение соответствующего элемента. Если оно равно 0, я знаю, что среди слов, которые нужно подсвечивать, нет таких, которые бы начинались с "i". Если элемент не равен нулю, он содержит указатель на структуру WORDINFO, в которой находится информация о слове, которое должно быть подсвечено.

Я читаю слова, которые нужно подсвечивать, и создаю структуру WORDINFO для каждого из них.

```
WORDINFO struct
    WordLen dd ? ; длина слова: используется для быстрого сравнения
    pszWord dd ? ; указатель на слово
    pColor dd ? ; указатель на dword, содержащий цвет, которым
                  ; нужно подсвечивать слово
    NextLink dd ? ; указатель на следующую структуру WORDINFO
WORDINFO ends
```

Как вы можете видеть, я использовал длину слова для ускорения процесса сравнения. Если первый символ слова совпадает, мы можем сравнить его длину с доступными словами. Каждый dword в ASMSyntaxArray содержит указатель на начало ассоциированного с ним массива WORDINFO. Например dword, который представляет символ "i", будет содержать указатель на связанный список слов, которые начинаются с "i". Поле pColor указывает на слово, содержащее цвет, которым будет подсвечиваться слово. pszWord указывает на слово, которое будет подсвечиваться, преобразованное к нижнему регистру.

Память для связанного списка занимает из кучи программы по умолчанию, поэтому очищать ее легко и быстро: никакой чистки не требуется вообще.

Список слов сохраняется в файле под названием "wordfile.txt", я получаю к нему доступ через API-функцию GetPrivateProfileString. Есть десять различных подсветок символов, начинающиеся с C1 до C10. Массив цветов называется ASMCOLORArray. Поле pColor каждой структуры WORDINFO указывает на один из элементов ASMCOLORArray. Поэтому можно легко изменять подсветку синтаксиса на лету: вам всего лишь нужно изменить dword в ASMCOLORArray и все слова, которые используют этот цвет, будут использовать новое значение.

ПРИМЕР

Здесь исходник. Возьмите его из архива ;)

АНАЛИЗ

Первое действие, совершаемое до вызова WinMain, это вызов FillHiliteInfo. Эта функция считывает содержимое wordfile.txt и обрабатывает его.

```
FillHiliteInfo proc uses edi
    LOCAL buffer[1024]:BYTE
    LOCAL pTemp:DWORD
    LOCAL BlockSize:DWORD
    invoke RtlZeroMemory,addr ASMSyntaxArray,sizeof ASMSyntaxArray

    Initialize ASMSyntaxArray to zero.
    invoke GetModuleFileName,hInstance,addr buffer,sizeof buffer
    invoke strlen,addr buffer
    mov ecx,eax
    dec ecx
    lea edi,buffer
```

```

add edi,ecx
std
mov al,"\"
repne scasb
cld
inc edi
mov byte ptr [edi],0
invoke lstrcat,addr buffer,addr WordFileName

```

Создаем путь до wordfile.txt: я предполагаю, что он всегда находится в той же папке, что и сама программа.

```

invoke GetFileAttributes,addr buffer
.if eax!=-1

```

Я использую этот метод как быстрый путь для проверки, существует ли файл.

```

mov BlockSize,1024*10
invoke HeapAlloc,hMainHeap,0,BlockSize
mov pTemp,eax

```

Резервируем блок памяти, чтобы сохранять слова. По умолчанию - 10 килобайт. Память занимаем из кучи по умолчанию.

```

@@:
invoke GetPrivateProfileString,addr ASMSection,
    addr C1Key,addr ZeroString,pTemp,
    BlockSize,addr buffer
.if eax!=0

```

Я использовал GetPrivateProfileString, чтобы получить содержимое каждого ключевого слова в wordfile.txt, начиная с C1 и до C10.

```

inc eax
.if eax==BlockSize ; the buffer is too small
add BlockSize,1024*10
invoke HeapReAlloc,hMainHeap,0,pTemp,BlockSize
mov pTemp,eax
jmp @B
.endif

```

Проверяем, достаточно ли велик блок памяти. Если это не так, мы увеличиваем размер на 10k, пока он не окажется достаточно велик.

```

mov edx,offset ASMCOLORArray
invoke ParseBuffer,hMainHeap,
    pTemp,eax,edx,addr ASMSyntaxArray

```

Передаем слова, хэндл на блок памяти, размер данных, считанных из wordfile.txt, адрес dword с цветом, который будет использоваться для подсветки слов и адреса ASMSyntaxArray.

Теперь давайте посмотрим, что делает ParseBuffer. В сущности, эта функция принимает буфер, содержащий слова, которые должны быть выделены, парсит их на отдельные слова и сохраняет каждое из них в массиве структур WORDINFO, к которому можно легко получить доступ из ASMSyntaxArray.

```

ParseBuffer proc uses edi esi hHeap:DWORD,
    pBuffer:DWORD, nSize:DWORD,
    ArrayOffset:DWORD, pArray:DWORD
LOCAL buffer[128]:BYTE
LOCAL InProgress:DWORD
mov InProgress,FALSE

```

InProgress - это флаг, который я использовал, чтобы суметь определить, начался ли процесс сканирования. Если значение равно FALSE, vs

```

lea esi,buffer
mov edi,pBuffer
invoke CharLower,edi

```

esi указывает на наш буфер, который будет содержать слово, взятое нами из списка слов. edi указывает на строку-список слов. Чтобы упростить поиск, мы приводим все символы к нижнему регистру.

```
    mov ecx,nSize
SearchLoop:
    or ecx,ecx
    jz Finished
    cmp byte ptr [edi]," "
    je EndOfWord
    cmp byte ptr [edi],9    ; tab
    je EndOfWord
```

Сканируем весь список слов в буфере, ориентируясь по пробелам, по которым определяем конец или начало слова.

```
    mov InProgress,TRUE
    mov al,byte ptr [edi]
    mov byte ptr [esi],al
    inc esi
SkipIt:
    inc edi
    dec ecx
    jmp SearchLoop
```

Если оказывается, что байт не является пробелом, то мы копируем его в буфер, где создается слово, а затем продолжаем поиск.

```
EndOfWord:
    cmp InProgress,TRUE
    je WordFound
    jmp SkipIt
```

Если пробел был найден, то мы проверяем значение в InProgress. Если значение равно TRUE, мы можем предположить, что был достигнут конец слова и мы можем поместить то, что у нас получилось в наш буфер (на который указывает esi) в структуру WORDINFO. Если значение равно FALSE, мы продолжаем сканирование дальше.

```
WordFound:
    mov byte ptr [esi],0
    push ecx
    invoke HeapAlloc,hHeap,HEAP_ZERO_MEMORY,sizeof WORDINFO
```

Если был достигнут конец слова, мы прибавляем 0 к буферу, чтобы сделать из слова строку формата ASCIIZ. Затем мы занимаем для этого слова блок памяти из кучи размером в WORDINFO.

```
    push esi
    mov esi,eax
    assume esi:ptr WORDINFO
    invoke strlen,addr buffer
    mov [esi].WordLen,eax
```

Мы получаем длину слова в нашем буфере и сохраняем ее в поле WordLen структуры WORDINFO, чтобы использовать в дальнейшем при быстром сравнении.

```
    push ArrayOffset
    pop [esi].pColor
```

Сохраняем адрес dword'a, который содержит цвет для подсветки слова, в поле pColor.

```
    inc eax
    invoke HeapAlloc,hHeap,HEAP_ZERO_MEMORY,eax
    mov [esi].pszWord,eax
    mov edx,eax
    invoke strcpy,edx,addr buffer
```

Занимает блок памяти из кучи, чтобы сохранить само слово. В настоящее время структура WORDINFO готова для вставки в соответствующий связанный список.

```
mov eax,pArray
movzx edx,byte ptr [buffer]
shl edx,2    ; multiply by 4
add eax,edx
```

pArray содержит адрес ASMSyntaxArray. Нам нужно перейти к dword'у, у которого тот же индекс, что и у значения первого символа слова. Поэтому мы помещаем первый символ в edx, а затем умножаем edx на 4 (так как каждый элемент в ASMSyntaxArray равен 4 байтам), а затем добавляем получившееся смещение к адресу ASMSyntaxArray. Теперь адрес нужного dword'a находится в eax.

```
.if dword ptr [eax]==0
    mov dword ptr [eax],esi
.else
    push dword ptr [eax]
    pop [esi].NextLink
    mov dword ptr [eax],esi
.endif
```

Проверяем значение dword'a. Если оно равно 0, это означает, что в настоящее время ключевых слов, которые начинаются с этого символа, нет. Затем мы помещаем адрес текущей структуры WORDINFO в этот dword.

Если значение dword'a не равно 0, это означает, что есть по крайней мере одно ключевое слово, которое начинается с этого символа. Затем мы вставляем эту структуру WORDINFO в голову связанного списка и обновляем его поле NextLink, чтобы оно указывало на структуру WORDINFO.

```
pop esi
pop ecx
lea esi,buffer
mov InProgress,FALSE
jmp SkipIt
```

После того как операция завершена, мы начинаем следующую цикл сканирования, пока не будет достигнут конец буфера.

```
invoke SendMessage,hwndRichEdit,EM_SETTYPOGRAPHYOPTIONS,
    TO_SIMPLELINEBREAK,TO_SIMPLELINEBREAK
invoke SendMessage,hwndRichEdit,EM_GETTYPOGRAPHYOPTIONS,1,1
.if eax==0    ; means this message is not processed
    mov RichEditVersion,2
.else
    mov RichEditVersion,3
    invoke SendMessage,hwndRichEdit,EM_SETEDITSTYLE,
        SES_EMULATESYSEDIT,SES_EMULATESYSEDIT
.endif
```

После того, как контрол RichEdit создан, нам нужно определить его версию. Этот шаг необходим, так как поведение EM_POSFROMCHAR отличается в зависимости от версии RichEdit, а EM_POSFROMCHAR жизненно важна для нашей процедуры подсветки. Я никого не видел документированного способа определения версии RichEdit, поэтому я пошел окольным путем. Я устанавливаю опцию, которая свойственна версии 3.0 и немедленно возвращает его значение. Если я могу получить значение, я предполагаю, что версия этого контрола 3.0.

Если вы используете контрол RichEdit версии 3.0, вы можете заметить, что обновление цвета фонт на больших файлах занимает довольно много времени. Похоже, что эта проблема существует только в версии 3.0. Я нашел способ обойти это, заставив контрол эмулировать поведение контрола edit, послав сообщение EM_SETEDITSTYLE.

После того, как мы получили информацию о версии контрола, мы переходим к сабклассированию контрола RichEdit. Сейчас мы рассмотрим новую процедуру для обработки сообщений.

```
NewRichEditProc proc hwnd:DWORD, uMsg:DWORD,
```

```

wParam:DWORD, lParam:DWORD
.....
.....
.if uMsg==WM_PAINT
    push edi
    push esi
    invoke HideCaret,hWnd
    invoke CallWindowProc,OldWndProc,hWnd,
        uMsg,wParam,lParam
    push eax

```

Мы обрабатываем сообщение WM_PAINT. Во-первых, мы прячем курсор, чтобы избежать уродливых артефактов после подсветки. Затем мы передаем сообщение оригинальной процедуре richedit, чтобы оно обновило окно. Когда CallWindowProc возвращает управление, текст обновляется согласно его обычному цвету/бэкграунду. Теперь мы можем сделать подсветку синтаксиса.

```

mov edi,offset ASMSyntaxArray
invoke GetDC,hWnd
mov hdc,eax
invoke SetBkMode,hdc,TRANSPARENT

```

Сохраняем адрес ASMSyntaxArray в edi. Затем мы получаем хэндл контекста устройства и делаем бэкграунд текста прозрачными, чтобы при выводе нами текста использовался текущий бэкграундный цвет.

```

invoke SendMessage,hWnd,EM_GETRECT,0,addr rect
invoke SendMessage,hWnd,EM_CHARFROMPOS,0,addr rect
invoke SendMessage,hWnd,EM_LINEFROMCHAR,eax,0
invoke SendMessage,hWnd,EM_LINEINDEX,eax,0

```

Мы хотим получить видимый текст, поэтому сначала нам требуется узнать размеры области, которую необходимо форматировать, послав ему EM_GETRECT. Затем мы получаем индекс ближайшего к левому верхнему углу символа с помощью сообщения EM_CHARFROMPOS. Как только мы получаем индекс первого символа, мы начинаем делать цветовую подсветку, начиная с этой позиции. Но эффект может быть не так хорош, как если бы начали с первого символа линии, в которой находится символ. Вот почему мне нужно было получить номер линии, в которой находится первый видимый символ, с помощью сообщения EM_LINEFROMCHAR. Чтобы получить первый символ этой линии, я посылаю сообщение EM_LINEINDEX.

```

mov txrange.chrg.cpMin,eax
mov FirstChar,eax
invoke SendMessage,hWnd,EM_CHARFROMPOS,0,addr rect.right
mov txrange.chrg.cpMax,eax

```

Как только мы получили индекс первого символа, сохраняем его на будущее в переменной FirstChar. Затем мы получаем последний видимый символ, посылая ему EM_CHARFROMPOS, передавая нижний правый угол формируемой области в lParam.

```

push rect.left
pop RealRect.left
push rect.top
pop RealRect.top
push rect.right
pop RealRect.right
push rect.bottom
pop RealRect.bottom
invoke CreateRectRgn,RealRect.left,RealRect.top,
    RealRect.right,RealRect.bottom
mov hRgn,eax
invoke SelectObject,hdc,hRgn
mov hOldRgn,eax

```

Во время подсветки синтаксиса, я заметил один побочный эффект этого метода: если у контрола richedit'a есть отступ (который вы можете указать, послав сообщение EM_SETMARGINS контролу RichEdit), DrawText пишет поверх него. Поэтому мне требуется создать функцией CreateRectRgn

ограничительный регион, в который будут выводиться результат выполнения функций GDI.

Затем нам требуется подсветить комментарии и убрать их с нашего пути. Мой метод состоит в поиске ";" и подсветке текста цветом комментария, пока не будет достигнут перевод каретки. Я не буду анализировать здесь эту процедуру: она довольно длинна и сложна. Достаточно сказать, что когда подсвечены все комментарии, мы замещаем их нулями в нашем буфере, чтобы слова к комментариям не обрабатывались позже.

```
mov ecx,BufferSize
lea esi,buffer
.while ecx>0
    mov al,byte ptr [esi]
    .if al==" " || al==0Dh || al=="/" || al=="|" || al=="|" || \
        al=="+" || al=="-" || al=="*" || al=="&" || al=="<" \
        || al==">" || al=="=" || al=="(" || al==")" || al=="{" \
        || al=="}" || al=="[" || al=="]" || al=="^" || al=="|" \
        || al==9
        mov byte ptr [esi],0
    .endif
    dec ecx
    inc esi
.endw
```

Как только мы избавились от комментариев, мы разделяем слова в буфере, замещая символы "разделения" нулями. С помощью этого метода нам не нужно заботиться о символах разделения во время обработки слов в дальнейшем, так как у нас будет только один символ разделения.

```
lea esi,buffer
mov ecx,BufferSize
.while ecx>0
    mov al,byte ptr [esi]
    .if al!=0
```

Ищем в буфере первый символ, не равный NULL, т.е. первый символ слова.

```
push ecx
invoke strlen,esi
push eax
mov edx,eax
```

Получаем длину слова и помещаем его в edx.

```
movzx eax,byte ptr [esi]
.if al>"A" && al<="Z"
    sub al,"A"
    add al,"a"
.endif
```

Конвертируем символ в нижний регистр (если он в верхнем).

```
shl eax,2
add eax,edi ; edi содержит указатель на массив указателей
; на структуры WORDINFO
.if dword ptr [eax]!=0
```

После этого мы переходим к соответствующему dword'у в ASMSyntaxArray и проверяем равно ли его значение 0. Если это так, мы можем перейти к следующему слову.

```
mov eax,dword ptr [eax]
assume eax:ptr WORDINFO
.while eax!=0
    .if edx==[eax].WordLen
```

Если значение dword'a не равно нулю, он указывает на связанный список структур WORDINFO. Мы делаем пробег по связанному списку, сравнивая длину слова в нашем буфере с длиной слова в структуре WORDINFO. Это быстрый тест, проводимый до полноценного сравнения слов, что должно сохранить несколько тактов.

```

pushad
invoke lstrcmpi,[eax].pszWord,esi
.if eax==0

```

Если длины обоих слов равны, мы переходим к сравнению слов с помощью lstrcmpi.

```

popad
mov ecx,esi
lea edx,buffer
sub ecx,edx
add ecx,FirstChar

```

Мы создаем индекс символа из адресов первого символа совпадающего слова в буфере. Сначала мы получаем его относительное смещение из стартового адреса буфера, а затем добавляем индекс символа первого видимого символа к нему.

```

pushad
.if RichEditVersion==3
    invoke SendMessage,hWnd,EM_POSFROMCHAR,addr rect,ecx
.else
    invoke SendMessage,hWnd,EM_POSFROMCHAR,ecx,0
    mov ecx,eax
    and ecx,0FFFFh
    mov rect.left,ecx
    shr eax,16
    mov rect.top,eax
.endif
popad

```

Узнав индекс первого символа слова, которое должно быть подсвечено, мы переходим к получению его координат с помощью сообщения EM_POSFROMCHAR. Тем не менее это сообщение интерпретируется различным образом в RichEdit 2.0 или 3.0. В RichEdit 2.0 wParam содержит индекс символа, а lParam не используется. Сообщение возвращается в eax. В RichEdit 3.0 wParam - это указатель на структуру POINT, которая будет заполнена координатой, а lParam содержит индекс символа.

Как вы можете видеть, передача неверных аргументов EM_POSFROMCHAR может привести к непредсказуемым результатам. Вот почему я должен определить версию RichEdit.

```

mov edx,[eax].pColor
invoke SetTextColor,hdc,dword ptr [edx]
invoke DrawText,hdc,esi,-1,addr rect,0

```

Как только мы получили координату, с которой надо начинать, мы устанавливаем цвет текста, который указан в структуре WORDINFO.

В заключение я хочу сказать, что этот метод можно улучшить различными способами. Например я получаю весь текст, который начинается от первой до последней видимой линии. Если линии очень длинные, качество может пострадать из-за обработки невидимых слов. Вы можете оптимизировать это, получая видимый текст полинейно. Также можно улучшить алгоритм поиска. Поймите меня правильно: подсветка синтаксиса, используемая в этом примере, быстрая, но она может быть быстрее. :)

PE. Урок 1. Обзор PE формата

Автор: lczelion, пер. Aquila

Дата: 2002-06-06

Это полностью переписанный вариант старых PE-туториалов N1, которые я считаю худшими туториалами, когда-либо мной написанных. Поэтому я решил заменить их этими.

PE означает Portable Executable. Это 'родной' файловый формат Win32. Его спецификации происходят от Unix Coff (common object file format). "Portable executable" означает, что файловый формат универсален для платформы win32: загрузчик PE любой win32-платформы распознает и использует этот файловый формат даже когда Windows запускается на не PC CPU-платформе, хотя это не означает, что ваши PE можно будет портировать на другие CPU-платформы без изменений. Каждый win32-исполняемый файл (кроме VxD и 16-битных DLL) использует PE-формат. Даже драйвера ядра NT используют PE-формат. Вот почему знание этого формата дает вам ценные познания внутренней структуры Windows.

Давайте посмотрим на общую компоновку PE-файла.

DOS MZ-заголовок
DOS stub
Заголовок PE
Таблица секций
Секция 1
Секция 2
Секция ...
Секция n

Вышприведенная таблица представляет собой общую структуру PE-файла. Все PE-файлы (даже 32-битные DLL) должны начинаться с обычного досовского MZ-заголовка. Обычно он нам не очень интересен, так как нужен лишь для того, чтобы если программа будет вдруг запущена из-под DOS'а, он мог распознать ее как исполняемый файл и мог запустить DOS-stub, который находится за MZ-заголовком. DOS-stub, фактически, является полноценным exe, который запускается операционной системой, не знающей о PE-формате. Он может просто отображать строку вроде "This program requires Windows" или может быть полноценной DOS-программой. Нам DOS-stub не очень интересен: он обычно предоставляется ассемблером/компилятором. В большинстве случаев, он просто использует int 21h, сервис 9, чтобы напечатать строку, которая отображает "This program cannot run in DOS mode".

После DOS-stub'а идет PE-заголовок. PE-заголовок - это общее название структуры под названием IMAGE_NT_HEADERS. Эта структура содержит много основных полей, используемых PE-загрузчиком. Скоро мы подробно с ней ознакомимся. В случае, если программа запускается операционной системой, которая знает о PE-формате, PE-загрузчик может найти смещение PE-заголовка в заголовке DOS-MZ. После этого он может пропустить DOS-stub и перейти напрямую к PE-заголовку, который является настоящим заголовком исполняемого файла.

Настоящее содержимое PE-файла разделено на блоки, называемые секциями. Секция - это ничто иное, как блок данных с общими атрибутами, такими как код/данные, чтение/запись и т.д. PE-файл можно сравнить с логическим диском. PE-заголовок - это загрузочный сектор, а секции - это файлы на диске. Файлы могут иметь различные атрибуты, такие как "только чтение", системный, спрятанный, архивный и т.п. Я хочу, чтобы было предельно ясно, что группирование данные

производится на основе их атрибутов. Не играет роли, как используются код/данные, если данные/код в PE-файле имеют одинаковые атрибуты, они могут быть сгруппированы в секцию. Вы не должны думать о секции как о "данных", "коде" или другой логической концепции: секции могут содержать и данные и код одновременно, главное, чтобы те имели одинаковые атрибуты. Если у вас есть данные, и вы хотите, чтобы они были доступны только для чтения, вы можете поместить эти данные в секцию, помеченную соответствующим атрибутом.

Если мы будем рассматривать файл в PE-формате как логический диск, PE-заголовок как бут-сектор, а секции как файлы, у нас все еще недостаточно информации, чтобы найти, где на диске находятся файлы, то есть мы еще не обсуждали эквивалент директории в PE-формате. Непосредственно за PE-заголовком следует таблица секций, представляющая собой массив структур. Каждая структура содержит информацию о каждой секции в PE-файле, такую как ее атрибут, смещение в файле, виртуальное смещение. Если в файле 5 секций, то будет ровно 5 членов в этом массиве структур.

Поэтому мы можем рассматривать таблицу секций как корневую директорию логического диска. Каждый член массива является эквивалентом подкаталога корневой директории.

Вот и все об общей структуре PE-файла. Я кратко изложу основные шаги, выполняющиеся при загрузке PE-файла в память:

- Когда PE-файл запускается, PE-загрузчик проверяет DOS MZ-заголовок, для того, чтобы определить смещение PE-заголовка. Если оно найдено, то загрузчик переходит к PE-заголовку.
- PE-загрузчик проверяет, является ли PE-заголовок целым и неиспорченным. Если это так, то он переходит к концу PE-заголовка.
- За ним незамедлительно следует таблица секций. PE-загрузчик считывает информацию о секциях и загружает эти секции в память. Также он устанавливает каждой секции атрибуты, указанные в таблице секций.
- После того, как PE-файл загружен в память, PE-загрузчик обрабатывает логические части PE-файла, например таблицу импорта.

Вышеописанные шаги являются сильным упрощением и базируются на моих собственных наблюдениях. Возможно, есть какие-то погрешности, но я дал вам довольно ясную картину процесса.

Вам следует скачать описание PE-формата, сделанное LUEVELSMEYER'ом. Оно очень подробное и может послужить вам справочником.

РЕ. Урок 2. Правильность РЕ файла

Автор: lczelion, пер. Aquila

Дата: 2002-06-06

В этом tutorialе мы научимся, как проверить, является ли файл РЕ-файлом.

Скачайте пример (находится в архиве wasm_files.zip: files/recovered/1002002/pe-tut02.zip).

ПРИМЕР

Как вы можете проверить, является ли данный файл РЕ-файлом? На этот вопрос трудно сразу ответить. Это зависит от того, с какой степенью надежности вы хотите это сделать. Вы можете проверить каждый параметр файла в РЕ-формата, а можете ограничиться проверкой самых важных из них. Как правило, проверять все параметры бессмысленно. Если критические структуры верны, мы можем допустить, что файл РЕ-формата. И мы сделаем это допущение.

Основная структура, которую мы будем проверять - это РЕ-заголовок. Поэтому нам нужно больше узнать о нем. Фактически РЕ-заголовок - это структура под названием IMAGE_NT_HEADERS. Она определена следующим образом:

```
IMAGE_NT_HEADERS STRUCT
    Signature dd ?
    FileHeader IMAGE_FILE_HEADER <>
    OptionalHeader IMAGE_OPTIONAL_HEADER32 <>
IMAGE_NT_HEADERS ENDS
```

Signature - это слово, которое содержит значение 50h, 45h, 00h, 00h. Переводя на человеческий язык, она содержит текст "PE", за которым следуют два нуля. Этот член является сигнатурой РЕ, поэтому мы будем использовать его для того, чтобы определить, является ли данный файл РЕ-формата.

FileHeader - это структура, которая содержит информацию о физической структуре РЕ-файла, такой как количество секций, устройство, на которое ориентирован данный файл и так далее.

OptionalHeader - это структура, которая содержит информацию о логической структуре РЕ-файла. Несмотря на "Optional" в ее имени, этот член всегда присутствует.

Наша цель ясна. Если значение сигнатуры в IMAGE_NT_HEADERS равно "PE", за которым следуют два нуля, тогда файла является РЕ. Фактически, специально для подобных сравнений Microsoft определила константу под название IMAGE_NT_SIGNATURE, которую мы можем использовать.

```
IMAGE_DOS_SIGNATURE equ 5A4Dh
IMAGE_OS2_SIGNATURE equ 454Eh
IMAGE_OS2_SIGNATURE_LE equ 454Ch
IMAGE_VXD_SIGNATURE equ 454Ch
IMAGE_NT_SIGNATURE equ 4550h
```

Следующий вопрос: как мы можем узнать, где начинается РЕ-заголовок? Ответ прост: DOS MZ-заголовок содержит файловое смещение РЕ-заголовка. DOS MZ-заголовок определен как структура IMAGE_DOS_HEADER. Параметр e_lfanew этой структуры содержит файловое смещение РЕ-заголовка.

Теперь мы выполним следующие шаги:

- Проверяем, верный ли у данного файла DOS MZ-заголовок, сравнивая первое слово этого файла со значением IMAGE_DOS_SIGNATURE.
- Если у файла верный DOS-заголовок, используем значение параметра e_lfanew, чтобы найти РЕ-заголовок.

- Сравниваем первое слово PE-заголовка со значением IMAGE_NT_HEADER. Если оба значения совпадают, тогда мы можем предположить, что этот файл является Portable Executable.

ПРИМЕР

```
.386
.model flat,stdcall
option casemap:none
include \masm32\include\windows.inc
include \masm32\include\kernel32.inc
include \masm32\include\comdlg32.inc
include \masm32\include\user32.inc
includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib
includelib \masm32\lib\comdlg32.lib

SEH struct
PrevLink dd ? ; the address of the previous seh structure
CurrentHandler dd ? ; the address of the exception handler
SafeOffset dd ? ; The offset where it's safe to continue execution
PrevEsp dd ? ; the old value in esp
PrevEbp dd ? ; The old value in ebp
SEH ends

.data
AppName db "PE tutorial no.2",0
ofn OPENFILENAME <>
FilterString db "Executable Files (*.exe, *.dll)",0,"*.exe;*.dll",0
db "All Files",0,"*.*",0,0
FileOpenError db "Cannot open the file for reading",0
FileOpenMappingError db "Cannot open the file for memory mapping",0
FileMappingError db "Cannot map the file into memory",0
FileValidPE db "This file is a valid PE",0
FileInvalidPE db "This file is not a valid PE",0

.data?
buffer db 512 dup(?)
hFile dd ?
hMapping dd ?
pMapping dd ?
ValidPE dd ?

.code
start proc
LOCAL seh:SEH
mov ofn.lStructSize,SIZEOF ofn
mov ofn.lpstrFilter, OFFSET FilterString
mov ofn.lpstrFile, OFFSET buffer
mov ofn.nMaxFile,512
mov ofn.Flags, OFN_FILEMUSTEXIST or OFN_PATHMUSTEXIST
or OFN_LONGNAMES or OFN_EXPLORER or OFN_HIDEREADONLY
invoke GetOpenFileName, ADDR ofn
.if eax==TRUE
invoke CreateFile, addr buffer, GENERIC_READ,
FILE_SHARE_READ, NULL, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL
.if eax!=INVALID_HANDLE_VALUE
mov hFile, eax
invoke CreateFileMapping, hFile, NULL, PAGE_READONLY,0,0,0
.if eax!=NULL
mov hMapping, eax
invoke MapViewOfFile,hMapping,FILE_MAP_READ,0,0,0
.if eax!=NULL
mov pMapping,eax
assume fs:nothing
push fs:[0]
pop seh.PrevLink
mov seh.CurrentHandler,offset SEHHandler
mov seh.SafeOffset,offset FinalExit
lea eax,seh
mov fs:[0], eax
mov seh.PrevEsp,esp
```

```

        mov seh.PrevEbp,ebp
        mov edi, pMapping
        assume edi:ptr IMAGE_DOS_HEADER
        .if [edi].e_magic==IMAGE_DOS_SIGNATURE
            add edi, [edi].e_lfanew
            assume edi:ptr IMAGE_NT_HEADERS
            .if [edi].Signature==IMAGE_NT_SIGNATURE
                mov ValidPE, TRUE
            .else
                mov ValidPE, FALSE
            .endif
        .else
            mov ValidPE,FALSE
        .endif
FinalExit:
        .if ValidPE==TRUE
            invoke MessageBox, 0, addr FileValidPE, addr AppName, MB_OK+MB_ICONINFORMATION
        .else
            invoke MessageBox, 0, addr FileInvalidPE, addr AppName, MB_OK+MB_ICONINFORMATION
        .endif
        push seh.PrevLink
        pop fs:[0]
        invoke UnmapViewOfFile, pMapping
    .else
        invoke MessageBox, 0, addr FileMappingError, addr AppName, MB_OK+MB_ICONERROR
    .endif
    invoke CloseHandle,hMapping
    .else
        invoke MessageBox, 0, addr FileOpenMappingError, addr AppName, MB_OK+MB_ICONERROR
    .endif
    invoke CloseHandle, hFile
    .else
        invoke MessageBox, 0, addr FileOpenError, addr AppName,
        MB_OK+MB_ICONERROR
    .endif
    .endif
    invoke ExitProcess, 0
start endp

SEHHandler proc uses edx pExcept:DWORD, pFrame:DWORD, pContext:DWORD, pDispatch:DWORD
    mov edx,pFrame
    assume edx:ptr SEH
    mov eax,pContext
    assume eax:ptr CONTEXT
    push [edx].SafeOffset
    pop [eax].regEip
    push [edx].PrevEsp
    pop [eax].regEsp
    push [edx].PrevEbp
    pop [eax].regEbp
    mov ValidPE, FALSE
    mov eax,ExceptionContinueExecution
    ret
SEHHandler endp
end start

```

АНАЛИЗ

Программа открывает файл и проверяет, является ли DOS-заголовок верным, если это так, она проверяет, является ли PE-заголовок верным. Если и это так, она решает, что данный файл - PE. В этом примере я использовал structured exception handling (SEH), поэтому мы не должны проверять любую возможную ошибку, если ошибка происходит, мы предполагаем, что она произошла из-за того, что файл не являлся верным PE. Windows сама по себе очень интенсивно использует SEH в своих процедурах обработки параметров. Если вы заинтересовались SEH'ом, читайте соответствующую статью Jeremy Gordon'a.

Программа отображает окно открытия файла, и когда пользователь выбирает исполняемый файл, она открывает файл и загружает его в память. Перед тем, как проводить проверку файла, она

устанавливает SEH.

```
assume fs:nothing
push fs:[0]
pop seh.PrevLink
mov seh.CurrentHandler,offset SEHHandler
mov seh.SafeOffset,offset FinalExit
lea eax,seh
mov fs:[0], eax
mov seh.PrevEsp,esp
mov seh.PrevEbp,ebp
```

Мы начинаем с того, что устанавливаем режим использования регистра fs "nothing". Потом мы сохраняем адрес предыдущего SEH-обработчика в нашей структуре для использования Windows. Мы сохраняем адрес нашего SEH-обработчика, адрес где стартует обработка исключения, если происходит ошибка, текущие значения esp и ebp, так что наш SEH-обработчик может получить состояние нормально состояние стека перед тем, как продолжать программу.

```
mov edi, pMapping
assume edi:ptr IMAGE_DOS_HEADER
.if [edi].e_magic==IMAGE_DOS_SIGNATURE
```

После установления SEH'а, мы продолжаем проверку. Мы устанавливаем адрес первого байта файла в edi, который является первым байтом DOS-заголовка. Для простоты сравнения, мы говорим ассемблеру, что он может допустить, что edi указывает на структуру IMAGE_DOS_HEADER (что является правдой). Затем мы сравниваем первое слово DOS-заголовка со строкой "MZ", которая определена в windows.inc под названием IMAGE_DOS_SIGNATURE. Если сравнение положительно, мы переходим к PE-заголовку. Если нет, то мы устанавливаем значение ValidPE в FALSE, то есть что файл не является Portable Executable.

```
add edi, [edi].e_lfanew
assume edi:ptr IMAGE_NT_HEADERS
.if [edi].Signature==IMAGE_NT_SIGNATURE
    mov ValidPE, TRUE
.else
    mov ValidPE, FALSE
.endif
```

Чтобы добраться до PE-заголовка, нам нужно значение, находящееся в e_lfanew DOS-заголовка. Это поле содержит смещение в файле PE-заголовка, относительно начала файла. Поэтому мы добавляем это значение к edi и получаем первый байт PE-заголовка. Это то место, где может произойти ошибка. Если файл на самом деле не PE-файл, значение в e_lfanew будет неверным и использование его будет подобно использованию случайного указателя. Если мы не используем SEH, мы должны сравнить e_lfanew с размером файла, что некрасиво. Если все идет хорошо, мы сравниваем первое двойное слово PE-заголовка со строкой "PE". Снова мы можем использовать уже определенную константу под названием IMAGE_NT_SIGNATURE. Если результат сравнения верен, мы предполагаем, что файл является правильным PE.

Если значение в e_lfanew неверно, может произойти ошибка и наш SEH-обработчик получит управление. Он просто восстанавливает указатель на стек, bsaе-указатель и продолжает выполнение программы с метки FinalExit.

```
FinalExit:
.if ValidPE==TRUE
    invoke MessageBox, 0, addr FileValidPE, addr AppName,
    MB_OK+MB_ICONINFORMATION
.else
    invoke MessageBox, 0, addr FileInvalidPE, addr AppName,
    MB_OK+MB_ICONINFORMATION
.endif
```

Вышеприведенный код сам по себе очень прост. Он проверяет значение в ValidPE и отображает соответствующее сообщение.

```
push seh.PrevLink  
pop fs:[0]
```

Когда SEH больше не используется, мы убираем его из SEH-цепи.

PE. Урок 3. Файловый заголовок

Автор: lczelion, пер. Aquila

Дата: 2002-06-06

В этом tutorialе мы изучим файловый заголовок PE.

Давайте кратко повторим то, что мы уже изучили:

- DOS MZ-заголовок называется IMAGE_DOS_HEADER. Только два из его члена важны для нас: e_magic, который содержит строку "MZ" и e_lfanew, которая содержит файловое смещение PE-заголовка.
- Мы используем значение в e_magic, чтобы убедиться, что файл имеет правильный DOS заголовок, сравнивая его со значением IMAGE_DOS_SIGNATURE. Если оба значения совпадают, мы можем быть уверены, что файл имеет правильный DOS-заголовок.
- Чтобы перейти к PE-заголовку, мы должны передвинуть файловый указатель на смещение, указанное значением в e_lfanew.
- Первое слово PE-заголовка должно содержать строку "PE", за которым следуют два нуля. Мы сравниваем значение в этом двойном слове со значением IMAGE_NT_SIGNATURE. Если они оба совпадают, мы можем допустить, что PE-заголовок верен.

Мы изучим больше о PE-заголовке в этом tutorialе. Официальное название PE-заголовка - это IMAGE_NT_HEADERS. Чтобы освежить вашу память, я покажу ее ниже.

```
IMAGE_NT_HEADERS STRUCT
    Signature dd ?
    FileHeader IMAGE_FILE_HEADER <>
    OptionalHeader IMAGE_OPTIONAL_HEADER32 <>
IMAGE_NT_HEADERS ENDS
```

Signature - это PE-сигнатура, "PE" следующее за двумя нулями. Вы уже знаете и используете это параметр.

FileHeader - это структура, которая содержит информацию о физическом составе/свойствах PE-файла вообще.

OptionalHeader - это также структура, которая содержит информацию о логическом составе PE-файла.

Самая интересная часть - это OptionalHeader. Тем не менее, некоторые поля в FileHeader также важны. Мы изучим FileHeader в этом tutorialе, так что мы можем перейти к изучению OptionalHeader'a в следующих tutorialах.

```
IMAGE_FILE_HEADER STRUCT
    Machine WORD ?
    NumberOfSections WORD ?
    TimeDateStamp dd ?
    PointerToSymbolTable dd ?
    NumberOfSymbols dd ?
    SizeOfOptionalHeader WORD ?
    Characteristics WORD ?
IMAGE_FILE_HEADER ENDS
```

Имя поля	Значение
Machine	CPU платформа, для которой предназначен этот файл. Для платформы Intel это значение равно IMAGE_FILE_MACHINE_I386. Я попытался использовать 14Dh и 14Eh, упоминающиеся в pe.txt от LUEVELSMEYER, но Windows отказалась запустить ее. Это поле едва ли представляет для нас какой-либо интерес, кроме быстро пути не дать программе быть запущенной.

Имя поля	Значение
NumberOfSection	Количество секций в файле. Нам понадобится изменять данный параметр, если мы захотим добавить или убрать секцию из файла.
TimeDataStamp	Дата и время, когда был создан файл. Бесполезен для нас.
PointerToSymbolTable	используется для отладки.
NumberOfSymbols	используется для отладки.
SizeOfOptionalHeader	Размер параметра OptionalHeader'a, который следует непосредственно за этой структурой. Должен быть установлен в правильное значение.
Characteristics	Содержит флаги для файла, например является ли этот файл exe или dll.

В кратце, только три параметра более менее полезны для нас: Machine, NumberOfSections и Characteristics. Как правило, вы не должны изменять значения Machine и Characteristics, но вы должны использовать это значение NumberOfSections, когда переходите к таблице секций.

Я опережаю события, но для того, чтобы проиллюстрировать использование NumberOfSections, я должен немного отвлечься на таблицу секций.

Таблица секций - это массив структур. Каждая структура содержит информацию об секции. То есть, если файл содержит три секции, будет 3 члена в этом массиве. Вам нужно значение параметра NumberOfSections, чтобы вы знали как много членов в этом массиве. За этим массивом следует ряд нулей, и Windows, по-моему мнению, использует этот факт, чтобы определять конец данного массива структур. Попробуйте установить значение NumberOfSections выше, чем на самом деле - Windows по-прежнему будет способен работать с файлом. Почему же мы не можем игнорировать данный параметр? Существует несколько причин. Спецификация PE не указывает, что таблица секций должна кончаться структурой, состоящей из нулей. Может случиться ситуация, когда последний член массива вплотную прилегает к первой секции, без всякого свободного места. Другой причиной является импорты. При компоновке в новом стиле информация следует непосредственно за последним членом таблицы секций. Вот почему вам нужен NumberOfSections.

PE. Урок 4. Опциональный заголовок

Автор: lczelion, пер. Aquila

Дата: 2002-06-06

Мы изучили DOS-заголовок и некоторые члены PE-заголовка. Теперь перед нами последний, самый большой и, вероятно, самый важный член PE-заголовка - опциональный заголовок.

Чтобы освежить вашу память, напомним: опциональный заголовок - это структура, являющаяся последним членом IMAGE_NT_HEADERS. Она содержит информацию о логической компоновке PE-файла. В этой структуре 31 поле. Некоторые из них важны, некоторые бесполезны. Я объясню только те, которые действительно могут понадобиться.

Есть слово, которое часто используется по отношению к формату PE: RVA. RVA расшифровывается как относительный виртуальный адрес (relative virtual address). Вы знаете что такое виртуальный адрес. Сложное, на первый взгляд, словосочетание RVA служит для обозначения простой концепции. Просто представьте, что RVA - это расстояние до ссылающейся точки в виртуальном адресном пространстве. Я уверен, что вы знакомы с файловым смещением: RVA точно тоже самое, что и файловое смещение. Тем не менее, RVA задается относительно положения в виртуальном адресном пространстве, а не файла. Я покажу вам пример. Если PE-файл загружается по адресу 400000h в виртуальном адресном пространстве и программа начинает выполняться с адреса 401000h, мы можем сказать, что программа начинается с RVA 1000h. RVA определяется относительно стартового виртуального адреса модуля.

Почему RVA используется в PE-формате? Это помогает уменьшить время загрузки файла PE-загрузчиком. Так как модуль может находиться где угодно в виртуальном адресном пространстве, PE-загрузчику было бы очень трудно пофиксить адреса все перемещаемых объектов в модуле. С другой стороны, если все эти объекты используют RVA, PE-загрузчику нет надобности фиксировать чтобы то ни было: он просто перемещает весь модуль в новый стартовый виртуальный адрес. Это вроде концепции абсолютного и относительного путей: RVA схож с относительным путем, виртуальный адрес - с абсолютным.

РЕ. Урок 5. Таблица секций

Автор: lczelion, пер. Aquila

Дата: 2002-06-06

Скачайте пример (находится в архиве wasm_files.zip: files/recovered/1002005/pe-tut05.zip).

ТЕОРИЯ

Мы изучили DOS-заголовок и РЕ-заголовок. Осталась таблица секций. Таблица секций - это массив структур, следующий непосредственно за РЕ-заголовком. Размер массива определен в поле NumberOfSections структуры IMAGE_FILE_HEADER. Структура секции называется IMAGE_SECTION_HEADER.

```
IMAGE_SIZEOF_SHORT_NAME equ 8

IMAGE_SECTION_HEADER STRUCT
    Name1 db IMAGE_SIZEOF_SHORT_NAME dup(?)
    union Misc
        PhysicalAddress dd ?
        VirtualSize dd ?
    ends
    VirtualAddress dd ?
    SizeOfRawData dd ?
    PointerToRawData dd ?
    PointerToRelocations dd ?
    PointerToLinenumbers dd ?
    NumberOfRelocations dw ?
    NumberOfLinenumbers dw ?
    Characteristics dd ?
IMAGE_SECTION_HEADER ENDS
```

Как обычно, не все члены полезны. Я расскажу только о тех, которые действительно полезны.

Поле	Значение
Name1	Очевидно, что имя этого поля - "name", но так как слово "name" является ключевым словом в MASM'e, мы используем вместо него "Name1". Заметьте, что максимальная длина этого поля 8 байтов. Имя - это всего лишь название, ничего больше. Вы можете использовать любое имя или даже оставить это поле пустым. Заметьте, что null в конце может и не находиться.
VirtualAddress	RVA секции. РЕ-загрузчик использует значение в этом поле, когда мэппирует секцию в память. То есть, если значение в этом поле равняется 1000h и файл загружен в 400000h, секция будет загружена в 401000h.
SizeOfRawData	Размер секции, выравненный согласно установкам в РЕ-заголовке. РЕ-загрузчик проверяет значение в этом поле, чтобы знать, сколько байтов он должен загрузить в память.
PointerToRawData	Файловое смещение на начало секции. РЕ-загрузчик использует это значение для того, чтобы найти где она начинается.
Characteristics	Содержит флаги, такие как содержит ли эта секция исполняемый код, инициализированные данные, неинициализированные данные, можно читать и писать в секцию.

Теперь когда мы знаем о структуре IMAGE_SECTION_HEADER, давайте посмотрим, как мы можем эмулировать работу, выполняемую РЕ-загрузчиком:

- Прочитаем NumberOfSections в IMAGE_FILE_HEADER, чтобы узнать, сколько секций в файле.

- Используем значение в `SizeOfHeaders` в качестве файлового смещения таблицы секций и передвинем файловый указатель на это смещение.
- Проходим по массиву структур, изучая каждый элемент.
- Просматривая каждую структуру, мы получаем значение `PointerToRawData` и перемещаем файловый указатель на это смещение. Затем мы читаем значение в `SizeOfRawData`, чтобы узнать сколько байтов мы должны загрузить в память. Читаем значение в `VirtualAddress` и добавляем значение к `ImageBase`, чтобы получить виртуальный адрес секции, с которого ей следует начинаться. И тогда мы готовы промэппировать секцию в память и пометить атрибут памяти согласно флагам, указанным в `Characteristics`.
- Идем по массиву, пока все секции не будут обработаны.

Заметьте, что мы не используем имя секции: оно не необходимо.

ПРИМЕР

Этот пример открывает PE-файл и проходит по таблице секций, показывая информацию о секциях в `listview`-контроле.

```
.386
.model flat,stdcall
option casemap:none
include \masm32\include\windows.inc
include \masm32\include\kernel32.inc
include \masm32\include\comdlg32.inc
include \masm32\include\user32.inc
include \masm32\include\comctl32.inc
includelib \masm32\lib\comctl32.lib
includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib
includelib \masm32\lib\comdlg32.lib

IDD_SECTIONTABLE equ 104
IDC_SECTIONLIST equ 1001

SEH struct
PrevLink dd ? ; адрес предыдущей seh-структуры
CurrentHandler dd ? ; адрес нового обработчика исключения
SafeOffset dd ? ; смещение с которого безопасно продолжить выполнение программы
PrevEsp dd ? ; старое значение esp
PrevEbp dd ? ; старое значение ebp
SEH ends

.data
AppName db "PE tutorial no.5",0
ofn OPENFILENAME <>
FilterString db "Executable Files (*.exe, *.dll)",0,"*.exe;*.dll",0
db "All Files",0,"*.*",0,0
FileOpenError db "Cannot open the file for reading",0
FileOpenMappingError db "Cannot open the file for memory mapping",0
FileMappingError db "Cannot map the file into memory",0
FileInvalidPE db "This file is not a valid PE",0
template db "%08lx",0
SectionName db "Section",0
VirtualSize db "V.Size",0
VirtualAddress db "V.Address",0
SizeOfRawData db "Raw Size",0
RawOffset db "Raw Offset",0
Characteristics db "Characteristics",0

.data?
hInstance dd ?
buffer db 512 dup(?)
hFile dd ?
hMapping dd ?
pMapping dd ?
ValidPE dd ?
NumberOfSections dd ?
```

```

.code
start proc
LOCAL seh:SEH
    invoke GetModuleHandle,NULL
    mov hInstance,eax
    mov ofn.lStructSize,SIZEOF ofn
    mov ofn.lpstrFilter, OFFSET FilterString
    mov ofn.lpstrFile, OFFSET buffer
    mov ofn.nMaxFile,512
    mov ofn.Flags, OFN_FILEMUSTEXIST or OFN_PATHMUSTEXIST or OFN_LONGNAMES
    or OFN_EXPLORER or OFN_HIDEREADONLY
    invoke GetOpenFileName, ADDR ofn
    .if eax==TRUE
        invoke CreateFile, addr buffer, GENERIC_READ,
            FILE_SHARE_READ, NULL, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL
        .if eax!=INVALID_HANDLE_VALUE
            mov hFile, eax
            invoke CreateFileMapping, hFile, NULL, PAGE_READONLY,0,0,0
            .if eax!=NULL
                mov hMapping, eax
                invoke MapViewOfFile,hMapping,FILE_MAP_READ,0,0,0
                .if eax!=NULL
                    mov pMapping,eax
                    assume fs:nothing
                    push fs:[0]
                    pop seh.PrevLink
                    mov seh.CurrentHandler,offset SEHHandler
                    mov seh.SafeOffset,offset FinalExit
                    lea eax,seh
                    mov fs:[0], eax
                    mov seh.PrevEsp,esp
                    mov seh.PrevEbp,ebp
                    mov edi, pMapping
                    assume edi:ptr IMAGE_DOS_HEADER
                    .if [edi].e_magic==IMAGE_DOS_SIGNATURE
                        add edi, [edi].e_lfanew
                        assume edi:ptr IMAGE_NT_HEADERS
                        .if [edi].Signature==IMAGE_NT_SIGNATURE
                            mov ValidPE, TRUE
                        .else
                            mov ValidPE, FALSE
                        .endif
                    .else
                        mov ValidPE,FALSE
                    .endif
                .endif
            .endif
        .endif
    .endif
FinalExit:
    push seh.PrevLink
    pop fs:[0]
    .if ValidPE==TRUE
        call ShowSectionInfo
    .else
        invoke MessageBox, 0, addr FileInvalidPE, addr AppName,
            MB_OK+MB_ICONINFORMATION
    .endif
    invoke UnmapViewOfFile, pMapping
    .else
        invoke MessageBox, 0, addr FileMappingError, addr AppName,
            MB_OK+MB_ICONERROR
    .endif
    invoke CloseHandle,hMapping
    .else
        invoke MessageBox, 0, addr FileOpenMappingError, addr AppName,
            MB_OK+MB_ICONERROR
    .endif
    invoke CloseHandle, hFile
    .else
        invoke MessageBox, 0, addr FileOpenError, addr AppName, MB_OK+MB_ICONERROR
    .endif
    .endif
    invoke ExitProcess, 0
    invoke InitCommonControls

```

```

start endp

SEHHandler proc uses edx
pExcept:DWORD,pFrame:DWORD,pContext:DWORD,pDispatch:DWORD
    mov edx,pFrame
    assume edx:ptr SEH
    mov eax,pContext
    assume eax:ptr CONTEXT
    push [edx].SafeOffset
    pop [eax].regEip
    push [edx].PrevEsp
    pop [eax].regEsp
    push [edx].PrevEbp
    pop [eax].regEbp
    mov ValidPE, FALSE
    mov eax,ExceptionContinueExecution
    ret
SEHHandler endp

DlgProc proc uses edi esi hDlg:DWORD, uMsg:DWORD, wParam:DWORD,
lParam:DWORD
    LOCAL lvc:LV_COLUMN
    LOCAL lvi:LV_ITEM
    .if uMsg==WM_INITDIALOG
        mov esi, lParam
        mov lvc.imask,LVCF_FMT or LVCF_TEXT or LVCF_WIDTH or LVCF_SUBITEM
        mov lvc.fmt,LVCFMT_LEFT
        mov lvc.lx,80
        mov lvc.iSubItem,0
        mov lvc.pszText,offset SectionName
        invoke
SendDlgItemMessage,hDlg,IDC_SECTIONLIST,LVM_INSERTCOLUMN,0,addr lvc
        inc lvc.iSubItem
        mov lvc.fmt,LVCFMT_RIGHT
        mov lvc.pszText,offset VirtualSize
        invoke
SendDlgItemMessage,hDlg,IDC_SECTIONLIST,LVM_INSERTCOLUMN,1,addr lvc
        inc lvc.iSubItem
        mov lvc.pszText,offset VirtualAddress
        invoke
SendDlgItemMessage,hDlg,IDC_SECTIONLIST,LVM_INSERTCOLUMN,2,addr lvc
        inc lvc.iSubItem
        mov lvc.pszText,offset SizeOfRawData
        invoke
SendDlgItemMessage,hDlg,IDC_SECTIONLIST,LVM_INSERTCOLUMN,3,addr lvc
        inc lvc.iSubItem
        mov lvc.pszText,offset RawOffset
        invoke
SendDlgItemMessage,hDlg,IDC_SECTIONLIST,LVM_INSERTCOLUMN,4,addr lvc
        inc lvc.iSubItem
        mov lvc.pszText,offset Characteristics
        invoke
SendDlgItemMessage,hDlg,IDC_SECTIONLIST,LVM_INSERTCOLUMN,5,addr lvc
        mov ax, NumberOfSections
        movzx eax,ax
        mov edi,eax
        mov lvi.imask,LVIF_TEXT
        mov lvi.iItem,0
        assume esi:ptr IMAGE_SECTION_HEADER
        .while edi>0
            mov lvi.iSubItem,0
            invoke RtlZeroMemory,addr buffer,9
            invoke lstrcpy,addr buffer,addr [esi].Name1,8
            lea eax,buffer
            mov lvi.pszText,eax
            invoke
SendDlgItemMessage,hDlg,IDC_SECTIONLIST,LVM_INSERTITEM,0,addr lvi
            invoke wsprintf,addr buffer,addr template,[esi].Misc.VirtualSize
            lea eax,buffer
            mov lvi.pszText,eax
            inc lvi.iSubItem
    
```

```

        invoke SendDlgItemMessage,hDlg,IDC_SECTIONLIST,LVM_SETITEM,0,addr lvi
        invoke wsprintf,addr buffer,addr template,[esi].VirtualAddress
        lea eax,buffer
        mov lvi.pszText,eax
        inc lvi.iSubItem
        invoke SendDlgItemMessage,hDlg,IDC_SECTIONLIST,LVM_SETITEM,0,addr lvi
        invoke wsprintf,addr buffer,addr template,[esi].SizeOfRawData
        lea eax,buffer
        mov lvi.pszText,eax
        inc lvi.iSubItem
        invoke SendDlgItemMessage,hDlg,IDC_SECTIONLIST,LVM_SETITEM,0,addr lvi
        invoke wsprintf,addr buffer,addr template,[esi].PointerToRawData
        lea eax,buffer
        mov lvi.pszText,eax
        inc lvi.iSubItem
        invoke SendDlgItemMessage,hDlg,IDC_SECTIONLIST,LVM_SETITEM,0,addr lvi
        invoke wsprintf,addr buffer,addr template,[esi].Characteristics
        lea eax,buffer
        mov lvi.pszText,eax
        inc lvi.iSubItem
        invoke SendDlgItemMessage,hDlg,IDC_SECTIONLIST,LVM_SETITEM,0,addr lvi
        inc lvi.iItem
        dec edi
        add esi, sizeof IMAGE_SECTION_HEADER
    .endw
.elseif
    uMsg==WM_CLOSE
        invoke EndDialog,hDlg,NULL
.else
    mov eax,FALSE
    ret
.endif
mov eax,TRUE
ret
DlgProc endp

ShowSectionInfo proc uses edi
    mov edi, pMapping
    assume edi:ptr IMAGE_DOS_HEADER
    add edi, [edi].e_lfanew
    assume edi:ptr IMAGE_NT_HEADERS
    mov ax,[edi].FileHeader.NumberOfSections
    movzx eax,ax
    mov NumberOfSections,eax
    add edi,sizeof IMAGE_NT_HEADERS
    invoke DialogBoxParam, hInstance, IDD_SECTIONTABLE,NULL, addr DlgProc, edi
    ret
ShowSectionInfo endp
end start

```

АНАЛИЗ

В этом примере частично используется код примера ко второму tutorialу. После этого проверяется, является ли файл верным PE, а затем вызывается функция ShowSectionInfo.

```

ShowSectionInfo proc uses edi
    mov edi, pMapping
    assume edi:ptr IMAGE_DOS_HEADER
    add edi, [edi].e_lfanew
    assume edi:ptr IMAGE_NT_HEADERS

```

Мы используем edi в качестве указателя на данные в PE-файле. Во-первых, мы присваиваем ему значение pMapping, которое является адресом DOS-заголовка. Затем мы добавляем к нему значение e_lfanew - теперь edi содержит адрес PE-заголовка.

```

    mov ax,[edi].FileHeader.NumberOfSections
    mov NumberOfSections,ax

```

Так как нам нужно перейти к таблице секций, мы должны получить количество секций в этом файле. Это значение параметра NumberOfSections в заголовке файла. Не забудьте, что этот

параметр размером в слово.

```
add edi, sizeof IMAGE_NT_HEADERS
```

Сейчас edi содержит адрес PE-заголовка. Последний мы добавляем к первому, и в результате edi будет содержать адрес таблицы секций.

```
invoke DialogBoxParam, hInstance, IDD_SECTIONTABLE, NULL, addr DlgProc, edi
```

Вызываем DialogBoxParam, чтобы показать диалоговое окно, содержащее listview-контрол. Отметим, что мы передаем адрес таблицы секций в качестве последнего параметра. Это значение будет доступно в lParam во время обработки сообщения WM_INITDIALOG.

Во время этого мы сохраняем значение lParam (адрес таблицы секций) в esi, номер секций в edi, а затем выводим listview-контрол. Когда все готово, мы входим в цикл, который должен вставить информацию о каждой секции в этот контрол. Эта часть очень проста.

```
.while edi>0
    mov lvi.iSubItem, 0
```

Помещаем эту строку в первую колонку.

```
invoke RtlZeroMemory, addr buffer, 9
invoke lstrcpy, addr buffer, addr [esi].Name1, 8
lea eax, buffer
mov lvi.pszText, eax
```

Мы отобразим имя секции, но перед этим мы должны отконвертировать его в ASCIIZ-строку.

```
invoke SendDlgItemMessage, hDlg, IDC_SECTIONLIST, LVM_INSERTITEM, 0, addr lvi
```

Затем мы отобразим его в первой колонке.

Мы продолжаем действовать по этой схеме, пока не выведем последнее значение. Тогда мы должны переходить к следующей структуре.

```
dec edi
add esi, sizeof IMAGE_SECTION_HEADER
.endw
```

Мы понижаем значение в edi после обработки каждой из секций. И добавляем размер IMAGE_SECTION_HEADER к esi, что он содержал адрес следующей структуры IMAGE_SECTION_HEADER.

Вот, что необходимо сделать, чтобы обработать таблицу секций:

- Убедиться, что файл является верным PE.
- Перейти к началу PE-заголовка.
- Получить количество секций из поля NumberOfSections в файловом заголовке.
- Перейти к таблице секций либо добавив ImageBase к SizeOfHeaders или добавив адрес PE-заголовка к размеру DOS-заголовка (таблица секций идет непосредственно за PE-заголовком). Если вы не хотите использовать file mapping, вам нужно переместить файловый указатель на таблицу секций посредством SetFilePointer. Файловое смещение таблицы секций находится в SizeOfHeaders (это один из параметров IMAGE_OPTIONAL_HEADER).
- Обработать каждую структуру IMAGE_SECTION_HEADER.

РЕ. Урок 6. Таблица импорта

Автор: lczelion, пер. Aquila

Дата: 2002-06-06

В этом tutorialе мы изучим таблицу импорта. Сначала я вас должен предупредить: этот tutorial довольно долгий и сложный для тех, кто не знаком с таблицей импорта. Вам может потребоваться перечитать данное руководство несколько раз и даже проанализировать затрагиваемые здесь структуры под дебаггером.

Скачайте пример (находится в архиве `wasm_files.zip`: `files/article41/pe-tut06.zip`).

ТЕОРИЯ

Прежде всего, вам следует знать, что такое импортируемая функция. Импортируемая функция - это функция, которая находится не в модуле вызывающего, но вызывается им, поэтому употребляется слово "импорт". Функции импорта физически находятся в одной или более DLL. В модуле вызывающего находится только информация о функциях. Эта информация включает имена функций и имена DLL, в которых они находятся.

Как мы можем узнать, где находится эта информация в РЕ-файле? Мы должны обратиться за ответом к директории данных. Я освежу вашу память. Вот РЕ-заголовок:

```
IMAGE_NT_HEADERS STRUCT
    Signature dd ?
    FileHeader IMAGE_FILE_HEADER <>
    OptionalHeader IMAGE_OPTIONAL_HEADER <>
IMAGE_NT_HEADERS ENDS
```

Последний член опционального заголовка - это директория данных:

```
IMAGE_OPTIONAL_HEADER32 STRUCT
    ....
    LoaderFlags dd ?
    NumberOfRvaAndSizes dd ?
    DataDirectory IMAGE_DATA_DIRECTORY 16 dup(<>)
IMAGE_OPTIONAL_HEADER32 ENDS
```

Директория данных - это массив структур `IMAGE_DATA_DIRECTORY`. Всего 16 членов. Если вы помните, таблица секций - это корневая директория секций РЕ-файлов, вы можете также думать о директории данных как о корневой директории логических компонентов, сохраненных внутри этих секций. Чтобы быть точным, директория данных содержит местонахождение и размеры важных структур данных РЕ-файла. Каждый параметр содержит информацию о важной структуре данных.

Параметр	Информация
0	Символы экспорта
1	Символы импорта
2	Ресурсы
3	Исключение
4	Безопасность
5	Base relocation
6	Отладка

Параметр	Информация
7	Строка копирайта
8	Unknown
9	Thread local storage (TLS)
10	Загрузочная информация
11	Bound Import
12	Таблица адресов импорта
13	Delay Import
14	COM descriptor

Теперь, когда вы знаете, что содержит каждый из членов директории данных, мы можем изучить их поподробнее. Каждый из элементов директории данных - это структура `IMAGE_DATA_DIRECTORY`, которая имеет следующее определение:

```
IMAGE_DATA_DIRECTORY STRUCT
    VirtualAddress dd ?
    isize dd ?
IMAGE_DATA_DIRECTORY ENDS
```

`VirtualAddress` - это относительный виртуальный адрес (RVA) структуры данных. Например, если эта структура для символов импорта, это поле содержит RVA массива `IMAGE_IMPORT_DESCRIPTOR`.

`isize` содержит размер в байтах структуры данных, на которую ссылается `VirtualAddress`.

Это главная схема по нахождению важной структуры данных в PE-файле:

- От DOS-заголовка вы переходите к PE-заголовку
- Получаете адрес директории данных в опциональном заголовке.
- Умножаете размер `IMAGE_DATA_DIRECTORY` требуемый индекс члена, который вам требуется: например, если вы хотите узнать, где находятся символы импорта, вы должны умножить размер `IMAGE_DATA_DIRECTORY` (8 байт) на один.
- Добавляете результат к адресу директории данных, и теперь у вас есть адрес структуры `IMAGE_DATA_DIRECTORY`, которая содержит информацию о желаемой структуре данных.

Теперь мы начнем обсуждение собственно таблицы импорта. Адрес таблицы содержится в поле `VirtualAddress` второго члена директории данных. Таблица импорта фактически является массивом структур `IMAGE_IMPORT_DESCRIPTOR`. Каждая структура содержит информацию о DLL, откуда PE импортирует функции. Например, если PE импортирует функции из 10 разных DLL, этот массив будет состоять из 10 элементов. Конец массива отмечается элементом, содержащим одни нули. Теперь мы можем подробно проанализировать структуру:

```
IMAGE_IMPORT_DESCRIPTOR STRUCT
    union
        Characteristics dd ?
        OriginalFirstThunk dd ?
    ends
    TimeDateStamp dd ?
    ForwarderChain dd ?
    Name1 dd ?
    FirstThunk dd ?
IMAGE_IMPORT_DESCRIPTOR ENDS
```

Первый член этой структуры - объединение. Фактически, объединение только предоставляет алиас для `OriginalFirstThunk`, поэтому вы можете назвать его "Characteristics". Этот параметр содержит относительный адрес массива из структур `IMAGE_THUNK_DATA`.

Что такое IMAGE_THUNK_DATA? Это объединение размером в двойное слово. Обычно мы используем его как указатель на структуру IMAGE_IMPORT_BY_NAME. Заметьте, что IMAGE_THUNK_DATA содержит указатели на структуру IMAGE_IMPORT_BY_NAME, а не саму структуру.

Воспринимайте это следующим образом: есть несколько структур IMAGE_IMPORT_BY_NAME. Мы собираем RVA этих структур (IMAGE_THUNK_DATA'ы) в один массив и прерываем его нулем. Затем мы помещаем RVA массива в OriginalFirstThunk.

Структура IMAGE_IMPORT_BY_NAME содержит информацию о функции импорта. Теперь давайте посмотрим, как выглядит структура IMAGE_IMPORT_BY_NAME.

```
IMAGE_IMPORT_BY_NAME STRUCT
    Hint dw ?
    Name1 db ?
IMAGE_IMPORT_BY_NAME ENDS
```

Hint содержит соответствующего индекса таблицы экспорта DLL, в которой находится функция. Это поле создано для использования PE-загрузчиком, чтобы он мог быстро найти функцию в таблице экспорта. Это значение не является необходимым и некоторые линкеры могут устанавливать значение этого поля равным нулю.

Name1 содержит имя импортируемой функции в формате ASCIIZ. Хотя этот параметр определен как байт, на самом деле он переменного размера. Так было сделано лишь потому, что нельзя представить в структуре поле переменного размера. Структура была определена для того, чтобы вы могли обращаться к данным через описательные имена.

О TimeDateStamp и ForwarderChain мы поговорим позже, когда разберем остальные параметры.

Name1 содержит RVA имени DLL, то есть, указатель на ее имя. Это строка в формате ASCIIZ.

FirstThunk очень похожа на OriginalFirstThunk, то есть, он содержит RVA массива из структур IMAGE_THUNK_DATA (хотя это другой массив).

Если вы все еще смущены, посмотрите на это так: есть несколько структур IMAGE_IMPORT_BY_NAME. Вы создаете два массива, затем заполняете их RVA'ми этих структур, так что оба массива будут содержать абсолютно одинаковые значения (то есть, будут дублировать друг друга). Теперь вы можете присвоить RVA первого массива OriginalFirstThunk'у и RVA второго массива FirstThunk'у.

OriginalFirstThunk	IMAGE_IMPORT_BY_NAME	FirstThunk
IMAGE_THUNK_DATA --->	Function 1	<--- IMAGE_THUNK_DATA
IMAGE_THUNK_DATA --->	Function 2	<--- IMAGE_THUNK_DATA
IMAGE_THUNK_DATA --->	Function 3	<--- IMAGE_THUNK_DATA
IMAGE_THUNK_DATA --->	Function 4	<--- IMAGE_THUNK_DATA
... --->	...	<--- ...
IMAGE_THUNK_DATA --->	Function n	<--- IMAGE_THUNK_DATA

Теперь вы должны понять, что я имею ввиду. Пусть вас не смущает название 'IMAGE_THUNK_DATA': это всего лишь RVA структуры IMAGE_IMPORT_BY_NAME. Если вы мысленно замените слово IMAGE_THUNK_DATA на RVA, вы поймете это. Количество элементов в массиве OriginalFirstThunk и FirstThunk зависит от количества функций, импортируемых PE из DLL. Например, если PE-файл импортирует 10 функций из kernel32.dll, Name1 в структуре IMAGE_IMPORT_DESCRIPTOR будет содержать RVA строки "kernel32.dll" и в каждом массиве будет 10 IMAGE_THUNK_DATA.

Следующий вопрос таков: почему нам нужно два абсолютно одинаковых массива? Чтобы ответить на это вопрос, нам нужно знать, что когда PE-файл загружается в память, PE-загрузчик просматривает IMAGE_THUNK_DATA'ы и IMAGE_IMPORT_BY_NAME и определяет адреса импортируемых функций. Затем он замещает IMAGE_THUNK_DATA'ы в массиве, на который

ссылается FirstThunk настоящими адресами функций. Поэтому когда PE-файл готов к запуску, вышеприведенная картина становится такой:

OriginalFirstThunk	IMAGE_IMPORT_BY_NAME	FirstThunk
IMAGE_THUNK_DATA --->	Function 1	Address of Function 1
IMAGE_THUNK_DATA --->	Function 2	Address of Function 2
IMAGE_THUNK_DATA --->	Function 3	Address of Function 3
IMAGE_THUNK_DATA --->	Function 4	Address of Function 4
... --->	...	...
IMAGE_THUNK_DATA --->	Function n	Address of Function n

Массив RVA'ов, на который ссылется OriginalFirstThunk остается прежним, так что если возникает нужда найти имена функций импорта, PE-загрузчик сможет их найти.

Надо сказать, что некоторые функции экспортируются через ординалы, то есть не по имени, а по их позиции. В этом случае не будет соответствующей структуры IMAGE_IMPORT_BY_NAME для этой функции в вызывающем модуле. Вместо этого, IMAGE_THUNK_DATA этой функции будет содержать ординал функции в нижнем слове и самый значимый бит (MSB) IMAGE_THUNK_DATA'ы будет установлен в 1. Например, если функция экспортируется только через ординал и тот равен 1234h, IMAGE_THUNK_DATA этой функции будет содержать 80001234h. Микрософт предоставляет константу для проверки MSB, IMAGE_ORDINAL_FLAG32. Он имеет значение 80000000h.

Представьте, что мы хотим создать список BCEX импортируемых функций PE-файла. Для этого нам потребуется сделать следующие шаги.

- Убедиться, что файл является Portable Executable
- От DOS-заголовка перейти к PE-заголовку
- Получить адрес директории данных в OptionalHeader
- Перейти ко второму элементу директории данных. Извлечь значение VirtualAddress
- Использовать это значение, чтобы перейти к первой структуре IMAGE_IMPORT_DESCRIPTOR
- Проверьте значение OriginalFirstThunk. Если оно не равно нулю, следуйте RVA в OriginalFirstThunk, чтобы перейти к RVA-массиву. Если OriginalFirstThunk равен нулю, используйте вместо него значение FirstThunk. Некоторые линкеры генерируют PE-файлы с 0 в OriginalFirstThunk. Это считается багом. Только для того, чтобы подстраховаться, мы сначала проверяем значение OriginalFirstThunk.
- Мы сравниваем значение каждого элемента массива с IMAGE_ORDINAL_FLAG32. Если MSB равен единице, значит функция экспортируется через ординал и мы можем получить его из нижнего слова элемента.
- Если MSB равен нулю, используйте значение элемента как RVA на IMAGE_IMPORT_BY_NAME, пропустите Hint, и вы у имени функции.
- Перейдите к следующему элементу массива и извлекайте имена пока не будет достигнут конец массива (он кончается null'ом). Сейчас мы получили имена функций, импортированных из данной DLL. Переходим к следующей DLL.
- Перейдите к следующему IMAGE_IMPORT_DESCRIPTOR'у и обработайте его. Делайте это, пока не обработаете весь массив (массив IMAGE_IMPORT_DESCRIPTOR кончается элементом с полями, заполненными нулями).

ПРИМЕР

Этот пример открывает PE-файл и отображает имена всех импортируемых функций в edit control'е. Также он показывает значения в структурах IMAGE_IMPORT_DESCRIPTOR.

```

.386
.model flat,stdcall
option casemap:none
include \masm32\include\windows.inc
include \masm32\include\kernel32.inc
include \masm32\include\comdlg32.inc
include \masm32\include\user32.inc
includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib
includelib \masm32\lib\comdlg32.lib

IDD_MAINDLG equ 101
IDC_EDIT equ 1000
IDM_OPEN equ 40001
IDM_EXIT equ 40003

DlgProc proto :DWORD,:DWORD,:DWORD,:DWORD
ShowImportFunctions proto :DWORD
ShowTheFunctions proto :DWORD,:DWORD
AppendText proto :DWORD,:DWORD

SEH struct
PrevLink dd ? ; адрес предыдущей seh-структуры
CurrentHandler dd ? ; адрес нового обработчика исключений
SafeOffset dd ? ; смещение, по которому безопасно выполнять выполнения
PrevEsp dd ? ; старое значение esp
PrevEbp dd ? ; старое значение ebp
SEH ends

.data
AppName db "PE tutorial no.6",0
ofn OPENFILENAME <>
FilterString db "Executable Files (*.exe, *.dll)",0,"*.exe;*.dll",0
db "All Files",0,"*.*",0,0
FileOpenError db "Cannot open the file for reading",0
FileOpenMappingError db "Cannot open the file for memory mapping",0
FileMappingError db "Cannot map the file into memory",0
NotValidPE db "This file is not a valid PE",0
CRLF db 0Dh,0Ah,0
ImportDescriptor db 0Dh,0Ah,"===== [ IMAGE_IMPORT_DESCRIPTOR
]===== ",0
IDTemplate db "OriginalFirstThunk = %1X",0Dh,0Ah
db "TimeDateStamp = %1X",0Dh,0Ah
db "ForwarderChain = %1X",0Dh,0Ah
db "Name = %s",0Dh,0Ah
db "FirstThunk = %1X",0
NameHeader db 0Dh,0Ah,"Hint Function",0Dh,0Ah
db "-----",0
NameTemplate db "%u %s",0
OrdinalTemplate db "%u (ord.)",0

.data?
buffer db 512 dup(?)
hFile dd ?
hMapping dd ?
pMapping dd ?
ValidPE dd ?

.code
start:
invoke GetModuleHandle,NULL
invoke DialogBoxParam, eax, IDD_MAINDLG,NULL,addr DlgProc, 0
invoke ExitProcess, 0

DlgProc proc hDlg:DWORD, uMsg:DWORD, wParam:DWORD, lParam:DWORD
.if uMsg==WM_INITDIALOG
invoke SendDlgItemMessage,hDlg,IDC_EDIT,EM_SETLIMITTEXT,0,0
.elseif uMsg==WM_CLOSE
invoke EndDialog,hDlg,0
.elseif uMsg==WM_COMMAND
.if lParam==0

```

```

    mov eax,wParam
    .if ax==IDM_OPEN
        invoke ShowImportFunctions,hDlg
    .else ; IDM_EXIT
        invoke SendMessage,hDlg,WM_CLOSE,0,0
    .endif
    .endif
    .else
        mov eax,FALSE
        ret
    .endif
    mov eax,TRUE
    ret
DlgProc endp

```

```

SEHHandler proc uses edx pExcept:DWORD, pFrame:DWORD, pContext:DWORD,
pDispatch:DWORD
    mov edx,pFrame
    assume edx:ptr SEH
    mov eax,pContext
    assume eax:ptr CONTEXT
    push [edx].SafeOffset
    pop [eax].regEip
    push [edx].PrevEsp
    pop [eax].regEsp
    push [edx].PrevEbp
    pop [eax].regEbp
    mov ValidPE, FALSE
    mov eax,ExceptionContinueExecution
    ret
SEHHandler endp

```

```

ShowImportFunctions proc uses edi hDlg:DWORD
    LOCAL seh:SEH
    mov ofn.lStructSize,SIZEOF
ofn mov ofn.lpstrFilter, OFFSET FilterString
    mov ofn.lpstrFile, OFFSET buffer
    mov ofn.nMaxFile,512
    mov ofn.Flags, OFN_FILEMUSTEXIST or OFN_PATHMUSTEXIST or OFN_LONGNAMES
or OFN_EXPLORER or OFN_HIDEREADONLY
    invoke GetOpenFileName, ADDR ofn
    .if eax==TRUE
        invoke CreateFile, addr buffer, GENERIC_READ, FILE_SHARE_READ, NULL,
OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL
        .if eax!=INVALID_HANDLE_VALUE
            mov hFile, eax
            invoke CreateFileMapping, hFile, NULL, PAGE_READONLY,0,0,0
            .if eax!=NULL
                mov hMapping, eax
                invoke MapViewOfFile,hMapping,FILE_MAP_READ,0,0,0
                .if eax!=NULL
                    mov pMapping,eax
                    assume fs:nothing
                    push fs:[0]
                    pop seh.PrevLink
                    mov seh.CurrentHandler,offset SEHHandler
                    mov seh.SafeOffset,offset FinalExit
                    lea eax,seh
                    mov fs:[0], eax
                    mov seh.PrevEsp,esp
                    mov seh.PrevEbp,ebp
                    mov edi, pMapping
                    assume edi:ptr IMAGE_DOS_HEADER
                    .if [edi].e_magic==IMAGE_DOS_SIGNATURE
                        add edi, [edi].e_lfanew
                        assume edi:ptr IMAGE_NT_HEADERS
                        .if [edi].Signature==IMAGE_NT_SIGNATURE
                            mov ValidPE, TRUE
                        .else
                            mov ValidPE, FALSE
                        .endif
                    .endif
                .endif
            .endif
        .endif
    .endif
ShowImportFunctions endp

```

```

        .else
            mov ValidPE,FALSE
        .endif
FinalExit:
    push seh.PrevLink
    pop fs:[0]
    .if ValidPE==TRUE
        invoke ShowTheFunctions, hDlg, edi
    .else
        invoke MessageBox,0, addr NotValidPE, addr AppName,
MB_OK+MB_ICONERROR
    .endif
    invoke UnmapViewOfFile, pMapping
    .else
        invoke MessageBox, 0, addr FileMappingError, addr AppName,
MB_OK+MB_ICONERROR
    .endif
    invoke CloseHandle,hMapping
    .else
        invoke MessageBox, 0, addr FileOpenMappingError, addr AppName,
MB_OK+MB_ICONERROR
    .endif
    invoke CloseHandle, hFile
    .else
        invoke MessageBox, 0, addr FileOpenError, addr AppName,
MB_OK+MB_ICONERROR
    .endif
    .endif
    .endif
    ret
ShowImportFunctions endp

```

```

AppendText proc hDlg:DWORD,pText:DWORD
    invoke SendDlgItemMessage,hDlg,IDC_EDIT,EM_REPLACESEL,0,pText
    invoke SendDlgItemMessage,hDlg,IDC_EDIT,EM_REPLACESEL,0,addr CRLF
    invoke SendDlgItemMessage,hDlg,IDC_EDIT,EM_SETSEL,-1,0
    ret
AppendText endp

```

RVAToOffset PROC uses edi esi edx ecx pFileMap:DWORD,RVA:DWORD

```

    mov esi,pFileMap
    assume esi:ptr IMAGE_DOS_HEADER
    add esi,[esi].e_lfanew
    assume esi:ptr IMAGE_NT_HEADERS
    mov edi,RVA ; edi == RVA
    mov edx,esi
    add edx,sizeof IMAGE_NT_HEADERS
    mov cx,[esi].FileHeader.NumberOfSections
    movzx ecx,cx
    assume edx:ptr IMAGE_SECTION_HEADER
    .while ecx>0 ; check all sections
        .if edi>=[edx].VirtualAddress
            mov eax,[edx].VirtualAddress
            add eax,[edx].SizeOfRawData
            .if edi < eax ; The address is in this section
                mov eax,[edx].VirtualAddress
                sub edi,eax
                mov eax,[edx].PointerToRawData
                add eax,edi ; eax == file offset
                ret
            .endif
        .endif
        add edx,sizeof IMAGE_SECTION_HEADER
        dec ecx
    .endw
    assume edx:nothing
    assume esi:nothing
    mov eax,edi
    ret
RVAToOffset endp

```

ShowTheFunctions proc uses esi ecx ebx hDlg:DWORD, pNTHdr:DWORD

```

LOCAL temp[512]:BYTE
invoke SetDlgItemText,hDlg,IDC_EDIT,0
invoke AppendText,hDlg,addr buffer
mov edi,pNTHdr
assume edi:ptr IMAGE_NT_HEADERS
mov edi,[edi].OptionalHeader.DataDirectory[ sizeof
IMAGE_DATA_DIRECTORY].VirtualAddress
invoke RVAToOffset,pMapping,edi
mov edi,eax
add edi,pMapping
assume edi:ptr IMAGE_IMPORT_DESCRIPTOR
.while !([edi].OriginalFirstThunk==0 && [edi].TimeDateStamp==0 &&
[edi].ForwarderChain==0 && [edi].Name1==0 && [edi].FirstThunk==0)
    invoke AppendText,hDlg,addr ImportDescriptor
    invoke RVAToOffset,pMapping,[edi].Name1
    mov edx,eax
    add edx,pMapping
    invoke wsprintf,addr temp,addr IDTemplate,
[edi].OriginalFirstThunk,[edi].TimeDateStamp,[edi].ForwarderChain,edx,[edi].F
    invoke AppendText,hDlg,addr temp
    .if [edi].OriginalFirstThunk==0
        mov esi,[edi].FirstThunk
    .else
        mov esi,[edi].OriginalFirstThunk
    .endif
    invoke RVAToOffset,pMapping,esi
    add eax,pMapping
    mov esi,eax
    invoke AppendText,hDlg,addr NameHeader
    .while dword ptr [esi]!=0
        test dword ptr [esi],IMAGE_ORDINAL_FLAG32
        jnz ImportByOrdinal
        invoke RVAToOffset,pMapping,dword ptr [esi]
        mov edx,eax
        add edx,pMapping
        assume edx:ptr IMAGE_IMPORT_BY_NAME
        mov cx,[edx].Hint
        movzx ecx,cx
        invoke wsprintf,addr temp,addr NameTemplate,ecx,addr [edx].Name1
        jmp ShowTheText
ImportByOrdinal:
    mov edx,dword ptr [esi]
    and edx,0FFFFh
    invoke wsprintf,addr temp,addr OrdinalTemplate,edx
ShowTheText:
    invoke AppendText,hDlg,addr temp
    add esi,4
    .endw
    add edi,sizeof IMAGE_IMPORT_DESCRIPTOR
    .endw
    ret
ShowTheFunctions endp
end start

```

АНАЛИЗ

Программа показывает диалоговое окно открытия файла, когда пользователь выбирает Open в меню. Она проверяет, является ли файл верным PE и затем вызывает ShowTheFunctions.

```

ShowTheFunctions proc uses esi ecx ebx hDlg:DWORD, pNTHdr:DWORD
    LOCAL temp[512]:BYTE

```

Резервируем 512 байтов стекового пространства для операций со строками.

```

    invoke SetDlgItemText,hDlg,IDC_EDIT,0

```

Очищаем edit control

```
invoke AppendText,hDlg,addr buffer
```

Вставьте имя PE-файла в edit control. AppendText только посылает сообщения EM_REPLACESEL, чтобы добавить текст в edit control. Заметьте, что он посылает EM_SETSEL с wParam = -1 и lParam = 0 edit control'у, чтобы сдвинуть курсор к концу текста.

```
mov edi,pNTHdr
assume edi:ptr IMAGE_NT_HEADERS
mov edi, [edi].OptionalHeader.DataDirectory[sizeof
IMAGE_DATA_DIRECTORY].VirtualAddress
```

Получаем RVA символов импорта. Сначала edi указывает на PE-заголовок. Мы используем его, чтобы перейти ко 2nd члену директории данных и получить значение параметра VirtualAddress.

```
invoke RVAToOffset,pMapping,edi
mov edi,eax
add edi,pMapping
```

Здесь скрывается одна из ловушек для новичков PE-программирования. Большинство из адресов в PE-файле - это RVA и RVA имеют значение только, когда загружены в память PE-загрузчиком. В нашем случае мы мэппируем файл в память, но не так, как это делает PE-загрузчик. Поэтому мы не можем напрямую использовать эти RVA. Каким-то образом мы должны конвертировать эти RVA в файловые смещения. Специально для этого я написал функцию RVAToOffset. Я не буду детально детально анализировать ее здесь. Достаточно сказать, что она проверяет сверяет данный RVA с RVA'ми началами и концами всех секций в PE-файле и использует значение в поле PointerToRawData из структуры IMAGE_SECTION_HEADER, чтобы сконвертировать RVA в файловое смещение.

Чтобы использовать эту функцию, вы передаете ей два параметра: указатель на мэппированный файл и RVA, которое вы хотите сконвертировать. Она возвращает в eax файловое смещение. В вышеприведенном отрывке кода мы должны добавить указатель на промэппированный файл к файловому оффсету, чтобы сконвертировать его в виртуальный адрес. Кажется сложным, не правда ли? :)

```
assume edi:ptr IMAGE_IMPORT_DESCRIPTOR
.while !([edi].OriginalFirstThunk==0 && [edi].TimeStamp==0 &&
[edi].ForwarderChain==0 && [edi].Name1==0 && [edi].FirstThunk==0)
```

edi теперь указывает на первую структуру IMAGE_IMPORT_DESCRIPTOR. Мы будем обрабатывать массив, пока не найдем структуру с нулями в всех полях, которая отмечает конец массива.

```
invoke AppendText,hDlg,addr ImportDescriptor
invoke RVAToOffset,pMapping, [edi].Name1
mov edx,eax
add edx,pMapping
```

Мы хотим отобразить значения текущей структуры IMAGE_IMPORT_DESCRIPTOR в edit control'е. Name1 отличается от других параметров тем, что оно содержит RVA имени DLL. Поэтому мы должны сначала сконвертировать его в виртуальный адрес.

```
invoke wsprintf, addr temp, addr IDTemplate,
[edi].OriginalFirstThunk,[edi].TimeStamp,[edi].ForwarderChain,edx,[edi].F
invoke AppendText,hDlg,addr temp
```

Отображаем значения текущего IMAGE_IMPORT_DESCRIPTOR.

```
.if [edi].OriginalFirstThunk==0
    mov esi,[edi].FirstThunk
.else
    mov esi,[edi].OriginalFirstThunk
.endif
```

Затем мы готовимся к обработке массива IMAGE_THUNK_DATA. Обычно мы должны выбрать массив, на который ссылается OriginalFirstThunk. Тем не менее, некоторые линкеры ошибочно

помещают в это поле 0, поэтому мы сначала должны проверить, не равен ли OriginalFirstThunk нулю. Если это так, мы используем массив, на который указывает FirstThunk.

```
invoke RVAToOffset,pMapping,esi
add eax,pMapping
mov esi,eax
```

Снова значение в OriginalFirstThunk/FirstThunk - это RVA. Мы должны сконвертировать его в виртуальный адрес.

```
invoke AppendText,hDlg,addr NameHeader
.while dword ptr [esi]!=0
```

Теперь мы готовы к обработке массива IMAGE_THUNK_DATA'ов, чтобы найти имена функций, импортируемых из DLL. Мы будем обрабатывать этот массив, пока не найдем элемент, содержащий 0.

```
test dword ptr [esi],IMAGE_ORDINAL_FLAG32
jnz ImportByOrdinal
```

Первая вещь, которую мы должны сделать с IMAGE_THUNK_DATA - это сверить ее с IMAGE_ORDINAL_FLAG32. Если MSB IMAGE_THUNK_DATA'ы равен 1, функция экспортируется через ординал, поэтому нам нет нужды обрабатывать ее дальше. Мы можем извлечь ее ординал из нижнего слова IMAGE_THUNK_DATA'ы и перейти к следующему IMAGE_THUNK_DATA-слову.

```
invoke RVAToOffset,pMapping,dword ptr [esi]
mov edx,eax
add edx,pMapping
assume edx:ptr IMAGE_IMPORT_BY_NAME
```

Если MSB IMAGE_THUNK_DATA'ы равен 0, тогда та содержит RVA структуры IMAGE_IMPORT_BY_NAME. Нам требуется сначала сконвертировать ее в виртуальный адрес.

```
mov cx, [edx].Hint
movzx ecx,cx
invoke wsprintf,addr temp,addr NameTemplate,ecx,addr [edx].Name1
jmp ShowTheText
```

Hint - это поле размером в слово. Мы должны сконвертировать ее в значение размером в двойное слово, перед тем, как передать его wsprintf. И мы показываем и Hint и имя функции в edit control'е.

```
ImportByOrdinal:
    mov edx,dword ptr [esi]
    and edx,0FFFFh
    invoke wsprintf,addr temp,addr OrdinalTemplate,edx
```

Если функция экспортируется только через ординал, мы обнуляем верхнее слово и отображаем ординал.

```
ShowTheText:
    invoke AppendText,hDlg,addr temp
    add esi,4
```

После добавления имени функции/ординала в edit control, мы переходим к следующему IMAGE_THUNK_DATA.

```
.endw
add edi,sizeof IMAGE_IMPORT_DESCRIPTOR
```

После обработки всех dword'ов IMAGE_THUNK_DATA в массив, мы переходим к следующему IMAGE_IMPORT_DESCRIPTOR'у, чтобы обработать функции импорта из других DLL.

Дополнительно:

Тutorial был бы незаконченным, если бы я не упомянул о bound import'е. Чтобы объяснить, что это такое, я должен немного отвлечься. Когда PE-загрузчик загружает PE-файл в память, он проверяет таблицу импорта и загружает требуемые DLL в адресное пространство процесса. Затем он

пробегают через массив `IMAGE_THUNK_DATA`, примерно так же как мы, и замещают `IMAGE_THUNK_DATA`'ы реальными адресами функций импорта. Этот шаг требует времени. Если программист каким-то образом смог бы верно предсказать адреса функций, PE-загрузчику не потребовалось бы фиксировать `IMAGE_THUNK_DATA`'ы каждый раз, когда запускается PE. `Bound import` - продукт этой идеи.

Если облечь это в простые слова, существует утилита под названием `bind.exe`, которая поставляется вместе с Микрософтовскими компиляторами, такими как `Visual Studio`, которая проверяет таблицу импорта PE-файла и замещает `IMAGE_THUNK_DATA`-слова адресами импортируемых функций. Когда файл загружается, PE-загрузчик должен проверить, верны ли адреса. Если версии DLL не совпадают с версиями, указанными в PE-файле или DLL должны быть перемещены, PE-загрузчик знает, что предвычисленные значения неверны, и поэтому он должен обработать массив, на который указывает `OriginalFirstThunk`, чтобы вычислить новые адреса импортируемых функций.

`Bound import` не играет большой роли в нашем примере, потому что мы по умолчанию используем `OriginalFirstThunk`. За дополнительной информацией о `bound import`'е, я рекомендую обратиться к `pe.txt`'у LUEVELSMEYER'a.

PE. Урок 7. Таблица экспорта

Автор: lczelion, пер. Aquila

Дата: 2002-06-06

Мы изучили одну часть динамической линковки под названием таблицы импорта в предыдущем tutorialе. Теперь мы узнаем о другой стороне медали, таблице экспорта.

Скачайте пример (находится в архиве `wasm_files.zip: files/article42/pe-tut07.zip`).

ТЕОРИЯ

Когда PE-загрузчик запускает программу, он загружает соответствующие DLL в адресное пространство процесса. Затем он извлекает информацию об импортированных функциях из основной программы. Он использует информацию, чтобы найти в DLL адреса функций, которые будут вызываться из основного процесса. Место в DLL, где PE-загрузчик ищет адреса функций - таблица экспорта.

Когда DLL/EXE экспортирует функцию, чтобы та была использована другой DLL/EXE, она может сделать это двумя путями: она может экспортировать функцию по имени или только по ординалу. Скажем, если в DLL есть функция под названием "GetSysConfig", она может указать другой DLL или EXE, что если они хотят вызвать функцию, они должны указать ее имя, то есть, GetSysConfig. Другой путь - экспортировать функцию по ординалу. Что такое ординал? Ординал - это 16-битный номер, который уникальным образом идентифицирует функцию в определенном DLL. Этот номер уникален только для той DLL, на которую он ссылается. Например, в вышеприведенном примере, DLL может решить экспортировать функцию через ординал, скажем, 16. Тогда другая DLL/EXE, которая хочет вызвать эту функцию должна указать этот номер в GetProcAddress. Это называется экспортом только через ординал.

Экспорт только через ординал не очень рекомендуется, так как это может вызвать проблемы при распространении DLL. Если DLL апгрейдится или апдейтится, человек, занимающийся ее программированием не может изменять ординалы функции, иначе программы, использующие эту DLL, перестанут работать.

Теперь мы можем анализировать структуру экспорта. Как и в случае с таблицей импорта, вы можете узнать, где находится таблица экспорта, из директории данных. В этом случае таблица экспорта - это первый член директории данных. Структура экспорта называется IMAGE_EXPORT_DIRECTORY. В структуре 11 параметров, но только несколько из них по настоящему нужны.

Поле	Значение
nName	Настоящее имя модуля. Это поле необходимо, потому что имя файла может измениться. Если подобное произойдет, PE-загрузчик будет использовать это внутреннее имя.
nBase	Число, которое вы должны сопоставить с ординалами, чтобы получить соответствующие индексы в массиве адресов функций.
NumberOfFunctions	Общее количество функций/символов, которые экспортируются из модуля.

Поле	Значение
NumberOfNames	Количество функций/символов, которые экспортируются по имени. Это значение не является числом ВСЕХ функций/символов в модуле. Это значение может быть равно нулю. В этом случае, модуль может экспортировать только по имени. Если нет функции/символа, который бы экспортировались, RVA таблицы экспорта в директории данных будет равен нулю.
AddressOfFunctions	RVA, который указывает на массив RVA функций/символов в модуле. Вкратце, RVA на все функции в модуле находятся в массиве и это поле указывает на начало массива.
AddressOfNames	RVA, которое указывает на массив RVA имен функций в модуле.
AddressOfNameOrdinals	RVA, которое указывает на 16-битный массив, который содержит ординалы, ассоциированные с именами функций в массиве AddressOfNames.

Вышеприведенная таблица может не дать вам ясного понимания, что такое таблица экспортов. Упрощенное объяснение ниже прояснит суть концепции.

Таблица экспортов существует для использования PE-загрузчиком. Прежде всего, модуль должен где-то сохранить адреса всех экспортированных функций где-то PE-загрузчик сможет их найти. Он держит их в массиве, на который ссылается поле AddressOfFunctions. Количество элементов в массиве находится в NumberOfFunctions. Таким образом, если модуль экспортирует 40 функций, массив будет также состоять из 40 элементов, NumberOfFunctions будет содержать значение 40. Теперь, если некоторые функции экспортируются по именам, модуль должен сохранить их имена в файле. Он сохраняет RVA имен в массиве, чтобы PE-загрузчик может их найти. На это массив ссылается AddressOfNames и количество имен находится в NumberOfNames.

Подумайте о работе, выполняемой PE-загрузчиком. Он знает имена экспортируемых функций, он должен каким-то образом получить адреса этих функций. До нынешнего момента модуль имел два массива: имена и адреса, но между ними не было связи. Теперь нам нужно что-то, что свяжет имена функций с их адресами. PE-спецификация использует индексы в массиве адресов в качестве элементарной линковки. Таким образом, если PE-загрузчик найдет имя, которое он ищет в массиве имен, он может также получить индекс в таблице адресов для этого имени. Индексы находятся в другом массиве, на который указывает поле AddressOfNameOrdinals. Так как этот массив существует в качестве посредника между именами и адресами, он должен содержать такое же количество элементов, как и массив имен, то есть, каждое имя может иметь один и только один ассоциированный с ним адрес. Чтобы линковка работала, оба массива имен и индексов, должны соответствовать друг другу, то есть, первый элемент в массиве индексов должен содержать индекс для первого имени и так далее.

AddressOfNames	AddressOfNameOrdinals
RVA of Name 1	Index of Name 1
<-->	
RVA of Name 2	Index of Name 2
<-->	
RVA of Name 3	Index of Name 3
<-->	
RVA of Name 4	Index of Name 4
<-->	
...	...
<-->	
RVA of Name N	Index of Name N

Если у нас есть имя экспортируемой функции и нам требуется получить ее адрес в модуле, мы должны сделать следующее.

- Перейти к PE-загрузчику
- Прочитать виртуальный адрес таблицы экспорта в директории данных
- Перейти к таблице экспорта и получить количество имен (NumberOfNames)
- Параллельно просмотреть массивы, на который указывают AddressOfNames и AddressOfNameOrdinals, ища нужное имя. Если имя найдено в массиве AddressOfNames, вы должны извлечь значение в ассоциированном элементе массива AddressOfNameOrdinals. Например, если вы нашли RVA необходимого имени в 77-ом элементе массива AddressOfNames, вы должны извлечь значение, сохраняемое в 77-ом элементе массива AddressOfNameOrdinals. Если вы просмотрели все NumberOfElements элементов массива и ничего не нашли, вы знаете, что имя находится не в этом модуле.
- Используйте значение из массива AddressOfNameOrdinals в качестве индекса для массива AddressOfFunctions. Например, если значение равно 5, вы должны извлечь значение 5-го элемента массива AddressOfFunctions. Это значение будет являться RVA функции.

Сейчас мы можем переключить наше внимание на параметр nBase из структур. Вы уже знаете, что массив AddressOfFunctions содержит адреса всех экспортируемых символов в модуле. И PE-загрузчик использует индексы этого массива, чтобы найти адреса функций. Давайте представим ситуацию, что мы используем индексы этого массива как ординалы. Так как программист может указать любое число в качестве стартового ординала в .def-файле, например, 200, это значит, что в массиве AddressOfFunctions должно быть по крайней мере 200 элементов. Хотя первые 200 элементов не будут использоваться, они должны существовать, так как PE-загрузчик может использовать индексы, чтобы найти правильные адреса. Это совсем нехорошо. Параметр nBase существует для того, чтобы решить эту проблему. Если программист указывает начальным ординалом 200, значением nBase будет 200. Когда PE-загрузчик считывает значение в nBase, он знает, что первые 200 элементов не существуют, и что он должен вычитать от ординала значение nBase, чтобы получить настоящий индекс нужного элемента в массиве AddressOfFunctions. Используя nBase, нет надобности в 200 пустых элементах.

Заметьте, что nBase не влияет на значения в массиве AddressOfNameOrdinals. Несмотря на имя "AddressOfNameOrdinals", этот массив содержит индексы в массиве, а не ординалы.

Обсудив nBase, мы можем продолжить.

Предположите, что у нас есть ординал функции и нам нужно получить адрес этой функции, тогда мы должны сделать следующее:

- Перейти к PE-заголовку
- Получить RVA таблицы экспортов из директории данных
- Перейти к таблице экспортов и получить значение nBase
- Вычесть из ординала значение nBase, и теперь у вас есть индекс нужного элемента массива AddressOfFunctions.
- Сравните индекс со значением NumberOfFunctions. Если индекс больше или равен ему, ординал неверен.
- Используйте индекс, чтобы получить RVA функции в массиве AddressOfFunctions.

Заметьте, что получение адреса функции из ординала гораздо проще и быстрее, по ее имени. Нет необходимости обрабатывать AddressOfNames и AddressOfNameOrdinals. Выигрыш в качестве смазывается трудностями в поддержке модуля.

Резюмируя: если вы хотите получить адрес функции, вам нужно обработать как AddressOfNames, так и AddressOfNameOrdinals, чтобы получить индекс элемента массива AddressOfFunctions. Если у вас есть ординал, вы можете сразу перейти к массиву AddressOfFunctions, после того как ординал

скорректирован с помощью nBase.

Если функция экспортируется по имени, вы можете использовать как ее имя, так и ее ординал в GetProcAddress. Но что, если функция экспортируется только через ординал?

"Функция экспортируется только через ординал" означает, что функция не имеет входов в AddressOfNames и AddressOfNameOrdinals. Помните два поля - NumberOfFunctions и NumberOfNames. Существование этих двух полей - это свидетельство того, что некоторые функции могут не иметь имен. Количество функций должно быть по крайней мере равно количеству имен. Функции, которые не имеют имен, экспортируются только через их ординалы. Например, если есть 70 функций, а массив AddressOfNames состоит только из 40 элементов, это означает, что в модуле есть 30 функций, экспортирующиеся только через их ординалы. Как мы можем узнать, какие функции экспортируются подобным образом? Это нелегко. Вы должны узнать это методом исключения, то есть элементы массива AddressOfFunctions, которые не упоминаются в массиве AddressOfNameOrdinals, содержат RVA функций, экспортирующиеся только через ординалы.

ПРИМЕР

Это пример схож с рассмотренным в предыдущем примере. Тем не менее, он отображает значения некоторых членов структуры IMAGE_EXPORT_DIRECTORY, а также отображает RVA, ординалы и имена экспортируемых функций. Заметьте, что этот пример не отображает функции, которые экспортируются только через ординалы.

```
.386
.model flat,stdcall
option casemap:none
include \masm32\include\windows.inc
include \masm32\include\kernel32.inc
include \masm32\include\comdlg32.inc
include \masm32\include\user32.inc
includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib
includelib \masm32\lib\comdlg32.lib

IDD_MAINDLG equ 101
IDC_EDIT equ 1000
IDM_OPEN equ 40001
IDM_EXIT equ 40003

DlgProc proto :DWORD,:DWORD,:DWORD,:DWORD
ShowExportFunctions proto :DWORD
ShowTheFunctions proto :DWORD,:DWORD
AppendText proto :DWORD,:DWORD

SEH struct
    PrevLink dd ?
    CurrentHandler dd ?
    SafeOffset dd ?
    PrevEsp dd ?
    PrevEbp dd ?
SEH ends

.data
AppName db "PE tutorial no.7",0
ofn OPENFILENAME <>
FilterString db "Executable Files (*.exe, *.dll)",0,"*.exe;*.dll",0
            db "All Files",0,"*.*",0,0
FileOpenError db "Cannot open the file for reading",0
FileOpenMappingError db "Cannot open the file for memory mapping",0
FileMappingError db "Cannot map the file into memory",0
NotValidPE db "This file is not a valid PE",0
NoExportTable db "No export information in this file",0
```

```

CRLF db 0Dh,0Ah,0
ExportTable db 0Dh,0Ah,"=====[ IMAGE_EXPORT_DIRECTORY ]=====",0Dh,0Ah
            db "Name of the module: %s",0Dh,0Ah
            db "nBase: %lu",0Dh,0Ah
            db "NumberOfFunctions: %lu",0Dh,0Ah
            db "NumberOfNames: %lu",0Dh,0Ah
            db "AddressOfFunctions: %lX",0Dh,0Ah
            db "AddressOfNames: %lX",0Dh,0Ah
            db "AddressOfNameOrdinals: %lX",0Dh,0Ah,0
Header db "RVA Ord. Name",0Dh,0Ah
        db "-----",0
template db "%lX %u %s",0

.data?
buffer db 512 dup(?)
hFile dd ?
hMapping dd ?
pMapping dd ?
ValidPE dd ?

.code
start:
invoke GetModuleHandle,NULL
invoke DialogBoxParam, eax, IDD_MAINDLG,NULL,addr DlgProc, 0
invoke ExitProcess, 0

DlgProc proc hDlg:DWORD, uMsg:DWORD, wParam:DWORD, lParam:DWORD
.if uMsg==WM_INITDIALOG
    invoke SendDlgItemMessage,hDlg,IDC_EDIT,EM_SETLIMITTEXT,0,0
.elseif uMsg==WM_CLOSE
    invoke EndDialog,hDlg,0
.elseif uMsg==WM_COMMAND
    .if lParam==0
        mov eax,wParam
        .if ax==IDM_OPEN
            invoke ShowExportFunctions,hDlg
        .else ; IDM_EXIT
            invoke SendMessage,hDlg,WM_CLOSE,0,0
        .endif
    .endif
.else
    mov eax,FALSE
    ret
.endif
mov eax,TRUE
ret
DlgProc endp

SEHHandler proc uses edx pExcept:DWORD, pFrame:DWORD, pContext:DWORD,
pDispatch:DWORD
mov edx,pFrame
assume edx:ptr SEH
mov eax,pContext
assume eax:ptr CONTEXT
push [edx].SafeOffset
pop [eax].regEip
push [edx].PrevEsp
pop [eax].regEsp
push [edx].PrevEbp
pop [eax].regEbp
mov ValidPE, FALSE
mov eax,ExceptionContinueExecution
ret
SEHHandler endp

ShowExportFunctions proc uses edi hDlg:DWORD
LOCAL seh:SEH
mov ofn.lStructSize,SIZEOF ofn
mov ofn.lpstrFilter, OFFSET FilterString
mov ofn.lpstrFile, OFFSET buffer
mov ofn.nMaxFile,512

```

```

mov ofn.Flags, OFN_FILEMUSTEXIST or OFN_PATHMUSTEXIST or OFN_LONGNAMES or
OFN_EXPLORER or OFN_HIDEREADONLY
invoke GetOpenFileName, ADDR ofn
.if eax==TRUE
    invoke CreateFile, addr buffer, GENERIC_READ, FILE_SHARE_READ, NULL,
    OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL
    .if eax!=INVALID_HANDLE_VALUE
        mov hFile, eax
        invoke CreateFileMapping, hFile, NULL, PAGE_READONLY,0,0,0
        .if eax!=NULL
            mov hMapping, eax
            invoke MapViewOfFile,hMapping,FILE_MAP_READ,0,0,0
            .if eax!=NULL
                mov pMapping,eax
                assume fs:nothing
                push fs:[0]
                pop seh.PrevLink
                mov seh.CurrentHandler,offset SEHHandler
                mov seh.SafeOffset,offset FinalExit
                lea eax,seh
                mov fs:[0], eax
                mov seh.PrevEsp,esp
                mov seh.PrevEbp,ebp
                mov edi, pMapping
                assume edi:ptr IMAGE_DOS_HEADER
                .if [edi].e_magic==IMAGE_DOS_SIGNATURE
                    add edi, [edi].e_lfanew
                    assume edi:ptr IMAGE_NT_HEADERS
                    .if [edi].Signature==IMAGE_NT_SIGNATURE
                        mov ValidPE, TRUE
                    .else
                        mov ValidPE, FALSE
                    .endif
                .else
                    mov ValidPE,FALSE
                .endif
            .endif
FinalExit:
            push seh.PrevLink
            pop fs:[0]
            .if ValidPE==TRUE
                invoke ShowTheFunctions, hDlg, edi
            .else
                invoke MessageBox,0, addr NotValidPE, addr AppName,
                MB_OK+MB_ICONERROR
            .endif
            invoke UnmapViewOfFile, pMapping
        .else
            invoke MessageBox, 0, addr FileMappingError, addr AppName,
            MB_OK+MB_ICONERROR
        .endif
        invoke CloseHandle,hMapping
    .else
        invoke MessageBox, 0, addr FileOpenMappingError, addr AppName,
        MB_OK+MB_ICONERROR
    .endif
    invoke CloseHandle, hFile
    .else
        invoke MessageBox, 0, addr FileOpenError, addr AppName,
        MB_OK+MB_ICONERROR
    .endif

.endif
ret
ShowExportFunctions endp

AppendText proc hDlg:DWORD,pText:DWORD
invoke SendDlgItemMessage,hDlg,IDC_EDIT,EM_REPLACESEL,0,pText
invoke SendDlgItemMessage,hDlg,IDC_EDIT,EM_REPLACESEL,0,addr CRLF
invoke SendDlgItemMessage,hDlg,IDC_EDIT,EM_SETSEL,-1,0
ret
AppendText endp

```


RVAToFileMap PROC uses edi esi edx ecx pFileMap:DWORD,RVA:DWORD

```

mov esi,pFileMap
assume esi:ptr IMAGE_DOS_HEADER
add esi,[esi].e_lfanew
assume esi:ptr IMAGE_NT_HEADERS
mov edi,RVA ; edi == RVA
mov edx,esi
add edx,sizeof IMAGE_NT_HEADERS
mov cx,[esi].FileHeader.NumberOfSections
movzx ecx,cx
assume edx:ptr IMAGE_SECTION_HEADER
.while ecx>0
    .if edi>=[edx].VirtualAddress
        mov eax,[edx].VirtualAddress
        add eax,[edx].SizeOfRawData
        .if edi < eax
            mov eax,[edx].VirtualAddress
            sub edi,eax
            mov eax,[edx].PointerToRawData
            add eax,edi
            add eax,pFileMap
            ret
        .endif
    .endif
    add edx,sizeof IMAGE_SECTION_HEADER
    dec ecx
.endw
assume edx:nothing
assume esi:nothing
mov eax,edi
ret
RVAToFileMap endp

```

ShowTheFunctions proc uses esi ecx ebx hDlg:DWORD, pNTHdr:DWORD

LOCAL temp[512]:BYTE

LOCAL NumberOfNames:DWORD

LOCAL Base:DWORD

```

mov edi,pNTHdr
assume edi:ptr IMAGE_NT_HEADERS
mov edi,[edi].OptionalHeader.DataDirectory.VirtualAddress
.if edi==0
    invoke MessageBox,0, addr NoExportTable,addr AppName,MB_OK+MB_ICONERROR

    ret
.endif
invoke SetDlgItemText,hDlg,IDC_EDIT,0
invoke AppendText,hDlg,addr buffer
invoke RVAToFileMap,pMapping,edi
mov edi,eax
assume edi:ptr IMAGE_EXPORT_DIRECTORY
mov eax,[edi].NumberOfFunctions
invoke RVAToFileMap, pMapping,[edi].nName
invoke wsprintf, addr temp,addr ExportTable, eax, [edi].nBase,
[edi].NumberOfFunctions, [edi].NumberOfNames, [edi].AddressOfFunctions,
[edi].AddressOfNames, [edi].AddressOfNameOrdinals
invoke AppendText,hDlg,addr temp
invoke AppendText,hDlg,addr Header
push [edi].NumberOfNames
pop NumberOfNames
push [edi].nBase
pop Base
invoke RVAToFileMap,pMapping,[edi].AddressOfNames
mov esi,eax
invoke RVAToFileMap,pMapping,[edi].AddressOfNameOrdinals
mov ebx,eax
invoke RVAToFileMap,pMapping,[edi].AddressOfFunctions
mov edi,eax
.while NumberOfNames>0
    invoke RVAToFileMap,pMapping,dword ptr [esi]
    mov dx,[ebx]

```

```

    movzx edx,dx
    mov ecx,edx
    shl edx,2
    add edx,edi
    add ecx,Base
    invoke wsprintf, addr temp,addr template,dword ptr [edx],ecx,eax
    invoke AppendText,hDlg,addr temp
    dec NumberOfNames
    add esi,4
    add ebx,2
.endw
ret

ShowTheFunctions endp
end start

```

АНАЛИЗ

```

mov edi,pNTHdr
assume edi:ptr IMAGE_NT_HEADERS
mov edi, [edi].OptionalHeader.DataDirectory.VirtualAddress
.if edi==0
    invoke MessageBox,0, addr NoExportTable,addr AppName,MB_OK+MB_ICONERROR
    ret
.endif

```

После того, как программа убеждается, что файл является верным PE, она переходит к директории данных и получает виртуальный адрес таблицы экспорта. Если виртуальный адрес равен нулю, в файле нет ни одного экспортируемого символа.

```

mov eax,[edi].NumberOfFunctions
invoke RVAToFileMap, pMapping,[edi].nName
invoke wsprintf, addr temp,addr ExportTable, eax, [edi].nBase,
[edi].NumberOfFunctions, [edi].NumberOfNames, [edi].AddressOfFunctions,
[edi].AddressOfNames, [edi].AddressOfNameOrdinals
invoke AppendText,hDlg,addr temp

```

Мы отображаем важную информацию в структуре IMAGE_EXPORT_DIRECTORY в edit control'e.

```

push [edi].NumberOfNames
pop NumberOfNames
push [edi].nBase
pop Base

```

Так как мы хотим перечислить все имена функций, нам требуется знать, как много имен в таблице экспорта. nBase используется, когда мы хотим сконвертировать индексы, содержащиеся в массиве AddressOfFunctions в ординалы.

```

invoke RVAToFileMap,pMapping,[edi].AddressOfNames
mov esi,eax
invoke RVAToFileMap,pMapping,[edi].AddressOfNameOrdinals
mov ebx,eax
invoke RVAToFileMap,pMapping,[edi].AddressOfFunctions
mov edi,eax

```

Адреса трех массивов сохранены в esi, ebx и edi, готовые для использования.

```

.while NumberOfNames>0

```

Продолжаем, пока все имена не будут обработаны.

```

invoke RVAToFileMap,pMapping,dword ptr [esi]

```

Так как esi указывает на массив RVA экспортируемых функций, разыменование ее даст RVA настоящего имени. Мы сконвертируем ее в виртуальный адрес, который будет передан затем

wsprintf.

```
mov dx,[ebx]
movzx edx,dx
mov ecx,edx
add ecx,Base
```

ebx указывает на массив ординалов. Элементы этого массива размеров в слово.

Таким образом мы сначала должны сконвертировать значение в двойное слово. edx и ecx содержит индекс массива AddressOfFunctions. Мы добавляем значение nBase к ecx, чтобы получить номер ординала функции.

```
shl edx,2
add edx,edi
```

Мы умножаем индекс на 4 (каждый элемент в массиве AddressOfFunctions размером 4 байта), а затем, добавляем адрес массива AddressOfFunctions к нему. Таким образом edx указывает на RVA функции.

```
invoke wsprintf, addr temp,addr template,dword ptr [edx],ecx,eax
invoke AppendText,hDlg,addr temp
```

Мы отображаем RVA, ординал и имя функции в edit control'e.

```
dec NumberOfNames
add esi,4
add ebx,2
.endw
```

Обновим счетчик и адреса текущих элементов массивов AddressOfNames и AddressOfNameOrdinals. Продолжаем, пока все имена не будут обработаны.

VXD. Урок 1. Основы

Автор: lczelion, пер. Aquila

Дата: 2002-06-06

В этой серии tutorиалов, я предполагаю, что вы, читатель, знакомы с операциями защищенного режима 80x86, такими как виртуальный 8086-режим, paging, GDT, LDT, IDT. Если вы не знаете о них, прочитайте сначала интелловскую документацию по адресу <http://developet.intel.com/design/pentium/manuals/>

Windows 95 - это мультиветвенная операционная система, выполняющаяся на самом привилегированном уровне, ring 0. Все приложения запускаются на ring 3, наименее привилегированном уровне. Таким образом, приложения ограничены в том, что они могут делать в системе. Они не могут выполнять привилегированные инструкции процессора, не могут получить доступ к портам ввода/вывода напрямую и так далее. Вы, без сомнения, знакомы с тремя большими системными компонентами: gdi32, kernel32 и user32. Вы, наверное, думали, что такой важный код должен выполняться в ring 0. Но на самом деле, они выполняются в ring 3, как и все остальные приложения. Вот почему они имеют не больше привилегий, чем, скажем, калькулятор или "минер". Настоящая сила системы под контролем менеджера виртуальной машины (VMM - virtual machine manager) и драйверов виртуальных устройств.

Ничего бы этого не случилось, если бы не DOS, усложняющий ситуацию. Во время эпохи Windows 3.x, существовало множество успешных DOS-программ на рынке. Windows 3.x должна была быть способной выполнять их вместе с Windows-программами, иначе бы она потерпела коммерческий провал.

Эту диллему не так легко решить. DOS- и Windows- программы разительно отличаются друг от друга. DOS-программы плохи в том смысле, что они берут под свой контроль все в системе: клавиатуру, CPU, память, диск и т.д.

Они не знают, как взаимодействовать с другими программами, в то время как Windows-программы используют кооперативную многозадачность, то есть каждая Windows-программа должна передавать контроль другим программам через GetMessage или PeekMessage.

Решение в том, чтобы выполнять каждую DOS-программу на виртуальной 8086-машине, в то время как другие Windows-программы выполняются на другой виртуальной машине под названием системная виртуальная машина. Windows ответственна за то, чтобы выделять CPU-время каждой машине по кругу. Таким образом, под Windows 3.x Windows-программы используют кооперативную многозадачность, но виртуальные машины используют преимущественную многозадачность.

Что такое виртуальная машина? Виртуальная машина - это фикция, созданная полностью программным образом. Виртуальная машина реагирует на действия выполняющейся программы, также как и настоящая машина. Таким образом, программа не знает, что она выполняется в виртуальной машине и это не мешает ей. Пока виртуальная машина отвечает программе также, как настоящая машина, она должна расцениваться как настоящая.

Вы можете думать о интерфейсе между настоящей машиной и ее программным обеспечением как о некоей разновидности API. Этот необычный API состоит из прерываний, вызовов BIOS и портах ввода/вывода. Если Windows может точно воспроизвести этот API, программы, выполняющиеся на виртуальной машине будут вести себя точно также, как и на настоящей.

Это тот момент, когда на сцену выступает VMM и VxD. Чтобы координировать и надзирать за виртуальными машинами, Windows требуется программа, специально предназначенная для этого. Эта программа называется менеджер виртуальных машин.

Менеджер виртуальных машин

VMM - это программа, выполняющаяся в 32-битном защищенном режиме. Ее основная задача заключается в создании и поддержке рабочей среды виртуальных машин. Она ответственна за создание, выполнение и прерывание виртуальных машин. VMM является одной из многих системных VxD и находится в файле VMM32.VXD в вашей системной директории. Давайте проанализируем порядок загрузки Windows 95.

- io.sys загружается в память.
- обрабатывается config.sys и autoexec.bat
- вызывается win.com
- win.com запускает VMM32.VXD, которая фактически является простым DOS EXE-файлом.
- VMM32.VXD загружает VMM в расширенную память, используя драйвер XMS.
- VMM инициализирует сам себя и другие стандартные драйвера виртуальных устройств.
- VMM переключает машину в защищенный режим и создает системную виртуальную машину.
- Виртуальное устройство оболочки, которое загружается последним, запускает Windows на системной виртуальной машине путем запуска krnl386.exe.
- krnl386.exe загружает все другие файлы, заканчивая оболочку Windows 95.

Как вы можете видеть, VMM - это первый VxD, загружаемый в память. Он создает системную виртуальную машину и инициализирует другие VxD. Он также предоставляет этим VxD различные сервисы.

Поведение VMM и VxD сильно отличается от обычных программ. Они, по большей части, находятся в спящем состоянии. Пока приложения выполняются в системе, эти VxD не активны. Они будут пробуждаться, когда произойдут прерывания/ошибки/события, которые потребуют их участия.

VxD должны синхронизировать свои доступы к сервисам VMM. Есть некоторые ситуации, в которых небезопасно вызывать сервисы VMM, например, когда обрабатывается какое-то аппаратное прерывание. В это время, VMM не может гарантировать ответ на ваш запрос. Вы как создатель VxD должны быть предельно осторожны в том, что вы делаете. Помните это, нет никого, кто будет обрабатывать вашу ошибку. Вы абсолютно одни в ring 0.

Виртуальные драйвера устройств

VxD - это аббревиатура Virtual Device Driver. x - это замена имени устройства, например, виртуальный драйвер клавиатуры, виртуальный драйвер мыши и так далее. VxD - это ключи к успешной виртуализации железа. Помните, что DOS-программы думают, что под ними вся система. Когда они запускаются в виртуальной машине, Windows должна предоставить им воплощения реальных устройств. VxD - это они и есть. VxD обычно воплощают какие-либо аппаратные устройства. Например, когда dos-программа думает, что она взаимодействует с клавиатурой, на самом деле она взаимодействует с виртуальным устройством-клавиатурой. VxD обычно берет контроль над каким-то "железным" устройством и распределяет доступ к нему между виртуальными машинами.

Тем не менее, такого правила, что VxD должен быть ассоциирован с каким-то железом, нет. Это правда, что VxD были спроектированы для виртуализации аппаратных устройств, но мы также можем рассматривать их как ring-0 DLL. Например, если вы хотите получить возможности, которые могут быть достигнуты только в ring 0, вы можете написать VxD, который выполнит эту работу за вас. Таким образом, вы можете рассматривать VxD как расширение вашей программы, так как он не виртуализирует никакого реального устройства.

Прежде, чем перейти к рассказу о том, как создавать свои VxD, давайте я сообщу несколько важных моментов относительно VxD.

- VxD существуют только в Windows 9x. Они не будут выполняться на Windows NT. Поэтому, если ваша программа полагается на VxD, она не будет работать под платформой Windows NT.
- VxD - это наиболее мощные объекты системы. Так как они могут делать все, что угодно, они очень опасны. Неисправный VxD может привести к "падению" системы. Против таких VxD нет защиты.
- Обычно существует много путей, чтобы достигнуть желаемой цель без обращения к VxD. Подумайте дважды, прежде, чем прибегнуть к ним. Только если нет возможности выполнить задачу в ring 3, используйте их.

В Windows 95 существует два типа VxD.

- Static VxD
- Dynamic VxD

Статические VxD - это такие VxD, которые загружаются во время системной загрузки и остаются загруженными, пока система не прекратит работу. Этот тип VxD появился еще во времена Windows 3.x. Динамические VxD доступны только под Windows 9x. Динамические VxD могут загружаться и выгружаться по мере надобности. Большая часть этих VxD - это VxD, которые контролируют устройства Plug and Play. Эти драйвера загружаются Менеджером Конфигурации и Input Output Supervisor'ом. Вы также можно загружать/выгружать динамические VxD из ваших win32-приложений.

Взаимодействие между VxD

VxD, включая VMM, могут взаимодействовать друг с другом тремя путями:

- Управляющие сообщения
- Сервисные API
- Callback'и

Управляющие сообщения: VMM посылает системные управляющие сообщения всем загруженным VxD, когда происходит какое-то интересное событие. В этом отношении контрольные сообщения похожи на windows-сообщения приложений ring-3. У каждого VxD есть функция, которая получает и обрабатывает контрольные сообщения, называемая управляющей функцией устройства. Существует около 50 системных управляющих сообщений. Почему их так мало? Зачастую в системе загружено много VxD, а так как управляющее сообщение шлется всем загруженным VxD, то система могла бы повиснуть, если бы был слишком много управляющих сообщений. Поэтому существуют только по-настоящему важные управляющие сообщения, такие как создание виртуальной машины, ее уничтожение и так далее. В добавление к системным управляющим сообщениями VxD может определить свои собственные управляющие сообщения, чтобы взаимодействовать с другими VxD, понимающими эти сообщения.

Сервисные API: VxD, включая VMM, обычно экспортирует множество публичных функций, которые могут вызываться другими VxD. Эти функции называются сервисами VxD. Механизм вызова этих VxD отличается от того, как это происходит в приложениях ring-3. Каждый VxD, который экспортирует сервисы VxD должен иметь уникальный ID номер. Вы можете получить эти ID от Микрософта. ID - это 16-битный номер, который уникальным образом идентифицирует VxD. Например,

UNDEFINED_DEVICE_ID	EQU 00000H
VMM_DEVICE_ID	EQU 00001H
DEBUG_DEVICE_ID	EQU 00002H
VPICD_DEVICE_ID	EQU 00003H
VDMAD_DEVICE_ID	EQU 00004H
VTD_DEVICE_ID	EQU 00005H

Вы можете видеть, что ID VMM - 1, VPICD - 3 и так далее. VMM использует эти уникальные ID, чтобы найти VxD, которая экспортирует требуемые сервисы. Вы также можете выбрать требующийся вам сервис по его индексу в service branch table. Когда VxD экспортирует сервисы,

она сохраняет в таблице адреса сервисов. VMM будет использовать предоставленный индекс, чтобы найти адрес желаемого сервиса из сервисной таблицы. Например, если вы хотите вызвать GetVersion, которая является первым сервисом, вы должны указать 0 (индексы начинаются с нуля). Реальный механизм вызовов сервисов VxD использует int 20h. Ваш код будет выглядеть как int 20, за которым следует двойное слово, состоящее из ID устройства и индекса сервиса. Например, если вы хотите вызвать сервис под номером 1, который экспортируется VxD с ID 000Dh, код будет выглядеть так:

```
int 20h
dd 000D0001h
```

Вернее слово двойного слова, которое следует за инструкцией int 20h, содержит ID устройства. Нижнее слово индекс элемента таблицы сервисов. Когда генерируется int20h, VMM получает контроль и проверяет двойное слово, следующее за инструкцией прерывания. Затем он извлекает ID устройства и использует его, чтобы найти VxD, а после использует индекс чтобы найти требуемого сервиса этого VxD.

Вы можете видеть, что эти операции пожирают время. VMM должна потратить время, чтобы найти VxD и адрес желаемого сервиса. Поэтому VMM немножко жульничает. После первого успешного вызова int 20h, VMM сохраняет связь. Это выглядит как замещение int20h и последующего слова на прямой вызов сервиса. Поэтому вышеприведенный кусок кода будет трансформирован в:

```
call dword ptr [VxD_Service_Address]
```

Этот прием работает, так как int 20h + dword занимает 6 байтов, что в точности равно инструкции call dword ptr. Поэтому последовательные вызовы быстры и эффективны. Этот метод имеет свои за и против. Положительным является то, что снижается загрузка VMM и VxD загрузчика, потому что они не должны фиксировать все сервисные вызовы VxD во время загрузки. Вызовы, которые никогда не загружались, останутся немодифицированными. Среди недостатков данного подхода можно назвать то, что он делает невозможным выгрузить статические VxD, сервисы которого используются, так как в противном случае, другие VxD, использующие сервисы выгруженного драйвера, завесили бы систему обращения к неправильным адресам памяти. Нет механизма "разлиновки". Из этого неизбежно следует, что динамические VxD не подходят для предоставления каких-либо сервисов.

Callback'и: callback'и или callback'овые функции - это функции в VxD, которые существуют. Callback'и не являются открытыми как сервисы. Они являются приватными функциями, чьи адреса VxD передает другим VxD в специальных ситуациях. Например, когда VxD обрабатывает хардварное прерывание, теми его функциями, которые могут вызвать page faults, не могут пользоваться другие VxD. VxD может дать адрес одной из своей собственных функций (callback) VMM, чтобы тот мог вызвать функцию, когда он может допускать page faults, и тогда VxD может продолжить свою работу когда вызывается его callback-функция. Идея обратного вызова (callback) используется не только в VxD. Многие Windows API используют ее. Вероятно, лучший пример - это оконная процедура. Вы указываете адрес процедуры окна в структуре WNDCLASS или WNDCLASSEX и передаете ее Windows с помощью функции RegisterClass или RegisterClassEx. Windows будет вызывать вашу процедуру окна, когда для этого окна появятся сообщения. Другим примером может являться hook-процедуры. Ваше приложение дает Windows адрес hook-процедуры, чтобы Windows могла вызвать ее, когда происходит событие, которое интересно приложению.

Приведенных три метода служат для взаимодействия между VxD. Если также интерфейсы для Windows-, V86- и Win32-приложений. Я расскажу о интерфейсе VxD для win32 приложений в следующих нескольких туториалах.

VxD. Урок 2. Менеджер виртуальных машин

Автор: lczelion, пер. Aquila

Дата: 2002-06-06

Менеджер виртуальных машин (VMM) - это настоящая операционная система, лежащая в основании Windows 95. Она создает и поддерживает рабочую среду для управления виртуальными машинами.

Она также предоставляет множество важных сервисов другим VxD. Три главных сервиса следующие:

- Управление памятью
- Обработка прерываний
- Переключение ветвей

Управление памятью

VMM использует способность Intel 80386 и более поздних процессоров создавать 32-битное виртуальное адресное пространство для системной VM. Она разделяет адресное пространство на четыре различных области.

- V86-область, начинающаяся с адреса 0h до 10FFEFh. Этот регион принадлежит выполняющийся в данный момент виртуальной машине.
- Приватная область памяти приложения от 4MB до 2GB. Эта область, в которой выполняется win32-приложение. Каждый win32-процесс будет иметь свои собственные приватные 2GB (минус 4 GB).
- Общая область приложений от 2 GB до 3 GB. Эта область общая для всех приложений в системной машине. Эта область, где находятся системные DLL (user32, kernel32 и gdi32). Все Win16-приложения также выполняются здесь, так как могут читать/писать из и в другие win16-приложения. В этой области win16-приложения могут видеть все другие Win16-приложения. Промэппированные файлы также находятся здесь, как и память, зарезервированная с помощью вызовов DPMI.
- Общая системная область от адреса 3GB до 4GB, где находятся VMM и VxD.

VMM предоставляет три типа сервисов памяти VxD:

- Сервисы, основанные на страницах памяти. Этот тип сервисов резервирует/управляет памятью, организованную в страницы по 4 KB. Это самый низкий уровень сервисов памяти, который может быть доступен. Все другие сервисы памяти используют эти сервисы в качестве основы.
- Сервисы "кучи". Управляет меньшими блоками памяти. Это самый высокий уровень управления памятью.
- Списковые сервисы. Блоки памяти фиксированного размера подходят для воплощения связанных списков.

Обработка прерываний

Прерывания защищенного режима сгруппированы в Interrupt Descriptor Table (IDT). VMM управляет IDT виртуальных машин с помощью VxD. Как правило, VMM обрабатывает практически все элементы IDT'ов. Она предоставляет обработчики прерываний первого уровня, которые сохраняют состояние прерванной программы в стеке и передают управление обработчикам прерываний второго уровня, которые могут быть предоставлены различными VxD для собственно обработки прерывания. Когда обработчик второго уровня заканчивает свою работу, он передает управление специальной процедуре, восстанавливающей состояние прерванной программы и продолжает выполнение в месте, где прервалось выполнение.

Вышеприведенное описание сильно упрощено. Восстановление может не быть немедленным, потому что порция времени, выделенная прерванной виртуальной машине, может истечь. VxD могут устанавливать обработчики прерываний с помощью сервисов VMM, таких как Set_PM_Int или Hook_V86_Int_Chain. VxD не должны модифицировать элементы IDT напрямую (но вы можете это делать, если вы знаете, что вы делаете).

Управление ветвями

VMM использует два компонента для воплощения упреждающей многозадачности между ветвями и виртуальными машинами.

- основной планировщик
- разделитель машинного времени или вторичный планировщик

Задача основного планировщика - выбор ветви с наибольшим приоритетом, которую нужно выполнить. Этот выбор происходит, пока VMM обслуживает прерывание (такое как прерывание таймера). Результат определяет какая ветвь/виртуальная машина получит контроль, когда VMM прекратит обработку прерывания. Основной планировщик работает по правилу "все или ничего". Либо ветвь будет запущена, либо нет. Выбирается только одна ветвь. VMM и другие VxD могут повышать/понижать приоритет выполнения ветвей, VMM повысит приоритет обработчика прерывания, чтобы у него был шанс выполнить свою задачу в максимально короткий срок.

Вторичный планировщик использует сервисы основного планировщика, чтобы резервировать время CPU для ветвей, которые разделяют высочайший приоритет выполнения, путем выделения каждой ветви кванта машинного времени (time-slice). Когда ветвь выполняется, пока ее time-slice не истек, вторичный планировщик повышает приоритет выполнения следующей ветви, чтобы она была выбрана основным планировщиком для запуска.

Вы можете получить больше информации по этой теме из "Системного программирования для Windows 95" Вальтера Онеи (Walter Oney's Systems Programming for Windows 95) и документации по Windows 95 DDK.

VxD. Урок 3. Каркас драйвера

Автор: lczelion, пер. Aquila

Дата: 2002-06-06

Теперь, когда вы знаете кое-что о VMM и VxD, мы должны изучить как программировать VxD. Вам необходимо иметь Windows 95/98 Device Driver Development Kit. Windows 95 DDK доступно только подписчикам MSDN. Тем не менее, Windows 98 DDK доступно без каких-либо гарантий со стороны Микрософта. Вы также можете использовать Windows 98 DDK, чтобы разрабатывать VxD, даже если ориентированы на WDM. Вы можете скачать Windows 98 DDK с <http://www.microsoft.com/hwdev/ddk/>

Вы можете скачать весь пакет, около 30 мегабайт, или же вы можете выборочно скачать только то, что вам нужно. Если вы поступите так, то не забудьте скачать документацию к Windows 95 DDK, которая включена в other.exe.

Windows 98 DDK содержит MASM версии 6.11d. Вам следует проапгрейдить ее до последней версии. Где скачать свежую версию, можно узнать на моей странице.

Windows 9x DDK содержит некоторые основные заголовочные файлы, которые не включены в MASM32.

Вы можете скачать пример для этого tutorials [здесь](#).

Формат LE

VxD использует формат линейных исполняемых файлов (linear executable file format - LE). Этот формат был спроектирован для OS/2 версии 2.0. Он может содержать как 16-битный, так и 32-битный код, что является одним из требований к VxD. Помните, что VxD начали свою историю еще в эпоху Windows 3.x. В то время Windows загружалась из DOS'a, поэтому VxD должны были выполнять определенные действия в реальном режиме, прежде чем Windows переключала машину в защищенный режим. 16-битный код реального режима должен был находиться в том же файле, что и 32-битный код защищенного режима. Поэтому файловый LE-формат был очевидным выбором. Драйвера Windows NT не имеют дела с реальным режимом, поэтому им не надо использовать LE-формат. Вместо этого они используют PE-формат.

Код и данные в LE-файле хранятся в сегментах с различными атрибутами выполнения. Они приводятся ниже.

- LCODE - 'page-locked' код и данные. Этот сегмент "заперт" в памяти. Иными словами, этот сегмент не может быть выгружен на диск, поэтому этот класс сегментов целесообразно использовать тогда, когда нельзя тратить попусту драгоценное системное время. Код и данные должны всегда присутствовать в памяти. Особенно это нужно для обработчиков аппаратных прерываний.
- PCODE - выгружаемый код. Выгрузка на диск и загрузка кода в память регулируется VMM. Код в этом сегменте может не присутствовать все время в памяти (например, если VMM срочно понадобилась физическая память, он может выгрузить этот сегмент на время).
- PDATA - то же самое, только это сегмент с данными, а не с кодом.
- ICODE - код только для инициализации. Код в этом сегменте используется только во время инициализации VxD. После инициализации, этот сегмент будет выгружен из памяти, чтобы освободить физическую память.
- DBCODE - код и данные только для отладки. Код и данные в этом сегменте используются только тогда, когда вы запускаете VxD под отладчиком. Например, код может содержать обработчик для контрольного сообщения Debug_Query.
- SCODE - статические код и данные. Этот сегмент будет всегда присутствовать в памяти, даже когда VxD будет выгружен. Этот сегмент особенно полезен для динамических VxD, так как они

могут выгружаться много раз во время рабочей Windows-сессии, в то время как требуется, чтобы сохранялось их конфигурация/состояние.

- RCODE - инициализационные код и данные реального режима. Этот сегмент содержит 16-битные код и данные для инициализации в реальном режиме.
- 16ICODEUSE16 - инициализационные данные защищенного режима. Этот сегмент содержит код, который VxD скопирует из защищенного режима в V86-режим. Например, если вы хотите скопировать какой-то код V86-режима, этот код должен находиться в этом сегменте. Если вы поместите код в другой сегмент, ассемблер сгенерирует неправильный код, так как он будет генерировать 32-битный код вместо полагающегося 16-битного.
- MCODE - "запертые" строки сообщений. Этот сегмент содержит строки сообщений, которые скомпилированы с помощью макросов сообщений VMM. Это поможет вам создать интернациональные версии вашего драйвера.

Все это не значит, что ваш VxD обязан иметь все эти сегменты. Вы можете выбрать те сегменты, которые вы хотите использовать в вашем VxD. Например, если ваш VxD не имеет инициализации реального режима, у него не будет секции RCODE. Как правило, вы будете использовать LCODE, PCODE и PDATA. За вами, как за создателем VxD, остается выбор нужных сегментов. Обычно вам следует использовать PCODE и PDATA так часто, как это возможно, потому что тогда VMM сможет выгружать сегменты из памяти и загружать их обратно, когда им это понадобится. Вы должны использовать LCODE для обработчиков аппаратных прерываний и сервисов, которые будут вызываться этими обработчиками.

Вам не нужно использовать эти классы сегментов напрямую. Вы должны объявить сегменты на основе этих классов. Объявления сегментов находятся в файле определения модуля. (.def). Вот пример такого файла для VxD:

```
VXD FIRSTVXD

SEGMENTS
  _LPTEXT    CLASS 'LCODE'    PRELOAD NONDISCARDABLE
  _LTEXT     CLASS 'LCODE'    PRELOAD NONDISCARDABLE
  _LDATA     CLASS 'LCODE'    PRELOAD NONDISCARDABLE

  _TEXT      CLASS 'LCODE'    PRELOAD NONDISCARDABLE
  _DATA      CLASS 'LCODE'    PRELOAD NONDISCARDABLE
  CONST      CLASS 'LCODE'    PRELOAD NONDISCARDABLE
  _TLS       CLASS 'LCODE'    PRELOAD NONDISCARDABLE

  _BSS       CLASS 'LCODE'    PRELOAD NONDISCARDABLE
  _LMGTABLE   CLASS 'MCODE'    PRELOAD NONDISCARDABLE IOPL
  _LMSGDATA   CLASS 'MCODE'    PRELOAD NONDISCARDABLE IOPL
  _IMSGTABLE  CLASS 'MCODE'    PRELOAD DISCARDABLE IOPL

  _IMSGDATA   CLASS 'MCODE'    PRELOAD DISCARDABLE IOPL
  _ITEXT      CLASS 'ICODE'    DISCARDABLE
  _IDATA      CLASS 'ICODE'    DISCARDABLE
  _PTEXT      CLASS 'PCODE'    NONDISCARDABLE

  _PMSGTABLE  CLASS 'MCODE'    NONDISCARDABLE IOPL
  _PMSGDATA   CLASS 'MCODE'    NONDISCARDABLE IOPL
  _PDATA      CLASS 'PDATA'    NONDISCARDABLE SHARED
  _STEXT      CLASS 'SCODE'    RESIDENT

  _SDATA      CLASS 'SCODE'    RESIDENT
  _DBOSTART   CLASS 'DBOCODE'  PRELOAD NONDISCARDABLE CONFORMING
  _DBOCODE    CLASS 'DBOCODE'  PRELOAD NONDISCARDABLE CONFORMING
  _DBODATA    CLASS 'DBOCODE'  PRELOAD NONDISCARDABLE CONFORMING

  _16ICODE    CLASS '16ICODE'  PRELOAD DISCARDABLE
  _RCODE      CLASS 'RCODE'

EXPORTS
  FIRSTVXD_DDB @1
```

Первое утверждение задает имя VxD. Имя VxD должно быть задано в верхнем регистре. Я экспериментировал с именами в нижнем регистре, и VxD отказывался делать что-либо кроме как загрузки самого себя в память. Затем идут определения сегментов. Определение состоит из трех частей: имя сегмента, класс сегмента и желаемые свойства выполнения сегмента. Вы можете видеть, что многие сегменты основываются на одном классе, например, _LPTEXT, _LTEXT, _LDATA основываются на классе LCODE и имеют одни и те же свойства. Этих сегменты объявлены для того, чтобы сделать программирование легче. Например, LCODE может содержать и код и данные. Программисту будет проще поместить данные _LDATA, а код в _LTEXT. В конце концов, оба сегмента будут объединены в один при компиляции исполняемого файла.

VxD экспортирует один и только один символ - это device descriptor block (DDB). Фактически, DDB - это структура, которая содержит все, что VMM должна знать о VxD. Вы должны экспортировать DDB в файле определения модуля. Большую часть времени вы будете использовать вышеприведенный .DEF файл в своих новых VxD-проектах. Вам следует только изменить имя VxD в первой и последней линиях .DEF-файла. Определения сегментов - это перегиб в asm'овском VxD-проекте. Вы получите много предупреждений, но это будет компилироваться.

Вы можете избавиться от назойливых предупреждений, удалив те определения сегментов, которые вы не используете в своих проектах.

vmm.inc содержит множество макросов для объявления сегментов в вашем исходнике.

```

_LTEXT    VxD_LOCKED_CODE_SEG

_PTEXT    VxD_PAGEABLE_CODE_SEG
_DBCODE   VxD_DEBUG_ONLY_CODE_SEG

_ITEXT    VxD_INIT_CODE_SEG

_LDATA    VxD_LOCKED_DATA_SEG

_IDATA    VxD_IDATA_SEG

_PDATA    VxD_PAGEABLE_DATA_SEG

_STEXT    VxD_STATIC_CODE_SEG

_SDATA    VxD_STATIC_DATA_SEG

_DBODATA  VxD_DEBUG_ONLY_DATA_SEG

_16ICODE  VxD_16BIT_INIT_SEG

_RCODE    VxD_REAL_INIT_SEG

```

У каждого макроса есть необходимая завершающая часть. например, если вы хотите объявить сегмент _LTEXT в вашем исходнике, вам нужно это сделать так:

```
VxD_LOCKED_CODE_SEG <поместите сюда свой код> VxD_LOCKED_CODE_ENDS
```

Каркас VxD

Теперь, когда вы знаете о сегментах в LE-файлах, мы можем перейти к исходнику. Вы сможете заметить, что макросы очень часто применяются в VxD-программировании, так как они того стоят, позволяя упростить программисту работу и, иногда, сделать исходник более портатбельным. Если это вам интересно, вы можете прочитать определения этих макросов в различных заголовочных файлах, таких как vmm.inc.

Вот исходник каркас VxD:

```

.386p
include vmm.inc

DECLARE_VIRTUAL_DEVICE FIRSTVXD,1,0, FIRSTVXD_Control,
```

```

    UNDEFINED_DEVICE_ID, UNDEFINED_INIT_ORDER

    Begin_control_dispatch FIRSTVXD
    End_control_dispatch FIRSTVXD

end

```

На первый взгляд, исходник не похож на ассемблерный код. Это происходит из-за использования макросов. Давайте проанализируем этот исходный код и вы вскоре поймете его.

```
.386p
```

Указывает ассемблеру, что мы хотим использовать набор инструкций 60386, включая привилегированные инструкции. Вы также можете использовать .486p или .586p.

```
include vmm.inc
```

Вы должны включать vmm.inc в каждый исходник VxD, так как он содержит определения макросов, которые вы будете использовать. Вы можете подключить другие файлы, если они вам потребуются.

```

DECLARE_VIRTUAL_DEVICE FIRSTVXD,1,0, FIRSTVXD_Control,
    UNDEFINED_DEVICE_ID, UNDEFINED_INIT_ORDER

```

Как было сказано раньше, VMM получает всю необходимую информацию о том, что ему необходимо знать о VxD из DDB. Это структура, которая содержит жизненно важную информацию о VxD, такую как имя VxD, ID устройства, входные адреса VxD сервисов (если они есть) и так далее. Вы можете найти эту структуру в vmm.inc. Она определена как VxD_Desc_Block. Вы экспортируете эту структуру в .DEF-файле. В этой структуре 22 параметра, но, как правило, вы будете использовать только некоторые из них. Поэтому vmm.inc содержит макрос, которое инициализировать и заполнять параметры структуры за вас. Это макрос называется DECLARE_VIRTUAL_DEVICE. Он имеет следующий формат:

```

Declare_Virtual_Device Name, MajorVer, MinorVer, CtrlProc, DeviceID, \
    InitOrder, V86Proc, PMProc, RefData

```

Вы можете заметить, что имена в VxD-исходнике не зависят от регистра. Вы можете использовать символы верхнего или нижнего регистра или их комбинацию. Давайте проанализируем каждый из членов Declare_virtual_device.

- Имя - имя VxD. Максимальная длина - 8 символов. Оно должно быть введено в верхнем регистре. Имя должно быть уникальным среди всех VxD системы. Макрос также использует имя, чтобы создать имя DDB, прибавляя '_DDB' к имени VxD. Поэтому, если вы используете 'FIRSTVXD' в качестве имени своего драйвера, макрос Declare_Virtual_Device объявит имя DDB как FIRSTVXD_DDB. Помните, что вы также должны экспортировать DDB в .DEF-файле.
- MajorVerand, MinorVer - основная и дополнительная версии VxD.
- CtrlProc - имя контрольной процедуры устройства вашего VxD. Контрольная процедура устройства (device control procedure) - это функция, которая получает и обрабатывает контрольные сообщения. Вы можете считать эту процедуру аналогом процедуры окна. Так как мы используем макрос Begin_Control_Dispatch, чтобы создать нашу контрольную процедуру устройства, нам следует использовать стандартное имя вида VxDName_Control. Begin_Control_Dispatch прибавляет '_Control', к имени, которое ему передается (и мы обычно передаем ему имя VxD), поэтому нам следует указывать имя нашего VxD в параметре CtrlProc с прибавленным к нему '_Control'.
- DeviceID - 16-битное уникальное значение VxD. ID потребуется вам только тогда, если ваш VxD должен обрабатывать одну из следующих ситуаций.
- Ваш VxD экспортирует VxD сервисы для использования другими VxD. Так как интерфейс int20 использует device ID, чтобы обнаруживать и находить VxD, наличие уникального идентификатора является обязательным.

- Ваш VxD оповещает о своем существовании приложения реального режима во время инициализации через int 2Fh, функция 1607h.
- Какие-то программы реального режима (TSR) будут использовать прерывание 2Fh, функцию 1605h, чтобы загрузить ваш VxD.
- Если VxD не нуждается в уникальном device ID, вы можете указать в этом поле UNDEFINED_DEVICE_ID. Если вам требуется уникальное ID, вам нужно попросить его у Microsoft'a.
- InitOrderInitialization - порядок загрузки VxD. У каждого VxD есть свой загрузочный номер. Например:

```
VMM_INIT_ORDER      EQU 00000000H
DEBUG_INIT_ORDER    EQU 00000000H
DEBUGCMD_INIT_ORDER EQU 00000000H

PERF_INIT_ORDER      EQU 00090000H
APM_INIT_ORDER       EQU 00100000H
```

- Вы можете видеть, что VMM, DEBUG и DEBUGCMD - это первые VxD, которые загружаются, за ними следуют PERF и APM. VxD с наименьшим значением загружается первым. Если вашему VxD требуются сервисы других VxD во время инициализации, вам следует указать значение данного поля большее, чем у VxD, сервисы которого вам потребуются. Если вашему VxD порядок загрузки не важен, укажите UNDEFINED_INIT_ORDER.
- V86Proc и PMProc - VxD может экспортировать API, для использования программами V86 и защищенного режима. V86Proc и PMProc задают адреса этих API. Помните, что VxD существует в основном для управления виртуальными машинами, в том числе и теми, что отличаются от системной виртуальной машины. Вот почему VxD зачастую предоставляют поддержку API для DOS-программ и программ защищенного режима. Если вы не экспортируете эти API, вы можете пропустить эти поля.
- RefDataReference - данные, используемые Input Output Supervisor (IOS). Единственным случаем, когда вам нужно будет использовать это поле - это когда вы программируете драйвер, работающий с IOS.

Затем идет Begin_Control_Dispatch.

```
Begin_control_dispatch FIRSTVXD
End_control_dispatch FIRSTVXD
```

Этот макрос и его заключительная часть определяет контрольную процедуру устройства, которая будет вызываться при поступлении контрольных сообщений. Вы должны указать первую половину имени этой процедуры, в нашем примере мы используем 'FIRSTVXD'. Макрос прибавит _Control к имени, которое вы укажете. Это им должно совпадать с тем, что вы указали в параметре CtrlProc, передаваемый макросу Declare_virtual_device. Процедура всегда находится в "запертом" сегменте (VxD_LOCKED_CODE_SEG). Вышеприведенная процедура не делает ничего. Вы должны указать, какие контрольные сообщения должны обрабатываться вашим VxD и функции, которые будут их обрабатывать. Для этих целей используется макрос Control_Dispatch.

```
Control_Dispatchmessage, function
```

Например, если ваш VxD обрабатывает только сообщение Device_Init, контрольная процедура устройства будет выглядеть так:

```
Begin_Control_Dispatch FIRSTVXD
Control_Dispatch Device_Init, OnDeviceInit
End_Control_DispatchFIRSTVXD
```

OnDeviceInit - это имя функции, которая будет обрабатывать сообщение Device_Init. Вы можете назвать эту функцию как угодно. Вы заканчиваете VxD заключительной директивой.

Подводя резюме, можно сказать, что VxD, как минимум, должно иметь DDB и контрольную процедуру устройства. Вы объявляете DDB с помощью макроса Declare_Virtual_Device и

контрольную процедуру устройства с помощью макроса `Begin_Control_Dispatch`. Вы должны экспортировать DDB, указав его имя в директиве `EXPORT` в `.DEF`-файле.

Компилирование VxD

Процесс компиляции такой же, как и при компиляции обычного win32-приложения. Вы направливаете `ml.exe` на `asm`-исходник, а затем линкуете объектник с помощью `link.exe`. Есть только отличия в параметрах, передаваемых `ml.exe` и `link.exe`.

```
ml-coff -c -Cx -DASM6 -DBLD_COFF -DIS_32 firstvxd.asm
```

`-coff` - указывает объектный формат COFF

`-c` - только ассемблирование. Вызов линкера не производится, так как мы будем вызывать `link.exe` вручную.

`-Cx` - сохранять регистр публичных, внешних имен.

`-D` - определяет текстовый макрос. Например, `-DBLD_COFF` определяет текстовый макрос `BLD_COFF`, который будет использоваться в ассемблировании. Если вы знакомы с `c`-программированием, это идентично:

```
#define BLD_COFF

#define IS_32
#define ASM6

link -vxd -def:firstvxd.def firstvxd.obj
```

`-vxd` указывает, что мы хотим создать VxD из объектного файла.

`-def:<.DEF файл>` задает имя файла определения модуля VxD.

Я считаю более правильным использовать `make`-файлы, но вы также можете создать `bat`-файл, чтобы автоматизировать компиляцию. Вот мой `make`-файл.

```
NAME=firstvxd

$(NAME).vxd:$(NAME).obj
    link -vxd -def:$(NAME).def $(NAME).obj

$(NAME).obj:$(NAME).asm
    ml -coff -c -Cx -DASM6 -DBLD_COFF -DIS_32 $(NAME).asm
```

VxD. Урок 4. Основы программирования

Автор: lczelion, пер. Aquila

Дата: 2002-06-06

Мы знаем, как создать VxD, который не делает ничего. В этом tutorialе мы сделаем его более функциональным, добавив обработчики контрольных сообщений.

Инициализация и завершение VxD

Есть два типа VxD: статический и динамический. Каждый тип отличается методом загрузки. Они также получают разные загрузочные и завершающие сообщения.

Статический VxD:

VMM загружает статический VxD когда:

- Резидентные программы реального режима обращаются к прерыванию int 2Fh, 1605h, чтобы загрузить его.
- VxD указан в регистре в следующем ключе:

```
HKEY_LOCAL_MACHINE\System\CurrentControlSet\Services\VxD\key\StaticVx
```

- VxD указан в system.in в секции [386enh]:

```
device=pathname
```

Во время периода разработки, я предполагаю, что вы загружаете VxD из system.ini, потому что если в вашем VxD будет ошибка, из-за которой Windows не сможет загрузиться, вы сможете отредактировать system.ini из DOS'a. Вы не сможете ничего сделать, если VxD прописываться в регистре. (Можно отредактировать регистр с помощью regedit.exe, но под ДОСом это будет сложнее, чем под виндами - прим. Aquila).

Когда VMM загружает ваш статический VxD, ваш VxD получить три системных контрольных сообщения в следующем порядке:

- Sys_Critical_Init VMM посылает это контрольное сообщение после переключения в защищенный режим, но прежде, чем будут разрешены прерывания. Большинство VxD не нуждается в обработке этих сообщений, кроме тех случаев, когда:
- Ваш VxD перехватывает некоторые прерывания, которые будут вызываться другими VxD или программа. Так как прерывания запрещены, когда вы обрабатываете эти контрольные сообщения, вы можете быть уверенным, что прерывания, которые вы перехватываете, не будут вызываться в то время, когда вы их перехватываете.
- Ваш VxD предоставляет некоторые VxD сервисы, которые будут вызываться во время инициализации другими VxD. Например, некоторым VxD, загружающимся после вашего VxD, может потребоваться вызвать один из сервисов вашего VxD во время обработки контрольного сообщения Device_Init. Так как сообщение Sys_Critical_Init посылается перед сообщением Device_Init, вы должны инициализировать ваши сервисы во время сообщения Sys_Critical_Init.
- Если вы обрабатываете это сообщение, вам следует проводить инициализацию так быстро, как это возможно, чтобы предотвратить потерю вызовов хардварных прерываний (которые, как вы помните, запрещены).

- Device_Init - VMM посылает контрольное сообщение после того, как прерывания были разрешены. Большинство VxD проводят инициализацию в качестве ответа на это сообщение. Так как прерывания разрешены, можно проводить объемные по времени операции без опасения того, что хардварные прерывания будут потеряны. Вам следует проводить инициализацию здесь (если потребуется).

- Init_Complete - после того, как все VxD обработают сообщение Device_Init, но прежде чем VMM освободит все инициализационные сегменты (классы ICODE и RCODE), VMM посылает это контрольное сообщение. Мало VxD требуется обрабатывать это сообщение.

Ваше VxD должно очищать флаг переноса, если инициализация прошла успешно, в противном случае вы должны установить флаг переноса. Вам не нужно обрабатывать ни одно из этих сообщений, если ваш VxD не требует инициализации.

Когда наступает время прервать выполнение статического VxD, VMM посылает следующие контрольные сообщения:

- Когда ваш VxD получает это сообщение, Windows 95 находится в стадии завершения работы. Все другие виртуальные машины, кроме системной виртуальной машины уже уничтожены. Тем не менее CPU еще находится в защищенном режиме и запускать код реального режима на виртуальной машине еще безопасно. Kernel32.dll в это время уже выгружен.

- Sys_Critical_Exit2 - ваш VxD получит это сообщение, когда все VxD уже обработали System_Exit2 и прерывания запрещены.

Большинство VxD не нуждается в обработке этих двух сообщений, кроме тех случаев, когда вы хотите подготовить систему к переводу в реальный режим. Вы должны знать, что когда Windows 95 завершает работу, она входит в реальный режим. Поэтому если ваш VxD сделал что-то, что может сделать систему нестабильной в этом режиме, ему следует восстановить изменения.

Вы можете задать вопрос, почему эти два сообщения имеют на конце "2"? Помните, что когда VMM загружает статические VxD, она загружает VxD с наименьшим значением загрузочного порядка, чтобы VxD могли полагаться на сервисы VxD, загружаемых раньше них. Например, если VxD2 полагается на сервисы VxD1, она должна указать ее инициализационный порядок большим, чем порядок VxD1. Загрузочный порядок должен быть:

```
..... VxD1 ==> VxD2 ==> VxD3 .....
```

Теперь, во время выгрузки, становится понятным, почему VxD, инициализированные раньше, должны и выгружаться раньше, чтобы они могли все еще могли вызывать сервисы VxD, которые загружались до них. В вышеприведенном примере, порядок должен быть следующим:

```
.... VxD3 ==> VxD2 ==> VxD1.....
```

В вышеприведенном примере, если VxD2 вызывал какие-то сервисы VxD1 во время инициализации, он все еще может нуждаться в них во время выгрузки. System_Exit2 и Sys_Critical_Exit2 шлются в порядке, обратном порядку инициализации. Это означает, что когда VxD2 получает эти сообщения, VxD1 еще не провел деинициализацию и он все еще может нуждаться в сервисах VxD1. Сообщения System_Exit и Sys_Critical_Exit не шлются в обратном порядке. Это означает, что когда вы получаете эти два сообщения, вы не можете быть уверенным, что вы все еще можете вызывать сервисы VxD, который загружался до вас. Эти сообщения не следует использовать для новых VxD. Есть еще два сообщения выхода:

- Device_Reboot_Notify2 - уведомляет VxD о том, что VMM собирается перезагрузить систему. Прерывания все еще доступны.

- Crit_Reboot_Notify2 - уведомляет VxD о том, что VMM собирается перезагрузить систему. Прерывания запрещены.

Теперь вы можете предположить, что существуют сообщения Device_Reboot_Notify и Crit_Reboot_Notify, но они посылаются не в порядке, обратном порядку инициализации.

Динамический VxD:

Динамические VxD могут динамически загружаться и выгружаться во время рабочих сессий Windows 95. Эта возможность не доступна под Windows 3.x. Основной целью динамических VxD является поддержка динамической переконфигурации железа, таких как устройств Plug'n'Play. Тем не менее, вы можете их загружать/выгружать из вашего win32-приложения, делая их идеальными ring-0'выми расширениями вашего приложения.

Пример в предыдущем tutorialе был статическим VxD. Вы можете сконвертировать этот пример в динамический VxD, добавив ключевое слово 'DYNAMIC' к VXD-выражению в .DEF-файле.

```
VXD FIRSTVXD DYNAMIC
```

Вот и все, что вы должны сделать, чтобы переконвертировать статический VxD в динамический. Динамические VxD могут быть загружены следующим образом:

- Помещением их в папку \SYSTEM\IOSUBSYS в директории Windows. VxD в этой директории загружаются Input Output Supervisor'ом (IOS). VxD в этой папке должны поддерживать layer device driver'a, поэтому, возможно, это не самая лучшая идея загружать ваш VxD этим путем.
- Используя сервис VxD-загрузчика. VxD-загрузчик - это статический VxD, который может динамически загружать VxD. Вы можете вызывать его сервисы из других VxD или из 16-битного кода.
- Используя CreateFile API из Win32-приложения. Вы указываете динамический VxD, который вы хотите загрузить в следующем формате:

```
\\.\pathname
```

- Например, если вы хотите загрузить динамический VxD под названием FirstVxD, который находится в текущей директории, вам следует указать следующий путь:

```
.data

VxDName db "\\.\FirstVxD.VXD",0
.....
.data?
hDevice dd ?

.....
.code
.....
invoke CreateFile, addr VxDName,0,0,0, FILE_FLAG_DELETE_ON_CLOSE,0

mov hDevice,eax
.....
invoke CloseHandle,hDevice
.....
```

FILE_FLAG_DELETE_ON_CLOSE - флаг, указывающий, что VxD выгружается, когда хэнгл, возвращенный CreateFile, будет закрыт.

Если вы используете CreateFile, чтобы загрузить динамический VxD, VxD должен поддерживать сообщение w32_DeviceIoControl. VWIN32 посылает это контрольное сообщение вашему динамическому VxD, когда он загружается в первый раз через CreateFile. VxD должен вернуть 0 в eax'a в качестве ответа на это сообщение.

Сообщение `w32_DeviceloControl` также посылается, когда приложение вызывает `DeviceloControl API`, чтобы взаимодействовать с `VxD`. Мы изучим интерфейс `DeviceloControl` в следующем разделе.

Динамический `VxD` получает одно сообщение во время инициализации:

```
Sys_Dynamic_Device_Init
```

И одно сообщение во время завершения:

```
Sys_Dynamic_Device_Exit
```

Динамический `VxD` не получает сообщения `Sys_Critical_Init`, `Device_Init` и `Init_Complete`, потому что эти сообщения посылаются во время инициализации системной `VM`. В ином случае, динамический `VxD` получает все другие контрольные сообщения, когда он находится в памяти. Он может делать все, что и статический `VxD`. То есть, хотя при загрузке динамического `VxD` используются совершенно другие механизмы и посылаются другие сообщения инициализации/завершения, он обладает теми же возможностями, что и статический `VxD`.

Другие системные контрольные сообщения

Во время нахождения `VxD` в памяти, он получит много других контрольных сообщений. Некоторые из них относятся к управлению виртуальными машинами, а некоторые к другим событиям. Например, существуют следующие контрольные сообщения, связанные с виртуальными машинами:

- `Create_VM`
- `VM_Critical_Init`
- `VM_Suspend`
- `VM_Resume`
- `Close_VM_Notify`
- `Destroy_VM`

На вас лежит ответственность выбрать, какие из сообщений обрабатывать.

Создание процедур внутри VxD

Вы объявляете процедуру в `VxD` внутри сегмента. Вам следует определить сначала сегмент, а затем поместить внутрь него процедуру. Например, если вы хотите, чтобы ваша функция была в выгружаемом ('pageable') сегменте, вам следует определить сначала сегмент, примерно так:

```
VxD_PAGEABLE_CODE_SEG  
  
[Ваша процедура]  
  
VxD_PAGEABLE_CODE_ENDS
```

Вы можете поместить много процедур внутри сегмента. Вы, как создатель `VxD`, должны решить, в каком сегменте вам следует содержать свои процедуры. Если ваши процедуры должны быть в памяти все время (например, обработчики аппаратных прерываний), поместите их в залоченный сегмент. В противном случае вам придется поместить их в выгружаемый сегмент.

Вы определяете вашу процедуру с помощью макросов `BeginProc` и `EndProc`.

```
BeginProc name  
  
EndProc name
```

name - это имя процедуры. Макрос BeginProc может принимать несколько параметров, вам следует проконсультироваться с документацией Win95 DDK за подробностями. Но большую часть времени вам надо будет передавать только имя процедуры.

Вам следует использовать макросы BeginProc-EndProc, так как они предоставляют больше функциональности, чем proc-endp.

Соглашения в программировании VxD

Использование регистров

VxD может использовать любой регистр общего назначения, FS и GS. Но вам следует избегать модифицирования сегментных регистров. Главным образом, вам не следует менять CS и SS, пока вы не уверены в том, что вы делаете. Вы можете использовать DS и ES так долго, пока вы не забываете восстанавливать их значения, когда вы возвращаетесь. Два флага особенно важны: флаги направления и прерывания. Вам не следует запрещать прерывания на длительный период времени, и если вы модифицируете флаг направления, не забывайте восстанавливать его предыдущее значение перед возвратом.

Передача параметров

Есть два типа передачи данных VxD-сервиса: основанные на регистрах и на стеке. В первом случае вы передаете данные сервисам через различные регистры и вы можете проверить флаг переноса после вызова сервиса, чтобы узнать, прошел ли вызов успешно. Сохранение значений в регистре не гарантируется. В случае с сервисами, принимающими значения через стек, вы загоняете в него параметры и получаете возвращаемое значение в eax. Такие сервисы сохраняют значения ebx, esi, edi и ebp. Большинство из регистровых сервисов ведут свое происхождение от Windows 3.x. Практически всегда вы сможете узнать к какому виду принадлежит тот или иной сервис по его названию. Если имя начинается с подчеркивания, например, '_HeapAllocate', это стековый (C) сервис (кроме нескольких сервисов экспортированных из VWIN32.VXD). Если имя сервиса не начинается с подчеркивания, это регистровый сервис.

Вызов VxD-сервисов

Вызов VMM- и VxD-сервисов осуществляется через макросы VMMCall и VxDCall. Оба макроса имеют одинаковый синтаксис. VMMCall используется, когда вам нужно вызвать VxD-сервисы, экспортированные VMM, а VxDCall используется, когда вы вызываете сервисы, экспортированные чем-то другим.

```
VMMCall service ; для вызова регистровых сервисов
VMMCall _service, <argument list> ; для вызова стековых сервисов
```

VMMCall и VxDCall фактически являются int 20h, за которым следует dword, что я уже объяснял в предыдущих tutorиалах, но макросы более удобны. В случае со стековыми сервисами, вы должны заключать список аргументов в угловые скобки.

```
VMMCall _HeapAllocate, <<size mybuffer>, HeapLockedIfDP>
```

`_HeapAllocate` - это стековый сервис. Он принимает два параметра. Мы должны заключить их в угловые скобки. Тем не менее, первый параметр - это выражение, которое макрос может интерпретировать неправильно, поэтому мы помещаем их внутри других угловых скобок.

Плоские адреса

Ассемблер и линкер более старой версии генерировали неправильные адреса, когда вы использовали оператор `'offset'`. Поэтому VxD-программисты используют `'offset flat:'` вместо `'offset'`. `vmi.inc` содержит макрос, которое делает это проще - `'OFFSET32'`. Поэтому, если вы хотите использовать оператор `'offset'`, вам следует использовать `'OFFSET32'`.

Заметьте: я экспериментировал с оператором `'offset'` перед написанием этого tutorials. Он генерировал корректные адреса, поэтому я думаю, что баг был убран в MASM 6.14. Но лучше подстраховаться и использовать `OFFSET32`.

VxD. Урок 5. Message Box

Автор: lczelion, пер. Aquila

Дата: 2002-06-06

В предыдущих туториалах мы изучили основы VxD-программирования. Теперь пришло время применить на практике полученные знания. В этом туториале мы создадим простой статический VxD, который будет отображать message box всякий раз, когда будет создаваться/уничтожаться виртуальная машина.

Скачайте пример здесь (находится в архиве wasm_files.zip: files/recovered/1003005/vxdmessage.zip).

Перехват создания и уничтожения VM

Когда создается виртуальная машина, VMM посылает контрольное сообщение Create_VM всем VxD. Также, когда VM завершает свою работу, она посылает всем VxD сообщение VM_Terminate и VM_Terminate2. Наша задача проста: обработать сообщения Create_VM и VM_Terminate2 в нашей контрольной процедуре устройства. Когда наш VxD получает эти два контрольных сообщения, он отображает message box на экране.

Когда наш VxD получает сообщение Create_VM или VM_Terminate2, ebx содержит хэндл VM. VM-хэндл можно считать уникальным ID виртуальной машины. Каждая виртуальная машина имеет свой уникальный ID (хэндл VM). Вы можете использовать его так же, как вы используете ID процесса, передавая его в качестве параметра сервисам, которым он требуется.

При ближайшем рассмотрении VM-хэндл оказывается 32-битным линейным адресом контрольного блока VM (VMCB). VMCB - это структура, которая содержит некоторую важную информацию о VM. Она определена как:

```
cb_s STRUC
CB_VM_Status      DD ?
CB_High_Linear    DD ?
CB_Client_Pointer DD ?
CB_VMID           DD ?
CB_Signature      DD ?
cb_s ENDS
```

- CB_VM_Status содержит битовые флаги, с помощью которых вы можете узнать о состоянии VM.
- CB_High_Linear - это стартовый линейный адрес зеркала виртуальной машине в общем системном регионе (выше 3 гигабайт). Эта концепция требует объяснения. Под Windows 95, VxD не должен работать с V86-регионом напрямую, вместо этого VMM мэппирует все V86-регионы каждой виртуальной машины в общий системный регион. Когда VxD хочет изменить/обратиться к памяти в V86-регион виртуальной машины, ему следует сделать это, обратившись к соответствующей области в общем системном регионе. Например, если видеопамять находится по адресу 0B000h и вашему VxD требуется обратиться к этому адресу, ему следует добавить значение в CB_High_Linear к 0B8000h и обратиться к этой области. Изменения, которые вы сделаете в зеркале отобразятся в виртуальной машине. Использование зеркала во многих случаях является лучшим вариантом, так как вы можете изменять виртуальную машину, даже если она не является текущей VM.
- CB_Client_Pointer содержит адреса клиентской структуры регистров. Эта структура содержит значения всех регистров прерванного приложения V86- или защищенного режима. Если ваш VxD хочет узнать/модифицировать состояние такого приложения, он может модифицировать поля клиентской структуры регистров и изменения распространяться на приложение, когда VMM продолжит его выполнения.

- CB_VMID - числовой идентификатор виртуальной машины. VMM назначает этот номер, когда создает VM. У системной VM VMID равен одному.
- CB_Signature содержит строку "VMcb". Это поле используется для проверки того, является ли хэндл VM верным.

Отображение MessageBox'a

VxD может использовать сервисы Virtual Shell Device, чтобы взаимодействовать с пользователями. Один такой сервис, который мы используем, называется SHELL_Message. SHELL_Message - это регистровый сервис. Вы можете ему параметры через регистры.

- ebx - хэндл виртуальной машины, которая ответственна за сообщение.
- eax - флаги MessageBox. Вы можете посмотреть их в shell.inc. Они начинаются с MB_.
- ecx - 32-битное линейный адрес сообщения, которое должно быть отображено.
- edi - 32-битный линейный адрес заголовка message box'a.
- esi - 32-битный линейный адрес callback-функций, необходимой для того, чтобы узнать реакцию пользователя. Если вам это не нужно, используйте NULL.
- edx - дополнительные данные, которые будут передаваться callback-функции (если вы указали ее в esi).

По возвращении флаг переноса очищается, если вызов прошел успешно. Флаг переноса устанавливается в противном случае.

ПРИМЕР

```
.386p
include vmm.inc
include shell.inc

DECLARE_VIRTUAL_DEVICE MESSAGE,1,0, MESSAGE_Control,
UNDEFINED_DEVICE_ID, UNDEFINED_INIT_ORDER

Begin_control_dispatch MESSAGE
    Control_Dispatch Create_VM, OnVMCreate

    Control_Dispatch VM_Terminate2, OnVMClose
End_control_dispatch MESSAGE

VxD_PAGEABLE_DATA_SEG
    MsgTitle db "VxD MessageBox",0
    VMCreated db "A VM is created",0
    VMDestroyed db "A VM is destroyed",0

VxD_PAGEABLE_DATA_ENDS

VxD_PAGEABLE_CODE_SEG

BeginProc OnVMCreate
    mov ecx, OFFSET32 VMCreated
CommonCode:
    VMCall Get_sys_vm_handle

    mov eax,MB_OK+MB_ICONEXCLAMATION
    mov edi, OFFSET32 MsgTitle
    xor esi,esi
    xor edx,edx

    VxDCall SHELL_Message
    ret
EndProc OnVMCreate

BeginProc OnVMClose
```

```

        mov ecx,OFFSET32 VMDestroyed
        jmp CommonCode

EndProc OnVMClose
VxD_PAGEABLE_CODE_ENDS

end

```

Анализ:

```

Begin_control_dispatch MESSAGE
    Control_Dispatch Create_VM, OnVMCreate
    Control_Dispatch VM_Terminate2, OnVMClose
End_control_dispatch MESSAGE

```

VxD обрабатывает два контрольных сообщения, CreateVM и VM_Terminate2. После получения сообщения Create_VM, он вызывает процедуру OnVMCreate. А когда он получает сообщение VM_Terminate2, вызывается процедура OnVMC.

```

VxD_PAGEABLE_DATA_SEG

    MsgTitle db "VxD MessageBox",0
    VMCreated db "A VM is created",0
    VMDestroyed db "A VM is destroyed",0

VxD_PAGEABLE_DATA_ENDS

```

Мы помещаем данные в выгружаемый сегмент.

```

BeginProcOnVMCreate
    mov ecx, OFFSET32VMCreated
CommonCode:
    VMCall Get_sys_vm_handle
    mov eax,MB_OK+MB_ICONEXCLAMATION
    mov edi, OFFSET32 MsgTitle
    xor esi,esi
    xor edx,edx
    VxDCall SHELL_Message
    ret
EndProcOnVMCreate

```

Процедура OnVMCreate создается с помощью макросов BeginProc и EndProc. Она помещает параметры для сервиса SHELL_Message в регистры. Так как мы хотим отображать message box в системной VM, мы не можем использовать значения в ebx (которое является хэндлом созданной VM). Вместо этого мы используем VMM-сервис Get_Sys_VM_Handle, чтобы получить хэндл системной виртуальной машины. Этот сервис возвращает хэндл VM в ebx. Мы помещаем адрес сообщения и заголовка в ecx и edi. Нам не нужно знать ответ пользователя, поэтому мы обнуляем esi и edx. Когда все параметры находятся в соответствующих регистрах, мы вызываем SHELL_Message, чтобы отобразить message box.

```

BeginProcOnVMClose
    mov ecx,OFFSET32 VMDestroyed
    jmp CommonCode
EndProcOnVMClose

```

Процедура OnVMClose достаточно проста. Так как она использует идентичный с OnVMCreate код, она инициализирует ecx адресом другого сообщения, а затем переходит к коду внутри OnVMCreate.

Файл определения модуля

```

VXD MESSAGE

SEGMENTS
    _LPTEXT    CLASS 'LCODE'    PRELOAD NONDISCARDABLE
    _LTEXT     CLASS 'LCODE'    PRELOAD NONDISCARDABLE

    _LDATA     CLASS 'LCODE'    PRELOAD NONDISCARDABLE
    _TEXT      CLASS 'LCODE'    PRELOAD NONDISCARDABLE

```


_DATA	CLASS 'LCODE'	PRELOAD NONDISCARDABLE
_CONST	CLASS 'LCODE'	PRELOAD NONDISCARDABLE
_TLS	CLASS 'LCODE'	PRELOAD NONDISCARDABLE
_BSS	CLASS 'LCODE'	PRELOAD NONDISCARDABLE
_LMGTABLE	CLASS 'MCODE'	PRELOAD NONDISCARDABLE IOPL
_LMSGDATA	CLASS 'MCODE'	PRELOAD NONDISCARDABLE IOPL
_IMSGTABLE	CLASS 'MCODE'	PRELOAD DISCARDABLE IOPL
_IMSGDATA	CLASS 'MCODE'	PRELOAD DISCARDABLE IOPL
_ITEXT	CLASS 'ICODE'	DISCARDABLE
_IDATA	CLASS 'ICODE'	DISCARDABLE
_PTEXT	CLASS 'PCODE'	NONDISCARDABLE
_PMSGTABLE	CLASS 'MCODE'	NONDISCARDABLE IOPL
_PMSGDATA	CLASS 'MCODE'	NONDISCARDABLE IOPL
_PDATA	CLASS 'PDATA'	NONDISCARDABLE SHARED
_STEXT	CLASS 'SCODE'	RESIDENT
_SDATA	CLASS 'SCODE'	RESIDENT
_DBOSTART	CLASS 'DBOCODE'	PRELOAD NONDISCARDABLE CONFORMING
_DBOCODE	CLASS 'DBOCODE'	PRELOAD NONDISCARDABLE CONFORMING
_DBODATA	CLASS 'DBOCODE'	PRELOAD NONDISCARDABLE CONFORMING
_16ICODE	CLASS '16ICODE'	PRELOAD DISCARDABLE
_RCODE	CLASS 'RCODE'	

EXPORTS

MESSAGE_DDB @1

Процесс компиляции

```
ml -coff -c -Cx -DASM6 -DBLD_COFF -DIS_32 message.asm
```

```
link -vxd -def:message.def message.obj
```

Инсталляция VxD Поместите message.vxd в директорию \system добавьте следующую линию в секции [386enh] system.ini

```
device=message.vxd
```

Перезагрузите компьютер

Тестирование VxD

Создайте DOS box. Вы увидите message box, отображающий сообщение "A VM is created". Когда вы закроете DOS box, появится окошко с сообщением "A VM is destroyed".

VxD. Урок 6. Интерфейс DeviceIoControl

Автор: lczelion, пер. Aquila

Дата: 2002-06-06

В этом tutorialе мы изучим динамические VxD. В частности, мы узнаем, как создавать, загружать и использовать их.

Скачайте пример здесь (находится в архиве wasm_files.zip: files/article48/shellmsg.zip).

Интерфейсы VxD

Всего VxD предоставляет 4 интерфейса.

- VxD-сервисы
- V86-интерфейс
- Интерфейс защищенного режима
- Win32-интерфейс DeviceIoControl

Мы уже знаем о VxD-сервисах. V86- и PM-интерфейсы являются функциями, которые можно вызывать из V86- и PM-приложений соответственно. Так как V86- и PM-приложения 16-битные, мы не можем использовать эти два интерфейса в win32-приложении. Вместе с Win32 Microsoft добавляет другой интерфейс для win32-приложений, чтобы они могли вызывать сервисы VxD: интерфейс DeviceIoControl.

Интерфейс DeviceIoControl

Чтобы не усложнять, скажу, что интерфейс DeviceIoControl - это путь для win32-приложений вызывать функции VxD. Не путайте функции, вызываемые через DeviceIoControl с VxD-сервисами: это не одно и то же. Например, функция 1, вызываемая через DeviceIoControl, может быть не тем же самым, что VxD-сервис 1. Вы должны считать функции DeviceIoControl как об отдельной группе функций, созданных специально для использования win32-приложениями. Так как это интерфейс, здесь есть две стороны:

Со стороны win32-приложения:

Оно должно вызвать CreateFile, чтобы открыть/загрузить VxD. Если вызов прошел успешно, VxD будет загружен в память и CreateFile возвратит хэндл VxD в eax.

Затем вы вызываете API-функцию DeviceIoControl, чтобы выбрать нужную функцию. DeviceIoControl имеет следующий синтаксис:

```
DeviceIoControl PROTO  hDevice:DWORD,\
                        dwIoControlCode:DWORD,\
                        lpInBuffer:DWORD,\
                        nInBufferSize:DWORD,\
                        lpOutBuffer:DWORD,\
                        nOutBufferSize:DWORD,\
                        lpBytesReturned:DWORD,\
                        lpOverlapped:DWORD
```

- `hDevice` - это хэндл `VxD`, возвращенного `CreateFile`'ом.
- `dwIoControlCode` - это значение, которое указывает операцию, которую должен выполнить `VxD`. Вы должны каким-то образом достать список допустимых значений `dwIoControlCode` для данного `VxD`, прежде, чем вы узнаете, какую операцию вам нужно совершить. Но, как правило, вы будете являться тем, кто программирует `VxD`, поэтому вы будете знать этот список.
- `lpInBuffer` - это адрес буфера, который содержит данные, необходимые `VxD` для выполнения операции, указанные в `dwIoControlCode`. Если операция не требует данных, вы можете передать `NULL`.
- `nInBufferSize` - это размер в байтах данных в буфере, на который указывает `lpInBuffer`.
- `lpOutBuffer` - это адрес буфера, который `VxD` заполнит выходными данными, когда операция будет успешно произведена. Если операция не предполагает выходных данных, это поле должно равняться `NULL`'у.
- `nOutBufferSiz` - это размер в байтах буфера, на который указывает `lpOutbuffer`.
- `lpBytesReturned` - адрес переменной типа `dword`, которая получит размер данных, вписанных `VxD` в `lpOutBuffer`.
- `lpOverlapped` - это адрес структуры `OVERLAPPED`, если вы хотите, чтобы операция была асинхронной. Если вы хотите подождать, пока операция будет выполнена, поместите `NULL` в это поле.

Со стороны VxD:

Он всего лишь обрабатывает сообщение `w32_deviceIoControl`. Когда `VxD` получает это сообщение, его регистры имеют следующие значения:

- `ebx` содержит хэндл `VM`.
- `esi` - это указатель на структуру `DIOCPParams`, которая содержит информацию, которую ему передало win32-приложение.

`DIOCPParams` определен следующим образом:

```
DIOCPParams STRUC
    Internal1          DD ?
    VMHandle           DD ?
    Internal2          DD ?
    dwIoControlCode    DD ?
    lpvInBuffer        DD ?
    cbInBuffer         DD ?
    lpvOutBuffer       DD ?
    cbOutBuffer        DD ?
    lpcbBytesReturned  DD ?
    lpOverlapped       DD ?
    hDevice            DD ?
    tagProcess         DD ?
DIOCPParams ENDS
```

- `Internal1` - это указатель на клиентскую структуру регистров win32-приложения.
- `VMHandle` - комментарий не требуется.
- `Internal2` - это указатель на device descriptor block (DDB).
- `dwIoControlCode`, `lpvInBuffer`, `cbInBuffer`, `lpvOutBuffer`, `cbOutBuffer`, `lpcbBytesReturned`, `lpOverlapped` - это параметры, которые были переданы `DeviceIoControl`.
- `hDevice` - это хэндл ring3-устройства.
- `tagProcess` - это тэг процесса.

Из структуры `DIOCPParams` вы получите всю информацию, переданную win32-приложению.

Ваш `VxD` должен, по крайней мере, обрабатывать `DIOC_Open` (значение, передаваемое в `dwIoControlCode`), которое `VWIN32` пошлет `VxD`, когда win32-приложение вызовет `CreateFile`, чтобы

открыть ваш VxD. Если VxD готов, он должен вернуть 0 в eax, это будет означать, что вызов CreateFile прошел успешно. Если ваш VxD не готов, он должен вернуть ненулевое значение в eax, что будет означать неуспешный вызов CreateFile. Кроме DIOC_Open, VxD получить от VWIN32 код DIOC_Closehandle, когда win32-приложение закроет хэндл устройства.

Минимальный каркас динамического VxD, который можно загрузить с помощью CreateFile:

```
.386p

include vmm.inc
include vwin32.inc

DECLARE_VIRTUAL_DEVICE DYNVXD,1,0, DYNVXD_Control,\
    UNDEFINED_DEVICE_ID, UNDEFINED_INIT_ORDER

Begin_control_dispatch DYNVXD
    Control_Dispatch w32_DeviceIoControl, OnDeviceIoControl
End_control_dispatch DYNVXD

VxD_PAGEABLE_CODE_SEG
BeginProc OnDeviceIoControl
    assume esi:ptr DIOCPParams

    .if [esi].dwIoControlCode==DIOC_Open
        xor eax,eax
    .endif
    ret

EndProc OnDeviceIoControl
VxD_PAGEABLE_CODE_ENDS

end

;-----
;  Module Definition File
;-----

VXD DYNVXD DYNAMIC

SEGMENTS

    _LPTEXT      CLASS 'LCODE'  PRELOAD NONDISCARDABLE
    _LTEXT       CLASS 'LCODE'  PRELOAD NONDISCARDABLE
    _LDATA       CLASS 'LCODE'  PRELOAD NONDISCARDABLE
    _TEXT        CLASS 'LCODE'  PRELOAD NONDISCARDABLE

    _DATA        CLASS 'LCODE'  PRELOAD NONDISCARDABLE
    CONST        CLASS 'LCODE'  PRELOAD NONDISCARDABLE
    _TLS         CLASS 'LCODE'  PRELOAD NONDISCARDABLE
    _BSS         CLASS 'LCODE'  PRELOAD NONDISCARDABLE

    _LMGTABLE    CLASS 'MCODE'  PRELOAD NONDISCARDABLE IOPL
    _MSGDATA     CLASS 'MCODE'  PRELOAD NONDISCARDABLE IOPL
    _MSGTABLE    CLASS 'MCODE'  PRELOAD DISCARDABLE IOPL
    _MSGDATA     CLASS 'MCODE'  PRELOAD DISCARDABLE IOPL

    _ITEXT       CLASS 'ICODE'  DISCARDABLE
    _IDATA       CLASS 'ICODE'  DISCARDABLE
    _PTEXT       CLASS 'PCODE'  NONDISCARDABLE
    _PMSGTABLE   CLASS 'MCODE'  NONDISCARDABLE IOPL

    _PMSGDATA    CLASS 'MCODE'  NONDISCARDABLE IOPL
    _PDATA       CLASS 'PDATA'  NONDISCARDABLE SHARED
    _STEXT       CLASS 'SCODE'  RESIDENT
    _SDATA       CLASS 'SCODE'  RESIDENT

    _DBOSTART    CLASS 'DBOCODE' PRELOAD NONDISCARDABLE CONFORMING
    _DBOCODE     CLASS 'DBOCODE' PRELOAD NONDISCARDABLE CONFORMING
    _DBODATA     CLASS 'DBOCODE' PRELOAD NONDISCARDABLE CONFORMING
    _16ICODE     CLASS '16ICODE' PRELOAD DISCARDABLE
```

```

        _RCODE          CLASS 'RCODE'

EXPORTS

        DYNVXD_DDB      @1

```

Полный пример

Ниже находится исходный код win32-приложения, которое загружает динамический VxD и вызывает функцию в VxD через DeviceIoControl API.

```

; VxDLoader.asm

.386
.model flat,stdcall
include windows.inc
include kernel32.inc

includelib kernel32.lib
include user32.inc
includelib user32.lib

.data
    AppName db "DeviceIoControl",0
    VxDName db "\\.\shellmsg.vxd",0

    Success db "The VxD is successfully loaded!",0
    Failure db "The VxD is not loaded!",0
    Unload db "The VxD is now unloaded!",0
    MsgTitle db "DeviceIoControl Example",0

    MsgText db "I'm called from a VxD!",0
    InBuffer dd offset MsgTitle
              dd offset MsgText

.data?

    hVxD dd ?

.code
start:
    invoke CreateFile,addr
        VxDName,0,0,0,FILE_FLAG_DELETE_ON_CLOSE,0
        .if eax!=INVALID_HANDLE_VALUE
            mov hVxD,eax
            invoke MessageBox,NULL,addr Success,addr
                AppName,MB_OK+MB_ICONINFORMATION
            invoke DeviceIoControl,hVxD,1,addr
                InBuffer,8,NULL,NULL,NULL,NULL
            invoke CloseHandle,hVxD

            invoke MessageBox,NULL,addr Unload,addr
                AppName,MB_OK+MB_ICONINFORMATION
        .else
            invoke MessageBox,NULL,addr Failure,NULL,MB_OK+MB_ICONERROR

        .endif
    invoke ExitProcess,NULL
end start

```

Далее следует исходный код динамического VxD, который загружается vxdloader.asm.

```

; ShellMsg.asm

.386p
include vmm.inc
include wwin32.inc

```

```

include shell.inc

DECLARE_VIRTUAL_DEVICE SHELLMSG,1,0, SHELLMSG_Control,\
    UNDEFINED_DEVICE_ID, UNDEFINED_INIT_ORDER

Begin_control_dispatch SHELLMSG
    Control_Dispatch w32_DeviceIoControl, OnDeviceIoControl
End_control_dispatch SHELLMSG

VxD_PAGEABLE_DATA_SEG
    pTitle dd ?
    pMessage dd ?

VxD_PAGEABLE_DATA_ENDS

VxD_PAGEABLE_CODE_SEG

BeginProc OnDeviceIoControl
    assume esi:ptr DIOCPParams
    .if [esi].dwIoControlCode==DIOC_Open
        xor eax,eax

    .elseif [esi].dwIoControlCode==1
        mov edi,[esi].lpvInBuffer
        ;-----
        ; copy the message title to buffer
        ;-----
        VMMSyscall _lstrlen, <[edi]>
        inc eax
        push eax

        VMMSyscall _HeapAllocate,<eax,HEAPZEROINIT>
        mov pTitle,eax
        pop eax
        VMMSyscall _lstrcpy, <pTitle,[edi],eax>

        ;-----
        ; copy the message text to buffer
        ;-----
        VMMSyscall _lstrlen, <[edi+4]>

        inc eax
        push eax
        VMMSyscall _HeapAllocate,<eax,HEAPZEROINIT>
        mov pMessage,eax

        pop eax
        VMMSyscall _lstrcpy, <pMessage,[edi+4],eax>
        mov edi,pTitle
        mov ecx,pMessage

        mov eax,MB_OK
        VMMSyscall Get_Sys_VM_Handle
        VxDCall SHELL_sysmodal_Message
        VMMSyscall _HeapFree,pTitle,0

        VMMSyscall _HeapFree,pMessage,0
        xor eax,eax
    .endif
    ret

EndProc OnDeviceIoControl
VxD_PAGEABLE_CODE_ENDS

end

```

Анализ:

Мы начнем с VxDLoader.asm.

```
invoke CreateFile,addr VxDName,0,0,0,FILE_FLAG_DELETE_ON_CLOSE,0
.if eax!=INVALID_HANDLE_VALUE
    mov hVxD,eax

    ....
.else
    invoke MessageBox,NULL,addr Failure,NULL,MB_OK+MB_ICONERROR
.endif
```

Мы вызывает CreateFile, чтобы загрузить динамический VxD. Обратите внимание на флаг FILE_FLAG_DELETE_ON_CLOSE. Этот флаг указывает Windows выгрузить VxD, когда VxD-хэндл, возвращенный CreateFile, будет закрыт. Если вызов CreateFile прошел успешно, мы сохраняем хэндл VxD для будущего использования.

```
invoke MessageBox,NULL,addr Success,addr AppName,MB_OK+MB_ICONINFORMATION
invoke DeviceIoControl,hVxD,1,addr InBuffer,8,NULL,NULL,NULL,NULL
invoke CloseHandle,hVxD
invoke MessageBox,NULL,addr Unload,addr AppName,MB_OK+MB_ICONINFORMATION
```

Программа отображает окошко с сообщением, когда VxD загружается/выгружается. Она вызывает DeviceIoControl с dwIoControlCode равным 1 и передает адрес InBuffer в параметре lpInBuffer, а размер InBuffer (8) в nInBufferSize. InBuffer - это массив из двух dword-элементов: каждый элемент содержит текстовую строку.

```
MsgTitle db "DeviceIoControl Example",0
MsgText db "I'm called from a VxD!",0
InBuffer dd offset MsgTitle
          dd offset MsgText
```

Теперь мы переводим наше внимание на VxD. Он обрабатывает только сообщение w32_deviceIoControl. Когда он получает это сообщение, вызывается процедура OnDeviceControl.

```
BeginProc OnDeviceIoControl
    assume esi:ptr DIOCParams

    .if [esi].dwIoControlCode==DIOC_Open
        xor eax,eax
```

OnDeviceIoControl обрабатывает код DIOC_Open, возвращая в eax 0.

```
.elseif [esi].dwIoControlCode==1
    mov edi,[esi].lpvInBuffer
```

Она также обрабатывает контрольный код 1. Вначале она извлекает данные из lpInBuffer, который состоит из двух двойных слов. Для извлечения она помещает адрес массива в edi. Первый dword - это адрес текста, который будет использован для заголовка окна сообщения. Второй dword - это адрес текста сообщения.

```
;-----
; copy the message title to buffer
;-----
VMCall _lstrlen, <[edi]>
inc eax

push eax
VMCall _HeapAllocate,<eax,HEAPZEROINIT>
mov pTitle,eax
pop eax

VMCall _lstrcpy, <pTitle,[edi],eax>
```

Она подсчитывает длину заголовка окна сообщения с помощью вызова VMM-сервиса `_lstrlen`. Значение в `eax`, возвращенное `_lstrlen` - это длина строки. Мы повышаем значение на один, что учесть завершающий ноль. Затем мы занимаем достаточно большой блок памяти, чтобы поместить в него строку с завершающим NULL'ом, с помощью вызова `_HeapAllocate`. Флаг `HEAPZEROINIT` указывает `_heapAllocate` обнулить зарезервированную память. Затем мы копируем строку из адресного пространства win32-приложения в блок памяти, который мы заняли. Затем мы делаем ту же операцию над текстовой строкой, которую мы используем как текст сообщения.

```
mov edi,pTitle
mov ecx,pMessage
mov eax,MB_OK
VMMCall Get_Sys_VM_Handle
VxDCall SHELL_sysmodal_Message
```

Мы сохраняем адреса заголовка и сообщения в `edi` и `ecx`. Помещаем желаемый флаг в `eax`, получаем хэндл системной виртуальной машины с помощью вызова `Get_Sys_VM_handle` и затем вызываем `SHELL_Sysmodal_Message`. `SHELL_Sysmodal_Message` - это системная модальная версия `SHELL_Message`. Она замораживает систему, пока пользователь не ответит на `message box`.

```
VMMCall _HeapFree,pTitle,0
VMMCall _HeapFree,pMessage,0
```

Когда `SHELL_Sysmodal_Message` возвращается, мы можем освободить блок памяти вызовом `_HeapFree`.

Заключение

Интерфейс `DeviceIoControl` делает идеальным использование динамического `VxD`, такого как `ring0-DLL`-расширение вашего win32-приложения.

VxD. Урок 7. Время приложения

Автор: lczelion, пер. Aquila

Дата: 2002-06-06

Скачайте пример (находится в архиве wasm_files.zip: files/article49/vxdexecute.zip).

Немного теории

Время приложений обычно называется "appu time". Это просто означает время, когда системная виртуальная машина достаточно стабильна, чтобы позволить взаимодействие VxD и приложений ring-3, особенно 16-битных. Например, во время приложений VxD может загружать и вызывать функции 16-битных DLL. Это время недоступно под Windows 3.1x. Под Windows 3.1 VxD должен получить адрес требуемой функции в 16-битной DLL и симулировать дальний вызов к этому адресу. Тем не менее, VxD может вызывать только те функции, которые безопасны для прерываний, например PostMessage. Под Windows 95 VxD может вызывать почти любую функцию с помощью времени приложений.

Просто запомните, что когда VxD получает сообщение, что настало время приложений, он может загружать 16-битные DLL и вызывать экспортируемые ими функции. Как же VxD узнает, что настало это время? Он должен зарегистрировать событие времени приложения с помощью VxD оболочки. Когда системная VM будет находиться в стабильном состоянии, Shell VxD вызовет callback-функцию, указанную VxD, когда он регистрировал данное событие. VxD оболочки вызовет вашу callback-функцию только один раз на каждую регистрацию события времени приложения. Это вроде того, как задать работу. Вы идете в рекрутинговое агенство, регистрируете ваше имя/телефонный номер. Затем идете домой. Когда работа будет доступна, агенство уведомит вас об этой хорошей новости. После этого они больше никогда вам не позвонят (если вы еще раз не сходите и не зарегистрируете). Может пройти некоторое время, прежде чем событие времени приложений станет доступно. Эти события не доступны в следующих обстоятельствах:

- во время загрузки системы и завершения работы
- когда системная виртуальная машина находится в критической секции или ждет сигнала

Обработка события времени приложения

Вы можете зарегистрировать событие времени приложения, вызвав функцию _SHEL_CallAtAppyTime, которая имеет следующее определение:

```
VxDCall _SHELL_CallAtAppyTime, <<OFFSET32 pfnCallback>, dwRefData, dwFlags, dwTimeout>
```

• pfnCallBack - плоский адрес функции обратного вызова, которая должна будет вызываться во время события времени приложения. Функция получит два параметра, dwRefData и dwFlags, которые идентичны двум параметрам, переданным _SHELL_CallAtAppyTime. Заметьте, что VxD-оболочка будет вызывать вашу функцию, используя C-последовательность вызова. Иначе говоря, вы должны будете объявить вашу функцию обратного вызова примерно так:

```
BeginProcOnAppyTime, CCALL, PUBLIC
ArgVardwRefData,DWORD ; declare argument
name and type
ArgVar dwFlags, DWORD
EnterProc
```

```
<Ваш код здесь>

LeaveProc

Return
EndProcOnAppyTime
```

- dwRefData - дополнительные данные, которые Shell VxD должна передать вашей callback-функции. Это может быть что угодно.
- dwFlags - флаги событий. Могут быть следующими:
- CAAFL_RING0 Ring zero event.
- CAAFL_TIMEOUT Time out the event after the duration specified by dwTimeout.
- Проще говоря, если вы хотите ждать событие времени приложения только в течении определенного периода, используйте флаг CAAFL_TIMEOUT. Если вы хотите ждать это событие бесконечно. Мне не известно, что в действительности делает CAAFL_RING0.
- dwTimeout - период времени, которое VxD будет ждать событие времени приложения. Я не смог найти никакой информации о том, какие единицы измерения должны использоваться.

Этот сервис асинхронен, что означает то, что он сразу возвращается после регистрации вашей функции обратного вызова. Если вызов сервиса прошел успешно, он возвращает хэндл события времени приложения в eax'e. В обратном случае он возвращает 0.

Вы можете отменить регистрацию времени приложения, вызвав _SHELL_CancelAppyTimeEvent, которая принимает только один параметр, хэндл события времени приложения, возвращенный _SHELL_CallAtAppyTime.

На всякий случай, прежде, чем вызывать _SHELL_CallAtAppyTime, вам следует узнать у системы, будет ли доступно событие времени приложения. Например, что, если вы зарегистрируете событие времени приложения, когда система будет завершать работу? Ваша функция обратного вызова никогда не будет вызвана! Вы можете проверить доступность необходимого события с помощью _SHELL_QueryAppyTimeAvailable (у этого сервиса нет параметров). Он возвращает 0 в eax, если время приложения не будет доступно, то есть, если система прекращает работу или сервер сообщений получает ошибки GP. Этот сервис не говорит вам, что сейчас время приложения: он только вам говорит, что могут быть события времени приложения, поэтому, на всякий случай, вам стоит вызывать сначала _SHELL_QueryAppyTimeAvailable, и если он возвратит не нулевое значение в eax, вам следует перейти к вызову _SHELL_CallAtAppyTime.

Сервисы оболочки времени приложения

Когда наступает время приложения, есть несколько сервисов Shell'a, которые вы можете вызвать:

- _SHELL_CallDll
- _SHELL_FreeLibrary
- _SHELL_GetProcAddress
- _SHELL_LoadLibrary
- _SHELL_LocalAllocEx
- _SHELL_LocalFree

Эти шесть сервисов позволяют VxD вызывать 16-битные функции в 16-битных DLL/EXE, таких как WinHelp. Тем не менее, так как мы движемся к 32-битному миру (и 64-битному в будущем), я не буду вдаваться в детали относительно данной темы. Если вам интересно, вы можете прочитать о них в Windows 95/98 DDK.

Есть также другие сервисы SHELL'a, доступные только во время времени приложения, которые я нахожу более полезными: _SHELL_ShellExecute и _SHELL_BroadcastSystemMessage. С помощью

`_SHELL_BroadcastSystemMessage` вы можете послать сообщение всем окнам верхнего уровня и всем VxD за один вызов! Если время приложения доступно, вы можете посылать сообщения и окнам и VxD. Если время приложения недоступно, вы можете посылать сообщения только VxD.

`_SHELL_ShellExecute` - это `ring0`-расширение функции `ShellExecute` режима `ring-3`. Фактически, этот сервис вызывает `ShellExecute`. С помощью этого Shell-сервиса, вы можете запускать/открывать/печатать любой файл. У `_SHELL_ShellExecute` следующее определение:

```
VxDCall _SHELL_ShellExecute, <OFFSET32 ShexPacket>
```

Она получает только один параметр, плоский адрес структуры `SHEXPACKET`. Она возвращает значени из `ShellExecute` в `eax`. Давайте детально проанализируем структуру `SHEXPACKET`.

- `shex_dwTotalSize` - размер в байтах структуры `SHEXPACKET` плюс дополнительные параметр, `rgchBaggage`, который немедленно следует за этой структурой. Я объясню вкратце, что такое `rgchBaggage`.
- `shex_dwSize` - размер в байтах структуры `SHEXPACKET` не считая `rgchBaggage`. Вместе с `shex_dwTotalSize` может вычислить размер `rgchBaggage`.
- `shex_ibOp` - операция, которая должна быть выполнена. Если вы укажете 0, это будет означать, что вы хотите открыть файла. Если файл является запусчным, то это запустит его. Если вам нужно выполнить другую операцию, вы должны указать имя операции где-то в `rgchBaggage` и это поле должно содержать дистанцию в байтах от начала этой структуры `SHEXPACKET` до первого символа строки формата `ASCIIZ`, которая задает имя операции, которую вы хотите выполнить. Размер `SHEXPACKET` - 32 байта. Если строка с именем операции следует сразу за структурой `SHEXPACKET`, то значени `shex_ibOp` должно быть равным 32. Определены три операции, которые можно выполнить - "open", "print" и "explore".
- `shex_ibFile` - относительная дистанция от начала этой структуры до `ASCIIZ`-строки, задающей имя файла, которое вы хотите послать `ShellExecute` (принцип тот же самый, что и в случае с `shex_ibOp`).
- `shex_ibParams` - опциональные параметры, которые вы хотите передать файлу, указанному в `shex_ibFile`. Если файл - это документ или вы не хотите передавать никаких параметров, задайте 0. Если вы хотите передать какие-то параметры, поместите строки где-нибудь за структурой и поместите относительную дистанцию от начала структуры до этой строки в этом поле.
- `shex_ibDir` - рабочая директория. Укажите 0, если вы хотите использовать Windows-директорию или вы можете указать строку с желаемой директорией где-нибудь после уже упоминавшейся структуры и поместить относительный адрес от начала структуры на эту строку в данной поле.
- `shex_nCmdShow` - как должно показываться окно. Это одно из значений, которое вы обычно передает `ShowWindow`, например `SW_XXXX`-значение. Посмотрите эти значения в `window.inc`.

Все параметры размера `DWORD`. Теперь несколько слов относительно того самого параметра `rgchBaggage`, о котором я обещал рассказать. Это не так так просто. `rgchBaggage` - это параметр структуры `SHEXPACKET`, но он не может быть включен в определение структуры, поскольку его размер непостоянен. Проверьте `shell.inc`, вы увидите, что `rgchBaggage` не определен в `SHEXPACKET`, хотя документация Windows 9x DDK утверждает, что это член данной структуры.

Что такое `rgchBaggage`? Это просто массив `ASCIIZ`-структуры, которая следует за структурой `SHEXPACKET`. Внутри этого массива вы можете поместить имя операции, которую вам нужно выполнить над файлом, имя файла и имя рабочей директории. `SHELL VxD` получает смещение строки в `rgchBaggage` добавляя относительный адрес строки к адресу структуры. Например, если `SHEXPACKET` начинается в `60000h`, а строка следует прямо за ней, тогда дистанция между структурой и строкой - это размер самой структуры, 32 байта (`20h`). `Shell VxD` будет знать, что строка находится по адресу `60020h`.

Пример

Это будет очень простой пример, который покажет вам как зарегистрировать событие времени приложения и использовать _SHELL_ShellExecute. VxD будет динамический, который будет загружаться простым win32-приложением. Когда пользователь нажмет кнопку "run Calculator", win32-приложение вызовет DeviceIoControl, чтобы дать VxD команду зарегистрировать событие времени приложения и загрузить calc.exe, которое находится на Windows-директории.

```
;-----  
;  
; VxD Source Code  
;-----  
  
.386p  
include \masm\include\vmx.inc  
include \masm\include\win32.inc  
include \masm\include\shell.inc  
  
VxDName TEXT EQU <VXDEXEC>  
ControlName TEXT EQU <VXDEXEC_Control>  
VxDMajorVersion TEXT EQU <1>  
VxDMinorVersion TEXT EQU <0>  
  
VxD_STATIC_DATA_SEG  
VxD_STATIC_DATA_ENDS  
  
VXD_LOCKED_CODE_SEG  
;-----  
; Имя VxD должно быть набрано в верхнем регистре  
;-----  
  
DECLARE_VIRTUAL_DEVICE %VxDName,%VxDMajorVersion,%VxDMinorVersion,  
%ControlName,UNDEFINED_DEVICE_ID,UNDEFINED_INIT_ORDER  
  
Begin_control_dispatch %VxDName  
Control_Dispatch W32_DEVICEIOCONTROL, OnDeviceIoControl  
End_control_dispatch %VxDName  
  
BeginProc OnDeviceIoControl  
assume esi:ptr DIOCPARAMS  
.if [esi].dwIoControlCode==1  
VxDCall _SHELL_CallAtAppTime,<<OFFSET32 OnAppTime>,0,0,0>  
.endif  
xor eax,eax  
  
ret  
EndProc OnDeviceIoControl  
VXD_LOCKED_CODE_ENDS  
  
VXD_PAGEABLE_CODE_SEG  
BeginProc OnAppTime, CCALL  
ArgVar RefData,DWORD  
ArgVar TheFlag,DWORD  
EnterProc  
mov File.shex_dwTotalSize, sizeof SHEXPACKET  
add File.shex_dwTotalSize, sizeof EXENAME  
mov File.shex_dwSize, sizeof SHEXPACKET  
mov File.shex_ibOp, 0  
mov File.shex_ibFile, sizeof SHEXPACKET  
mov File.shex_ibParams, 0  
mov File.shex_ibDir, 0  
mov File.shex_dwReserved, 0  
mov File.shex_nCmdShow, 1  
VxDCall _SHELL_ShellExecute, <OFFSET32 File>  
LeaveProc
```

```

    Return
EndProc OnAppyTime
VXD_PAGEABLE_CODE_ENDS

VXD_PAGEABLE_DATA_SEG
    File SHEXPACKET <>
    EXENAME db "calc.exe",0
VXD_PAGEABLE_DATA_ENDS

end

```

Анализ

VxD ожидает сообщений от DeviceIoControl, сервис 1. Когда он получит это сообщение, он регистрирует событие времени приложения.

```
VxDCall _SHELL_CallAtAppyTime,<<OFFSET32 OnAppyTime>,0,0,0>
```

Он передает плоский адрес функции OnAppyTime _SHELL_CallAtAppyTime, чтобы Shell VxD вызвал ее, когда произойдет событие времени приложения.

```
BeginProc OnAppyTime, CCALL
```

Мы объявляем функцию с помощью BeginProc. Так как Shell VxD вызовет OnAppyTime, используя C-соглашение о передаче параметров, нам требуется указать атрибут CCALL.

```

    ArgVar RefData,DWORD
    ArgVar TheFlag,DWORD
EnterProc
...
LeaveProc
Return

```

Так как Shell VxD вызовет OnAppyTime с двумя параметрами, мы должны соответствующим образом настроить границы стека. Макрос ArgVar отвечает именно за это (вызывается для каждого параметра). Вот его синтакс:

```
ArgVar varname, size, used
```

varname - это имя параметра. Вы можете использовать любое имя, которое хотите. size - это, конечно, размер параметра в байтах. Вы можете использовать BYTE, WORD, DWORD или 1, 2, 4. used обычно опускается.

Сразу после вызовов макроса ArgVar нам нужно использовать макросы EnterProc и LeaveProc, чтобы отметить начало и конец инструкций в процедуре, чтобы локальные переменные и параметры могли использоваться корректно. Используйте макрос Return, чтобы передать управление вызывающему.

```

mov File.shex_dwTotalSize,sizeof SHEXPACKET
add File.shex_dwTotalSize,sizeof EXENAME
mov File.shex_dwSize,sizeof SHEXPACKET
mov File.shex_ibOp,0
mov File.shex_ibFile,sizeof SHEXPACKET
mov File.shex_ibParams,0
mov File.shex_ibDir,0
mov File.shex_dwReserved,0
mov File.shex_nCmdShow,1
VxDCall _SHELL_ShellExecute,<OFFSET32 File>

```

Инструкции внутри процедуры просты: инициализируйте структуру SHEXPACKET и вызовите сервис _SHELL_ShellExecute. Заметьте, что shex_dwTotalSize содержит комбинированный размер структуры SHEXPACKET и строки, которая следует за ней. Это в простом случае. Если строка

следует не сразу за ней, вы должны вычислить дистанцию между первым байтом строки и последним байтом строки самостоятельно. `shex_ibFile` содержит размер структуры, так как имя программы следует сразу за ней. `shex_ibDir` равен нулю, что означает то, что мы хотим использовать директорию `Windows` в качестве рабочей директории. Заметьте, что это не означает то, что программа должна быть в директории `Windows`. Программа может быть где угодно, главное, чтобы `Windows` мог ее найти. `shex_nCmdShow` равен 1 (это значение `SW_SHOWNORMAL`).

```
File SHEXPACKET <>
EXEName db "calc.exe",0
```

Мы определили структуру `SHEXPACKET`, за которой сразу следует имя программы, которую мы хотим запустить.

VxD. Урок 8. Client Register Structure

Автор: lczelion, пер. Aquila

Дата: 2002-06-06

В этом tutorialе мы изучим другую важную структуру под названием client register structure (CRS). Скачайте пример (находится в архиве wasm_files.zip: files/recovered/1003008/vxdbeep.zip).

Немного теории:

VxD очень сильно отличаются от обычных win32/win16/DOS-приложений. VxD, как правило, находятся в спящем состоянии, пока обычные приложения занимаются своим делом. Они поступают как наблюдатели, которые надзирают за другими ring3-приложения и корректируют их, когда они делают что-нибудь неправильно.

- Происходит прерывание
- VMM получает контроль
- VMM сохраняет значения регистров
- Сервисы VMM вызывают другие VxD
- VMM возвращает контроль прерванной программе

Самым интересным из вышесказанного является то, что VMM может оказывать влияние на прерванные приложения только одним образом - модифицируя сохраненные значения регистров. Например, если VMM считает, что прерванная программа должна продолжить выполнение с другого адреса, она может поменять значение CS:IP (значения которых были сохранены до прерывания программы), что повлечет за собой изменение хода программы - она продолжит выполнение по новому адресу, содержащемуся в CS:IP.

VMM сохраняет значения регистров в месте прерывания программы в CRS.

Client_Reg_Struct STRUC

```
Client_EDI DD ?
Client_ESI DD ?
Client_EBP DD ?
Client_res0 DD ?
Client_EBX DD ?
Client_EDX DD ?
Client_ECX DD ?
Client_EAX DD ?
Client_Error DD ?
Client_EIP DD ?
Client_CS DW ?
Client_res1 DW ?
Client_EFlags DD ?
Client_ESP DD ?
Client_SS DW ?
Client_res2 DW ?
Client_ES DW ?
Client_res3 DW ?
Client_DS DW ?
Client_res4 DW ?
Client_FS DW ?
Client_res5 DW ?
Client_GS DW ?
Client_res6 DW ?
Client_Alt_EIP DD ?
Client_Alt_CS DW ?
Client_res7 DW ?
Client_Alt_EFlags DD ?
Client_Alt_ESP DD ?
Client_Alt_SS DW ?
Client_res8 DW ?
```

```

Client_Alt_ES DW ?
Client_res9 DW ?
Client_Alt_DS DW ?
Client_res10 DW ?
Client_Alt_FS DW ?
Client_res11 DW ?
Client_Alt_GS DW ?
Client_res12 DW ?

Client_Reg_Struc ENDS

```

Вы можете видеть, что в этой структуре два множества параметров:

Client_xxx и Client_Alt_xxx. Это требует небольшого объяснения. В данном VM существует две ветви выполнения: V86 и защищенного режима. Если прерывание происходит, когда активна V86-программа, параметры Client_xxx будут содержать значения регистров V86-программы, а Client_Alt_xxx будут содержать значения регистров PM-программы. И наоборот, если прерывание происходит, когда активна PM-программа, Client_xxx будут содержать значения регистров PM-программы, а Client_Alt_xxx будут содержать значения регистров V86-программы. Client_resX зарезервированы и не используются.

У вас может появиться вопрос после анализа структуры: что если я хочу изменить только байт в структуре, например al? Вышеприведенная структура включает регистры размером только в слово и двойное слово. Не пугайтесь. Взгляните на vmm.inc. Есть две дополнительные структуры специально для этой цели: Client_Word_Reg_Struc и Client_Byte_Reg_Struc. Если вы хотите получить доступ к регистрам размером в слово и байт, приведите Client_Word_Reg_Struc к Client_Word_Reg_Struc или Client_Byte_Reg_Struc соответственно.

Следующий вопрос: как мы можем получить указатель на CRS?

Это довольно просто: большую часть времени VMM держит адрес CRS и ebp, когда она вызывает наш VxD. CRS в этом случае описывает состояние текущей виртуальной машины. Также вы можете получить этот указатель из VM-хэндла. Помните, что хэндл VM - это линейный адрес контрольного блока VM.

```

cb_s STRUC
CB_VM_Status DD ?
CB_High_Linear DD ?
CB_Client_Pointer DD ?
CB_VMID DD ?
CB_Signature DD ?
cb_s ENDS

```

CB_Client_Pointer содержит указатель на CRS этой VM. Например, вы можете получить указатель на CRS текущей VM с помощью следующего кода:

```

VMMSysCall Get_Cur_VM_Handle ; возвращает хэндл текущей VM в ebx
assume ebx:ptr cb_s
mov ebp,[ebx+CB_Client_Pointer] ; указатель на CRS

```

Теперь, когда мы изучили CRS, мы можем перейти к ее непосредственному применению. Мы будем использовать CRS, чтобы передавать значения регистров MS-DOS-прерыванию 21h, сервису 2h (отображение символа). Этот сервис MS-DOS получает символ, который должен быть отображен, в dl. Если мы передаем символ 07h этому сервису, он проиграет некий звук с помощью спикера.

Помните, что этот int 21h - это сервис MS-DOS, который доступен в режиме V86, так как мы можем вызвать V86-прерывание из VxD? Один путь - это использовать сервис Exes_Int. Этот сервис VMM получает номер прерывания, который нужно вызвать в eax. Тем не менее, он должен внутри особого блока кода, который начинается с Begin_Nest_V86_Exes и заканчивается End_Nest_Exes. Поэтому если нам нужно вызывать int 21h, сервис 2, нам потребуется изменить Client_ah и Client_Dl в CRS внутри такого блока выполнения и затем сохранить значение 21h в eax. Когда все будет готово, можно вызывать Exes_Int.

Пример:

Пример является динамическим VxD, который использует int21h, сервис 2, чтобы спикер проиграл некоторый звук.

```
.386p
include \masm\include\vmx.inc
include \masm\include\win32.inc
include \masm\include\v86mmgr.inc

VxDName TEXT EQU
ControlName TEXT EQU
VxDMajorVersion TEXT EQU <1>
VxDMinorVersion TEXT EQU <0>

VxD_STATIC_DATA_SEG
VxD_STATIC_DATA_ENDS

VXD_LOCKED_CODE_SEG
;-----
; Помните: имя VxD должно использоваться в верхнем регистре, иначе ничего
; не будет работать.
;-----

DECLARE_VIRTUAL_DEVICE %VxDName,%VxDMajorVersion,%VxDMinorVersion, \
%ControlName,UNDEFINED_DEVICE_ID,UNDEFINED_INIT_ORDER

Begin_control_dispatch %VxDName
    Control_Dispatch W32_DEVICEIOCONTROL, OnDeviceIoControl
End_control_dispatch %VxDName

VXD_LOCKED_CODE_ENDS

VXD_PAGEABLE_CODE_SEG
BeginProc OnDeviceIoControl
    assume esi:ptr DIOCPARAMS
    .if [esi].dwIoControlCode==1
        Push_Client_State
        VMCall Begin_Nest_V86_Exec
        assume ebp:ptr Client_Byte_Reg_Struct
        mov [ebp].Client_d1,7
        mov [ebp].Client_ah,2
        mov eax,21h
        VMCall Exec_Int
        VMCall End_Nest_Exec
        Pop_Client_State
    EndI:
    .endif
    xor eax,eax
    ret
EndProc OnDeviceIoControl
VXD_PAGEABLE_CODE_ENDS

end
```

Анализ:

```
Push_Client_State
```

Здесь особо нечего анализировать. Когда VxD получает сообщение DeviceIoControl, ebp уже указывает на CRS текущей VM. Мы вызываем макрос Push_Client_State, чтобы сохранить текущее состояние клиентских регистров в стеке. Позже мы восстановим их значение с помощью Pop_Client_State.

```
VMCall Begin_Nest_V86_Exec
```

Начинаем особый блок выполнения с помощью вызова Begin_Nest_V86_Exec.

```
assume ebp:ptr Client_Byte_Reg_Struct
mov [ebp].Client_d1,7
```

```
mov [ebp].Client_ah,2
```

Изменяем значения регистров dl и ah в CRS. Эти измененные значения будут использованы прерыванием.

```
mov eax,21h  
VMMSyscall Exec_Int
```

Exec_Int получает номер прерывания из eax. Мы помещаем в него нужное значение и вызываем Exec_Int.

```
VMMSyscall End_Nest_Exec  
Pop_Client_State
```

Когда Exec_Int возвращает значение, мы заканчиваем особый блок выполнения и восстанавливаем сохраненные значения клиентских регистров из стека.

Вы услышите, как ваш PC-спикер проигрывает 'bell'-символ (07h).

VxD. Урок 9. Менеджер V86-памяти

Автор: lczelion, пер. Aquila

Дата: 2002-06-06

В предыдущих туториалах вы узнаете, как эмулировать вызов V86-прерывания. Тем не менее, есть одна проблема, которая еще не затрагивалась: обмен данных между VxD и V86-кодом. Мы изучим, как использовать менеджер V86-памяти для этого.

Скачайте пример (находится в архиве `wasm_files.zip`: `files/recovered/1003009/vxdlabel.zip`).

Немного теории

Если ваш VxD работает с некоторыми V86-процедурами, рано или поздно ему потребуется передать и получить большой объем данных от V86-программы. Использовать регистры для этой цели неудобно. Вашей следующей попыткой может стать резервирование блока памяти в `ring0` и передача указателя на блок памяти через какой-нибудь регистр, чтобы V86-код мог воспользоваться этими данными. Если вы сделаете так, это, вполне вероятно, вызовет крах системы, потому что адресация режима V86 требует пару `segment:offset`, а не линейный адрес.

Есть много способов решить эту проблему. Тем не менее, я выбрал простой путь, чтобы показать сервисы, предоставляемые менеджером V86-памяти. Если каким-то образом вы сможете найти свободный блок памяти в V86-регионе, который вы захотите использовать в качестве буфера передачи данных, это решит одну из проблем. Тем не менее, останется проблема трансляции указателя в надлежащий формат. Вы можете решить обе проблемы, используя сервисы менеджера V86-памяти.

Менеджер V86-памяти - это статический VxD, который управляет памятью для V86-приложений. Он также предоставляет сервисы EMS и XMS V86-приложениям и API-сервисы трансляции другим VxD. API-трансляция, фактически, является процессом копирования данных из `ring0` в буфер в V86-регионе и затем передача V86-адреса данных V86-коду. Никакого волшебства.

Менеджер V86-памяти управляет буфером трансляции, который является блоком памяти в V86-регионе для копирования данных от VxD в V86-регион и обратно. Изначально этот буфер трансляции равен четырем килобайтам. Вы можете повысить размер этого буфера, вызвав `V86MMGR_Set_Mapping_Info`.

Теперь, когда вы знаете о буфере трансляции, как мы можем скопировать данные туда и обратно? Это вопрос вовлекает два сервиса: `V86MMGR_Allocate_Buffer` и `V86MMGR_Free_Buffer`.

`V86MMGR_Allocate_Buffer` резервирует блок памяти из буфера трансляции и опционально копирует данные из `ring0` в зарезервированный V86-блок. `V86MMGR_Free_Buffer` делает обратное: он опционально копирует данные из зарезервированного V86-блока в `ring0`-буфер и освобождает блок памяти, зарезервированный `V86MMGR_Allocate_Buffer`.

Заметьте, что менеджер V86-памяти управляет зарезервированными буферами как стеком. Это означает, что резервирование/освобождение должны соблюдать правило "первый вошел/первый вышел". Поэтому, если вы сделаете два вызова `V86MMGR_Allocate_Buffer`, первый вызов `V86MMGR_Free_Buffer` освободит буфер зарезервированный вторым вызовом `V86MMGR_Allocate_Buffer`.

Теперь мы можем перейти к анализируемому определению `V86MMGR_Allocate_Buffer`. Этот сервис получает данные через регистры.

- `ebx` - хэндл текущей виртуальной машины
- `ebp` - указатель на CRS текущей виртуальной машины

- esx - количество байтов, которое нужно зарегистрировать в буфере трансляции.
- carry flag - очищается, если вы не хотите копировать данные из ring0-буфер в зарезервированный блок. Установите флаг, если вы хотите копировать данные из ring0-буфера в зарезервированный блок.
- fs:esi - селектор:смещение блока ring0-памяти, содержащая данные, которые должны быть скопированы в зарезервированный буфер. Игнорируется, если carry flag очищен.

Если вызов прошел успешно, флаг переноса будет очищен и esx будер содержать количество байтов, которые были зарезервированы в буфере трансляции. Это значение может быть меньше, чем переданное вам значение, поэтому вам следует сохранить содержимое esx, чтобы затем использовать для передачи V86MMGR_Free_Buffer. edi содержит V86-адрес зарезервированного блока, причем сегмент находится в верхнем слове, а смещение в нижнем. Флаг переноса устанавливается, если происходит ошибка.

V86MMGR_Free_Buffer принимает точно такие же параметры, как и V86MMGR_Allocate_Buffer.

Что в действительно происходит, когда вы вызываете V86MMGR_Allocate_Buffer? Вы резервирует блок памяти в V86-регионе текущей виртуальной машины и получаете V86-адрес этого блока в edi. Мы можем использовать эти сервисы для передачи и получения данных от V86-прерываний.

Кроме API трансляции менеджер V86-памяти также предлагает другим VxD сервисы API мэппирования. API мэппирования - это процесс мэппирования некоторых страниц в расширенной памяти в V86-регион каждой виртуальной машины. Вы можете использовать V86MMGR_Map_Pages, чтобы делать это. С помощью этого сервиса, страницы мэппируются в то же линейное пространство каждой виртуальной машины. Это тратит адресное пространство впустую, если вы хотите работать только с одной VM. Также API мэппирования медленнее, чем API трансляции, поэтому вам лучше использовать последнюю так часто, как это возможно. API-мэппирование требуется для некоторой V86-операции, которые требуются для доступа к тому же линейному пространству и должны присутствовать во всех виртуальных машинах.

Пример

Этот пример использует API трансляции вместе с int21h, 440Dh, младший код 66h, Get Media ID, чтобы получить букву первого жесткого диска.

```
;-----
;                               VxDLabel.asm
;-----
.386p
include \masm\include\vmx.inc
include \masm\include\win32.inc
include \masm\include\v86mmgr.inc

VxDName TEXT EQU
ControlName TEXT EQU
VxDMajorVersion TEXT EQU <1>
VxDMinorVersion TEXT EQU <0>

VxD_STATIC_DATA_SEG
VxD_STATIC_DATA_ENDS

VxD_LOCKED_CODE_SEG
;-----
; Remember: The name of the vxd MUST be uppercase else it won't work/unload
;-----

DECLARE_VIRTUAL_DEVICE %VxDName,%VxDMajorVersion,%VxDMinorVersion,
%ControlName,UNDEFINED_DEVICE_ID,UNDEFINED_INIT_ORDER

Begin_control_dispatch %VxDName
    Control_Dispatch W32_DEVICEIOCONTROL, OnDeviceIoControl
End_control_dispatch %VxDName
VxD_LOCKED_CODE_ENDS
```

```

VXD_PAGEABLE_CODE_SEG
BeginProc OnDeviceIoControl
    assume esi:ptr DIOCPParams
    .if [esi].dwIoControlCode==1
        VMMSysCall Get_Sys_VM_Handle
        mov Handle,ebx
        assume ebx:ptr cb_s
        mov ebp,[ebx+CB_Client_Pointer]
        mov ecx,sizeof MID
        stc
        push esi
        mov esi,OFFSET32 MediaID
        push ds
        pop fs
        VxDCall V86MMGR_Allocate_Buffer
        pop esi
        jc EndI
        mov AllocSize,ecx
        Push_Client_State
        VMMSysCall Begin_Nest_V86_Exec
        assume ebp:ptr Client_Byte_Reg_Struct
        mov [ebp].Client_ch,8
        mov [ebp].Client_cl,66h
        assume ebp:ptr Client_word_reg_struct
        mov edx,edi
        mov [ebp].Client_bx,3 ; drive A
        mov [ebp].Client_ax,440dh
        mov [ebp].Client_dx,dx
        shr edx,16
        mov [ebp].Client_ds,dx
        mov eax,21h
        VMMSysCall Exec_Int
        VMMSysCall End_Nest_Exec
        Pop_Client_State
        ;-----
        ; retrieve the data
        ;-----
        mov ecx,AllocSize
        stc
        mov ebx,Handle
        push esi
        mov esi,OFFSET32 MediaID
        push ds
        pop fs
        VxDCall V86MMGR_Free_Buffer
        pop esi
        mov edx,esi
        assume edx:ptr DIOCPParams
        mov edi,[edx].lpvOutBuffer
        mov esi,OFFSET32 MediaID.midVollabel
        mov ecx,11
        rep movsb
        mov byte ptr [edi],0
        mov ecx,[edx].lpcbBytesReturned
        mov dword ptr [edx],11
    EndI:
    .endif
    xor eax,eax
    ret
EndProc OnDeviceIoControl
VXD_PAGEABLE_CODE_ENDS

```

```

VXD_PAGEABLE_DATA_SEG
MID struct
    midInfoLevel dw 0
    midSerialNum dd ?
    midVollabel db 11 dup(?)
    midFileSysType db 8 dup(?)
MID ends
MediaID MID <>

```

```

Handle dd ?
AllocSize dd ?
VXD_PAGEABLE_DATA_ENDS

end

;-----
;                               Label.asm
; The win32 VxD loader.
;-----
.386
.model flat,stdcall
option casemap:none

include \masm32\include\windows.inc
include \masm32\include\user32.inc
include \masm32\include\kernel32.inc
includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib

DlgProc PROTO :DWORD,:DWORD,:DWORD,:DWORD
.data
Failure db "Cannot load VxDLabel.VXD",0
AppName db "Get Disk Label",0
VxDName db "\\.\vxdLabel.vxd",0
OutputTemplate db "Volume Label of Drive C",0

.data?
hInstance HINSTANCE ?
hVxD dd ?
DiskLabel db 12 dup(?)
BytesReturned dd ?

.const
IDD_VXDRUN     equ 101
IDC_LOAD       equ 1000

.code
start:
invoke GetModuleHandle, NULL
mov     hInstance,eax
invoke DialogBoxParam, hInstance, IDD_VXDRUN ,NULL,addr DlgProc,NULL

invoke ExitProcess,eax

DlgProc proc hDlg:HWND, uMsg:UINT, wParam:WPARAM, lParam:LPARAM
.if uMsg==WM_INITDIALOG
invoke CreateFile,addr VxDName,0,0,0,0,FILE_FLAG_DELETE_ON_CLOSE,0
.if eax==INVALID_HANDLE_VALUE
invoke MessageBox,hDlg,addr Failure,addr
AppName,MB_OK+MB_ICONERROR
mov     hVxD,0
invoke EndDialog,hDlg,NULL
.else
mov     hVxD,eax
.endif
.elseif uMsg==WM_CLOSE
.if hVxD!=0
invoke CloseHandle,hVxD
.endif
invoke EndDialog,hDlg,0
.ELSEIF uMsg==WM_COMMAND
mov     eax,wParam
mov     edx,wParam
shr     edx,16
.if dx==BN_CLICKED
.if ax==IDC_LOAD
invoke DeviceIoControl,hVxD,1,NULL,0,addr DiskLabel,12,\
addr BytesReturned,NULL

```

```

        invoke MessageBox,hDlg,addr DiskLabel,addr OutputTemplate, \
            MB_OK+MB_ICONINFORMATION
    .endif
.endif
.ELSE
    mov eax,FALSE
    ret
.ENDIF
mov eax,TRUE
ret
DlgProc endp
end start

```

Анализ

Мы проанализируем label.asm, который является win32-приложением, загружающим VxD.

```

    invoke DeviceIoControl,hVxD,1,NULL,0,addr DiskLabel,12,\
        addr BytesReturned,NULL

```

Он вызывает DeviceIoControl с кодом устройства равным 1, без входного буфера, с указателем на буфер вывода и его размер. DiskLabel - это буфер для получения метки тома, который возвратит VxD. Количество байтов, которые будут фактически возвращены, будут сохранены в переменной BytesReturned. Этот пример демонстрирует, как передавать данные VxD и как получить их от него: вы передаете входной/выходной буфер VxD, и тот читает/записывает в предложенный буфер.

Сейчас мы проанализируем VxD-код.

```

VMMCall Get_Sys_VM_Handle
mov Handle,ebx
assume ebx:ptr cb_s
mov ebp,[ebx+CB_Client_Pointer]

```

Когда VxD получает сообщение W32_DeviceIoControl, он вызывает Get_Sys_VM_Handle, чтобы получить хэндл системой VM и сохраняет ее в переменную под названием Handle. Затем он извлекает указатель на CRS из контрольного блока VM.

```

mov ecx,sizeof MID
stc
push esi
mov esi,OFFSET32 MediaID
push ds
pop fs
VxDCall V86MMGR_Allocate_Buffer
pop esi
jc EndI
mov AllocSize,ecx

```

Теперь он подготавливает параметры, которые будут переданы V86MMGR_Allocate_Buffer. Мы должны инициализировать зарезервированный буфер, начинаем с инструкции stc. Мы помещаем смещение MediaID в esi и селектор в fs, а затем вызываем V86MMGR_Allocate_Buffer. Помните, что esi содержит указатель на DIOCParm, чтобы мы могли сохранить их с помощью push esi и pop esi.

```

Push_Client_State
VMMCall Begin_Nest_V86_Exec
assume ebp:ptr Client_Byte_Reg_Struc
mov [ebp].Client_ch,8
mov [ebp].Client_cl,66h
assume ebp:ptr Client_word_reg_struct
mov edx,edi
mov [ebp].Client_bx,3 ; drive C
mov [ebp].Client_ax,440dh

```

Мы подготавливаем значения в CRS для int 21h, 440Dh minor code 66h, указав, что мы хотим получить media ID диска C. Мы также копируем значение edi в edx (edi содержит V86-адрес блока памяти, зарезервированного с помощью V86MMGR_Allocate_Buffer).

```

mov [ebp].Client_dx,dx

```

```
shr edx,16
mov [ebp].Client_ds,dx
```

Так как int 21h, 440Dh, minor code 66h принимает указатель на структуру MID в ds:dx, мы должны разделить пару "сегмент:смещение" в edx'е на две части и поместить их в соответствующие образы регистров.

```
mov eax,21h
VMMCall Exec_Int
VMMCall End_Nest_Exec
Pop_Client_State
```

Когда все готово, мы вызываем Exec_Int, чтобы произвести вызов прерывания.

```
mov ecx,AllocSize
stc
mov ebx,Handle
push esi
mov esi,OFFSET32 MediaID
push ds
pop fs
VxDCall V86MMGR_Free_Buffer
pop esi
```

После возвращения Exec_Int, зарезервированный буфер будет заполнен необходимой нам информацией. Следующий шаг - это получить информацию. Мы достигнем этой цели, вызвав V86MMGR_Free_Buffer. Этот сервис освобождает блок памяти, зарезервированный V86MMGR_Allocate_Memory и копирует данных в зарезервированном блоке и указанных блок ring0-памяти. Как и в случае с V86MMGR_Allocate_Memory, если вы хотите скопировать операцию, вы должны установить флаг переноса перед вызовом сервиса.

```
mov edx,esi
assume edx:ptr DIOCPParams
mov edi,[edx].lpvOutBuffer
mov esi,OFFSET32 MediaID.midVolLabel
mov ecx,11
rep movsb
mov byte ptr [edi],0
mov ecx,[edx].lpcbBytesReturned
mov dword ptr [edx],11
```

Теперь информация находится в ring0-буфере, мы копируем метку тома в буфер, предоставленный win32-приложением. Мы можем получить доступ к этому буферу, используя поле lpvOutBuffer структуры DIOCPParams.

ODBC. Урок 1. Основы

Автор: lczelion, пер. SheSan

Дата: 2002-06-06

Это первая "консультация" из целой серии, которая имеет дело с базами данных, программируемых в win32asm. Программирование баз данных становится всё более актуально в современном мире. В настоящее время существует множество различных форматов баз данных. Если мы желаем изучить файловый формат баз данных, чтобы программировать их используя win32asm, то нам необходимо множество времени и желания.

К счастью, Микрософт имеет технологию, которая значительно помогает нам в этом отношении. Она называется **ODBC**, что означает **Open Database Connectivity**, т.е. представляет собой спецификацию интерфейса для доступа к базам данных различных форматов. По сути, это некий интерфейс API, такой же как и Windows API, который имеет дело с программированием баз данных. То есть, с возможностями ODBC API, нам открывается сравнительно лёгкий путь для доступа к целому ряду баз данных.

Как же функционирует ODBC? Какова её структура? Вы должны получить ясное представление об архитектуре ODBC перед его использованием. ODBC включает в себя четыре компонента:

- **Приложение (ваша программа)-ODBC менеджер-ODBC Драйвера-Источник Данных (базы данных)**

Центральным компонентом является менеджер ODBC. Вы можете понимать под этим термином некоего мастера управляющего работой ODBC. Вы сообщаете ему, что вы хотите сделать и он передаёт ваше желание своим рабочим (драйверам ODBC) которые и выполняют эту работу. Если рабочие имеют некоторые сообщения для вас, они сообщают об этом мастеру (менеджеру ODBC) и он передает сообщения вам.

Согласно этой модели, вы не работаете непосредственно с драйверами баз данных. Все действия по управлению драйверами ODBC осуществляет менеджер, его задачей является трансляция ваших желаний в реальность. Каждый драйвер ODBC знает всё о базе данных для которой он был разработан. Таким образом каждый компонент делает все возможное, чтобы упростить работу.

Ваша программа <----> ODBC менеджер<----> ODBC Драйвера <----> Базы Данных

Менеджер ODBC поставляется Микрософт. Проверьте вашу Панель Управления. Если ваша машина имеет правильно установленный ODBC, то вы найдете Источник Данных ODBC (ODBC Data Sources) здесь. Что касается драйверов ODBC, то Микрософт предоставляет их с продуктами, и вы можете всегда получить новые драйвера ODBC от поставщиков баз данных. Устанавливая новые драйверы ODBC, мы даём возможность нашей машине использовать новые базы данных, о которых она не знала прежде.

ODBC API - просты в использовании, но в любом случае вам необходимо обладать некоторыми знаниями о SQL и базах данных. Например: значение области, первичный ключ, записи, столбцы, колонки и т.п... Если у вас нет таких знаний, то я советую сначала их приобрести. Как вы могли заметить, менеджер ODBC пытается спрятать детали реализации от вашей программы, в замен он предлагает некоторый интерфейс для работы с базами данных, а конкретно с драйверами ODBC. Драйвера ODBC отличаются в своих возможностях, поэтому приложения должны обладать возможностью, чтобы обнаружить поддерживает ли драйвер ODBC конкретную характеристику. ODBC предоставляет три уровня услуг, называемых **Уровнями Соответствия Интерфейса** (Interface Conformance Levels). Это ядро, Уровень 1 и Уровень 2. Каждый драйвер ODBC должен включать все характеристики определенные на данном уровне ядра. С точки зрения приложений, ODBC API делятся между тремя уровнями. Если специфическая функция помечена как ядерная, то

это означает, что вы можете использовать её не проверяя, поддерживающееся ли она конкретным драйвером ODBC. Если это функция уровня 1 или 2, то вам необходимо убедиться, что драйвер ODBC поддерживает её перед её использованием. Вы можете получить подробные сведения о ODBC API из MSDN.

Вам нужно знать некоторые термины ODBC перед началом программирования.

- **Окружение (Environment).** Это просто глобальный контекст, чтобы иметь доступ к данным. Если вы знакомы с DAO, то вы можете понимать это как рабочую область. Он содержит информацию, которая относится ко всей ODBC-сессии, это, например, описатели соединения в течение этого сеанса. Вы должны получить контекст среды прежде, чем сможете начать работать с ODBC.
- **Соединение (Connection).** Определяет драйвер ODBC и источник данных (базу данных). Вы можете иметь многочисленные связи с другими базами данных в той же среде.
- **Инструкция (Statement).** ODBC использует SQL как язык. Таким образом инструкция может быть простым запросом SQL который будет выполнен ODBC.

Ниже шаги, которые обычно необходимо выполнять при программировании с ODBC:

1. Подключится к источнику данных
2. Построить и выполнить одну или более инструкций SQL
3. Изучить результирующие записи (если имеются)
4. Отключится от источника данных

Мы узнаем как выполнять каждый шаг приведённый выше на следующих уроках.

ODBC. Урок 2. Соединение с базой данных

Автор: lczelion, пер. SheSan

Дата: 2002-06-06

На этой консультации, мы изучим механику использования ODBC API.

Ваша программа не общается непосредственно с драйверами ODBC, она пользуется услугами менеджера ODBC. Управление работой менеджера осуществляется с помощью API функций к которым вы можете обращаться непосредственно из своей программы, вы должны только подключить `odbc32.inc` и `odbc32.lib`, а так же `windows.inc`.

Шаги которые необходимо предпринять для соединения с базой данных следующие:

1. **Получить идентификатор окружения.** Вам нужно делать это только один раз за ODBC-сеанс. Как только вы получите идентификатор, вы сможете модифицировать свойства окружения так, чтобы они удовлетворяли вашим требованиям. Вы можете понимать этот шаг как создание некой рабочей области.
2. **Указать какую версию ODBC ваша программа хочет использовать.** Вы можете выбрать между версиями ODBC 2.x и 3.x. Они различны во многих аспектах, поэтому этот шаг - необходим. Таким образом менеджер ODBC сможет решить какой синтаксис он должен использовать, чтобы связаться с вашей программой и проинтерпретировать её команды.
3. **Выделить память для идентификатора соединения.** Этот шаг может рассматриваться как создание пустого соединения. Вы не обязаны определять нужный для соединения с базой данных драйвер.
4. **Установить связь.** Вы вызываете функцию ODBC, чтобы установить связь.

Когда вы закончите работу с базой данных, вы должны закрыть связь с ней следующим образом:

1. **Отключится от источника данных**
2. **Уничтожить идентификатор соединения**
3. **Уничтожить идентификатор окружения** (если вы не желаете использовать это окружение для других соединений)

ВЫДЕЛЕНИЕ ПАМЯТИ ДЛЯ ИДЕНТИФИКАТОРА

В версиях ODBC до 3.x, вам нужно вызывать отдельные функции, чтобы выделить память для идентификатора окружения, соединения и инструкции (**SQLAllocEnv**, **SQLAllocConnect**, **SQLAllocStmt**). Теперь под ODBC 3.x все функции заменяются **SQLAllocHandle**, которая имеет следующий синтаксис:

```
SQLRETURN SQLAllocHandle(SQLSMALLINT HandleType,  
                          SQLHANDLE InputHandle,  
                          SQLHANDLE * OutputHandlePtr  
                          );
```

Вышеуказанный синтаксис может утомить ваш взор, поэтому мы его немного упростим.

```
SQLAllocHandle proto HandleType:DWORD,  
                  InputHandle:DWORD,  
                  OutputHandlePtr:DWORD
```

SQLRETURN определен как тип **SQLSMALLINT**, **SQLSMALLINT** определен как короткое целое, т.е. слово (16 бит). Таким образом функция возвращает выходную величину в `ax`, а не в `eax`, что является очень важным. Однако параметр передается функции под Win32 в 32-битном стеке. Поэтому, даже если параметр определен как словный (16-бит) вы должны его расширить до 32-бит.

Вот почему **HandleType**- dword вместо word. Вы можете свериться с библиотекой импорта: **odbc32.lib**. Вход для **SQLAllocHandle** это **_SQLAllocHandle@12**. Эта запись означает, что комбинированный размер параметров составляет 12 байт (3 слова). Тем не менее, это не означает, что функциональный прототип C - неверный. **SQLAllocHandle** будет только использовать младшее слово **HandleType** и игнорировать старшее слово. Таким образом функциональный прототип C - корректен пока наш asm-функциональный прототип соответствует сказанному выше.

С типом SQL разберёмся более обстоятельно. Рассмотрим входные функциональные параметры и обратную величину.

- **HandleType** - константа, которая определяет тип идентификатора, для которого вы хотите распределить память. Возможные величины:

Значение	Описание
SQL_HANDLE_ENV	идентификатор окружения
SQL_HANDLE_DBC	идентификатор соединения
SQL_HANDLE_STMT	идентификатор запроса
SQL_HANDLE_DESC	идентификатор дескриптора

Дескриптор - это коллекция метаданных, которые описывают параметры инструкций SQL или столбцов с набором результатов, которые обрабатываются приложением или драйвером.

- **InputHandle**- идентификатор родительского "контекста". Если вы хотите выделить память для идентификатора подключения, вы должны получить идентификатор окружения, потому что подключение будет сделано в контексте этого окружения. Если вы хотите выделить память для идентификатора окружения, этот параметр должен быть **SQL_HANDLE_NULL** (остерегайтесь значения **SQL_HANDLE_NULL** в windows.inc версии 1.18 и ниже, т.к. там допущена ошибка и определено ненадлежащим образом как 0L. Вы должны стереть "L", иначе ваша программа не будет транслироваться. Что касается операторных и дескрипторных идентификаторов, то вы должны передать идентификатор подключения как этот параметр.

- **OutputHandlePtr** указывает на dword переменную, которая получит распределенный идентификатор, если запрос успешен.

Возможные возвращаемые значения **SQLAllocHandle** могут быть:

Значение	Описание
SQL_SUCCESS	Функция завершена успешно
SQL_SUCCESS_WITH_INFO	Функция завершена успешно, но с предупреждением
SQL_ERROR	Функция потерпела неудачу.
SQL_INVALID_HANDLE	Идентификатор переданный функции, недействителен

Выполнилась ли функция успешно или потерпела неудачу, вы можете получить подробную информацию относительно этого, вызывая **SQLGetDiagRec** или **SQLGetDiagField**. Они играют ту же самую роль, что и GetLastError в Win32 API.

Пример:

```
.data?
hEnv dd ?

.code
    invoke SQLAllocHandle, SQL_HANDLE_ENV, SQL_HANDLE_NULL, addr hEnv
    .if ax==SQL_SUCCESS || ax==SQL_SUCCESS_WITH_INFO
```

ВЫБОР ВЕРСИИ ODBC

После выделения памяти для идентификатора окружения вы должны установить атрибут окружения, **SQL_ATTR_ODBC_VERSION**, в соответствующее значение. Установка значения атрибута окружения делается, вызовом **SQLSetEnvAttr**. К настоящему времени вы должны знать, что имеются также функции **SQLSetConnectAttr** и **SQLSetStmtAttr**. **SQLSetEnvAttr** определена как:

```
SQLSetEnvAttr proto EnvironmentHandle:DWORD,  
                  Attribute:DWORD,  
                  ValuePtr:DWORD,  
                  StringLength:DWORD
```

- **EnvironmentHandle**. Содержит идентификатор окружения, атрибут которого вы хотите установить.
- **Attribute**. Константа, которая представляет атрибут, который вы хотите установить. Для нашей цели, это - **SQL_ATTR_ODBC_VERSION**. Вы можете искать полный список в MSDN.
- **ValuePtr**. Значение этого параметра зависит от атрибута, который вы хотите установить. Если атрибут - 32-разрядное значение, этот параметр обрабатывается как значение, которое вы хотите установить. Если атрибут - текстовая строка или двоичный буфер, то это интерпретируется как указатель на строку или буфер. Если вы определяете **SQL_ATTR_ODBC_VERSION**, то имеются два возможных значения, которые вы можете использовать: **SQL_OV_ODBC3** и **SQL_OV_ODBC2**, для ODBC версий 3.x и 2.x соответственно.
- **StringLength**. Размер значения, указанного **ValuePtr**. Если значение - строка или двоичный буфер, этот параметр должен быть действителен. Если атрибут, который вы хотите установить - dword, этот параметр игнорируется. С тех пор как **SQL_ATTR_ODBC_VERSION** атрибут содержит значение dword, вы можете передавать NULL как этот параметр.

Список возможных возвращаемых значений идентичен **SQLAllocHandle**.

Пример:

```
.data?  
hEnv dd ?  
  
.code  
    invoke SQLAllocHandle, SQL_HANDLE_ENV, SQL_HANDLE_NULL, addr hEnv  
    .if ax==SQL_SUCCESS || ax==SQL_SUCCESS_WITH_INFO  
        invoke SQLSetEnvAttr, hEnv, SQL_ATTR_ODBC_VERSION, SQL_OV_ODBC3, NULL  
        .if ax==SQL_SUCCESS || ax==SQL_SUCCESS_WITH_INFO
```

ВЫДЕЛЕНИЕ ПАМЯТИ ДЛЯ ИДЕНТИФИКАТОРА ПОДКЛЮЧЕНИЯ

Этот шаг подобен выделению памяти для ид. окружения, вы также вызываете **SQLAllocHandle**, но передаёте другое значение параметра.

Пример:

```
.data?  
hEnv dd ?  
hConn dd ?  
  
.code  
    invoke SQLAllocHandle, SQL_HANDLE_ENV, SQL_HANDLE_NULL, addr hEnv  
    .if ax==SQL_SUCCESS || ax==SQL_SUCCESS_WITH_INFO  
        invoke SQLSetEnvAttr, hEnv, SQL_ATTR_ODBC_VERSION, SQL_OV_ODBC3, NULL  
        .if ax==SQL_SUCCESS || ax==SQL_SUCCESS_WITH_INFO  
            invoke SQLAllocHandle, SQL_HANDLE_DBC, hEnv, addr hConn
```

```
.if ax==SQL_SUCCESS || ax==SQL_SUCCESS_WITH_INFO
```

УСТАНОВКА СВЯЗИ

Теперь мы готовы сделать попытку фактического подключения к источнику данных через выбранный ODBC драйвер. Имеются фактически три функции ODBC, которые мы можем использовать, чтобы достичь этой цели. Они предлагают различную степень "выбора", который вы можете сделать.

Функция	Уровень	Описание
SQLConnect	Ядро	Это - самая простая функция. Требуется только DSN (название источника Данных) и необязательное название пользователя и пароль. Она не предлагает интерфейса GUI типа подсказки пользователю в виде диалогого окна для получения дополнительной информации. Вы должны использовать эту функцию, если вы уже имеете DSN для заданной базы данных.
SQLDriverConnect	Ядро	Эта функция предлагает более широкий спектр услуг чем SQLConnect . Вы можете соединяться с источником данных, который не определен в системной информации, то есть без DNS. Кроме того, вы можете определить, отобразит ли эта функция диалоговое окно, запрашивающее пользователя для получения дополнительной информации. Например, если вы опустили имя файла базы данных, она будет инструктировать ODBC драйвер об отображении диалогового окна, запрашивающего пользователя выбрать базу данных, для соединения с ней.
SQLBrowseConnect	Уровень 1	Эта функция предлагает перечисление источников данных во время выполнения. Она обеспечивает более гибкий интерфейс в сравнении с SQLDriverConnect , потому что вы можете вызывать SQLBrowseConnect несколько раз последовательно, каждый раз запрашивая пользователя для получения более конкретной информации, пока наконец вы не получите рабочую строку подключения.

Я буду исследовать сначала SQLConnect. Чтобы использовать SQLConnect, вы должны знать кое-что относительно DSN. DSN расшифровывается как Название Источника Данных, т.е. это строка, которая уникально идентифицирует источник данных. DSN идентифицирует строение данных, которое содержит информацию о том, как соединиться с удельным источником данных. Информация включает и то, какой ODBC-драйвер использовать и с какой базой данных соединиться. Вы создаете, изменяете и удаляете DSN, используя 32-разрядного ODBC Администратора в панели управления.

SQLConnect имеет следующий синтаксис:

```

SQLConnect proto ConnectionHandle:DWORD
                pDSN:DWORD,
                DSNLength:DWORD,
                pUserName:DWORD,
                NameLength:DWORD,
                pPassword:DWORD,
                PasswordLength:DWORD

```

- **ConnectionHandle.** Идентификатор подключения который вы хотите использовать.
- **pDSN.** Указатель на DSN-строку.
- **DSNLength.** Длина DSN-строки.
- **pUserName.** Указатель на строку содержащую имя пользователя.
- **NameLength.** Длина строки содержащей имя пользователя.
- **pPassword.** Указатель на строку содержащую пароль ассоциированный с данным именем пользователя.
- **PasswordLength.** Длина пароля

По минимуму, **SQLConnect** требует идентификатор соединения, DSN и их длину: имя пользователя и пароль необязательны, если источник данных не требует их. Список возможных возвращаемых значений идентичен таковому SQLAllocHandle. Предположим мы имеем DSN называемый "Продажи" в нашей системе, и мы хотим соединиться с ним. Мы можем сделать это следующим образом:

```

.data
DSN db "Sales",0

.code

.....
invoke SQLConnect, hConn, addr DSN, sizeof DSN,0,0,0,0

```

Один из недостатков **SQLConnect** - то, что, вы должны создать DSN прежде, чем сможете соединяться с источником данных. SQLDriverConnect предлагает более гибкий вариант. Она имеет следующий синтаксис:

```

SQLDriverConnect proto ConnectionHandle:DWORD,
                    hWnd:DWORD,
                    pInConnectString:DWORD,
                    InStringLength:DWORD,
                    pOutConnectString:DWORD,
                    OutBufferSize:DWORD,
                    pOutConnectStringLength:DWORD,
                    DriverCompletion:DWORD

```

- **ConnectionHandle.** Идентификатор соединения.
- **hWnd.** Дескриптор вашего окна. Если вы передадите NULL как параметр, драйвер не будет запрашивать пользователя для получения дополнительной информации (если необходимо).
- **pInConnectString.** Указатель на строку подключения. Это - ASCIIZ строка, которая отформатирована, согласно специфике ODBC драйвера, с которым вы хотите соединиться. Она описывает название драйвера и источника данных а так же некоторые дополнительные параметры. Полное описание строки подключения можно найти в MSDN. Здесь я не буду углубляться в подробности.
- **InStringLength.** Длина строки соединения.
- **pOutConnectString.** Указатель на буфер, который будет заполнен законченной строкой подключения. Размер этого буфера должен быть по крайней мере 1,024 байта. Это может звучать запутывающе. Если строка подключения, которую вы передаёте функции, не закончена, то в этом случае, ODBC драйвер может запрашивать пользователя для получения дополнительной информации. ODBC драйвер в этом случае создает законченную строку подключения из всей располагаемой информации и помещает её в буфер. Даже если строка подключения, которую вы составили, была функциональна, этот буфер будет заполнен большим количеством атрибутов. Цель этого параметра - сохранить законченную строку подключения для будущего подключения.

- **OutBufferSize.** Размер буфера, указанного **pOutConnectString**.
- **pOutConnectStringLength.** Указатель на dword переменную, которая получит фактическую длину законченной строки подключения, возвращенной ODBC драйвером.
- **DriverCompletion.** Флаг, который определяет запросит ли ODBC менеджер/драйвер пользователя для получения дополнительной информации. Однако, флаг зависит от того, передаёте ли вы дескриптор окна **hWnd** параметру **SQLDriverConnect**. Если вы не делали этого, ODBC менеджер/драйвер не будет запрашивать пользователя, даже если этот флаг инструктирует об этом.

Значение	Описание
SQL_DRIVER_PROMPT	ODBC драйвер запрашивает пользователя относительно информации. Эта информация используется для создания строки подключения.
SQL_DRIVER_COMPLETE / SQL_DRIVER_COMPLETE_REQUIRED	ODBC драйвер запросит пользователя только, если строка подключения, составленная в вашей программе не закончена.
SQL_DRIVER_NOPROMPT	ODBC драйвер не будет запрашивать пользователя для получения дополнительной информации.

Пример:

```
.data
strConnect db "DBQ=c:\data\test.mdb;DRIVER={Microsoft Access Driver (*.mdb)}";,0

.data?
buffer db 1024 dup(?)
OutStringLength dd ?

.code
.....
invoke SQLDriverConnect, hConn, hWnd, addr strConnect, sizeof strConnect,
    addr buffer, sizeof buffer, addr OutBufferSize, SQL_DRIVER_COMPLETE
```

РАЗЪЕДИНЕНИЕ С ИСТОЧНИКОМ ДАННЫХ

После того, как подключение сделано успешно, вы можете создать одну или большее количество инструкций и сделать запрос источнику данных. Я буду исследовать эту часть на следующей консультации. Пока, давайте предположим, что вы уже отработали с источником данных, и должны разъединиться с ним, вызывая **SQLDisconnect**. Эта функция проста (Это отражение грубой и грустной действительности о том, что разрушение - намного проще чем конструкция или созидание). Требуется только один параметр, маркер подключения.

```
invoke SQLDisconnect, hConn
```

УДАЛЕНИЕ ИДЕНТИФИКАТОРОВ ПОДКЛЮЧЕНИЯ И СРЕДЫ

После успешного разъединения вы можете уничтожить идентификаторы подключения и среды, вызывая **SQLFreeHandle**. Это - новая функция, вводимая в ODBC 3.x., она заменяет **SQLFreeConnect**, **SQLFreeEnv** и **SQLFreeStmt**. **SQLFreeHandle** имеет следующий синтаксис:

```
SQLFreeHandle proto HandleType:DWORD, Handle:DWORD
```

- **HandleType.** Константа, которая идентифицирует тип идентификатора, который вы передаёте этой функции как второй параметр. Возможные значения - те же самые, как и в

SQLAllocHandle-Handle. Идентификатор который вы хотите удалить.

Например:

```
invoke SQLFreeHandle, SQL_HANDLE_DBC, hConn  
invoke SQLFreeHandle, SQL_HANDLE_ENV, hEnv
```

ODBC. Урок 3. Подготовка и Использование Инструкций

Автор: lczelion, пер. SheSan

Дата: 2002-06-06

На этой консультации, мы продолжим изучение приёмов программирования ODBC. Мы изучим, как взаимодействовать с источником данных через ODBC. Из предыдущей консультации мы рассмотрели как осуществить подключение к источнику данных. Это - первый шаг. Подключение определяет путь данных между вами и источником данных. Это пассивно. Чтобы взаимодействовать с источником данных, вы должны использовать инструкции. Вы можете понимать эти инструкции как команды, которые вы посылаете источнику данных. "Команда" должна быть написана на SQL. Используя инструкции, вы можете изменить строение источника данных, сделать запрос для получения некоторых данных, модифицировать и удалить данные.

Шаги для подготовки и использования инструкций следующие:

- Выделить память для операторного идентификатора
- Создать инструкцию SQL
- Выполнить инструкцию
- Уничтожить инструкцию

ВЫДЕЛЕНИЕ ПАМЯТИ ДЛЯ ОПЕРАТОРНОГО ИДЕНТИФИКАТОРА

Вы выделяете память для операторного идентификатора вызывая **SQLAllocHandle**, передав ей соответствующие параметры.

Например:

```
.data?  
hStmt dd ?  
.code  
.....  
invoke SQLAllocHandle, SQL_HANDLE_STMT, hConn, addr hStmt
```

СОЗДАНИЕ SQL-ИНСТРУКЦИИ

Здесь, вы должны помочь себе сами. Вы должны узнать о грамматике SQL. Например, если вы хотите создать таблицу, вы должны понять как это делается.

ВЫПОЛНЕНИЕ ИНСТРУКЦИИ

Имеются четыре пути выполнения инструкции, в зависимости от того, когда они откомпилированы и кто определяет их.

Метод	Описание
Немедленное выполнение	Ваша программа определяет инструкцию SQL. Инструкция компилируется и выполняется во время выполнения за один шаг.
Подготовленное Выполнение	Ваша программа также определяет инструкцию SQL. Однако, подготовка и выполнение разделены на два шага: сначала инструкция SQL компилируется, а затем она выполняется. Используя этот метод, вы можете откомпилировать инструкцию SQL лишь однажды а затем выполнять ту же самую инструкцию SQL определённое число раз. Это экономит время.
Процедуры	Инструкции SQL компилируются и сохраняются в источнике данных. Ваша программа вызывает их во время выполнения.
Каталог	Инструкции SQL жестко закодированы в ODBC драйвер. Цель функций каталога - вернуть предопределенные наборы исхода, типа имен таблиц в базе данных. В целом, функции каталога используются, чтобы получить информацию относительно источника данных. Ваша программа вызывает их во время выполнения.

Эти четыре метода имеют "за" и "против". Немедленное выполнение хорошо, когда вы выполняете специфическую инструкцию SQL только однажды. Если бы вы должны были выполнить инструкцию несколько раз последовательно, то вам лучше было бы использовать подготовленное выполнение, потому что инструкция SQL будет откомпилирована лишь однажды. В этом случае последовательное выполнение, будет быстрее, потому что инструкция уже откомпилирована. Процедуры - лучший выбор, если вы хотите оптимизации по скорости. Поскольку процедуры уже откомпилированы и сохранены в источнике данных, они выполняются довольно быстро. Плохо то, что не все данные поддерживают процедуры. Каталог - для получения информации относительно строения источника данных.

На данной консультации, мы сосредоточимся на немедленном и подготовленном выполнении, потому что они выполняются на стороне нашего приложения. Запись процедур осуществляется в DBMS формате. Мы используем функции каталога в будущем.

НЕМЕДЛЕННОЕ ВЫПОЛНЕНИЕ

Чтобы выполнить вашу инструкцию SQL немедленно, вызовите функцию **SQLExecDirect**, которая имеет следующий синтаксис:

```
SQLExecDirect proto StatementHandle:DWORD,
                pStatementText:DWORD,
                TextLength:DWORD
```

- **StatementHandle**. Идентификатор инструкции которую вы хотите выполнить
- **PStatementText**. Указатель на строку SQL которую вы хотите выполнить
- **TextLength**. Длина строки SQL.

Возможные возвращаемые значения:

Значение	Описание
SQL_SUCCESS	Операция успешна.
SQL_SUCCESS_WITH_INFO	Операция успешна, но может быть предупреждение.
SQL_ERROR	Операция потерпела неудачу.

Значение	Описание
SQL_INVALID_HANDLE	Операторный идентификатор , который вы передали функции, был недействителен.
SQL_NEED_DATA	Если инструкция SQL включает один или большее количество параметров, и вы не сумели передать их ей перед выполнением, вы получите это возвращаемое значение. В этом случае вы должны представить параметры через SQLPARAMDATA или SQLPUTDATA.
SQL_NO_DATA	Если ваша инструкция SQL не возвращает результата, то вы получите этот параметр. Если этот параметр получен, то ваша функция выполнена успешно но она не возвращает никаких результатов.
SQL_STILL_EXECUTING	Если вы выполняете инструкцию асинхронно, SQLExecDirect немедленно возвращает это значение, указывая, что инструкция обрабатывается. По умолчанию, ODBC драйверы работают в синхронном режиме, который является наилучшим, если вы используете параллельные процессы OS. Если вы хотите асинхронного выполнения, вы можете установить операторный атрибут с SQLSETSTMATTR.

Пример:

```
.data
SQLStmt db "select * from Sales",0
.data?
hStmt dd ?
.code
.....
invoke SQLAllocHandle, SQL_HANDLE_STMT, hConn, addr hStmt
.if ax==SQL_SUCCESS || ax==SQL_SUCCESS_WITH_INFO
    invoke SQLExecDirect, hStmt, addr SQLStmt, sizeof SQLStmt
```

ПОДГОТОВЛЕННОЕ ВЫПОЛНЕНИЕ

Процесс выполнения инструкции SQL разделен на две фазы. В первой фазе, вы должны подготовить инструкцию, вызывая **SQLPrepare**. Затем, вы вызываете **SQLExecute**, чтобы фактически выполнить инструкцию. Используя подготовленное выполнение, вы можете исполнять с помощью **SQLExecute** ту же самую инструкцию SQL любое число раз. **SQLPrepare** принимает те же самые три параметра, что и **SQLExecDirect**, так что я не буду показывать его определение здесь. **SQLExecute** имеет следующий синтаксис:

```
SQLExecute proto StatementHandle:DWORD
```

Пример:

```
.data
SQLStmt db "select * from Sales",0
.data?
hStmt dd ?
.code
.....
invoke SQLAllocHandle, SQL_HANDLE_STMT, hConn, addr hStmt
.if ax==SQL_SUCCESS || ax==SQL_SUCCESS_WITH_INFO
    invoke SQLPrepare, hStmt, addr SQLStmt, sizeof SQLStmt
    invoke SQLExecute, hStmt
```

Вы можете задаться вопросом, каково преимущество подготовленного выполнения по сравнению с немедленным выполнением. От вышеупомянутых примеров, это не очевидно. Мы должны знать кое что относительно операторных параметров, чтобы быть квалифицированными в оценке этого.

ОПЕРАТОРНЫЕ ПАРАМЕТРЫ

Параметры, как упомянуто здесь, являются переменными, которые используются инструкциями SQL. Например, если мы имеем таблицу названную "служащие"(employee), которая имеет три поля: "имя"(name), "фамилия", и "Номер_телефона"(telephoneNo), и мы должны найти номер телефона служащего по имени Bob, то мы можем использовать следующую инструкцию SQL:

```
select telephoneNo from employee where name='Bob'
```

Эта инструкция SQL будет работать так как мы хотим, но что, если мы хотим найти телефон другого служащего? Если вы не используете параметр, вы не имеете свободы выбора, но можно создать новую строку SQL и откомпилировать/выполнить её снова.

Теперь скажем, что мы не можем допустить эту неэффективность. Мы можем использовать параметр в наших интересах. В нашем примере выше, мы должны заменить строку/значение?. Строка SQL будет:

```
select telephoneNo from employee where name=?
```

Подумайте немного относительно этого: Как ODBC драйвер может знать то, какое значение он должен вставить вместо параметра? Ответ: мы должны снабдить его требуемым значением. Метод которым мы можем сделать это назван закреплением параметра. По сути, это - процесс соединения маркера параметра с переменной в вашем приложении. В вышеупомянутом примере, мы должны создать строковый буфер и затем сообщать ODBC драйверу, что, когда требуется действительное значение параметра, оно должно быть получено значение из строкового буфера, который мы снабдили значениями. Как только параметр связан с переменной, он будет продолжать быть связанным с этой переменной, пока не произойдёт связывания с другой переменной, или пока все параметры не будут лишены этой связи, вызовом **SQLFreeStmt** с **SQL_RESET_PARAMS** в качестве параметра.

Вы связываете параметр с переменной, вызывая **SQLBindParameter**, который имеет следующий синтаксис:

```
SQLBindParameter proto StatementHandle:DWORD,  
                      ParameterNumber:DWORD,  
                      InputOutputType:DWORD,  
                      ValueType:DWORD,  
                      ParameterType:DWORD,  
                      ColumnSize:DWORD,  
                      DecimalDigits:DWORD,  
                      ParameterValuePtr:DWORD,  
                      BufferLength:DWORD,  
                      pStrLenOrIndPtr:DWORD
```

- **StatementHandle**. Идентификатор инструкции.
- **ParameterNumber**. Номер параметра, начинающийся от 1. Это - путь ODBC использования, чтобы опознавать маркеры параметра. Если имеются три параметра, то крайний левый - параметр номер 1, а крайний правый параметр - параметр номер 3.
- **InputOutputType**. Флаг, который определяет, является ли этот параметр для ввода или вывода. Ввод здесь означает, что ODBC драйвер, захватывает значение в параметре для своего собственного использования, в то время как вывод означает, что ODBC драйвер, разместит результат в параметре, когда операция будет выполнена. Большую часть времени, мы используем параметр для ввода. Выходной параметр обычно связывается с сохраненными процедурами. Два возможных значения: **SQL_PARAM_INPUT**, **SQL_PARAM_INPUT_OUTPUT** и **SQL_PARAM_OUTPUT**.

- **ValueType**. Определяет тип переменной или буфера в вашем приложении, который вы хотите связать с параметром. Имеется список констант для располагаемых типов. Их имена начинаются с SQL_C_
- **ParameterType**. Определяют тип SQL параметра. Например, если параметр SQL - текстовое поле, то вы должны поместить значение SQL_CHAR здесь. Вы должны искать законченный список в MSDN.
- **ColumnSize**. Размер параметра, вы можете понимать это поле как размер столбца (поля), связанного с маркером параметра. В нашем примере, маркер параметра используется как критерий в столбце "название". Если этот столбец определен как 20 байтов по размеру, то вы должны передать 20 в ColumnSize.
- **DecimalDigits**. Число десятичных разрядов столбца, связанного с маркером параметра.
- **ParameterValuePtr**. Указатель на буфер для данных параметра.
- **BufferLength**. Размер буфера, указанного ParameterValuePtr.
- **pStrLenOrIndPtr**. Указатель на dword переменную, которая содержит одно из следующих значений:
 - *Длина значения параметра, сохраненного в буфере, указанном ParameterValuePtr*. Это значение игнорируется, если тип параметра не текстовая строка или двоичный. Не путайте это с **BufferLength**. Пример прояснит это. Предположим, что параметр - текстовая строка. Столбец - шириной 20 байт, т.е. вы распределяете 21-байтовый буфер и передаете его адрес в **ParameterValuePtr**. Как раз перед запросом **SQLExecute**, вы помещали строку "Bob" в буфер, эта строка - длиной в 3 байта, таким образом вы нуждаетесь в передаче значения 3 переменной, указанной **pStrLenOrIndPtr**.
 - **SQL_NTS**. Значение параметра - строка с нулевым символом в конце.
 - **SQL_NULL_DATA**. Значение параметра NULL.
 - **SQL_DEFAULT_PARAM**. Процедура должна использовать значение параметра по умолчанию, с большим приоритетом чем значение, полученное от приложения. Это соответствует только для процедур, которые имеют заданные по умолчанию значения параметра.
 - **SQL_DATA_AT_EXEC**. Данные для параметра будут посланы в SQLPUTDATA. Так как данных может быть слишком много, чтобы они сохранялись в памяти, вы сообщаете ODBC драйверу, что вы пошлете данные к нему через **SQLPUTDATA**.

Вы конечно можете сказать, что очень много возможных значений для pStrLenOrIndPtr, но главным образом, мы будем использовать только первую или третью опцию

Пример:

```
.data
SQLString db "select telephoneNo from employee where name=?",0
Sample1 db "Bob",0
Sample2 db "Mary",0
.data?
buffer db 21 dup(?)
StrLen dd ?
.code
.....
invoke SQLPrepare, hStmt, addr SQLString, sizeof SQLString
invoke SQLBindParameter, hStmt, 1, SQL_PARAM_INPUT, SQL_C_CHAR,
    SQL_CHAR, 20, 0, addr buffer, sizeof buffer, addr StrLen
;=====
; First run
;=====
    invoke lstrcpy, addr buffer, addr Sample1
    mov StrLen, sizeof Sample1
    invoke SQLExecute, hStmt
;=====
; Second run
;=====
    invoke lstrcpy, addr buffer, addr Sample2
    mov StrLen, sizeof Sample2
```

invoke SQLExecute, hStmt

Замечание! Мы связываем параметр с буфером только однажды и затем, мы изменяем содержание буфера и вызываем **SQLEXECUTE** несколько раз. Никакой потребности в вызове **SQLPREPARE** не возникает, т.к. ODBC драйвер знает, где найти то, что требует параметр, потому что мы "сказали" ему об этом, вызыв **SQLBindParameter**.

Мы пока игнорируем записи, возвращаемые запросом. Доступ и использование набора возвращаемых результатов - предмет будущих консультаций.

Предположим, что вы выполнили некоторую инструкцию SQL и хотите выполнить новую инструкцию, вы не должны распределять новый операторный идентификатор . Вы должны просто развязать параметры (если необходимо) вызывая **SQLFreeStmt** с параметром **SQL_UNBIND** и **SQL_RESET_PARAMS**. Тогда вы сможете использовать операторный идентификатор для следующей инструкции SQL.

УДАЛЕНИЕ ИНСТРУКЦИИ

Это делается вызовом **SQLFreeHandle**.

ODBC. Урок 4. Возвращаемые величины

Автор: lczelion, пер. SheSan

Дата: 2002-06-06

На этой консультации, вы узнаете как извлекать записи возвращенные инструкциями SQL.

Мы назовём группу записей возвращаемых инструкцией - набором результатов. Общие шаги для получения установленного результата следующие:

1. Определить доступен ли набор результатов.
2. Связать множество столбцов результатов с переменными
3. Выборка столбца

Когда вы проанализируете набор результатов, вам нужно уничтожить его вызвав **SQLCloseCursor**.

ОПРЕДЕЛЕНИЕ ДОСТУПНОСТИ НАБОРА РЕЗУЛЬТАТОВ

Иногда как вы уже знаете в результате выполнения инструкции SQL создаётся набор результатов. Если инструкция SQL - не возвращает набора результатов, то естественно, что никакого доступа к ним быть не может. Тем не менее, иногда вы даже и не знаете, что возвращает инструкция SQL, это возникает тогда, когда вы позволяете пользователю вводить заказные инструкции SQL. В этом случае вы должны проверить был ли создан набор результатов вызовом **SQLNumResultCols**. Эта функция возвращает количество столбцов (полей) в наборе результатов (если они существуют) и имеет следующий синтаксис:

```
SQLNumResultCols proto StatementHandle:DWORD, pNumCols:DWORD
```

- **StatementHandle**. Идентификатор инструкции.
- **pNumCols**. Указатель на переменную типа dword которая возвращает число столбцов в наборе результатов.

Если величина в переменной указанной **pNumCols** это 0, значит результат не устанавливался.

СВЯЗЫВАНИЕ СТОЛБЦОВ

В некотором отношении, понятие - идентичное той же связи переменной с параметром инструкции SQL. Вы ассоциируете(связываете) переменную со специфическим столбцом в наборе результатов. В этом случае используется функция **SQLBindCol**, которая имеет следующий синтаксис:

```
SQLBindCol proto StatementHandle:DWORD,  
                ColumnNumber:DWORD,  
                TargetValuePtr:DWORD,  
                BufferLength:DWORD,  
                pStrLenOrIndPtr:DWORD
```

- **StatementHandle**. Идентификатор инструкции
- **ColumnNumber**. Номер столбца в наборе результатов, с которым будет произведено связывание. Номер столбца начинается с 1. Столбец 0 - столбец закладки.
- **TargetType**. Константа, которая указывает тип переменной (буфера) указанного **TargetValuePtr**.

- **TargetValuePtr**. Указатель на переменную или буфер, который будет связан со столбцом. Когда вы вызываете **SQLFetch**, чтобы извлечь столбец из набора результатов, переменная или буфер будут заполняться величиной из связанного столбца.
- **BufferLength**. Размер буфера указанного **TargetValuePtr**.
- **pStrLenOrIndPtr**. Ищите подробное описание на консультации рассматривающей функцию **SQLBindParameter**

Пример:

```
.data?
buffer db 21 dup(?)
DataLength dd ?      ;will be filled with the length of the string
                      ;in buffer after SQLFetch is called.

.code
.....
invoke SQLBindCol, hStmt, 1, SQL_C_CHAR,
    addr buffer, 21, addr DataLength
```

ВЫБОРКА СТОЛБЦА

Это - совсем просто. Функция **SQLFetch** извлекает столбец из набора результатов в связанные с ним переменные. После того, как функция **SQLFetch** вызвана, курсор обновляется. Вы можете понимать курсор как указатель записи. Он указывает на столбец который будет возвращаться когда ф-я **SQLFetch** вызвана. Например, если набор результатов имеет 4 столбца, курсор позиционируется на первом столбце когда создаётся набор результатов. Когда вызывается ф-я **SQLFetch**, курсор передвигается на 1 столбец. Таким образом, если вы вызываете ф-ю **SQLFetch** 4 раза, то получается, что столбцов которые можно выбрать больше нет. Тогда говорят, что курсор указывает на конец файла (EOF). **SQLFetch** имеет следующий синтаксис:

```
SQLFetch proto StatementHandle:DWORD
```

Эта функция возвращает **SQL_NO_DATA**, если столбец больше недоступен.

Пример:

```
.data?
buffer db 21 dup(?)
DataLength dd ?

.code
.....
invoke SQLBindCol, hStmt, 1, SQL_C_CHAR, addr buffer, 21, addr DataLength
invoke SQLFetch, hStmt
```

ODBC. Урок 5. ODBC пример

Автор: lczelion, пер. SheSan

Дата: 2002-06-06

На этой консультации, мы обобщим всё, что мы знаем на данный момент. Мы напишем программу, которая использует ODBC API. Для простоты я выбирал базу данных Microsoft Access (Microsoft Access 97) для этой программы.

Загрузите пример (находится в архиве wasm_files.zip: files/article56/odbc5.zip).

Примечание: Если вы используете windows.inc версии 1.18 или ниже, вы должны прежде устранить небольшой баг. Запустив поиск в файле windows.inc для **SQL_NULL_HANDLE**, вы найдете строку:

```
SQL_NULL_HANDLE equ 0L
```

Удалите символ "L" после нуля, вот так:

```
SQL_NULL_HANDLE equ 0
```

Эта программа построена на базе диалогового окна с простым меню. Когда пользователь выбирает пункт меню "connect", он пытается подключиться к test.mdb, т.е. своей базе данных. После того, как связь будет установлена, программа отобразит конечную полную строку связи возвращенную драйвером ODBC. После этого, пользователь может выбрать пункт меню "View All Records", чтобы заполнить контрол listview данными из базы. Кроме того, пользователь может выбрать "Query", чтобы найти специфическую запись. Программа представит небольшой диалоговый блок, предлагающий пользователю набрать имя человека, которого он хочет искать. Когда пользователь нажимает кнопку ОК или клавишу Enter, программа выполняет запрос выбрать запись(записи), которые сопоставлены имени. Когда пользователь сделал всё, что ему было необходимо, он может выбрать пункт меню "disconnect", чтобы отключиться от базы данных.

Теперь давайте посмотрим на исходный код:

```
.386
.model flat,stdcall
include \masm32\include\windows.inc
include \masm32\include\kernel32.inc
include \masm32\include\odbc32.inc
include \masm32\include\comctl32.inc
include \masm32\include\user32.inc
includelib \masm32\lib\odbc32.lib
includelib \masm32\lib\comctl32.lib
includelib \masm32\lib\kernel32.lib
includelib \masm32\lib\user32.lib
```

```
IDD_MAINDLG      equ 101
IDR_MAINMENU     equ 102
IDC_DATA_LIST    equ 1000
IDM_CONNECT      equ 40001
IDM_DISCONNECT   equ 40002
IDM_QUERY        equ 40003
IDC_NAME         equ 1000
IDC_OK           equ 1001
IDC_CANCEL       equ 1002
IDM_CUSTOMQUERY  equ 40004
IDD_QUERYDLG     equ 102
```

```
DlgProc proto hDlg:DWORD, uMsg:DWORD, wParam:DWORD, lParam:DWORD
QueryProc proto hDlg:DWORD, uMsg:DWORD, wParam:DWORD, lParam:DWORD
SwitchMenuState proto :DWORD
ODBCConnect proto :DWORD
ODBCDisconnect proto :DWORD
RunQuery proto :DWORD
```

```

.data?
hInstance dd ?
hEnv dd ?
hConn dd ?
hStmt dd ?
Conn db 256 dup(?)
StrLen dd ?
hMenu dd ? ; handle to the main menu
hList dd ? ; handle to the listview control
TheName db 26 dup(?)
TheSurname db 26 dup(?)
TelNo db 21 dup(?)
NameLength dd ?
SurnameLength dd ?
TelNoLength dd ?
SearchName db 26 dup(?)
ProgPath db 256 dup(?)
ConnectString db 1024 dup(?)

.data
SQLStatement db "select * from main",0
WhereStatement db " where name=?",0
strConnect db "DRIVER={Microsoft Access Driver (*.mdb)};DBQ=",0
DBName db "test.mdb",0
ConnectCaption db "Complete Connection String",0
Disconnect db "Disconnect successful",0
AppName db "ODBC Test",0
AllocEnvFail db "Environment handle allocation failed",0
AllocConnFail db "Connection handle allocation failed",0
SetAttrFail db "Cannot set desired ODBC version",0
NoData db "You must type the name in the edit box",0
ExecuteFail db "Execution of SQL statement failed",0
ConnFail db "Connection attempt failed",0
AllocStmtFail db "Statement handle allocation failed",0
Heading1 db "Name",0
Heading2 db "Surname",0
Heading3 db "Telephone No.",0

.code
start:
    invoke GetModuleHandle, NULL
    mov hInstance,eax
    call GetProgramPath
    invoke DialogBoxParam, hInstance, IDD_MAINDLG,0,addr DlgProc,0
    invoke ExitProcess,eax
    invoke InitCommonControls
DlgProc proc hDlg:DWORD, uMsg:DWORD, wParam:DWORD, lParam:DWORD
    .if uMsg==WM_INITDIALOG
        invoke GetMenu, hDlg
        mov hMenu,eax
        invoke GetDlgItem, hDlg, IDC_DATA LIST
        mov hList,eax
        call InsertColumn
    .elseif uMsg==WM_CLOSE
        invoke GetMenuState, hMenu, IDM_CONNECT,MF_BYCOMMAND
        .if eax==MF_GRAYED
            invoke ODBCDisconnect, hDlg
        .endif
        invoke EndDialog,hDlg, 0
    .elseif uMsg==WM_COMMAND
        .if lParam==0
            mov eax,wParam
            .if ax==IDM_CONNECT
                invoke ODBCConnect,hDlg
            .elseif ax==IDM_DISCONNECT
                invoke ODBCDisconnect,hDlg
            .elseif ax==IDM_QUERY
                invoke RunQuery,hDlg
            .elseif ax==IDM_CUSTOMQUERY
                invoke DialogBoxParam, hInstance, IDD_QUERYDLG,hDlg,
                    addr QueryProc, 0
        .endif
    .endif
DlgProc endp

```

```

        .endif
    .endif
    .else
        mov eax,FALSE
        ret
    .endif
    mov eax,TRUE
    ret
DlgProc endp

GetProgramPath proc
    invoke GetModuleFileName, NULL,addr ProgPath,sizeof ProgPath
    std
    mov edi,offset ProgPath
    add edi,sizeof ProgPath-1
    mov al,"\"
    mov ecx,sizeof ProgPath
    repne scasb
    cld
    mov byte ptr [edi+2],0
    ret
GetProgramPath endp

SwitchMenuState proc Flag:DWORD
    .if Flag==TRUE
        invoke EnableMenuItem, hMenu, IDM_CONNECT, MF_GRAYED
        invoke EnableMenuItem, hMenu, IDM_DISCONNECT, MF_ENABLED
        invoke EnableMenuItem, hMenu, IDM_QUERY, MF_ENABLED
        invoke EnableMenuItem, hMenu, IDM_CUSTOMQUERY, MF_ENABLED
    .else
        invoke EnableMenuItem, hMenu, IDM_CONNECT, MF_ENABLED
        invoke EnableMenuItem, hMenu, IDM_DISCONNECT, MF_GRAYED
        invoke EnableMenuItem, hMenu, IDM_QUERY, MF_GRAYED
        invoke EnableMenuItem, hMenu, IDM_CUSTOMQUERY, MF_GRAYED
    .endif
    ret
SwitchMenuState endp

ODBCConnect proc hDlg:DWORD
    invoke SQLAllocHandle, SQL_HANDLE_ENV, SQL_NULL_HANDLE, addr hEnv
    .if ax==SQL_SUCCESS || ax==SQL_SUCCESS_WITH_INFO
        invoke SQLSetEnvAttr, hEnv,SQL_ATTR_ODBC_VERSION, SQL_OV_ODBC3,0
        .if ax==SQL_SUCCESS || ax==SQL_SUCCESS_WITH_INFO
            invoke SQLAllocHandle, SQL_HANDLE_DBC, hEnv, addr hConn
            .if ax==SQL_SUCCESS || ax==SQL_SUCCESS_WITH_INFO
                invoke lstrcpy,addr ConnectString,addr strConnect
                invoke lstrcat,addr ConnectString, addr ProgPath
                invoke lstrcat, addr ConnectString,addr DBName
                invoke SQLDriverConnect, hConn, hDlg, addr ConnectString,
                    sizeof ConnectString, addr Conn, sizeof Conn,addr StrLen,
                    SQL_DRIVER_COMPLETE
                .if ax==SQL_SUCCESS || ax==SQL_SUCCESS_WITH_INFO
                    invoke SwitchMenuState,TRUE
                    invoke MessageBox,hDlg, addr Conn,addr ConnectCaption,
                        MB_OK+MB_ICONINFORMATION
                .else
                    invoke SQLFreeHandle, SQL_HANDLE_DBC, hConn
                    invoke SQLFreeHandle, SQL_HANDLE_ENV, hEnv
                    invoke MessageBox, hDlg, addr ConnFail, addr AppName,
                        MB_OK+MB_ICONERROR
                .endif
            .else
                invoke SQLFreeHandle, SQL_HANDLE_ENV, hEnv
                invoke MessageBox, hDlg, addr AllocConnFail,
                    addr AppName, MB_OK+MB_ICONERROR
            .endif
        .else
            invoke SQLFreeHandle, SQL_HANDLE_ENV, hEnv
            invoke MessageBox, hDlg, addr SetAttrFail, addr AppName,
                MB_OK+MB_ICONERROR
        .endif
    .endif

```

```

        .else
            invoke MessageBox, hDlg, addr AllocEnvFail, addr AppName,
                MB_OK+MB_ICONERROR
        .endif
        ret
ODBCConnect endp

ODBCDisconnect proc hDlg:DWORD
    invoke SQLDisconnect, hConn
    invoke SQLFreeHandle, SQL_HANDLE_DBC, hConn
    invoke SQLFreeHandle, SQL_HANDLE_ENV, hEnv
    invoke SwitchMenuState, FALSE
    invoke ShowWindow,hList, SW_HIDE
    invoke MessageBox,hDlg,addr Disconnect, addr AppName,
        MB_OK+MB_ICONINFORMATION
    ret
ODBCDisconnect endp

InsertColumn proc
    LOCAL lvc:LV_COLUMN
    mov lvc.imask,LVCF_TEXT+LVCF_WIDTH
    mov lvc.pszText,offset Heading1
    mov lvc.lx,150
    invoke SendMessage,hList, LVM_INSERTCOLUMN,0,addr lvc
    mov lvc.pszText,offset Heading2
    invoke SendMessage,hList, LVM_INSERTCOLUMN, 1 ,addr lvc
    mov lvc.pszText,offset Heading3
    invoke SendMessage,hList, LVM_INSERTCOLUMN, 3 ,addr lvc
    ret
InsertColumn endp

FillData proc
    LOCAL lvi:LV_ITEM
    LOCAL row:DWORD

    invoke SQLBindCol, hStmt,1,SQL_C_CHAR, addr TheName,
        sizeof TheName,addr NameLength
    invoke SQLBindCol, hStmt,2,SQL_C_CHAR, addr TheSurname,
        sizeof TheSurname,addr SurnameLength
    invoke SQLBindCol, hStmt,3,SQL_C_CHAR, addr TelNo,
        sizeof TelNo,addr TelNoLength
    mov row,0
    .while TRUE
        mov byte ptr ds:[TheName],0
        mov byte ptr ds:[TheSurname],0
        mov byte ptr ds:[TelNo],0
        invoke SQLFetch, hStmt
        .if ax==SQL_SUCCESS || ax==SQL_SUCCESS_WITH_INFO
            mov lvi.imask,LVIF_TEXT+LVIF_PARAM
            push row
            pop lvi.iItem
            mov lvi.iSubItem,0
            mov lvi.pszText, offset TheName
            push row
            pop lvi.lParam
            invoke SendMessage,hList, LVM_INSERTITEM,0, addr lvi
            mov lvi.imask,LVIF_TEXT
            inc lvi.iSubItem
            mov lvi.pszText,offset TheSurname
            invoke SendMessage,hList,LVM_SETITEM, 0,addr lvi
            inc lvi.iSubItem
            mov lvi.pszText,offset TelNo
            invoke SendMessage,hList,LVM_SETITEM, 0,addr lvi
            inc row
        .else
            .break
        .endif
    .endw
    ret
FillData endp

RunQuery proc hDlg:DWORD

```

```

invoke ShowWindow, hList, SW_SHOW
invoke SendMessage, hList, LVM_DELETEALLITEMS,0,0
invoke SQLAllocHandle, SQL_HANDLE_STMT, hConn, addr hStmt
.if ax==SQL_SUCCESS || ax==SQL_SUCCESS_WITH_INFO
    invoke SQLExecDirect, hStmt, addr SQLStatement, sizeof SQLStatement
    .if ax==SQL_SUCCESS || ax==SQL_SUCCESS_WITH_INFO
        invoke FillData
    .else
        invoke ShowWindow, hList, SW_HIDE
        invoke MessageBox,hDlg,addr ExecuteFail, addr AppName,
            MB_OK+MB_ICONERROR
    .endif
    invoke SQLCloseCursor, hStmt
    invoke SQLFreeHandle, SQL_HANDLE_STMT, hStmt
.else
    invoke ShowWindow, hList, SW_HIDE
    invoke MessageBox,hDlg,addr AllocStmtFail, addr AppName,
        MB_OK+MB_ICONERROR
.endif
ret
RunQuery endp
QueryProc proc hDlg:DWORD, uMsg:DWORD, wParam:DWORD, lParam:DWORD
    .if uMsg==WM_CLOSE
        invoke SQLFreeHandle, SQL_HANDLE_STMT, hStmt
        invoke EndDialog, hDlg,0
    .elseif uMsg==WM_INITDIALOG
        invoke ShowWindow, hList, SW_SHOW
        invoke SQLAllocHandle, SQL_HANDLE_STMT, hConn, addr hStmt
        .if ax==SQL_SUCCESS || ax==SQL_SUCCESS_WITH_INFO
            invoke lstrcpy, addr Conn, addr SQLStatement
            invoke lstrcat, addr Conn, addr WhereStatement
            invoke SQLBindParameter,hStmt, 1, SQL_PARAM_INPUT,
                SQL_C_CHAR, SQL_CHAR,25,0, addr SearchName,25,
                addr StrLen
            invoke SQLPrepare, hStmt, addr Conn, sizeof Conn
        .else
            invoke ShowWindow, hList, SW_HIDE
            invoke MessageBox,hDlg,addr AllocStmtFail, addr AppName,
                MB_OK+MB_ICONERROR
            invoke EndDialog, hDlg,0
        .endif
    .elseif uMsg==WM_COMMAND
        mov eax, wParam
        shr eax,16
        .if ax==BN_CLICKED
            mov eax,wParam
            .if ax==IDC_OK
                invoke GetDlgItemText, hDlg, IDC_NAME, addr SearchName, 25
                .if ax==0
                    invoke MessageBox, hDlg,addr NoData, addr AppName,
                        MB_OK+MB_ICONERROR
                    invoke GetDlgItem, hDlg, IDC_NAME
                    invoke SetFocus, eax
                .else
                    invoke strlen,addr SearchName
                    mov StrLen,eax
                    invoke SendMessage, hList, LVM_DELETEALLITEMS,0,0
                    invoke SQLExecute, hStmt
                    invoke FillData
                    invoke SQLCloseCursor, hStmt
                .endif
            .else
                invoke SQLFreeHandle, SQL_HANDLE_STMT, hStmt
                invoke EndDialog, hDlg,0
            .endif
        .endif
    .else
        mov eax,FALSE
        ret
    .endif
mov eax,TRUE

```

```

    ret
QueryProc endp
end start

```

АНАЛИЗ

```

start:
    invoke GetModuleHandle, NULL
    mov hInstance, eax
    call GetProgramPath

```

Когда программа стартует, она получает описатель экземпляра, затем обнаруживает свой собственный путь. Это делается с расчётом на то, что база данных, test.mdb, находится в той же папке, что и программа.

```

GetProgramPath proc
    invoke GetModuleFileName, NULL, addr ProgPath, sizeof ProgPath
    std
    mov edi, offset ProgPath
    add edi, sizeof ProgPath-1
    mov al, "\"
    mov ecx, sizeof ProgPath
    repne scasb
    cld
    mov byte ptr [edi+2], 0
    ret
GetProgramPath endp

```

Функция **GetProgramPath** вызывает **GetModuleFileName**, чтобы получить полный путь и имя программы. После этого мы ищем последний символ "\" в этом пути, чтобы отделить имя файла от пути заменяя имя файла нулём. Таким образом мы получили путь к программе в ProgPath.

Программа затем отображает основное диалоговое окно, используя **DialogBoxParam**. Сначала, как только основное диалоговое окно загружается, мы получаем дескрипторы меню и контроля listview. Затем создаём три столбца в элементе управления listview (поскольку мы знаем заблаговременно, что множество результатов состоит из трех столбцов. Далее мы переходим к заполнению таблицы созданной на первом шаге.)

После этого мы ждем действия пользователя. Если пользователь выбирает "connect" из меню, он вызывает функцию **ODBCConnect**

```

ODBCConnect proc hDlg:DWORD
    invoke SQLAllocHandle, SQL_HANDLE_ENV,
        SQL_NULL_HANDLE, addr hEnv
    .if ax==SQL_SUCCESS || ax==SQL_SUCCESS_WITH_INFO

```

Первое, что мы делаем это выделяем память для идентификатора окружения, используя **SQLAllocHandle**:

```

        invoke SQLSetEnvAttr, hEnv, SQL_ATTR_ODBC_VERSION,
            SQL_OV_ODBC3, 0
    .if ax==SQL_SUCCESS || ax==SQL_SUCCESS_WITH_INFO

```

После того как мы получили ид. окружения, мы устанавливаем его параметры сообщая о том, что мы будем использовать синтаксис ODBC 3.x вызывая **SQLSetEnvAttr**:

```

        invoke SQLAllocHandle, SQL_HANDLE_DBC, hEnv, addr hConn
    .if ax==SQL_SUCCESS || ax==SQL_SUCCESS_WITH_INFO

```

Если все идет хорошо, то мы можем начать соединение выделив память для идентификатора соединения, используя **SQLAllocHandle**:

```

        invoke lstrcpy, addr ConnectString, addr strConnect
        invoke lstrcat, addr ConnectString, addr ProgPath
        invoke lstrcat, addr ConnectString, addr DBName

```

Конструируем строку соединения, которую мы будем использовать при подключении:

```

invoke SQLDriverConnect, hConn, hDlg, addr ConnectString,
    sizeof ConnectString, addr Conn, sizeof Conn,
    addr StrLen, SQL_DRIVER_COMPLETE
.if ax==SQL_SUCCESS || ax==SQL_SUCCESS_WITH_INFO
    invoke SwitchMenuState,TRUE
    invoke MessageBox,hDlg, addr Conn,addr ConnectCaption,
        MB_OK+MB_ICONINFORMATION

```

Когда строка соединения - готова, мы вызываем **SQLDriverConnect**, чтобы попытаться подключиться к test.mdb, используя MS Access драйвер ODBC. Если файл test.mdb не обнаружен, то драйвер ODBC известит пользователя об этом поскольку мы определили флаг **SQL_DRIVER_COMPLETE**. Когда ф-я **SQLDriverConnect** успешно выполнится, переменная Conn заполнится полной строкой связи созданной драйвером ODBC. Мы отображаем её используя окно сообщений. **SwitchMenuState** - простая функция, она прорисовывает серым цветом пункты соответствующего меню.

Сейчас связь с базой данных будет установлена до тех пор пока пользователь не разорвёт её.

Когда пользователь выбирает пункт меню "View All Records", процедура диалогового окна вызывает **RunQuery**:

```

RunQuery proc hDlg:DWORD
    invoke ShowWindow, hList, SW_SHOW
    invoke SendMessage, hList, LVM_DELETEALLITEMS,0,0

```

С тех пор как элемент управления listview был создан, он оставался невидимым, но теперь самое время, чтобы показать его. Также нам нужно удалить все пункты (если имеются) с контрола listview.

```

invoke SQLAllocHandle, SQL_HANDLE_STMT, hConn, addr hStmt
.if ax==SQL_SUCCESS || ax==SQL_SUCCESS_WITH_INFO

```

Затем, программа выделяет память для идентификатора инструкции.

```

invoke SQLExecDirect, hStmt, addr SQLStatement,
    sizeof SQLStatement
.if ax==SQL_SUCCESS || ax==SQL_SUCCESS_WITH_INFO

```

Выполняем инструкцию SQL используя **SQLExecDirect**. Я решаю здесь использовать **SQLExecDirect**, поскольку эта инструкция SQL исполняется лишь однажды:

```

invoke FillData

```

После выполнения инструкции SQL, набор результатов должен быть возвращен. Мы извлекаем данные результатов в элемент управления listview используя функцию **FillData**:

```

FillData proc
    LOCAL lvi:LV_ITEM
    LOCAL row:DWORD

    invoke SQLBindCol, hStmt,1,SQL_C_CHAR, addr TheName,
        sizeof TheName,addr NameLength
    invoke SQLBindCol, hStmt,2,SQL_C_CHAR, addr TheSurname,
        sizeof TheSurname,addr SurnameLength
    invoke SQLBindCol, hStmt,3,SQL_C_CHAR, addr TelNo,
        sizeof TelNo,addr TelNoLength

```

Здесь, набор результатов уже возвращен. Нам нужно связать все три колонки набора результатов с буфером. Мы вызываем ф-ю **SQLBindCol** чтобы сделать это. Имейте в виду, что нам нужен отдельный вызов для каждой колонки, и мы не должны связывать все столбцы: только столбцы, из которых нам нужно получить данные.

```

mov row,0
.while TRUE
    mov byte ptr ds:[TheName],0
    mov byte ptr ds:[TheSurname],0
    mov byte ptr ds:[TelNo],0

```


Мы инициализируем буфер значением NULL в случае, если в столбце(столбцах) нет данных. Лучше использовать длину данных возвращенных в переменных которые мы определили в **SQLBindCol**. В нашем примере, мы могли бы проверить величины в **NameLength**, **SurnameLength**, **TelNoLength**, чтобы получить фактическую длину возвращенных строк.

```
invoke SQLFetch, hStmt
.if ax==SQL_SUCCESS || ax==SQL_SUCCESS_WITH_INFO
    mov lvi.imask, LVIF_TEXT+LVIF_PARAM
    push row
    pop lvi.iItem
    mov lvi.iSubItem, 0
    mov lvi.pszText, offset TheName
    push row
    pop lvi.lParam
    invoke SendMessage, hList, LVM_INSERTITEM, 0, addr lvi
```

Остальное - просто. Вызываем **SQLFetch**, чтобы извлечь колонку из набора результата а затем сохранить величины в буферах на контроле listview. Когда колонка становится недоступна (мы достигли конца файла), **SQLFetch** возвращает **SQL_NO_DATA** и мы выходим из бесконечного цикла.

```
invoke SQLCloseCursor, hStmt
invoke SQLFreeHandle, SQL_HANDLE_STMT, hStmt
```

Когда мы получили набор результатов, мы должны закрыть курсор, используя **SQLCloseCursor**, а затем освободить идентификатор инструкции используя **SQLFreeHandle**.

Когда пользователь выбирает пункт меню "Query", программа отображает другое диалоговое окно, приглашающее пользователя ввести имя по которому будет осуществлён поиск.

```
.elseif uMsg==WM_INITDIALOG
    invoke ShowWindow, hList, SW_SHOW
    invoke SQLAllocHandle, SQL_HANDLE_STMT, hConn, addr hStmt
    .if ax==SQL_SUCCESS || ax==SQL_SUCCESS_WITH_INFO
        invoke lstrcpy, addr Conn, addr SQLStatement
        invoke lstrcat, addr Conn, addr WhereStatement
        invoke SQLBindParameter, hStmt, 1, SQL_PARAM_INPUT,
            SQL_C_CHAR, SQL_CHAR, 25, 0, addr SearchName,
            25, addr StrLen
        invoke SQLPrepare, hStmt, addr Conn, sizeof Conn
```

Первая вещь, которую делает диалоговое окно - показывает элемент listview управления. Затем распределяет память для идентификатора инструкции и наконец, создает инструкцию SQL. Эта инструкция SQL означает вернуть все значения содержащиеся в таблице main, где имя имеет значение ?.

```
select * from main where name=?
```

Затем, оно вызывает **SQLBindParameter**, чтобы соединить ид. инструкции с буфером **SearchName**. Так что когда инструкция SQL выполняется, драйвер ODBC может получить строку, которая ему нужна из **SearchName**. Потом оно вызывает **SQLPrepare**, чтобы скомпилировать инструкцию SQL. Здесь отложенное выполнение разумно т.к. мы подготавливаем/компилируем утверждение SQL только раз и затем, мы будем использовать его много раз. Когда инструкция SQL скомпилирована, последующее выполнение - намного быстрее.

```
.if ax==IDC_OK
    invoke GetDlgItemText, hDlg, IDC_NAME,
        addr SearchName, 25
    .if ax==0
        invoke MessageBox, hDlg, addr NoData,
            addr AppName, MB_OK+MB_ICONERROR
        invoke GetDlgItem, hDlg, IDC_NAME
        invoke SetFocus, eax
    .else
```

Когда пользователь занес некоторое имя в окно редактирования и нажал ввод, программа извлекает текст из этого окна редактирования и проверяет действительно ли он был введен, если нет - возвращается значение NULL. В этом случае отображается сообщение и устанавливается клавиатурный фокус на окно редактирования, чтобы подсказать пользователю о необходимости ввода имени.

```
invoke strlen,addr SearchName
mov StrLen,eax
invoke SendMessage, hList, LVM_DELETEALLITEMS,0,0
invoke SQLExecute, hStmt
invoke FillData
invoke SQLCloseCursor, hStmt
```

Если имя набирается в окне редактирования, мы находим его длину и сохраняем в StrLen для использования драйвером ODBC (вспомните, что фактически мы получили адрес **StrLen** для ф-ции SQLBindParameter). Затем мы вызываем ф-ю **SQLExecute**, передавая ей в качестве входного параметра ид. инструкции, чтобы выполнить подготовленную инструкцию SQL. Когда происходит возврат из ф-ции **SQLExecute**, мы вызываем **FillData**, чтобы отобразить результат в элементе управления listview. Поскольку мы не имеем более полезной информации, мы закрываем курсор вызывая **SQLCloseCursor**.